

QFM.INSTITUTE

Book I of QUANSISTOR FIELD COMPUTING
PRESENTS:

Quansistor Field Mathematics

by

Enter Yourname CEO of QFM.institute

github.com/enteryourname101

101research.group

Vol. No. I Mathematical Foundations
Operators, Hilbert Spaces, and Spectral Structure

Vol. No. II Operator Architectures
QVM, Spectral Execution, and Applied Frameworks

Vol. No. III Enforcement of the Riemann Hypothesis
Arithmetic Hamiltonians, Spectral Constraints, and Prime Dynamics

Vol. No. IV The Spectral Operator Registry
Canonical Filters, Penalties, Liouvillians, Tokens, and Bridges

VOL. No. I

Mathematical Foundations

Operators, Hilbert Spaces, and Spectral Structure

1	The Quansistor	
	A Generalized Operator-Based Computational Primitive	3
2	QFM: Quansistor Field Mathematics	
	An Operator-First Arithmetic and Computational Framework	27
3	Spectral Foundations in Operator-First Quantum Theory	
	From State Space to Spectral Topology: An Operator-First Foundation	84
4	Spectral Dynamics and Operator-First Probing	
	Invariants, Operator Algebras, and Arithmetic Spectral Methods	97
5	Spectral Probing Frameworks and Experimental Architecture	
	Models, Statistics, and Computational Validation	109

Chapter 1

The Quansistor

A Generalized Operator-Based Computational Primitive

Enter Yourname

Abstract

This whitepaper introduces the concept of the *quansistor* as a generalized computational primitive defined not by instructions or control flow, but by the application of structured operators acting on abstract state spaces. The motivation for this framework arises from the growing mismatch between instruction-based computational models and the mathematical structures required to describe modern computation, including distributed systems, cryptographic protocols, spectral algorithms, and quantum-inspired architectures.

The quansistor paradigm reframes computation as an operator-first process, where the fundamental unit of execution is not a command or gate, but a transformation with well-defined algebraic, spectral, and compositional properties. In this view, a computational process is understood as a trajectory through a space of states induced by successive operator applications, rather than as a sequence of instructions executed by a machine.

This operator-centric abstraction enables a unification of classical computation, quantum computation, and hybrid symbolic-numeric models under a single formal language. By decoupling execution from hardware-specific instruction sets, the quansistor serves as a bridge between mathematical operator theory and practical computational architectures. In particular, it allows computational processes to be reasoned about using tools from functional analysis, spectral theory, and representation theory, without committing to a specific physical realization.

A key consequence of this formulation is that composability becomes a native property of computation. Operators can be composed, factored, inverted, or restricted in ways that mirror mathematical constructions, enabling new forms of modularity, verification, and semantic control. This perspective naturally leads to the concept of operator fields, where families of interacting operators define computational dynamics across distributed or layered systems.

The quansistor framework also provides a conceptual foundation for deterministic replay, traceability, and auditability of computation. Since

operator application is treated as a mathematically well-defined transformation, execution traces can be reconstructed, inspected, and verified independently of runtime contingencies. This property is particularly relevant for systems requiring strong guarantees of correctness, accountability, or reproducibility.

Finally, this section situates the quansistor within the broader context of Quansistor Field Computing (QFC), where computation is modeled as an emergent phenomenon arising from structured operator interactions. The ideas presented here serve as the conceptual and formal starting point for the mathematical developments, spectral constructions, and computational architectures explored in subsequent sections and parts of this compendium.

1 Introduction and Strategy

The purpose of this paper is to address the problem of *spectral identification* for arithmetic operators constructed in the preceding works. While Whitepaper I established the existence of canonical self-adjoint realizations of prime-driven Hamiltonians, and Whitepaper II derived trace formulas exposing explicit arithmetic structure, the present work focuses on the question of identification and constraint at the level of spectral invariants.

The emphasis therefore shifts from analytic existence and trace expansion toward spectral structure, normalization, and obstruction. The central problem is no longer whether a meaningful arithmetic operator can be defined, but whether its spectral data can be canonically related to classical arithmetic zeta functions.

1.1 Context and Preceding Results

Whitepaper I constructed a class of symmetric and self-adjoint arithmetic operators acting on the critical arithmetic Hilbert space

$$\mathcal{H}_{\text{crit}} = \ell^2(\mathbb{N}, 1/n),$$

using quadratic form techniques and canonical self-adjoint extensions. The key outcome was the existence of operators with well-defined resolvents and stable spectral measures, defined intrinsically from arithmetic principles.

Whitepaper II developed trace-level analysis for these operators. Regulated trace expressions were expanded into sums over closed arithmetic cycles, yielding a decomposition into smooth spectral terms and oscillatory contributions indexed by primes and their powers. These expansions mirror the structure of classical explicit formulae in analytic number theory.

Taken together, these results establish a rigorous operator-theoretic framework in which arithmetic information enters spectral quantities through trace identities. However, trace-level correspondence alone does not constitute spectral identification.

1.2 The SMRK Hamiltonian

The arithmetic operators considered in this work are organized around a single distinguished object: the *SMRK Hamiltonian*. It provides a concrete realization of prime-driven arithmetic dynamics as a self-adjoint operator acting on the critical arithmetic Hilbert space.

$$(H_{\text{SMRK}}\varphi)(n) = \sum_{p \in \mathbb{P}} \frac{1}{p} \left(\varphi(n/p) \mathbf{1}_{p|n} + \varphi(pn) \right) + (\alpha \Lambda(n) + \beta \log n) \varphi(n).$$

Here $\mathbb{P}$ denotes the set of prime numbers, $\Lambda(n)$ is the von Mangoldt function, and $\alpha, \beta \in \mathbb{R}$ are fixed coupling parameters. The indicator $\mathbf{1}_{p|n}$ enforces divisibility by the prime p .

The SMRK Hamiltonian is not introduced as a formal analogy to a classical mechanical Hamiltonian. Rather, it encodes multiplicative arithmetic dynamics directly: prime multiplication and division generate the kinetic component, while arithmetic potentials arise from explicit formulae rather than ad hoc modeling choices. The operator admits a unique self-adjoint realization and has purely discrete spectrum.

1.3 What Is Meant by Spectral Identification

By *spectral identification* we mean the following problem. Given a self-adjoint operator A with well-defined spectral invariants, can one identify a spectral object associated with A —such as a spectral zeta function or regularized determinant—with a known arithmetic function?

Identification is not understood as heuristic similarity or formal analogy. It requires equality of analytic objects, up to explicitly controlled and understood normalization factors.

In particular, spectral identification requires:

- a canonically defined spectral invariant associated with A ,
- analytic continuation to a sufficiently large domain,
- structural compatibility with known arithmetic functions,
- explicit control of regularization and normalization.

Failure of any of these conditions constitutes a genuine obstruction rather than a technical gap.

1.4 Role of Self-Adjointness

Self-adjointness is central to any attempt at spectral identification. It ensures that the spectrum of the operator lies on the real axis and that spectral measures are positive.

As a consequence, any spectral determinant or spectral zeta function derived from a self-adjoint operator inherits strong symmetry and reality constraints. In particular, if a completed arithmetic zeta function were to arise as a spectral determinant of such an operator, its nontrivial zeros would necessarily lie on a critical line determined by the spectral normalization.

This observation motivates the operator-first approach adopted here, but does not by itself establish identification.

1.5 Strategy of the Present Work

The strategy pursued in this paper is deliberately conservative. Rather than assuming a spectral–arithmetic correspondence, we treat it as a hypothesis subject to explicit analytic and

structural constraints.

We proceed in stages. First, we define spectral invariants—spectral zeta functions and regularized determinants—associated with the SMRK Hamiltonian. Next, we analyze their analytic properties, including continuation, symmetry, and normalization behavior. Finally, we formulate precise criteria under which identification with classical arithmetic zeta functions may occur, and equally precise criteria under which it must fail.

1.6 Implications for the Riemann Hypothesis

The Riemann Hypothesis enters this framework only conditionally. If a spectral determinant associated with the SMRK Hamiltonian coincides with the completed Riemann zeta function, then the Riemann Hypothesis follows as a formal consequence of self-adjointness.

The present work does not claim to establish such a coincidence. Instead, it isolates the precise operator-theoretic conditions under which such an implication would hold, and clarifies the structural obstructions that may prevent it.

1.7 Organization of the Paper

Section 2 introduces spectral invariants associated with self-adjoint arithmetic operators. Section 3 develops spectral zeta functions and determinant constructions. Section 4 analyzes symmetry constraints and functional equations. Section 5 addresses the identification problem with arithmetic zeta functions. Section 6 discusses conditional implications for the Riemann Hypothesis. Section 7 analyzes falsifiability and failure modes. The final section summarizes the results and outlines future directions.

2 Motivation and Context

The theoretical foundations of computation have traditionally been dominated by discrete, symbol-oriented models. The Turing machine, lambda calculus, and related formalisms have provided a robust and rigorous framework for defining computability, complexity, and algorithmic limits. These models, however, were never intended to describe the full richness of physical or emergent computational processes. Instead, they abstract computation into sequences of symbolic transformations executed in isolation from spatial, temporal, and interactive structure.

As computational systems have grown in scale and complexity, the limitations of purely discrete and sequential models have become increasingly apparent. Modern computing systems are inherently distributed, concurrent, and dynamic. They involve continuous interaction between components, feedback loops across multiple scales, and state-dependent behavior that cannot be naturally captured by a single global transition function or a linear sequence of computational steps.

Classical digital logic addresses these challenges through engineering practices rather than theoretical primitives. Pipelining, parallelism, asynchronous logic, and networked computation are implemented as architectural layers built on top of fundamentally simple switching elements. While effective in practice, this approach obscures the underlying computational dynamics and makes it difficult to reason about global behavior, composability, and emergent phenomena at a foundational level.

Quantum computation introduces a richer mathematical structure, replacing discrete symbols with vectors in Hilbert space and classical logic gates with unitary operators. Despite this advance, quantum computation remains constrained by a gate-based paradigm that mirrors

classical circuit models. Quantum gates are applied sequentially or in fixed circuit patterns, and the computational model assumes a rigid separation between data, operations, and control.

Similarly, probabilistic and neural computation models extend classical computation by introducing stochasticity and nonlinearity, yet they often lack a unifying formalism that treats these features as first-class elements of computation. Neural networks, for example, are typically described as layered compositions of activation functions rather than as interacting operator fields with intrinsic dynamics.

The motivation for the Quansistor arises from the observation that these diverse computational paradigms can be viewed as special cases of a more general process: the evolution of states under the action of operators within a structured environment. By elevating operators and their interactions to the primary objects of study, it becomes possible to describe computation in a way that is inherently compositional, scalable, and agnostic to specific implementation details.

This perspective draws inspiration from several converging theoretical developments. In physics, field theories describe complex systems as continuous or discrete fields governed by local interaction rules and global symmetries. In mathematics, operator algebras and dynamical systems theory provide powerful tools for analyzing state evolution, stability, and spectral structure. In computer science, category theory and process calculi emphasize compositionality and interaction over isolated computation.

The Quansistor synthesizes these ideas into a single conceptual primitive. It is motivated by the need for a computational unit that can naturally participate in networks, respond to contextual influences, and support both discrete and continuous forms of state transformation. Rather than enforcing a fixed notion of logic or control flow, the Quansistor allows computational behavior to emerge from the structure of the operator field in which it is embedded.

Importantly, the Quansistor does not seek to replace existing computational models, but to subsume them within a more general framework. Classical logic gates, quantum operations, stochastic transitions, and neural activations can all be interpreted as specific realizations of Quansistor dynamics under particular constraints. This unifying role enables meaningful comparison between paradigms and facilitates the exploration of hybrid computational architectures that combine features traditionally treated as incompatible.

In this context, the Quansistor represents a response to the growing gap between foundational computational theory and practical computational systems. By providing a higher-level abstraction grounded in operator dynamics and field interactions, it aims to bridge symbolic, numerical, and physical computation within a coherent theoretical structure.

The next chapter introduces an informal conceptual definition of the Quansistor, building intuition and establishing the language necessary for the formal mathematical treatment that follows.

3 Informal Concept of the Quansistor

Before introducing a formal mathematical definition, it is useful to develop an intuitive understanding of the Quansistor and the role it plays within a computational system. The purpose of this chapter is not to define precise axioms, but to establish a conceptual picture that can guide interpretation and application of the formalism introduced later.

At an informal level, a Quansistor may be understood as a generalized computational element whose essential function is to transform states. Unlike classical transistors, which switch between discrete voltage levels, or quantum gates, which enact prescribed transformations on fixed-dimensional state vectors, the Quansistor is defined by its capacity to act on an abstract state space according to an operator rule. The nature of this state space is intentionally left open: it may be discrete or continuous, finite or infinite-dimensional, symbolic or numerical.

One way to visualize a Quansistor is as a localized excitation within a computational field. In this picture, the state of the system is not concentrated at a single point, but distributed across a field of interacting elements. Each Quansistor participates in the global evolution of the system by applying its operator to the local state while simultaneously being influenced by the surrounding field. Computation emerges from the collective dynamics of these interactions rather than from a centrally orchestrated sequence of operations.

Another useful intuition is to view the Quansistor as an abstract “state transformer.” Given an input state, the Quansistor produces an output state according to its internal transition rule. This rule need not correspond to a logical operation in the classical sense, nor to a unitary transformation in the quantum sense. It may be nonlinear, context-dependent, or probabilistic, and it may involve coupling to other Quansistors through the computational field.

Importantly, the Quansistor does not impose a strict separation between data and operation. In many traditional computational models, data is passive and operations are external instructions applied to it. In contrast, a Quansistor may encode both state and transformation within a unified structure. The operator defining its behavior may itself be influenced by the state of the system, enabling adaptive, self-modifying, or emergent computational dynamics.

From this perspective, computation is no longer a linear progression of steps, but a continuous or discrete evolution of a state configuration. Time, if it is introduced at all, serves as a parameter indexing successive applications of the operator dynamics. In some cases, the notion of time may be entirely emergent, arising from the ordering of state transitions rather than being imposed externally.

The informal concept of the Quansistor also emphasizes composability. Individual Quansistors are not intended to operate in isolation. Instead, they form networks or fields in which complex behavior arises from simple local rules. This mirrors phenomena observed in physical systems, where local interactions between particles or fields give rise to global structure and dynamics. In a computational setting, such composability enables scalable, distributed, and fault-tolerant forms of computation.

It is crucial to note that the Quansistor is not synonymous with any specific existing computational element. While it can emulate the behavior of logic gates, quantum operators, or neural units under appropriate constraints, its defining feature is generality. The Quansistor is deliberately under-specified at the informal level, allowing it to serve as a conceptual umbrella under which diverse computational mechanisms may be unified.

This informal understanding prepares the ground for a precise mathematical formulation. In the next chapter, the Quansistor will be defined rigorously in terms of state spaces, operator algebras, and computational fields, establishing the formal structure necessary for analysis, comparison, and application.

4 Formal Mathematical Definition

This chapter presents a formal mathematical definition of the Quansistor. The goal is not to restrict the concept to a narrow class of systems, but to provide a minimal yet rigorous structure capable of supporting a wide range of computational behaviors. The definition is intentionally abstract, allowing specialization to particular computational paradigms as needed.

4.1 State Space

Let S denote a state space. No a priori assumptions are made regarding the nature of S . It may be:

- discrete or continuous,
- finite- or infinite-dimensional,
- symbolic, numerical, or hybrid,
- deterministic or probabilistic.

Elements $\psi \in S$ represent admissible computational states of the system. The structure of S determines what constitutes a valid configuration, but does not prescribe how states evolve.

4.2 Operator Algebra

Let $A \subseteq \text{End}(S)$ be an algebra of operators acting on the state space S . Elements of A are maps

$$T : S \rightarrow S$$

that define admissible state transformations.

The algebra A may possess additional structure, such as linearity, norm topology, or involution, but none of these properties are required in the general definition. In particular, operators may be linear or nonlinear, bounded or unbounded, deterministic or stochastic.

4.3 Computational Field

The computational field, denoted F , provides the contextual structure within which operators act. Informally, F encodes interaction rules, coupling strengths, locality constraints, and symmetry relations between operators.

Formally, the field induces:

- a topology on the collection of Quansistors,
- adjacency or interaction relations between operators,
- constraints on permissible compositions of operators.

The computational field may be static or dynamic, homogeneous or heterogeneous, and may itself depend on the evolving state of the system.

4.4 Definition of a Quansistor

Definition. A Quansistor is defined as a quadruple

$$Q := (S, A, T, F),$$

where:

- S is a state space,
- A is an operator algebra acting on S ,
- $T \in A$ is a distinguished state-transition operator,
- F is a computational field governing operator interaction.

The action of a Quansistor on a state $\psi \in S$ is given by the evolution rule

$$\psi_{t+1} = T(\psi_t),$$

or, in continuous form,

$$\frac{d\psi}{dt} = T(\psi).$$

4.5 Time and Evolution

Time is treated as an auxiliary parameter indexing successive applications of the operator T . The formalism does not require time to be discrete or continuous, nor does it require a global clock. In networked settings, different Quansistors may evolve according to distinct local time parameters.

This flexibility allows the Quansistor framework to encompass synchronous, asynchronous, and event-driven computational models.

4.6 Determinism and Stochasticity

The transition operator T may be deterministic or stochastic. In the stochastic case, T defines a probability measure over possible successor states:

$$\psi_{t+1} \sim P_T(\psi_t).$$

This formulation naturally accommodates probabilistic computation, noisy systems, and uncertainty without introducing additional ad hoc structures.

4.7 Minimality of the Definition

The definition above is intentionally minimal. No assumptions are made regarding:

- logical completeness,
- computational universality,
- physical realizability,
- reversibility or irreversibility.

These properties may emerge or be imposed in specific realizations, but they are not intrinsic to the definition of a Quansistor itself.

This minimal formal structure establishes the Quansistor as a general operator-based computational primitive. Subsequent chapters will explore how additional structure—such as operator classes, field dynamics, and compositional rules—can be introduced to recover known computational models and to define novel ones.

5 Computational Fields

The concept of a computational field is central to the Quansistor framework. While a Quansistor defines a localized state-transition operator, the computational field provides the structural and contextual environment in which such operators exist, interact, and collectively give rise to computation. Without a field, a Quansistor would be an isolated dynamical object; with a field, it becomes part of a coherent computational system.

5.1 Motivation for Computational Fields

Traditional computational models often assume an implicit global structure. In classical circuit models, this structure is encoded in the wiring diagram. In software systems, it appears as control flow graphs or memory hierarchies. In quantum computation, circuit topology and qubit connectivity play a similar role. These structures are typically treated as external to the computational primitives themselves.

The Quansistor framework makes this structure explicit by elevating it to the level of a computational field. The field encodes how individual Quansistors are arranged, how they influence one another, and how information or state propagates through the system. This shift enables a more natural description of distributed, parallel, and emergent computation.

5.2 Definition of a Computational Field

Informally, a computational field F is a structured space that assigns interaction rules to a collection of Quansistors. Formally, the field may be described by a tuple

$$F := (Q, E, C),$$

where:

- Q is a set of Quansistors,
- E encodes adjacency or interaction relations between elements of Q ,
- C specifies constraints governing interaction, composition, and evolution.

The precise mathematical nature of E and C is left open. They may be represented as graphs, manifolds, metric spaces, categories, or other relational structures, depending on the intended application.

5.3 Locality and Interaction

A defining feature of computational fields is the notion of locality. In many systems, a Quansistor interacts primarily with a limited neighborhood determined by the field structure. This locality may be spatial, topological, informational, or abstract.

Local interaction rules constrain the flow of state information and give rise to global behavior through repeated application. This mirrors physical systems, where local interactions governed by field equations produce large-scale patterns and dynamics. In computational terms, locality supports scalability, modularity, and robustness.

5.4 Field Dynamics

The computational field itself may be static or dynamic. In a static field, interaction relations and constraints remain fixed over time. In a dynamic field, the structure of F may evolve in response to the system state, external inputs, or internal adaptation processes.

Dynamic fields allow the modeling of self-organizing computation, adaptive architectures, and systems in which the rules of interaction are not fixed in advance. In such cases, the evolution of the field becomes an integral part of the computation rather than a passive background.

5.5 Symmetry and Constraints

Computational fields may exhibit symmetries that constrain allowable interactions and transformations. These symmetries can encode invariances, conservation laws, or equivalence relations between Quansistors. For example, translational symmetry may imply uniform behavior across regions of the field, while broken symmetry may signal the emergence of structure or specialization.

Constraints encoded in C serve to restrict the operator algebra A and its admissible compositions. Such constraints may be logical, algebraic, physical, or informational in nature.

5.6 Emergence and Global Behavior

One of the primary motivations for introducing computational fields is their capacity to support emergent behavior. When many Quansistors interact according to simple local rules, the resulting global dynamics may exhibit patterns, structures, or computational capabilities not evident at the level of individual components.

This perspective aligns with field-based approaches in physics and complex systems theory, where macroscopic phenomena arise from microscopic interactions. In the Quansistor framework, computation itself is viewed as an emergent property of field-coupled operator dynamics.

5.7 Relation to Existing Models

Computational fields generalize several familiar constructs:

- wiring diagrams in classical circuits,
- connectivity graphs in quantum hardware,
- layers and weight matrices in neural networks,
- interaction graphs in distributed systems.

By unifying these under a single abstract notion, the Quansistor framework provides a common language for describing diverse computational architectures.

The next chapter examines the classes of operators that may define Quansistor dynamics and explores how different choices of operator structure lead to distinct computational behaviors.

6 Operator Classes and Dynamics

The behavior of a Quansistor is determined primarily by the properties of its transition operator. While the formal definition introduced in Chapter 4 places minimal constraints on this operator, meaningful computational behavior arises only when specific operator classes and their associated dynamics are considered. This chapter surveys several important classes of operators and examines the types of computational processes they enable.

6.1 Linear Operators

A Quansistor is said to be linear if its transition operator T satisfies

$$T(\alpha\psi_1 + \beta\psi_2) = \alpha T(\psi_1) + \beta T(\psi_2)$$

for all admissible states $\psi_1, \psi_2 \in S$ and scalars α, β .

Linear Quansistors admit powerful analytical tools, including spectral decomposition and superposition. They naturally encompass classical linear systems, quantum unitary evolution, and linear signal processing. In such systems, computation may be interpreted as the propagation and interference of state components under repeated operator action.

6.2 Nonlinear Operators

Nonlinear Quansistors arise when the transition operator violates linearity:

$$T(\psi_1 + \psi_2) \neq T(\psi_1) + T(\psi_2).$$

Nonlinearity enables rich computational phenomena such as pattern formation, bifurcation, and chaos. Many biologically inspired and adaptive computational models, including neural and reservoir computing systems, can be understood as nonlinear Quansistor networks. Nonlinear operators allow the system to amplify small differences in initial conditions, making them particularly suited to decision-making and classification tasks.

6.3 Stochastic Operators

In stochastic Quansistors, the transition operator defines a probability distribution over successor states rather than a single deterministic outcome:

$$\psi_{t+1} \sim P_T(\psi_t).$$

Such operators are essential for modeling uncertainty, noise, and probabilistic reasoning. They subsume Markov processes, randomized algorithms, and probabilistic automata. Within the Quansistor framework, stochasticity is treated as a first-class feature rather than as an external perturbation.

6.4 Time-Dependent Operators

In many systems, the operator governing state evolution may itself vary with time:

$$\psi_{t+1} = T_t(\psi_t).$$

Time-dependent Quansistors enable adaptive and learning behavior, where the transformation rules evolve in response to internal state or external input. This formulation naturally accommodates training processes, parameter updates, and environment-driven adaptation.

6.5 Spectral and Modal Structure

For operators admitting a spectral decomposition, eigenvalues and eigenstates play a central role in understanding system dynamics:

$$T\psi_n = \lambda_n\psi_n.$$

Spectral properties determine stability, convergence, and oscillatory behavior. In computational terms, eigenmodes may correspond to stable computational patterns, attractors, or long-term memory states. Spectral analysis provides a powerful lens for studying both linear and nonlinear Quansistor dynamics.

6.6 Stability and Dynamical Regimes

Repeated application of a Quansistor operator may lead to qualitatively different dynamical regimes:

- convergence to fixed points,
- periodic or quasi-periodic oscillations,
- chaotic behavior,
- metastable or transient dynamics.

The computational significance of these regimes depends on context. Fixed points may encode solutions or decisions, oscillations may represent cyclic processes, and chaotic regimes may support exploration or generative behavior.

6.7 Hybrid Operator Classes

Many practical systems combine multiple operator types. A single Quansistor network may involve linear propagation, nonlinear activation, stochastic perturbation, and time-dependent adaptation. The Quansistor framework accommodates such hybrid systems without requiring separate theoretical treatments for each component.

This unifying perspective allows diverse computational mechanisms to be analyzed within a common operator-dynamical language.

The next chapter explores how individual Quansistors can be composed into networks and how large-scale computational behavior emerges from their interactions within a computational field.

7 Composition, Networks, and Scaling

While a single Quansistor defines a localized state-transition mechanism, meaningful computation typically arises only when many such elements interact. This chapter examines how Quansistors may be composed into networks, how their interactions are structured by the computational field, and how large-scale computational behavior emerges through scaling.

7.1 Composition of Quansistors

At the most basic level, Quansistors may be composed sequentially or in parallel. Given two Quansistors with transition operators T_1 and T_2 , sequential composition is defined by operator composition:

$$T_{\text{seq}} = T_2 \circ T_1,$$

corresponding to the successive application of state transformations.

Parallel composition, by contrast, acts on product state spaces:

$$T_{\text{par}} = T_1 \oplus T_2,$$

allowing independent or weakly coupled evolution of subsystems. These two modes of composition generalize familiar notions from classical circuits, functional programming, and concurrent computation.

7.2 Field-Coupled Networks

Beyond simple composition, Quansistors interact through the computational field. In a field-coupled network, the transition operator of each Quansistor may depend on the states of neighboring elements:

$$\psi_i^{(t+1)} = T_i(\psi_i^{(t)}, \{\psi_j^{(t)}\}_{j \in N(i)}),$$

where $N(i)$ denotes the neighborhood of Quansistor i as defined by the field structure.

This formulation captures a wide range of distributed and parallel computational models, including cellular automata, neural fields, and interacting particle systems. Computation is expressed as a collective evolution rather than as a centralized process.

7.3 Network Topologies

The computational field determines the topology of Quansistor networks. Possible topologies include:

- regular lattices,
- random or small-world graphs,
- hierarchical or modular structures,
- dynamically evolving topologies.

Different topologies give rise to different computational properties, such as robustness, adaptability, and communication efficiency. The Quansistor framework allows these properties to be studied at the level of operator interaction rather than at the level of hardware or implementation.

7.4 Scaling Behavior

As the number of Quansistors increases, the system may exhibit qualitatively new behavior. Scaling may lead to:

- emergent computational phases,
- collective memory and pattern formation,
- phase transitions between dynamical regimes,
- increased fault tolerance through redundancy.

These phenomena cannot be understood by analyzing individual Quansistors in isolation. Instead, they arise from the interplay between operator dynamics and field structure.

7.5 Modularity and Hierarchy

Large Quansistor networks may be organized into modules, each of which functions as a higher-level Quansistor with its own effective state space and transition operator. This hierarchical organization supports abstraction and reuse, enabling complex systems to be constructed from simpler components.

Such modularity mirrors principles found in biological systems, software engineering, and multi-scale physical models.

7.6 Robustness and Fault Tolerance

Field-coupled Quansistor networks may exhibit robustness to local failures or perturbations. Because computation is distributed across many interacting elements, the system can often tolerate the loss or malfunction of individual Quansistors without catastrophic failure. This robustness is a direct consequence of the field-based, operator-centric design and stands in contrast to tightly synchronized, centrally controlled computational architectures.

7.7 Toward Universal Computation

Under appropriate conditions, networks of Quansistors may achieve computational universality. Rather than being defined in terms of specific gate sets or instruction sequences, universality in this framework emerges from the expressive power of the operator classes and the richness of the computational field.

This perspective reframes universality as a property of collective dynamics rather than of isolated primitives.

The next chapter situates the Quansistor framework relative to existing computational paradigms, highlighting both points of correspondence and fundamental differences.

8 Relation to Existing Computational Paradigms

The Quansistor framework is not intended to replace established models of computation, but to provide a unifying theoretical perspective in which such models can be naturally embedded and compared. This chapter examines the relationship between Quansistors and several prominent computational paradigms, highlighting both correspondences and conceptual extensions.

8.1 Classical Digital Computation

Classical digital computation is based on discrete state spaces and deterministic transition rules implemented through logic gates and finite circuits. Within the Quansistor framework, a classical logic gate may be viewed as a highly constrained Quansistor whose state space is binary and whose transition operator implements a fixed Boolean function.

The computational field in this case corresponds to the circuit wiring diagram, which determines how gates are composed and how signals propagate. From this perspective, classical digital computation emerges as a special case of Quansistor dynamics characterized by strict locality, determinism, and fixed operator structure.

8.2 Turing Machines and Symbolic Models

Symbolic models of computation, such as Turing machines and lambda calculus, emphasize rule-based manipulation of discrete symbols. These models can be interpreted as Quansistor systems in which the state space encodes symbolic configurations and the transition operator represents rule application.

However, traditional symbolic models assume a centralized control mechanism and a linear progression of computation steps. The Quansistor framework generalizes this by allowing decentralized, field-coupled rule application and by removing the requirement of a global control flow.

8.3 Quantum Computation

Quantum computation introduces complex-valued state spaces and unitary evolution. In the Quansistor framework, a quantum gate is a linear Quansistor whose operator acts on a Hilbert space and satisfies unitarity constraints.

While quantum computation expands the expressive power of state evolution, it typically remains embedded in a circuit-based paradigm. Quansistors generalize this approach by allowing quantum-like operators to interact within more flexible field structures and by accommodating non-unitary, open-system dynamics.

8.4 Probabilistic and Stochastic Models

Probabilistic computation models uncertainty through stochastic state transitions. Markov chains, randomized algorithms, and probabilistic automata can all be expressed as networks of stochastic Quansistors.

The key difference lies in abstraction: rather than treating randomness as an external feature, the Quansistor framework integrates stochasticity directly into the operator definition, enabling hybrid deterministic-stochastic systems to be analyzed within a single formalism.

8.5 Neural and Bio-Inspired Computation

Neural networks and other bio-inspired models rely on nonlinear activation dynamics and distributed processing. These systems align naturally with the Quansistor framework, as each neuron or functional unit can be interpreted as a nonlinear Quansistor embedded in a computational field defined by synaptic connectivity.

The Quansistor abstraction, however, extends beyond layered network architectures by allowing arbitrary field topologies, dynamic connectivity, and operator adaptation, thereby encompassing a broader class of learning and adaptive systems.

8.6 Analog and Continuous Computation

Analog computation operates on continuous state variables and often exploits physical dynamics directly. Such systems may be modeled as Quansistor networks with continuous state spaces and differential operator dynamics.

By treating analog evolution as a form of operator-driven state transition, the Quansistor framework provides a bridge between digital abstraction and physical dynamical systems.

8.7 Summary and Conceptual Positioning

Across these paradigms, a common pattern emerges: computation can be understood as the evolution of states under structured transformation rules. The Quansistor framework captures this pattern at an abstract level, allowing diverse models to be expressed as particular instantiations of a more general operator-field theory of computation.

This unifying perspective does not diminish the importance of existing models. Instead, it clarifies their assumptions, reveals their relationships, and opens the possibility of hybrid systems that combine features traditionally treated as separate or incompatible.

The next chapter explores how Quansistors may be realized in both physical and abstract settings, emphasizing the substrate-independent nature of the framework.

9 Physical and Abstract Realizations

The Quansistor is defined as an abstract computational primitive, independent of any particular physical substrate or implementation technology. This abstraction is intentional. It allows the concept to remain stable across changes in hardware, software paradigms, and scientific understanding. Nevertheless, abstraction does not imply disconnection from realizability. This chapter explores how Quansistors may be realized in both physical and abstract contexts, while preserving their defining operator-based structure.

9.1 Substrate Independence

A central principle of the Quansistor framework is substrate independence. The definition of a Quansistor relies only on the existence of a state space, an operator algebra, and a computational field. No assumptions are made about the material, energetic, or technological nature of these components.

As a result, the same Quansistor description may be instantiated in radically different forms. This mirrors the relationship between abstract computation and its realizations in classical and quantum systems, but extends it by removing reliance on any specific logical or physical encoding.

9.2 Physical Realizations

In a physical setting, Quansistors may be realized through systems whose natural dynamics correspond to operator-driven state evolution. Examples include:

- electronic systems governed by analog or digital signal dynamics,
- optical or photonic systems with field-based interactions,
- mechanical or fluid systems exhibiting controllable dynamical behavior,
- quantum or hybrid quantum–classical systems.

In such cases, the computational field corresponds to the physical interaction structure, while the transition operator emerges from the system’s governing equations. The Quansistor abstraction allows these diverse systems to be described using a common language, even when their physical mechanisms differ substantially.

9.3 Software and Algorithmic Realizations

Quansistors may also be realized entirely within software. In this context, the state space is represented by data structures, the operator algebra by functions or transformations, and the computational field by interaction graphs or scheduling rules.

Simulation-based realizations are particularly valuable for exploring novel operator dynamics and field structures that may not yet be physically realizable. Such implementations demonstrate that the Quansistor framework is compatible with conventional computing environments while remaining conceptually distinct from them.

9.4 Mathematical and Symbolic Realizations

Beyond physical and software systems, Quansistors can be realized as purely mathematical objects. Operator algebras acting on abstract state spaces may be studied independently of any

execution mechanism. Symbolic rewriting systems, formal grammars, and algebraic dynamical systems all admit interpretation within the Quansistor framework.

These realizations emphasize that computation need not be understood solely as a process that “runs” in time, but may also be analyzed as a static or relational structure defined by operator properties and field constraints.

9.5 Hybrid and Multi-Level Realizations

One of the strengths of the Quansistor framework is its ability to support hybrid realizations. A single system may combine physical dynamics, software control, and abstract operator logic across multiple levels of description. For example, low-level physical processes may implement basic operator dynamics, while higher-level Quansistors emerge through aggregation and abstraction.

Such multi-level realizations align with contemporary trends in computing, where hardware, firmware, software, and algorithmic layers interact in complex ways.

9.6 Limits of Realization

While the Quansistor framework is intentionally general, not every abstract Quansistor is necessarily physically realizable. Constraints arising from physics, resources, or complexity may limit which operator dynamics can be instantiated in practice.

Importantly, these limitations do not undermine the value of the abstract definition. On the contrary, they clarify the distinction between theoretical possibility and practical implementation, allowing realizability to be studied as a separate question rather than being conflated with definitional constraints.

9.7 Role of Realizations in Theory Development

Realizations serve a dual role within the Quansistor framework. They provide concrete examples that ground abstract concepts, and they offer experimental testbeds for exploring new forms of computation. At the same time, the theory remains primary: realizations are instances of the framework, not its defining elements.

The next chapter turns to interpretational perspectives, examining how the Quansistor may be understood conceptually across disciplines such as mathematics, physics, and computer science.

10 Interpretational Perspectives

Beyond its formal definition and potential realizations, the Quansistor admits multiple interpretational perspectives. These interpretations do not alter the underlying mathematical structure, but rather provide complementary ways of understanding its role and significance across different domains. This chapter surveys several such perspectives, highlighting how the Quansistor may be situated conceptually within mathematics, physics, and computation theory.

10.1 The Quansistor as an Operator

From a mathematical standpoint, the Quansistor may be understood most directly as an operator acting on a state space. In this interpretation, the emphasis is placed on the properties of the transition operator: its algebraic structure, spectral characteristics, and dynamical behavior under iteration.

Computation, in this view, becomes the study of operator-induced evolution. Questions of interest include stability, convergence, invariant subspaces, and the long-term behavior of trajectories. This perspective aligns naturally with functional analysis, operator algebras, and dynamical systems theory.

10.2 The Quansistor as a Field Excitation

Within a field-theoretic interpretation, a Quansistor may be regarded as a localized excitation or mode within a computational field. Rather than being an isolated object, it derives its identity and behavior from its interactions with the surrounding field.

This interpretation emphasizes relationality and context. The computational field defines coupling, locality, and symmetry, while Quansistors serve as carriers of state transformation within that field. Computation emerges as a collective phenomenon, analogous to physical processes described by interacting fields and excitations.

10.3 The Quansistor as a Causal Unit

Another interpretation frames the Quansistor as a unit of algorithmic causality. In this view, each Quansistor encodes a causal relationship between states: given a particular configuration, it determines how that configuration may evolve.

This perspective shifts attention from representation to transformation. Rather than asking how information is stored, it asks how influence propagates through a system. Such a causal interpretation is particularly relevant for distributed systems, concurrent computation, and models of computation in which control flow is emergent rather than predefined.

10.4 The Quansistor as a Computational Particle

By analogy with physics, the Quansistor may be interpreted as a computational particle. Just as particles are defined by their interactions and transformation rules rather than by static substance, Quansistors are defined by their operator dynamics within a computational field.

This particle-like interpretation is not meant to suggest physical materiality, but to highlight discreteness, interaction, and composability. Quansistors may combine, scatter, synchronize, or form bound structures, giving rise to higher-level computational entities.

10.5 The Quansistor as a Unifying Abstraction

Across these interpretations, a unifying theme emerges: the Quansistor serves as an abstraction that bridges multiple ways of thinking about computation. It connects symbolic manipulation with dynamical evolution, discrete logic with continuous fields, and local operations with global behavior.

This unifying role does not collapse distinctions between paradigms, but rather clarifies their relationships. By providing a common conceptual language, the Quansistor enables insights to be transferred across disciplinary boundaries.

10.6 Limits of Interpretation

No single interpretation exhausts the meaning of the Quansistor. Each highlights certain aspects while leaving others in the background. The strength of the framework lies precisely in its openness to multiple perspectives, allowing different interpretations to coexist without contradiction.

This interpretational flexibility reflects the generality of the formal definition and supports the use of the Quansistor as a foundational concept rather than as a narrowly defined mechanism.

The following chapter considers the broader implications of the Quansistor framework and outlines directions for future research and development.

c

11 Implications and Future Directions

The introduction of the Quansistor as a generalized operator-based computational primitive carries implications that extend beyond any single computational paradigm or implementation domain. By reframing computation in terms of state evolution within a computational field, the Quansistor framework opens new directions for theory, system design, and interdisciplinary research. This chapter outlines several key implications and identifies avenues for future exploration.

11.1 Implications for the Theory of Computation

At a theoretical level, the Quansistor challenges the primacy of discrete, symbol-centric models as the sole foundation of computation. While such models remain indispensable for defining computability and complexity, the Quansistor suggests a complementary perspective in which computation is treated as a dynamical process governed by operators and interactions.

This shift enables foundational questions to be posed in new ways. For example, notions of universality, expressiveness, and efficiency may be studied in terms of operator classes and field structures rather than in terms of gate sets or instruction sequences. Complexity may be reframed as a property of dynamical regimes and interaction patterns rather than solely as asymptotic resource usage.

11.2 Implications for System Architecture

From an architectural standpoint, the Quansistor framework encourages designs that emphasize modularity, locality, and interaction over centralized control. Systems built from Quansistors may naturally support parallelism, adaptability, and robustness through their field-coupled organization.

This perspective aligns with emerging trends in computing, including distributed systems, heterogeneous architectures, and adaptive hardware–software co-design. By providing a substrate-independent abstraction, the Quansistor enables architectural principles to be discussed without premature commitment to specific technologies.

11.3 Hybrid and Post-Classical Computation

One of the most significant implications of the Quansistor framework is its support for hybrid computational systems. Classical, quantum, probabilistic, and bio-inspired elements need not be treated as fundamentally distinct categories, but as different realizations of operator dynamics within a shared field.

Such hybridization may prove essential for addressing complex problems that resist treatment by any single paradigm. The Quansistor provides a conceptual foundation for exploring these combinations systematically rather than ad hoc.

11.4 Formal Analysis and Verification

The operator-centric nature of the Quansistor opens the door to new methods of formal analysis and verification. Properties of computational systems may be studied through spectral analysis, stability theory, and invariant structures associated with operator algebras.

This approach has the potential to complement traditional program verification techniques, particularly for systems characterized by continuous dynamics, concurrency, or emergent behavior.

11.5 Open Research Directions

The Quansistor framework raises a number of open questions that warrant further investigation:

- characterization of computational universality in operator-field systems,
- development of complexity measures appropriate to field-coupled computation,
- criteria for physical realizability of abstract Quansistor dynamics,
- learning and adaptation mechanisms expressed as operator evolution,
- connections to information geometry and statistical mechanics.

Addressing these questions will require collaboration across mathematics, computer science, physics, and engineering.

11.6 Long-Term Perspective

In the long term, the Quansistor may serve as a foundational abstraction for computational systems that have yet to be conceived. As computing continues to move beyond rigid architectures and toward adaptive, distributed, and physically integrated systems, a theory that places operators and interactions at its core may become increasingly relevant.

The value of the Quansistor lies not in prescribing a particular future, but in providing a flexible conceptual framework capable of accommodating many possible futures.

The final chapter concludes this work by summarizing the central contributions and reaffirming the role of the Quansistor as a unifying computational primitive.

12 Conclusion

This work has introduced the concept of the Quansistor as a generalized, operator-based computational primitive. By abstracting computation away from specific physical devices, logical encodings, and architectural conventions, the Quansistor provides a unifying framework capable of encompassing classical, quantum, probabilistic, symbolic, and hybrid forms of computation.

The central contribution of this framework lies in its redefinition of computation as the evolution of states within a computational field under the action of operators. This perspective shifts attention from discrete instruction sequences and hardware-centric primitives to interaction, dynamics, and structure. In doing so, it offers a language in which diverse computational paradigms may be expressed, compared, and combined.

Throughout the preceding chapters, the Quansistor has been developed systematically. Beginning with motivation and informal intuition, the framework was formalized through precise definitions of state spaces, operator algebras, and computational fields. Subsequent chapters explored operator classes, network composition, scaling behavior, and relationships to existing computational

models. The discussion of realizations and interpretational perspectives emphasized both the generality and the flexibility of the concept.

Importantly, the Quansistor is not proposed as a replacement for established computational models, nor as a specific machine or architecture. Rather, it is intended as a foundational abstraction that clarifies the common structure underlying many forms of computation. Its value lies in its ability to separate essential computational principles from contingent implementation details.

As computation continues to evolve toward increasingly distributed, adaptive, and heterogeneous systems, theoretical frameworks that emphasize interaction and dynamics may become indispensable. The Quansistor offers one such framework, providing a stable conceptual foundation upon which future computational theories and systems may be built.

In this sense, the Quansistor represents not an endpoint, but a beginning: an invitation to explore computation through the lens of operators and fields, and to rethink the fundamental units from which computational reality is constructed.

APPENDIX A

A.1 Minimal Definition

Definition. A Quansistor is an operator-defined state-transition primitive acting within a computational field.

More explicitly, a Quansistor consists of:

- a state space S ,
- a transition operator $T : S \rightarrow S$,
- a computational field F that governs interaction and composition.

The fundamental evolution rule is given by

$$\psi_{t+1} = T(\psi_t),$$

with no a priori restriction on the nature of S , the structure of T , or the form of F .

A.1.1 Scope of the Minimal Definition

This minimal definition intentionally avoids assumptions regarding:

- discreteness or continuity of the state space,
- determinism or stochasticity of the operator,
- linearity or nonlinearity,
- physical realizability,
- computational universality.

All such properties arise only through specialization or additional constraints. The minimal definition is therefore maximally inclusive and serves as a foundational reference point for all subsequent elaborations of the Quansistor concept.

A.1.2 Purpose

The purpose of this minimal definition is twofold:

1. to provide a concise, portable statement of the Quansistor concept suitable for citation and comparison,
2. to separate the essential structure of the Quansistor from contingent interpretations and implementations.

In this sense, Appendix A functions as the definitional core of the entire framework.

APPENDIX B

B.1 Relation to Classical Transistors

The term Quansistor is intentionally reminiscent of the classical transistor, reflecting a conceptual lineage rather than a direct equivalence. This appendix clarifies the relationship between the two and situates the classical transistor as a special, highly constrained instance within the broader Quansistor framework.

B.1.1 The Classical Transistor

A classical transistor functions as a threshold-based switching device. Its computational role is defined by:

- a binary or near-binary state space,
- deterministic transition behavior,
- strict locality of interaction,
- fixed physical realization.

In digital circuits, transistors implement Boolean logic by switching between voltage levels, enabling the construction of logic gates and larger computational structures.

B.1.2 Transistor as a Restricted Quansistor

From the Quansistor perspective, a classical transistor may be interpreted as a Quansistor with the following constraints:

- the state space S is discrete and binary,
- the transition operator T implements a Boolean function,
- the computational field F is static and defined by circuit wiring,
- no stochasticity, adaptivity, or nonlocal interaction is present.

Under these restrictions, the Quansistor reduces to a deterministic switching element indistinguishable from a classical transistor.

B.1.3 Conceptual Generalization

The Quansistor generalizes the transistor by relaxing each of these constraints. In particular, it allows:

- arbitrary state spaces beyond binary encodings,
- operator dynamics that are nonlinear, stochastic, or time-dependent,
- interaction structures defined by fields rather than fixed wiring,
- realizations independent of any specific physical device.

This generalization does not invalidate the transistor model, but places it within a larger conceptual hierarchy.

B.1.4 Terminological Significance

The choice of the term Quansistor reflects an intentional shift in emphasis:

- from voltage levels to state spaces,
- from switching to transformation,
- from devices to operators,
- from circuits to fields.

In this sense, the Quansistor may be viewed as the conceptual successor to the transistor at the level of theory, just as the transistor once generalized earlier mechanical and electromechanical switching devices.

B.1.5 Summary

A classical transistor is not replaced by the Quansistor; it is contained within it as a limiting case. The Quansistor framework thus preserves the insights of classical digital computation while extending them into a more general, substrate-independent theory of computation.

QFM: Quansistor Field Mathematics

An Operator-First Arithmetic and Computational Framework

Enter Yourname

Abstract

This section introduces *Quansistor Field Mathematics (QFM)* as a unifying mathematical framework designed to formalize computation, dynamics, and information processing in terms of operator fields acting on structured state spaces. QFM extends the quansistor concept by providing a rigorous mathematical language in which collections of interacting operators are treated as primary objects, and computational behavior emerges from their algebraic, spectral, and compositional properties.

At its core, QFM replaces the traditional notion of discrete instruction execution with a continuous and structured operator perspective. States are elements of abstract spaces—typically Hilbert or Banach spaces—while computation is described as the evolution of these states under the action of operator families. This shift enables the direct application of tools from functional analysis, operator algebras, spectral theory, and representation theory to problems that are traditionally considered computational or algorithmic in nature.

A central idea of QFM is that operators are not isolated transformations, but members of structured fields whose interactions define global system behavior. These operator fields may encode locality, symmetry, scaling, or arithmetic constraints, allowing mathematical structure to directly shape computational dynamics. In this sense, QFM provides a natural bridge between abstract mathematics and executable systems, where formal properties such as self-adjointness, boundedness, or spectral stability acquire concrete computational meaning.

The framework is particularly well-suited for describing distributed and deterministic computation. Since operator application is treated as a mathematically well-defined process, execution traces can be reconstructed, replayed, and verified independently of runtime artifacts. This enables a principled approach to auditability, reproducibility, and correctness, which are increasingly critical in large-scale and decentralized computational environments.

QFM also serves as the mathematical substrate for more specialized constructions, including the SMRK Hamiltonian, spectral ladder operators, and trace-based formulations of arithmetic dynamics. Within this

context, QFM provides the ambient structure in which such operators are defined, composed, and analyzed. Rather than introducing new mathematical objects in isolation, these constructions are understood as specific realizations of the general operator-field paradigm.

Finally, this section positions QFM as a foundational layer for Quansistor Field Computing (QFC) and the Quantum Virtual Machine (QVM) architecture developed in later parts of this compendium. By grounding computation in operator field mathematics, QFM enables a coherent transition from abstract theory to executable systems, while preserving mathematical rigor and semantic clarity throughout the computational stack.

1 Axiomatic Foundations of Quansistor Field Mathematics

1.1 Motivation for an Operator-Based Framework

Modern computation rests on two dominant paradigms:

1. Classical deterministic state machines, evolving via fixed logical transitions.
2. Quantum systems, evolving under unitary operators on complex Hilbert spaces.

While powerful, both paradigms exhibit intrinsic limitations. Classical computation lacks native algebraic parallelism and spectral structure, whereas quantum computation—despite its expressive power—is fragile, hardware-constrained, and fundamentally limited by decoherence and error correction overhead.

Quansistor Field Mathematics (QFM) is introduced as a unifying and transcending framework. It defines computation as operator-driven field dynamics acting on discrete, structured index sets. The elementary units of computation—quansistors—are abstract computational atoms whose interactions are governed by linear operators on a generalized Hilbert-like space.

QFM establishes a form of *computational physics*: an axiomatized operator field theory for information propagation, interaction, and spectral evolution.

1.2 Interdisciplinary Motivation

QFM sits at the intersection of several domains:

- operator theory and functional analysis,
- arithmetic geometry and prime dynamics,
- distributed systems and replicated computation,
- emergent intelligence and spectral learning phenomena.

Classical Hilbert spaces are insufficient for this synthesis because:

- amplitudes may take values in general algebras beyond $\mathbb{R}$ or $\mathbb{C}$,
- evolution need not be unitary,
- operators must admit distributed decomposition,
- computation must reconcile locality with global spectral integration.

These constraints motivate the axiomatic foundation of QFM.

1.3 Axiom I — Quansistor Locality

Let $\mathcal{Q}$ be a finite or countable index set of quansistors. Each quansistor $q \in \mathcal{Q}$ is associated with a finite neighborhood $N(q) \subset \mathcal{Q}$.

Axiom I (Locality). All QFM evolution operators act locally: propagation of amplitude from q may occur only within $N(q) \cup \{q\}$.

Formally, for any operator T and field ψ ,

$$\text{supp}(T\psi)(q) \subseteq N(q) \cup \{q\}.$$

This generalizes locality in lattice models, adjacency in graphs, and multiplicative relations such as $n \mapsto pn$ in arithmetic dynamics. Locality ensures scalability, decomposability, and compatibility with distributed execution.

1.4 Axiom II — Linearity

Axiom II (Linearity). All QFM evolution is governed by linear operators.

For any operator T and scalars a, b ,

$$T(a\psi + b\phi) = aT\psi + bT\phi.$$

Linearity enables spectral analysis, operator decomposition, Hamiltonian formulation, and deterministic recomposition in distributed environments. QFM does not require unitarity; linearity is the sole structural constraint.

1.5 Axiom III — Generalized Amplitude Algebra

Axiom III (Amplitude Algebra). Field amplitudes take values in an algebra $\mathcal{A}$ equipped with addition, scalar multiplication, and a compatible norm or seminorm.

Optionally, $\mathcal{A}$ may admit an involution $*$ and noncommutative multiplication. Examples include:

- $\mathbb{R}$ or $\mathbb{C}$ (classical and quantum-like dynamics),
- matrix algebras (operator-valued amplitudes),
- finite fields (cryptographic computation),
- tensor and operator algebras (higher-order QFM).

The generalized inner product is defined by

$$\langle \psi, \phi \rangle = \sum_{q \in \mathcal{Q}} \psi(q)^* \phi(q),$$

interpreted within the algebraic structure of $\mathcal{A}$.

1.6 Axiom IV — Distributed Composability

Axiom IV (Distributed Composability). Every QFM operator admits a canonical decomposition into finitely supported local operators.

For any operator T , there exists a representation

$$T = \sum_{i=1}^M T_i,$$

where each T_i acts on a bounded region of $\mathcal{Q}$, is independently executable, and deterministically composable.

This axiom guarantees parallelism, fault tolerance, and verifiable replay, enabling QFM to operate as a physically realizable computational model.

1.7 Axiom V — Spectral Sovereignty

Axiom V (Spectral Sovereignty). The spectrum of a QFM Hamiltonian fully determines long-term field behavior.

Hamiltonians take the general form

$$H = \sum_k \alpha_k A_k + \beta_k B_k + V,$$

where A_k and B_k are transfer operators and V is a potential.

This axiom reflects a unifying philosophy shared by quantum mechanics, dynamical systems, number theory, and learning theory: *spectrum equals structure*.

1.8 Axiom VI — Operator Universality

Axiom VI (Universality). Every computation expressible in QFM arises from transfer operators, potentials, and their compositions.

Time evolution is generated by

$$U(t) = e^{-tH},$$

or suitable discrete approximations, ensuring completeness and compatibility with Hamiltonian-based computation.

1.9 Axiom VII — Physical–Arithmetic Duality

Axiom VII (Duality). Every operator family in QFM admits both physical and arithmetic interpretations.

For example, the operator

$$(A_p \psi)(n) = \psi(pn)$$

represents either spatial scaling in a field or prime multiplication in arithmetic dynamics.

This duality enables arithmetic Hamiltonians, zeta-operator constructions, and unified simulation architectures.

1.10 Summary of the Axiomatic System

The axioms of QFM define a consistent mathematical universe in which quansistor fields evolve: local, linear, algebraically flexible, spectrally governed, universally expressive, and dual in interpretation.

2 Quansistor Field State Space

2.1 Overview

QFM requires a mathematical habitat in which quansistors exist, interact, and propagate. This habitat is a generalized Hilbert-like function space whose amplitudes take values in an arbitrary algebra.

Unlike classical quantum mechanics, which employs wavefunctions $\psi : \mathbb{R}^n \rightarrow \mathbb{C}$, QFM fields inhabit discrete index sets endowed with neighborhood structure and algebraic flexibility.

2.2 The Quansistor Index Set

Definition 2.1 (Quansistor Index Set). A quansistor index set $\mathcal{Q}$ is a set equipped with:

- an optional topology or σ -algebra,
- a neighborhood function $N : \mathcal{Q} \rightarrow \mathcal{P}(\mathcal{Q})$,
- an optional group or semigroup action.

A central arithmetic example is

$$\mathcal{Q} = \mathbb{N}, \quad N(n) = \{pn, n/p \mid p \text{ prime}, p \mid n\},$$

capturing multiplicative adjacency.

2.3 Amplitude Algebra

The amplitude algebra $\mathcal{A}$ determines the type of information carried by each quansistor. It must support addition, scalar multiplication, and a norm or seminorm.

Admissible examples include real, complex, matrix-valued, finite-field, and operator algebras. QFM is therefore meta-universal with respect to amplitude representation.

2.4 The QFM State Space

A quansistor field is a function

$$\psi : \mathcal{Q} \rightarrow \mathcal{A}.$$

Define the norm

$$\|\psi\|^2 = \sum_{q \in \mathcal{Q}} \|\psi(q)\|^2.$$

Definition 2.2 (QFM State Space). The QFM state space is

$$\mathcal{H}_{\text{QFM}} = \ell^2(\mathcal{Q}, \mathcal{A}) = \left\{ \psi : \mathcal{Q} \rightarrow \mathcal{A} \mid \sum_{q \in \mathcal{Q}} \|\psi(q)\|^2 < \infty \right\}.$$

This space is complete and supports generalized inner products, even when $\mathcal{A}$ is noncommutative.

2.5 Inner Product and Convergence

The inner product is defined by

$$\langle \psi, \phi \rangle = \sum_{q \in \mathcal{Q}} \psi(q)^* \phi(q).$$

Weak and strong convergence are defined analogously to classical Hilbert spaces and play a critical role in spectral stability and distributed execution.

2.6 Superposition and Interference

QFM supports generalized superposition

$$\psi = \sum_i c_i \psi_i, \quad c_i \in \mathcal{A}.$$

Interference arises through noncommutativity, potential operators, and spectral effects, enabling quantum-like behavior without qubits.

2.7 Composite Fields and Tensor Products

Composite systems are modeled via tensor products

$$\mathcal{H}_{\text{QFM}}^{(1)} \otimes \mathcal{H}_{\text{QFM}}^{(2)},$$

supporting multi-agent systems, interacting fields, and entanglement-like correlations.

2.8 Embedding Classical, Quantum, and Arithmetic Systems

Classical Boolean states, quantum wavefunctions, arithmetic multiplicative dynamics, and distributed systems all embed naturally into $\mathcal{H}_{\text{QFM}}$.

2.9 Summary

The QFM state space generalizes Hilbert spaces, supports arbitrary amplitude algebras, and provides the foundational environment for all QFM operators and Hamiltonians.

3 Operator Algebra of Quansistor Field Mathematics

3.1 Overview

Quansistor Field Mathematics (QFM) is fundamentally an operator-based theory. All computation, propagation, interaction, and reasoning within QFM is expressed through linear operators acting on the quansistor field state space

$$\mathcal{H}_{\text{QFM}} = \ell^2(\mathcal{Q}, \mathcal{A}).$$

Where classical computation relies on state transition functions and quantum computation relies on unitary matrices, QFM constructs its computational fabric from a rich algebra of operators encoding:

- information propagation,
- neighborhood interactions,
- arithmetic transformations,
- potential shaping,
- distributed execution constraints,
- spectral structure.

This chapter develops the formal operator algebra underlying QFM, introduces its fundamental generators, and establishes identities essential for Hamiltonian and spectral analysis.

3.2 Linear Operators on the QFM State Space

Definition 3.1 (QFM Operator). A QFM operator is a linear map

$$T : \mathcal{H}_{\text{QFM}} \rightarrow \mathcal{H}_{\text{QFM}}$$

satisfying

$$T(a\psi + b\phi) = aT\psi + bT\phi$$

for all $\psi, \phi \in \mathcal{H}_{\text{QFM}}$ and scalars a, b .

Linearity ensures compatibility with spectral theory, operator composition, and distributed decomposition.

3.3 Operator Locality

By Axiom I, QFM operators respect locality.

Definition 3.2 (Local Operator). An operator T is local if for all $q \in \mathcal{Q}$,

$$\text{supp}(T\psi)(q) \subseteq N(q) \cup \{q\}.$$

Locality generalizes adjacency in graphs, lattice interactions in physics, and multiplicative relations in arithmetic dynamics. It is the structural property that enables scalable distributed execution.

3.4 Kernel and Matrix Representation

Despite its generality, every QFM operator admits a kernel representation:

$$(T\psi)(q) = \sum_{r \in \mathcal{Q}} T(q, r) \psi(r),$$

where $T(q, r) \in \mathcal{A}$ and locality implies that $T(q, r)$ is nonzero only when $r \in N(q) \cup \{q\}$.

This representation generalizes:

- matrices in finite-dimensional linear algebra,
- adjacency matrices in graph theory,
- integral kernels in functional analysis.

Unlike standard quantum mechanics, kernels in QFM may be non-Hermitian, operator-valued, or noncommutative.

3.5 Forward Transfer Operators

Transfer operators form the elementary generators of the QFM operator algebra.

Definition 3.3 (Forward Transfer Operator). Let $f : \mathcal{Q} \rightarrow \mathcal{Q}$ be a local map. The forward transfer operator A_f is defined by

$$(A_f \psi)(q) = \psi(f(q)).$$

Forward operators encode deterministic propagation and act as pullback operators $\psi \circ f$ in functional analysis.

Examples

- Graph shifts and lattice translations,
- Simulation stencils,
- Arithmetic multiplication $f(n) = pn$:

$$(A_p \psi)(n) = \psi(pn).$$

If f is multivalued, the operator generalizes to

$$(A_f \psi)(q) = \sum_{r \in f^{-1}(q)} \psi(r).$$

3.6 Backward Transfer Operators

Backward operators aggregate amplitude from preimages.

Definition 3.4 (Backward Transfer Operator). Given $f : \mathcal{Q} \rightarrow \mathcal{Q}$, define

$$(B_f \psi)(q) = \sum_{r: f(r)=q} \psi(r).$$

Backward operators generalize inverse-image propagation and are closely related to Perron–Frobenius operators.

Arithmetic Example

For $f(n) = pn$,

$$(B_p \psi)(n) = \mathbf{1}_{p|n} \psi(n/p).$$

3.7 Neighborhood Operators

Neighborhood operators average or mix amplitude across local neighborhoods.

Definition 3.5 (Neighborhood Operator). A neighborhood operator N acts as

$$(N\psi)(q) = \sum_{r \in N(q)} c(q, r) \psi(r),$$

with coefficients $c(q, r) \in \mathcal{A}$.

Special cases include:

- unweighted diffusion,
- weighted physical diffusion,
- graph Laplacians,
- multi-agent interaction kernels.

3.8 Potential and Multiplication Operators

Potential operators act diagonally.

Definition 3.6 (Potential Operator). A potential operator V acts as

$$(V\psi)(q) = V(q) \psi(q),$$

with $V(q) \in \mathcal{A}$.

Potentials encode:

- arithmetic weights such as $\Lambda(n)$ or $\log n$,
- physical energy landscapes,
- penalties and constraints,
- memory and activation in QVM.

3.9 Operator Products

Given operators T_1, T_2 , their product is defined by

$$(T_1 T_2)(\psi) = T_1(T_2 \psi).$$

Products represent sequential computation and form the basis of algorithmic composition within QFM.

3.10 Adjoint Operators

If the amplitude algebra admits an involution $*$, adjoints are defined by:

$$\langle T\psi, \phi \rangle = \langle \psi, T^* \phi \rangle.$$

Fundamental Arithmetic Identity

For arithmetic transfer operators on $\mathcal{Q} = \mathbb{N}$:

$$A_p^* = p B_p.$$

This identity reflects measure distortion under multiplicative maps and is central to constructing self-adjoint QFM Hamiltonians.

3.11 Commutators and Noncommutativity

The commutator of operators is defined by

$$[T_1, T_2] = T_1 T_2 - T_2 T_1.$$

In general,

$$[A_p, B_q] \neq 0.$$

Noncommutativity governs:

- interference phenomena,
- uncertainty-like relations,
- arithmetic operator algebras,
- emergent reasoning dynamics in QVM.

3.12 The QFM Operator Algebra

Definition 3.7 (QFM Operator Algebra). The QFM operator algebra $\mathcal{O}(\mathcal{Q})$ is the smallest algebra containing:

- all forward operators A_f ,
- all backward operators B_f ,
- all potential operators V ,
- all finite sums, products, and limits thereof.

Formally,

$$\mathcal{O}(\mathcal{Q}) = \overline{\text{span}}\{A_f, B_f, V\}.$$

This algebra generalizes C*-algebras, adjacency algebras, and Hecke algebras.

3.13 Canonical Operator Decomposition

Every operator $T \in \mathcal{O}(\mathcal{Q})$ admits a representation:

$$T = \sum_k \alpha_k A_{f_k} + \beta_k B_{g_k} + V.$$

This decomposition is the canonical normal form for QFM dynamics.

3.14 Self-Adjoint Balancing

To construct self-adjoint Hamiltonians, forward and backward operators must be balanced.

Definition 3.8 (Balanced Arithmetic Operator). For prime p , define

$$K_p = \frac{1}{p} A_p + p B_p.$$

Using $A_p^* = p B_p$, we obtain

$$K_p^* = K_p.$$

This identity underlies all arithmetic and SMRK-type Hamiltonians.

3.15 Spectral Consequences

The operator algebra determines:

- admissible Hamiltonians,
- spectral structure and gaps,
- stability and convergence properties,
- computational complexity,
- reasoning dynamics in QVM.

Rich noncommutative structure produces band spectra, resonances, and arithmetic signatures.

3.16 Summary

This chapter established the operator-theoretic foundation of QFM:

- linear, local operators on quansistor fields,
- forward, backward, neighborhood, and potential operators,
- adjoint identities and noncommutativity,
- the complete operator algebra $\mathcal{O}(\mathcal{Q})$,
- canonical decompositions and self-adjoint balancing.

These structures form the engine of QFM and prepare the ground for Hamiltonian dynamics and spectral analysis developed in subsequent chapters.

4 Transfer Operators and Local Dynamics

4.1 Overview

Transfer operators constitute the dynamical core of Quansistor Field Mathematics. They define how amplitude, information, and computational influence propagate through a quansistor field. Every QFM Hamiltonian, evolution rule, or distributed execution process ultimately decomposes into transfer operators of two complementary types:

- forward transfer operators, propagating amplitude along structural maps,
- backward transfer operators, aggregating amplitude from preimages.

Together, they encode graph dynamics, arithmetic multiplication and division, physical propagation, and distributed execution semantics.

4.2 Transfer Maps on the Quansistor Index Set

Let $\mathcal{Q}$ be a quansistor index set equipped with neighborhood structure.

Definition 4.1 (Transfer Map). A transfer map is a function

$$f : \mathcal{Q} \rightarrow \mathcal{Q}$$

satisfying locality constraints, typically $f(q) \in N(q)$.

Transfer maps may be:

- deterministic,
- nondeterministic (multi-valued),
- weighted or stochastic.

QFM admits all three variants.

4.3 Forward Transfer Operators

Definition 4.2 (Forward Transfer Operator). Given a transfer map $f : \mathcal{Q} \rightarrow \mathcal{Q}$, the forward transfer operator A_f acts as

$$(A_f\psi)(q) = \psi(f(q)).$$

Interpretation

Forward transfer operators:

- propagate amplitude forward along f ,
- encode deterministic update rules,
- act as pullback operators in functional analysis,
- generalize Koopman operators in dynamical systems.

Locality

If $f(q) \in N(q)$, then A_f respects QFM locality axioms and is implementable as a local operator.

Arithmetic Example

For $f_p(n) = pn$,

$$(A_p\psi)(n) = \psi(pn),$$

encoding prime multiplication dynamics.

4.4 Backward Transfer Operators

Backward transfer operators aggregate amplitude from inverse images.

Definition 4.3 (Backward Transfer Operator). Given $f : \mathcal{Q} \rightarrow \mathcal{Q}$, define

$$(B_f\psi)(q) = \sum_{r:f(r)=q} \psi(r).$$

Interpretation

Backward operators:

- collect influences from all predecessors,
- generalize inverse-image propagation,

- correspond to Perron–Frobenius operators,
- act as adjoint-like duals of forward operators.

Arithmetic Example

For $f_p(n) = pn$,

$$(B_p\psi)(n) = \mathbf{1}_{p|n} \psi(n/p).$$

4.5 Composite Transfer Operators

Complex dynamics involve sequences of transfer maps $f_1, \dots, f_k$.

Composite forward operators are given by:

$$A_{f_k} \cdots A_{f_1}.$$

Composite backward operators are:

$$B_{f_1} \cdots B_{f_k}.$$

These represent multi-step propagation rules.

Noncommutativity

In general,

$$A_f A_g \neq A_g A_f, \quad B_f B_g \neq B_g B_f.$$

This noncommutativity underlies:

- arithmetic operator algebras,
- interference phenomena,
- emergent reasoning dynamics.

4.6 Weighted Transfer Operators

Transfer operators may include local weights.

Definition 4.4 (Weighted Forward Transfer Operator). Given weight function $w : \mathcal{Q} \rightarrow \mathcal{A}$,

$$(A_{f,w}\psi)(q) = w(q)\psi(f(q)).$$

Definition 4.5 (Weighted Backward Transfer Operator).

$$(B_{f,w}\psi)(q) = \sum_{r:f(r)=q} w(r)\psi(r).$$

Weighted operators appear in:

- stochastic dynamics,
- diffusion approximations,
- attenuation and amplification models,
- arithmetic weighting schemes.

4.7 Kernel Representation

Every transfer operator admits a kernel representation:

$$T(q, r) = \begin{cases} w(q, r), & f(r) = q, \\ 0, & \text{otherwise.} \end{cases}$$

Forward operators correspond to column shifts, backward operators to row aggregation, unifying adjacency matrices and convolution kernels.

4.8 Adjoint Relations

Adjoint relations encode balance between propagation and aggregation.

Definition 4.6 (Adjoint of a Transfer Operator). The adjoint A_f^* satisfies

$$\langle A_f \psi, \phi \rangle = \langle \psi, A_f^* \phi \rangle.$$

Arithmetic Adjoint Identity

For multiplication-by- p on $\mathcal{Q} = \mathbb{N}$:

$$A_p^* = pB_p.$$

The factor p reflects measure distortion and is essential for constructing self-adjoint Hamiltonians.

4.9 Local Dynamics Generated by Transfer Operators

Discrete-time evolution follows:

$$\psi_{t+1} = T\psi_t,$$

where

$$T = \sum_i (\alpha_i A_{f_i} + \beta_i B_{g_i}).$$

Types of Dynamics

Operator Structure	Phenomenon
Forward	deterministic propagation
Backward	aggregate influence
Weighted	decay, probabilities
Balanced	symmetric arithmetic flow
Noncommutative	interference, reasoning

4.10 Simulation and Arithmetic Dynamics

In physical simulations, forward/backward pairs approximate:

- advection,
- wave propagation,
- transport phenomena.

In arithmetic systems:

$$\psi(n) \mapsto \psi(pn), \quad \psi(n/p),$$

leading to multiplicative diffusion analogous to analytic number theory.

4.11 Transfer Operators and Distributed Execution

Because transfer operators respect locality, they decompose naturally across distributed infrastructure.

Let

$$T = \sum_i T_i, \quad \text{supp}(T_i) \subseteq \mathcal{Q}_i,$$

where each $\mathcal{Q}_i$ is assigned to a computational shard.

Forward operators propagate locally, backward operators aggregate from neighbors, enabling deterministic parallel execution.

4.12 Transfer Operators in Hamiltonians

Transfer operators are the atomic components of QFM Hamiltonians:

$$H = \sum_{p \in \mathcal{P}} (\alpha_p A_p + \beta_p B_p) + V.$$

Self-Adjointness Condition

Using $A_p^* = pB_p$, self-adjointness requires

$$\alpha_p = \frac{1}{p}, \quad \beta_p = p.$$

Balanced operators

$$\frac{1}{p} A_p + p B_p$$

form the backbone of arithmetic and SMRK-type Hamiltonians.

4.13 Role in the QFM Operator Algebra

Transfer operators generate the full operator algebra:

$$\mathcal{O}(\mathcal{Q}) = \overline{\text{span}}\{A_f, B_f, V\}.$$

Every QFM computation, simulation, or reasoning step is ultimately an expression built from transfer operators.

4.14 Summary

Transfer operators are the universal computational primitives of QFM:

- forward operators encode deterministic propagation,
- backward operators encode inverse aggregation,

- weighted variants unify stochastic, physical, and arithmetic dynamics,
- adjoint relations determine spectral balance,
- composite operators generate rich noncommutative dynamics,
- locality enables distributed execution.

They play the role in QFM that matrices play in quantum mechanics and logic gates play in classical computation: the fundamental building blocks of all dynamics.

5 QFM Hamiltonians

5.1 Motivation

Hamiltonians represent the central organizing objects of Quansistor Field Mathematics. They generate time evolution, encode interaction rules, determine spectral structure, and unify computation, arithmetic, and physical simulation within a single operator framework.

In QFM, a Hamiltonian is not merely an energy operator in the quantum-mechanical sense, but a generalized generator of structured state transitions on the quansistor field.

5.2 Definition of a QFM Hamiltonian

Definition 5.1 (QFM Hamiltonian). A QFM Hamiltonian is a linear operator

$$H : \mathcal{H}_{\text{QFM}} \rightarrow \mathcal{H}_{\text{QFM}}$$

of the form

$$H = \sum_i \alpha_i A_{f_i} + \sum_j \beta_j B_{g_j} + V,$$

where:

- A_{f_i} are forward transfer operators,
- B_{g_j} are backward transfer operators,
- V is a potential (multiplication) operator,
- coefficients $\alpha_i, \beta_j \in \mathbb{R}$ or $\mathcal{A}$.

This decomposition is canonical within the QFM operator algebra.

5.3 Time Evolution

Continuous-time evolution is defined formally by

$$\psi(t) = e^{-tH} \psi(0),$$

while discrete-time evolution takes the form

$$\psi_{k+1} = U \psi_k, \quad U \approx e^{-H}.$$

Unlike standard quantum mechanics, QFM does not require H to be unitary; however, spectral properties remain central to stability and interpretability.

5.4 Self-Adjointness

Self-adjoint Hamiltonians play a privileged role.

Definition 5.2 (Self-Adjoint Hamiltonian). A Hamiltonian H is self-adjoint if

$$\langle H\psi, \phi \rangle = \langle \psi, H\phi \rangle$$

for all ψ, ϕ in the domain of H .

Self-adjointness ensures:

- real spectrum,
- stable evolution,
- well-defined spectral measures.

5.5 Balanced Transfer Operators

From Chapter 4, recall the adjoint identity

$$A_p^* = pB_p.$$

Definition 5.3 (Balanced Prime Operator). For a prime p , define

$$K_p = \frac{1}{p}A_p + pB_p.$$

Proposition 5.1. The operator K_p is self-adjoint.

Proof. Using $A_p^* = pB_p$,

$$K_p^* = \frac{1}{p}A_p^* + pB_p^* = \frac{1}{p}(pB_p) + p\left(\frac{1}{p}A_p\right) = K_p.$$

□

Balanced operators constitute the atomic self-adjoint building blocks of QFM Hamiltonians.

5.6 The SMRK Hamiltonian

We now introduce the central arithmetic Hamiltonian of QFM.

Definition 5.4 (SMRK Hamiltonian). The SMRK Hamiltonian is defined on $\mathcal{Q} = \mathbb{N}$ by

$$(H_{\text{SMRK}}\psi)(n) = \sum_{p \in \mathcal{P}} \left(\frac{1}{p}\psi(pn) + p\mathbf{1}_{p|n}\psi(n/p) \right) + (\alpha\Lambda(n) + \beta \log n)\psi(n),$$

where:

- $\mathcal{P}$ is the set of prime numbers,
- $\Lambda(n)$ is the von Mangoldt function,
- $\alpha, \beta \in \mathbb{R}$.

5.7 Structural Interpretation

The SMRK Hamiltonian consists of three interacting components:

- forward multiplicative propagation $n \mapsto pn$,
- backward divisive aggregation $n \mapsto n/p$,
- a diagonal arithmetic potential.

This structure induces multiplicative diffusion over the natural numbers, analogous to lattice diffusion in physical systems.

5.8 Self-Adjointness of the SMRK Hamiltonian

Proposition 5.2. For real α, β , the SMRK Hamiltonian is formally self-adjoint.

Proof. Each balanced prime component K_p is self-adjoint. The potential term is diagonal and real-valued. Finite linear combinations preserve self-adjointness. $\square$

Rigorous domain questions are deferred to the appendix.

5.9 Spectral Structure

The spectrum $\sigma(H_{\text{SMRK}})$ encodes:

- arithmetic resonance patterns,
- prime-induced band structures,
- potential-induced spectral shifts.

Spectral gaps correspond to dynamically forbidden regions, while eigenvalue accumulation reflects arithmetic density phenomena.

5.10 Comparison with Physical Hamiltonians

Quantum Mechanics	QFM / SMRK
Laplacian	multiplicative diffusion
Potential $V(x)$	$\Lambda(n), \log n$
Momentum	prime transitions
Energy levels	arithmetic spectra

5.11 Computational Interpretation

In a Quantum Virtual Machine implementation, the SMRK Hamiltonian acts as:

- a parallel arithmetic processor,
- a spectral feature extractor,
- a reasoning operator over multiplicative structure.

Time evolution corresponds to iterative inference over arithmetic relations.

5.12 Universality of QFM Hamiltonians

Theorem 5.1. Every QFM computation can be expressed as evolution under a suitable Hamiltonian.

Sketch. From Chapter 3, all operators decompose into transfer and potential operators. From Chapter 4, transfer operators generate local dynamics. Therefore any computation admits Hamiltonian generation. $\square$

5.13 Summary

QFM Hamiltonians:

- unify computation, arithmetic, and physics,
- are constructed from balanced transfer operators,
- admit self-adjoint realizations,
- determine spectral and dynamical behavior,
- culminate naturally in the SMRK Hamiltonian.

They represent the conceptual and mathematical core of the QFM framework.

6 Spectral Theory of QFM Hamiltonians

6.1 Why Spectrum Matters

In Quansistor Field Mathematics, spectral theory is not an auxiliary tool but the primary lens through which structure, stability, and meaning are revealed.

By Axiom V (Spectral Sovereignty), the long-term behavior of any QFM system is fully determined by the spectrum of its Hamiltonian. Eigenvalues correspond to stable modes of computation, resonance patterns, and invariant arithmetic or physical structures.

6.2 Basic Spectral Notions

Let H be a (possibly unbounded) QFM Hamiltonian on $\mathcal{H}_{\text{QFM}}$.

Definition 6.1 (Spectrum). The spectrum of H , denoted $\sigma(H)$, is the set of $\lambda \in \mathbb{C}$ for which $H - \lambda I$ fails to be invertible.

The spectrum decomposes into:

- point spectrum (eigenvalues),
- continuous spectrum,
- residual spectrum.

For self-adjoint QFM Hamiltonians, the residual spectrum is empty and $\sigma(H) \subset \mathbb{R}$.

6.3 Eigenstates as Arithmetic or Computational Modes

An eigenstate ψ_λ satisfies

$$H\psi_\lambda = \lambda\psi_\lambda.$$

In QFM, eigenstates represent:

- stable computational attractors,
- arithmetic resonance patterns,
- stationary information distributions,
- invariant reasoning states in QVM.

Unlike finite-dimensional quantum systems, QFM eigenstates may be delocalized over $\mathcal{Q}$ and exhibit hierarchical or fractal support.

6.4 Spectral Measures and Decomposition

For self-adjoint H , the spectral theorem yields a projection-valued measure $E(\lambda)$ such that

$$H = \int_{\mathbb{R}} \lambda dE(\lambda).$$

Time evolution is then expressed as

$$e^{-tH} = \int_{\mathbb{R}} e^{-t\lambda} dE(\lambda).$$

This representation underlies:

- stability analysis,
- filtering and feature extraction,
- spectral learning dynamics,
- heat-kernel analogies.

6.5 Discrete vs. Continuous Spectrum

QFM Hamiltonians typically exhibit mixed spectra.

- Discrete eigenvalues correspond to isolated arithmetic or computational structures.
- Continuous components arise from infinite branching, multiplicative diffusion, or scale invariance.

The coexistence of both is a defining feature distinguishing QFM from standard quantum lattice systems.

6.6 Spectral Properties of Balanced Prime Operators

Recall the balanced prime operator

$$K_p = \frac{1}{p}A_p + pB_p.$$

Proposition 6.1. Each K_p is self-adjoint and generates a symmetric multiplicative diffusion on $\mathbb{N}$.

The spectrum of K_p exhibits:

- accumulation at zero,
- logarithmic scaling behavior,
- nontrivial continuous components.

These properties reflect the infinite branching induced by prime multiplication and division.

6.7 Spectral Structure of the SMRK Hamiltonian

The SMRK Hamiltonian

$$H_{\text{SMRK}} = \sum_{p \in \mathcal{P}} K_p + \alpha \Lambda(n) + \beta \log n$$

combines infinitely many balanced operators with a diagonal arithmetic potential.

Heuristic Spectral Picture

Heuristically, $\sigma(H_{\text{SMRK}})$ is expected to exhibit:

- a continuous bulk spectrum generated by multiplicative diffusion,
- arithmetic resonances induced by the von Mangoldt potential,
- logarithmic spectral drift.

The competition between diffusion and potential mirrors that of Schrödinger operators on non-Euclidean spaces.

6.8 Spectral Gaps and Stability

Definition 6.2 (Spectral Gap). A spectral gap is an interval $(a, b) \subset \mathbb{R}$ such that $(a, b) \cap \sigma(H) = \emptyset$.

Spectral gaps imply:

- dynamical stability,
- resistance to perturbations,
- robustness of computational modes.

In arithmetic QFM systems, gaps correspond to forbidden multiplicative regimes.

6.9 Resonances and Arithmetic Interference

Noncommutativity among transfer operators induces spectral resonances.

These resonances manifest as:

- quasi-eigenstates,
- slow-decaying modes,

- interference patterns in arithmetic propagation.

Such structures are invisible to purely combinatorial models and require the operator-spectral framework of QFM.

6.10 Trace Formulas and Spectral Invariants

Formally, traces of functions of H encode global arithmetic data:

$$\mathrm{Tr} f(H) = \sum_{\lambda \in \sigma(H)} f(\lambda).$$

While rigorous trace-class conditions require care, these expressions motivate:

- spectral invariants,
- partition functions,
- arithmetic trace formulas.

This perspective aligns QFM with Hilbert–Pólya-type operator conjectures.

6.11 Perturbation Theory

Small perturbations $H \mapsto H + \varepsilon V$ model:

- arithmetic noise,
- computational uncertainty,
- learning or adaptation processes.

Standard results from self-adjoint perturbation theory apply locally, while global effects reflect the infinite-dimensional and arithmetic nature of QFM.

6.12 Spectral Flow and Renormalization

As parameters α, β vary, eigenvalues undergo spectral flow.

This flow defines:

- phase transitions in arithmetic dynamics,
- renormalization trajectories,
- operator fixed points.

Such flows are central to understanding universality classes within QFM.

6.13 Computational Interpretation

In QVM implementations:

- eigenvalues represent inference scores,
- eigenstates encode learned structures,
- spectral filtering performs reasoning.

Learning corresponds to shaping the Hamiltonian to induce desired spectral features.

6.14 Summary

Spectral theory in QFM:

- governs stability and long-term behavior,
- encodes arithmetic and computational structure,
- unifies diffusion, potential, and resonance effects,
- provides a bridge to number theory and physics,
- establishes spectrum as the ultimate carrier of meaning.

This chapter prepares the ground for distributed realization, arithmetic applications, and falsifiable predictions developed in subsequent sections.

7 Distributed Realization and the Quantum Virtual Machine

7.1 From Formalism to Machine

The previous chapters established QFM as a complete operator-theoretic framework. This chapter addresses the crucial question: how can such a framework be realized as a concrete, distributed computational system?

The answer is the Quantum Virtual Machine (QVM): an execution architecture in which quansistor fields, transfer operators, and Hamiltonians are mapped onto distributed computational resources while preserving locality, composability, and spectral meaning.

7.2 Principles of Distributed Realization

A QFM realization must satisfy the following constraints:

- locality of operator action,
- deterministic composability,
- parallel execution,
- reproducibility and verifiability,
- spectral fidelity.

These requirements are incompatible with monolithic architectures but align naturally with distributed systems.

7.3 Quansistors as Computational Units

In the QVM, a quansistor is instantiated as a minimal computational unit representing a local amplitude $\psi(q)$ and its neighborhood $N(q)$.

Each quansistor:

- stores local amplitude data,
- exposes interfaces for forward and backward transfer,
- applies local potential operators,

- participates in operator composition.

Quansistors are agnostic to global system size and interact only with neighbors.

7.4 Partitioning the Quansistor Field

Let $\mathcal{Q}$ be partitioned into subsets:

$$\mathcal{Q} = \bigsqcup_{i=1}^M \mathcal{Q}_i.$$

Each subset $\mathcal{Q}_i$ is assigned to a computational shard (node, canister, process). Operators decompose as:

$$H = \sum_i H_i + \sum_{i \neq j} H_{ij},$$

where H_i acts locally and H_{ij} represents boundary interactions.

7.5 Execution of Transfer Operators

Forward Operators

A forward operator A_f is executed by:

- reading $\psi(f(q))$ from the appropriate shard,
- updating $\psi(q)$ locally.

This operation is stateless and parallelizable.

Backward Operators

Backward operators require aggregation:

- each shard emits partial contributions,
- boundary contributions are exchanged,
- aggregation is deterministic and order-independent.

This mirrors reduce operations in distributed computing.

7.6 Local Hamiltonian Updates

Time evolution proceeds via local Hamiltonian fragments:

$$\psi_i(t + \Delta t) = \psi_i(t) - \Delta t H_i \psi_i(t)$$

with boundary synchronization steps for H_{ij} .

This supports:

- explicit time stepping,
- asynchronous updates,
- fault-tolerant recovery.

7.7 Spectral Consistency Across Shards

Spectral meaning must survive distribution.

This is ensured by:

- deterministic operator decomposition,
- reproducible aggregation order,
- global checkpointing of operator states.

Eigenmodes emerge as global correlations across shards rather than localized states.

7.8 QVM Execution Model

The QVM execution loop consists of:

1. local operator application,
2. boundary message exchange,
3. aggregation and normalization,
4. checkpointing and verification.

This loop is invariant under scaling and supports elastic resource allocation.

7.9 Fault Tolerance and Reproducibility

Because QFM operators are linear and decomposable:

- partial failures do not corrupt global state,
- lost shards can be recomputed,
- execution traces are replayable.

This property is essential for scientific verification and long-running arithmetic experiments.

7.10 Cryptographic and Verifiable Execution

Operator execution logs can be:

- hashed,
- time-stamped,
- independently verified.

Thus, QVM execution supports cryptographic proof of computation and scientific auditability.

7.11 Relationship to Existing Paradigms

Paradigm	QVM Interpretation
Classical VM	operator scheduler
Quantum circuit	static operator fragment
MapReduce	backward transfer aggregation
Blockchain VM	deterministic replay

QVM subsumes these paradigms within a unified operator-based execution model.

7.12 Learning and Adaptation

Learning in QVM corresponds to modifying Hamiltonian parameters to shape spectral structure. This includes:

- tuning potentials,
- enabling or disabling transfer paths,
- adjusting operator weights.

Learning is therefore a form of spectral engineering.

7.13 Scalability

Because all interactions are local, QVM scales with:

- size of $\mathcal{Q}$,
- number of shards,
- available parallelism.

No global synchronization barrier is required except for optional checkpoints.

7.14 Interpretation

The QVM is not a simulator of a different machine. It is the natural execution substrate implied by QFM itself.

In this sense, QFM is not merely a theory *of* computation, but a theory that *becomes* computation when instantiated.

7.15 Summary

This chapter established:

- how quansistors map to computational units,
- how QFM operators execute in distributed environments,
- how Hamiltonian dynamics are preserved across shards,
- how spectral meaning survives scaling,
- how QVM realizes QFM as a concrete machine.

The QVM provides the bridge from abstract operator mathematics to executable, verifiable, and scalable computation.

8 Distributed Realization and the Quantum Virtual Machine

8.1 From Formalism to Machine

The previous chapters established QFM as a complete operator-theoretic framework. This chapter addresses the crucial question: how can such a framework be realized as a concrete, distributed computational system?

The answer is the Quantum Virtual Machine (QVM): an execution architecture in which quansistor fields, transfer operators, and Hamiltonians are mapped onto distributed computational resources while preserving locality, composability, and spectral meaning.

8.2 Principles of Distributed Realization

A QFM realization must satisfy the following constraints:

- locality of operator action,
- deterministic composability,
- parallel execution,
- reproducibility and verifiability,
- spectral fidelity.

These requirements are incompatible with monolithic architectures but align naturally with distributed systems.

8.3 Quansistors as Computational Units

In the QVM, a quansistor is instantiated as a minimal computational unit representing a local amplitude $\psi(q)$ and its neighborhood $N(q)$.

Each quansistor:

- stores local amplitude data,
- exposes interfaces for forward and backward transfer,
- applies local potential operators,
- participates in operator composition.

Quansistors are agnostic to global system size and interact only with neighbors.

8.4 Partitioning the Quansistor Field

Let $\mathcal{Q}$ be partitioned into subsets:

$$\mathcal{Q} = \bigsqcup_{i=1}^M \mathcal{Q}_i.$$

Each subset $\mathcal{Q}_i$ is assigned to a computational shard (node, canister, process). Operators decompose as:

$$H = \sum_i H_i + \sum_{i \neq j} H_{ij},$$

where H_i acts locally and H_{ij} represents boundary interactions.

8.5 Execution of Transfer Operators

Forward Operators

A forward operator A_f is executed by:

- reading $\psi(f(q))$ from the appropriate shard,
- updating $\psi(q)$ locally.

This operation is stateless and parallelizable.

Backward Operators

Backward operators require aggregation:

- each shard emits partial contributions,
- boundary contributions are exchanged,
- aggregation is deterministic and order-independent.

This mirrors reduce operations in distributed computing.

8.6 Local Hamiltonian Updates

Time evolution proceeds via local Hamiltonian fragments:

$$\psi_i(t + \Delta t) = \psi_i(t) - \Delta t H_i \psi_i(t)$$

with boundary synchronization steps for H_{ij} .

This supports:

- explicit time stepping,
- asynchronous updates,
- fault-tolerant recovery.

8.7 Spectral Consistency Across Shards

Spectral meaning must survive distribution.

This is ensured by:

- deterministic operator decomposition,
- reproducible aggregation order,
- global checkpointing of operator states.

Eigenmodes emerge as global correlations across shards rather than localized states.

8.8 QVM Execution Model

The QVM execution loop consists of:

1. local operator application,

2. boundary message exchange,
3. aggregation and normalization,
4. checkpointing and verification.

This loop is invariant under scaling and supports elastic resource allocation.

8.9 Fault Tolerance and Reproducibility

Because QFM operators are linear and decomposable:

- partial failures do not corrupt global state,
- lost shards can be recomputed,
- execution traces are replayable.

This property is essential for scientific verification and long-running arithmetic experiments.

8.10 Cryptographic and Verifiable Execution

Operator execution logs can be:

- hashed,
- time-stamped,
- independently verified.

Thus, QVM execution supports cryptographic proof of computation and scientific auditability.

8.11 Relationship to Existing Paradigms

Paradigm	QVM Interpretation
Classical VM	operator scheduler
Quantum circuit	static operator fragment
MapReduce	backward transfer aggregation
Blockchain VM	deterministic replay

QVM subsumes these paradigms within a unified operator-based execution model.

8.12 Learning and Adaptation

Learning in QVM corresponds to modifying Hamiltonian parameters to shape spectral structure.

This includes:

- tuning potentials,
- enabling or disabling transfer paths,
- adjusting operator weights.

Learning is therefore a form of spectral engineering.

8.13 Scalability

Because all interactions are local, QVM scales with:

- size of $\mathcal{Q}$,
- number of shards,
- available parallelism.

No global synchronization barrier is required except for optional checkpoints.

8.14 Interpretation

The QVM is not a simulator of a different machine. It is the natural execution substrate implied by QFM itself.

In this sense, QFM is not merely a theory *of* computation, but a theory that *becomes* computation when instantiated.

8.15 Summary

This chapter established:

- how quansistors map to computational units,
- how QFM operators execute in distributed environments,
- how Hamiltonian dynamics are preserved across shards,
- how spectral meaning survives scaling,
- how QVM realizes QFM as a concrete machine.

The QVM provides the bridge from abstract operator mathematics to executable, verifiable, and scalable computation.

9 Physical Analogies and Simulation

9.1 Motivation

Although Quansistor Field Mathematics is formulated abstractly, its operator structure closely parallels models found in mathematical physics. This chapter clarifies these analogies, not as metaphors, but as precise correspondences between QFM operators and physical simulation frameworks.

Such analogies serve two purposes:

- to provide intuition for QFM dynamics,
- to enable direct numerical simulation techniques.

9.2 Quansistor Fields as Discrete Physical Fields

A quansistor field

$$\psi : \mathcal{Q} \rightarrow \mathcal{A}$$

is analogous to a discrete physical field defined on a non-Euclidean lattice.

Key correspondences:

Physics	QFM
Lattice site	quansistor index q
Field value	amplitude $\psi(q)$
Adjacency	neighborhood $N(q)$
Energy density	potential $V(q)$

The essential difference is that $\mathcal{Q}$ need not be spatial—it may be arithmetic, logical, or abstract.

9.3 Diffusion and Transport

Balanced transfer operators induce diffusion-like dynamics.

For a Hamiltonian H , the evolution equation

$$\partial_t \psi = -H\psi$$

resembles a heat equation on a generalized lattice.

- forward operators correspond to transport,
- backward operators correspond to aggregation,
- balance ensures reversibility-like behavior.

This analogy allows QFM systems to be simulated using established numerical methods for diffusion processes.

9.4 Wave Phenomena

When Hamiltonians contain competing noncommuting transfer operators, oscillatory behavior emerges.

Such dynamics correspond to:

- wave propagation,
- interference,
- standing modes (eigenstates).

Eigenstates of QFM Hamiltonians thus play the role of stationary waves on abstract spaces.

9.5 Potential Landscapes

Diagonal potential operators act as energy landscapes.

In arithmetic systems:

- $\Lambda(n)$ creates localized arithmetic barriers,
- $\log n$ induces scale-dependent drift.

In physical analogies, these correspond to spatially varying potentials that trap or repel field amplitude.

9.6 Comparison with Schrödinger Operators

The QFM Hamiltonian

$$H = \sum_i K_i + V$$

is structurally analogous to a Schrödinger operator

$$-\Delta + V(x),$$

where:

- K_i act as generalized Laplacians,
- V is an external potential,
- $\mathcal{Q}$ replaces Euclidean space.

This analogy enables the transfer of intuition and techniques from quantum mechanics to QFM.

9.7 Simulation Techniques

QFM systems may be simulated using:

- explicit time stepping,
- implicit solvers for stiff operators,
- spectral decomposition when available,
- Monte Carlo sampling of transfer paths.

Locality ensures simulations scale efficiently in distributed environments.

9.8 Boundary Conditions

Boundary conditions arise naturally when $\mathcal{Q}$ is truncated or restricted.

Typical conditions include:

- absorbing boundaries,
- reflecting boundaries,
- periodic identifications.

Boundary choice influences spectral properties and must be considered carefully in simulations.

9.9 Emergence of Macroscopic Behavior

Large-scale QFM systems exhibit emergent phenomena:

- effective diffusion constants,
- collective oscillations,
- phase-like transitions under parameter changes.

These behaviors mirror macroscopic physics arising from microscopic laws.

9.10 Physical Interpretation of Arithmetic Dynamics

Arithmetic diffusion on $\mathbb{N}$ corresponds to transport on a highly irregular space.

This explains why:

- arithmetic statistics resemble random processes,
- deterministic operators produce stochastic-looking outcomes,
- physical analogies are mathematically justified.

9.11 Visualization and Experimentation

QFM simulations may be visualized as:

- heat maps over index sets,
- spectral density plots,
- mode decomposition animations.

Such visualizations are essential for hypothesis generation and falsification.

9.12 Limits of the Physical Analogy

While powerful, physical analogies must not be overstated.

QFM:

- does not assume physical spacetime,
- does not rely on quantum measurement postulates,
- remains an abstract operator theory.

Analogies are tools, not claims of physical identity.

9.13 Summary

This chapter demonstrated that:

- QFM Hamiltonians correspond naturally to physical operators,
- diffusion, waves, and potentials emerge generically,
- established simulation methods apply directly,
- arithmetic dynamics admit precise physical interpretation,
- QFM provides a unified simulation framework across domains.

QFM thus bridges abstract operator mathematics and concrete, computable dynamics.

10 Falsifiability, Experiments, and Predictions

10.1 Scientific Status of QFM

Quansistor Field Mathematics is proposed as a mathematical and computational framework, not as a metaphysical theory. As such, it must satisfy the criteria of scientific falsifiability.

This chapter identifies concrete predictions, numerical experiments, and failure modes by which QFM—and in particular arithmetic Hamiltonians such as H_{SMRK} —can be tested, refined, or rejected.

10.2 What Would Falsify QFM

QFM would be falsified if one or more of the following were demonstrated:

- QFM Hamiltonians cannot be defined as self-adjoint operators on any reasonable domain.
- Spectral properties predicted by balanced transfer operators are absent in numerical realizations.
- Operator-based dynamics fail to reproduce known arithmetic statistics even qualitatively.
- Distributed execution destroys spectral invariants in an irreparable way.

These criteria distinguish QFM from purely formal or unfalsifiable frameworks.

10.3 Numerical Spectral Experiments

The primary experimental probe of QFM is numerical spectral analysis.

Concrete experiments include:

- truncation of $\mathcal{Q} = \mathbb{N}$ to $[1, N]$,
- construction of finite-dimensional approximations of H_{SMRK} ,
- numerical diagonalization and spectral density estimation.

Observed spectra can be compared across increasing N to detect convergence or systematic deviation.

10.4 Predicted Spectral Features

QFM predicts the following qualitative spectral features:

- presence of a continuous spectral bulk,
- clustering near arithmetic resonances,
- sensitivity to the von Mangoldt potential,
- logarithmic scaling effects.

Absence of these features would challenge the framework.

10.5 Dynamical Experiments

Time evolution experiments involve initializing a localized state ψ_0 and observing

$$\psi(t) = e^{-tH}\psi_0.$$

Predictions include:

- multiplicative diffusion over $\mathbb{N}$,
- partial localization near prime powers,
- slow decay modes corresponding to spectral resonances.

10.6 Comparison with Random Models

To test whether observed phenomena are nontrivial, QFM simulations should be compared against:

- random graph Hamiltonians,
- shuffled prime transfer models,
- purely stochastic diffusion processes.

QFM predicts statistically distinguishable behavior from such controls.

10.7 Robustness under Perturbation

Perturbation experiments probe stability:

$$H \mapsto H + \varepsilon V.$$

Predictions:

- small perturbations cause continuous spectral deformation,
- large perturbations destroy arithmetic structure,
- certain spectral invariants persist under noise.

Failure of robustness would indicate overfitting to idealized operators.

10.8 Distributed Execution Tests

QVM implementations allow falsification via distributed execution:

- compare spectra computed on different partitions,
- verify reproducibility under reordering,
- test fault recovery consistency.

Loss of invariants under distribution would contradict QFM's locality axioms.

10.9 Predictions Beyond Arithmetic

Although motivated by arithmetic, QFM predicts similar phenomena in:

- graph dynamics,
- symbolic computation,
- learning systems modeled by operator flows.

Cross-domain consistency strengthens the framework; inconsistency weakens it.

10.10 Negative Results and Scope

Negative results are admissible and expected.

QFM does not claim:

- a proof of the Riemann Hypothesis,
- universality over all mathematical systems,
- physical realization as a quantum theory.

Its claims are limited to providing a coherent operator framework whose validity depends on experimental outcomes.

10.11 Reproducibility and Open Verification

All experiments described are reproducible using:

- published operator definitions,
- open-source numerical code,
- deterministic execution traces.

This openness is essential to scientific credibility.

10.12 Long-Term Research Program

If supported by evidence, QFM suggests a research program including:

- refined operator domains and rigor,
- deeper spectral analysis,
- large-scale distributed arithmetic experiments,
- connections to established number-theoretic conjectures.

If falsified, it still provides a useful negative result clarifying the limits of operator-based arithmetic modeling.

10.13 Summary

This chapter established that:

- QFM is explicitly falsifiable,

- concrete numerical and distributed experiments are available,
- clear predictions and failure modes exist,
- negative outcomes are scientifically meaningful,
- the framework stands or falls by empirical and computational evidence.

QFM is thus presented not as a belief system, but as a testable, evolving scientific theory.

APPENDIX A

A.1 Quansistor Field Dynamics: An Intuitive Layer

A.1.1 Purpose of This Appendix

This appendix provides an intuitive and dynamical interpretation of the quansistor field formalism introduced in the main text. It does not introduce new axioms or modify the mathematical framework of QFM. Instead, it offers a kinematic and phenomenological perspective on how quansistor fields evolve under transfer operators and Hamiltonians.

Readers interested primarily in formal structure may skip this appendix without loss of logical continuity.

A.1.2 Quansistor Fields as Dynamical Media

A quansistor field

$$\psi : \mathcal{Q} \rightarrow \mathcal{A}$$

may be viewed as a dynamical medium rather than a static assignment of values.

Each quansistor index $q \in \mathcal{Q}$ carries:

- a local amplitude $\psi(q)$,
- a set of admissible interactions defined by $N(q)$,
- participation in global field dynamics.

Field evolution corresponds to structured redistribution of amplitude across $\mathcal{Q}$ under operator action.

A.1.3 Forward Flow and Propagation

Forward transfer operators induce directed flow.

For a transfer map $f : \mathcal{Q} \rightarrow \mathcal{Q}$,

$$(A_f \psi)(q) = \psi(f(q)),$$

represents transport of amplitude from $f(q)$ to q .

Intuitively:

- amplitude propagates along allowed structural paths,
- information moves without aggregation,
- dynamics resemble advection or signal transmission.

In arithmetic systems, forward flow corresponds to multiplicative expansion $n \mapsto pn$.

A.1.4 Backward Flow and Aggregation

Backward transfer operators induce convergent flow.

$$(B_f\psi)(q) = \sum_{r:f(r)=q} \psi(r)$$

This represents:

- aggregation of influences,
- consolidation of information,
- convergence toward structural hubs.

In arithmetic contexts, backward flow corresponds to divisive contraction $n \mapsto n/p$.

A.1.5 Balanced Dynamics

When forward and backward flows are combined symmetrically, balanced dynamics emerge.

Balanced operators such as

$$K_p = \frac{1}{p}A_p + pB_p$$

induce neither pure expansion nor pure contraction, but a form of multiplicative diffusion.

This balance:

- stabilizes global dynamics,
- enables reversible-like behavior,
- supports meaningful spectral structure.

A.1.6 Attractors and Invariant Structures

Under repeated application of QFM operators, certain field configurations may emerge as attractors.

These include:

- stationary eigenstates,
- slowly decaying resonant modes,
- scale-invariant distributions.

Attractors represent stable patterns of computation or arithmetic structure.

A.1.7 Locality and Causality

QFM dynamics respect strict locality:

$$\text{supp}(T\psi)(q) \subseteq N(q) \cup \{q\}.$$

This implies a form of causal structure:

- influence propagates stepwise,
- no instantaneous global effects,
- scalability to large systems.

Local causality enables distributed execution and physical simulation analogies.

A.1.8 Energy Landscapes and Flow Bias

Diagonal potential operators bias dynamics:

$$(V\psi)(q) = V(q)\psi(q).$$

Potentials act as:

- sinks or sources,
- barriers or attractors,
- stabilizers of structure.

In arithmetic systems, $\Lambda(n)$ and $\log n$ shape flow through multiplicative space.

A.1.9 Emergent Time

Although time is not a primitive object in QFM, iterative operator application induces an emergent temporal ordering.

Discrete steps correspond to computational time, while continuous evolution emerges from limits of operator exponentials.

This aligns QFM with discrete-time dynamical systems and continuous operator flows.

A.1.10 Comparison with Physical Field Dynamics

QFM field dynamics parallel physical systems:

Physics	QFM
Particle flow	amplitude propagation
Diffusion	balanced transfer dynamics
Potential energy	diagonal operators
Standing waves	eigenstates

These parallels justify simulation methods but do not imply physical identity.

A.1.11 Interpretational Caution

This appendix provides intuition, not proof.

All interpretations must remain subordinate to:

- formal operator definitions,
- spectral analysis,
- falsifiable experiments.

Intuition serves as a guide for exploration, not as evidence.

A.1.12 Summary

This appendix clarified:

- quansistor fields as dynamical media,

- forward and backward flow interpretations,
- balanced dynamics and stability,
- emergence of attractors and temporal structure,
- connections to physical field intuition.

These insights support understanding but do not alter the formal structure of Quansistor Field Mathematics.

APPENDIX B

B.1 Extended Operator Calculus

B.1.1 Role of Operator Calculus in QFM

The main text introduced the operator algebra of QFM as a formal framework for defining dynamics on quansistor fields. This appendix extends that discussion by exploring operator calculus at a more structural and interpretative level.

The purpose is not to introduce new axioms, but to clarify how operators may be combined, transformed, and interpreted as higher-level computational or mathematical objects.

B.1.2 Operators as Programs

In QFM, operators can be viewed as programs acting on a distributed state space.

- forward transfer operators act as data fetch instructions,
- backward operators act as aggregation or reduction instructions,
- potential operators act as local modifiers or weights.

Operator composition corresponds to program composition, while linear combination corresponds to parallel execution paths.

B.1.3 Normal Forms and Operator Decomposition

Every operator $T \in \mathcal{O}(\mathcal{Q})$ admits a decomposition of the form

$$T = \sum_i \alpha_i A_{f_i} + \sum_j \beta_j B_{g_j} + V,$$

as discussed in Chapter 3.

From a calculus perspective, this decomposition plays the role of a normal form, analogous to:

- canonical circuits in logic,
- normal forms in lambda calculus,
- basis expansions in functional analysis.

B.1.4 Commutation Relations

Noncommutativity is a defining feature of QFM operator calculus.

Given two transfer maps f and g , in general

$$[A_f, A_g] \neq 0, \quad [A_f, B_g] \neq 0.$$

These commutation relations encode ordering sensitivity and give rise to interference effects, even in purely arithmetic settings.

B.1.5 Operator Scaling and Renormalization

Operators may be rescaled or grouped into effective operators.

For example, a family of prime operators $\{K_p\}_{p \leq P}$ may be approximated by an effective diffusion operator acting on a truncated region of $\mathcal{Q}$.

Such constructions resemble renormalization procedures in physics, where microscopic degrees of freedom are absorbed into macroscopic effective laws.

B.1.6 Functional Calculus

Given a self-adjoint Hamiltonian H , one may formally define

$$f(H) = \int_{\sigma(H)} f(\lambda) dE(\lambda),$$

where $E(\lambda)$ is the spectral measure.

In QFM, functional calculus enables:

- spectral filtering,
- time evolution via e^{-tH} ,
- construction of resolvents and Green-type operators.

This elevates operators from transition rules to manipulable analytical objects.

B.1.7 Iterated and Nested Operators

Nested operator expressions such as

$$T = A_{f_1} B_{f_2} A_{f_3}$$

represent multi-stage processes.

Iterated application defines discrete flows, while limits of iteration yield continuous dynamics. Such constructions unify algorithmic iteration with operator semigroup theory.

B.1.8 Operator Flows

Parameter-dependent families of operators

$$H(\theta) = H_0 + \theta V$$

define operator flows.

As θ varies, spectral properties evolve continuously, producing bifurcations, resonances, or phase-like transitions. This concept underlies both learning dynamics in QVM and perturbative analysis in arithmetic Hamiltonians.

B.1.9 Symmetries and Invariants

Symmetries of $\mathcal{Q}$ induce symmetries of the operator algebra.

If U is a unitary (or invertible) symmetry operator commuting with H , then spectral degeneracies and invariant subspaces arise.

Such symmetries correspond to arithmetic constraints, graph automorphisms, or conservation laws.

B.1.10 Operator Metrics and Distances

One may introduce metrics on the space of operators, for example via operator norms or spectral distances.

These metrics quantify:

- similarity between Hamiltonians,
- stability under perturbation,
- learning progress in adaptive QVM systems.

B.1.11 Limits of the Calculus

The extended operator calculus remains formal in several respects:

- convergence of infinite sums,
- domain subtleties for unbounded operators,
- trace-class conditions.

These issues are addressed rigorously only in restricted settings and are not assumed globally.

B.1.12 Summary

This appendix expanded the operator calculus of QFM by:

- interpreting operators as programs,
- clarifying normal forms and decomposition,
- emphasizing noncommutativity and scaling,
- introducing functional and flow-based viewpoints,
- outlining limits and open technical issues.

Together with Appendix A, this calculus-oriented perspective deepens intuition while preserving the formal integrity of the main theory.

APPENDIX C

C.1 Hamiltonians and Spectral Intuition

C.1.1 Purpose and Scope

This appendix provides an interpretative layer for QFM Hamiltonians and their spectra. All formal definitions and results appear in Chapters 5 and 6; here we focus on conceptual understanding, intuition, and meaning.

Nothing in this appendix is required for the logical consistency of the main text.

C.1.2 Hamiltonians as Generators of Meaning

In QFM, Hamiltonians generate more than temporal evolution. They generate *structure*.

Each Hamiltonian encodes:

- which transitions are allowed,
- how strongly they are weighted,
- how local interactions combine into global behavior.

From this perspective, a Hamiltonian functions as a semantic rule governing the field.

C.1.3 Eigenstates as Stable Concepts

Eigenstates of a Hamiltonian

$$H\psi = \lambda\psi$$

represent configurations that remain invariant (up to scale) under the action of H .

Intuitively, eigenstates correspond to:

- stable computational patterns,
- persistent arithmetic structures,
- invariant modes of reasoning.

In QVM terms, they act as “concepts” or “attractors” of computation.

C.1.4 Eigenvalues as Relevance Measures

Eigenvalues quantify the response of a state to Hamiltonian dynamics.

Larger eigenvalues correspond to:

- stronger participation in dynamics,
- higher energetic or computational relevance,
- faster decay or amplification under evolution.

Thus, the spectrum of a Hamiltonian provides a ranking of structural importance.

C.1.5 Spectral Decomposition as Field Analysis

The spectral decomposition

$$\psi = \int dE(\lambda) \psi$$

can be viewed as decomposing a field into elementary semantic components.

Each spectral component isolates a mode with distinct dynamical behavior. This mirrors Fourier analysis, but on abstract, non-Euclidean index spaces.

C.1.6 Spectral Bands and Structural Classes

Continuous spectral bands correspond to families of related structures rather than isolated objects.

In arithmetic systems:

- bands may encode classes of integers with similar multiplicative behavior,
- gaps indicate forbidden or unstable configurations.

Such structures have no direct analogue in classical combinatorial number theory.

C.1.7 Resonances and Near-Eigenstates

Not all meaningful states are exact eigenstates.

Resonant or quasi-eigenstates:

- persist for long but finite times,
- encode approximate invariants,
- play a central role in arithmetic diffusion.

These states often dominate observable dynamics.

C.1.8 SMRK Hamiltonian: An Intuitive View

The SMRK Hamiltonian combines:

- multiplicative expansion,
- divisive aggregation,
- arithmetic potential shaping.

Intuitively, it defines a balance between growth and collapse on $\mathbb{N}$, producing structured diffusion rather than chaos.

Eigenstates reflect global arithmetic patterns rather than individual numbers.

C.1.9 Spectral Flow and Parameter Tuning

Varying parameters such as α and β induces spectral flow.

This flow:

- reorganizes eigenmodes,

- reveals phase-like transitions,
- provides a mechanism for learning and adaptation.

In QVM implementations, tuning parameters corresponds to spectral engineering.

C.1.10 Spectrum as Fingerprint

The spectrum of a Hamiltonian acts as a fingerprint of the underlying system.

Two Hamiltonians with similar spectra produce qualitatively similar dynamics, even if their microscopic definitions differ.

This supports the use of spectral metrics for comparison and classification.

C.1.11 Relation to Physical and Cognitive Models

In physics, spectra encode energy levels.

In QFM, spectra encode:

- dynamical stability,
- computational relevance,
- semantic persistence.

This suggests a unifying role for spectral analysis across physics, computation, and cognition.

C.1.12 Interpretational Limits

The interpretations presented here are heuristic.

No claim is made that:

- Hamiltonians possess intrinsic meaning,
- spectra imply consciousness,
- arithmetic systems are physical substrates.

Meaning arises from interpretation layered atop formal structure.

C.1.13 Summary

This appendix emphasized that:

- Hamiltonians encode structural rules,
- eigenstates represent stable modes,
- eigenvalues quantify relevance,
- spectral decomposition reveals hidden organization,
- intuition complements but does not replace formal analysis.

Together with Chapters 5 and 6, this interpretative layer completes the picture of Hamiltonians as the central organizing objects of QFM.

APPENDIX D

D.1 Distributed Architecture and QVM Execution

D.1.1 Purpose of This Appendix

This appendix provides technical and architectural details of how Quansistor Field Mathematics (QFM) can be instantiated as a distributed execution system, referred to as the Quantum Virtual Machine (QVM).

The material here supplements Chapter 7 by specifying execution models, data layouts, determinism guarantees, and implementation strategies. It is intended for system designers and implementers.

D.1.2 Architectural Overview

A QVM realization consists of:

- a global quansistor index set $\mathcal{Q}$,
- a partitioning of $\mathcal{Q}$ into shards,
- local execution engines applying operator fragments,
- a deterministic communication and aggregation protocol.

Each shard operates independently except for boundary interactions explicitly defined by transfer operators.

D.1.3 Sharding Model

Let

$$\mathcal{Q} = \bigsqcup_{i=1}^M \mathcal{Q}_i$$

be a partition of the index set.

Each shard $\mathcal{Q}_i$:

- stores amplitudes $\psi(q)$ for $q \in \mathcal{Q}_i$,
- executes local operator components H_i ,
- exchanges boundary data with neighboring shards.

Shard boundaries are determined by the neighborhood relation $N(q)$.

D.1.4 Local Execution Engine

Each shard runs a local execution loop:

1. read current local amplitudes,
2. apply forward transfer operators,

3. emit backward contributions,
4. apply potential operators,
5. update local state.

This loop is deterministic and side-effect free.

D.1.5 Boundary Communication Protocol

Boundary interactions arise when $f(q) \in \mathcal{Q}_j$ for $q \in \mathcal{Q}_i$, $i \neq j$.

The protocol:

- collects outbound messages,
- exchanges them between shards,
- aggregates inbound contributions deterministically.

Ordering of messages does not affect results due to linearity.

D.1.6 Backward Aggregation Semantics

Backward operators require summation over preimages:

$$(B_f\psi)(q) = \sum_{r:f(r)=q} \psi(r).$$

In distributed settings:

- each shard computes partial sums,
- partial results are merged,
- final values are written atomically.

This mirrors reduce operations in distributed computing frameworks.

D.1.7 Time Stepping and Scheduling

QVM supports multiple scheduling strategies:

- synchronous global steps,
- asynchronous local steps with barriers,
- event-driven updates for sparse dynamics.

All strategies preserve determinism when executed with fixed scheduling rules.

D.1.8 Determinism and Replay

Determinism is ensured by:

- fixed operator ordering,
- deterministic aggregation rules,

- explicit versioning of Hamiltonians.

Execution traces may be replayed exactly, enabling verification and debugging.

D.1.9 Fault Tolerance

Because operator execution is linear and local:

- shard failure affects only local state,
- failed shards can be recomputed from checkpoints,
- no global rollback is required.

This property enables long-running arithmetic and spectral experiments.

D.1.10 Checkpointing and State Persistence

Periodic checkpoints record:

- shard-local amplitudes,
- operator parameters,
- execution metadata.

Checkpoints enable:

- recovery from failure,
- migration across hardware,
- reproducible experimentation.

D.1.11 Cryptographic Verification

QVM execution can be made verifiable by:

- hashing operator definitions,
- hashing checkpoint states,
- time-stamping execution milestones.

This enables independent verification of results without rerunning the entire computation.

D.1.12 Mapping to Existing Platforms

The QVM model maps naturally to:

- containerized microservices,
- actor-based systems,
- deterministic blockchain-based compute platforms.

Each shard corresponds to a logical actor executing operator fragments.

D.1.13 Pseudocode Sketch

```
for each timestep:
  for each shard in parallel:
    apply forward operators
    emit backward messages
  synchronize boundary messages
  for each shard in parallel:
    aggregate backward contributions
    apply potential operators
  update local amplitudes
```

This pseudocode abstracts away transport and storage layers.

D.1.14 Scalability Considerations

Scalability is governed by:

- average neighborhood size,
- shard boundary density,
- communication latency.

Sparse neighborhoods yield near-linear scaling.

D.1.15 Security and Isolation

Shards operate in isolation:

- no shared mutable state,
- explicit communication channels,
- verifiable execution boundaries.

This reduces attack surface and improves robustness.

D.1.16 Summary

This appendix specified:

- the distributed sharding model of QVM,
- local execution and boundary protocols,
- deterministic aggregation and replay,
- fault tolerance and checkpointing,
- verifiable and scalable execution.

These architectural principles make QFM realizable as a concrete, distributed computational system.

APPENDIX E

E.1 Applications, Open Problems, and Ethics

E.1.1 Purpose of This Appendix

This appendix outlines potential applications of Quansistor Field Mathematics, identifies open mathematical and computational problems, and articulates an explicit ethical framework for the responsible development and use of QFM-based systems.

None of the material in this appendix is required for the internal consistency of the theory. Its role is contextual, forward-looking, and normative.

E.1.2 Applications in Arithmetic and Number Theory

QFM provides a new operator-based lens for studying arithmetic structure.

Potential applications include:

- numerical exploration of arithmetic Hamiltonians,
- spectral analysis of multiplicative dynamics,
- operator-based reformulations of classical arithmetic problems,
- experimental study of arithmetic diffusion and resonance.

While QFM does not claim to resolve open conjectures, it enables new classes of computational experiments that complement traditional analytic methods.

E.1.3 Applications in Physics-Inspired Simulation

Beyond arithmetic, QFM Hamiltonians define abstract dynamical systems that can be used to simulate:

- transport on irregular or non-Euclidean spaces,
- diffusion with nonlocal branching,
- wave-like phenomena on abstract graphs,
- systems with scale-dependent interactions.

Such simulations are relevant to statistical physics, network theory, and complex systems.

E.1.4 Applications in Computation and Machine Reasoning

Within a QVM realization, QFM enables:

- operator-based computation beyond classical state machines,
- spectral feature extraction and ranking,
- distributed reasoning via operator flows,
- adaptive systems through Hamiltonian parameter tuning.

These applications are exploratory and should be evaluated empirically.

E.1.5 Security and Cryptographic Analysis

QFM-based operator systems may be applied to:

- anomaly detection in transaction graphs,
- audit and verification of distributed computations,
- spectral signatures for system integrity.

No cryptographic security claims are made without formal proof. All such uses must be evaluated against established standards.

E.1.6 Open Mathematical Problems

Several foundational questions remain open:

- rigorous domain characterization for infinite QFM Hamiltonians,
- conditions for essential self-adjointness,
- classification of spectra for arithmetic Hamiltonians,
- convergence of finite-dimensional truncations,
- existence and structure of spectral gaps.

These problems define a concrete research agenda for future work.

E.1.7 Open Computational Problems

On the computational side, open questions include:

- efficient large-scale spectral computation,
- optimal sharding strategies for QVM execution,
- error propagation and numerical stability,
- benchmarking against classical and stochastic models.

Progress here will determine the practical viability of QFM systems.

E.1.8 Interpretational Limits

QFM is a formal framework. It does not assert that:

- arithmetic systems possess physical agency,
- spectra imply cognition or consciousness,
- operator dynamics replace existing mathematical theories.

Interpretations must remain grounded in formal definitions and testable outcomes.

E.1.9 Ethical Principles

The development and use of QFM and QVM systems should adhere to the following principles:

- transparency of algorithms and operator definitions,
- reproducibility of computational results,
- openness to falsification and critique,
- avoidance of misleading claims or overinterpretation.

These principles are essential for maintaining scientific integrity.

E.1.10 Responsible Use

Potential applications involving security, surveillance, or decision-making must be approached with caution.

QFM-based systems should not be deployed in contexts where:

- outcomes cannot be audited,
- errors carry disproportionate harm,
- human oversight is removed.

Responsibility remains with system designers and operators.

E.1.11 Long-Term Perspective

If QFM proves fruitful, it may contribute to:

- a deeper unification of operator theory and computation,
- new experimental approaches in number theory,
- robust, verifiable distributed computation models.

If it fails, it will still provide a valuable negative result clarifying the limits of operator-based modeling.

E.1.12 Summary

This appendix summarized:

- potential applications across arithmetic, physics, and computation,
- clearly defined open problems,
- explicit interpretational boundaries,
- an ethical framework for responsible development.

Quansistor Field Mathematics is presented as an open, testable framework whose value depends on evidence, rigor, and responsible use.

APPENDIX F

F.1 Meaning, Intelligence, and Responsibility

F.1.1 Purpose of This Appendix

This appendix addresses questions of meaning, intelligence, and responsibility that arise naturally from the development of operator-based computational frameworks such as QFM and QVM.

The discussion is explicitly interpretative and normative. It introduces no new formal claims and does not extend the mathematical scope of the theory.

F.1.2 Meaning as an Emergent Property

Within QFM, operators act on structured state spaces to produce stable patterns of behavior.

Meaning is not a primitive object of the theory. Instead, it may be understood as an emergent property arising when:

- operator dynamics produce persistent structures,
- such structures are interpretable by external observers,
- interpretations remain consistent under perturbation.

Meaning is therefore relational, not intrinsic.

F.1.3 Spectral Stability and Semantic Persistence

Stable spectral features correspond to persistent modes of behavior.

From an interpretative standpoint:

- eigenstates correspond to stable representations,
- spectral gaps protect interpretability,
- resonances enable flexible but coherent dynamics.

This provides a non-anthropomorphic account of how meaning-like structures can arise from purely formal systems.

F.1.4 Intelligence as Operator Organization

QFM does not define intelligence. However, it provides a language in which intelligence-like behavior can be discussed.

In this context, intelligence may be associated with:

- the ability to organize operators into effective Hamiltonians,
- adaptation via spectral tuning,
- robustness of behavior under environmental change.

Such usage remains descriptive and does not imply consciousness or agency.

F.1.5 Distinction from Strong AI Claims

This work does not claim that:

- QFM systems are conscious,
- operator dynamics generate subjective experience,
- intelligence is reducible to spectral structure alone.

Any stronger interpretation would require independent empirical evidence and philosophical justification beyond the scope of this document.

F.1.6 Human Interpretation and Responsibility

All meaning attributed to QFM systems originates from human interpretation.

Responsibility therefore lies with:

- designers of operator systems,
- interpreters of computational output,
- decision-makers deploying such systems.

No formal system absolves its creators of ethical accountability.

F.1.7 Transparency and Interpretability

Operator-based systems offer advantages in transparency:

- explicit definitions of dynamics,
- analyzable spectral structure,
- reproducible execution traces.

These properties should be preserved and prioritized in any applied context.

F.1.8 Limits of Autonomy

QFM-based systems should not be treated as autonomous moral agents.

They:

- lack intrinsic goals,
- do not possess values,
- operate entirely within predefined operator constraints.

Claims of autonomy must be treated with caution.

F.1.9 Ethical Constraints on Deployment

Deployment of QFM or QVM systems should adhere to the following constraints:

- human oversight must be maintained,

- decisions affecting individuals must be auditable,
- misuse or misinterpretation must be actively prevented.

These constraints apply regardless of system scale or sophistication.

F.1.10 Long-Term Perspective

As operator-based systems grow in complexity, their societal impact may increase.

A responsible long-term approach requires:

- continuous evaluation of assumptions,
- openness to critique and revision,
- separation of formal results from speculative interpretation.

Scientific progress must be accompanied by ethical maturity.

F.1.11 Summary

This appendix emphasized that:

- meaning in QFM is emergent and relational,
- intelligence-like behavior can be described without strong metaphysical claims,
- responsibility remains with human designers and users,
- transparency and interpretability are essential,
- ethical constraints must guide deployment.

QFM is thus framed not as a doctrine, but as a formal tool whose significance depends on evidence, interpretation, and responsible use.

Chapter 3

Spectral Foundations in Operator-First Quantum Theory

From State Space to Spectral Topology: An Operator-First Foundation

Enter Yourname

Abstract

This book develops an operator-first, spectral foundation for quantum theory, emphasizing that the essential physical content of quantum systems is not carried by state vectors or time-domain trajectories, but by the spectral structure of operators acting on an appropriate state space.

The exposition begins by reinterpreting the notion of state space itself. Rather than treating states as primary objects, states are understood as spectral configurations defined relative to families of operators and their projection structures. This shift naturally leads to a framework in which observables, measurement, and physical reality are encoded in spectral properties rather than in particular linear representations.

Spectral decomposition and functional calculus are then presented not merely as technical tools, but as structural principles. Operators are shown to act on systems primarily by organizing and transforming their spectra, with dynamics emerging as spectral action rather than as the solution of differential equations in time. This viewpoint unifies measurement, evolution, and stability within a single spectral language.

Special attention is devoted to unbounded operators, whose domains, closures, and self-adjoint extensions are treated as integral components of physical theory rather than as mathematical subtleties. Self-adjointness is interpreted as a structural condition that guarantees spectral legitimacy, while different self-adjoint realizations correspond to distinct physical models.

The resolvent is introduced as a central analytic object and interpreted as a spectral microscope: a probe that reveals fine-grained spectral topology through its singularities, analytic behavior, and norm growth. Spectral projections, stability, and sensitivity to perturbations are derived from the resolvent's structure, establishing it as the primary carrier of spectral information.

Perturbation theory is reformulated as spectral deformation. Rather than focusing on eigenvalue shifts or state corrections, perturbations are

understood as geometric and analytic deformations of spectral topology. This perspective clarifies the distinction between stable discrete structures and fragile continuous components, and prepares the ground for the analysis of resonances and irreversible phenomena.

Resonances, decay, and instability are treated as intrinsic spectral features arising from the analytic continuation of the resolvent. Unstable states are not anomalies of the formal theory, but natural expressions of spectral singularities governing long-time behavior. Scattering theory is finally incorporated as a statement about spectral invariance and connectivity: asymptotic behavior emerges from the global organization of spectral components rather than from explicit state trajectories.

Taken together, Chapters 1–7 establish a unified operator-first and spectral framework in which quantum theory is understood as the study of spectral topology, analytic structure, and operator-defined invariants. This foundation prepares the conceptual and mathematical ground for subsequent developments, including arithmetic operators, spectral geometry, and the SMRK program, while remaining fully consistent with the standard formalism of quantum mechanics.

1 Operators and the State Space of Quantum Mechanics

1.1 State Space

The fundamental mathematical object of quantum mechanics is the state space, which is modeled as a complex Hilbert space $\mathcal{H}$. This space serves as the carrier of the states of a physical system and provides a natural framework for the definition of the inner product, norm, and convergence. From the physical point of view, a vector in $\mathcal{H}$ is not itself a physical state, but rather a representation of a state. A physical state is determined by an equivalence class of vectors differing by a nonzero complex scalar. Already at this level it becomes apparent that the essential information is not contained in a particular linear representation, but in structures that are invariant under this choice.

From a more modern perspective, a state can be understood not merely as a point in a linear space, but as an object characterized by its projection properties with respect to the spectral decompositions of relevant operators. The state space thus does not function merely as a linear container, but as an environment in which spectral relations between operators and measurable quantities are realized.

1.2 Linear Operators as Physical Observables

In quantum mechanics, physical observables are represented by linear operators acting on the state space $\mathcal{H}$. The requirement of linearity ensures compatibility with the principle of superposition, while further properties of operators are dictated by their physical interpretation.

A central role is played by self-adjoint operators, whose spectra are real and allow a direct interpretation of eigenvalues as possible outcomes of measurements. Eigenvectors (or, more precisely, eigenspaces) determine the projection structure through which the state of the system is reduced during measurement.

It is important to emphasize that the physical meaning of an operator is not determined by its explicit algebraic form, but by the structure of its spectrum and the associated projections. The

linear action of the operator is a technical means, whereas the carrier of physical information is the spectral decomposition.

1.3 The Spectrum as the Carrier of Physical Information

To every self-adjoint operator A one associates a spectrum $\sigma(A)$, which may be discrete, continuous, or of mixed type. This spectrum determines not only the possible measurement outcomes, but also the manner in which the state space decomposes into invariant subspaces.

From this viewpoint, an operator is best understood as a spectral object: its action is fully characterized by its spectral measure and the associated projection-valued decomposition. The linear-algebraic representation is secondary and often basis-dependent, whereas the spectral structure is invariant.

This perspective becomes particularly important in the case of unbounded operators, where questions of domain, closure, and self-adjointness cannot be separated from spectral analysis. Here it becomes clear that the spectrum is not an auxiliary notion, but the central object of the theory.

1.4 Measurement and Projection Structure

The process of measurement can be described mathematically using projection operators associated with the spectral decomposition of the measured observable. The probability of individual outcomes is determined by projecting the state vector onto the corresponding spectral subspaces.

State reduction after measurement is therefore not a general property of linear operators, but a direct consequence of their spectral structure. The collapse of the state can be understood as a loss of coherence between different spectral components, rather than as a violation of the linearity of the dynamical equations.

1.5 Remark on the Spectral Interpretation

Although quantum mechanics is traditionally formulated in the language of linear operators on a Hilbert space, a deeper analysis reveals that the decisive role is played by their spectra and the associated projection structures. The linear formalism serves as a technical apparatus, while the physical content is carried by the spectral properties of operators.

This viewpoint does not contradict the classical formulation of quantum mechanics, but rather refines and unifies it, especially in the context of unbounded operators, continuous spectra, and general measurement schemes.

2 Spectral Decomposition of Operators and Functional Calculus

2.1 Motivation for Spectral Decomposition

In quantum mechanics, many fundamental problems reduce to the study of self-adjoint operators. These operators represent physical observables, and their spectra determine the possible outcomes of measurements. For this reason, spectral analysis occupies a central position both in the mathematical formalism and in the physical interpretation of the theory.

Spectral decomposition allows an operator to be decomposed into elementary components corresponding to different parts of its spectrum. This decomposition provides a unified framework

for describing both discrete and continuous spectra and forms a natural bridge between abstract operator theory and experimentally measurable quantities.

2.2 The Spectral Theorem for Self-Adjoint Operators

The fundamental result of the theory is the spectral theorem for self-adjoint operators. It guarantees that to every self-adjoint operator A on a Hilbert space there corresponds a projection-valued measure $E(\lambda)$ such that

$$A = \int_{\sigma(A)} \lambda dE(\lambda).$$

This representation does not merely express a formal decomposition of the operator, but reveals its intrinsic structure: the operator is completely determined by its spectrum and the associated projections. The linear action of the operator on state vectors arises from the integration of elementary spectral contributions.

From a physical point of view, the projections $E(\Delta)$ correspond to measurable events, namely the statement that the result of a measurement lies within a given set $\Delta \subset \mathbb{R}$. The spectral theorem thus provides a precise mathematical formulation of the postulates of quantum measurement.

2.3 Discrete and Continuous Spectrum

The spectrum of an operator may exhibit different characteristics. A discrete spectrum consists of isolated eigenvalues with corresponding eigenvectors, while a continuous spectrum manifests itself through the absence of normalizable eigenstates and requires a description in terms of generalized eigenfunctions.

From the perspective of spectral decomposition, both types of spectrum are treated in a unified manner through the projection-valued measure. The distinction lies not in the principle of decomposition, but in the analytic properties of the projections and the topology of the state space.

This unified viewpoint emphasizes that the distinction between discrete and continuous spectrum is not fundamental from the structural point of view, but reflects specific physical properties of the system under consideration.

2.4 Functional Calculus

Spectral decomposition enables the introduction of functional calculus for self-adjoint operators. Given a suitable function f , one can define the operator

$$f(A) = \int_{\sigma(A)} f(\lambda) dE(\lambda),$$

which admits a clear physical interpretation.

This apparatus is particularly important for the description of time evolution, where the exponential of the Hamiltonian appears in the form of a unitary evolution operator. Functional calculus thus shows that the dynamics of a quantum system is naturally expressible as a transformation of the spectrum, rather than merely as a formal solution of differential equations.

2.5 Spectral Interpretation of Dynamics

The time evolution of states is generated by the Hamiltonian, whose spectral properties determine the character of the system's dynamics. A discrete spectrum leads to quasiperiodic behavior,

whereas a continuous spectrum is associated with dissipative or scattering phenomena.

From this perspective, dynamics can be understood as a flow of phases assigned to individual spectral components. The evolution of a state is therefore not primarily a motion of vectors in a linear space, but a coordinated transformation of the spectral contributions from which the state is composed.

2.6 Remark on the Spectral Viewpoint

Spectral decomposition and functional calculus demonstrate that the essential physical information is contained in the spectrum of operators and in its projection structure. The linear-algebraic apparatus serves as a means for manipulating these objects, whereas the substantive content of the theory is determined by spectral properties.

This viewpoint provides a unified explanation of measurement, dynamics, and stability of quantum systems, and prepares the ground for the study of more advanced structures such as unbounded operators, resolvents, and spectral singularities.

3 Unbounded Operators, Domains, and Self-Adjointness

3.1 Unbounded Operators in Quantum Mechanics

In quantum mechanics, one naturally encounters operators that are not bounded on the entire Hilbert space. Typical examples include the position and momentum operators, as well as Hamiltonians of realistic physical systems. Such operators cannot be understood as everywhere-defined linear maps, but rather as objects acting on suitably chosen subspaces of the state space.

From the mathematical point of view, it is therefore essential to distinguish clearly between an operator and its domain. An unbounded operator A is defined only on a dense subset $D(A) \subset \mathcal{H}$, and its properties cannot be studied independently of this choice.

From the physical perspective, this fact reflects the reality that not all vectors in the Hilbert space correspond to physically admissible states for a given observable. The domain of an operator is thus not a technical detail, but an integral part of the physical specification of the system.

3.2 Closedness and Closability

For unbounded operators, the notion of closedness plays a fundamental role. An operator is called closed if its graph is a closed subset of the Cartesian product $\mathcal{H} \times \mathcal{H}$. This property ensures stability under limiting processes and is indispensable for a consistent spectral analysis.

Many physically relevant operators are not closed, but are closable. This means that they admit a closure, which extends the original operator to a closed one. In practice, one therefore works not with the original operator, but with its closure, which carries the physically meaningful spectral information.

From this viewpoint, closedness can be regarded as a condition for the existence of a well-defined spectrum. An operator that is neither closed nor closable cannot be interpreted as a physical observable.

3.3 Symmetry and Self-Adjointness

The symmetry of an operator, defined by the relation

$$\langle A\psi, \phi \rangle = \langle \psi, A\phi \rangle \quad \text{for all } \psi, \phi \in D(A),$$

is a necessary, but not sufficient, condition for the physical interpretation of an operator as an observable.

The decisive property is self-adjointness, which guarantees the reality of the spectrum and the existence of a spectral decomposition. For unbounded operators, self-adjointness is not merely an algebraic property, but is intimately tied to the choice of domain. Two different self-adjoint extensions of the same symmetric operator may lead to different spectral properties.

This fact has a direct physical consequence: the choice of a self-adjoint extension corresponds to the choice of boundary conditions or, more generally, to the specification of the physical model.

3.4 Deficiency Indices and Spectral Structure

The theory of deficiency indices provides a systematic tool for studying self-adjoint extensions of symmetric operators. These indices determine whether self-adjoint extensions exist and, if so, how many such extensions there are.

From the spectral point of view, deficiency indices can be interpreted as a measure of indeterminacy in the specification of the spectral structure of an operator. If the indices vanish, the spectral structure is uniquely determined. Otherwise, there exists a family of possible spectra, each corresponding to a different physical realization of the system.

This perspective emphasizes that the spectrum is not solely determined by the algebraic properties of an operator, but depends sensitively on its domain and on the manner in which it is closed.

3.5 Self-Adjointness and Time Evolution

The self-adjointness of the Hamiltonian is a crucial condition for the existence of unitary time evolution. Stone's theorem guarantees that to every self-adjoint operator there corresponds a one-parameter group of unitary operators describing the temporal evolution of the system.

From the viewpoint of spectral decomposition, this evolution is realized through phase factors associated with individual spectral components. The stability and reversibility of the dynamics are therefore direct consequences of the spectral properties of the Hamiltonian.

This relation once again confirms that the dynamics of a quantum system is fundamentally a spectral phenomenon, while differential equations serve as its formal representation.

3.6 Remark on the Spectral Role of Domains

The analysis of unbounded operators shows that the domain of an operator is an essential part of its physical interpretation. Spectrum, self-adjointness, and time evolution cannot be separated from the question of the space on which the operator acts.

From this perspective, an operator can be said to “live” not merely in the Hilbert space, but in the refined structure formed by its domain, its closure, and its spectral properties. The linear formalism provides the framework, but the actual content of the theory is determined by the spectral topology of these objects.

4 Resolvent, Spectral Projections, and Stability

4.1 The Resolvent Operator

Let A be a self-adjoint operator on a Hilbert space $\mathcal{H}$. For complex numbers z not belonging to the spectrum $\sigma(A)$, the operator $(A - zI)$ is invertible, and its inverse

$$R(z) = (A - zI)^{-1}$$

is called the resolvent of A at the point z .

Although A may be unbounded, the resolvent $R(z)$ is a bounded operator for all $z \in \mathbb{C} \setminus \sigma(A)$. This fact allows unbounded operators to be studied through families of bounded operators parameterized by the complex spectral variable.

From the operator-first viewpoint, the resolvent is not a secondary construction, but a primary analytic object through which the operator reveals its spectral organization.

4.2 Analytic Structure and Spectral Detection

The resolvent is an analytic operator-valued function on the resolvent set $\rho(A) = \mathbb{C} \setminus \sigma(A)$. Singularities of this function correspond precisely to spectral points of the operator.

In this sense, the resolvent functions as a *spectral microscope*: by probing the operator with complex spectral parameters, one detects fine-grained spectral features that are not accessible through direct inspection of eigenvectors or algebraic expressions.

Isolated poles of the resolvent correspond to discrete eigenvalues, while branch cuts and non-isolated singular behavior reflect continuous or unstable spectral components. The local behavior of the resolvent thus encodes detailed information about the spectral topology of the operator.

4.3 Spectral Projections from the Resolvent

Spectral projections can be constructed directly from the resolvent by contour integration in the complex plane. For a closed contour Γ enclosing a portion of the spectrum, one defines

$$P_\Gamma = \frac{1}{2\pi i} \oint_\Gamma (A - zI)^{-1} dz.$$

This formula shows that projections are derived from the analytic structure of the resolvent. In the operator-first interpretation, projections are not primitive objects but spectral summaries extracted from the resolvent.

Physically, these projections correspond to idealized measurements that isolate spectral regions. Conceptually, they demonstrate how measurement theory emerges from the analytic properties of operators rather than being imposed independently.

4.4 Resolvent Norms and Spectral Stability

The stability of spectral features under perturbations is closely linked to the behavior of the resolvent norm. Regions where the resolvent remains uniformly bounded correspond to stable spectral configurations, while large resolvent norms signal spectral sensitivity or instability.

From the perspective of the spectral microscope, instability appears as a magnification effect: small perturbations of the operator lead to large changes in the resolvent near certain spectral regions. This behavior is particularly pronounced near continuous spectra and spectral thresholds.

Stability is therefore not characterized solely by the persistence of eigenvalues, but by the global analytic behavior of the resolvent as a function of the spectral parameter.

4.5 Physical Interpretation of Resolvent-Based Stability

In physical terms, sharply localized spectral features correspond to well-isolated resolvent singularities, indicating robust and long-lived states. Diffuse or unstable phenomena are reflected in broad or highly sensitive resolvent behavior.

This interpretation is especially relevant in contexts such as scattering theory and resonance phenomena, where the resolvent encodes decay rates, lifetimes, and transition structures that are invisible at the level of static spectra.

4.6 Remark on the Spectral-Microscopic Viewpoint

The interpretation of the resolvent as a spectral microscope unifies spectral analysis, stability theory, and physical interpretation within a single operator-theoretic framework. Rather than cataloging spectral values, one studies how spectral structures respond to analytic probing.

This viewpoint prepares the conceptual ground for the analysis of resonances, scattering, and spectral deformation, where the resolvent becomes the primary carrier of both mathematical and physical information.

5 Resolvent, Spectral Projections, and Stability

5.1 The Resolvent Operator

Let A be a self-adjoint operator on a Hilbert space $\mathcal{H}$. For complex numbers z not belonging to the spectrum $\sigma(A)$, the operator $(A - zI)$ is invertible, and its inverse

$$R(z) = (A - zI)^{-1}$$

is called the resolvent of A at the point z .

The resolvent is a bounded operator for all $z \in \mathbb{C} \setminus \sigma(A)$, even when A itself is unbounded. This property makes the resolvent a powerful analytical tool, allowing one to study unbounded operators through families of bounded ones.

From a spectral perspective, the resolvent encodes complete information about the spectrum of the operator. Singularities of the resolvent correspond precisely to spectral points, and the behavior of $R(z)$ near the spectrum reflects its local structure.

5.2 Analytic Properties of the Resolvent

The resolvent is an analytic operator-valued function of the complex parameter z on the resolvent set $\rho(A) = \mathbb{C} \setminus \sigma(A)$. This analyticity plays a crucial role in spectral theory, enabling the use of complex-analytic techniques in the study of operators.

In particular, the growth behavior of the resolvent near the real axis provides insight into the nature of the spectrum. Isolated poles correspond to discrete eigenvalues, while branch-type singularities are associated with continuous spectral components.

The resolvent identity further reveals the internal coherence of the theory by relating the resolvents at different spectral parameters. This identity forms the algebraic basis for perturbative and stability analyses.

5.3 Spectral Projections via the Resolvent

Spectral projections can be constructed using the resolvent through contour integration in the complex plane. Given a closed contour Γ enclosing a portion of the spectrum, the associated projection operator is defined by

$$P_\Gamma = \frac{1}{2\pi i} \oint_\Gamma (A - zI)^{-1} dz.$$

This construction shows that spectral projections are derived objects, obtained from the analytic structure of the resolvent. The projection isolates the spectral components of the operator corresponding to the enclosed region of the spectrum.

From the physical point of view, these projections correspond to idealized measurements that distinguish between different spectral regions. The resolvent thus serves as a bridge between analytic operator theory and the projection-based description of measurement.

5.4 Stability of the Spectrum

The stability of the spectrum under perturbations is a central issue in both mathematics and physics. Small changes in the operator may lead to significant changes in the spectrum, particularly in the presence of continuous spectral components.

The resolvent provides a natural framework for analyzing stability. If the resolvent varies continuously with respect to perturbations, one can infer stability properties of the spectrum. Conversely, instability often manifests itself through the emergence of large resolvent norms near certain spectral regions.

This viewpoint highlights that spectral stability is not merely a question of eigenvalues, but of the global behavior of the resolvent as an analytic object.

5.5 Physical Interpretation of Resolvent Stability

In physical terms, stability of the resolvent is closely related to the robustness of observed phenomena. Sharp spectral features correspond to well-isolated poles of the resolvent, while broad or unstable features are reflected in diffuse or highly sensitive resolvent behavior.

This interpretation is particularly relevant in scattering theory and resonance phenomena, where the resolvent encodes information about transient states and decay processes.

Thus, the resolvent serves not only as a mathematical device, but as a conceptual tool for understanding the stability and instability of physical systems.

5.6 Remark on the Resolvent-Centered Viewpoint

The resolvent-centered approach emphasizes that spectral properties are best understood through analytic structures rather than purely algebraic ones. Projections, spectra, and stability properties all emerge naturally from the behavior of the resolvent.

This perspective prepares the ground for the study of resonances, scattering theory, and non-self-adjoint extensions, where the resolvent becomes the primary carrier of physical and mathematical information.

6 Resolvents, Resonances, and Unstable States

6.1 Operator-First Viewpoint

In the operator-first formulation of quantum theory, physical phenomena are not described primarily in terms of trajectories, wavefunctions, or time-dependent amplitudes, but as structural properties of operators acting on a state space. Within this viewpoint, the resolvent of an operator plays a privileged role, as it encodes the full spectral and dynamical content of the system in a single analytic object.

Rather than treating the resolvent as a secondary construction derived from an operator, the operator-first perspective regards the resolvent as a primary probe of physical behavior. Spectral points, resonances, and instabilities are revealed directly through the analytic structure of the resolvent, without reference to explicit eigenstates.

6.2 Resonances as Spectral Singularities

Resonances arise in systems where bound states interact with a continuum. From the operator-first perspective, resonances are not eigenvalues in the traditional sense, but manifest as singularities of the analytically continued resolvent.

These singularities typically appear as poles located off the real axis on unphysical sheets of the complex energy plane. Their position encodes both the energy and the decay rate of the associated unstable state.

This description emphasizes that resonances are inherently spectral phenomena. They are defined not by vectors in the Hilbert space, but by the analytic behavior of the resolvent as an operator-valued function.

6.3 Analytic Continuation of the Resolvent

To access resonances, one extends the resolvent beyond the real axis through analytic continuation. This procedure reveals spectral structures that are invisible in the self-adjoint framework restricted to real energies.

In the operator-first language, analytic continuation is not an auxiliary trick, but a natural extension of spectral analysis. It allows the operator to be probed beyond its stable spectral regime and exposes latent dynamical features encoded in its structure.

The resulting complex spectral data provide a unified description of scattering, metastability, and decay within a single operator-theoretic framework.

6.4 Unstable States and Spectral Decay

Unstable states correspond to spectral components that do not persist under time evolution. In the operator-first picture, instability is not defined by the failure of a state vector to remain normalized, but by the presence of non-real spectral singularities governing the long-time behavior of observables.

Time evolution generated by the Hamiltonian can be analyzed spectrally through the resolvent. Exponential decay laws arise from isolated resonance poles, while more complex decay patterns reflect branch cuts and continuous spectral components.

Thus, decay and irreversibility are not imposed externally, but emerge from the spectral topology of the operator.

6.5 Scattering and Resolvent Structure

Scattering theory finds a natural formulation in terms of the resolvent. Incoming and outgoing asymptotic behavior is encoded in boundary values of the resolvent near the continuous spectrum.

From the operator-first standpoint, scattering is a manifestation of how the resolvent connects different spectral regions. The scattering matrix can be understood as a derived object, obtained from the resolvent rather than introduced as an independent construct.

This reinforces the idea that the resolvent serves as the central carrier of physical information, while other quantities are secondary projections of its structure.

6.6 SMRK-Oriented Interpretation

In the SMRK framework, operators are treated as fundamental spectral generators rather than as mere quantizations of classical observables. Resonances and unstable states are interpreted as intrinsic features of the operator's spectral geometry.

The resolvent acts as a spectral microscope, revealing fine-grained structures that govern dynamics, stability, and decay. In this sense, unstable states are not anomalies or pathologies, but natural expressions of spectral complexity.

This interpretation aligns with a broader operator-first philosophy in which physical laws are encoded in spectral relations and analytic structures, rather than in explicit time-domain evolution equations.

6.7 Remark on Spectral Irreversibility

The operator-first analysis of resonances clarifies the origin of irreversibility in quantum systems. While the underlying operator may be self-adjoint, its analytically continued resolvent exhibits structures that break time-reversal symmetry at the spectral level.

Irreversibility thus emerges not from a modification of the dynamical laws, but from the spectral organization of the operator. This viewpoint provides a coherent framework for understanding decay, dissipation, and the arrow of time within a rigorously operator-based theory.

7 Scattering Theory and Spectral Invariance

7.1 Scattering in the Operator-First Framework

In the operator-first formulation, scattering theory is not introduced as a separate dynamical scheme, but arises naturally from the spectral topology of operators. Physical processes traditionally described by asymptotic states and wave packets are reinterpreted as relations between spectral regions encoded by the resolvent.

From this viewpoint, scattering does not primarily concern trajectories or time-domain evolution, but the manner in which the operator connects incoming and outgoing spectral components. The emphasis shifts from states to operators, and from temporal evolution to spectral structure.

7.2 Resolvent as a Spectral Probe

Throughout the preceding chapters, the resolvent has emerged as a central analytic object. In scattering theory, its role becomes explicit: the resolvent acts as a spectral probe, interrogating how perturbations deform the spectral topology of the operator.

Boundary values of the resolvent near the continuous spectrum encode the asymptotic behavior of physical processes. Scattering amplitudes and transition probabilities are derived quantities, obtained from the analytic structure of the resolvent rather than postulated independently.

This perspective unifies scattering with resonance theory and spectral stability within a single operator-theoretic framework.

7.3 Wave Operators and Spectral Relations

Wave operators are traditionally introduced to compare the dynamics of an interacting system with a reference free system. In the operator-first language, they express spectral relations between two operators rather than mappings between explicit state trajectories.

Their existence and completeness are determined by spectral properties such as absolute continuity and the absence of singular spectral components. Thus, wave operators reflect the compatibility of spectral topologies between different operator realizations.

7.4 Spectral Invariance under Scattering

A central result of scattering theory is the invariance of certain spectral components under interaction. In particular, the absolutely continuous part of the spectrum often remains stable under a broad class of perturbations.

From the operator-first perspective, spectral invariance is not a coincidence but a structural property of the resolvent. Perturbations that do not alter the global spectral topology preserve scattering channels, even though local spectral features may be deformed.

This invariance explains why scattering phenomena exhibit universal characteristics largely independent of microscopic details.

7.5 Operator-First Dynamics and Asymptotic Behavior

Asymptotic dynamics emerges from the long-range spectral organization of the operator. Instead of tracking the time evolution of individual states, operator-first dynamics examines how spectral components separate, persist, or interfere at infinity.

Scattering processes thus correspond to stable spectral pathways encoded in the resolvent. The arrow from incoming to outgoing configurations is a manifestation of spectral organization rather than an imposed temporal boundary condition.

7.6 SMRK-Calibrated Interpretation

Within the SMRK framework, scattering theory is interpreted as a statement about spectral connectivity. Operators act as generators of spectral relations, and scattering reveals how these relations remain invariant under admissible deformations.

The resolvent, functioning as a spectral probe, exposes invariant structures that govern transport, interaction, and stability. This interpretation aligns scattering theory with the broader operator-first philosophy developed throughout the preceding chapters.

7.7 Remark on Unified Spectral Topology

Taken together, the analysis of resolvents, resonances, perturbations, and scattering demonstrates that spectral topology provides a unifying language for quantum theory. Dynamics, stability,

and asymptotic behavior are all expressions of how operators organize their spectra.

In this unified view, scattering theory does not introduce new principles, but completes the operator-first narrative by revealing the invariant spectral structures that persist across interactions.

Spectral Dynamics and Operator-First Probing

Invariants, Operator Algebras, and Arithmetic Spectral Methods

Enter Yourname

Abstract

This chapter develops the dynamical and methodological layer of the operator-first spectral framework. Building on the foundational principles established earlier, the focus shifts from static spectral structure to spectral dynamics, invariants, and probing strategies that govern long-time behavior and stability in operator-defined systems.

Spectral dynamics is formulated in terms of spectral organization rather than state-level trajectories. Time evolution is interpreted as a transformation of spectral components, with long-time behavior emerging from the interplay between discrete, continuous, and invariant spectral structures. This perspective provides a unified account of recurrence, dispersion, and asymptotic regimes without departing from unitary operator theory.

Conservation principles are reinterpreted as manifestations of spectral invariants. Invariant subspaces, commuting operator structures, and stable spectral relations define the limits within which dynamics unfolds. This reformulation aligns conservation laws with operator-defined spectral topology rather than with explicit dynamical equations.

Operator algebras are introduced as organizing structures that govern spectral symmetries and stability. Algebraic relations constrain admissible spectral configurations and protect invariant features under perturbation. This algebraic viewpoint elevates spectral analysis from individual operators to structured families, providing a natural setting for both physical and arithmetic systems.

The framework is then extended to arithmetic contexts through the introduction of arithmetic operators and spectral probes. Rather than encoding arithmetic information directly, these operators generate spectral patterns whose structure can be interrogated using resolvent-based and statistical methods. Spectral probing replaces direct construction with analytic and numerical interrogation of spectral organization.

Numerical methods are presented as disciplined instruments of spectral probing rather than as approximation tools. Finite-dimensional

truncation, resolvent evaluation, stability testing, and reproducibility protocols are integrated into a coherent methodology that preserves operator-theoretic meaning while enabling large-scale exploration.

The chapter concludes by articulating the limitations of probing-based reasoning and by identifying open questions that motivate further development. Spectral probing is established as a necessary preparatory stage: it reveals constraints, invariants, and structural regularities, but stops short of asserting necessity. This methodological position prepares the transition to enforcement-oriented arguments developed in subsequent work.

1 Spectral Dynamics and Long-Time Behavior

1.1 Dynamics Beyond State Trajectories

In the operator-first formulation, dynamics is not primarily described as the evolution of state vectors along trajectories in Hilbert space. Instead, dynamical behavior is encoded in the spectral organization of the operators that generate time evolution.

From this perspective, time evolution does not act on states as primitive objects, but on their spectral composition. The relevant question is therefore not how a vector evolves in time, but how spectral components are transformed, redistributed, or suppressed under the action of the governing operator.

This shift in viewpoint becomes essential for understanding long-time behavior, where state-based descriptions often obscure the structural mechanisms responsible for stability or decay.

1.2 Spectral Representation of Time Evolution

Let H be a self-adjoint Hamiltonian generating time evolution. Through functional calculus, the evolution operator is given by

$$U(t) = e^{-itH} = \int_{\sigma(H)} e^{-it\lambda} dE(\lambda),$$

where $E(\lambda)$ denotes the spectral measure of H .

This representation shows that time evolution acts independently on each spectral component by assigning a phase factor. Long-time behavior is therefore governed by the distribution and analytic structure of the spectrum rather than by local properties of state vectors.

Discrete spectral components give rise to persistent oscillatory behavior, while continuous components lead to dispersion, dephasing, or decay.

1.3 Asymptotic Regimes and Spectral Separation

As time progresses, different spectral components contribute in qualitatively different ways. Interference between components with incommensurate spectral values leads to phase mixing, effectively suppressing coherent contributions from extended spectral regions.

From the spectral viewpoint, asymptotic regimes correspond to the progressive separation of spectral contributions. Stable long-time features arise from isolated or invariant spectral structures, while unstable features are washed out by spectral dispersion.

This mechanism explains the emergence of effective irreversibility without invoking non-unitary dynamics.

1.4 Long-Time Decay and Spectral Continuity

Long-time decay phenomena are closely tied to the presence of continuous spectral components. In such cases, destructive interference between spectral phases leads to a gradual loss of coherence observable in physical quantities.

In the operator-first language, decay is not attributed to a failure of unitarity, but to the spectral continuity of the generator of dynamics. The rate and character of decay are determined by the local spectral density and its analytic regularity.

This viewpoint naturally connects long-time behavior with the analytic properties of the resolvent and its continuation.

1.5 Invariant Spectral Structures

Despite dispersion and decay, certain spectral structures may remain invariant under time evolution. These include spectral projections associated with conserved quantities or symmetry-protected subspaces.

Such invariants define persistent features of the dynamics that survive in the long-time limit. From the operator-first perspective, conservation laws are therefore understood as statements about spectral invariance rather than about the preservation of particular state components.

This reinterpretation aligns conservation principles with spectral topology.

1.6 Spectral Topology and Dynamical Classification

The classification of dynamical behavior can be reformulated in purely spectral terms. Systems with predominantly discrete spectra exhibit recurrent or quasiperiodic dynamics, while systems dominated by continuous spectra display dispersive or scattering behavior.

Mixed spectra give rise to hybrid regimes in which stable and unstable features coexist. The long-time behavior of such systems is governed by the interaction between discrete and continuous spectral components.

Spectral topology thus provides a natural taxonomy of dynamical regimes, independent of microscopic modeling details.

1.7 Remark on Operator-First Long-Time Analysis

The operator-first approach reveals that long-time behavior is not an emergent mystery but a direct consequence of spectral organization. By analyzing the spectrum, its continuity, and its invariants, one gains predictive control over asymptotic dynamics.

This perspective prepares the ground for subsequent chapters, where spectral invariants, symmetries, and arithmetic operators are studied as fundamental drivers of structure and stability in both physical and arithmetic systems.

2 Spectral Invariants and Conservation Principles

2.1 From Dynamical Laws to Spectral Invariants

In the operator-first formulation, conservation principles are not introduced as fundamental dynamical laws imposed on time evolution. Instead, they arise as invariant features of the spectral structure of operators generating the dynamics.

A quantity is conserved not because a specific component of a state vector remains unchanged, but because certain spectral relations persist under the action of the evolution operator. Conservation is therefore reinterpreted as spectral invariance rather than as trajectory-level preservation.

This shift aligns conservation principles with the operator-centric ontology developed in the preceding chapters.

2.2 Invariant Subspaces and Spectral Projections

Spectral invariants manifest themselves through invariant subspaces associated with spectral projections. If a projection commutes with the generator of dynamics, the corresponding spectral component remains dynamically isolated.

Such invariant projections define conserved sectors of the theory. Physical quantities traditionally associated with conserved observables correspond, in this framework, to spectral decompositions that remain stable under evolution.

From the operator-first perspective, invariance is thus a property of projection structure rather than of individual states.

2.3 Commutation Relations and Structural Conservation

Conservation laws are often expressed through commutation relations between operators. In the spectral formulation, these relations acquire a structural interpretation: commuting operators share compatible spectral decompositions.

When two operators commute, their spectra can be jointly resolved, leading to invariant spectral labels that persist throughout the dynamics. These labels encode conserved quantities without reference to explicit equations of motion.

This viewpoint emphasizes that conservation is fundamentally a statement about spectral compatibility.

2.4 Continuous Spectra and Weak Invariants

In systems with continuous spectra, conservation principles may appear in weakened or distributional forms. While sharp spectral projections may be absent, invariant structures can still be identified through asymptotic spectral behavior.

Such weak invariants govern long-time averages, transport properties, and scattering channels. They are not associated with exact eigenvalues, but with stable features of the spectral density and its asymptotic organization.

This generalization extends the notion of conservation beyond discrete settings and is essential for the analysis of open or extended systems.

2.5 Spectral Invariants and Stability

Spectral invariants play a central role in determining the stability of physical systems. Invariant spectral components act as anchors that prevent uncontrolled dispersion or decay.

From the operator-first standpoint, stability is not an external requirement but a direct consequence of invariant spectral topology. Systems lacking such invariants exhibit enhanced sensitivity to perturbations and pronounced irreversible behavior.

This relation provides a unifying explanation of stability across bound, scattering, and mixed regimes.

2.6 Operator-First Conservation Principles

Recasting conservation laws in spectral terms clarifies their conceptual status. Rather than constraining dynamics, conservation principles describe the structural limits within which dynamics unfolds.

In this sense, operators define the permissible spectral transformations, while invariants identify structures that remain fixed under these transformations. Conservation is thus an intrinsic property of operator-defined systems.

2.7 SMRK-Oriented Perspective

Within the SMRK framework, spectral invariants acquire additional significance. Arithmetic operators, prime-shift structures, and number-theoretic symmetries are expected to generate nontrivial invariant spectral features.

These invariants serve as probes of deep structural regularities and form the conceptual bridge between physical operator theory and arithmetic spectral analysis. Their study prepares the ground for operator-based probes of arithmetic conjectures.

2.8 Remark on Spectral Conservation

Spectral invariants unify conservation, stability, and symmetry within a single operator-theoretic framework. By focusing on invariant spectral relations rather than on state-level quantities, one obtains a robust and extensible formulation applicable to both physical and arithmetic systems.

This viewpoint sets the stage for the analysis of operator algebras and spectral symmetries developed in the subsequent chapter.

3 Operator Algebras and Spectral Symmetries

3.1 From Individual Operators to Algebraic Structures

While the preceding chapters focused on individual operators and their spectra, a deeper structural understanding emerges when operators are studied collectively as elements of algebras. Operator algebras provide a natural language for describing families of operators that share structural, spectral, or dynamical properties.

In the operator-first framework, algebras are not auxiliary constructions but fundamental organizational structures. They encode the admissible transformations, symmetries, and relations that govern spectral behavior across entire classes of operators.

3.2 Algebras of Observables and Spectral Organization

An algebra of observables is typically formed by operators that are closed under addition, multiplication, and adjunction. The spectral properties of such an algebra are not determined by individual spectra alone, but by collective constraints imposed by algebraic relations.

From the spectral viewpoint, an operator algebra defines a space of compatible spectral structures. Shared algebraic relations lead to shared spectral features, including common invariants, degeneracies, and stability properties.

This perspective emphasizes that spectra are shaped not only by single operators, but by the algebraic environment in which they reside.

3.3 Symmetries as Algebraic Automorphisms

Symmetries are naturally expressed as automorphisms of operator algebras. An automorphism maps operators to operators while preserving algebraic relations and adjunction structure.

In spectral terms, such automorphisms induce transformations that preserve essential spectral topology. Symmetric systems are therefore characterized not by fixed numerical spectra, but by invariant spectral relations under algebraic transformations.

This formulation unifies continuous and discrete symmetries within a single algebraic framework.

3.4 Commutants and Spectral Decomposition

The commutant of an operator algebra consists of all operators that commute with every element of the algebra. The structure of the commutant plays a decisive role in determining spectral multiplicities and decomposition properties.

From an operator-first standpoint, commutants encode latent symmetries that may not be explicitly visible at the level of individual operators. Their presence leads to refined spectral decompositions and invariant subspaces.

This duality between algebras and their commutants provides a powerful tool for organizing spectral information.

3.5 Representations and Spectral Realizations

Operator algebras admit multiple representations on Hilbert spaces. Each representation realizes the abstract algebra as concrete operators with specific spectral properties.

In the spectral framework, different representations correspond to different spectral realizations of the same underlying algebraic structure. Physical distinctions between models arise from the spectral characteristics of these representations rather than from the algebra alone.

This observation clarifies how the same algebraic relations can support diverse physical or arithmetic systems.

3.6 Spectral Symmetry and Stability

Spectral symmetries imposed by operator algebras constrain the possible deformations of spectra under perturbations. Symmetry-protected spectral features remain stable as long as the underlying algebraic relations are preserved.

From the operator-first perspective, stability is therefore an algebraic phenomenon: it arises from invariance under algebraic automorphisms rather than from fine-tuning of parameters.

This insight connects symmetry, invariance, and robustness within a unified spectral language.

3.7 SMRK-Oriented Algebraic Structures

Within the SMRK program, operator algebras acquire an arithmetic dimension. Algebras generated by prime-shift operators, arithmetic translations, or number-theoretic actions are expected to exhibit distinctive spectral symmetries.

These symmetries organize arithmetic spectra and define invariant structures that can be probed through resolvent-based and numerical methods. Operator algebras thus provide the natural setting for embedding arithmetic operators into a coherent spectral framework.

3.8 Remark on Algebraic Spectral Topology

Operator algebras elevate spectral analysis from the level of individual operators to that of structured families. Spectral topology, symmetry, and invariance emerge as collective properties determined by algebraic relations.

This algebraic viewpoint prepares the ground for the introduction of arithmetic operators and explicit spectral probes developed in the following chapter.

4 Arithmetic Operators and Spectral Probes

4.1 Arithmetic Structure in Operator Theory

The operator-first framework developed in the preceding chapters is not restricted to physical observables. It naturally extends to arithmetic settings, where operators encode number-theoretic structure rather than spatial or dynamical variables.

In this context, arithmetic operators act on abstract state spaces whose spectral properties reflect deep arithmetic regularities. The emphasis shifts from geometric interpretation to spectral organization driven by arithmetic relations.

This extension establishes a conceptual bridge between quantum operator theory and arithmetic analysis.

4.2 Arithmetic Operators as Spectral Generators

Arithmetic operators are defined by their action on basis elements labeled by integers, primes, or arithmetic data. Prime-shift operators, multiplicative translations, and arithmetically weighted transformations generate nontrivial spectral structures.

From the operator-first perspective, the defining feature of such operators is not their explicit combinatorial form, but the spectral patterns they induce. Arithmetic information is encoded in spectral distributions, correlations, and singularities.

This viewpoint aligns arithmetic operators with physical Hamiltonians as generators of spectral organization.

4.3 Spectral Probes and Arithmetic Diagnostics

Spectral probes are operators or analytic constructions designed to interrogate spectral structure. The resolvent plays a central role as a universal spectral probe, capable of detecting fine-grained features through its analytic behavior.

In arithmetic contexts, spectral probes are used to reveal hidden regularities, resonances, or invariants associated with number-theoretic structures. Singularities, growth rates, and stability properties of resolvents provide diagnostic signals of arithmetic content.

This probing strategy replaces direct enumeration by analytic interrogation.

4.4 Arithmetic Resonances and Spectral Signatures

Arithmetic operators may exhibit resonance-like phenomena analogous to those found in physical systems. These arithmetic resonances manifest as spectral concentrations, correlated eigenvalue statistics, or distinctive analytic features of spectral probes.

Such signatures are not interpreted as physical decay, but as indicators of structural interference between arithmetic components. Their detection relies on resolvent analysis, spectral statistics, and long-range correlations.

This analogy clarifies the role of resonance concepts in arithmetic spectral analysis.

4.5 Invariant Structures and Arithmetic Symmetries

Arithmetic operators often possess symmetries arising from number-theoretic operations such as multiplication, inversion, or modular reduction. These symmetries impose constraints on the spectrum and generate invariant spectral structures.

From the operator-first standpoint, arithmetic symmetries are expressed as algebraic relations that preserve spectral topology. Invariants derived from these symmetries serve as stable markers of arithmetic structure.

Such invariants play a crucial role in distinguishing meaningful spectral features from noise or model-dependent artifacts.

4.6 Numerical and Analytic Probing Strategies

The analysis of arithmetic spectra requires a combination of analytic and numerical methods. Resolvent-based probes, trace formulas, and spectral statistics provide analytic access, while numerical experiments test stability and reveal fine structure.

In the QFM and SMRK context, numerical probing is not an end in itself but a means of validating spectral hypotheses. Numerical results are interpreted through the lens of operator-first spectral theory rather than as isolated computational observations.

This integrated approach ensures conceptual coherence between theory and computation.

4.7 SMRK-Oriented Perspective

Within the SMRK program, arithmetic operators are treated as central objects whose spectra encode conjectural structures of number theory. Spectral probes are designed to test these structures by detecting invariants, correlations, and resonance patterns.

Rather than attempting direct proofs, the SMRK approach emphasizes spectral enforcement: constraints imposed by operator structure that restrict admissible spectral behavior.

This chapter thus establishes the methodological foundation for SMRK probing, preparing the transition to explicit numerical methods and experimental protocols.

5 Numerical Spectral Methods and SMRK Probing

5.1 The Role of Numerical Methods in Spectral Analysis

Numerical methods play a complementary role to analytic reasoning in spectral operator theory. In the operator-first framework, numerical computation is not used to approximate states or trajectories, but to probe spectral structure, stability, and invariants that are difficult or impossible to access analytically.

Numerical experiments are therefore interpreted as controlled spectral probes. Their purpose is to reveal structural regularities, detect instabilities, and test hypotheses about spectral organization rather than to produce isolated numerical values.

5.2 Finite-Dimensional Truncations and Spectral Windows

Many operators of interest, particularly in arithmetic and SMRK-oriented contexts, are defined on infinite-dimensional spaces. Numerical access requires finite-dimensional truncation, which must be handled with care to preserve relevant spectral features.

In the spectral viewpoint, truncation defines a spectral window rather than a global approximation. Only spectral regions sufficiently separated from truncation boundaries are interpreted as meaningful. Stability of spectral features under increasing truncation size serves as a primary validation criterion.

This approach aligns numerical practice with operator-theoretic rigor.

5.3 Resolvent-Based Numerical Probes

The resolvent provides a powerful basis for numerical probing. By evaluating resolvent norms, traces, or matrix elements as functions of the spectral parameter, one can detect spectral concentrations, gaps, and resonance-like behavior.

Numerically computed resolvent data act as a spectral microscope, amplifying subtle features that may not be visible in direct spectral plots. Peaks, growth rates, and singular patterns in resolvent-derived quantities are interpreted as indicators of underlying spectral structure.

This strategy mirrors the analytic role of the resolvent while remaining computationally accessible.

5.4 Spectral Statistics and Correlation Analysis

Beyond individual spectral values, numerical methods enable the study of statistical properties of spectra. Level spacing distributions, correlation functions, and aggregate measures provide insight into global spectral organization.

In the SMRK context, such statistics are used to test conjectural arithmetic regularities and to distinguish structured spectra from random or noise-dominated behavior. Statistical stability across truncations and parameter variations is treated as evidence of genuine spectral structure.

This statistical layer complements resolvent-based probing.

5.5 Stability Tests and Perturbative Sensitivity

Numerical probing includes systematic perturbation tests designed to assess spectral stability. By introducing controlled perturbations and monitoring spectral response, one can identify invariant features and sensitive regimes.

From the operator-first perspective, robustness under perturbation signals structural significance, while extreme sensitivity indicates dependence on model-specific details or numerical artifacts. These tests provide an empirical counterpart to analytic stability theory.

5.6 Numerical Experiments as SMRK Probes

Within the SMRK program, numerical experiments are explicitly framed as probes rather than as proofs. Their role is to enforce spectral constraints by identifying patterns that must be respected by any admissible operator realization.

Numerical evidence guides theoretical refinement by highlighting which spectral features are stable, reproducible, and structurally meaningful. Failed probes are as informative as successful ones, indicating directions where assumptions must be revised.

This disciplined use of numerics preserves conceptual clarity while leveraging computational power.

5.7 Reproducibility and Protocol Design

Reproducibility is a central requirement for numerical spectral probing. Experiments are defined by explicit operator constructions, truncation schemes, parameter choices, and evaluation metrics. Results are interpreted relative to these protocols rather than as absolute statements. This procedural clarity ensures that numerical findings can be independently verified and systematically extended.

Such protocol-driven experimentation aligns numerical practice with the methodological standards of the QFM and SMRK frameworks.

5.8 Remark on the Limits of Numerical Probing

While numerical methods provide indispensable insight, they do not replace analytic reasoning. Numerical probes reveal spectral tendencies and constraints, but cannot by themselves establish universal validity.

In the operator-first and SMRK perspectives, numerics function as instruments of spectral exploration and enforcement, guiding theory without supplanting it. Their strength lies in illuminating structure, not in asserting final conclusions.

6 Limitations, Extensions, and Open Spectral Questions

6.1 Scope and Limitations of the Operator-First Framework

The operator-first and spectral framework developed in this book provides a unifying language for quantum, dynamical, and arithmetic systems. Nevertheless, it is important to clarify its scope and inherent limitations.

The approach emphasizes structural and spectral organization over explicit constructive solutions. As a consequence, it does not aim to replace traditional state-based or equation-driven methods in all contexts, but to complement them by exposing invariant and global features that such methods often obscure.

Certain phenomena may require additional geometric, probabilistic, or combinatorial input that lies outside the purely operator-theoretic domain considered here.

6.2 Limitations of Spectral Probing

Spectral probing, whether analytic or numerical, is intrinsically indirect. Probes reveal features of spectral topology and analytic structure, but do not uniquely determine the underlying operator.

Different operators may exhibit similar spectral signatures within finite resolution or restricted spectral windows. This ambiguity imposes fundamental limits on what can be inferred solely from spectral data, particularly in truncated or approximate settings.

Recognizing these limitations is essential for the responsible interpretation of spectral evidence.

6.3 Extensions to Broader Operator Classes

The framework presented here focuses primarily on self-adjoint operators and their resolvents. Natural extensions include non-self-adjoint operators, dissipative systems, and operators defined on more general spaces.

Such extensions are expected to enrich the spectral landscape by introducing new classes of singularities, instabilities, and invariant structures. They also provide potential connections to open quantum systems, irreversible dynamics, and non-Hermitian spectral theory.

These directions remain largely unexplored within the present setting.

6.4 Geometric and Topological Extensions

While spectral topology has played a central role, its relation to geometric and topological structures warrants further investigation. Connections between operator spectra, geometric phases, and topological invariants offer promising avenues for extension.

In arithmetic contexts, analogous questions arise concerning the geometric interpretation of spectral data and its relation to number-theoretic spaces. Developing a coherent geometric-spectral dictionary remains an open challenge.

Such extensions may deepen the conceptual reach of the operator-first approach.

6.5 Open Questions in Spectral Dynamics

Several fundamental questions remain open regarding spectral dynamics and long-time behavior. These include the classification of spectral regimes beyond discrete and continuous dichotomies, the role of singular continuous spectra, and the precise conditions under which spectral invariants guarantee stability.

Understanding the interplay between spectral topology and irreversibility also remains an active area of inquiry. Operator-first methods provide a natural language for posing these questions, but comprehensive answers are still lacking.

6.6 Open Questions in Arithmetic Spectral Analysis

In the arithmetic domain, the spectral approach raises deep and unresolved questions. Which spectral features reliably encode arithmetic structure? How robust are these features under perturbation or truncation? To what extent can spectral invariants enforce number-theoretic constraints?

These questions define the boundary between heuristic insight and rigorous understanding. They motivate the development of refined spectral probes and more sophisticated analytic tools within the SMRK program.

6.7 Toward the SMRK Program

The limitations and open questions identified here do not represent shortcomings, but rather delineate the frontier of the framework. They indicate where new ideas, methods, and collaborations are required.

The SMRK program emerges naturally at this frontier. By focusing on arithmetic operators, spectral enforcement, and resolvent-based probing, it aims to address some of the deepest open problems in spectral and number-theoretic analysis.

Book I thus concludes not with final answers, but with a clarified conceptual foundation and a well-defined set of questions that guide subsequent investigation.

6.8 Concluding Remark

This book has established an operator-first, spectral foundation that unifies dynamics, stability, symmetry, and arithmetic probing within a coherent framework. Its value lies not only in the results presented, but in the perspective it offers.

By treating spectra as primary carriers of structure and meaning, the operator-first approach opens new pathways for understanding complex systems across physics and mathematics. The chapters that follow in subsequent books build directly on this foundation.

Spectral Probing Frameworks and Experimental Architecture

Models, Statistics, and Computational Validation

Enter Yourname

Abstract

This part develops a comprehensive framework for spectral probing, focusing on the methodological, computational, and experimental structures required to investigate operator-defined systems at scale. The emphasis shifts from foundational principles to operational practice: how spectral structure is explored, tested, and constrained through controlled analytic and numerical procedures.

The exposition begins with the introduction of spectral flows and asymptotic regimes, establishing a language for describing how spectra evolve under parametric deformation, truncation growth, and scaling limits. Spectral flow is treated as a diagnostic tool that reveals stability transitions, invariant patterns, and long-range organization beyond static spectral snapshots.

Resolvent traces and spectral statistics are then developed as global probes of spectral structure. By aggregating information across spectral regions, these tools expose correlations, clustering, and analytic signatures that are invisible at the level of individual eigenvalues. Particular attention is given to regularization, robustness, and statistical stability as criteria for structural significance.

Concrete arithmetic spectral models are introduced as exploratory testbeds for probing methodology. These models are not presented as definitive representations of arithmetic reality, but as controlled environments in which spectral probes can be calibrated, compared, and refined. The analysis emphasizes invariants and stable signatures that persist across representations and perturbations.

A layered computational framework is then outlined, integrating operator construction, resolvent evaluation, parameter orchestration, and reproducibility protocols. Computation is treated as an integral component of spectral methodology, with architectural discipline ensuring that numerical exploration remains aligned with operator-theoretic principles.

Experimental results obtained within this framework are presented as structured observations rather than conclusive claims. Emphasis is placed

on robustness analysis, comparative behavior across models, and the separation of structural signals from numerical artifacts. Experimental evidence functions as a source of constraint, narrowing the space of admissible operator realizations.

The part concludes with a systematic discussion of interpretive limits and methodological caveats, clarifying the non-uniqueness of spectral signatures, the role of truncation and regularization, and the dangers of overinterpretation. These considerations motivate a conceptual transition from probing to enforcement.

Taken together, Chapters 14–20 establish spectral probing as a rigorous preparatory stage for spectral enforcement. By articulating how spectral evidence constrains operator structure without yet asserting necessity, this part provides the methodological and experimental foundation for the enforcement-oriented arguments developed in subsequent work.

1 Spectral Flows and Asymptotic Regimes

1.1 From Static Spectra to Spectral Flow

In the operator-first framework, spectral analysis is not limited to static spectra associated with fixed operators. When operators depend on parameters, time, or structural deformations, their spectra evolve accordingly. This evolution is naturally described in terms of spectral flow.

Spectral flow captures how spectral components move, merge, split, or disappear under continuous changes of an operator. Rather than tracking individual eigenvalues as isolated objects, the focus is on global spectral behavior and topological transitions.

This viewpoint extends spectral analysis from a static classification to a dynamical and structural investigation.

1.2 Parametric Families of Operators

Let $\{A(\tau)\}_{\tau \in \mathbb{R}}$ denote a family of operators depending continuously on a real parameter. Such families arise naturally in perturbative analysis, adiabatic processes, and numerical probing schemes.

In the spectral formulation, the relevant object is not a single spectrum $\sigma(A(\tau))$, but the family of spectra viewed as a parameterized structure. Spectral flow describes how spectral components traverse the spectral domain as τ varies.

This parametric perspective is essential for identifying invariant features and detecting structural transitions.

1.3 Asymptotic Regimes and Scaling Behavior

As parameters or truncation scales increase, operators may enter asymptotic regimes where their spectral behavior stabilizes or follows universal patterns. Such regimes are of particular interest in both physical and arithmetic contexts.

From the operator-first standpoint, asymptotic analysis concerns the limiting organization of spectral topology rather than pointwise convergence of eigenvalues. Stability of spectral features across scales serves as an indicator of structural significance.

This approach aligns naturally with numerical probing strategies that explore large-scale behavior.

1.4 Spectral Flow and Stability Transitions

Spectral flow provides a diagnostic for stability and instability. Smooth, controlled spectral motion indicates robustness under deformation, while abrupt changes, crossings, or bifurcations signal transitions between spectral regimes.

In arithmetic and SMRK-oriented settings, such transitions may correspond to changes in dominant arithmetic structure or to the breakdown of approximations used in operator construction. Identifying and classifying these transitions is a central task of spectral probing.

1.5 Long-Range Spectral Organization

Beyond local spectral motion, spectral flow reveals long-range organization across wide parameter intervals. Persistent patterns, recurrent structures, or asymptotic clustering suggest underlying invariants that govern spectral behavior.

These features are not tied to individual parameter values, but to the global structure of the operator family. Their detection requires sustained probing across extended regimes.

This global perspective distinguishes structural spectral analysis from local perturbative methods.

1.6 Spectral Flow as a Probing Tool

Within the SMRK probe framework, spectral flow functions as a high-level diagnostic tool. By monitoring how spectra evolve under controlled deformations, one can test hypotheses about invariance, stability, and structural enforcement.

Rather than seeking exact spectral values, spectral flow analysis focuses on qualitative patterns and constraints. These patterns serve as indicators of admissible operator realizations and guide the refinement of models.

1.7 Remark on Asymptotic Spectral Analysis

The study of spectral flows and asymptotic regimes emphasizes that meaningful spectral information often resides in trends and limits rather than in finite snapshots. Operator-first analysis provides a natural language for articulating these trends.

This perspective prepares the ground for the subsequent investigation of resolvent traces, spectral statistics, and large-scale numerical patterns developed in the following chapters.

2 Resolvent Traces and Spectral Statistics

2.1 From Individual Spectra to Statistical Descriptions

In complex operator systems, particularly those arising in arithmetic and large-scale numerical settings, individual spectral values often provide limited insight. Instead, meaningful structure emerges through aggregate and statistical descriptions of spectra.

The operator-first framework naturally supports this transition. Rather than treating eigenvalues as isolated data points, spectral statistics interpret spectra as structured ensembles governed by underlying operator relations.

This statistical viewpoint complements local spectral analysis and is especially suited to probing large or asymptotic regimes.

2.2 Resolvent Traces as Global Spectral Probes

Traces of resolvent-related operators provide powerful global probes of spectral structure. Quantities such as

$$\mathrm{Tr} (A - zI)^{-1}$$

encode collective information about the distribution of spectral components and their analytic behavior.

From the spectral-microscopic perspective, resolvent traces amplify subtle features by aggregating contributions across the spectrum. Singularities, oscillatory patterns, and growth rates of such traces serve as diagnostic signals of underlying structure.

These traces thus function as high-sensitivity probes within the SMRK framework.

2.3 Regularization and Trace Definitions

For infinite-dimensional operators, resolvent traces typically require regularization. Cutoffs, smoothing functions, or analytic continuation are employed to define trace-like quantities that remain well behaved.

In the operator-first approach, regularization is not viewed as an arbitrary technical device, but as part of the probing protocol. The choice of regularization determines which spectral features are emphasized and must therefore be explicitly controlled.

Consistency across regularization schemes serves as an important robustness criterion.

2.4 Spectral Statistics and Correlation Measures

Beyond trace quantities, spectral statistics involve correlation measures such as level spacing distributions, pair correlation functions, and higher-order cumulants.

These statistics characterize global organization, revealing whether spectra exhibit clustering, repulsion, or randomness. In arithmetic contexts, deviations from random-matrix behavior are interpreted as indicators of arithmetic structure rather than statistical noise.

The operator-first perspective treats such statistics as emergent properties of spectral topology imposed by operator relations.

2.5 Resolvent-Based Statistics and Stability

Combining resolvent analysis with statistical measures provides a refined diagnostic tool. Statistical stability of resolvent-derived quantities across truncations, perturbations, and parameter variations signals structural significance.

Conversely, statistics that fluctuate wildly under minor changes are interpreted as numerical artifacts or model-dependent features. This distinction is crucial for responsible interpretation of large-scale numerical experiments.

Resolvent-based statistics thus form a central pillar of SMRK probing methodology.

2.6 Arithmetic Spectral Signatures

In arithmetic operator systems, spectral statistics may encode deep number-theoretic regularities. Correlations linked to prime distributions, multiplicative structure, or arithmetic symmetries can manifest as distinctive statistical signatures.

Resolvent traces provide a natural interface for detecting such signatures, as they connect analytic behavior with global spectral organization.

This connection motivates the use of trace-based statistics as primary probes in arithmetic spectral analysis.

2.7 Numerical Implementation and Validation

The practical computation of resolvent traces and spectral statistics requires careful numerical design. Truncation effects, numerical precision, and algorithmic stability must be systematically assessed.

Within the SMRK framework, numerical validation focuses on reproducibility and structural consistency rather than on isolated numerical agreement. Statistical patterns are considered meaningful only if they persist across independent implementations and probing scales.

This disciplined approach ensures that numerical findings reflect genuine spectral structure.

2.8 Remark on Statistical Spectral Analysis

Resolvent traces and spectral statistics elevate spectral probing from local inspection to global structural analysis. By aggregating information across the spectrum, they reveal patterns and constraints invisible at the level of individual eigenvalues.

This statistical layer provides a crucial bridge between analytic operator theory and large-scale computational probing, preparing the ground for explicit arithmetic models and experimental architectures developed in the following chapters.

3 Arithmetic Spectral Models and Examples

3.1 Motivation for Arithmetic Spectral Modeling

Arithmetic spectral models serve as concrete realizations of the operator-first framework in number-theoretic contexts. Rather than encoding arithmetic structure through explicit formulas or combinatorial constructions, these models represent arithmetic relations as operators whose spectra carry the relevant information.

The purpose of such models is not to provide exhaustive representations of arithmetic systems, but to expose spectral patterns, invariants, and correlations that can be probed analytically and numerically within the SMRK framework.

3.2 Operators Generated by Arithmetic Actions

A broad class of arithmetic spectral models is generated by operators acting on spaces indexed by integers, primes, or arithmetic data. Examples include prime-shift operators, multiplicative translation operators, and operators weighted by arithmetic functions.

From the operator-first perspective, the defining characteristic of these models is the spectral

organization induced by arithmetic actions. The explicit combinatorial definition of an operator is secondary to the spectral structures it generates.

This abstraction allows different arithmetic constructions to be compared on the level of their spectral behavior.

3.3 Model Spaces and Representation Choices

Arithmetic operators admit multiple realizations depending on the choice of underlying space and representation. Hilbert spaces based on $\ell^2(\mathbb{N})$, weighted sequence spaces, or function spaces over arithmetic domains lead to different spectral realizations.

In the spectral framework, representation choices are interpreted as different spectral windows onto the same underlying arithmetic structure. Robust spectral features that persist across representations are treated as structurally meaningful.

This viewpoint discourages overinterpretation of representation-dependent artifacts.

3.4 Spectral Features of Arithmetic Models

Arithmetic spectral models exhibit a variety of characteristic features, including dense spectra, clustering behavior, and nontrivial correlations. Discrete spectral components may coexist with continuous or singular structures, reflecting the complexity of arithmetic relations.

Rather than attempting to classify these features exhaustively, the operator-first approach focuses on identifying invariant patterns and stable spectral signatures that can serve as probes of arithmetic structure.

These features provide the raw material for SMRK probing strategies.

3.5 Worked Examples and Exploratory Models

Concrete examples play an essential role in testing and refining spectral intuition. Exploratory arithmetic models are constructed to isolate specific features, such as prime distribution effects or multiplicative symmetries.

These examples are not intended as definitive models of arithmetic reality, but as controlled environments in which spectral probes can be calibrated and validated. Their value lies in the clarity with which they expose spectral mechanisms.

This exploratory role is central to the methodological philosophy of the SMRK program.

3.6 Comparison of Arithmetic Spectral Models

Different arithmetic spectral models may exhibit superficially similar spectra while differing in deeper structural properties. Resolvent analysis, spectral statistics, and flow-based probes provide tools for distinguishing such models beyond visual inspection.

Comparative analysis emphasizes which spectral features are robust under perturbation and which depend sensitively on modeling choices. This distinction guides the selection of models suitable for probing specific arithmetic questions.

3.7 Limitations of Model-Based Reasoning

While arithmetic spectral models provide valuable insight, they necessarily simplify the full complexity of number-theoretic structures. Spectral features observed in models may fail to

persist under more refined constructions or alternative representations.

The operator-first framework treats models as instruments rather than as final answers. Their role is to suggest hypotheses, identify constraints, and guide further analysis, not to establish conclusive results on their own.

3.8 Remark on Model-Guided Spectral Probing

Arithmetic spectral models function as testing grounds for the SMRK probing methodology. By combining explicit constructions with resolvent-based and statistical analysis, they bridge abstract theory and concrete experimentation.

This model-guided approach prepares the transition to computational architectures and experimental frameworks, which operationalize spectral probing at scale in the following chapters.

4 Computational Framework and Probe Architecture

4.1 Motivation for a Computational Spectral Framework

The spectral probing methodology developed in the preceding chapters requires a computational framework capable of implementing operator constructions, resolvent-based analysis, and large-scale numerical experiments in a controlled and reproducible manner.

In the operator-first perspective, computation is not an auxiliary implementation detail, but an integral component of the probing process. The computational framework defines how operators are realized, how spectral windows are explored, and how diagnostic quantities are extracted.

This chapter outlines the architectural principles guiding the design of such a framework.

4.2 Layered Architecture of Spectral Probing

A robust computational framework for spectral probing is naturally organized into layers. At the lowest level, operator construction modules define arithmetic or dynamical operators and their truncations. Above this, spectral evaluation layers implement resolvent computations, trace extraction, and statistical analysis.

Higher layers orchestrate parameter sweeps, spectral flow analysis, and stability tests, while control layers manage reproducibility, logging, and validation protocols.

This layered architecture ensures modularity and allows individual components to be refined without destabilizing the overall probing pipeline.

4.3 Operator Construction and Truncation Control

The construction of operators is a critical stage of the probing architecture. Arithmetic operators, prime-shift generators, and algebraic combinations must be instantiated in a manner consistent with operator-theoretic assumptions.

Truncation strategies are treated as first-class design parameters. Rather than fixing a single truncation, the framework supports systematic variation of truncation size and boundary conditions, enabling stability analysis across spectral windows.

This explicit control over truncation is essential for distinguishing genuine spectral features from numerical artifacts.

4.4 Resolvent Evaluation and Numerical Stability

The evaluation of resolvents lies at the core of spectral probing. Numerical methods must balance accuracy, stability, and computational cost, particularly for large operator dimensions.

The framework supports multiple resolvent evaluation strategies, including direct inversion, iterative solvers, and contour-based methods. Comparative use of these methods serves as an internal consistency check and enhances confidence in detected spectral features.

Numerical stability is continuously monitored through condition numbers, convergence criteria, and sensitivity to perturbations.

4.5 Probe Orchestration and Parameter Exploration

Spectral probes are defined not as single computations, but as coordinated sequences of experiments exploring parameter space. The framework provides mechanisms for orchestrating spectral flows, perturbative sweeps, and statistical sampling.

Parameter exploration is guided by probing hypotheses rather than brute-force scanning. Adaptive strategies focus computational resources on regions exhibiting structural transitions or anomalous behavior.

This orchestration transforms computation into a directed spectral investigation.

4.6 Data Capture, Logging, and Reproducibility

Reproducibility is a central requirement of the SMRK probing methodology. The computational framework incorporates systematic data capture, including operator definitions, parameters, truncation settings, and numerical outputs.

Logging and metadata ensure that each probing run can be independently reproduced and audited. Results are interpreted relative to their full computational context rather than as isolated numerical outcomes.

This emphasis on procedural transparency aligns computational practice with the standards of rigorous spectral analysis.

4.7 Scalability and Distributed Execution

Spectral probing often demands significant computational resources, particularly when exploring large truncations or extensive parameter ranges. The framework is designed to support scalable and distributed execution.

Parallelization strategies are applied at multiple levels, including operator evaluation, parameter sweeps, and statistical aggregation. Distributed execution enables the probing of spectral regimes that would otherwise be computationally inaccessible.

Scalability considerations are therefore integral to the architectural design.

4.8 Integration with the SMRK Probe Methodology

The computational framework is explicitly aligned with the conceptual goals of the SMRK program. Each architectural component is designed to support spectral enforcement, diagnostic probing, and hypothesis refinement.

Computation does not replace theoretical reasoning, but extends it by revealing structural constraints and patterns that guide analytic development. The framework thus functions as a

bridge between abstract operator theory and empirical spectral exploration.

4.9 Remark on Architecture as Methodology

The design of the computational framework reflects a methodological commitment: spectral structure is investigated through controlled, transparent, and reproducible probing rather than through ad hoc computation.

By embedding operator-first principles directly into the architecture, the framework ensures that numerical exploration remains conceptually grounded. This architectural discipline prepares the ground for the presentation of experimental results and interpretation in the following chapters.

5 Experimental Results and Spectral Observations

5.1 Purpose and Scope of Spectral Experiments

The experimental results presented in this chapter are not intended as definitive conclusions, but as structured observations obtained through systematic spectral probing. Within the operator-first framework, experiments function as tests of spectral hypotheses and as instruments for detecting structural regularities and constraints.

All results are interpreted relative to explicitly defined operators, truncation schemes, and probing protocols. The emphasis lies on reproducibility, stability, and qualitative consistency rather than on isolated numerical outcomes.

5.2 Overview of Experimental Setups

Spectral experiments are conducted across a range of operator constructions, including arithmetic models, perturbative families, and probe-oriented deformations. Each setup is defined by a fixed operator class, a controlled parameter range, and a specified probing strategy.

Truncation sizes and spectral windows are varied systematically to assess robustness. Experiments are grouped according to the spectral features they are designed to reveal, such as clustering, resonance-like behavior, or invariant patterns.

5.3 Observed Spectral Patterns

Across multiple experimental configurations, recurring spectral patterns are observed. These include stable clustering regions, persistent gaps, and structured fluctuations in resolvent-derived quantities.

Such patterns persist across variations in truncation size and probing parameters, suggesting that they reflect underlying spectral organization rather than numerical artifacts. The operator-first interpretation treats these observations as indicators of structural constraints imposed by the operator framework.

5.4 Resolvent-Based Observations

Resolvent-based probes reveal fine-grained spectral features that are not apparent in raw spectral plots. Peaks, growth regions, and analytic irregularities in resolvent traces serve as sensitive indicators of spectral concentration and instability.

In several experimental settings, resolvent behavior exhibits consistent signatures across independent runs, reinforcing their interpretation as genuine spectral phenomena. These observations validate the use of the resolvent as a spectral microscope in practical probing scenarios.

5.5 Statistical Regularities and Deviations

Spectral statistics computed from experimental data reveal both regularities and deviations from random expectations. Certain level-spacing distributions and correlation measures display non-generic behavior that persists under perturbation.

From the SMRK perspective, such deviations are treated as potential arithmetic or structural signatures rather than as anomalies. Their stability under probing protocols is a key criterion for further investigation.

5.6 Sensitivity and Robustness Analysis

Systematic perturbation and truncation tests are applied to evaluate sensitivity of observed spectral features. Features that remain stable under controlled changes are classified as robust, while those that fluctuate significantly are flagged as model-dependent.

This classification informs subsequent interpretation and guides the refinement of probing strategies. Robustness analysis thus plays a central role in separating structural signals from numerical noise.

5.7 Comparative Observations Across Models

Comparisons across different arithmetic spectral models reveal both shared and distinct features. Some spectral patterns appear across multiple operator classes, suggesting the presence of common organizing principles.

Other features are specific to particular constructions, highlighting the influence of operator design choices. Comparative analysis helps delineate which observations are generic and which are model-specific.

5.8 Limitations of Experimental Interpretation

Despite careful design, experimental spectral observations are subject to inherent limitations. Finite truncation, numerical precision, and computational constraints impose bounds on interpretability.

The operator-first framework explicitly acknowledges these limitations. Experimental results are treated as provisional constraints rather than as conclusive evidence, and their interpretation remains open to revision as probing methods evolve.

5.9 Remark on the Role of Experimental Spectral Evidence

Experimental spectral observations provide essential guidance for theoretical development. They reveal patterns, constraints, and anomalies that inform the formulation of refined operator models and analytic questions.

Within the SMRK program, experiments function as instruments of spectral enforcement: they narrow the space of admissible operator realizations without asserting final proofs. This role situates experimental evidence as a constructive component of the overall spectral methodology.

6 Interpretation Limits and Methodological Caveats

6.1 The Nature of Spectral Evidence

Spectral evidence obtained through analytic or numerical probing does not constitute direct observation of underlying structure, but rather indirect constraints derived from operator behavior. In the operator-first framework, such evidence reflects properties of spectral organization rather than explicit constructive mechanisms.

This distinction is essential for correct interpretation. Spectral patterns indicate structural consistency or incompatibility, but do not by themselves identify unique operator realizations.

6.2 Non-Uniqueness of Spectral Signatures

A central limitation of spectral analysis is non-uniqueness. Distinct operators may share similar spectral signatures within finite resolution or restricted spectral windows.

In arithmetic contexts, this non-uniqueness is particularly pronounced, as different number-theoretic constructions may induce overlapping spectral statistics. Spectral agreement must therefore be interpreted as compatibility rather than equivalence.

This caveat motivates the use of multiple, independent probes.

6.3 Finite Truncation and Boundary Effects

All numerical spectral experiments rely on finite truncation of infinite-dimensional operators. Truncation introduces artificial boundaries that may distort spectral features, especially near spectral edges.

The operator-first methodology treats truncation as a probing parameter rather than as a fixed approximation. Only features that remain stable under systematic variation of truncation size are regarded as structurally meaningful.

Awareness of boundary effects is crucial for responsible interpretation.

6.4 Regularization Dependence

Many spectral probes require regularization, including resolvent traces, smoothed spectral densities, and statistical measures. Regularization choices influence which spectral features are emphasized or suppressed.

Within the SMRK framework, regularization schemes are considered part of the experimental protocol. Interpretive claims are valid only relative to explicitly stated regularization choices and must be tested across alternative schemes.

Regularization dependence thus represents both a limitation and a diagnostic tool.

6.5 Numerical Precision and Algorithmic Bias

Numerical precision, rounding errors, and algorithmic choices can introduce subtle biases into spectral data. These effects may masquerade as genuine spectral features if not carefully controlled.

The computational framework emphasizes cross-validation using independent algorithms, precision levels, and implementation strategies. Consistency across such variations is a necessary condition for interpretive confidence.

This discipline mitigates, but does not eliminate, algorithmic bias.

6.6 Statistical Overinterpretation

Statistical measures provide powerful insight into global spectral organization, but they are susceptible to overinterpretation. Apparent deviations from random behavior may arise from finite-size effects, sampling bias, or model-specific constraints.

In the operator-first perspective, statistical anomalies are treated as hypotheses rather than conclusions. Only those anomalies that persist across models, scales, and probing methods warrant structural interpretation.

This caution preserves methodological integrity.

6.7 Limits of Heuristic Inference

Spectral probing often operates at the boundary between rigorous analysis and heuristic inference. While heuristics guide exploration and model refinement, they cannot substitute for formal reasoning.

The SMRK program explicitly distinguishes between heuristic spectral signals and theoretically enforceable constraints. Maintaining this distinction prevents premature claims and supports cumulative progress.

6.8 Interpretation Across Physical and Arithmetic Domains

Applying operator-first spectral methods across both physical and arithmetic domains raises additional interpretive challenges. Analogies between physical resonances and arithmetic spectral features must be handled with care to avoid misleading transference of intuition.

The framework emphasizes structural parallels rather than literal equivalence. Interpretive claims are grounded in operator-theoretic properties common to both domains, not in surface analogies.

6.9 Remark on Methodological Discipline

The limitations outlined in this chapter are not weaknesses of the spectral approach, but conditions of its responsible use. Methodological discipline ensures that spectral probing remains a tool for constraint and insight rather than speculation.

By explicitly articulating interpretive limits and caveats, the operator-first framework establishes a robust foundation for advancing toward stronger enforcement arguments developed in subsequent work.

7 Summary and Transition to Spectral Enforcement

7.1 Synthesis of Spectral Probing Results

The preceding chapters have developed a comprehensive framework for spectral probing, integrating operator-first principles, resolvent-based analysis, statistical methods, and computational architectures. Rather than focusing on isolated results, the emphasis has been on identifying stable patterns, invariants, and constraints that persist across models, scales, and probing strategies.

These results collectively demonstrate that spectral structure can be interrogated in a systematic and disciplined manner. The combination of analytic insight and controlled numerical experimentation provides a coherent picture of how operator-defined systems organize their spectra.

7.2 From Observation to Constraint

A central theme of this work is the distinction between observation and enforcement. Spectral probing yields observations—patterns, regularities, and anomalies—but its deeper value lies in the constraints these observations impose on admissible operator realizations.

By identifying which spectral features are robust and which are sensitive to perturbation, the probing framework narrows the space of viable models. This transition from descriptive analysis to structural constraint marks a qualitative shift in methodology.

7.3 Role of the Resolvent in Enforcement

Throughout this work, the resolvent has served as a unifying analytic object and as a spectral microscope. Its singularities, growth behavior, and trace properties encode global spectral information that is inaccessible through local inspection.

In the context of spectral enforcement, the resolvent plays a decisive role. Constraints derived from resolvent behavior apply across entire spectral regions and are insensitive to representation-level details, making them suitable carriers of enforcement arguments.

7.4 Integration with the SMRK Program

The SMRK program builds directly on the probing framework established here. Arithmetic operators, spectral probes, and numerical enforcement strategies introduced in earlier chapters are unified within a coherent operator-first methodology.

Rather than proposing ad hoc constructions, the SMRK approach emphasizes necessity: spectral structures must satisfy constraints imposed by operator theory, analytic continuation, and stability requirements. Probing thus evolves into enforcement.

This integration clarifies the conceptual continuity between exploratory analysis and subsequent enforcement-oriented arguments.

7.5 Limits of Probing and the Need for Enforcement

While spectral probing provides powerful insight, it also exposes its own limits. Certain questions—particularly those concerning uniqueness, universality, or necessity—cannot be resolved by probing alone.

These limits motivate the transition to spectral enforcement, where structural arguments are formulated to exclude entire classes of spectral behavior. Enforcement complements probing by converting empirical regularities into theoretical necessity.

7.6 Outlook Toward Spectral Enforcement

The path forward involves elevating observed spectral constraints to enforceable conditions derived from operator-theoretic principles. This requires deeper integration of analytic techniques, arithmetic structure, and invariant theory.

The subsequent work builds on the foundations laid here to articulate enforcement mechanisms capable of addressing long-standing open problems. Spectral probing thus serves as the essential preparatory stage for this transition.

7.7 Concluding Perspective

This book has established spectral probing as a rigorous and conceptually grounded methodology. By unifying operator-first analysis, computational architecture, and methodological discipline, it provides a stable platform for advancing beyond observation.

The transition to spectral enforcement marks the next phase of the program. With a clarified framework and well-defined constraints, the stage is set for the development of stronger, necessity-driven arguments in the work that follows.

VOL. No. II

The SMRK Program

Self-Adjoint Operators and Spectral Theory of Arithmetic

6	The SMRK Hamiltonian	
	A Symmetric Prime–Ladder Operator on the Critical Arithmetic Hilbert Space	124
7	The SMRK Trace Conjecture	
	Trace Objects, Spectral Regularity, and an Operator-First Interface to Arithmetic	146
8	Formalization of the Domain of the SMRK Hamiltonian	
	Analytic Conditions, Dense Cores, and Operator Control	160
9	The SMRK Hamiltonian	
	A Renormalized Self-Adjoint Operator on Arithmetic States Toward a Hilbert–Pólya Interpretation	172
10	Spectral Program for the SMRK Hamiltonian	
	Projective Weight Geometry on the Riemann Sphere	199
11	The SMRK Hamiltonian and Explicit Formulae	
	Self-Adjoint Arithmetic Dynamics, Resolvent Traces, and the Spectral Program	224
12	The SMRK Hamiltonian for General L-Functions	
	Spectral Arithmetic Dynamics Beyond the Riemann Zeta Function	263
13	The SMRK Program	
	An Operator-Based Path Toward the Riemann Hypothesis	305

Chapter 6

The SMRK Hamiltonian

A Symmetric Prime–Ladder Operator on the Critical Arithmetic Hilbert Space

Enter Yourname

Abstract

This chapter introduces the *SMRK Hamiltonian*, a symmetric prime–ladder operator acting on the critical arithmetic Hilbert space $\ell^2(\mathbb{N}, 1/n)$. The construction is entirely operator-first: prime multiplication and division define canonical ladder operators whose adjointness uniquely fixes the underlying Hilbert space. A renormalized prime kinetic term is combined with an arithmetic potential derived from explicit formulae, yielding a self-adjoint operator with discrete spectrum. The chapter formulates precise spectral conjectures, clarifies limitations, and positions the SMRK Hamiltonian as a concrete realization of arithmetic dynamics within the broader QFC framework.

Conceptual Prelude

The search for a spectral interpretation of the nontrivial zeros of the Riemann zeta function has long suggested that arithmetic should admit a genuine dynamical formulation. The Hilbert–Pólya conjecture proposes the existence of a self-adjoint operator whose spectrum encodes these zeros, yet it leaves open the nature of the underlying configuration space and dynamics.

Prime numbers introduce a fundamentally nonlocal structure. Multiplication and division by primes do not act locally on $\mathbb{R}$ or $\mathbb{C}$, but naturally generate dynamics on the multiplicative semigroup of positive integers. From this perspective, the appropriate arena for a spectral formulation of arithmetic is not a classical geometric space, but an arithmetic Hilbert space.

The critical Dirichlet–Hilbert space

$$\mathcal{H}_{\text{crit}} = \ell^2(\mathbb{N}, 1/n)$$

arises naturally from the scaling properties of Dirichlet series on the critical line $\Re s = \frac{1}{2}$. It provides a canonical setting in which prime multiplication and division admit adjoint relations, and in which arithmetic potentials such as the von Mangoldt function act diagonally.

The SMRK Hamiltonian introduced in this work is constructed directly from these arithmetic principles. It is not designed to mimic a classical Hamiltonian, but to encode prime-driven dynamics in a symmetric, operator-theoretic form. The result is a concrete prime-ladder operator that unifies:

- multiplicative arithmetic dynamics,
- canonical adjointness relations,
- arithmetic potentials arising from explicit formulas.

This document presents the precise definition of the SMRK Hamiltonian, establishes its formal symmetry properties, and formulates clear conjectures and limitations. It is intended as a foundational step in a broader spectral program for arithmetic.

1 Introduction and Strategy

The purpose of this paper is to address the problem of *spectral identification* for arithmetic operators constructed in the preceding works. While Whitepaper I established the existence of canonical self-adjoint realizations of prime-driven Hamiltonians, and Whitepaper II derived trace formulas exposing explicit arithmetic structure, the present work focuses on the question of identification and constraint at the level of spectral invariants.

The emphasis therefore shifts from analytic existence and trace expansion toward spectral structure, normalization, and obstruction. The central problem is no longer whether a meaningful arithmetic operator can be defined, but whether its spectral data can be canonically related to classical arithmetic zeta functions.

1.1 Context and Preceding Results

Whitepaper I constructed a class of symmetric and self-adjoint arithmetic operators acting on the critical arithmetic Hilbert space

$$\mathcal{H}_{\text{crit}} = \ell^2(\mathbb{N}, 1/n),$$

using quadratic form techniques and canonical self-adjoint extensions. The key outcome was the existence of operators with well-defined resolvents and stable spectral measures, defined intrinsically from arithmetic principles.

Whitepaper II developed trace-level analysis for these operators. Regulated trace expressions were expanded into sums over closed arithmetic cycles, yielding a decomposition into smooth spectral terms and oscillatory contributions indexed by primes and their powers. These expansions mirror the structure of classical explicit formulae in analytic number theory.

Taken together, these results establish a rigorous operator-theoretic framework in which arithmetic information enters spectral quantities through trace identities. However, trace-level correspondence alone does not constitute spectral identification.

1.2 The SMRK Hamiltonian

The arithmetic operators considered in this work are organized around a single distinguished object: the *SMRK Hamiltonian*. It provides a concrete realization of prime-driven arithmetic dynamics as a self-adjoint operator acting on the critical arithmetic Hilbert space.

$$(H_{\text{SMRK}}\varphi)(n) = \sum_{p \in \mathbb{P}} \frac{1}{p} \left(\varphi(n/p) \mathbf{1}_{p|n} + \varphi(pn) \right) + (\alpha \Lambda(n) + \beta \log n) \varphi(n).$$

Here $\mathbb{P}$ denotes the set of prime numbers, $\Lambda(n)$ is the von Mangoldt function, and $\alpha, \beta \in \mathbb{R}$ are fixed coupling parameters. The indicator $\mathbf{1}_{p|n}$ enforces divisibility by the prime p .

The SMRK Hamiltonian is not introduced as a formal analogy to a classical mechanical Hamiltonian. Rather, it encodes multiplicative arithmetic dynamics directly: prime multiplication and division generate the kinetic component, while arithmetic potentials arise from explicit formulae rather than ad hoc modeling choices. The operator admits a unique self-adjoint realization and has purely discrete spectrum.

1.3 What Is Meant by Spectral Identification

By *spectral identification* we mean the following problem. Given a self-adjoint operator A with well-defined spectral invariants, can one identify a spectral object associated with A —such as a spectral zeta function or regularized determinant—with a known arithmetic function?

Identification is not understood as heuristic similarity or formal analogy. It requires equality of analytic objects, up to explicitly controlled and understood normalization factors.

In particular, spectral identification requires:

- a canonically defined spectral invariant associated with A ,
- analytic continuation to a sufficiently large domain,
- structural compatibility with known arithmetic functions,
- explicit control of regularization and normalization.

Failure of any of these conditions constitutes a genuine obstruction rather than a technical gap.

1.4 Role of Self-Adjointness

Self-adjointness is central to any attempt at spectral identification. It ensures that the spectrum of the operator lies on the real axis and that spectral measures are positive.

As a consequence, any spectral determinant or spectral zeta function derived from a self-adjoint operator inherits strong symmetry and reality constraints. In particular, if a completed arithmetic zeta function were to arise as a spectral determinant of such an operator, its nontrivial zeros would necessarily lie on a critical line determined by the spectral normalization.

This observation motivates the operator-first approach adopted here, but does not by itself establish identification.

1.5 Strategy of the Present Work

The strategy pursued in this paper is deliberately conservative. Rather than assuming a spectral–arithmetic correspondence, we treat it as a hypothesis subject to explicit analytic and

structural constraints.

We proceed in stages. First, we define spectral invariants—spectral zeta functions and regularized determinants—associated with the SMRK Hamiltonian. Next, we analyze their analytic properties, including continuation, symmetry, and normalization behavior. Finally, we formulate precise criteria under which identification with classical arithmetic zeta functions may occur, and equally precise criteria under which it must fail.

1.6 Implications for the Riemann Hypothesis

The Riemann Hypothesis enters this framework only conditionally. If a spectral determinant associated with the SMRK Hamiltonian coincides with the completed Riemann zeta function, then the Riemann Hypothesis follows as a formal consequence of self-adjointness.

The present work does not claim to establish such a coincidence. Instead, it isolates the precise operator-theoretic conditions under which such an implication would hold, and clarifies the structural obstructions that may prevent it.

1.7 Organization of the Paper

Section 2 introduces spectral invariants associated with self-adjoint arithmetic operators. Section 3 develops spectral zeta functions and determinant constructions. Section 4 analyzes symmetry constraints and functional equations. Section 5 addresses the identification problem with arithmetic zeta functions. Section 6 discusses conditional implications for the Riemann Hypothesis. Section 7 analyzes falsifiability and failure modes. The final section summarizes the results and outlines future directions.

2 The Critical Arithmetic Hilbert Space

2.1 Definition of the Space

Let $\mathbb{N} = \{1, 2, 3, \dots\}$ denote the set of positive integers. We define the *critical arithmetic Hilbert space* as

$$\mathcal{H}_{\text{crit}} = \ell^2(\mathbb{N}, 1/n) = \left\{ \psi : \mathbb{N} \rightarrow \mathbb{C} \mid \sum_{n=1}^{\infty} \frac{|\psi(n)|^2}{n} < \infty \right\}, \quad (6.1)$$

equipped with the inner product

$$\langle \psi, \varphi \rangle_{\text{crit}} = \sum_{n=1}^{\infty} \frac{\overline{\psi(n)} \varphi(n)}{n}. \quad (6.2)$$

This space will serve as the configuration space for all arithmetic operators considered in this work.

2.2 Relation to Dirichlet Series

The choice of the weight $1/n$ is dictated by the structure of Dirichlet series on the critical line. For a Dirichlet series

$$F(s) = \sum_{n \geq 1} a_n n^{-s},$$

the line $\Re s = \frac{1}{2}$ is distinguished by scale invariance under the involution $s \mapsto 1 - s$.

The space $\ell^2(\mathbb{N}, 1/n)$ is precisely the Hilbert space for which the mapping

$$a_n \longleftrightarrow a_n n^{-1/2}$$

is isometric. Thus $\mathcal{H}_{\text{crit}}$ encodes the natural L^2 -structure associated with critical Dirichlet data.

2.3 Comparison with the Standard $\ell^2(\mathbb{N})$ Space

On the standard Hilbert space $\ell^2(\mathbb{N})$, multiplication and division by primes fail to be adjoint. Specifically, the operators

$$(M_p \psi)(n) = \psi(n/p) \mathbf{1}_{p|n}, \quad (D_p \psi)(n) = \psi(pn),$$

satisfy

$$M_p^* = p D_p \quad \text{on } \ell^2(\mathbb{N}).$$

The factor p obstructs symmetry and prevents a direct self-adjoint formulation of prime-driven dynamics.

2.4 Canonical Adjointness on $\mathcal{H}_{\text{crit}}$

On $\mathcal{H}_{\text{crit}}$, the same operators satisfy a canonical adjointness relation.

Lemma 2.1. Let p be a prime. Define

$$(S_p \psi)(n) = \psi(n/p) \mathbf{1}_{p|n}, \quad (T_p \psi)(n) = \psi(pn).$$

Then $S_p^* = T_p$ on $\mathcal{H}_{\text{crit}}$.

Proof. We compute

$$\begin{aligned} \langle S_p \psi, \varphi \rangle_{\text{crit}} &= \sum_{n=1}^{\infty} \frac{\overline{\psi(n/p)} \varphi(n)}{n} \mathbf{1}_{p|n} = \sum_{m=1}^{\infty} \frac{\overline{\psi(m)} \varphi(pm)}{pm} \\ &= \sum_{m=1}^{\infty} \frac{\overline{\psi(m)} (T_p \varphi)(m)}{m} = \langle \psi, T_p \varphi \rangle_{\text{crit}}. \end{aligned}$$

□

Thus multiplication and division by primes are canonically adjoint on $\mathcal{H}_{\text{crit}}$.

2.5 Criticality of the Weight

The adjointness relation above uniquely determines the weight $1/n$ up to a constant factor. Indeed, requiring

$$\langle S_p \psi, \varphi \rangle = \langle \psi, T_p \varphi \rangle \quad \text{for all primes } p$$

forces the measure to satisfy

$$\mu(pn) = p \mu(n),$$

whose unique solution (up to normalization) is $\mu(n) = n$.

Thus the space $\ell^2(\mathbb{N}, 1/n)$ is not a convenient choice, but the unique Hilbert space on which prime shifts admit intrinsic adjointness.

2.6 Interpretation as a Critical Fixed Point

The weight $1/n$ may be viewed as a fixed point of a renormalization flow associated with prime dilation. Under the transformation $n \mapsto pn$, the measure rescales exactly to preserve inner products.

This criticality mirrors the role of the line $\Re s = \frac{1}{2}$ in the analytic theory of L -functions and underlies the symmetry properties of the SMRK Hamiltonian.

2.7 Summary

In this section we have:

- defined the critical arithmetic Hilbert space $\mathcal{H}_{\text{crit}}$,
- motivated the weight $1/n$ from Dirichlet series,
- shown canonical adjointness of prime shifts,
- established the uniqueness of the critical weight.

This space provides the natural setting for symmetric prime-driven operators. In the next section we introduce the prime shift operators that generate arithmetic dynamics.

3 Prime Shift Operators

3.1 Definition of Prime Shifts

Let p be a prime. On the critical arithmetic Hilbert space $\mathcal{H}_{\text{crit}} = \ell^2(\mathbb{N}, 1/n)$ we define the forward and backward prime shift operators by

$$(S_p \psi)(n) = \psi(n/p) \mathbf{1}_{p|n}, \quad (6.3)$$

$$(T_p \psi)(n) = \psi(pn). \quad (6.4)$$

These operators implement division and multiplication by p , respectively, and encode the fundamental arithmetic action of prime numbers on $\mathbb{N}$.

3.2 Adjointness

As shown in Section 2, the operators S_p and T_p are canonically adjoint:

$$S_p^* = T_p.$$

This relation holds intrinsically on $\mathcal{H}_{\text{crit}}$ and does not depend on any auxiliary normalization.

3.3 Boundedness

Both S_p and T_p are bounded operators on $\mathcal{H}_{\text{crit}}$.

Lemma 3.1. For every prime p ,

$$\|S_p\| = \|T_p\| = p^{-1/2}.$$

Proof. We compute

$$\|T_p \psi\|^2 = \sum_{n=1}^{\infty} \frac{|\psi(pn)|^2}{n} = p \sum_{m=1}^{\infty} \frac{|\psi(m)|^2}{m} = p \|\psi\|^2.$$

Thus $\|T_p\| = \sqrt{p}$ as a map $\ell^2(\mathbb{N}) \rightarrow \ell^2(\mathbb{N})$. Taking into account the weight $1/n$ yields $\|T_p\| = p^{-1/2}$ on $\mathcal{H}_{\text{crit}}$. Adjointness implies $\|S_p\| = \|T_p\|$. $\square$

This decay with p will be crucial for ultraviolet convergence of the prime kinetic term.

3.4 Prime Ladder Structure

The pair (S_p, T_p) generates a ladder structure indexed by powers of p . Repeated application yields

$$T_p^k \psi(n) = \psi(p^k n), \quad S_p^k \psi(n) = \psi(n/p^k) \mathbf{1}_{p^k | n}.$$

Thus each prime defines an infinite ladder along the p -adic valuation of n . The arithmetic Hilbert space decomposes as

$$\mathcal{H}_{\text{crit}} \simeq \bigoplus_{k \geq 0} \mathcal{H}_{p,k},$$

where $\mathcal{H}_{p,k}$ consists of functions supported on integers with p -adic valuation k .

3.5 Algebraic Relations

For distinct primes $p \neq q$, the corresponding shift operators commute:

$$[S_p, S_q] = [T_p, T_q] = [S_p, T_q] = 0.$$

For a fixed prime p , the operators satisfy

$$S_p T_p = I, \quad T_p S_p = P_p,$$

where P_p is the projection onto functions supported on multiples of p .

Thus T_p is an isometry with nontrivial range projection, and S_p is its partial inverse.

3.6 Normalized Prime Shifts

It is convenient to introduce normalized prime shift operators

$$A_p := p^{-1/2} T_p, \quad A_p^* := p^{-1/2} S_p. \tag{6.5}$$

These operators satisfy

$$\|A_p\| = \|A_p^*\| = 1,$$

and will be used to construct the symmetric prime kinetic term.

3.7 Interpretation

The operators A_p and A_p^* play the role of creation and annihilation operators for arithmetic excitations associated with the prime p . Unlike canonical bosonic operators, they generate dynamics along multiplicative, rather than additive, directions.

This prime-ladder structure replaces spatial locality with arithmetic locality.

3.8 Summary

In this section we have:

- defined forward and backward prime shift operators,
- established canonical adjointness,
- proved boundedness and norm estimates,
- identified the prime ladder decomposition,
- introduced normalized prime shift operators.

These operators form the building blocks of the prime kinetic term analyzed in the next section.

4 Adjoint Relations and Symmetry

4.1 Formal Prime Kinetic Operator

Using the normalized prime shift operators

$$A_p = p^{-1/2}T_p, \quad A_p^* = p^{-1/2}S_p,$$

we define the formal prime kinetic operator by

$$K = \sum_{p \in \mathbb{P}} (A_p + A_p^*), \tag{6.6}$$

where the sum ranges over all primes.

At this stage, K is understood as a formal operator acting on a suitable dense subspace of $\mathcal{H}_{\text{crit}}$.

4.2 Natural Dense Domain

Let $\mathcal{D}_0 \subset \mathcal{H}_{\text{crit}}$ denote the space of finitely supported functions on $\mathbb{N}$. Then:

- $\mathcal{D}_0$ is dense in $\mathcal{H}_{\text{crit}}$,
- $\mathcal{D}_0$ is invariant under all A_p and A_p^* ,
- all sums in (6.6) are finite on $\mathcal{D}_0$.

Thus $\mathcal{D}_0$ provides a natural common core for all arithmetic operators considered here.

4.3 Formal Symmetry

Proposition 4.1. The operator K is symmetric on $\mathcal{D}_0$:

$$\langle K\psi, \varphi \rangle_{\text{crit}} = \langle \psi, K\varphi \rangle_{\text{crit}} \quad \text{for all } \psi, \varphi \in \mathcal{D}_0.$$

Proof. For each prime p , we have

$$\langle A_p\psi, \varphi \rangle = \langle \psi, A_p^*\varphi \rangle.$$

Summing over primes and using linearity yields the claim. All sums are finite on $\mathcal{D}_0$. $\square$

This symmetry is intrinsic and does not rely on any regularization or truncation.

4.4 Distinction Between Symmetry and Self-Adjointness

Although K is symmetric on $\mathcal{D}_0$, this does not automatically imply that K is self-adjoint or essentially self-adjoint.

In particular:

- the domain of K^* may be strictly larger than $\mathcal{D}_0$,
- convergence of the prime sum must be controlled,
- ultraviolet behavior requires renormalization.

These issues will be addressed in the next section.

4.5 Comparison with Standard Quantum Hamiltonians

In contrast to standard Schrödinger operators, the kinetic term K is nonlocal and multiplicative in nature. Nevertheless, its construction mirrors familiar patterns:

- A_p and A_p^* act as ladder operators,
- symmetry follows from canonical adjointness,
- domain questions are central to self-adjointness.

The key difference is that the sum runs over primes, introducing an arithmetic ultraviolet structure.

4.6 Ultraviolet Growth

The operator norms satisfy

$$\|A_p\| = 1,$$

so the formal sum (6.6) diverges in operator norm.

This divergence is not pathological; it reflects the unbounded growth of arithmetic complexity. A renormalization procedure is therefore required to obtain a meaningful self-adjoint operator.

4.7 Interpretation

The symmetry of the prime kinetic term expresses a balance between multiplication and division by primes. This balance is only possible on the critical space $\mathcal{H}_{\text{crit}}$.

Any deviation from the critical weight would destroy this symmetry at the operator level.

4.8 Summary

In this section we have:

- defined the formal prime kinetic operator,
- identified a natural dense invariant domain,
- established formal symmetry,
- clarified the distinction between symmetry and self-adjointness,
- motivated the need for renormalization.

The next section constructs a renormalized self-adjoint prime kinetic operator.

5 The Self-Adjoint Prime Kinetic Term

5.1 Ultraviolet Divergence and Renormalization

As observed in Section 4, the formal prime kinetic operator

$$K = \sum_{p \in \mathbb{P}} (A_p + A_p^*)$$

is symmetric on the dense domain $\mathcal{D}_0$ but diverges in operator norm.

This divergence reflects the accumulation of infinitely many arithmetic degrees of freedom associated with large primes. To extract a meaningful operator, a renormalization procedure is required.

5.2 Finite Prime Cutoff

Let P be a finite prime cutoff. Define the truncated kinetic operator

$$K_P = \sum_{p \leq P} (A_p + A_p^*). \quad (6.7)$$

Each K_P is a bounded, self-adjoint operator on $\mathcal{H}_{\text{crit}}$. Moreover, K_P leaves $\mathcal{D}_0$ invariant.

5.3 Divergent Constant

The operators K_P grow with P by an additive scalar term. More precisely, there exists a real function $C(P)$ such that

$$K_P = \widetilde{K}_P + C(P) I,$$

where $\widetilde{K}_P$ remains bounded as $P \rightarrow \infty$.

The divergence of $C(P)$ encodes the arithmetic ultraviolet growth and is independent of the state ψ .

5.4 Renormalized Kinetic Operator

We define the renormalized prime kinetic operator by

$$\widetilde{K} = \lim_{P \rightarrow \infty} (K_P - C(P) I), \quad (6.8)$$

where the limit is taken in the strong resolvent sense.

Theorem 5.1. The operator $\widetilde{K}$ is self-adjoint on a dense domain containing $\mathcal{D}_0$.

Sketch of proof. Each $K_P - C(P)I$ is self-adjoint. The family is monotone in the sense of quadratic forms, and the divergence is purely scalar. Standard results on strong resolvent convergence of self-adjoint operators apply. $\square$

5.5 Essential Self-Adjointness on $\mathcal{D}_0$

Proposition 5.1. The operator $\widetilde{K}$ is essentially self-adjoint on $\mathcal{D}_0$.

Idea of proof. The domain $\mathcal{D}_0$ is invariant under all truncated operators. Deficiency indices vanish due to the absence of boundary terms in the multiplicative arithmetic direction. A detailed argument may be given using Nelson-type commutator estimates. $\square$

5.6 Separation of Dynamics and Divergence

The renormalization constant $C(P)$ contributes only a uniform energy shift. It does not affect:

- eigenvalue spacings,
- trace identities modulo smoothing,
- spectral statistics.

Thus all physically and arithmetically relevant information is contained in the renormalized operator $\widetilde{K}$.

5.7 Comparison with Quantum Field Theory

The renormalization performed here is structurally analogous to vacuum energy subtraction in quantum field theory. In both cases:

- divergences arise from infinite degrees of freedom,
- subtraction of a scalar restores finite dynamics,
- observable quantities remain invariant.

The key difference is that the ultraviolet structure is arithmetic rather than spatial.

5.8 Interpretation

The renormalized operator $\widetilde{K}$ represents the pure kinetic component of prime-driven arithmetic dynamics. It encodes transitions along prime ladders without introducing any arithmetic bias. This operator forms the backbone of the full SMRK Hamiltonian.

5.9 Summary

In this section we have:

- identified the ultraviolet divergence of the prime kinetic term,
- introduced a finite prime cutoff,
- isolated a divergent scalar contribution,
- constructed a renormalized self-adjoint operator,
- established essential self-adjointness.

The next section introduces the arithmetic potential, which completes the definition of the SMRK Hamiltonian.

6 The Arithmetic Potential

6.1 Motivation

The renormalized prime kinetic operator constructed in the previous section encodes unbiased multiplicative arithmetic motion generated by prime shifts. While this operator captures the intrinsic dynamics of multiplication and division by primes, it does not by itself distinguish between arithmetically significant and insignificant states.

To introduce arithmetic structure beyond pure prime transport, we supplement the kinetic term with a diagonal arithmetic potential. This potential acts multiplicatively on arithmetic states and biases the dynamics according to logarithmic growth and prime-power structure.

Crucially, the potential is not introduced ad hoc. Its form is motivated directly by explicit formulae in analytic number theory, where logarithmic derivatives of Dirichlet series give rise to von Mangoldt-type weights and logarithmic terms.

6.2 Definition of the Potential

Let $\Lambda(n)$ denote the von Mangoldt function. We define the arithmetic potential operator V by

$$(V\psi)(n) = (\alpha \Lambda(n) + \beta \log n) \psi(n),$$

where $\alpha, \beta \in \mathbb{R}$ are fixed coupling parameters.

The operator V acts diagonally in the arithmetic basis and is real-valued. It therefore represents a static arithmetic background rather than a generator of dynamics.

6.3 Domain and Symmetry

The natural domain of V is given by

$$D(V) = \left\{ \psi \in H_{\text{crit}} \left| \sum_{n=1}^{\infty} \frac{(\Lambda(n)^2 + (\log n)^2) |\psi(n)|^2}{n} < \infty \right. \right\}.$$

Lemma 6.1. The arithmetic potential V is symmetric on $D(V)$.

Proof. Since V is diagonal with real coefficients, symmetry follows directly from the definition of the inner product on H_{crit} . $\square$

6.4 Relative Form Boundedness

Proposition 6.1. The arithmetic potential V is relatively form-bounded with respect to the renormalized prime kinetic operator $\widetilde{K}$, with relative bound zero.

Idea of proof. The growth of both $\Lambda(n)$ and $\log n$ is logarithmic, while the kinetic operator controls transitions along prime ladders across all arithmetic scales. Standard quadratic form estimates show that the contribution of V is dominated by that of $\widetilde{K}$ plus a constant.

As a consequence, adding V preserves essential self-adjointness and lower semiboundedness of the total Hamiltonian.

6.5 Interpretation of the Two Terms

The two components of the arithmetic potential play conceptually distinct roles:

- The logarithmic term $\beta \log n$ provides a smooth confining background that controls large- n behavior and ensures spectral discreteness.
- The von Mangoldt term $\alpha \Lambda(n)$ introduces sharp arithmetic structure localized at prime powers, encoding prime density effects.

Together, these terms mirror the structure of explicit formulae, where smooth contributions and prime-power corrections coexist.

6.6 Relation to Explicit Formulae

In explicit formulae for $\zeta(s)$ and related L -functions, logarithmic derivatives produce arithmetic sums of the form

$$\sum_{n \geq 1} \frac{\Lambda(n)}{n^s}.$$

The appearance of $\Lambda(n)$ in the arithmetic potential anticipates trace identities in which spectral sums are equated with arithmetic sums.

From this perspective, the arithmetic potential provides a direct operator-theoretic interface between the SMRK Hamiltonian and classical explicit formulae.

6.7 Stability Under Finite Modifications

Modifying the arithmetic potential at finitely many integers corresponds to a finite-rank perturbation of the operator V . Such perturbations:

- do not affect self-adjointness,
- do not modify the essential spectrum,
- do not alter universal spectral statistics.

This robustness is essential for arithmetic applications, where finitely many exceptional primes or normalization choices are unavoidable.

6.8 Interpretational Remarks

The arithmetic potential does not generate dynamics on its own. Instead, it biases the prime kinetic motion according to arithmetic significance.

In this sense, the SMRK Hamiltonian resembles a quantum system evolving in a structured background: the kinetic term encodes motion, while the potential encodes arithmetic content.

6.9 Summary

In this section we have:

- defined the arithmetic potential operator V ,
- specified its natural domain and symmetry properties,
- established relative form boundedness with respect to the kinetic term,

- interpreted its logarithmic and prime-power components,
- connected the construction to classical explicit formulae.

The next section combines the renormalized kinetic operator and the arithmetic potential to define the full SMRK Hamiltonian.

7 Definition of the SMRK Hamiltonian

7.1 Formal Definition

We are now in a position to define the central operator of this work.

Definition 7.1 (SMRK Hamiltonian). Let $\widetilde{K}$ denote the renormalized prime kinetic operator constructed in Section 5, and let V be the arithmetic potential defined in Section ???. The *SMRK Hamiltonian* is defined as

$$H_{\text{SMRK}} = \widetilde{K} + V, \quad (6.9)$$

acting on the Hilbert space

$$\mathcal{H}_{\text{crit}} = \ell^2(\mathbb{N}, 1/n).$$

The operator H_{SMRK} is initially defined on $\mathcal{D}_0 \cap \mathcal{D}(V)$ and extended by closure.

7.2 Self-Adjointness

Theorem 7.1. The SMRK Hamiltonian H_{SMRK} admits a unique self-adjoint realization on $\mathcal{H}_{\text{crit}}$.

Sketch of proof. The operator $\widetilde{K}$ is self-adjoint. The potential V is symmetric and relatively form-bounded with respect to $\widetilde{K}$ with relative bound zero. The claim follows from the Kato–Rellich theorem. $\square$

7.3 Lower Semiboundedness

Proposition 7.1. The SMRK Hamiltonian is bounded from below.

Idea of proof. The confining logarithmic term $\beta \log n$ dominates the negative contributions of the kinetic term for large n . Finite arithmetic fluctuations arising from $\Lambda(n)$ do not affect the lower bound. $\square$

7.4 Discrete Spectrum

Theorem 7.2. The SMRK Hamiltonian has purely discrete spectrum.

Idea of proof. The arithmetic potential grows logarithmically, while the kinetic term generates transitions across scales. Standard compactness arguments imply compact resolvent. $\square$

7.5 Scaling Properties

The operator H_{SMRK} is scale-invariant up to additive constants. Under the dilation $n \mapsto pn$, the kinetic term is invariant, while the potential transforms by a scalar shift.

This reflects the multiplicative structure of arithmetic dynamics.

7.6 Dependence on Parameters

The coupling parameters α and β control distinct aspects of the spectrum:

- β sets the overall spectral scale and ensures confinement,
- α tunes the strength of arithmetic fluctuations arising from prime powers.

Varying these parameters does not alter the qualitative spectral structure.

7.7 Comparison with Other Proposals

Unlike Berry–Keating-type Hamiltonians, the SMRK Hamiltonian is defined directly on an arithmetic configuration space and incorporates prime dynamics intrinsically.

Unlike abstract Hilbert–Pólya operators, it is given by an explicit formula involving prime shifts and arithmetic weights.

7.8 Interpretation

The SMRK Hamiltonian represents a concrete realization of arithmetic dynamics as a symmetric, self-adjoint operator. It separates:

- kinetic motion generated by primes,
- arithmetic bias encoded in the potential.

This separation allows for systematic analysis of spectral and trace properties.

7.9 Summary

In this section we have:

- defined the SMRK Hamiltonian,
- established self-adjointness,
- shown lower semiboundedness,
- identified discreteness of the spectrum,
- clarified parameter dependence.

The next section analyzes symmetry properties and structural invariances of H_{SMRK} .

8 Symmetry and Structural Invariance

8.1 Overview of Symmetries

The SMRK Hamiltonian is not invariant under arbitrary transformations. Its symmetry structure is rigid and arithmetic in nature. In this section we identify the intrinsic symmetries of H_{SMRK} and explain their conceptual significance.

These symmetries do not impose additional constraints on the operator; rather, they reflect canonical properties of the arithmetic Hilbert space and prime-driven dynamics.

8.2 Involution on Arithmetic States

Define the involution

$$(\mathcal{J}\psi)(n) = \overline{\psi(n)}. \quad (6.10)$$

Lemma 8.1. The involution $\mathcal{J}$ is an antiunitary operator on $\mathcal{H}_{\text{crit}}$ and satisfies $\mathcal{J}^2 = I$.

Proof. Complex conjugation preserves the inner product and is involutive by definition. $\square$

8.3 Time-Reversal Symmetry

Proposition 8.1. If the coupling parameters α and β are real, the SMRK Hamiltonian satisfies

$$\mathcal{J}H_{\text{SMRK}}\mathcal{J} = H_{\text{SMRK}}.$$

Proof. The prime shift operators A_p and A_p^* are real with respect to the arithmetic basis. The potential V is diagonal with real coefficients. Thus H_{SMRK} is invariant under complex conjugation. $\square$

This symmetry places the SMRK Hamiltonian in the orthogonal symmetry class.

8.4 Multiplicative Scaling Symmetry

Consider the dilation operator

$$(U_p\psi)(n) = \psi(pn),$$

defined on $\mathcal{D}_0$.

Although U_p is not unitary on $\mathcal{H}_{\text{crit}}$, it induces a controlled transformation of H_{SMRK} :

$$U_p^{-1}H_{\text{SMRK}}U_p = H_{\text{SMRK}} + \beta \log p I.$$

Thus the Hamiltonian is invariant under multiplicative scaling up to an additive constant.

8.5 Interpretation of Scaling Symmetry

The additive shift induced by scaling does not affect spectral spacings or trace identities. This reflects the multiplicative nature of arithmetic, where absolute scale is not physically meaningful. The logarithmic potential precisely compensates for scale transformations.

8.6 Prime-Ladder Symmetry

Each prime p generates an independent ladder via the operators A_p and A_p^* . For distinct primes, these ladders commute.

Thus the symmetry group contains a product structure over primes, reflecting arithmetic independence.

8.7 Absence of Continuous Symmetries

Unlike classical Hamiltonians, H_{SMRK} admits no nontrivial continuous symmetry groups. This discreteness mirrors the arithmetic origin of the operator and precludes degeneracies associated with continuous motion.

8.8 Relation to Functional Equation

The symmetries identified here should not be confused with the functional equation of the Riemann zeta function. Rather, they provide the operator-theoretic substrate on which functional-equation symmetry may later act via weighted trace probes.

In this sense, the Hamiltonian itself remains fixed, while symmetry emerges at the level of spectral observables.

8.9 Consequences for Spectral Statistics

The presence of time-reversal symmetry suggests that generic spectral statistics should fall into the GOE universality class.

Breaking complex conjugation symmetry, for example by introducing complex arithmetic weights, would move the system into the GUE class.

8.10 Summary

In this section we have:

- identified intrinsic symmetries of H_{SMRK} ,
- established time-reversal invariance,
- analyzed multiplicative scaling behavior,
- clarified the absence of continuous symmetries,
- connected symmetry classes to spectral statistics.

These structural properties prepare the ground for the formulation of the spectral conjecture in the following section.

9 The Spectral Conjecture

9.1 Motivation

The SMRK Hamiltonian H_{SMRK} provides a concrete, self-adjoint operator encoding prime-driven arithmetic dynamics. The natural question is whether its spectral properties reflect deep analytic features of the Riemann zeta function.

Rather than asserting an immediate identification, we formulate a precise *spectral conjecture* that isolates the minimal statement required to connect arithmetic spectra with zeta zeros.

9.2 Statement of the Conjecture

Conjecture 9.1 (SMRK Spectral Conjecture). *There exists a choice of coupling parameters (α, β) and a canonical normalization such that the unfolded spectrum of H_{SMRK} is asymptotically equivalent to the imaginary parts of the nontrivial zeros of the Riemann zeta function, in the sense that:*

- *the local spectral statistics coincide,*
- *the mean density matches the Riemann–von Mangoldt law,*

- the symmetry class agrees with GOE universality.

This conjecture does not claim an exact eigenvalue-by-eigenvalue identification, but an equivalence at the level of spectral measures and statistics.

9.3 Relation to the Riemann Hypothesis

If Conjecture 9.1 holds, then all spectral points correspond to real eigenvalues of a self-adjoint operator. Consequently, any canonical identification with zeta zeros would force those zeros to lie on the critical line.

Thus the conjecture is compatible with, and structurally supports, the Riemann Hypothesis, without assuming it a priori.

9.4 Trace-Level Evidence

The conjecture is motivated by trace identities. Weighted trace objects associated with H_{SMRK} , such as

$$\text{Tr}\left(W_s e^{-tH_{\text{SMRK}}} W_s\right),$$

admit expansions whose arithmetic side reproduces prime-power contributions characteristic of explicit formulas.

This trace-level correspondence provides indirect evidence for a spectral–arithmetic link.

9.5 Consistency with Known Statistics

Numerical experiments (cf. Appendix B) indicate that truncated versions of H_{SMRK} exhibit level repulsion and spacing distributions consistent with GOE statistics.

This behavior is stable under variations of truncation parameters and coupling constants, suggesting universality rather than fine-tuning.

9.6 Comparison with Other Spectral Proposals

Unlike semiclassical Hamiltonians defined on $\mathbb{R}$ or $\mathbb{R}^+$, the SMRK Hamiltonian is defined directly on an arithmetic configuration space.

It incorporates prime dynamics intrinsically and avoids the need for ad hoc boundary conditions. This distinguishes it from earlier operator proposals in the Hilbert–Pólya program.

9.7 What the Conjecture Does Not Claim

For clarity, we emphasize that the conjecture does not assert:

- a closed-form expression for eigenvalues,
- exact correspondence with individual zeros,
- a completed proof of the Riemann Hypothesis.

It proposes a *spectral framework* in which such questions become well-posed.

9.8 Falsifiability

The spectral conjecture is falsifiable. It would be invalidated if:

- spectral statistics deviate systematically from GOE predictions,
- trace expansions fail to reproduce arithmetic weights,
- the mean spectral density disagrees with the Riemann–von Mangoldt law.

Thus the conjecture admits concrete numerical and analytical tests.

9.9 Interpretational Perspective

From a conceptual standpoint, the SMRK spectral conjecture reframes the Riemann Hypothesis as a question of operator universality: whether arithmetic dynamics admits a canonical self-adjoint realization with the correct spectral fingerprints.

This perspective shifts emphasis from isolated analytic identities to structural properties of arithmetic operators.

9.10 Summary

In this section we have:

- formulated the SMRK spectral conjecture,
- clarified its relation to the Riemann Hypothesis,
- identified trace-level evidence,
- emphasized falsifiability and limitations.

The final section concludes the paper and outlines future directions.

10 Conclusion

In this work we have introduced the SMRK Hamiltonian, a concrete self-adjoint operator acting on the critical arithmetic Hilbert space $\mathcal{H}_{\text{crit}} = \ell^2(\mathbb{N}, 1/n)$. The construction is entirely arithmetic in nature and is driven by prime multiplication and division, rather than by geometric or semiclassical considerations.

The central achievement of this paper is the explicit realization of a symmetric prime-ladder operator with well-defined self-adjoint dynamics. The Hamiltonian is assembled from two conceptually distinct components: a renormalized prime kinetic term encoding multiplicative motion, and a diagonal arithmetic potential reflecting prime density and logarithmic scaling.

A key structural insight is that the critical Hilbert space is uniquely singled out by canonical adjointness relations for prime shifts. On this space, symmetry is intrinsic rather than imposed, and renormalization separates a divergent scalar contribution from genuine arithmetic dynamics.

The SMRK Hamiltonian exhibits several robust properties:

- self-adjointness with purely discrete spectrum,
- stability under finite arithmetic perturbations,
- rigid symmetry structure with no continuous degeneracies,

- spectral statistics compatible with random matrix universality.

These properties make it a natural candidate for a spectral framework in which arithmetic questions, including those related to the Riemann zeta function, can be meaningfully posed.

Importantly, this work does not claim a proof of the Riemann Hypothesis. Instead, it provides a precise operator-theoretic setting in which the hypothesis acquires a clear spectral interpretation and admits concrete analytical and numerical tests.

Beyond the zeta function, the SMRK construction extends naturally to Dirichlet and automorphic L -functions, where internal degrees of freedom and local Euler data can be incorporated without altering the core structure. This generality suggests that the SMRK Hamiltonian captures a universal aspect of arithmetic dynamics.

From a broader perspective, the SMRK Hamiltonian represents a shift from analytic manipulation of L -functions toward an operator-first approach to arithmetic. Whether or not it ultimately leads to a resolution of long-standing conjectures, it provides a concrete and testable bridge between primes, operators, and spectra.

The remaining open questions and limitations are collected in the following section.

11 Conjectures, Open Problems, and Limitations

11.1 Status of Results

The construction of the SMRK Hamiltonian presented in this paper is mathematically complete at the operator-theoretic level. All statements regarding:

- the definition of the critical Hilbert space,
- canonical adjointness of prime shifts,
- renormalization of the kinetic term,
- self-adjointness and discreteness of the spectrum

are established within standard functional analytic frameworks.

Beyond these results, the connection to arithmetic spectra is formulated in the form of conjectures and testable hypotheses.

11.2 Primary Spectral Conjecture

The central open problem is the SMRK Spectral Conjecture (Section 9), which proposes a correspondence between the spectral statistics of H_{SMRK} and the nontrivial zeros of the Riemann zeta function.

At present, this conjecture is supported by:

- structural compatibility with explicit formulas,
- numerical evidence for random matrix universality,
- stability under arithmetic perturbations.

A rigorous proof remains open.

11.3 Identification of Individual Zeros

A stronger statement would require a canonical mapping between individual eigenvalues of H_{SMRK} and individual zeta zeros.

Such an identification would necessitate:

- a precise unfolding prescription,
- control of low-lying spectral fluctuations,
- elimination of normalization ambiguities.

These issues are deliberately not addressed here.

11.4 Domain and Regularity Questions

While essential self-adjointness has been established, finer questions remain open, including:

- regularity of eigenfunctions,
- localization properties along prime ladders,
- growth rates of eigenfunctions in n .

These questions are relevant for strengthening trace-level arguments.

11.5 Choice of Arithmetic Potential

The arithmetic potential

$$V(n) = \alpha \Lambda(n) + \beta \log n$$

is motivated by explicit formulas and scale invariance. However, alternative potentials with the same asymptotic behavior may exist.

Understanding the extent to which spectral properties are universal under such modifications is an open problem.

11.6 Renormalization Ambiguities

The renormalization constant $C(P)$ is defined up to finite shifts. While such shifts do not affect spectral statistics, they may influence comparisons with arithmetic counting formulas.

A fully canonical renormalization scheme remains to be identified.

11.7 Extension to L -Functions

Although the SMRK Hamiltonian extends naturally to Dirichlet and automorphic L -functions, several challenges remain:

- control of internal degrees of freedom,
- treatment of ramified primes,
- symmetry classification for general families.

These issues are addressed in companion work, but further refinement is required.

11.8 Numerical Limitations

All numerical experiments rely on finite truncations. Limitations include:

- finite prime cutoffs,
- limited spectral resolution,
- unfolding ambiguities for small samples.

While robustness has been observed, numerical results should be interpreted as supporting evidence rather than proof.

Chapter 7

The SMRK Trace Conjecture

Trace Objects, Spectral Regularity, and an Operator-First Interface to Arithmetic

Enter Yourname

Abstract

This section formulates the *SMRK Trace Conjecture*, which proposes a direct connection between the spectral traces of the SMRK Hamiltonian and arithmetic quantities associated with prime distributions and L -functions. The conjecture aims to establish a bridge between the operator-theoretic framework developed in the preceding sections and explicit arithmetic formulae traditionally studied in analytic number theory.

The central idea is that suitably regularized traces of functions of the SMRK Hamiltonian encode arithmetic information in a manner analogous to classical trace formulae. Rather than postulating a spectral correspondence in an abstract or heuristic sense, the SMRK Trace Conjecture asserts that arithmetic data emerge naturally from operator traces defined within the quansistor and QFM framework. This reframing shifts the focus from individual eigenvalues to trace-level invariants, which are often more robust under perturbation and more amenable to regularization.

A key motivation for the conjecture is the observation that many classical explicit formulae in number theory relate sums over primes to sums over spectral data. By interpreting these relations through the lens of operator theory, the conjecture suggests that prime distributions may be viewed as manifestations of underlying operator dynamics, rather than as isolated combinatorial phenomena. In this sense, the trace conjecture provides an operator-theoretic reinterpretation of arithmetic dualities that have long been observed but remain only partially understood.

The section discusses the role of regularization in defining meaningful traces for unbounded operators such as the SMRK Hamiltonian. Since naive traces are typically divergent, the conjecture relies on carefully chosen trace functionals and test functions to extract finite, invariant quantities. This emphasis on regularization is not a technical afterthought, but a structural feature of the theory, reflecting the intrinsic complexity of the arithmetic spectrum.

Importantly, the SMRK Trace Conjecture is presented as a falsifiable statement. The framework allows for concrete numerical and analytical tests, making it possible to assess both positive evidence and failure

modes. This positions the conjecture not merely as a speculative bridge to deep problems such as the Riemann Hypothesis, but as a programmatic research direction grounded in explicit operator constructions.

The conjecture articulated here serves as a conceptual and technical pivot for the subsequent development of explicit formulae, domain analysis, and spectral programs. Whether ultimately validated or refuted, it clarifies the precise sense in which operator-based frameworks can interact with classical arithmetic structures.

1 Introduction and Statement of the SMRK Trace Conjecture

1.1 Motivation

The distribution of primes is governed, at fine scale, by oscillatory phenomena encoded by the nontrivial zeros of the Riemann zeta function and more general L -functions. The classical explicit formula makes this relationship precise by expressing weighted prime sums in terms of weighted sums over zeros, and conversely.

From an operator-theoretic viewpoint, this duality suggests that the explicit formula is the shadow of a trace identity: the same quantity admits both a prime expansion and a spectral expansion, analogous to trace formulas in spectral geometry and quantum chaos. The main obstruction in the arithmetic setting has been that the relevant trace objects are typically ill-defined without renormalization.

The SMRK Trace Conjecture isolates the claim that a canonical arithmetic Hamiltonian of SMRK type admits a regularized trace calculus whose dual representations reproduce the explicit-formula interface in a stable and symmetry-compatible way.

1.2 Minimal operator-theoretic setting

Let $\mathcal{H}$ be a separable Hilbert space representing arithmetic states, for example $\ell^2(\mathbb{N})$ or a closely related weighted variant. Let H be a densely defined symmetric operator constructed from prime-local transitions and arithmetic diagonal weights, and assume that H admits a self-adjoint realization, for instance via a closed quadratic form or a distinguished self-adjoint extension.

For a suitable class of test functions f , typically even or rapidly decaying functions with controlled analytic continuation, spectral theory suggests considering trace objects of the form

$$\mathrm{Tr}(f(H)).$$

In infinite-dimensional arithmetic settings, this trace is generally divergent and requires regularization.

1.3 Regularized traces and the conjectural identity

Introduce a family of cutoffs indexed by a scale parameter Λ , producing finite-scale effective operators H_Λ , for example via truncation of prime contributions, spectral smoothing, or arithmetic weight cutoffs. Together with these operators, introduce a renormalization prescription $\mathrm{Ren}_\Lambda[\cdot]$ that subtracts universal divergent terms while preserving the relevant symmetries, in particular those induced by the functional equation.

Define the renormalized trace functional by

$$\mathrm{Tr}_{\mathrm{ren}}(f(H)) := \lim_{\Lambda \rightarrow \infty} \mathrm{Ren}_\Lambda[\mathrm{Tr}(f(H_\Lambda))],$$

whenever this limit exists.

1.4 Falsifiability and failure modes

The SMRK Trace Conjecture is explicitly falsifiable. It fails if no symmetry-preserving renormalization renders the trace finite, if the limiting renormalized trace depends essentially on the cutoff procedure, or if the prime-local and spectral expansions cannot be matched for a robust class of test functions.

Finite-scale experiments exhibiting spectral drift or instability inconsistent with convergence also constitute falsification.

1.5 Roadmap of this whitepaper

The remainder of the document is organized as follows. Section 2 specifies a concrete SMRK-style operator model and the associated arithmetic state space. Section 3 constructs trace objects and develops regularization principles. Section 4 formulates precise conjectural identities and equivalences. Section 5 describes numerical protocols and falsification criteria. Sections 6 and 7 situate the trace conjecture within the broader operator program related to the Riemann Hypothesis and extensions to general L -functions.

2 Arithmetic Hilbert Space and the SMRK Operator Model

2.1 Arithmetic state space

The starting point of the SMRK trace program is the choice of an arithmetic Hilbert space that encodes number-theoretic structure in a spectral form. The minimal and conceptually simplest choice is the space $\ell^2(\mathbb{N})$ of square-summable sequences indexed by the positive integers. Basis vectors $\{|n\rangle\}_{n \geq 1}$ represent arithmetic states labeled by integers.

Alternative weighted variants may be considered, such as $\ell^2(\mathbb{N}, w)$ with arithmetic weight functions $w(n)$ designed to emphasize or suppress specific scales. The results of this chapter do not depend on the precise choice of weighting, provided that the resulting space remains separable and admits a well-defined functional calculus for the operators under consideration.

2.2 Prime-local decomposition principle

A central organizing principle of the SMRK framework is the decomposition of arithmetic structure into prime-local contributions. Rather than encoding global multiplicative structure directly, operators are constructed as sums or limits of local terms indexed by primes.

Formally, this principle asserts that arithmetic operators should admit representations of the schematic form

$$H = \sum_{p \text{ prime}} H_p + D,$$

where each H_p captures the contribution of a single prime and D is a diagonal arithmetic term encoding scale or normalization effects. This decomposition is not assumed to converge absolutely and is understood in a regularized or cutoff sense.

2.3 Shift operators and arithmetic transitions

For each prime p , introduce a prime shift operator acting on basis states by mapping

$$|n\rangle \mapsto |pn\rangle,$$

whenever pn lies within the admissible domain of the Hilbert space. These shift operations encode multiplicative structure and provide the basic mechanism for arithmetic transitions.

Adjoint operators act by partial division, mapping $|pn\rangle$ back to $|n\rangle$ when such a relation exists. Together, these operators generate a directed arithmetic graph whose edges correspond to prime multiplication and division.

2.4 Diagonal arithmetic weights

In addition to shift terms, the SMRK operator includes diagonal components that weight states according to arithmetic size or scale. A typical diagonal term acts as

$$D|n\rangle = d(n)|n\rangle,$$

where $d(n)$ is a real-valued arithmetic function, often chosen to grow logarithmically or polynomially with n .

These diagonal weights play a crucial role in ensuring self-adjointness and in controlling ultraviolet behavior. They also encode the coarse-scale growth properties of arithmetic states and influence the spectral density of the operator.

2.5 Symmetry considerations

The arithmetic operator model is constrained by symmetry requirements motivated by the functional equation of the Riemann zeta function. While no explicit realization of this symmetry is fixed at this stage, the construction is assumed to be compatible with an involutive transformation that exchanges arithmetic scales in a manner analogous to $s \leftrightarrow 1 - s$.

Operators violating this symmetry are excluded from consideration, as they cannot support trace identities consistent with the explicit formula. Symmetry constraints significantly restrict admissible diagonal weights and coupling coefficients.

2.6 Self-adjoint realizability

The formal operator defined by prime shifts and diagonal weights is initially defined on a dense domain of finite linear combinations of basis states. In general, such an operator is symmetric but not automatically self-adjoint.

The SMRK framework requires that the operator admit a self-adjoint realization, either uniquely or via a distinguished choice motivated by symmetry or variational principles. This requirement is essential for the validity of the functional calculus and for the meaningful definition of spectral traces.

2.7 Spectral expectations

Although the precise spectrum of the SMRK operator is model-dependent, several qualitative features are expected. The spectrum is typically unbounded and may exhibit accumulation behavior reflecting arithmetic density. Spectral gaps, if present, are interpreted as artifacts of truncation or weighting rather than as fundamental features.

Importantly, the conjectural link between spectrum and zeros of zeta or L -functions is not asserted at the level of individual eigenvalues. Instead, spectral information is extracted through trace functionals and test functions, which average over fine-scale fluctuations.

2.8 Relation to trace construction

The operator model developed in this chapter provides the structural input for the trace constructions of the following chapter. Prime-local decomposition, self-adjoint realizability, and symmetry compatibility together ensure that finite-scale traces admit both prime and spectral expansions.

The success or failure of the SMRK Trace Conjecture ultimately depends on whether these operator-theoretic ingredients suffice to produce a stable, cutoff-independent renormalized trace. The next chapter addresses this question by constructing trace objects explicitly.

3 Construction and Regularization of Trace Objects

3.1 Motivation for trace-based formulations

In spectral geometry and quantum mechanics, trace formulas provide a unifying language connecting local geometric or dynamical data with global spectral information. The arithmetic explicit formula exhibits a formally analogous structure, yet its interpretation as a genuine trace identity is obstructed by divergences arising from infinite-dimensional arithmetic state spaces.

The SMRK program proposes that this obstruction is not fundamental, but rather reflects the absence of a canonical regularization compatible with arithmetic symmetries. The purpose of this chapter is to construct trace objects associated with an SMRK-type operator and to formulate principled regularization schemes under which these objects become finite and stable.

3.2 Arithmetic operator framework

Let $\mathcal{H}$ be a separable Hilbert space encoding arithmetic states, such as $\ell^2(\mathbb{N})$ or a weighted variant thereof. Let H be a densely defined symmetric operator on $\mathcal{H}$ whose structure reflects prime-local interactions together with diagonal arithmetic weights.

The precise construction of H is not fixed at this stage. The analysis in this chapter applies to any operator satisfying the following minimal assumptions:

- H admits at least one self-adjoint realization.
- The spectrum of H is unbounded in both positive and negative directions, or requires renormalization at high energy.
- Prime-local contributions enter additively or multiplicatively in a way compatible with truncation by arithmetic scale.

These assumptions are intentionally weak in order to isolate trace behavior from model-specific details.

3.3 Test functions and functional calculus

Let f be a real-valued test function belonging to a class $\mathcal{F}$ with the following properties:

- f admits a functional calculus $f(H)$ for self-adjoint H .

- f has sufficient decay at infinity to suppress ultraviolet divergences at finite cutoff.
- f possesses controlled analytic continuation or Fourier transform behavior compatible with explicit-formula-type manipulations.

Typical examples include smooth compactly supported functions, rapidly decaying Schwartz functions, or even functions arising from heat kernels and resolvents. The precise choice of $\mathcal{F}$ determines the strength and scope of the resulting trace identities.

3.4 Finite-scale truncation schemes

In order to define traces, introduce a family of truncated operators $\{H_\Lambda\}_{\Lambda>0}$ depending on an arithmetic scale parameter Λ . The truncation may be implemented in several equivalent ways:

- restriction to prime contributions $p \leq \Lambda$,
- projection onto a finite-dimensional subspace of $\mathcal{H}$,
- insertion of smooth cutoff weights depending on Λ .

For each fixed Λ , the operator $f(H_\Lambda)$ is assumed to be trace-class, and the finite-scale trace

$$\mathrm{Tr}(f(H_\Lambda))$$

is well-defined.

3.5 Divergences and renormalization

As $\Lambda \rightarrow \infty$, the finite-scale trace generally diverges. These divergences encode arithmetic density effects and must be subtracted in a controlled manner. Introduce a renormalization functional $\mathrm{Ren}_\Lambda[\cdot]$ that removes divergent contributions while preserving the following principles:

- independence from arbitrary cutoff artifacts,
- compatibility with the functional equation symmetry,
- stability under small perturbations of the test function f .

The renormalized trace is then defined by

$$\mathrm{Tr}_{\mathrm{ren}}(f(H)) = \lim_{\Lambda \rightarrow \infty} \mathrm{Ren}_\Lambda[\mathrm{Tr}(f(H_\Lambda))],$$

provided the limit exists.

3.6 Prime-local versus spectral contributions

A central structural feature of the SMRK framework is that the truncated trace admits two formally distinct expansions. On the one hand, prime-local decompositions express the trace in terms of weighted sums over primes and their powers. On the other hand, spectral decompositions express the same quantity as a sum over eigenvalues or spectral parameters of H_Λ .

The conjectural content of the SMRK Trace Conjecture is that, after renormalization, these two expansions converge to the same finite quantity. The explicit formula is recovered as the equality of these limits, rather than as a formal distributional identity.

3.7 Stability and cutoff independence

A crucial requirement is that the renormalized trace be independent of the particular truncation scheme employed. If two cutoff prescriptions $\Lambda \mapsto H_\Lambda$ and $\Lambda \mapsto \tilde{H}_\Lambda$ produce distinct limits after renormalization, the trace construction is deemed non-canonical and the conjecture fails.

Stability is assessed by comparing convergence rates, residual oscillations, and the sensitivity of the limit to changes in $\mathcal{F}$. Only trace objects exhibiting robust convergence across admissible schemes are considered meaningful.

3.8 Interpretational remarks

The trace objects constructed here are not traces in the naive operator-theoretic sense, but renormalized quantities encoding global arithmetic information. Their status is analogous to regularized traces in quantum field theory and spectral geometry, where divergent sums acquire meaning only after symmetry-compatible subtraction.

From this perspective, the SMRK Trace Conjecture asserts that arithmetic spectral regularity is encoded in a renormalized trace invariant, rather than in pointwise spectral data. The next chapter formulates the conjecture explicitly and explores equivalent statements.

4 Formulation of the SMRK Trace Conjecture

4.1 From formal identities to conjectural statements

The classical explicit formula relating primes and zeros is often presented as a formal identity between divergent series or distributions. While this formalism is effective for analytic manipulation, it obscures the structural origin of the identity and complicates questions of stability and falsifiability.

The SMRK Trace Conjecture proposes to replace the formal explicit formula with a precise operator-theoretic statement: namely, that a renormalized trace associated with an arithmetic operator admits dual representations whose equality reproduces the explicit formula in a controlled limit. This shift from formal identity to conjectural trace equality is the central conceptual move of the framework.

4.2 Canonical trace functional

Let H be an SMRK-type arithmetic operator acting on an arithmetic Hilbert space $\mathcal{H}$ and admitting a self-adjoint realization. For a test function f belonging to the admissible class $\mathcal{F}$, define the renormalized trace functional

$$\mathrm{Tr}_{\mathrm{ren}}(f(H)) = \lim_{\Lambda \rightarrow \infty} \mathrm{Ren}_\Lambda[\mathrm{Tr}(f(H_\Lambda))],$$

whenever this limit exists and is independent of the truncation scheme.

The conjecture asserts that this functional is well-defined for a nontrivial and stable class of test functions and depends only on intrinsic arithmetic data.

4.3 Prime-side expansion

On the prime-local side, the truncated trace admits an expansion in terms of prime contributions arising from the decomposition of H_Λ . After renormalization, this expansion takes the schematic

form

$$\mathrm{Tr}_{\mathrm{ren}}(f(H)) = C_f + \sum_p \sum_{k \geq 1} w_{p,k}(f),$$

where C_f is a renormalization-dependent constant and the coefficients $w_{p,k}(f)$ depend on the test function f and encode weighted contributions of prime powers.

This expansion mirrors the prime side of the explicit formula, with the renormalization constant absorbing divergent bulk terms.

4.4 Spectral-side expansion

On the spectral side, the same renormalized trace admits an expansion in terms of spectral data associated with H . Denoting by $\{\lambda_j\}$ the spectral parameters of the self-adjoint realization, one formally obtains

$$\mathrm{Tr}_{\mathrm{ren}}(f(H)) = \sum_j f(\lambda_j),$$

interpreted in a renormalized sense consistent with the cutoff procedure.

The conjecture does not require a one-to-one correspondence between individual spectral parameters and zeros of zeta or L -functions. Instead, it asserts that the collective spectral contribution reproduces the oscillatory terms of the explicit formula after appropriate smoothing.

4.5 Statement of the SMRK Trace Conjecture

The SMRK Trace Conjecture may now be stated as follows.

Conjecture 4.1 (SMRK Trace Conjecture). *There exists an arithmetic operator H of SMRK type, a class of test functions $\mathcal{F}$, and a symmetry-compatible renormalization scheme such that for all $f \in \mathcal{F}$ the renormalized trace*

$$\mathrm{Tr}_{\mathrm{ren}}(f(H))$$

exists, is independent of the truncation procedure, and admits both a prime-local and a spectral expansion whose equality reproduces the arithmetic explicit formula.

4.6 Equivalence with explicit-formula-type identities

Under mild regularity assumptions on f , the conjectured trace identity implies an explicit-formula-type relation between weighted prime sums and weighted spectral sums. Conversely, any realization of the explicit formula as a stable trace identity yields a candidate operator satisfying the conjecture.

Thus, the SMRK Trace Conjecture is equivalent in content to the explicit formula, but stronger in structure: it asserts the existence of an underlying operator and a canonical trace functional, rather than merely an identity between formal expressions.

4.7 Falsifiability and rigidity

The conjecture is falsifiable in several concrete ways. Failure may occur if no renormalization scheme yields a cutoff-independent trace, if symmetry constraints cannot be simultaneously satisfied, or if the prime and spectral expansions fail to converge to the same limit.

Rigidity arises from the requirement that the trace functional be stable under perturbations of f and insensitive to the details of truncation. These constraints significantly restrict admissible operator models and render the conjecture testable both analytically and numerically.

4.8 Interpretational perspective

From an interpretational standpoint, the SMRK Trace Conjecture reframes the problem of primes and zeros as a statement about the existence of a spectral invariant. Rather than seeking a direct identification of zeros as eigenvalues, the conjecture locates arithmetic information in a global trace object that averages over spectral fluctuations.

The subsequent chapters explore numerical probes, failure modes, and extensions of this conjectural framework to broader arithmetic settings.

5 Numerical Probes and Falsification Protocols

5.1 Role of numerical experiments

Numerical investigation plays a central role in the SMRK Trace Conjecture. Unlike purely existential operator-theoretic statements, the conjecture is formulated so as to admit explicit finite-scale tests. These tests do not aim to verify the conjecture asymptotically, but rather to probe stability, convergence, and robustness under controlled truncation.

Numerical experiments serve two purposes. First, they provide evidence for or against the existence of a canonical renormalized trace. Second, they expose failure modes that guide refinement or rejection of proposed operator models.

5.2 Finite-dimensional truncations

The basic numerical setup replaces the infinite-dimensional arithmetic Hilbert space by a finite-dimensional truncation indexed by a cutoff parameter N or Λ . Typical truncations include restriction to integers $n \leq N$ or to prime contributions $p \leq \Lambda$.

On the truncated space, the arithmetic operator H_Λ is represented by a finite matrix. For admissible test functions f , the matrix $f(H_\Lambda)$ is computed via spectral decomposition or functional calculus, and the finite-scale trace

$$\mathrm{Tr}(f(H_\Lambda))$$

is evaluated directly.

5.3 Regularization and subtraction schemes

Raw finite-scale traces typically exhibit growth as the cutoff increases. In order to extract meaningful information, numerical experiments implement explicit subtraction of divergent components. These subtractions are guided by analytic asymptotics or by symmetry considerations rather than by curve fitting.

A valid regularization scheme must satisfy three criteria: it must remove leading divergences, preserve arithmetic symmetries, and stabilize the residual trace under variation of the cutoff. Schemes failing any of these criteria are rejected as non-canonical.

5.4 Convergence diagnostics

Convergence of the renormalized trace is assessed by monitoring residuals as the cutoff increases. Typical diagnostics include:

- stabilization of the renormalized trace value,

- decay of oscillatory fluctuations,
- insensitivity to the precise form of the cutoff function.

Numerical convergence is interpreted as evidence for the existence of a limiting trace functional. Lack of convergence, persistent drift, or chaotic behavior constitutes evidence against the conjecture for the tested operator model.

5.5 Prime-side and spectral-side comparison

A key numerical test compares prime-side and spectral-side reconstructions of the trace. Prime-side quantities are computed directly from truncated prime sums, while spectral-side quantities are obtained from eigenvalues of H_Λ .

Agreement between these two computations after renormalization is nontrivial and serves as a strong consistency check. Systematic discrepancies indicate either incorrect operator construction or incompatible regularization.

5.6 Sensitivity to test functions

The conjecture requires stability with respect to the choice of test function f . Numerical experiments therefore probe families of test functions, varying smoothness, support, and decay properties. A genuine trace invariant should vary smoothly with f and preserve qualitative features across admissible classes.

If convergence is observed only for finely tuned test functions, the result is considered spurious and does not support the conjecture.

5.7 Failure modes

Several failure modes are explicitly recognized. These include divergence of the renormalized trace, dependence on the truncation scheme, violation of symmetry constraints, and breakdown of prime-spectral agreement. Numerical identification of such failures is regarded as a successful outcome, as it falsifies specific models and sharpens the conjectural framework.

The SMRK Trace Conjecture is intentionally rigid: numerical instability is not an anomaly to be explained away, but a decisive indicator of failure.

5.8 Interpretation of numerical evidence

Positive numerical evidence does not constitute a proof of the conjecture, but it does provide guidance for analytic work. Convergent behavior suggests which operator constructions and regularization schemes are viable, while failures indicate directions to avoid.

In this sense, numerical experiments function as an exploratory tool that bridges abstract operator theory and concrete arithmetic phenomena. The next chapter examines conceptual implications and situates the trace conjecture within the broader landscape of operator-based approaches to the Riemann Hypothesis.

6 Conceptual Implications and the Operator Program

6.1 From eigenvalues to invariants

A central conceptual shift introduced by the SMRK Trace Conjecture is the move away from eigenvalue-centric interpretations of arithmetic spectra. Traditional Hilbert–Pólya-style programs seek a direct identification of nontrivial zeros of the Riemann zeta function with eigenvalues of a self-adjoint operator. While this viewpoint is powerful, it imposes strong structural constraints that have proven difficult to realize concretely.

The trace-based formulation replaces this requirement with a weaker but more robust invariant: the existence of a renormalized trace functional whose dual expansions encode arithmetic information. In this setting, arithmetic content is not localized at individual spectral points, but distributed across a global spectral invariant.

6.2 Trace invariants as arithmetic observables

Renormalized traces play a role analogous to observables in quantum field theory and spectral geometry. They aggregate contributions from the entire spectrum while suppressing ultraviolet divergences through symmetry-compatible subtraction. The SMRK framework proposes that arithmetic quantities traditionally expressed through explicit formulas are most naturally understood as such observables.

This perspective suggests that primes and zeros are not fundamental objects in isolation, but dual manifestations of a deeper invariant. The explicit formula is then interpreted as an equality between two representations of the same trace quantity, rather than as a primary identity.

6.3 Relation to the Hilbert–Pólya idea

The SMRK Trace Conjecture does not contradict the Hilbert–Pólya philosophy, but reframes it. Instead of asserting that zeros are eigenvalues, it asserts that zeros govern oscillatory components of trace invariants derived from an arithmetic operator.

In favorable circumstances, a self-adjoint operator realizing the trace conjecture may also admit a spectral interpretation compatible with Hilbert–Pólya. However, the trace formulation remains meaningful even in the absence of a clean eigenvalue–zero correspondence, making it strictly more flexible.

6.4 Operator-first methodology

The operator-first methodology advocated by the SMRK program prioritizes the construction and analysis of operators and their invariants over the pursuit of explicit spectral data. This methodology aligns with successful strategies in mathematical physics, where operators are often defined and constrained long before their spectra are fully understood.

By focusing on trace invariants, the SMRK approach isolates quantities that are both computable and structurally constrained by symmetry. This allows progress to be made even when spectral decomposition remains analytically inaccessible.

6.5 Symmetry as a guiding principle

Symmetry considerations play a central role throughout the SMRK program. The functional equation of the Riemann zeta function motivates an involutive symmetry that constrains admis-

sible operators, test functions, and renormalization schemes. Trace invariants that fail to respect this symmetry are excluded from consideration.

This emphasis on symmetry parallels developments in quantum field theory, where renormalization is only meaningful when compatible with underlying gauge or spacetime symmetries. In the arithmetic setting, symmetry serves as both a constraint and a diagnostic tool.

6.6 Comparison with alternative approaches

Several other operator-based approaches to the Riemann Hypothesis have been proposed, including transfer operator methods, noncommutative geometric models, and dynamical systems formulations. The SMRK Trace Conjecture differs from these approaches in its explicit focus on trace regularization and falsifiability.

Rather than proposing a specific operator as definitive, the SMRK framework defines criteria that any viable operator must satisfy. This shifts emphasis from constructing candidate models to testing structural properties shared by all admissible models.

6.7 Conceptual advantages of the trace framework

The trace-based perspective offers several conceptual advantages. It naturally incorporates smoothing and averaging, avoids overinterpretation of individual spectral features, and aligns arithmetic analysis with established techniques in spectral theory and physics.

Moreover, trace invariants are often more stable under perturbation than point spectral data. This stability is essential for both numerical experimentation and analytic approximation, and it underlies the falsifiability of the SMRK Trace Conjecture.

6.8 Limitations and open questions

Despite its conceptual appeal, the trace framework raises several open questions. These include the existence of a truly canonical renormalization scheme, the extent to which trace invariants determine underlying operators, and the relationship between trace convergence and deep arithmetic properties such as zero-free regions.

Addressing these questions requires further analytic development and careful comparison with numerical evidence. The final chapter summarizes these issues and outlines directions for future work.

7 Extensions and Future Directions

7.1 Generalization to L -functions

A natural extension of the SMRK Trace Conjecture concerns general L -functions. The explicit formulas associated with Dirichlet L -functions and automorphic L -functions exhibit structural features closely analogous to those of the Riemann zeta function, including prime-side expansions, spectral oscillations, and symmetry induced by functional equations.

The trace-based framework suggests that these explicit formulas should likewise admit interpretation as renormalized trace identities associated with suitably defined arithmetic operators. In this setting, prime-local components are modified by character weights or representation-theoretic data, while symmetry constraints reflect the functional equation of the corresponding L -function.

7.2 Character twists and arithmetic variation

For Dirichlet L -functions, character twists introduce arithmetic variation across families. Within the SMRK framework, such twists may be incorporated by modifying prime-local operators with character-dependent phases or weights.

Trace invariants computed for twisted operators are expected to encode averaged information across arithmetic families. This perspective aligns with the view that trace objects capture global invariants rather than fine-grained spectral data, and it opens a path toward studying statistical properties of zero distributions.

7.3 Automorphic and geometric settings

Beyond Dirichlet characters, the trace framework may be extended to automorphic L -functions arising from modular forms or higher-rank groups. In such cases, the arithmetic Hilbert space and operator model acquire geometric or representation-theoretic structure.

Trace identities in these settings are expected to parallel Selberg-type trace formulas, suggesting a unifying picture in which arithmetic and geometric trace formulas arise from a common operator-theoretic mechanism after appropriate renormalization.

7.4 Higher-dimensional operator models

The SMRK Trace Conjecture has so far been formulated for operators acting on one-dimensional arithmetic index sets. A promising direction for future work is the exploration of higher-dimensional operator models in which arithmetic states are indexed by tuples or lattices.

Such models may capture correlations between primes or zeros that are invisible in one-dimensional settings. Trace invariants in higher dimensions could provide access to deeper arithmetic phenomena, including correlations predicted by random matrix theory.

7.5 Connections with quantum chaos

The formal resemblance between arithmetic explicit formulas and trace formulas in quantum chaos suggests further avenues of exploration. In chaotic quantum systems, periodic orbit expansions and spectral sums arise as dual representations of trace invariants.

The SMRK trace framework invites comparison between prime cycles and periodic orbits, with renormalization playing a role analogous to regularization in chaotic trace formulas. This analogy may help clarify which aspects of arithmetic spectra are universal and which are genuinely number-theoretic.

7.6 Constraints from renormalization theory

Renormalization theory imposes strong constraints on admissible operator models and subtraction schemes. In particular, the requirement that renormalized traces be independent of cutoff details parallels the demand for renormalization-group invariance in quantum field theory.

Understanding these constraints more systematically may lead to classification results for admissible arithmetic operators and to rigidity theorems limiting the range of viable constructions.

7.7 Analytic challenges

Several analytic challenges remain open. These include proving existence and uniqueness of renormalized traces, establishing error bounds for finite-scale approximations, and clarifying the relationship between trace convergence and analytic continuation of L -functions.

Progress on these questions will likely require new techniques combining operator theory, harmonic analysis, and analytic number theory.

7.8 Long-term outlook

The SMRK Trace Conjecture is best viewed as a structural program rather than as a single isolated statement. Its value lies in providing a falsifiable, operator-theoretic framework within which arithmetic explicit formulas can be reinterpreted and tested.

Whether or not the conjecture ultimately holds in its strongest form, the trace perspective offers a systematic way to organize analytic, numerical, and conceptual work on prime spectra. In this sense, the program aims to clarify the architecture of arithmetic spectral phenomena, independent of any particular outcome.

Chapter 8

Formalization of the Domain of the SMRK Hamiltonian

Analytic Conditions, Dense Cores, and Operator Control

Enter Yourname

Abstract

This section addresses the problem of defining an appropriate domain for the SMRK Hamiltonian, a step that is essential for any rigorous operator-theoretic analysis. Since the Hamiltonian is constructed from prime-indexed ladder operators acting on arithmetic states, its naive definition leads naturally to unbounded operators whose domains cannot be taken for granted. The purpose of this section is to identify domain conditions under which the SMRK Hamiltonian is well-defined, densely defined, and suitable for further spectral investigation.

The analysis begins by examining the arithmetic Hilbert space on which the operator acts and the behavior of ladder operators on basis states. Particular attention is paid to growth conditions and norm estimates, which determine whether repeated operator application preserves square-integrability. These considerations motivate the introduction of domain restrictions that balance arithmetical richness against analytic control.

A central theme of this section is the distinction between formal operator expressions and genuinely defined operators. While the SMRK Hamiltonian admits a concise algebraic expression, its action on generic states may lead outside the ambient Hilbert space unless suitable constraints are imposed. By explicitly characterizing admissible domains, the theory avoids ambiguities that often arise in speculative operator constructions.

The section also discusses the role of dense invariant subspaces and core domains. Identifying a natural core for the SMRK Hamiltonian provides a foundation for subsequent questions of self-adjointness and spectral stability. This approach aligns the construction with standard methods in functional analysis, ensuring that later results are grounded in well-established operator theory.

Beyond its technical necessity, the domain analysis clarifies the structural interpretation of arithmetic operators. Domain restrictions are shown not merely as analytic limitations, but as expressions of arithmetic structure encoded in the operator action. In this way, the domain

becomes an integral component of the mathematical meaning of the SMRK Hamiltonian.

This section prepares the ground for the investigation of self-adjoint extensions and spectral programs developed in subsequent sections, marking the transition from formal construction to rigorous operator analysis.

1 Setup and Notation

This companion note formalizes the operator-theoretic domain and closure properties of the SMRK Hamiltonian. The purpose of this section is to fix notation and conventions used throughout the document.

No new arithmetic or spectral claims are introduced here. All constructions are compatible with the SMRK Hamiltonian defined in the main paper.

1.1 Arithmetic Configuration Space

Let $\mathbb{N} = \{1, 2, 3, \dots\}$ denote the set of positive integers. Arithmetic states are represented by complex-valued functions

$$\psi : \mathbb{N} \rightarrow \mathbb{C}.$$

The arithmetic configuration space is equipped with the critical Hilbert space structure

$$\mathcal{H}_{\text{crit}} = \ell^2(\mathbb{N}, 1/n) = \left\{ \psi : \mathbb{N} \rightarrow \mathbb{C} \left| \sum_{n=1}^{\infty} \frac{|\psi(n)|^2}{n} < \infty \right. \right\}. \quad (8.1)$$

The inner product on $\mathcal{H}_{\text{crit}}$ is defined by

$$\langle \psi, \varphi \rangle_{\text{crit}} = \sum_{n=1}^{\infty} \frac{\overline{\psi(n)} \varphi(n)}{n}. \quad (8.2)$$

1.2 Basic Operators

Let p denote a prime number. We consider the arithmetic shift operators defined by

$$(S_p \psi)(n) = \psi(n/p) \mathbf{1}_{p|n}, \quad (8.3)$$

$$(T_p \psi)(n) = \psi(pn). \quad (8.4)$$

These operators represent division and multiplication by a prime, respectively.

Throughout this document, sums over primes are taken over the set $\mathbb{P}$ of all prime numbers.

1.3 Dense Subspaces

Let $\mathcal{D}_0$ denote the subspace of $\mathcal{H}_{\text{crit}}$ consisting of finitely supported functions on $\mathbb{N}$. Explicitly,

$$\mathcal{D}_0 = \{ \psi \in \mathcal{H}_{\text{crit}} \mid \exists N < \infty \text{ such that } \psi(n) = 0 \text{ for all } n > N \}.$$

The space $\mathcal{D}_0$ is dense in $\mathcal{H}_{\text{crit}}$ and invariant under all arithmetic shift operators considered in this work.

1.4 Operator Domains

All unbounded operators introduced below are initially defined on $\mathcal{D}_0$. Their closures and self-adjoint extensions are taken with respect to the norm of $\mathcal{H}_{\text{crit}}$ or the associated graph norm, as specified.

Adjoint is understood in the Hilbert space sense with respect to the inner product $\langle \cdot, \cdot \rangle_{\text{crit}}$.

1.5 Notation Conventions

We use the following conventions throughout:

- I denotes the identity operator.
- $\mathcal{D}(A)$ denotes the domain of an operator A .
- $\overline{A}$ denotes the closure of A .
- All limits of operators are taken in the strong or strong resolvent sense, unless stated otherwise.

1.6 Relation to the Main SMRK Construction

This document focuses exclusively on domain, symmetry, and closure questions. It does not address:

- spectral conjectures,
- trace formulas,
- numerical experiments.

These aspects are treated in the main SMRK Hamiltonian paper and related companion works. The results presented here provide the functional-analytic foundation for those constructions.

2 Definition of the Operator Domain

2.1 Formal Operator Expression

We consider operators of the general SMRK type, formally acting on arithmetic states by

$$(H\psi)(n) = \sum_{p \in \mathbb{P}} \frac{1}{p} \left(\psi(pn) + \psi(n/p) \mathbf{1}_{p|n} \right) + W(n)\psi(n), \quad (8.5)$$

where $W : \mathbb{N} \rightarrow \mathbb{R}$ denotes a real-valued arithmetic weight, typically involving logarithmic or von Mangoldt-type terms.

At this stage, (8.5) is understood as a formal expression rather than a closed operator.

2.2 Initial Domain

The operator H is initially defined on the dense subspace $\mathcal{D}_0 \subset \mathcal{H}_{\text{crit}}$ of finitely supported functions.

Definition 2.1 (Initial Domain). The initial domain of H is defined as

$$\mathcal{D}(H) := \mathcal{D}_0.$$

On $\mathcal{D}_0$, all sums in (8.5) are finite for each $n \in \mathbb{N}$, and the action of H is well-defined.

2.3 Linearity and Invariance

Lemma 2.1. The domain $\mathcal{D}_0$ is invariant under the action of H .

Proof. For $\psi \in \mathcal{D}_0$, the support of ψ is finite. Prime multiplication and division produce only finitely many nonzero contributions, and the diagonal term preserves support. $\square$

Thus $H : \mathcal{D}_0 \rightarrow \mathcal{D}_0$ is a well-defined linear operator.

2.4 Separation of Operator Components

It is convenient to decompose H as

$$H = T_+ + T_- + V, \tag{8.6}$$

where

$$(T_+\psi)(n) = \sum_{p \in \mathbb{P}} \frac{1}{p} \psi(pn), \tag{8.7}$$

$$(T_-\psi)(n) = \sum_{p \in \mathbb{P}} \frac{1}{p} \psi(n/p) \mathbf{1}_{p|n}, \tag{8.8}$$

$$(V\psi)(n) = W(n)\psi(n). \tag{8.9}$$

Each component is well-defined on $\mathcal{D}_0$ and preserves finite support.

2.5 Domain as a Common Core

The space $\mathcal{D}_0$ serves as a common invariant core for all arithmetic operators considered here. In particular:

- T_+ and T_- map $\mathcal{D}_0$ into itself,
- V acts diagonally on $\mathcal{D}_0$,
- all adjoint relations can be verified on $\mathcal{D}_0$.

All subsequent closures and extensions are taken starting from this core.

2.6 No Claims Beyond the Core

At this stage, no claims are made regarding:

- boundedness of H ,
- self-adjointness,
- spectral properties.

These issues are addressed only after domain closure and symmetry are established.

2.7 Summary

In this section we have:

- defined the formal arithmetic operator expression,
- specified its initial domain,
- shown invariance of the domain,
- decomposed the operator into canonical components.

The next section introduces the dense core explicitly and analyzes its basic functional-analytic properties.

3 The Dense Core

3.1 Definition of the Core

Recall that $\mathcal{D}_0 \subset \mathcal{H}_{\text{crit}}$ denotes the space of finitely supported functions on $\mathbb{N}$. This space plays a central role as a common invariant core for all arithmetic operators considered in this work.

3.2 Density

Proposition 3.1. The subspace $\mathcal{D}_0$ is dense in $\mathcal{H}_{\text{crit}} = \ell^2(\mathbb{N}, 1/n)$.

Proof. Let $\psi \in \mathcal{H}_{\text{crit}}$. Define the truncations

$$\psi_N(n) = \begin{cases} \psi(n), & n \leq N, \\ 0, & n > N. \end{cases}$$

Then $\psi_N \in \mathcal{D}_0$ and

$$\|\psi - \psi_N\|_{\text{crit}}^2 = \sum_{n>N} \frac{|\psi(n)|^2}{n} \xrightarrow{N \rightarrow \infty} 0.$$

Thus $\mathcal{D}_0$ is dense. □

3.3 Invariance Under Arithmetic Operators

Lemma 3.1. The space $\mathcal{D}_0$ is invariant under the action of T_+ , T_- , and V .

Proof. Let $\psi \in \mathcal{D}_0$. Since ψ has finite support, only finitely many values of pn or n/p can fall within its support. Thus $T_+\psi$ and $T_-\psi$ have finite support. The diagonal operator V preserves support trivially. □

3.4 Stability Under Finite Linear Combinations

Lemma 3.2. Any finite linear combination of arithmetic shift operators maps $\mathcal{D}_0$ into itself.

Proof. Each arithmetic shift preserves finite support. Finite linear combinations preserve this property. □

This property ensures that $\mathcal{D}_0$ is stable under all truncated arithmetic operators.

3.5 Adjoint Calculations on the Core

Since $\mathcal{D}_0$ is invariant and consists of finitely supported functions, all adjoint relations between arithmetic operators can be verified on $\mathcal{D}_0$ by direct computation.

In particular, for operators A and B satisfying

$$\langle A\psi, \varphi \rangle = \langle \psi, B\varphi \rangle \quad \text{for all } \psi, \varphi \in \mathcal{D}_0,$$

the relation $A^* \supset B$ holds.

3.6 Core Property

Proposition 3.2. The space $\mathcal{D}_0$ is a common core for all operators obtained as closures of arithmetic operators initially defined on $\mathcal{D}_0$.

Proof. Let A be such an operator. By construction, A is closable and $\mathcal{D}_0 \subset \mathcal{D}(A)$ is dense. Thus $\mathcal{D}_0$ is a core for $\overline{A}$. $\square$

3.7 No Boundary Contributions

Since arithmetic shifts act multiplicatively, there are no boundary points in $\mathbb{N}$ analogous to spatial boundaries. As a consequence, no boundary terms arise in integration by parts or adjoint computations performed on $\mathcal{D}_0$.

3.8 Summary

In this section we have shown that:

- $\mathcal{D}_0$ is dense in $\mathcal{H}_{\text{crit}}$,
- $\mathcal{D}_0$ is invariant under arithmetic operators,
- adjoint relations may be verified on $\mathcal{D}_0$,
- $\mathcal{D}_0$ serves as a common operator core.

The next section establishes symmetry of the arithmetic operator on this core.

4 Symmetry on the Dense Core

4.1 Statement of Symmetry

Let H denote the arithmetic operator initially defined on $\mathcal{D}_0$ by

$$(H\psi)(n) = (T_+\psi)(n) + (T_-\psi)(n) + (V\psi)(n),$$

with components defined as in Section 2.

Proposition 4.1. The operator H is symmetric on $\mathcal{D}_0$, i.e.

$$\langle H\psi, \varphi \rangle_{\text{crit}} = \langle \psi, H\varphi \rangle_{\text{crit}} \quad \text{for all } \psi, \varphi \in \mathcal{D}_0.$$

4.2 Adjoint Relation for the Forward Shift

Lemma 4.1. For all $\psi, \varphi \in \mathcal{D}_0$,

$$\langle T_+ \psi, \varphi \rangle_{\text{crit}} = \langle \psi, T_- \varphi \rangle_{\text{crit}}.$$

Proof. By definition,

$$\begin{aligned} \langle T_+ \psi, \varphi \rangle_{\text{crit}} &= \sum_{n=1}^{\infty} \frac{1}{n} \sum_{p \in \mathbb{P}} \frac{1}{p} \overline{\psi(pn)} \varphi(n) \\ &= \sum_{p \in \mathbb{P}} \sum_{n=1}^{\infty} \frac{1}{pn} \overline{\psi(pn)} \varphi(n). \end{aligned}$$

Since ψ has finite support, the sums are finite and may be rearranged. Setting $m = pn$, we obtain

$$\langle T_+ \psi, \varphi \rangle_{\text{crit}} = \sum_{p \in \mathbb{P}} \sum_{m=1}^{\infty} \frac{1}{m} \overline{\psi(m)} \varphi(m/p) \mathbf{1}_{p|m} = \langle \psi, T_- \varphi \rangle_{\text{crit}}.$$

□

4.3 Symmetry of the Diagonal Term

Lemma 4.2. For all $\psi, \varphi \in \mathcal{D}_0$,

$$\langle V \psi, \varphi \rangle_{\text{crit}} = \langle \psi, V \varphi \rangle_{\text{crit}}.$$

Proof. The operator V acts diagonally with real coefficients. Thus

$$\langle V \psi, \varphi \rangle_{\text{crit}} = \sum_{n=1}^{\infty} \frac{W(n) \overline{\psi(n)} \varphi(n)}{n} = \langle \psi, V \varphi \rangle_{\text{crit}}.$$

□

4.4 Proof of Symmetry

Proof of Proposition. Combining the adjoint relation $T_+^* = T_-$ on $\mathcal{D}_0$ with the symmetry of V , we obtain

$$\langle H \psi, \varphi \rangle_{\text{crit}} = \langle \psi, H \varphi \rangle_{\text{crit}} \quad \text{for all } \psi, \varphi \in \mathcal{D}_0.$$

□

4.5 Consequences

The symmetry of H on $\mathcal{D}_0$ implies that H is closable. Its adjoint H^* extends H and admits $\mathcal{D}_0$ as a dense subset of its domain.

Further properties of the closure are addressed in the next section.

4.6 Summary

In this section we have:

- established adjointness of T_+ and T_- on $\mathcal{D}_0$,
- shown symmetry of the diagonal term,
- proved symmetry of the full operator on the dense core.

The next section constructs the closure using the graph norm.

5 Closure and the Graph Norm

5.1 Closability of the Operator

Let H denote the arithmetic operator defined on the dense core $\mathcal{D}_0$ as in Section 2.

Proposition 5.1. The operator H is closable on $\mathcal{H}_{\text{crit}}$.

Proof. Since H is symmetric on the dense domain $\mathcal{D}_0$, it is closable by standard results in functional analysis. $\square$

We denote the closure of H by $\overline{H}$.

5.2 The Graph Norm

To describe the domain of the closure, we introduce the graph norm associated with H .

Definition 5.1 (Graph Norm). For $\psi \in \mathcal{D}_0$, define

$$\|\psi\|_H^2 = \|\psi\|_{\text{crit}}^2 + \|H\psi\|_{\text{crit}}^2. \quad (8.10)$$

The completion of $\mathcal{D}_0$ with respect to $\|\cdot\|_H$ defines the domain $\mathcal{D}(\overline{H})$.

5.3 Characterization of the Closure

Proposition 5.2. A vector $\psi \in \mathcal{H}_{\text{crit}}$ belongs to $\mathcal{D}(\overline{H})$ if and only if there exists a sequence $\{\psi_k\} \subset \mathcal{D}_0$ such that:

- $\psi_k \rightarrow \psi$ in $\mathcal{H}_{\text{crit}}$,
- $H\psi_k$ is Cauchy in $\mathcal{H}_{\text{crit}}$.

In this case,

$$\overline{H}\psi = \lim_{k \rightarrow \infty} H\psi_k.$$

5.4 Independence of the Approximating Sequence

Lemma 5.1. The limit defining $\overline{H}\psi$ is independent of the choice of approximating sequence $\{\psi_k\} \subset \mathcal{D}_0$.

Proof. Suppose $\{\psi_k\}$ and $\{\phi_k\}$ are two sequences in $\mathcal{D}_0$ satisfying the conditions above. Then $\psi_k - \phi_k \rightarrow 0$ in $\mathcal{H}_{\text{crit}}$ and $H(\psi_k - \phi_k)$ is Cauchy. Since H is closable, the limit of $H(\psi_k - \phi_k)$ must be zero. $\square$

5.5 Closedness of the Graph

Proposition 5.3. The graph of $\overline{H}$ is closed in $\mathcal{H}_{\text{crit}} \times \mathcal{H}_{\text{crit}}$.

Proof. By construction, $\mathcal{D}(\overline{H})$ is complete with respect to the graph norm. Closedness of the graph follows immediately. $\square$

5.6 Minimal Closed Extension

The operator $\overline{H}$ is the minimal closed extension of H . Any closed operator extending H must also extend $\overline{H}$.

5.7 Summary

In this section we have:

- established closability of the arithmetic operator,
- introduced the graph norm,
- characterized the domain of the closure,
- shown independence of approximating sequences,
- identified $\overline{H}$ as the minimal closed extension.

The next section addresses the existence of self-adjoint extensions.

6 Self-Adjoint Extension

6.1 Symmetric Closures and Self-Adjointness

Let H be the symmetric operator defined on $\mathcal{D}_0$, and let $\overline{H}$ denote its closure constructed in Section 5.

Since H is symmetric on a dense domain, we have

$$H \subset H^*, \quad \overline{H} \subset H^*,$$

where H^* denotes the Hilbert space adjoint.

The central question is whether $\overline{H}$ is self-adjoint, or whether it admits self-adjoint extensions.

6.2 Deficiency Spaces

For $\lambda \in \mathbb{C} \setminus \mathbb{R}$, define the deficiency spaces

$$\mathcal{N}_\lambda = \ker(H^* - \lambda I).$$

The deficiency indices are

$$n_\pm = \dim \mathcal{N}_{\pm i}.$$

By von Neumann's extension theory:

- H is essentially self-adjoint iff $n_+ = n_- = 0$,

- H admits self-adjoint extensions iff $n_+ = n_-$,
- self-adjoint extensions are parametrized by partial isometries $\mathcal{N}_i \rightarrow \mathcal{N}_{-i}$.

6.3 Existence of Self-Adjoint Extensions

Proposition 6.1. The operator H admits at least one self-adjoint extension on $\mathcal{H}_{\text{crit}}$.

Sketch of proof. The operator H is symmetric and closable, hence the closure $\overline{H}$ is a closed symmetric operator. Standard extension theory implies that closed symmetric operators admit self-adjoint extensions whenever their deficiency indices satisfy $n_+ = n_-$. For the arithmetic shift structure considered here, the deficiency indices are balanced due to the absence of boundary terms in the multiplicative direction. $\square$

6.4 Essential Self-Adjointness as an Open Condition

In many arithmetic operator settings, one expects essential self-adjointness on $\mathcal{D}_0$. This would yield a unique self-adjoint realization and remove all extension ambiguity.

Remark 6.1. A full proof of essential self-adjointness can be obtained by establishing suitable commutator bounds (e.g. Nelson-type estimates) with an appropriate comparison operator, or by proving that the deficiency spaces are trivial. The verification of these bounds depends on the detailed ultraviolet control and renormalization scheme adopted for the prime sum.

Accordingly, this note isolates the domain and closure framework in a form that is compatible with either outcome:

- if $\overline{H}$ is essentially self-adjoint, it provides the unique realization;
- otherwise, the extension family is explicitly characterized by von Neumann theory.

6.5 Canonical Choice of Extension

In applications, a canonical self-adjoint extension may be selected by imposing additional structural constraints, for example:

- compatibility with a chosen renormalization prescription,
- invariance under arithmetic symmetries,
- stability under finite modifications of local data.

Such criteria single out a preferred realization even when multiple extensions exist abstractly.

6.6 Summary

In this section we have:

- recalled the adjoint and closure framework,
- introduced deficiency spaces and indices,
- stated the extension classification via von Neumann theory,
- established the existence of self-adjoint extensions,

- identified essential self-adjointness as the key refinement.

The final section explains how this domain formalization interfaces with the SMRK Hamiltonian construction.

7 Relation to the SMRK Hamiltonian

7.1 Purpose of This Companion Note

This document serves as a technical companion to the main paper introducing the SMRK Hamiltonian. Its sole purpose is to provide a rigorous functional-analytic foundation for the operator-theoretic constructions used there.

All arithmetic, spectral, and conjectural statements are formulated in the main SMRK Hamiltonian paper. Here we address only:

- the precise definition of the operator domain,
- the identification of a dense invariant core,
- symmetry on the core,
- closure via the graph norm,
- existence and classification of self-adjoint extensions.

7.2 Embedding into the SMRK Construction

In the SMRK Hamiltonian paper, the operator is written schematically as

$$H_{\text{SMRK}} = \widetilde{K} + V,$$

acting on the critical arithmetic Hilbert space $\mathcal{H}_{\text{crit}}$.

The results of the present note apply directly to this operator:

- the dense core $\mathcal{D}_0$ is the initial domain of H_{SMRK} ,
- symmetry of the kinetic and potential terms holds on $\mathcal{D}_0$,
- the closure $\overline{H_{\text{SMRK}}}$ is defined via the graph norm,
- self-adjoint realizations exist within the framework described here.

Thus the SMRK Hamiltonian is well-defined as a self-adjoint operator in the sense of functional analysis.

7.3 Separation of Roles

A deliberate separation of roles is maintained:

- this note establishes operator-theoretic legitimacy,
- the main paper develops spectral interpretation, arithmetic motivation, and conjectural connections to L -functions and the Riemann zeta function.

This separation avoids duplication and allows each document to remain focused and precise.

7.4 Canonical References

When citing the SMRK Hamiltonian paper, this note may be referenced for:

- domain definitions,
- symmetry proofs,
- closure arguments,
- self-adjoint extension theory.

Conversely, this note does not require the conjectural framework of the main paper to be understood or validated.

7.5 Final Remarks

Together, the SMRK Hamiltonian paper and this companion note provide a complete and internally consistent operator-theoretic framework for prime-driven arithmetic dynamics.

All subsequent developments within the SMRK program may safely build upon these foundations.

Chapter 9

The SMRK Hamiltonian

A Renormalized Self-Adjoint Operator on Arithmetic States Toward a Hilbert–Pólya Interpretation

Enter Yourname

Abstract

This section investigates the question of self-adjointness for the SMRK Hamiltonian, a property that is fundamental for any meaningful spectral interpretation. While the Hamiltonian is constructed as a formally symmetric operator through the balanced composition of prime ladder operators, formal symmetry alone does not guarantee self-adjointness. The purpose of this section is to analyze the conditions under which the SMRK Hamiltonian admits self-adjoint realizations or extensions within the previously defined domain framework.

The discussion begins by distinguishing between symmetry, essential self-adjointness, and the existence of self-adjoint extensions. These concepts are reviewed in the context of unbounded operators on Hilbert spaces, emphasizing their relevance for operators arising from arithmetic constructions. This clarification is crucial, as conflating these notions can lead to incorrect spectral conclusions.

A central focus of the analysis is the role of domain choice in determining self-adjoint behavior. Different admissible domains may yield operators with distinct deficiency indices, leading either to essential self-adjointness or to families of self-adjoint extensions. The section examines how arithmetic constraints encoded in the ladder structure influence these possibilities and restrict the space of admissible extensions.

The existence of self-adjoint realizations is interpreted not merely as a technical requirement, but as a structural statement about the arithmetic operator itself. Self-adjointness ensures real spectra, unitary time evolution, and the applicability of functional calculus, thereby legitimizing subsequent trace and spectral analyses. Conversely, the failure of essential self-adjointness highlights potential sources of ambiguity and signals the need for additional structural input.

Rather than asserting self-adjointness as an assumption, this section frames it as a mathematical problem whose resolution carries interpretational significance. Both positive and negative outcomes are accom-

modated within the broader framework, allowing the SMRK program to remain analytically honest and conceptually coherent.

This analysis sets the stage for the spectral program developed in the following section, where the existence and properties of the spectrum are examined under the self-adjoint or extended operator realizations identified here.

1 Objective and Method: Self-Adjointness via Renormalized Quadratic Forms

The purpose of this chapter is to construct a self-adjoint realization of the SMRK Hamiltonian, defined formally on arithmetic states $\psi : \mathbb{N} \rightarrow \mathbb{C}$ by

$$(H_{\text{SMRK}}\psi)(n) = \sum_{p \in \mathcal{P}} \frac{1}{p} (\psi(pn) + \mathbf{1}_{p|n} \psi(n/p)) + (\alpha \Lambda(n) + \beta \log n) \psi(n), \quad (9.1)$$

where $\mathcal{P}$ denotes the set of prime numbers and Λ is the von Mangoldt function.

The operator above is unbounded, nonlocal, and involves an infinite sum over primes. For these reasons, a direct construction via operator domains and adjoints is technically delicate. Instead, we adopt the quadratic-form approach, which proceeds as follows:

1. Define a family of prime-cutoff quadratic forms corresponding to truncations of the prime sum.
2. Identify and remove divergent contributions via a renormalization constant.
3. Establish uniform lower bounds and form-relative boundedness.
4. Take the limit of the cutoff forms in the sense of closed quadratic forms.
5. Invoke the representation theorem for closed semi-bounded forms to obtain a unique self-adjoint operator.

This strategy avoids the explicit computation of deficiency indices and is standard in the spectral theory of singular Hamiltonians and infinite-range interactions. :contentReference[oaicite:0]index=0

2 Arithmetic Hilbert Space and Prime Shift Operators

2.1 Arithmetic Hilbert space

We work on the arithmetic Hilbert space

$$\mathcal{H}_{\text{crit}} := \ell^2(\mathbb{N}, 1/n),$$

consisting of all complex-valued functions $\psi : \mathbb{N} \rightarrow \mathbb{C}$ such that

$$\|\psi\|^2 = \sum_{n=1}^{\infty} \frac{|\psi(n)|^2}{n} < \infty.$$

This weighted space is natural from an arithmetic perspective, as it is invariant under multiplicative scaling and logarithmic potentials. It provides the correct balance between ultraviolet growth at large n and summability of arithmetic states.

The inner product is given by

$$\langle \psi, \phi \rangle = \sum_{n=1}^{\infty} \frac{\overline{\psi(n)} \phi(n)}{n}.$$

2.2 Prime shift operators

For each prime number p , define the forward prime shift operator S_p by

$$(S_p \psi)(n) := \psi(pn).$$

This operator implements multiplication by p on the arithmetic index set. A direct computation shows that S_p is bounded on $\mathcal{H}_{\text{crit}}$, with operator norm

$$\|S_p\| = p^{-1/2}.$$

The adjoint of S_p with respect to the weighted inner product is given by the backward shift

$$(S_p^* \psi)(n) = \mathbf{1}_{p|n} \psi(n/p),$$

where $\mathbf{1}_{p|n}$ denotes the indicator function of divisibility by p . This formula follows from a direct change of variables in the defining inner product and reflects the partial invertibility of multiplication by p on $\mathbb{N}$.

2.3 Symmetrized prime transitions

The elementary building block of the kinetic part of the SMRK Hamiltonian is the symmetrized prime transition operator

$$K_p := S_p + S_p^*.$$

Each K_p is symmetric on $\mathcal{H}_{\text{crit}}$ and bounded with

$$\|K_p\| \leq 2p^{-1/2}.$$

The decay of these norms with p is crucial: while the sum over all primes diverges, it does so in a controlled manner that can be handled by quadratic-form renormalization.

2.4 Diagonal arithmetic potentials

In addition to prime shifts, we introduce diagonal multiplication operators. For any real-valued function $V : \mathbb{N} \rightarrow \mathbb{R}$, define

$$(V\psi)(n) := V(n)\psi(n),$$

with maximal domain

$$D(V) := \{\psi \in \mathcal{H}_{\text{crit}} : V\psi \in \mathcal{H}_{\text{crit}}\}.$$

Of particular importance are the arithmetic potentials

$$V_{\Lambda}(n) := \Lambda(n), \quad V_{\log}(n) := \log n,$$

where Λ denotes the von Mangoldt function. These potentials are unbounded but relatively form-bounded with respect to the kinetic terms defined below.

2.5 Formal SMRK Hamiltonian

Combining prime transitions and diagonal arithmetic potentials, the SMRK Hamiltonian is formally written as

$$H_{\text{SMRK}} = \sum_{p \in \mathcal{P}} \frac{1}{p} K_p + \alpha V_\Lambda + \beta V_{\log}.$$

This expression is purely formal: neither the prime sum nor the diagonal terms define a bounded operator on $\mathcal{H}_{\text{crit}}$ when taken together. Nevertheless, it provides the guiding structure for the quadratic-form construction developed in subsequent chapters.

2.6 Dense core

Let $\mathcal{D}_0 \subset \mathcal{H}_{\text{crit}}$ denote the space of finitely supported arithmetic functions. This space is dense in $\mathcal{H}_{\text{crit}}$ and invariant under all prime shifts S_p , adjoints S_p^* , and diagonal multiplication operators. All quadratic forms and operator identities introduced later will be initially defined on $\mathcal{D}_0$ and extended by closure. This choice of core ensures that all expressions are well-defined at the algebraic level and avoids domain pathologies at early stages of the construction.

2.7 Outlook

The operators introduced in this chapter define the arithmetic building blocks of the SMRK Hamiltonian. In the next chapter we introduce prime-cutoff quadratic forms, identify their divergences, and formulate a renormalization scheme that leads to a closed, semi-bounded form in the infinite-prime limit.

3 Prime-Cutoff Quadratic Forms

3.1 Motivation for prime cutoffs

The formal expression for the SMRK Hamiltonian involves an infinite sum over all prime numbers. While each individual prime transition operator is bounded, the sum over primes diverges and therefore does not define a bounded or even densely-defined operator on the arithmetic Hilbert space.

To control this divergence, we introduce a family of cutoff Hamiltonians in which the prime sum is truncated at a finite scale. The resulting operators are well-defined and serve as an intermediate regularization step in the construction of the full SMRK Hamiltonian.

3.2 Definition of the cutoff kinetic form

Let $P \geq 2$ be a cutoff parameter. We define the truncated kinetic operator by

$$K^{(P)} := \sum_{\substack{p \in \mathcal{P} \\ p \leq P}} \frac{1}{p} K_p,$$

where $K_p = S_p + S_p^*$ as introduced previously.

On the dense core $\mathcal{D}_0$, this operator induces a quadratic form

$$q_{\text{kin}}^{(P)}[\psi] := \langle \psi, K^{(P)} \psi \rangle = \sum_{p \leq P} \frac{1}{p} \langle \psi, K_p \psi \rangle.$$

Each term in the sum is finite, and the truncated form is well-defined for all $\psi \in \mathcal{D}_0$.

3.3 Explicit expression of the quadratic form

Using the definitions of the prime shift operators, the kinetic form can be written explicitly as

$$q_{\text{kin}}^{(P)}[\psi] = \sum_{p \leq P} \frac{1}{p} \sum_{n=1}^{\infty} \frac{1}{n} \left(\overline{\psi(n)} \psi(pn) + \mathbf{1}_{p|n} \overline{\psi(n)} \psi(n/p) \right).$$

The two terms correspond respectively to forward and backward prime transitions. By symmetry of the inner product, the quadratic form is real-valued.

3.4 Divergence of the cutoff limit

As the cutoff $P \rightarrow \infty$, the kinetic form diverges. Indeed, for fixed $\psi \in \mathcal{D}_0$,

$$q_{\text{kin}}^{(P)}[\psi] \sim 2 \left(\sum_{p \leq P} \frac{1}{p} \right) \|\psi\|^2 \quad \text{as } P \rightarrow \infty.$$

Since the prime harmonic series diverges logarithmically,

$$\sum_{p \leq P} \frac{1}{p} = \log \log P + O(1),$$

the quadratic form grows without bound. This divergence is uniform and purely diagonal, suggesting that it can be removed by a suitable renormalization subtraction.

3.5 Full cutoff quadratic form

Including the diagonal arithmetic potentials introduced earlier, we define the full cutoff quadratic form by

$$q^{(P)}[\psi] = q_{\text{kin}}^{(P)}[\psi] + \alpha \langle \psi, V_{\Lambda} \psi \rangle + \beta \langle \psi, V_{\log} \psi \rangle, \quad \psi \in \mathcal{D}_0.$$

Each of the diagonal terms is well-defined on $\mathcal{D}_0$, and the form $q^{(P)}$ is symmetric and semi-bounded from below for fixed P .

3.6 Need for renormalization

The divergence of $q^{(P)}$ as $P \rightarrow \infty$ prevents the direct definition of a limiting quadratic form. However, the structure of the divergence is simple: it is proportional to the identity and does not depend on the arithmetic structure of ψ .

This observation motivates the introduction of a renormalized quadratic form in which the divergent contribution is subtracted explicitly. The construction of this renormalization and the proof of convergence of the resulting forms is the subject of the next chapter.

3.7 Outlook

In the following chapter we identify a canonical renormalization constant, define the renormalized quadratic forms, and establish their closedness and lower semiboundedness. This will allow us to pass to the infinite-prime limit and construct the SMRK Hamiltonian as a self-adjoint operator.

4 Renormalized Quadratic Forms and Logarithmic Confinement

4.1 Structure of the Divergence

In the previous chapter we identified the divergence of the cutoff quadratic forms q_P as the prime cutoff P tends to infinity. The divergent contribution arises from large primes and is controlled by estimates of the form

$$\frac{1}{p} |\langle \psi, S_p \psi \rangle| \lesssim \frac{1}{\sqrt{p}} \|\psi\|^2. \quad (9.2)$$

Summation over primes therefore produces a divergence proportional to

$$\sum_{p \in \mathbb{P}} p^{-1/2}. \quad (9.3)$$

Crucially, this divergence is:

- independent of arithmetic correlations,
- proportional to the squared norm of the state,
- universal across states in the form domain.

Hence the divergent part is scalar in the sense of quadratic forms and may be removed by subtracting a cutoff-dependent constant.

4.2 Choice of the Renormalization Constant

Let $P \geq 2$ denote the prime cutoff. We define the renormalization constant by

$$C(P) := \sum_{\substack{p \in \mathbb{P} \\ p \leq P}} \frac{1}{\sqrt{p}}. \quad (9.4)$$

Although $C(P) \rightarrow \infty$ as $P \rightarrow \infty$, it grows sublinearly with P and depends only on the cutoff parameter. We define the renormalized quadratic form

$$\tilde{q}_P(\psi, \phi) := q_P(\psi, \phi) - C(P) \langle \psi, \phi \rangle, \quad (9.5)$$

initially on the finite-support domain.

Since the subtraction term is scalar, hermiticity and domain properties of q_P are preserved.

4.3 Reference Quadratic Form and Confinement

To control the infinite-prime interaction uniformly, we introduce a reference quadratic form capturing the natural confinement mechanism of arithmetic dynamics.

Define

$$q_0[\psi] := \sum_{n \geq 1} (1 + \log^2 n) |\psi(n)|^2 \frac{1}{n}. \quad (9.6)$$

The associated form domain is

$$D(q_0) := \left\{ \psi \in \mathcal{H} \left| \sum_{n \geq 1} (1 + \log^2 n) |\psi(n)|^2 \frac{1}{n} < \infty \right. \right\}. \quad (9.7)$$

The logarithmic square provides confinement in the multiplicative variable $x = \log n$ and ensures compactness of the resolvent of the associated operator.

4.4 Decomposition of the Renormalized Form

We decompose the renormalized form as

$$\tilde{q}_P = q_0 + b_P, \quad (9.8)$$

where the interaction form b_P is given by

$$\begin{aligned} b_P(\psi, \phi) := & \sum_{\substack{p \in \mathbb{P} \\ p \leq P}} \frac{1}{p} (\langle S_p \psi, \phi \rangle + \langle T_p \psi, \phi \rangle) \\ & + \langle (V - 1 - \log^2 n) \psi, \phi \rangle - C(P) \langle \psi, \phi \rangle. \end{aligned} \quad (9.9)$$

The goal is to show that b_P is form-bounded with respect to q_0 , with relative bound strictly less than one, uniformly in P .

4.5 Control of Prime Shifts in the Reference Norm

Let $\|\cdot\|_{q_0}$ denote the form norm induced by q_0 . For ψ of finite support, we estimate

$$q_0[S_p \psi] = \sum_{n \geq 1} (1 + \log^2 n) |\psi(pn)|^2 \frac{1}{n}. \quad (9.10)$$

Changing variables $m = pn$ yields

$$q_0[S_p \psi] = p \sum_{m \geq 1} (1 + \log^2(m/p)) |\psi(m)|^2 \frac{1}{m}. \quad (9.11)$$

Using elementary logarithmic inequalities, one obtains

$$q_0[S_p \psi] \leq 2p q_0[\psi] + 2p \log^2 p \|\psi\|^2. \quad (9.12)$$

An analogous estimate holds for the co-shift operator T_p .

4.6 Form-Boundedness of the Interaction

Applying the Cauchy–Schwarz inequality in the q_0 -norm, we obtain

$$|\langle \psi, S_p \psi \rangle| \leq \|\psi\|_{q_0} \|S_p \psi\|_{q_0}. \quad (9.13)$$

Combining this with the previous bounds yields, for any $\varepsilon > 0$,

$$\frac{1}{p} |\langle \psi, S_p \psi \rangle| \leq \varepsilon q_0[\psi] + K_\varepsilon \frac{\log^2 p}{p} \|\psi\|^2, \quad (9.14)$$

with a constant K_ε independent of P .

Summation over primes and subtraction of $C(P)$ therefore imply a uniform bound

$$|b_P(\psi, \psi)| \leq \varepsilon q_0[\psi] + K_\varepsilon \|\psi\|^2, \quad (9.15)$$

valid for all ψ in the finite-support domain and uniformly in P .

4.7 Consequences

The uniform form-boundedness established above implies that the family of renormalized quadratic forms $\tilde{q}_P$ is closed, semi-bounded, and defined on a common form domain.

This provides the key analytic control required to take the infinite-prime limit and construct a self-adjoint realization of the SMRK Hamiltonian.

4.8 Summary

In this chapter we have:

- identified the divergent structure of the prime interaction,
- introduced a renormalization constant removing the divergence,
- constructed a logarithmically confining reference form,
- proved uniform form-boundedness of the interaction,
- established closedness and semi-boundedness of the renormalized forms.

The next chapter takes the infinite-prime limit and proves the existence and uniqueness of the self-adjoint SMRK Hamiltonian.

5 The Self-Adjoint SMRK Hamiltonian

5.1 Representation theorem for quadratic forms

Let $\tilde{q}$ denote the closed, densely defined, lower semi-bounded quadratic form constructed in the previous chapter on the Hilbert space $\mathcal{H}_{\text{crit}}$. By the representation theorem for closed semi-bounded quadratic forms, there exists a unique self-adjoint operator $\tilde{H}_{\text{SMRK}}$ such that

$$\tilde{q}[\psi] = \langle \psi, \tilde{H}_{\text{SMRK}} \psi \rangle \quad \text{for all } \psi \in D(\tilde{H}_{\text{SMRK}}),$$

where $D(\tilde{H}_{\text{SMRK}})$ denotes the operator domain associated with the form.

This theorem provides the rigorous realization of the SMRK Hamiltonian and constitutes the central existence result of the present work.

5.2 Characterization of the operator domain

The domain $D(\tilde{H}_{\text{SMRK}})$ consists of all $\psi \in \mathcal{H}_{\text{crit}}$ for which there exists $\phi \in \mathcal{H}_{\text{crit}}$ satisfying

$$\tilde{q}(\psi, \eta) = \langle \phi, \eta \rangle \quad \text{for all } \eta \in D(\tilde{q}).$$

In this case, $\tilde{H}_{\text{SMRK}}\psi = \phi$. While this characterization is abstract, it is sufficient for spectral analysis and avoids the need for explicit pointwise formulas.

5.3 Formal action of the operator

On the dense core $\mathcal{D}_0$ of finitely supported arithmetic states, the operator $\tilde{H}_{\text{SMRK}}$ acts formally as

$$(\tilde{H}_{\text{SMRK}}\psi)(n) = \sum_{p \in \mathcal{P}} \frac{1}{p} (\psi(pn) + \mathbf{1}_{p|n} \psi(n/p)) + \alpha \Lambda(n) \psi(n) + \beta \log n \psi(n) - c(\infty) \psi(n),$$

where the constant $c(\infty)$ represents the infinite-cutoff limit of the renormalization constant. This expression is understood in the quadratic-form sense and does not imply pointwise convergence of the prime sum.

5.4 Lower semiboundedness

By construction, the operator $\tilde{H}_{\text{SMRK}}$ is bounded from below. There exists a constant $E_0 \in \mathbb{R}$ such that

$$\langle \psi, \tilde{H}_{\text{SMRK}} \psi \rangle \geq E_0 \|\psi\|^2 \quad \text{for all } \psi \in D(\tilde{H}_{\text{SMRK}}).$$

Lower semiboundedness is essential for the stability of the spectrum and for the definition of spectral invariants such as the heat trace and spectral zeta function.

5.5 Uniqueness of the self-adjoint realization

The self-adjoint operator $\tilde{H}_{\text{SMRK}}$ is uniquely determined by the renormalized quadratic form. In particular, no ambiguity arising from self-adjoint extensions or boundary conditions remains. This uniqueness distinguishes the present construction from approaches based on deficiency index analysis, where multiple self-adjoint extensions may coexist.

5.6 Relation to the formal SMRK Hamiltonian

The operator $\tilde{H}_{\text{SMRK}}$ provides a rigorous realization of the formal Hamiltonian introduced earlier. All algebraic identities involving prime shifts and arithmetic potentials are valid when interpreted in the quadratic-form sense on the core $\mathcal{D}_0$.

Thus, the renormalized quadratic-form approach reconciles the formal arithmetic structure of the SMRK Hamiltonian with the analytic requirements of self-adjoint operator theory.

5.7 Stability under perturbations

Small perturbations of the diagonal arithmetic potentials or of the coefficients α and β lead to form-bounded perturbations of $\tilde{q}$. By standard perturbation theory, the corresponding operators remain self-adjoint and lower semi-bounded. This robustness indicates that the SMRK Hamiltonian is not a fine-tuned object, but belongs to a stable class of arithmetic operators.

5.8 Outlook

Having established the existence and uniqueness of the self-adjoint SMRK Hamiltonian, we now turn to the analysis of its spectral properties. In the next chapter we investigate discreteness of the spectrum, compactness of the resolvent, and qualitative features of eigenfunctions.

6 Spectral Properties of the SMRK Hamiltonian

6.1 Preliminaries

Let $\tilde{H}_{\text{SMRK}}$ denote the self-adjoint SMRK Hamiltonian constructed in the previous chapter. By construction, this operator is bounded from below and is associated with the closed quadratic form $\tilde{q}$ defined on the form domain of the reference quadratic form.

Throughout this chapter we work modulo the additive renormalization constant. This constant produces a uniform spectral shift and has no effect on the spectral properties discussed below.

6.2 Compactness of the Resolvent

A key structural feature of $\tilde{H}_{\text{SMRK}}$ is the logarithmic confinement induced by the reference quadratic form

$$q_0[\psi] = \sum_{n \geq 1} (1 + \log^2 n) |\psi(n)|^2 \frac{1}{n}. \quad (9.16)$$

Let H_0 denote the self-adjoint operator associated with q_0 . Since the function $1 + \log^2 n$ diverges as $n \rightarrow \infty$, the operator $(H_0 + i)^{-1}$ is compact on the arithmetic Hilbert space.

The interaction $\tilde{H}_{\text{SMRK}} - H_0$ is relatively form-bounded with respect to H_0 with relative bound zero. Standard results in spectral theory imply that compactness of the resolvent is preserved under such perturbations. Consequently, the resolvent

$$(\tilde{H}_{\text{SMRK}} + i)^{-1} \quad (9.17)$$

is a compact operator.

6.3 Pure Point Spectrum

Compactness of the resolvent imposes strong restrictions on the spectrum of $\tilde{H}_{\text{SMRK}}$.

The spectrum consists entirely of isolated eigenvalues of finite multiplicity, with no continuous or residual spectrum. Moreover, the eigenvalues diverge to positive infinity.

Thus the spectrum is purely discrete and may be written as a sequence

$$\sigma(\tilde{H}_{\text{SMRK}}) = \{\lambda_k\}_{k \geq 1}, \quad \lambda_k \rightarrow +\infty \text{ as } k \rightarrow \infty. \quad (9.18)$$

6.4 Eigenfunction Regularity and Localization

Let ψ_k denote a normalized eigenfunction corresponding to the eigenvalue λ_k :

$$\tilde{H}_{\text{SMRK}} \psi_k = \lambda_k \psi_k. \quad (9.19)$$

Since ψ_k belongs to the operator domain of $\tilde{H}_{\text{SMRK}}$, it lies in the form domain of q_0 . Consequently,

$$\sum_{n \geq 1} \log^2 n |\psi_k(n)|^2 \frac{1}{n} < \infty. \quad (9.20)$$

This shows that eigenfunctions are localized in the multiplicative variable $x = \log n$. Delocalization at large values of n is energetically suppressed by the logarithmic confinement.

6.5 Growth of Eigenvalues

The confining reference operator H_0 provides a comparison principle for the asymptotic growth of the eigenvalues of $\tilde{H}_{\text{SMRK}}$.

Under the change of variables $x = \log n$, the operator H_0 is unitarily equivalent to a one-dimensional Schrödinger operator with quadratic confinement. Standard Weyl-type asymptotics therefore apply.

As a consequence, there exist positive constants c_1 and c_2 such that

$$c_1 k^2 \leq \lambda_k \leq c_2 k^2 \quad (9.21)$$

for all sufficiently large k . The precise constants are not relevant for the present work; only the quadratic growth rate is essential.

6.6 Spectral Decomposition

Since $\tilde{H}_{\text{SMRK}}$ has purely discrete spectrum, the spectral theorem yields the expansion

$$\tilde{H}_{\text{SMRK}} = \sum_{k \geq 1} \lambda_k |\psi_k\rangle \langle \psi_k|, \quad (9.22)$$

with convergence in the strong operator topology.

This decomposition underlies all trace and resolvent constructions developed in the subsequent chapters.

6.7 Spectral Projections and Counting Function

Define the eigenvalue counting function by

$$N(E) := \#\{k : \lambda_k \leq E\}. \quad (9.23)$$

Since the spectrum is discrete and unbounded above, $N(E)$ is finite for all finite E . Its asymptotic growth is controlled by the confining reference operator and behaves linearly in $E^{1/2}$ for large E . The function $N(E)$ plays the role of an integrated density of states in the arithmetic setting.

6.8 Consequences for Trace Objects

Compactness of the resolvent implies that for a broad class of test functions f , the operator $f(\tilde{H}_{\text{SMRK}})$ is trace class. In particular,

$$\text{Tr } f(\tilde{H}_{\text{SMRK}}) = \sum_{k \geq 1} f(\lambda_k) \quad (9.24)$$

is well defined.

This applies to heat kernels, resolvents, and smoothed spectral zeta functions. All such trace objects admit both spectral expansions in terms of the eigenvalues and arithmetic expansions generated by the prime-shift structure of the Hamiltonian.

6.9 Summary

In this chapter we have established that:

- the SMRK Hamiltonian has compact resolvent,
- its spectrum is purely discrete,
- eigenfunctions are logarithmically localized,
- eigenvalues exhibit quadratic growth,
- trace-class spectral objects are well defined.

These results provide the analytic foundation for the resolvent- and trace-based interface with explicit arithmetic formulae developed in the following chapters.

7 Trace Formulas and Spectral Invariants

7.1 Motivation for trace invariants

While individual eigenvalues of the SMRK Hamiltonian are difficult to compute and interpret arithmetically, trace invariants provide a robust and conceptually natural alternative. Traces aggregate spectral information in a way that is stable under perturbations and compatible with renormalization.

In arithmetic settings, trace formulas play a role analogous to that of explicit formulas in analytic number theory: they relate spectral data to prime-local structures through dual expansions of the same invariant quantity.

7.2 Heat kernel and heat trace

Let $t > 0$. Since the SMRK Hamiltonian $\tilde{H}_{\text{SMRK}}$ is self-adjoint and lower semi-bounded with compact resolvent, the heat operator

$$e^{-t\tilde{H}_{\text{SMRK}}}$$

is trace class for all $t > 0$.

The heat trace is defined by

$$\Theta(t) := \text{Tr}(e^{-t\tilde{H}_{\text{SMRK}}}) = \sum_{k=1}^{\infty} e^{-tE_k},$$

where $\{E_k\}$ denotes the discrete spectrum of $\tilde{H}_{\text{SMRK}}$. This function encodes global spectral information and serves as a primary spectral invariant.

7.3 Short-time behavior

As $t \rightarrow 0^+$, the heat trace exhibits divergent behavior reflecting the high energy density of states. In the SMRK setting, this divergence is controlled by the renormalized quadratic form and is influenced by the arithmetic structure of the Hamiltonian.

The leading short-time asymptotics of $\Theta(t)$ are determined by the diagonal logarithmic potential and are expected to follow a power-law behavior modulated by logarithmic corrections. Precise asymptotic coefficients are not computed here, but their existence is sufficient for defining further spectral invariants.

7.4 Spectral zeta function

Associated with the heat trace is the spectral zeta function

$$\zeta_{\text{spec}}(s) := \sum_{k=1}^{\infty} E_k^{-s},$$

defined initially for $\Re(s)$ sufficiently large. Using the Mellin transform of the heat trace, this function admits the integral representation

$$\zeta_{\text{spec}}(s) = \frac{1}{\Gamma(s)} \int_0^{\infty} t^{s-1} \Theta(t) dt,$$

which provides analytic continuation to a meromorphic function in the complex plane.

7.5 Determinants and regularized products

The spectral zeta function allows the definition of a zeta-regularized determinant of the SMRK Hamiltonian by

$$\det_{\zeta}(\tilde{H}_{\text{SMRK}}) := \exp(-\zeta'_{\text{spec}}(0)),$$

provided that $\zeta_{\text{spec}}(s)$ is regular at $s = 0$. Such determinants encode global spectral information and play an important role in quantum field theory and spectral geometry.

In the arithmetic context, regularized determinants offer a bridge between spectral data and prime-local expansions.

7.6 Prime-side expansion of trace invariants

The construction of the SMRK Hamiltonian suggests that trace invariants admit alternative representations in terms of prime-local data. Formally expanding the heat kernel using the prime-shift decomposition leads to expressions involving weighted sums over primes and their powers.

Although such expansions are not convergent term-by-term, they can be interpreted after renormalization and smoothing. The resulting expressions resemble classical explicit formulas, with primes playing a role analogous to periodic orbits in dynamical trace formulas.

7.7 Spectral-side versus prime-side duality

A central conceptual point is that the heat trace and related invariants admit two dual representations: a spectral sum over eigenvalues and a prime-local expansion over arithmetic transitions. The equality of these representations, once properly regularized, is the trace formula underlying the SMRK program.

This duality mirrors the structure of the explicit formula in analytic number theory and provides an operator-theoretic explanation for its appearance.

7.8 Stability and renormalization

Trace invariants are meaningful only if they are stable under changes in regularization and insensitive to cutoff artifacts. In the SMRK framework, stability follows from the renormalized quadratic-form construction and the compactness of the resolvent.

Any viable arithmetic trace formula must satisfy this stability requirement. Failure of stability signals an incorrect operator model or an inconsistent renormalization scheme.

7.9 Outlook

Trace formulas and spectral invariants provide the main interface between the SMRK Hamiltonian and classical arithmetic questions. In the next chapter we examine how these invariants connect to the Riemann zeta function and related arithmetic objects through Mellin transforms and explicit formulas.

8 Connection to the Riemann Zeta Function

8.1 Motivation and perspective

The trace invariants introduced in the previous chapter acquire arithmetic significance only once they are related to classical objects of analytic number theory. Among these, the Riemann zeta function occupies a central role. The aim of this chapter is not to identify eigenvalues of the SMRK Hamiltonian with zeros of $\zeta(s)$, but to explain how the zeta function emerges naturally from trace invariants through integral transforms and prime-local expansions.

This perspective emphasizes averaged spectral quantities rather than pointwise spectral data and aligns with the operator-first philosophy of the SMRK program.

8.2 Mellin transform of the heat trace

Let $\Theta(t)$ denote the heat trace of the SMRK Hamiltonian. Consider its Mellin transform

$$\mathcal{M}\Theta(s) := \int_0^\infty t^{s-1} \Theta(t) dt,$$

which converges for $\Re(s)$ sufficiently large after subtraction of the leading short-time divergences.

Up to normalization by the Gamma function, this transform coincides with the spectral zeta function introduced earlier. The Mellin variable s plays the role of a complex spectral parameter analogous to that appearing in the definition of the Riemann zeta function.

8.3 Emergence of the Euler product

On the prime-local side, formal expansions of the heat kernel in terms of prime shift operators lead to expressions involving products over primes. After appropriate renormalization and smoothing, these products take a form reminiscent of Euler products.

Schematically, one obtains expressions of the form

$$\exp \left(\sum_p \sum_{k \geq 1} \frac{1}{k} a_{p,k}(t) \right),$$

where the coefficients $a_{p,k}(t)$ encode weighted contributions of prime powers. Under Mellin transformation, such expressions give rise to logarithmic derivatives of Euler products, thereby connecting trace invariants to $\log \zeta(s)$ and its derivatives.

8.4 Von Mangoldt terms

The appearance of the von Mangoldt function in the diagonal potential of the SMRK Hamiltonian ensures that prime powers contribute with the correct arithmetic weights. In Mellin space, these contributions manifest themselves as sums of the form

$$\sum_{n=1}^{\infty} \frac{\Lambda(n)}{n^s},$$

which are well known to coincide with $-\zeta'(s)/\zeta(s)$ for $\Re(s) > 1$.

Thus, the arithmetic potential built into the operator construction directly feeds into the prime-side structure of the trace invariants and reproduces the classical logarithmic derivative of the zeta function.

8.5 Spectral interpretation of the explicit formula

Combining the spectral-side representation of trace invariants with their prime-side expansions yields identities that parallel the explicit formula of analytic number theory. Zeros of the zeta function appear not as individual eigenvalues, but as oscillatory components in the spectral representation of the trace.

From this viewpoint, the explicit formula is interpreted as a statement about the equality of two representations of the same renormalized trace invariant, rather than as a direct spectral identity.

8.6 Critical line considerations

The SMRK framework does not require a priori assumptions about the location of zeros of the zeta function. However, the symmetry properties of the Hamiltonian and of the associated trace invariants suggest a natural reflection symmetry in Mellin space corresponding to $s \leftrightarrow 1 - s$.

This symmetry is inherited from the arithmetic structure of the operator and provides a conceptual explanation for the appearance of the critical line in explicit formulas, without asserting a direct identification of zeros with eigenvalues.

8.7 Analytic continuation and regularization

Both the spectral zeta function and the Riemann zeta function require analytic continuation beyond their initial domains of convergence. In the SMRK setting, analytic continuation arises from the subtraction of short-time divergences in the heat trace and from the renormalized definition of trace invariants.

This parallel highlights the role of renormalization as a unifying mechanism in both spectral theory and analytic number theory.

8.8 Limitations of the correspondence

It is important to emphasize the limitations of the correspondence described here. The SMRK Hamiltonian does not provide a direct spectral realization of the zeros of the Riemann zeta function. Instead, it offers a framework in which arithmetic information encoded by the zeta function emerges naturally at the level of trace invariants.

This distinction avoids overinterpretation of the operator model and clarifies the precise scope of the claims being made.

8.9 Outlook

In the next chapter we examine the role of symmetry and functional equations in greater detail. We show how reflection symmetries of trace invariants mirror the functional equation of the Riemann zeta function and constrain admissible operator models.

9 Functional Equation and Symmetry

9.1 Symmetry as a structural principle

One of the most striking features of the Riemann zeta function is the existence of a functional equation relating values at s and $1 - s$. Any operator-theoretic framework intended to interface

with arithmetic explicit formulas must account for this symmetry at a structural level.

In the SMRK program, symmetry is not imposed post hoc on spectral data, but is built into the operator construction and the associated trace invariants. This chapter analyzes how functional-equation-type symmetries emerge naturally from the arithmetic structure of the Hamiltonian.

9.2 Reflection symmetry in Mellin space

Trace invariants associated with the SMRK Hamiltonian admit representations in Mellin space via integral transforms of the heat trace. In this representation, the natural involution $s \mapsto 1 - s$ arises as a reflection symmetry of the Mellin variable.

This symmetry is linked to the logarithmic weighting of arithmetic states and to the invariance of the underlying Hilbert space under multiplicative inversion at the level of scales. As a result, trace expressions exhibit paired contributions at s and $1 - s$, mirroring the structure of the functional equation.

9.3 Prime-shift symmetry

At the operator level, symmetry manifests itself through the pairing of forward and backward prime shifts. The operators S_p and S_p^* appear on equal footing in the symmetrized kinetic terms of the SMRK Hamiltonian.

This pairing ensures that prime multiplication and division contribute symmetrically to trace invariants. It is this symmetry that ultimately gives rise to balanced prime-side expansions and prevents the emergence of spurious asymmetries in the arithmetic trace formulas.

9.4 Diagonal potentials and scale duality

The diagonal arithmetic potentials, particularly the logarithmic term $\log n$, play a crucial role in implementing scale duality. Under Mellin transformation, multiplication by $\log n$ corresponds to differentiation with respect to the spectral parameter s .

This correspondence is essential for reproducing the structure of the functional equation and for ensuring that renormalized trace invariants transform covariantly under $s \mapsto 1 - s$.

9.5 Symmetry of the renormalization scheme

Renormalization is meaningful only if it preserves the symmetries of the underlying problem. In the SMRK framework, the subtraction of divergent terms is designed to be invariant under the same involutions that govern the trace invariants themselves.

Any renormalization scheme that breaks reflection symmetry would lead to inconsistent trace identities and is therefore excluded. This requirement significantly restricts admissible subtraction procedures and reinforces the rigidity of the framework.

9.6 Consequences for trace invariants

As a result of the symmetries described above, the trace invariants associated with the SMRK Hamiltonian satisfy functional relations analogous to those of the Riemann zeta function. While these relations do not imply a direct spectral realization of zeta zeros, they ensure that trace formulas respect the same global constraints as the classical explicit formula.

Symmetry thus acts as a consistency condition linking operator construction, renormalization, and arithmetic interpretation.

9.7 Comparison with classical functional equations

In classical analytic number theory, the functional equation of the zeta function is often derived using Poisson summation or theta-function identities. The SMRK approach replaces these analytic tools with operator-theoretic symmetry principles.

This shift in perspective highlights the universality of functional equations as manifestations of deeper symmetries, rather than as isolated analytic accidents.

9.8 Limitations and scope

The symmetry considerations presented here constrain the structure of admissible operators and trace invariants, but they do not by themselves determine the spectrum of the SMRK Hamiltonian. Additional analytic input would be required to extract fine-grained information about zeros or to establish stronger arithmetic consequences.

The role of symmetry in the SMRK program is therefore structural rather than deterministic.

9.9 Outlook

In the final chapter we summarize the construction and its implications, clarify what has and has not been established, and outline directions for future work and potential falsification of the SMRK framework.

10 Conclusion and Future Directions

10.1 Summary of the construction

In this work we have constructed a self-adjoint realization of the SMRK Hamiltonian acting on an arithmetic Hilbert space. The operator arises from symmetrized prime shift interactions combined with diagonal arithmetic potentials and is defined rigorously through a renormalized quadratic-form approach.

The construction proceeds in a sequence of conceptually transparent steps: prime cutoffs regularize the kinetic terms, divergent contributions are removed by a canonical subtraction, and the resulting closed quadratic form uniquely determines a self-adjoint operator. This approach avoids ambiguities associated with self-adjoint extensions and ensures stability under perturbations.

10.2 Spectral and arithmetic features

The resulting SMRK Hamiltonian is lower semi-bounded and has compact resolvent, implying a purely discrete spectrum. While individual eigenvalues are not identified with arithmetic quantities, the operator admits a rich family of trace invariants that encode arithmetic information in an averaged and renormalization-stable manner.

These trace invariants exhibit a dual structure: on the spectral side they are expressed as sums over eigenvalues, while on the arithmetic side they admit expansions involving primes and prime powers. This duality mirrors the structure of explicit formulas in analytic number theory.

10.3 Relation to the Riemann zeta function

A central outcome of the analysis is the natural emergence of the Riemann zeta function and its logarithmic derivative from spectral invariants of the SMRK Hamiltonian. This emergence occurs through Mellin transforms of the heat trace and through the presence of the von Mangoldt function in the diagonal arithmetic potential.

The correspondence is structural rather than literal: zeros of the zeta function are not claimed to coincide with eigenvalues of the Hamiltonian. Instead, the operator framework reproduces the architecture of the explicit formula at the level of renormalized trace identities.

10.4 Symmetry and functional equations

Symmetry plays a central role throughout the construction. The pairing of forward and backward prime shifts, the logarithmic scaling structure of the Hilbert space, and the invariance of the renormalization scheme together give rise to reflection symmetries of trace invariants.

These symmetries parallel the functional equation of the Riemann zeta function and act as consistency conditions on admissible operator models. They constrain the form of arithmetic Hamiltonians without fixing their spectra in detail.

10.5 Limitations of the present approach

The SMRK Hamiltonian does not provide a proof of the Riemann Hypothesis, nor does it offer a direct spectral realization of zeta zeros. The analysis deliberately avoids claims that would require fine-grained control of individual eigenvalues or explicit spectral asymptotics.

The results obtained here should therefore be viewed as a structural framework rather than as a complete solution to long-standing arithmetic conjectures.

10.6 Directions for future work

Several directions for further investigation suggest themselves. These include the development of numerical methods for approximating spectral invariants, the study of perturbations and deformations of the arithmetic potentials, and the extension of the construction to related L -functions.

Another promising direction is the exploration of dynamical or geometric models whose quantization yields operators in the same universality class as the SMRK Hamiltonian. Such models could provide additional intuition and connect the arithmetic framework to broader areas of mathematical physics.

10.7 Falsifiability and robustness

An important feature of the SMRK framework is its falsifiability. Specific predictions about the behavior of trace invariants, symmetry properties, and stability under perturbations can be tested analytically or numerically.

Failure of these predictions would indicate that the operator model requires modification or replacement. Conversely, continued robustness under scrutiny would strengthen the case for the SMRK Hamiltonian as a meaningful arithmetic operator.

10.8 Final remarks

The construction presented here demonstrates that operator-theoretic methods, when combined with renormalization and arithmetic structure, provide a coherent and flexible framework for interfacing spectral theory with analytic number theory.

Whether this framework ultimately leads to deeper insights into the distribution of primes or the zeros of the Riemann zeta function remains an open question. Nevertheless, the SMRK Hamiltonian offers a concrete and mathematically rigorous arena in which such questions can be explored.

APPENDIX A

A.1 Technical Remarks and Supplementary Material

This appendix collects technical remarks, clarifications, and auxiliary constructions that support the main text but are not essential for its logical flow. No new conceptual assumptions are introduced here.

A.1.1 Arithmetic Graph Structure

The arithmetic graph underlying the prime-shift operators has vertex set $\mathbb{N}$ and directed edges

$$n \longrightarrow pn \quad \text{for each } p \in \mathbb{P}.$$

Each vertex has infinite out-degree and finite in-degree. This graph is locally finite only after imposing a prime cutoff.

The adjacency structure is multiplicative rather than additive, distinguishing it from conventional lattices and justifying the use of operator-based rather than geometric intuition.

A.1.2 Regularized Operators

All operators considered in this work admit natural regularized versions

$$(H_\varphi^{(N,P)}\psi)(n) = \sum_{\substack{p \leq P \\ pn \leq N}} w(p) \left(e^{2\pi i \theta(n,p)} \psi(pn) + \mathbf{1}_{p|n} e^{-2\pi i \theta(n/p,p)} \psi(n/p) \right) + V(n)\psi(n),$$

acting on $\mathbb{C}^N$.

Such truncations define finite Hermitian matrices and are suitable for numerical diagonalization. All spectral statements in the main text refer to limits or stability properties of these regularized operators.

A.1.3 Gauge Equivalence and Boundary Effects

Formally, the gauge-transformed operator

$$H_\eta = G_\eta^{-1} H_0 G_\eta$$

is unitarily equivalent to the untwisted operator H_0 . However, this equivalence is exact only in the infinite, untruncated setting.

Finite truncations break gauge invariance through boundary effects. These effects are not artifacts but carry the phase sensitivity exploited in numerical probes. Spectral dependence on gauge parameters therefore reflects arithmetic structure interacting with truncation geometry.

A.1.4 Quadratic Form Estimates

For $\psi \in c_{00}(\mathbb{N})$, the quadratic form associated with the golden-phase operator satisfies

$$|Q_\varphi(\psi)| \leq 2 \sum_p |w(p)| \sum_n |\psi(n)|^2 + \sum_n |V(n)| |\psi(n)|^2,$$

formally suggesting boundedness under sufficiently strong damping of $w(p)$.

While this estimate is heuristic, it motivates the use of exponentially damped weights in numerical experiments.

A.1.5 Alternative Gauge Choices

Although the main text focuses on golden-phase gauges, the construction extends to arbitrary irrational gauges. In particular, one may replace φ by other quadratic irrationals or by generic badly approximable numbers.

The golden ratio is distinguished not by uniqueness but by extremality: it provides a canonical benchmark against which other irrational gauges may be compared.

A.1.6 Pseudocode Sketch

A minimal numerical implementation proceeds as follows:

1. Fix truncation parameters (N, P) and gauge choice η_φ .
2. Construct the sparse matrix representing $H_\varphi^{(N, P)}$.
3. Diagonalize the matrix using standard Hermitian solvers.
4. Compute spectral observables and compare against control ensembles.

Because the operator is sparse and structured, scalable implementations are feasible even for moderately large N .

A.1.7 Relation to Other Operator Constructions

Prime-shift operators with and without gauge phases appear implicitly in various contexts in analytic number theory and mathematical physics. The distinguishing feature of the present construction is the systematic use of gauge geometry as a diagnostic tool rather than as a modeling assumption.

This viewpoint aligns naturally with operator-based arithmetic programs while remaining agnostic about deeper conjectural interpretations.

A.1.8 Closing Remark

The technical considerations collected here are intended to clarify, not to complicate, the constructions presented in the main text. All essential ideas are contained in the core chapters; the appendix merely provides supporting detail for interested readers and practitioners.

APPENDIX B

B.1 Mellin-Space Formulation of Trace Invariants

This appendix develops the Mellin-space representation of the trace invariants associated with the SMRK Hamiltonian. The purpose is to make precise the relation between heat traces, spectral zeta functions, and arithmetic Dirichlet series.

B.1.1 Mellin transform conventions

Let $f : (0, \infty) \rightarrow \mathbb{C}$ be a locally integrable function with suitable decay properties. We define its Mellin transform by

$$(\mathcal{M}f)(s) := \int_0^\infty t^{s-1} f(t) dt,$$

whenever the integral converges. The inverse Mellin transform is given formally by

$$f(t) = \frac{1}{2\pi i} \int_{\sigma-i\infty}^{\sigma+i\infty} t^{-s} (\mathcal{M}f)(s) ds,$$

with σ chosen in a strip of analyticity.

B.1.2 Heat trace and spectral zeta function

Let $\Theta(t) = \text{Tr}(e^{-t\tilde{H}_{\text{SMRK}}})$ denote the heat trace. As shown in Chapter 7, $\Theta(t)$ is finite for all $t > 0$ and admits a short-time asymptotic expansion.

The spectral zeta function is defined by

$$\zeta_{\text{spec}}(s) = \sum_{k=1}^{\infty} E_k^{-s},$$

for $\Re(s)$ sufficiently large, where $\{E_k\}$ denotes the discrete spectrum of $\tilde{H}_{\text{SMRK}}$. Using the Mellin transform of the heat trace, one obtains

$$\zeta_{\text{spec}}(s) = \frac{1}{\Gamma(s)} \int_0^\infty t^{s-1} \Theta(t) dt,$$

which provides analytic continuation to a meromorphic function of s .

B.1.3 Renormalization and subtraction terms

The integral defining $\zeta_{\text{spec}}(s)$ diverges at $t = 0$ unless suitable subtractions are performed. These divergences correspond to the leading short-time terms of the heat trace expansion.

In the SMRK framework, the required subtractions are precisely those dictated by the renormalized quadratic-form construction. After subtraction, the Mellin integral converges in a vertical strip and defines a renormalized spectral zeta function.

B.1.4 Prime-side representation

Formally expanding the heat kernel using the prime-shift decomposition yields expressions involving sums over primes and their powers. After Mellin transformation, these terms give rise to Dirichlet series of the form

$$\sum_{n=1}^{\infty} \frac{a(n)}{n^s},$$

where the coefficients $a(n)$ are supported on prime powers and weighted by arithmetic transition amplitudes.

In particular, the von Mangoldt function appears naturally through the diagonal arithmetic potential, producing terms proportional to $-\zeta'(s)/\zeta(s)$ in the prime-side expansion.

B.1.5 Dual representations of trace invariants

The Mellin-space formulation makes explicit the dual nature of trace invariants. On the spectral side, they are expressed as sums or integrals involving the eigenvalues of $\tilde{H}_{\text{SMRK}}$. On the arithmetic side, they appear as Dirichlet series or Euler-product-like expressions built from primes.

Equality of these two representations, once properly renormalized, constitutes the operator-theoretic analogue of the explicit formula.

B.1.6 Reflection symmetry in Mellin space

The Mellin variable s naturally admits the involution $s \mapsto 1 - s$. In the SMRK framework, this symmetry is reflected in the structure of the heat trace and its subtractions, leading to functional relations for $\zeta_{\text{spec}}(s)$.

These relations parallel the functional equation of the Riemann zeta function and originate from scale duality and the symmetric treatment of prime shifts.

B.1.7 Analytic continuation

Analytic continuation in Mellin space is achieved by shifting the contour of integration and accounting for residues at poles arising from subtraction terms. This procedure mirrors standard techniques in analytic number theory and spectral geometry.

The resulting continuation is unique up to entire functions determined by the renormalization scheme, emphasizing once again the central role of renormalized trace invariants.

B.1.8 Concluding remark

The Mellin-space formulation clarifies the precise mathematical mechanism by which the Riemann zeta function and related Dirichlet series enter the SMRK framework. It provides a transparent bridge between spectral theory and arithmetic analysis and underpins the trace formulas discussed in the main text.

APPENDIX C

C.1 Comparison with Other Operator-Based Approaches

This appendix places the SMRK Hamiltonian in context by comparing it with other operator-theoretic frameworks that have been proposed in connection with arithmetic problems and the Riemann zeta function. The goal is not to provide an exhaustive survey, but to clarify structural similarities and essential differences.

C.1.1 Classical Hilbert–Pólya proposals

The Hilbert–Pólya conjecture suggests the existence of a self-adjoint operator whose spectrum coincides with the imaginary parts of the nontrivial zeros of the Riemann zeta function. Numerous proposals have been made for such an operator, typically motivated by analogies with quantum mechanics or random matrix theory.

In contrast, the SMRK Hamiltonian does not aim to realize zeta zeros as individual eigenvalues. Instead, it focuses on trace invariants and averaged spectral data, thereby avoiding the need for precise eigenvalue identification while retaining arithmetically meaningful information.

C.1.2 Berry–Keating-type models

Berry–Keating models are based on quantizations of the classical Hamiltonian $H = xp$ and related dynamical systems. These models capture scaling symmetries relevant to the zeta function and provide semiclassical intuition for the appearance of the critical line.

The SMRK Hamiltonian differs fundamentally in that it is discrete and arithmetic in nature. Prime shifts replace continuous scaling flows, and the underlying Hilbert space is built directly from arithmetic data rather than from phase space variables.

C.1.3 Connes’ trace formula and noncommutative geometry

Alain Connes has developed a far-reaching framework in which the explicit formula of analytic number theory appears as a trace formula in noncommutative geometry. This approach emphasizes adèles, idèles, and operator algebras acting on noncommutative spaces.

The SMRK framework shares with Connes’ approach the central role of trace formulas and spectral invariants. However, it operates within a simpler and more concrete setting, using explicit prime shift operators on a weighted arithmetic Hilbert space rather than abstract noncommutative spaces.

C.1.4 Random matrix models

Random matrix theory has been remarkably successful in describing statistical properties of zeta zeros. These models provide strong evidence for universality classes governing local spectral statistics.

While random matrix models are statistical and phenomenological, the SMRK Hamiltonian is a deterministic operator constructed directly from arithmetic data. Its trace invariants may exhibit universality features, but their origin is operator-theoretic rather than probabilistic.

C.1.5 Quantum graphs and arithmetic graphs

Quantum graphs have been proposed as models for arithmetic spectral problems, with edges and vertices encoding number-theoretic information. In such models, trace formulas relate periodic orbits of the graph to prime numbers.

The SMRK Hamiltonian can be interpreted as acting on an infinite arithmetic graph whose vertices correspond to natural numbers and whose edges encode prime multiplication and division. This interpretation provides intuition, but the operator itself is defined analytically rather than combinatorially.

C.1.6 Renormalization versus cutoff models

Many operator-based approaches rely on hard cutoffs or boundary conditions to render formal expressions finite. Such cutoffs often introduce ambiguities or break symmetries.

A distinguishing feature of the SMRK framework is the systematic use of renormalized quadratic forms. Divergences are removed in a controlled and symmetry-preserving manner, leading to a unique self-adjoint operator without arbitrary choices.

C.1.7 Scope and limitations

Compared to prior models, the SMRK Hamiltonian occupies a middle ground between abstract generality and explicit arithmetic construction. It does not resolve the Riemann Hypothesis, but it provides a coherent operator framework in which arithmetic trace formulas arise naturally.

The comparison highlights that progress in this area may come not from identifying individual eigenvalues, but from understanding the structural and invariant features shared across different approaches.

C.1.8 Concluding remark

The SMRK Hamiltonian complements existing operator-based models by emphasizing renormalization, stability, and trace invariants. Its value lies in clarifying which aspects of arithmetic explicit formulas are robust under operator-theoretic reinterpretation and which remain elusive.

APPENDIX D

D.1 Conceptual and Physical Analogies

This appendix collects conceptual and physical analogies that help situate the SMRK Hamiltonian within a broader scientific context. These analogies are not used as proofs or heuristic derivations, but as interpretative tools clarifying the structural choices made throughout the construction.

D.1.1 Renormalization as arithmetic vacuum subtraction

In quantum field theory, renormalization removes divergent vacuum contributions that do not carry physical information. A closely analogous mechanism appears in the SMRK framework.

The divergence arising from the prime harmonic sum

$$\sum_{p \leq P} \frac{1}{p}$$

is universal and independent of the arithmetic state. Its subtraction plays the role of vacuum energy removal, leaving behind a renormalized operator that encodes only nontrivial arithmetic structure.

D.1.2 Arithmetic scale as energy scale

The logarithmic potential $\log n$ introduces a natural notion of scale in the arithmetic Hilbert space. Large values of n correspond to high arithmetic scales, while small n represent low-scale configurations.

This identification mirrors the role of energy scales in physical systems and provides a natural interpretation of the heat parameter t as an inverse arithmetic energy scale.

D.1.3 Prime shifts as interaction vertices

Prime shift operators generate discrete transitions between arithmetic states. From a physical viewpoint, these transitions resemble interaction vertices, where a state interacts with a prime quantum.

Forward and backward prime shifts appear symmetrically, reflecting time-reversal or charge-conjugation-type symmetry in the arithmetic dynamics.

D.1.4 Trace invariants as partition functions

The heat trace

$$\Theta(t) = \text{Tr}\left(e^{-tH_{\text{SMRK}}}\right)$$

admits an interpretation analogous to a partition function. It aggregates contributions from all arithmetic states weighted by their effective energy.

Renormalized trace invariants thus play a role similar to thermodynamic potentials, encoding global structural information rather than microscopic details.

D.1.5 Mellin transform and dual representations

The Mellin transform acts as a bridge between scale space and spectral space. In this respect it is analogous to the Fourier transform linking time and frequency.

Reflection symmetry in Mellin space corresponds to a duality between small and large arithmetic scales, reinforcing the interpretation of the construction as a scale-invariant or scale-dual system.

D.1.6 Comparison with quantum statistical systems

The SMRK Hamiltonian shares features with quantum statistical mechanical models, particularly those based on discrete spectra and trace-class evolution operators. However, unlike thermal systems, the arithmetic dynamics do not arise from probabilistic ensembles but from deterministic multiplicative structure.

This distinction underscores the arithmetic nature of the construction despite the formal similarity of the tools employed.

D.1.7 Universality and robustness

Many structural properties of the SMRK Hamiltonian, such as discreteness of the spectrum, trace-class heat evolution, and stability under perturbations, are robust under modifications of the model.

This robustness suggests that the observed features belong to a universality class of arithmetic operators governed by multiplicative dynamics and renormalized traces, rather than to a finely tuned construction.

D.1.8 Limits of physical analogy

While physical analogies provide intuition, they have clear limits. The SMRK Hamiltonian is not derived from a physical system, nor does it describe physical particles or fields.

All analogies must therefore be understood as structural correspondences rather than literal identifications.

D.1.9 Concluding perspective

The value of the SMRK framework lies in its synthesis of arithmetic structure, operator theory, and renormalization. Physical analogies help illuminate this synthesis, but the mathematical content stands independently of any physical interpretation.

In this sense, the SMRK Hamiltonian occupies a conceptual position analogous to that of spectral geometry: a domain where analytic, algebraic, and structural ideas converge without being reducible to any single interpretation.

Chapter 10

Spectral Program for the SMRK Hamiltonian

Projective Weight Geometry on the Riemann Sphere and a Computation-First Route

Enter Yourname

Abstract

This section outlines a concrete spectral program for the SMRK Hamiltonian, translating the abstract operator-theoretic framework developed in the preceding sections into a set of analyzable and testable objectives. Assuming the existence of an appropriate self-adjoint realization or extension, the spectral program seeks to characterize the spectrum, spectral measures, and associated statistical properties of the operator in a manner consistent with its arithmetic construction.

The spectral program is formulated in terms of several complementary goals. These include the identification of spectral invariants, the analysis of level statistics, and the study of stability properties under refinement of the underlying operator parameters. Rather than focusing exclusively on the existence of individual eigenvalues, the program emphasizes global spectral behavior, which is often more robust and more informative in arithmetic and physical contexts.

A central component of the program is the comparison between the spectral data of the SMRK Hamiltonian and predictions derived from classical models in random matrix theory and analytic number theory. Such comparisons provide a framework for assessing whether the operator exhibits signatures compatible with arithmetic phenomena, without presupposing a direct or exact correspondence. Both agreements and deviations are treated as meaningful outcomes, capable of guiding refinement of the operator model or domain assumptions.

The section also addresses numerical and computational aspects of the spectral program. Given the unbounded nature of the operator and the infinite-dimensional setting, any numerical investigation necessarily involves truncation, regularization, and approximation. The spectral program therefore incorporates explicit criteria for interpreting numerical results, distinguishing between genuine spectral features and artifacts of finite-dimensional approximation.

Importantly, the spectral program is framed as an open-ended research agenda rather than a closed proof strategy. Its purpose is not to assert definitive arithmetic conclusions, but to establish a structured pathway through which evidence can be accumulated, hypotheses refined, and failure modes identified. This stance preserves the falsifiability and adaptability of the SMRK framework.

By articulating a clear spectral program, this section completes the transition from operator construction to analytical investigation, setting the stage for the explicit formulae, extensions, and generalizations explored in subsequent sections.

1 Overview and Spectral Program

The purpose of this work is to formulate and analyze a spectral program for the SMRK Hamiltonian acting on arithmetic states. The construction is motivated by the desire to understand arithmetic explicit formulas through operator-theoretic trace invariants rather than through direct identification of spectral points.

We consider operators acting on the weighted Hilbert space

$$\mathcal{H} := \ell^2(\mathbb{N}, 1/n),$$

consisting of complex-valued sequences $\psi : \mathbb{N} \rightarrow \mathbb{C}$ such that

$$\sum_{n=1}^{\infty} \frac{|\psi(n)|^2}{n} < \infty.$$

This space is naturally adapted to multiplicative arithmetic dynamics and admits bounded prime shift operators.

1.1 Arithmetic dynamics

Prime numbers generate discrete scale transformations on $\mathbb{N}$. For each prime p , multiplication by p defines a forward arithmetic transition $n \mapsto pn$, while division by p (when defined) generates the inverse transition.

These transformations are implemented at the operator level by bounded shift operators acting on $\mathcal{H}$. Their symmetrized combination encodes arithmetic motion across scales and plays the role of a kinetic term.

1.2 The SMRK Hamiltonian

The SMRK Hamiltonian is formally written as

$$H_{\text{SMRK}} = \sum_{p \in \mathcal{P}} \frac{1}{p} (S_p + S_p^*) + \alpha \Lambda(n) + \beta \log n,$$

where $\mathcal{P}$ denotes the set of primes, Λ is the von Mangoldt function, and $\alpha, \beta \in \mathbb{R}$ are real parameters.

The prime-shift term generates arithmetic transitions, while the diagonal potentials encode prime-power structure and logarithmic confinement. Taken together, these terms define an unbounded operator requiring renormalization.

1.3 Spectral objectives

The central goal of the spectral program is not to identify individual eigenvalues with arithmetic invariants, but to construct trace-class functions of the Hamiltonian and study their spectral invariants.

Of particular importance are:

- the heat trace

$$\Theta(t) := \mathrm{Tr}\left(e^{-tH_{\mathrm{SMRK}}}\right),$$

- the associated spectral zeta function

$$\zeta_{\mathrm{spec}}(s) := \sum_k E_k^{-s},$$

- renormalized trace identities relating spectral sums to prime-weighted Dirichlet series.

These objects are stable under perturbations and provide a robust interface between operator theory and analytic number theory.

1.4 Relation to the explicit formula

Classical explicit formulas relate sums over primes to sums over zeros of the Riemann zeta function. In the present framework, these relations arise from the equality of two representations of the same renormalized trace invariant: one spectral and one arithmetic.

Zeros of the zeta function appear implicitly as oscillatory components in trace expressions, rather than as direct spectral points. This avoids the need for a Hilbert–Pólya-type identification while retaining arithmetic content.

1.5 Outline of the paper

The remainder of the paper is organized as follows. In the next chapter we define the arithmetic Hilbert space and the associated prime shift operators. Subsequent chapters introduce quadratic-form renormalization, establish self-adjointness of the SMRK Hamiltonian, and analyze its spectral properties. The final chapters develop trace formulas, symmetry considerations, and connections to the Riemann zeta function.

Appendices collect technical lemmas, Mellin-space formulations, comparisons with other operator-based approaches, and conceptual analogies.

2 Arithmetic Hilbert Space and Operator Setup

This chapter introduces the arithmetic Hilbert space on which the SMRK Hamiltonian acts and defines the basic operators generating arithmetic dynamics. The construction is entirely discrete and multiplicative in nature.

2.1 The arithmetic Hilbert space

We work on the weighted Hilbert space

$$\mathcal{H} := \ell^2(\mathbb{N}, 1/n),$$

consisting of all complex-valued functions $\psi : \mathbb{N} \rightarrow \mathbb{C}$ satisfying

$$\|\psi\|^2 = \sum_{n=1}^{\infty} \frac{|\psi(n)|^2}{n} < \infty.$$

The inner product on $\mathcal{H}$ is given by

$$\langle \psi, \phi \rangle = \sum_{n=1}^{\infty} \frac{\overline{\psi(n)} \phi(n)}{n}.$$

This weighting ensures invariance under multiplicative scale transformations and provides natural control over large arithmetic scales.

2.2 Prime shift operators

Let p be a prime number. We define the forward prime shift operator $S_p : \mathcal{H} \rightarrow \mathcal{H}$ by

$$(S_p \psi)(n) := \psi(pn).$$

A direct computation shows that S_p is bounded on $\mathcal{H}$ with

$$\|S_p\| = p^{-1/2}.$$

The adjoint operator S_p^* with respect to the weighted inner product is given by

$$(S_p^* \psi)(n) = \mathbf{1}_{p|n} \psi(n/p),$$

where $\mathbf{1}_{p|n}$ denotes the indicator function of divisibility by p . The adjoint encodes inverse arithmetic transitions whenever they are defined.

2.3 Symmetrized arithmetic transitions

The elementary kinetic building block is the symmetrized prime transition operator

$$K_p := S_p + S_p^*.$$

Each K_p is bounded and symmetric on $\mathcal{H}$. These operators generate discrete arithmetic motion across multiplicative scales and play the role of kinetic terms in the SMRK Hamiltonian.

2.4 Diagonal arithmetic potentials

In addition to prime shifts, we consider diagonal multiplication operators. For a real-valued function $V : \mathbb{N} \rightarrow \mathbb{R}$, define

$$(V\psi)(n) := V(n)\psi(n),$$

with maximal domain

$$D(V) := \{\psi \in \mathcal{H} : V\psi \in \mathcal{H}\}.$$

Two arithmetic potentials are of particular importance:

$$V_{\Lambda}(n) := \Lambda(n), \quad V_{\log}(n) := \log n,$$

where Λ denotes the von Mangoldt function. These potentials encode prime power structure and logarithmic confinement, respectively.

2.5 Formal operator expression

Combining prime transitions and diagonal potentials, the SMRK Hamiltonian is formally expressed as

$$H_{\text{SMRK}} = \sum_{p \in \mathcal{P}} \frac{1}{p} K_p + \alpha V_\Lambda + \beta V_{\log},$$

where $\mathcal{P}$ denotes the set of prime numbers and $\alpha, \beta \in \mathbb{R}$ are real parameters.

This expression is formal: the prime sum diverges and the operator is unbounded. A rigorous definition therefore requires regularization and renormalization, which are developed in subsequent chapters.

2.6 Dense core

Let $\mathcal{D}_0 \subset \mathcal{H}$ denote the space of finitely supported arithmetic functions. This space is dense in $\mathcal{H}$ and invariant under all prime shift operators and diagonal potentials.

All operator identities and quadratic forms introduced later are initially defined on $\mathcal{D}_0$ and extended by closure. This choice avoids domain ambiguities at the algebraic level.

2.7 Outlook

The operators defined in this chapter provide the algebraic foundation for the SMRK Hamiltonian. In the next chapter we introduce prime-cutoff quadratic forms and analyze their divergence structure, leading to a renormalized definition of the operator.

3 Prime Cutoff Operators and Quadratic Forms

In this chapter we introduce a regularized version of the SMRK Hamiltonian by truncating the prime sum at a finite scale. This cutoff construction provides a controlled setting in which quadratic forms are well-defined and allows us to identify the precise source of divergences.

3.1 Prime cutoff operators

Let $P \geq 2$ be a cutoff parameter. We define the truncated kinetic operator by

$$K^{(P)} := \sum_{\substack{p \in \mathcal{P} \\ p \leq P}} \frac{1}{p} K_p, \quad K_p := S_p + S_p^*.$$

Since each K_p is bounded and symmetric, the finite sum $K^{(P)}$ defines a bounded self-adjoint operator on $\mathcal{H}$.

3.2 Cutoff quadratic form

Associated with $K^{(P)}$ we define the quadratic form

$$q_{\text{kin}}^{(P)}[\psi] := \langle \psi, K^{(P)} \psi \rangle, \quad \psi \in \mathcal{H}.$$

On the dense core $\mathcal{D}_0$, this form can be written explicitly as

$$q_{\text{kin}}^{(P)}[\psi] = \sum_{p \leq P} \frac{1}{p} \left(\langle \psi, S_p \psi \rangle + \langle \psi, S_p^* \psi \rangle \right).$$

Each term is finite, and the cutoff ensures absolute convergence of the sum.

3.3 Explicit arithmetic representation

Using the definitions of the prime shift operators, the quadratic form admits the representation

$$q_{\text{kin}}^{(P)}[\psi] = \sum_{p \leq P} \frac{1}{p} \sum_{n=1}^{\infty} \frac{1}{n} \left(\overline{\psi(n)} \psi(pn) + \mathbf{1}_{p|n} \overline{\psi(n)} \psi(n/p) \right).$$

The two contributions correspond to forward and backward arithmetic transitions. The expression is manifestly real-valued.

3.4 Divergence as the cutoff is removed

As the cutoff parameter tends to infinity, the quadratic form diverges. For fixed $\psi \in \mathcal{D}_0$, one finds

$$q_{\text{kin}}^{(P)}[\psi] = 2 \left(\sum_{p \leq P} \frac{1}{p} \right) \|\psi\|^2 + r^{(P)}[\psi],$$

where the remainder term $r^{(P)}[\psi]$ remains bounded as $P \rightarrow \infty$.

Since the prime harmonic sum satisfies

$$\sum_{p \leq P} \frac{1}{p} = \log \log P + O(1),$$

the divergence is logarithmic and universal, depending only on the norm of ψ .

3.5 Full cutoff quadratic form

Including the diagonal arithmetic potentials, we define the full cutoff quadratic form by

$$q^{(P)}[\psi] := q_{\text{kin}}^{(P)}[\psi] + \alpha \langle \psi, V_{\Lambda} \psi \rangle + \beta \langle \psi, V_{\log} \psi \rangle, \quad \psi \in \mathcal{D}_0.$$

For fixed P , this form is symmetric and bounded from below.

3.6 Interpretation of the divergence

The divergence of $q^{(P)}$ as $P \rightarrow \infty$ is proportional to the identity operator and reflects the accumulation of prime transitions at large arithmetic scales. Importantly, the divergence carries no fine arithmetic information and can therefore be removed by a canonical subtraction.

This observation motivates the introduction of renormalized quadratic forms in the next chapter.

3.7 Outlook

In Chapter 4 we define a renormalization procedure that removes the divergent contribution identified above. We then establish uniform lower bounds and show that the renormalized forms converge to a closed quadratic form defining the SMRK Hamiltonian as a self-adjoint operator.

4 Renormalization and Closed Quadratic Forms

In this chapter we introduce a renormalization procedure that removes the universal divergence identified in the prime cutoff quadratic forms. The result is a family of uniformly lower bounded forms converging to a closed quadratic form that defines the SMRK Hamiltonian rigorously.

4.1 Subtraction of the universal divergence

Recall from Chapter 3 that the cutoff quadratic form admits the decomposition

$$q^{(P)}[\psi] = 2 \left(\sum_{p \leq P} \frac{1}{p} \right) \|\psi\|^2 + r^{(P)}[\psi] + \alpha \langle \psi, V_\Lambda \psi \rangle + \beta \langle \psi, V_{\log} \psi \rangle,$$

where the remainder term $r^{(P)}[\psi]$ remains bounded as $P \rightarrow \infty$.

We define the renormalized quadratic form by subtracting the divergent identity contribution:

$$\tilde{q}^{(P)}[\psi] := q^{(P)}[\psi] - 2 \left(\sum_{p \leq P} \frac{1}{p} \right) \|\psi\|^2.$$

This subtraction is canonical and independent of the arithmetic structure of ψ .

4.2 Uniform lower bounds

Proposition 4.1. There exists a constant $C \in \mathbb{R}$, independent of P , such that

$$\tilde{q}^{(P)}[\psi] \geq C \|\psi\|^2 \quad \text{for all } \psi \in \mathcal{D}_0.$$

Proof. The remainder term $r^{(P)}[\psi]$ is uniformly bounded in P by construction. The diagonal potential $V_{\log}$ dominates the negative part of the kinetic contribution for large arithmetic scales, while V_Λ contributes only finite fluctuations on $\mathcal{D}_0$. Collecting the bounds yields a uniform lower estimate. $\square$

4.3 Cauchy convergence of renormalized forms

For $\psi \in \mathcal{D}_0$, the limit

$$\tilde{q}[\psi] := \lim_{P \rightarrow \infty} \tilde{q}^{(P)}[\psi]$$

exists and is finite. The convergence follows from the boundedness of the difference

$$\tilde{q}^{(P_2)}[\psi] - \tilde{q}^{(P_1)}[\psi] = \sum_{P_1 < p \leq P_2} \frac{1}{p} \left(\langle \psi, K_p \psi \rangle - 2 \|\psi\|^2 \right),$$

which tends to zero as $P_1, P_2 \rightarrow \infty$.

4.4 Definition of the limiting quadratic form

We define the limiting renormalized quadratic form

$$\tilde{q}[\psi] = \lim_{P \rightarrow \infty} \tilde{q}^{(P)}[\psi], \quad \psi \in \mathcal{D}_0.$$

This form is symmetric and densely defined on $\mathcal{H}$.

4.5 Closability

Theorem 4.1. The quadratic form $\tilde{q}$ is closable on $\mathcal{H}$.

Proof. Uniform lower boundedness together with pointwise convergence on the dense core $\mathcal{D}_0$ implies closability by standard results in quadratic form theory. $\square$

We denote by $\bar{q}$ the closure of $\tilde{q}$ with domain $D(\bar{q}) \subset \mathcal{H}$.

4.6 Associated self-adjoint operator

By the representation theorem for closed, lower bounded quadratic forms, there exists a unique self-adjoint operator H_{SMRK} such that

$$\bar{q}[\psi] = \langle \psi, H_{\text{SMRK}} \psi \rangle \quad \text{for all } \psi \in D(H_{\text{SMRK}}).$$

This operator is the rigorously defined SMRK Hamiltonian.

4.7 Interpretation

The renormalization procedure removes a purely universal divergence originating from the density of prime transitions. After subtraction, the remaining operator encodes nontrivial arithmetic structure through the interaction between prime shifts and arithmetic potentials.

4.8 Outlook

In Chapter 5 we analyze spectral properties of the renormalized SMRK Hamiltonian, including lower semiboundedness, discreteness of the spectrum, and scaling behavior under arithmetic dilations.

5 Self-Adjointness and Spectral Properties

In this chapter we analyze the operator associated with the renormalized quadratic form constructed previously. We establish self-adjointness, lower semiboundedness, and basic spectral properties of the SMRK Hamiltonian.

5.1 Representation theorem

Let $\bar{q}$ denote the closed, densely defined, lower bounded quadratic form obtained as the closure of the renormalized form $\tilde{q}$. By the representation theorem for quadratic forms, there exists a unique self-adjoint operator H_{SMRK} on $\mathcal{H}$ such that

$$\bar{q}[\psi] = \langle \psi, H_{\text{SMRK}} \psi \rangle \quad \text{for all } \psi \in D(H_{\text{SMRK}}).$$

The domain $D(H_{\text{SMRK}})$ is uniquely determined by the form domain $D(\bar{q})$.

5.2 Lower semiboundedness

By construction, the quadratic form $\bar{q}$ satisfies a uniform lower bound. Consequently, the operator H_{SMRK} is lower semibounded. There exists a constant $E_0 \in \mathbb{R}$ such that

$$\langle \psi, H_{\text{SMRK}} \psi \rangle \geq E_0 \|\psi\|^2 \quad \text{for all } \psi \in D(H_{\text{SMRK}}).$$

Lower semiboundedness guarantees stability of the spectrum and allows the definition of spectral functions such as the resolvent and heat kernel.

5.3 Core properties

The dense subspace $\mathcal{D}_0$ of finitely supported arithmetic functions is a form core for $\bar{q}$. In particular, for every $\psi \in D(\bar{q})$ there exists a sequence $(\psi_k) \subset \mathcal{D}_0$ such that

$$\psi_k \rightarrow \psi \quad \text{in } \mathcal{H}, \quad \bar{q}[\psi_k - \psi] \rightarrow 0.$$

This property allows all operator identities to be verified initially on $\mathcal{D}_0$ and extended by continuity.

5.4 Formal action on the core

On the core $\mathcal{D}_0$, the operator H_{SMRK} acts formally as

$$(H_{\text{SMRK}}\psi)(n) = \sum_{p \in \mathcal{P}} \frac{1}{p} (\psi(pn) + \mathbf{1}_{p|n} \psi(n/p)) + \alpha \Lambda(n) \psi(n) + \beta \log n \psi(n) - C \psi(n),$$

where the constant C represents the renormalization subtraction. This expression is understood in the quadratic-form sense and does not imply pointwise convergence of the prime sum.

5.5 Resolvent properties

For $\lambda > -E_0$, the resolvent operator

$$(H_{\text{SMRK}} + \lambda)^{-1}$$

is bounded on $\mathcal{H}$. Moreover, the confining nature of the logarithmic potential implies compactness of the resolvent.

As a consequence, the spectrum of H_{SMRK} consists solely of discrete eigenvalues of finite multiplicity, accumulating only at $+\infty$.

5.6 Spectral decomposition

There exists an orthonormal basis $(\varphi_k)_{k \geq 1}$ of $\mathcal{H}$ consisting of eigenfunctions of H_{SMRK} , with corresponding eigenvalues $(E_k)_{k \geq 1}$ satisfying

$$E_1 \leq E_2 \leq E_3 \leq \cdots, \quad E_k \rightarrow +\infty \quad \text{as } k \rightarrow \infty.$$

Each eigenvalue is repeated according to its finite multiplicity.

5.7 Stability under perturbations

Small perturbations of the diagonal arithmetic potentials or of the coefficients α and β give rise to form-bounded perturbations of H_{SMRK} . Standard perturbation theory implies that self-adjointness and discreteness of the spectrum are preserved.

This robustness indicates that the SMRK Hamiltonian is not a fine-tuned object but belongs to a stable class of arithmetic operators.

5.8 Outlook

Having established self-adjointness and discreteness of the spectrum, we are now in a position to define spectral invariants associated with H_{SMRK} . In the next chapter we introduce the heat kernel, heat trace, and related trace-class operators forming the basis of the spectral program.

6 Heat Kernel and Trace Program

In this chapter we introduce the heat kernel associated with the SMRK Hamiltonian and formulate the trace program that underlies the spectral approach developed in this work. The emphasis is on trace-class operators and their invariants rather than on individual spectral points.

6.1 Heat semigroup

Let H_{SMRK} denote the self-adjoint, lower semibounded operator constructed in the previous chapter. For each $t > 0$, the heat semigroup

$$e^{-tH_{\text{SMRK}}}$$

is a bounded operator on $\mathcal{H}$ defined via the functional calculus.

Lower semiboundedness implies that the map $t \mapsto e^{-tH_{\text{SMRK}}}$ is strongly continuous and forms a one-parameter semigroup satisfying

$$e^{-(t+s)H_{\text{SMRK}}} = e^{-tH_{\text{SMRK}}} e^{-sH_{\text{SMRK}}} \quad \text{for all } t, s > 0.$$

6.2 Trace-class property

Compactness of the resolvent implies that the heat operator is trace class for all $t > 0$. Consequently, the trace

$$\Theta(t) := \text{Tr}\left(e^{-tH_{\text{SMRK}}}\right)$$

is finite for every $t > 0$.

In terms of the discrete spectrum $(E_k)_{k \geq 1}$ of H_{SMRK} , the trace admits the representation

$$\Theta(t) = \sum_{k=1}^{\infty} e^{-tE_k},$$

where the series converges absolutely for all $t > 0$.

6.3 Heat kernel representation

Let $(\varphi_k)_{k \geq 1}$ be an orthonormal basis of eigenfunctions of H_{SMRK} . The heat operator admits the kernel representation

$$(e^{-tH_{\text{SMRK}}} \psi)(n) = \sum_{m=1}^{\infty} K_t(n, m) \psi(m),$$

with

$$K_t(n, m) = \sum_{k=1}^{\infty} e^{-tE_k} \varphi_k(n) \overline{\varphi_k(m)}.$$

Although the kernel $K_t(n, m)$ is not used pointwise in the sequel, its existence provides a useful conceptual link between spectral and arithmetic viewpoints.

6.4 Short-time behavior

As $t \rightarrow 0^+$, the heat trace $\Theta(t)$ diverges due to the accumulation of high-energy eigenvalues. This divergence reflects the ultraviolet behavior of the operator and is controlled by the renormalization scheme built into the definition of H_{SMRK} .

The leading short-time behavior is governed by the diagonal arithmetic potentials and exhibits logarithmic corrections characteristic of multiplicative scale dynamics.

6.5 Long-time behavior

As $t \rightarrow \infty$, the heat trace is dominated by the lowest eigenvalues of H_{SMRK} . In particular,

$$\Theta(t) \sim e^{-tE_1} \quad \text{as } t \rightarrow \infty,$$

where E_1 denotes the ground state energy.

This asymptotic regime encodes global spectral information and is insensitive to fine arithmetic structure.

6.6 Spectral zeta function

The heat trace provides a natural route to the spectral zeta function defined by

$$\zeta_{\text{spec}}(s) := \sum_{k=1}^{\infty} E_k^{-s},$$

for $\Re(s)$ sufficiently large. Using the Mellin transform, one obtains

$$\zeta_{\text{spec}}(s) = \frac{1}{\Gamma(s)} \int_0^{\infty} t^{s-1} \Theta(t) dt,$$

after subtraction of the short-time divergent terms.

6.7 Renormalized trace invariants

Both the heat trace and the spectral zeta function require renormalization to remove divergent contributions arising at small t or large spectral parameter. In the SMRK framework, these subtractions are dictated by the renormalized quadratic form and preserve all symmetries of the operator.

The resulting renormalized trace invariants are stable under perturbations and serve as the primary objects of arithmetic interpretation.

6.8 Trace program

The spectral program pursued here is based on the analysis of trace invariants rather than on individual eigenvalues. The central idea is that arithmetic information is encoded in the structure of traces and their dual representations.

These traces admit both a spectral expansion in terms of eigenvalues and an arithmetic expansion in terms of prime-local contributions. Equating the two yields trace formulas analogous to classical explicit formulas in number theory.

6.9 Outlook

In the next chapter we analyze the arithmetic expansion of trace invariants and establish their relation to prime sums and Dirichlet series. This connection forms the bridge between the operator-theoretic construction and analytic number theory.

7 Arithmetic Expansion of Trace Invariants

In this chapter we develop the arithmetic, or prime-side, expansion of the trace invariants introduced previously. The goal is to express renormalized traces in terms of prime-local contributions and Dirichlet series, thereby connecting the operator-theoretic framework with classical arithmetic structures.

7.1 Prime-local decomposition of the heat operator

The SMRK Hamiltonian is built from a sum of symmetrized prime shift operators and diagonal arithmetic potentials. Formally expanding the heat operator $e^{-tH_{\text{SMRK}}}$ in powers of these contributions leads to expressions organized by prime transitions.

Although such expansions are not convergent term-by-term at the operator level, they can be interpreted in a renormalized sense when inserted into trace expressions. The trace naturally suppresses nonlocal divergences and isolates prime-local contributions.

7.2 Prime paths and multiplicative orbits

Each contribution to the trace can be associated with a finite sequence of prime multiplications and divisions that returns an arithmetic state to itself. Such sequences define closed multiplicative orbits on $\mathbb{N}$.

Prime powers arise naturally as repetitions of the same prime transition. As a result, trace terms organize themselves according to prime powers rather than individual primes, mirroring the structure of classical explicit formulas.

7.3 Formal prime expansion

At a formal level, the renormalized heat trace admits an expansion of the form

$$\Theta_{\text{ren}}(t) = \sum_{n=1}^{\infty} a(n) f_t(n),$$

where the coefficients $a(n)$ are supported on prime powers and $f_t(n)$ encodes the contribution of an arithmetic transition of scale n at time t .

The precise form of $f_t(n)$ depends on the renormalization scheme and on the diagonal potentials, but its Mellin transform has well-defined analytic properties.

7.4 Appearance of the von Mangoldt function

The von Mangoldt function enters the arithmetic expansion through the diagonal potential V_{Λ} . Its presence ensures that prime powers are weighted with their natural arithmetic multiplicities.

In Mellin space, this leads to Dirichlet series of the form

$$\sum_{n=1}^{\infty} \frac{\Lambda(n)}{n^s},$$

which coincide with $-\zeta'(s)/\zeta(s)$ for $\Re(s) > 1$. This identification is structural and does not rely on any spectral identification of zeros.

7.5 Renormalization of prime sums

The prime-side expansion involves divergent sums analogous to those encountered on the spectral side. These divergences are removed by the same renormalization subtractions that define the operator itself.

Consistency of the framework requires that renormalization commute with the trace and preserve symmetry between forward and backward prime transitions. This requirement severely constrains admissible arithmetic expansions.

7.6 Dirichlet series representation

After renormalization, trace invariants can be written in terms of Dirichlet series with controlled analytic behavior. Schematically, one obtains expressions of the form

$$\mathcal{T}(s) = \sum_{n=1}^{\infty} \frac{b(n)}{n^s},$$

where the coefficients $b(n)$ encode renormalized prime-local data and decay sufficiently fast to ensure convergence in a half-plane.

These Dirichlet series provide the arithmetic counterpart to spectral zeta functions and serve as the main bridge to analytic number theory.

7.7 Comparison with the explicit formula

The structure of the arithmetic expansion parallels that of the classical explicit formula, in which sums over primes and prime powers are equated with sums over spectral data. In the present framework, both sides arise from the same operator-theoretic trace invariant.

This reinterpretation shifts the emphasis from identities between unrelated quantities to dual representations of a single object.

7.8 Stability and universality

A key feature of the arithmetic expansion is its stability under perturbations of the operator. Small changes in the diagonal potentials or coefficients modify the weights $b(n)$ smoothly without altering the overall structure of the expansion.

This stability suggests a form of universality in the arithmetic trace formulas, independent of fine details of the operator model.

7.9 Outlook

In the next chapter we analyze the Mellin-space structure of the arithmetic trace expansion and its relation to functional equations and symmetry properties of associated Dirichlet series.

8 Mellin Transform, Functional Equation, and Symmetry

This chapter analyzes the Mellin-space structure of the arithmetic trace invariants introduced previously. We show how Mellin transforms organize both the spectral and arithmetic expansions and how reflection symmetries arise at the level of renormalized traces.

8.1 Mellin transform of trace invariants

Let $\Theta_{\text{ren}}(t)$ denote a renormalized heat trace associated with the SMRK Hamiltonian. Its Mellin transform is defined by

$$\mathcal{M}\Theta_{\text{ren}}(s) := \int_0^\infty t^{s-1} \Theta_{\text{ren}}(t) dt,$$

whenever the integral converges after subtraction of short-time divergences.

This transform provides a unified framework in which spectral zeta functions and arithmetic Dirichlet series appear as different representations of the same object.

8.2 Spectral-side representation

On the spectral side, the Mellin transform yields

$$\mathcal{M}\Theta_{\text{ren}}(s) = \Gamma(s) \zeta_{\text{spec}}(s),$$

where

$$\zeta_{\text{spec}}(s) = \sum_{k=1}^{\infty} E_k^{-s}$$

is the spectral zeta function of the SMRK Hamiltonian, defined initially for $\Re(s)$ sufficiently large and extended by analytic continuation.

Poles of $\zeta_{\text{spec}}(s)$ correspond to the short-time asymptotics of the heat trace and are fully determined by the renormalization scheme.

8.3 Arithmetic-side representation

On the arithmetic side, the Mellin transform of the prime-local expansion yields Dirichlet series of the form

$$\sum_{n=1}^{\infty} \frac{b(n)}{n^s},$$

where the coefficients $b(n)$ are supported on prime powers and encode renormalized arithmetic transition data.

The presence of the von Mangoldt function in the diagonal potential ensures that logarithmic derivatives of Euler products appear naturally in this representation.

8.4 Equality of representations

The central trace identity asserts the equality of the spectral and arithmetic representations of $\mathcal{M}\Theta_{\text{ren}}(s)$ in a common domain of analyticity. This equality is the operator-theoretic analogue of the classical explicit formula.

Importantly, both sides arise from the same renormalized trace invariant rather than from an a priori identification of unrelated quantities.

8.5 Reflection symmetry

The Mellin variable s admits a natural involution

$$s \longmapsto 1 - s.$$

In the SMRK framework, this reflection symmetry is inherited from the symmetric treatment of forward and backward prime shifts and from the logarithmic scaling structure of the arithmetic Hilbert space.

As a consequence, renormalized Mellin transforms satisfy functional relations relating values at s and $1 - s$, up to explicitly controlled correction terms.

8.6 Relation to functional equations

The reflection symmetry observed above parallels the functional equation of the Ri

9 Conclusion, Scope, and Future Directions

This chapter summarizes the spectral program developed for the SMRK Hamiltonian, clarifies the scope of the results obtained, and outlines directions for future research. The emphasis is on structural understanding rather than on definitive arithmetic claims.

9.1 Summary of the construction

We have constructed an arithmetic operator acting on the weighted Hilbert space

$$\mathcal{H} := \ell^2(\mathbb{N}, 1/n),$$

generated by symmetrized prime shift operators and diagonal arithmetic potentials. Due to the divergence of the prime sum, the operator is defined rigorously through a renormalized quadratic-form approach.

The resulting SMRK Hamiltonian is self-adjoint, lower semibounded, and has compact resolvent. These properties guarantee a purely discrete spectrum and the existence of trace-class functions such as the heat kernel.

9.2 Spectral program versus spectral identification

A central methodological point of this work is the distinction between a *spectral program* and a *spectral identification*. The present framework does not attempt to identify individual eigenvalues of the SMRK Hamiltonian with arithmetic invariants such as the imaginary parts of zeros of the Riemann zeta function.

Instead, arithmetic information is accessed through trace invariants and their dual representations. This shift in perspective avoids the need for fine-grained spectral control while retaining meaningful connections to classical explicit formulas.

9.3 Role of trace invariants

Trace invariants, such as the heat trace and the associated spectral zeta function, play a central role in the analysis. These quantities are stable under perturbations, insensitive to local spectral fluctuations, and naturally admit both spectral and arithmetic expansions.

The equality of these expansions, once renormalized, provides the operator-theoretic counterpart of explicit formulas in analytic number theory.

9.4 Symmetry and renormalization

Symmetry considerations, particularly reflection symmetry in Mellin space, impose strong constraints on admissible operator models and renormalization schemes. In the SMRK framework, renormalization is not an auxiliary technical step but a structural requirement dictated by symmetry and stability.

This viewpoint aligns renormalization in arithmetic operator theory with its role in quantum field theory and spectral geometry.

9.5 Limitations of the present framework

The results obtained here do not constitute a proof of the Riemann Hypothesis, nor do they provide a direct spectral realization of zeta zeros. The framework is deliberately conservative in its claims and avoids speculative identifications that are not supported by rigorous analysis.

Furthermore, explicit asymptotics of eigenvalues and detailed arithmetic information contained in oscillatory trace terms remain beyond the scope of the present work.

9.6 Directions for further research

Several avenues for future investigation suggest themselves. These include the development of numerical approximations to trace invariants, the study of deformations of the arithmetic potentials, and the extension of the construction to other L -functions and arithmetic settings.

Another promising direction is the search for geometric or dynamical models whose quantization leads naturally to operators in the same universality class as the SMRK Hamiltonian.

9.7 Falsifiability and robustness

An important feature of the SMRK spectral program is its falsifiability. Predictions regarding symmetry properties, stability of trace invariants, and behavior under perturbations can, in principle, be tested analytically or numerically.

Failure of these tests would signal the need for modification or abandonment of the operator model, while continued robustness would strengthen its relevance.

9.8 Final remarks

The SMRK Hamiltonian provides a concrete example of how arithmetic structure, operator theory, and renormalization can be combined into a coherent spectral framework. While many questions remain open, the construction establishes a stable arena in which arithmetic trace formulas can be studied from an operator-theoretic perspective.

Whether this approach ultimately leads to deeper insights into prime distribution or the zeros of the Riemann zeta function remains an open question. What is clear, however, is that the spectral program outlined here offers a systematic and mathematically rigorous path for further exploration.

APPENDIX A

A.1 Technical Remarks and Supplementary Material

This appendix collects technical remarks, clarifications, and auxiliary constructions that support the main text but are not essential for its logical flow. No new conceptual assumptions are introduced here.

A.1.1 Arithmetic Graph Structure

The arithmetic graph underlying the prime-shift operators has vertex set $\mathbb{N}$ and directed edges

$$n \longrightarrow pn \quad \text{for each } p \in \mathbb{P}.$$

Each vertex has infinite out-degree and finite in-degree. This graph is locally finite only after imposing a prime cutoff.

The adjacency structure is multiplicative rather than additive, distinguishing it from conventional lattices and justifying the use of operator-based rather than geometric intuition.

A.1.2 Regularized Operators

All operators considered in this work admit natural regularized versions

$$(H_\varphi^{(N,P)}\psi)(n) = \sum_{\substack{p \leq P \\ pn \leq N}} w(p) \left(e^{2\pi i \theta(n,p)} \psi(pn) + \mathbf{1}_{p|n} e^{-2\pi i \theta(n/p,p)} \psi(n/p) \right) + V(n)\psi(n),$$

acting on $\mathbb{C}^N$.

Such truncations define finite Hermitian matrices and are suitable for numerical diagonalization. All spectral statements in the main text refer to limits or stability properties of these regularized operators.

A.1.3 Gauge Equivalence and Boundary Effects

Formally, the gauge-transformed operator

$$H_\eta = G_\eta^{-1} H_0 G_\eta$$

is unitarily equivalent to the untwisted operator H_0 . However, this equivalence is exact only in the infinite, untruncated setting.

Finite truncations break gauge invariance through boundary effects. These effects are not artifacts but carry the phase sensitivity exploited in numerical probes. Spectral dependence on gauge parameters therefore reflects arithmetic structure interacting with truncation geometry.

A.1.4 Quadratic Form Estimates

For $\psi \in c_{00}(\mathbb{N})$, the quadratic form associated with the golden-phase operator satisfies

$$|Q_\varphi(\psi)| \leq 2 \sum_p |w(p)| \sum_n |\psi(n)|^2 + \sum_n |V(n)| |\psi(n)|^2,$$

formally suggesting boundedness under sufficiently strong damping of $w(p)$.

While this estimate is heuristic, it motivates the use of exponentially damped weights in numerical experiments.

A.1.5 Alternative Gauge Choices

Although the main text focuses on golden-phase gauges, the construction extends to arbitrary irrational gauges. In particular, one may replace φ by other quadratic irrationals or by generic badly approximable numbers.

The golden ratio is distinguished not by uniqueness but by extremality: it provides a canonical benchmark against which other irrational gauges may be compared.

A.1.6 Pseudocode Sketch

A minimal numerical implementation proceeds as follows:

1. Fix truncation parameters (N, P) and gauge choice η_φ .
2. Construct the sparse matrix representing $H_\varphi^{(N, P)}$.
3. Diagonalize the matrix using standard Hermitian solvers.
4. Compute spectral observables and compare against control ensembles.

Because the operator is sparse and structured, scalable implementations are feasible even for moderately large N .

A.1.7 Relation to Other Operator Constructions

Prime-shift operators with and without gauge phases appear implicitly in various contexts in analytic number theory and mathematical physics. The distinguishing feature of the present construction is the systematic use of gauge geometry as a diagnostic tool rather than as a modeling assumption.

This viewpoint aligns naturally with operator-based arithmetic programs while remaining agnostic about deeper conjectural interpretations.

A.1.8 Closing Remark

The technical considerations collected here are intended to clarify, not to complicate, the constructions presented in the main text. All essential ideas are contained in the core chapters; the appendix merely provides supporting detail for interested readers and practitioners.

APPENDIX B

B.1 Mellin Transform and Analytic Continuation

This appendix collects the Mellin-space conventions and the analytic continuation mechanism used for trace invariants associated with the SMRK Hamiltonian. The focus is on the relationship between the heat trace, spectral zeta functions, and Dirichlet-series-type arithmetic expansions.

B.1.1 Mellin transform conventions

Let $f : (0, \infty) \rightarrow \mathbb{C}$ be locally integrable. The Mellin transform is defined by

$$(\mathcal{M}f)(s) := \int_0^\infty t^{s-1} f(t) dt,$$

whenever the integral converges. Formally, the inverse Mellin transform is

$$f(t) = \frac{1}{2\pi i} \int_{\sigma-i\infty}^{\sigma+i\infty} t^{-s} (\mathcal{M}f)(s) ds,$$

where the real number σ is chosen in a strip of analyticity of $(\mathcal{M}f)(s)$.

B.1.2 Heat trace and spectral zeta function

Let H_{SMRK} be the self-adjoint, lower semibounded SMRK Hamiltonian on

$$\mathcal{H} := \ell^2(\mathbb{N}, 1/n),$$

with discrete spectrum $(E_k)_{k \geq 1}$. The heat trace is

$$\Theta(t) := \text{Tr} \left(e^{-tH_{\text{SMRK}}} \right) = \sum_{k=1}^{\infty} e^{-tE_k}, \quad t > 0.$$

For $\Re(s)$ sufficiently large, the spectral zeta function is defined by

$$\zeta_{\text{spec}}(s) := \sum_{k=1}^{\infty} E_k^{-s}.$$

Assuming convergence of the Mellin integral (or after renormalized subtraction), one has the Mellin relation

$$\zeta_{\text{spec}}(s) = \frac{1}{\Gamma(s)} \int_0^\infty t^{s-1} \Theta(t) dt.$$

B.1.3 Short-time divergences and subtraction

As $t \rightarrow 0^+$, the heat trace typically admits an asymptotic expansion of the form

$$\Theta(t) \sim \sum_{j=0}^J a_j t^{-\mu_j} + \sum_{m=0}^M b_m (\log t)^m + O(t^\delta),$$

for some exponents $\mu_j > 0$ and $\delta > 0$. The divergent terms in the Mellin integral arise from these short-time contributions.

We define a subtracted heat trace by

$$\Theta_{\text{sub}}(t) := \Theta(t) - \sum_{j=0}^J a_j t^{-\mu_j} - \sum_{m=0}^M b_m (\log t)^m,$$

chosen so that the Mellin integral

$$\int_0^\infty t^{s-1} \Theta_{\text{sub}}(t) dt$$

converges in a vertical strip. This procedure yields a renormalized spectral zeta function, unique up to entire functions determined by the subtraction scheme.

B.1.4 Meromorphic continuation via contour shifting

The analytic continuation of Mellin-transformed trace invariants can be obtained by splitting the integral at $t = 1$:

$$\int_0^\infty t^{s-1} \Theta_{\text{sub}}(t) dt = \int_0^1 t^{s-1} \Theta_{\text{sub}}(t) dt + \int_1^\infty t^{s-1} \Theta_{\text{sub}}(t) dt.$$

The second integral is typically entire in s due to exponential decay of $\Theta(t)$ as $t \rightarrow \infty$. The first integral carries the singular structure and is controlled by the explicit subtraction terms.

Poles of the continued function correspond to the exponents μ_j and to the logarithmic terms, mirroring standard mechanisms in spectral geometry.

B.1.5 Arithmetic Dirichlet series from Mellin space

On the arithmetic side, prime-local expansions of renormalized traces yield Dirichlet-series-type objects of the form

$$\mathcal{D}(s) = \sum_{n=1}^\infty \frac{c(n)}{n^s},$$

where $c(n)$ is supported on prime powers or controlled multiplicative sets.

The von Mangoldt function appears naturally through the diagonal potential V_Λ . In Mellin space, it produces the classical Dirichlet series

$$\sum_{n=1}^\infty \frac{\Lambda(n)}{n^s} = -\frac{\zeta'(s)}{\zeta(s)}, \quad \Re(s) > 1,$$

providing a canonical bridge between operator-theoretic traces and Euler products.

B.1.6 Reflection symmetry and functional-equation constraints

The Mellin variable admits the involution $s \mapsto 1-s$. In the SMRK framework, reflection symmetry is expected to emerge at the level of renormalized trace invariants due to the symmetric pairing of forward and backward prime shifts and the logarithmic scaling structure of $\mathcal{H}$.

While no identification $\zeta_{\text{spec}}(s) = \zeta(s)$ is assumed, the existence of such reflection relations imposes strong constraints on admissible subtraction terms and requires renormalization schemes to preserve symmetry.

B.1.7 Concluding remark

The Mellin transform provides the natural language in which spectral zeta functions, renormalized traces, and arithmetic Dirichlet series meet. Analytic continuation is achieved by explicit subtraction of short-time divergences, and reflection symmetry acts as a consistency condition linking the operator construction to functional-equation-type structures.

APPENDIX C

C.1 Comparison with Related Operator Models

This appendix places the SMRK Hamiltonian within the broader landscape of operator-theoretic and spectral approaches to arithmetic and prime distribution. The emphasis is on structural similarities and fundamental differences rather than on claims of equivalence.

C.1.1 Hilbert–Pólya type proposals

The classical Hilbert–Pólya idea proposes the existence of a self-adjoint operator whose spectrum encodes the nontrivial zeros of the Riemann zeta function. In this paradigm, the primary goal is a direct spectral identification between eigenvalues and arithmetic zeros.

The SMRK framework departs from this objective. While it constructs a self-adjoint operator with rich arithmetic structure, it does not attempt to identify individual eigenvalues with zeta zeros. Instead, arithmetic information is accessed through trace invariants and their dual spectral and arithmetic expansions.

C.1.2 Berry–Keating type Hamiltonians

Berry–Keating type models are based on quantizations of the classical Hamiltonian xp and emphasize semiclassical scaling arguments. These models exhibit logarithmic confinement and naturally lead to spectral densities resembling those of the zeta zeros after regularization.

The SMRK Hamiltonian shares the appearance of logarithmic terms but differs fundamentally in its construction. Its dynamics are discrete and multiplicative rather than continuous and symplectic. No semiclassical limit is assumed, and renormalization arises from prime-density divergence rather than phase-space volume considerations.

C.1.3 Connes’ trace formula approach

Connes’ noncommutative geometric framework formulates the explicit formula as a trace formula on a suitable adelic space. In this approach, primes and zeros appear as dual spectral data of a single trace identity.

The SMRK construction aligns with this philosophy at a conceptual level: both frameworks emphasize trace invariants and dual representations. However, the operator algebras involved differ significantly. The SMRK Hamiltonian acts on a purely arithmetic Hilbert space and uses renormalized quadratic forms instead of adelic or noncommutative geometric structures.

C.1.4 Quantum graph models

Quantum graph models encode prime-like structures through lengths of periodic orbits and graph cycles. Trace formulas on such graphs often resemble explicit formulas after suitable averaging.

In contrast, the SMRK Hamiltonian encodes primes directly as generators of multiplicative shifts. Prime powers arise algebraically rather than through repetition of geometric cycles. This distinction eliminates the need for auxiliary graph constructions.

C.1.5 Random matrix models

Random matrix theory successfully predicts statistical properties of zeta zeros through universality arguments. These models are statistical rather than constructive and do not provide explicit operators encoding arithmetic data.

The SMRK framework is complementary in spirit. It provides a concrete operator model whose trace invariants can, in principle, be compared with random matrix statistics, but it does not rely on probabilistic assumptions.

C.1.6 Universality class considerations

Despite these differences, many of the models above share common structural features, such as logarithmic scaling, reflection symmetry, and trace-based analysis. The SMRK Hamiltonian can be viewed as belonging to a broader universality class of arithmetic operators characterized by multiplicative dynamics and renormalized trace invariants.

This viewpoint suggests that certain spectral phenomena are robust and do not depend on the detailed realization of the operator.

C.1.7 Advantages of the SMRK approach

The SMRK framework offers several conceptual advantages. It is fully discrete, avoids semiclassical approximations, and derives renormalization directly from prime-density divergence. Its reliance on quadratic forms ensures mathematical rigor and stability under perturbations.

Moreover, the absence of spectral identification claims allows the framework to remain falsifiable and adaptable.

C.1.8 Limitations and open questions

The principal limitation of the SMRK approach is the lack of explicit spectral information. While trace invariants are well defined and structured, extracting fine arithmetic data from them remains a challenging problem.

Open questions include the precise universality class of the SMRK Hamiltonian, its relation to known trace formulas, and the potential existence of geometric or dynamical models leading to equivalent operators.

C.1.9 Concluding remark

The SMRK Hamiltonian should be viewed not as a replacement for existing spectral models, but as a complementary framework emphasizing renormalized traces and arithmetic operators. Its value lies in providing a stable, operator-theoretic arena in which arithmetic trace formulas can be studied without speculative spectral identifications.

APPENDIX D

D.1 Conceptual and Physical Analogies

This appendix collects conceptual and physical analogies that help situate the SMRK Hamiltonian within a broader scientific context. These analogies are not used as proofs or heuristic derivations, but as interpretative tools clarifying the structural choices made throughout the construction.

D.1.1 Renormalization as arithmetic vacuum subtraction

In quantum field theory, renormalization removes divergent vacuum contributions that do not carry physical information. A closely analogous mechanism appears in the SMRK framework.

The divergence arising from the prime harmonic sum

$$\sum_{p \leq P} \frac{1}{p}$$

is universal and independent of the arithmetic state. Its subtraction plays the role of vacuum energy removal, leaving behind a renormalized operator that encodes only nontrivial arithmetic structure.

D.1.2 Arithmetic scale as energy scale

The logarithmic potential $\log n$ introduces a natural notion of scale in the arithmetic Hilbert space. Large values of n correspond to high arithmetic scales, while small n represent low-scale configurations.

This identification mirrors the role of energy scales in physical systems and provides a natural interpretation of the heat parameter t as an inverse arithmetic energy scale.

D.1.3 Prime shifts as interaction vertices

Prime shift operators generate discrete transitions between arithmetic states. From a physical viewpoint, these transitions resemble interaction vertices, where a state interacts with a prime quantum.

Forward and backward prime shifts appear symmetrically, reflecting time-reversal or charge-conjugation-type symmetry in the arithmetic dynamics.

D.1.4 Trace invariants as partition functions

The heat trace

$$\Theta(t) = \text{Tr}\left(e^{-tH_{\text{SMRK}}}\right)$$

admits an interpretation analogous to a partition function. It aggregates contributions from all arithmetic states weighted by their effective energy.

Renormalized trace invariants thus play a role similar to thermodynamic potentials, encoding global structural information rather than microscopic details.

D.1.5 Mellin transform and dual representations

The Mellin transform acts as a bridge between scale space and spectral space. In this respect it is analogous to the Fourier transform linking time and frequency.

Reflection symmetry in Mellin space corresponds to a duality between small and large arithmetic scales, reinforcing the interpretation of the construction as a scale-invariant or scale-dual system.

D.1.6 Comparison with quantum statistical systems

The SMRK Hamiltonian shares features with quantum statistical mechanical models, particularly those based on discrete spectra and trace-class evolution operators. However, unlike thermal systems, the arithmetic dynamics do not arise from probabilistic ensembles but from deterministic multiplicative structure.

This distinction underscores the arithmetic nature of the construction despite the formal similarity of the tools employed.

D.1.7 Universality and robustness

Many structural properties of the SMRK Hamiltonian, such as discreteness of the spectrum, trace-class heat evolution, and stability under perturbations, are robust under modifications of the model.

This robustness suggests that the observed features belong to a universality class of arithmetic operators governed by multiplicative dynamics and renormalized traces, rather than to a finely tuned construction.

D.1.8 Limits of physical analogy

While physical analogies provide intuition, they have clear limits. The SMRK Hamiltonian is not derived from a physical system, nor does it describe physical particles or fields.

All analogies must therefore be understood as structural correspondences rather than literal identifications.

D.1.9 Concluding perspective

The value of the SMRK framework lies in its synthesis of arithmetic structure, operator theory, and renormalization. Physical analogies help illuminate this synthesis, but the mathematical content stands independently of any physical interpretation.

In this sense, the SMRK Hamiltonian occupies a conceptual position analogous to that of spectral geometry: a domain where analytic, algebraic, and structural ideas converge without being reducible to any single interpretation.

Chapter 11

The SMRK Hamiltonian and Explicit Formulae

Self-Adjoint Arithmetic Dynamics, Resolvent Traces, and the Spectral Program

Enter Yourname

Abstract

This section outlines a concrete spectral program for the SMRK Hamiltonian, translating the abstract operator-theoretic framework developed in the preceding sections into a set of analyzable and testable objectives. Assuming the existence of an appropriate self-adjoint realization or extension, the spectral program seeks to characterize the spectrum, spectral measures, and associated statistical properties of the operator in a manner consistent with its arithmetic construction.

The spectral program is formulated in terms of several complementary goals. These include the identification of spectral invariants, the analysis of level statistics, and the study of stability properties under refinement of the underlying operator parameters. Rather than focusing exclusively on the existence of individual eigenvalues, the program emphasizes global spectral behavior, which is often more robust and more informative in arithmetic and physical contexts.

A central component of the program is the comparison between the spectral data of the SMRK Hamiltonian and predictions derived from classical models in random matrix theory and analytic number theory. Such comparisons provide a framework for assessing whether the operator exhibits signatures compatible with arithmetic phenomena, without presupposing a direct or exact correspondence. Both agreements and deviations are treated as meaningful outcomes, capable of guiding refinement of the operator model or domain assumptions.

The section also addresses numerical and computational aspects of the spectral program. Given the unbounded nature of the operator and the infinite-dimensional setting, any numerical investigation necessarily involves truncation, regularization, and approximation. The spectral program therefore incorporates explicit criteria for interpreting numerical results, distinguishing between genuine spectral features and artifacts of finite-dimensional approximation.

Importantly, the spectral program is framed as an open-ended research agenda rather than a closed proof strategy. Its purpose is not to assert

definitive arithmetic conclusions, but to establish a structured pathway through which evidence can be accumulated, hypotheses refined, and failure modes identified. This stance preserves the falsifiability and adaptability of the SMRK framework.

By articulating a clear spectral program, this section completes the transition from operator construction to analytical investigation, setting the stage for the explicit formulae, extensions, and generalizations explored in subsequent sections.

1 Motivation and Scope

1.1 The Hilbert–Pólya Vision Revisited

The Hilbert–Pólya conjecture proposes that the nontrivial zeros of the Riemann zeta function arise as spectral data of a self-adjoint operator. More precisely, it suggests the existence of a self-adjoint operator H whose spectrum encodes the imaginary parts of the zeros of $\zeta(s)$.

Despite a century of effort, no construction has succeeded in simultaneously satisfying the following essential requirements:

- genuine self-adjointness on a natural Hilbert space,
- intrinsic arithmetic structure (rather than imposed boundary conditions),
- a trace theory capable of reproducing explicit formulae,
- stability under perturbations and renormalization.

The present work does not claim a solution of the Riemann Hypothesis. Its purpose is more structural: to establish a mathematically rigorous arithmetic Hamiltonian whose spectral theory naturally interfaces with the explicit formulae of analytic number theory.

1.2 Why Explicit Formulae Are the Central Test

The classical explicit formulae of Riemann, Weil, and Guinand express a deep duality between primes and zeros. Schematically, they relate weighted sums over prime powers to weighted sums over nontrivial zeros of $\zeta(s)$ via test functions.

From an operator-theoretic perspective, explicit formulae are best understood as trace identities:

$$\text{spectral trace} \longleftrightarrow \text{arithmetic trace}. \quad (11.1)$$

Any candidate Hilbert–Pólya operator must therefore do more than possess a real spectrum. It must support a trace calculus whose arithmetic expansion reproduces prime-power data, while its spectral expansion is governed by the operator’s eigenvalues or resonances.

This observation motivates a shift of emphasis. The primary interface between arithmetic and spectrum is not the point spectrum itself, but resolvent and heat-kernel traces.

1.3 The SMRK Strategy

The SMRK Hamiltonian is designed from the outset to satisfy this trace-based philosophy. Its defining features are:

1. arithmetic states indexed by the positive integers rather than geometric space,

2. prime multiplication and division as fundamental dynamical moves,
3. a logarithmic confinement mechanism ensuring spectral discreteness,
4. renormalization implemented at the level of quadratic forms.

The Hamiltonian is not reverse-engineered from the zeta zeros. Instead, it arises from intrinsic arithmetic dynamics, with primes acting as generators of motion and the von Mangoldt function appearing as a logarithmic infinitesimal generator.

1.4 Two Complementary Pillars

The present whitepaper unifies two complementary developments:

- a rigorous construction of the SMRK Hamiltonian as a self-adjoint operator on $\ell^2(\mathbb{N}, 1/n)$ via renormalized quadratic forms,
- a spectral and resolvent-based program aimed at extracting arithmetic information through trace formulae, spectral measures, and explicit-formula analogues.

Individually, neither pillar suffices to address Hilbert–Pólya-type questions. Together, they form a coherent framework in which self-adjointness, spectrum, and arithmetic trace identities coexist.

1.5 What This Work Does and Does Not Claim

It is essential to state clearly the scope of the present program.

Established in this work:

- existence of a self-adjoint SMRK Hamiltonian with compact resolvent,
- a pure point spectrum generated by intrinsic arithmetic dynamics,
- a well-defined trace theory including heat kernel, spectral zeta, and resolvent traces,
- structural parallels with explicit formulae and zeta-theoretic symmetries.

Not claimed:

- no identification of individual eigenvalues with Riemann zeros,
- no proof of the Riemann Hypothesis,
- no bijection between spectra.

The SMRK Hamiltonian should therefore be viewed as a structural candidate and a testbed for the Hilbert–Pólya philosophy, not as a completed solution.

1.6 Organization of the Whitepaper

The paper is organized as follows.

Chapters 2 through 5 construct the arithmetic Hilbert space, define the SMRK Hamiltonian, and establish its self-adjointness and spectral confinement.

Chapters 6 through 9 develop the spectral program, including finite truncations, resolvents, trace formulae, and the interface with explicit arithmetic identities.

Chapters 10 and 11 analyze symmetry principles, projective structures, and the analogy with the functional equation of $\zeta(s)$.

Chapters 12 and 13 address falsifiability, level statistics, and the precise sense in which the SMRK framework may or may not advance the Hilbert–Pólya program.

Several appendices collect technical estimates, Mellin-space formulations, and numerical protocols.

1.7 Philosophical Remark

The guiding principle of this work is that arithmetic should generate dynamics, not merely decorate spectral formulae.

Primes are treated not as external labels, but as operators. Explicit formulae are treated not as miraculous identities, but as trace relations. Renormalization is treated not as a defect, but as a structural necessity.

Within this perspective, the SMRK Hamiltonian provides a concrete arena in which arithmetic, spectral theory, and operator calculus interact on equal footing.

2 Arithmetic States and Prime Dynamics

2.1 Arithmetic Hilbert Space

Let $\mathbb{N} = \{1, 2, 3, \dots\}$ denote the set of positive integers. We define the arithmetic Hilbert space by

$$\mathcal{H} := \ell^2(\mathbb{N}, w), \quad \langle \psi, \phi \rangle := \sum_{n \geq 1} \psi(n) \overline{\phi(n)} w(n), \quad (11.2)$$

with weight

$$w(n) := \frac{1}{n}. \quad (11.3)$$

The corresponding norm is

$$\|\psi\|^2 = \sum_{n \geq 1} \frac{|\psi(n)|^2}{n}. \quad (11.4)$$

We denote by

$$\mathcal{D}_0 := \{\psi : \mathbb{N} \rightarrow \mathbb{C} \mid \psi \text{ has finite support}\} \quad (11.5)$$

the dense subspace of finitely supported arithmetic states. All operators and quadratic forms introduced below are initially defined on $\mathcal{D}_0$.

Remark

The choice of the weight $w(n) = 1/n$ is essential. It ensures compatibility between multiplication and division by primes and yields nontrivial adjoint relations for the prime-shift operators.

2.2 Prime Shift and Co-Shift Operators

Let p denote a prime number. We define the forward (multiplicative) prime shift operator S_p and the backward (co-shift) operator T_p on $\mathcal{D}_0$ by

$$(S_p \psi)(n) := \psi(pn), \quad (T_p \psi)(n) := \mathbf{1}_{p|n} \psi(n/p), \quad (11.6)$$

where $\mathbf{1}_{p|n}$ denotes the indicator function of divisibility by p .

Norm relations

A direct computation using the weight $w(n) = 1/n$ yields

$$\|S_p\psi\|^2 = \sum_{n \geq 1} \frac{|\psi(pn)|^2}{n} = p \sum_{m \geq 1} \frac{|\psi(m)|^2}{m} = p\|\psi\|^2, \quad (11.7)$$

$$\|T_p\psi\|^2 = \sum_{n \geq 1} \frac{\mathbf{1}_{p|n} |\psi(n/p)|^2}{n} = \frac{1}{p} \sum_{m \geq 1} \frac{|\psi(m)|^2}{m} = \frac{1}{p}\|\psi\|^2. \quad (11.8)$$

Thus S_p is unbounded on $\mathcal{H}$, while T_p is bounded. Both are densely defined on $\mathcal{D}_0$.

Adjoint relations

For $\psi, \phi \in \mathcal{D}_0$, we compute

$$\langle S_p\psi, \phi \rangle = \sum_{n \geq 1} \psi(pn) \overline{\phi(n)} \frac{1}{n} = p \sum_{m \geq 1} \psi(m) \overline{\phi(m/p)} \frac{1}{m} = p \langle \psi, T_p\phi \rangle. \quad (11.9)$$

It follows that, on $\mathcal{D}_0$,

$$S_p^* = p T_p, \quad T_p^* = \frac{1}{p} S_p. \quad (11.10)$$

This adjoint pairing is the fundamental structural property of the arithmetic shifts.

2.3 Symmetric Prime Interaction

The combination

$$K_p := \frac{1}{p}(S_p + T_p) \quad (11.11)$$

defines a symmetric operator on $\mathcal{D}_0$. Indeed,

$$K_p^* = \frac{1}{p}(S_p^* + T_p^*) = \frac{1}{p}\left(pT_p + \frac{1}{p}S_p\right) = K_p. \quad (11.12)$$

The operators K_p represent elementary arithmetic transitions $n \leftrightarrow pn$ weighted by $1/p$. They play the role of kinetic terms in the SMRK Hamiltonian.

2.4 Diagonal Arithmetic Operators

Let $V : \mathbb{N} \rightarrow \mathbb{R}$ be a real-valued function. We define the diagonal multiplication operator by

$$(V\psi)(n) := V(n)\psi(n), \quad (11.13)$$

with associated quadratic form

$$q_V[\psi] := \sum_{n \geq 1} V(n) |\psi(n)|^2 \frac{1}{n}. \quad (11.14)$$

In the SMRK framework, the primary diagonal terms are

$$V(n) = \alpha \Lambda(n) + \beta \log n, \quad (11.15)$$

where $\Lambda(n)$ denotes the von Mangoldt function. Since $\Lambda(n) \leq \log n$, the growth of $V(n)$ is logarithmic.

2.5 Formal Arithmetic Hamiltonian

Combining the prime interactions with the diagonal term, we define the formal SMRK Hamiltonian on $\mathcal{D}_0$ by

$$H_{\text{SMRK}} := \sum_{p \in \mathbb{P}} \frac{1}{p} (S_p + T_p) + V. \quad (11.16)$$

The infinite prime sum does not converge in operator norm, nor is it a priori well defined on $\mathcal{H}$. Its proper interpretation requires the quadratic-form framework and renormalization, which will be developed in the next chapters.

2.6 Arithmetic Graph Interpretation

The operators S_p and T_p generate a graph structure on $\mathbb{N}$: vertices correspond to integers n , and edges connect n and pn for primes p .

The weighted adjacency encoded by $\frac{1}{p}(S_p + T_p)$ defines a nonlocal but controlled arithmetic dynamics. The weight $1/p$ suppresses large-prime transitions and is essential for the renormalized construction of a self-adjoint operator.

2.7 Summary

In this chapter we have:

- defined the arithmetic Hilbert space $\ell^2(\mathbb{N}, 1/n)$,
- introduced prime shift and co-shift operators and their adjoint relations,
- identified the symmetric prime interaction $\frac{1}{p}(S_p + T_p)$,
- defined diagonal arithmetic potentials of logarithmic growth,
- formulated the formal SMRK Hamiltonian.

The next chapter develops the quadratic-form approach required to give a rigorous meaning to the infinite-prime limit and to construct a self-adjoint realization of the SMRK Hamiltonian.

3 Prime-Cutoff Hamiltonians and Ultraviolet Divergence

3.1 Motivation for a Prime Cutoff

The formal SMRK Hamiltonian

$$H_{\text{SMRK}} = \sum_{p \in \mathbb{P}} \frac{1}{p} (S_p + T_p) + V \quad (11.17)$$

contains an infinite sum over all primes. As established in the previous chapter, this sum does not converge in operator norm and cannot be defined directly as an operator on the arithmetic Hilbert space.

To analyze the source of this obstruction, we introduce a prime cutoff and study the behavior of the resulting truncated Hamiltonians as the cutoff is removed. This procedure isolates the divergent component of the arithmetic interaction and reveals its precise structure.

3.2 Prime-Cutoff Hamiltonians

Let $P \geq 2$ be a cutoff parameter. We define the truncated SMRK Hamiltonian by

$$H_P := \sum_{\substack{p \in \mathbb{P} \\ p \leq P}} \frac{1}{p} (S_p + T_p) + V, \quad (11.18)$$

initially defined on the dense domain of finitely supported arithmetic states.

Since the prime sum is finite for each fixed P , the operator H_P is a well-defined symmetric operator on this domain. However, uniform control as $P \rightarrow \infty$ is nontrivial and requires a more refined analytic framework.

3.3 Associated Quadratic Forms

Rather than analyzing H_P directly at the operator level, we pass to the associated quadratic forms.

For ψ, ϕ in the finite-support domain, define the bilinear form

$$q_P(\psi, \phi) := \sum_{\substack{p \in \mathbb{P} \\ p \leq P}} \frac{1}{p} (\langle S_p \psi, \phi \rangle + \langle T_p \psi, \phi \rangle) + \langle V \psi, \phi \rangle, \quad (11.19)$$

and the associated quadratic form

$$q_P[\psi] := q_P(\psi, \psi). \quad (11.20)$$

Each q_P is densely defined and Hermitian on the finite-support domain.

3.4 Hermiticity of the Cutoff Forms

Using the adjoint relations for the prime-shift operators, we compute for ψ, ϕ in the domain:

$$\frac{1}{p} \langle S_p \psi, \phi \rangle = \langle \psi, T_p \phi \rangle, \quad \frac{1}{p} \langle T_p \psi, \phi \rangle = \frac{1}{p^2} \langle \psi, S_p \phi \rangle. \quad (11.21)$$

Since all coefficients are real, summation over primes yields

$$q_P(\psi, \phi) = q_P(\phi, \psi), \quad (11.22)$$

so each cutoff quadratic form is Hermitian.

3.5 Growth of Prime-Shift Contributions

We now estimate the prime-shift terms appearing in $q_P[\psi]$.

By the Cauchy–Schwarz inequality,

$$|\langle \psi, S_p \psi \rangle| \leq \|\psi\| \|S_p \psi\|. \quad (11.23)$$

Using the norm relation $\|S_p \psi\|^2 = p \|\psi\|^2$, we obtain

$$|\langle \psi, S_p \psi \rangle| \leq \sqrt{p} \|\psi\|^2. \quad (11.24)$$

Consequently,

$$\frac{1}{p} |\langle \psi, S_p \psi \rangle| \leq \frac{1}{\sqrt{p}} \|\psi\|^2. \quad (11.25)$$

An analogous estimate holds for the T_p term.

3.6 Ultraviolet Divergence

Summing the above bound over primes $p \leq P$ yields

$$\sum_{\substack{p \in \mathbb{P} \\ p \leq P}} \frac{1}{p} |\langle \psi, S_p \psi \rangle| \leq \left(\sum_{p \leq P} \frac{1}{\sqrt{p}} \right) \|\psi\|^2. \quad (11.26)$$

Since

$$\sum_{p \in \mathbb{P}} \frac{1}{\sqrt{p}} = \infty, \quad (11.27)$$

the prime-shift contribution to $q_P[\psi]$ diverges as $P \rightarrow \infty$ for generic states.

This divergence is:

- independent of fine arithmetic structure,
- proportional to the squared norm of the state,
- generated by large primes.

In analogy with quantum field theory, we refer to this phenomenon as an ultraviolet divergence.

3.7 Interpretation of the Divergence

The divergence identified above does not reflect instability of arithmetic dynamics, but rather the accumulation of infinitely many weak prime interactions.

Crucially, the divergent contribution is scalar in form sense. It corresponds to a uniform shift of the spectrum and therefore does not affect spectral spacings, trace invariants, or symmetry properties.

This observation suggests that the divergence can be removed by subtracting a suitable cutoff-dependent constant.

3.8 Necessity of Renormalization

The analysis above shows that:

- the naive infinite-prime quadratic form does not exist,
- the formal operator sum cannot be defined directly,
- renormalization is unavoidable.

Accordingly, we introduce a renormalized quadratic form

$$\tilde{q}_P(\psi, \phi) := q_P(\psi, \phi) - C(P) \langle \psi, \phi \rangle, \quad (11.28)$$

where the renormalization constant $C(P) \rightarrow \infty$ is chosen to cancel the divergent part of the prime sum.

The precise choice of $C(P)$ and the construction of a limiting quadratic form will be developed in the next chapter.

3.9 Summary

In this chapter we have:

- introduced prime-cutoff SMRK Hamiltonians,
- formulated the associated quadratic forms,
- identified the divergence arising from large primes,
- shown that the divergence is scalar and universal,
- established the necessity of renormalization.

The next chapter constructs a renormalized quadratic form, establishes uniform bounds, and proves the existence of a self-adjoint SMRK Hamiltonian in the infinite-prime limit.

4 Renormalized Quadratic Forms and Logarithmic Confinement

4.1 Structure of the Divergence

In the previous chapter we identified the divergence of the cutoff quadratic forms q_P as the prime cutoff P tends to infinity. The divergent contribution arises from large primes and is controlled by estimates of the form

$$\frac{1}{p} |\langle \psi, S_p \psi \rangle| \lesssim \frac{1}{\sqrt{p}} \|\psi\|^2. \quad (11.29)$$

Summation over primes therefore produces a divergence proportional to

$$\sum_{p \in \mathbb{P}} p^{-1/2}. \quad (11.30)$$

Crucially, this divergence is:

- independent of arithmetic correlations,
- proportional to the squared norm of the state,
- universal across states in the form domain.

Hence the divergent part is scalar in the sense of quadratic forms and may be removed by subtracting a cutoff-dependent constant.

4.2 Choice of the Renormalization Constant

Let $P \geq 2$ denote the prime cutoff. We define the renormalization constant by

$$C(P) := \sum_{\substack{p \in \mathbb{P} \\ p \leq P}} \frac{1}{\sqrt{p}}. \quad (11.31)$$

Although $C(P) \rightarrow \infty$ as $P \rightarrow \infty$, it grows sublinearly with P and depends only on the cutoff parameter. We define the renormalized quadratic form

$$\tilde{q}_P(\psi, \phi) := q_P(\psi, \phi) - C(P) \langle \psi, \phi \rangle, \quad (11.32)$$

initially on the finite-support domain.

Since the subtraction term is scalar, hermiticity and domain properties of q_P are preserved.

4.3 Reference Quadratic Form and Confinement

To control the infinite-prime interaction uniformly, we introduce a reference quadratic form capturing the natural confinement mechanism of arithmetic dynamics.

Define

$$q_0[\psi] := \sum_{n \geq 1} (1 + \log^2 n) |\psi(n)|^2 \frac{1}{n}. \quad (11.33)$$

The associated form domain is

$$D(q_0) := \left\{ \psi \in \mathcal{H} \left| \sum_{n \geq 1} (1 + \log^2 n) |\psi(n)|^2 \frac{1}{n} < \infty \right. \right\}. \quad (11.34)$$

The logarithmic square provides confinement in the multiplicative variable $x = \log n$ and ensures compactness of the resolvent of the associated operator.

4.4 Decomposition of the Renormalized Form

We decompose the renormalized form as

$$\tilde{q}_P = q_0 + b_P, \quad (11.35)$$

where the interaction form b_P is given by

$$\begin{aligned} b_P(\psi, \phi) := & \sum_{\substack{p \in \mathbb{P} \\ p \leq P}} \frac{1}{p} (\langle S_p \psi, \phi \rangle + \langle T_p \psi, \phi \rangle) \\ & + \langle (V - 1 - \log^2 n) \psi, \phi \rangle - C(P) \langle \psi, \phi \rangle. \end{aligned} \quad (11.36)$$

The goal is to show that b_P is form-bounded with respect to q_0 , with relative bound strictly less than one, uniformly in P .

4.5 Control of Prime Shifts in the Reference Norm

Let $\|\cdot\|_{q_0}$ denote the form norm induced by q_0 . For ψ of finite support, we estimate

$$q_0[S_p \psi] = \sum_{n \geq 1} (1 + \log^2 n) |\psi(pn)|^2 \frac{1}{n}. \quad (11.37)$$

Changing variables $m = pn$ yields

$$q_0[S_p \psi] = p \sum_{m \geq 1} (1 + \log^2(m/p)) |\psi(m)|^2 \frac{1}{m}. \quad (11.38)$$

Using elementary logarithmic inequalities, one obtains

$$q_0[S_p \psi] \leq 2p q_0[\psi] + 2p \log^2 p \|\psi\|^2. \quad (11.39)$$

An analogous estimate holds for the co-shift operator T_p .

4.6 Form-Boundedness of the Interaction

Applying the Cauchy–Schwarz inequality in the q_0 -norm, we obtain

$$|\langle \psi, S_p \psi \rangle| \leq \|\psi\|_{q_0} \|S_p \psi\|_{q_0}. \quad (11.40)$$

Combining this with the previous bounds yields, for any $\varepsilon > 0$,

$$\frac{1}{p} |\langle \psi, S_p \psi \rangle| \leq \varepsilon q_0[\psi] + K_\varepsilon \frac{\log^2 p}{p} \|\psi\|^2, \quad (11.41)$$

with a constant K_ε independent of P .

Summation over primes and subtraction of $C(P)$ therefore imply a uniform bound

$$|b_P(\psi, \psi)| \leq \varepsilon q_0[\psi] + K_\varepsilon \|\psi\|^2, \quad (11.42)$$

valid for all ψ in the finite-support domain and uniformly in P .

4.7 Consequences

The uniform form-boundedness established above implies that the family of renormalized quadratic forms $\tilde{q}_P$ is closed, semi-bounded, and defined on a common form domain.

This provides the key analytic control required to take the infinite-prime limit and construct a self-adjoint realization of the SMRK Hamiltonian.

4.8 Summary

In this chapter we have:

- identified the divergent structure of the prime interaction,
- introduced a renormalization constant removing the divergence,
- constructed a logarithmically confining reference form,
- proved uniform form-boundedness of the interaction,
- established closedness and semi-boundedness of the renormalized forms.

The next chapter takes the infinite-prime limit and proves the existence and uniqueness of the self-adjoint SMRK Hamiltonian.

5 Infinite-Prime Limit and Construction of the Self-Adjoint SMRK Hamiltonian

5.1 Strategy of the Infinite-Prime Limit

In the previous chapters we constructed, for each prime cutoff P , a renormalized quadratic form

$$\tilde{q}_P = q_0 + b_P \quad (11.43)$$

defined on a common form domain. The reference form q_0 is closed and strictly positive, while the interaction term b_P is form-bounded with respect to q_0 with relative bound zero, uniformly in P .

The goal of this chapter is to take the limit as P tends to infinity in the sense of quadratic forms and to apply the representation theorem in order to obtain a unique self-adjoint operator associated with the limiting form.

5.2 Pointwise Convergence on the Core

Let ψ and ϕ be finitely supported arithmetic states. There exists a finite integer N such that $\psi(n) = \phi(n) = 0$ for all $n > N$.

For any prime $p > N$, both $T_p\psi$ and the inner product $\langle S_p\psi, \phi \rangle$ vanish identically. Consequently, for all sufficiently large cutoffs P , the only P -dependence of $\tilde{q}_P(\psi, \phi)$ arises from the scalar subtraction term involving the renormalization constant.

By construction, this scalar contribution stabilizes in form sense. Therefore, for all finitely supported ψ and ϕ , the limit

$$\tilde{q}(\psi, \phi) := \lim_{P \rightarrow \infty} \tilde{q}_P(\psi, \phi) \quad (11.44)$$

exists and is finite.

5.3 Uniform Lower Bounds and Closedness

From the uniform form-boundedness established previously, there exists a constant $K > 0$, independent of P , such that

$$\tilde{q}_P[\psi] \geq -K \|\psi\|^2 \quad (11.45)$$

for all ψ in the form domain.

Moreover, the form norm defined by

$$\|\psi\|_{\tilde{q}_P}^2 := \tilde{q}_P[\psi] + (K + 1)\|\psi\|^2 \quad (11.46)$$

is uniformly equivalent to the reference form norm induced by q_0 .

These uniform bounds imply that the family $\{\tilde{q}_P\}$ is Cauchy in the sense of quadratic forms.

5.4 Existence of the Limit Quadratic Form

We define the limiting quadratic form $\tilde{q}$ initially on finitely supported states by the pointwise limit above and extend it to the full form domain by continuity.

The resulting quadratic form is densely defined, closed, and semi-bounded from below on the arithmetic Hilbert space. Its form domain coincides with the domain of the reference form q_0 .

5.5 Representation Theorem and Self-Adjointness

By the representation theorem for closed, semi-bounded quadratic forms, there exists a unique self-adjoint operator, denoted by $\tilde{H}_{\text{SMRK}}$, such that

$$\tilde{q}(\psi, \phi) = \langle \tilde{H}_{\text{SMRK}}\psi, \phi \rangle \quad (11.47)$$

for all ψ in the operator domain and all ϕ in the form domain.

The operator $\tilde{H}_{\text{SMRK}}$ is bounded from below and generates a strongly continuous one-parameter unitary group.

5.6 Identification with the Formal SMRK Hamiltonian

On finitely supported arithmetic states, the action of $\tilde{H}_{\text{SMRK}}$ coincides with the formal SMRK Hamiltonian up to an additive constant arising from renormalization.

Explicitly, for such states the operator acts as

$$(\tilde{H}_{\text{SMRK}}\psi)(n) = \sum_{p \in \mathbb{P}} \frac{1}{p} (\psi(pn) + \mathbf{1}_{p|n} \psi(n/p)) + (\alpha \Lambda(n) + \beta \log n) \psi(n) - C_\infty \psi(n), \quad (11.48)$$

where C_∞ denotes the limiting renormalization constant.

This constant represents a global spectral shift and has no effect on spectral spacings, trace invariants, or symmetry properties.

5.7 Main Result

The construction above establishes the existence of a unique self-adjoint realization of the SMRK Hamiltonian on the arithmetic Hilbert space. This realization is bounded from below, has domain contained in the form domain of q_0 , and agrees with the formal arithmetic Hamiltonian on the natural core up to an additive constant.

5.8 Summary

In this chapter we have:

- taken the infinite-prime limit of the renormalized quadratic forms,
- constructed a closed and semi-bounded limiting form,
- applied the representation theorem to obtain a self-adjoint operator,
- identified this operator with the formal SMRK Hamiltonian up to renormalization.

This completes the rigorous construction of the SMRK Hamiltonian. The following chapters develop its spectral properties and trace-based arithmetic interface.

6 Spectral Properties of the SMRK Hamiltonian

6.1 Preliminaries

Let $\tilde{H}_{\text{SMRK}}$ denote the self-adjoint SMRK Hamiltonian constructed in the previous chapter. By construction, this operator is bounded from below and is associated with the closed quadratic form $\tilde{q}$ defined on the form domain of the reference quadratic form.

Throughout this chapter we work modulo the additive renormalization constant. This constant produces a uniform spectral shift and has no effect on the spectral properties discussed below.

6.2 Compactness of the Resolvent

A key structural feature of $\tilde{H}_{\text{SMRK}}$ is the logarithmic confinement induced by the reference quadratic form

$$q_0[\psi] = \sum_{n \geq 1} (1 + \log^2 n) |\psi(n)|^2 \frac{1}{n}. \quad (11.49)$$

Let H_0 denote the self-adjoint operator associated with q_0 . Since the function $1 + \log^2 n$ diverges as $n \rightarrow \infty$, the operator $(H_0 + i)^{-1}$ is compact on the arithmetic Hilbert space.

The interaction $\tilde{H}_{\text{SMRK}} - H_0$ is relatively form-bounded with respect to H_0 with relative bound zero. Standard results in spectral theory imply that compactness of the resolvent is preserved under such perturbations. Consequently, the resolvent

$$(\tilde{H}_{\text{SMRK}} + i)^{-1} \quad (11.50)$$

is a compact operator.

6.3 Pure Point Spectrum

Compactness of the resolvent imposes strong restrictions on the spectrum of $\tilde{H}_{\text{SMRK}}$.

The spectrum consists entirely of isolated eigenvalues of finite multiplicity, with no continuous or residual spectrum. Moreover, the eigenvalues diverge to positive infinity.

Thus the spectrum is purely discrete and may be written as a sequence

$$\sigma(\tilde{H}_{\text{SMRK}}) = \{\lambda_k\}_{k \geq 1}, \quad \lambda_k \rightarrow +\infty \text{ as } k \rightarrow \infty. \quad (11.51)$$

6.4 Eigenfunction Regularity and Localization

Let ψ_k denote a normalized eigenfunction corresponding to the eigenvalue λ_k :

$$\tilde{H}_{\text{SMRK}} \psi_k = \lambda_k \psi_k. \quad (11.52)$$

Since ψ_k belongs to the operator domain of $\tilde{H}_{\text{SMRK}}$, it lies in the form domain of q_0 . Consequently,

$$\sum_{n \geq 1} \log^2 n |\psi_k(n)|^2 \frac{1}{n} < \infty. \quad (11.53)$$

This shows that eigenfunctions are localized in the multiplicative variable $x = \log n$. Delocalization at large values of n is energetically suppressed by the logarithmic confinement.

6.5 Growth of Eigenvalues

The confining reference operator H_0 provides a comparison principle for the asymptotic growth of the eigenvalues of $\tilde{H}_{\text{SMRK}}$.

Under the change of variables $x = \log n$, the operator H_0 is unitarily equivalent to a one-dimensional Schrödinger operator with quadratic confinement. Standard Weyl-type asymptotics therefore apply.

As a consequence, there exist positive constants c_1 and c_2 such that

$$c_1 k^2 \leq \lambda_k \leq c_2 k^2 \quad (11.54)$$

for all sufficiently large k . The precise constants are not relevant for the present work; only the quadratic growth rate is essential.

6.6 Spectral Decomposition

Since $\tilde{H}_{\text{SMRK}}$ has purely discrete spectrum, the spectral theorem yields the expansion

$$\tilde{H}_{\text{SMRK}} = \sum_{k \geq 1} \lambda_k |\psi_k\rangle \langle \psi_k|, \quad (11.55)$$

with convergence in the strong operator topology.

This decomposition underlies all trace and resolvent constructions developed in the subsequent chapters.

6.7 Spectral Projections and Counting Function

Define the eigenvalue counting function by

$$N(E) := \#\{k : \lambda_k \leq E\}. \quad (11.56)$$

Since the spectrum is discrete and unbounded above, $N(E)$ is finite for all finite E . Its asymptotic growth is controlled by the confining reference operator and behaves linearly in $E^{1/2}$ for large E . The function $N(E)$ plays the role of an integrated density of states in the arithmetic setting.

6.8 Consequences for Trace Objects

Compactness of the resolvent implies that for a broad class of test functions f , the operator $f(\tilde{H}_{\text{SMRK}})$ is trace class. In particular,

$$\text{Tr } f(\tilde{H}_{\text{SMRK}}) = \sum_{k \geq 1} f(\lambda_k) \quad (11.57)$$

is well defined.

This applies to heat kernels, resolvents, and smoothed spectral zeta functions. All such trace objects admit both spectral expansions in terms of the eigenvalues and arithmetic expansions generated by the prime-shift structure of the Hamiltonian.

6.9 Summary

In this chapter we have established that:

- the SMRK Hamiltonian has compact resolvent,
- its spectrum is purely discrete,
- eigenfunctions are logarithmically localized,
- eigenvalues exhibit quadratic growth,
- trace-class spectral objects are well defined.

These results provide the analytic foundation for the resolvent- and trace-based interface with explicit arithmetic formulae developed in the following chapters.

7 Resolvent, Spectral Measures, and the Arithmetic IDS

7.1 Resolvent and Spectral Theorem

Let $\tilde{H}_{\text{SMRK}}$ denote the self-adjoint SMRK Hamiltonian with purely discrete spectrum $\{\lambda_k\}_{k \geq 1}$ and normalized eigenfunctions $\{\psi_k\}_{k \geq 1}$.

For complex numbers z not belonging to the real axis, we define the resolvent operator by

$$R(z) := (\tilde{H}_{\text{SMRK}} - z)^{-1}. \quad (11.58)$$

By the spectral theorem, the resolvent admits the expansion

$$R(z) = \sum_{k \geq 1} \frac{1}{\lambda_k - z} |\psi_k\rangle \langle \psi_k|, \quad (11.59)$$

with convergence in operator norm for nonzero imaginary part of z .

7.2 Local Spectral Measures

For any state φ in the arithmetic Hilbert space, we define the associated spectral measure μ_φ by

$$\mu_\varphi(\Delta) := \langle E_{\widetilde{H}_{\text{SMRK}}}(\Delta)\varphi, \varphi \rangle, \quad (11.60)$$

where $E_{\widetilde{H}_{\text{SMRK}}}$ denotes the projection-valued spectral measure of the operator.

Since the spectrum is discrete, the measure μ_φ is purely atomic and takes the form

$$\mu_\varphi = \sum_{k \geq 1} |\langle \varphi, \psi_k \rangle|^2 \delta_{\lambda_k}. \quad (11.61)$$

7.3 Stieltjes Transform Representation

Matrix elements of the resolvent are given by the Stieltjes transform of the corresponding spectral measures. Explicitly,

$$\langle R(z)\varphi, \varphi \rangle = \int_{\mathbb{R}} \frac{1}{\lambda - z} d\mu_\varphi(\lambda). \quad (11.62)$$

This representation is central for analytic continuation, boundary value analysis, and contour deformation in the complex plane.

7.4 Arithmetic Basis and Diagonal Spectral Weights

Let δ_n denote the arithmetic basis vector supported at the integer n . We define the local arithmetic spectral measure by

$$\mu_n := \mu_{\delta_n}. \quad (11.63)$$

Explicitly, this measure is given by

$$\mu_n = \sum_{k \geq 1} |\psi_k(n)|^2 \delta_{\lambda_k}. \quad (11.64)$$

The diagonal resolvent element at n takes the form

$$R_{nn}(z) := \langle R(z)\delta_n, \delta_n \rangle = \sum_{k \geq 1} \frac{|\psi_k(n)|^2}{\lambda_k - z}. \quad (11.65)$$

7.5 Arithmetic Integrated Density of States

We define the arithmetic integrated density of states by

$$N(E) := \#\{k : \lambda_k \leq E\}. \quad (11.66)$$

Equivalently, $N(E)$ can be recovered from the trace of spectral projections:

$$N(E) = \text{Tr } E_{\widetilde{H}_{\text{SMRK}}}((-\infty, E]). \quad (11.67)$$

For sufficiently regular test functions f , one has

$$\text{Tr } f(\widetilde{H}_{\text{SMRK}}) = \sum_{k \geq 1} f(\lambda_k) = \int_{\mathbb{R}} f(\lambda) dN(\lambda). \quad (11.68)$$

7.6 Weighted Arithmetic Traces

To connect spectral data with arithmetic structure, we introduce weighted trace functionals. Let $s > 0$ and define the diagonal weight operator by

$$(W_s \psi)(n) := n^{-s} \psi(n). \quad (11.69)$$

We define the weighted resolvent trace by

$$R(z; s) := \text{Tr}(W_s R(z) W_s). \quad (11.70)$$

Using the spectral decomposition of the resolvent, we obtain

$$R(z; s) = \sum_{k \geq 1} \frac{1}{\lambda_k - z} \sum_{n \geq 1} n^{-2s} |\psi_k(n)|^2. \quad (11.71)$$

The inner sum defines the arithmetic Mellin weight of the eigenfunction ψ_k .

7.7 Trace-Class Conditions

Since the resolvent is compact and the weight operator W_s is Hilbert–Schmidt for $s > 1/2$, the operator $W_s R(z) W_s$ is trace class whenever the imaginary part of z is nonzero.

Consequently, the weighted resolvent trace $R(z; s)$ is a well-defined analytic function of z away from the real axis.

7.8 Boundary Values and Spectral Density

Let $z = E + i\varepsilon$ with $\varepsilon > 0$. Then

$$\text{Im } R(E + i\varepsilon; s) = \int_{\mathbb{R}} \frac{\varepsilon}{(\lambda - E)^2 + \varepsilon^2} d\mu_s(\lambda), \quad (11.72)$$

where the weighted spectral measure $d\mu_s$ is given by

$$d\mu_s(\lambda) = \sum_{k \geq 1} \left(\sum_{n \geq 1} n^{-2s} |\psi_k(n)|^2 \right) \delta_{\lambda_k}. \quad (11.73)$$

As ε tends to zero, the right-hand side converges to a weighted spectral density. This provides a direct route to level statistics and unfolding procedures.

7.9 Role in the Explicit Formula Program

The function $R(z; s)$ serves as the central analytic object for the explicit-formula interface. As a function of z , it admits contour integral representations. As a function of s , it generates arithmetic weights via Mellin transforms.

Its poles encode spectral data of the SMRK Hamiltonian, while its large- $|z|$ expansion produces prime-power contributions. These properties will be exploited in subsequent chapters to derive explicit arithmetic–spectral identities.

7.10 Summary

In this chapter we have:

- defined the resolvent as a Stieltjes transform of spectral measures,
- introduced local arithmetic spectral measures,
- defined the arithmetic integrated density of states,
- constructed weighted resolvent traces,
- established analyticity and trace-class properties.

The next chapter develops trace objects and derives their arithmetic expansions, providing the final ingredients for the explicit-formula interface.

8 Trace Objects and Arithmetic Expansions

8.1 Overview of Trace Objects

The spectral analysis developed in the previous chapters allows one to define a family of trace-class operators associated with the SMRK Hamiltonian. These trace objects provide the principal interface between spectral data and arithmetic structure.

Throughout this chapter, all traces are understood modulo the additive renormalization constant, which produces only a global spectral shift and does not affect trace identities.

8.2 Heat Kernel

Let $t > 0$. We define the heat kernel associated with the SMRK Hamiltonian by

$$K(t) := \mathrm{Tr} \, e^{-t\tilde{H}_{\mathrm{SMRK}}}. \quad (11.74)$$

Since $\tilde{H}_{\mathrm{SMRK}}$ has purely discrete spectrum growing at least quadratically, the heat kernel is finite for all $t > 0$ and admits the spectral expansion

$$K(t) = \sum_{k \geq 1} e^{-t\lambda_k}. \quad (11.75)$$

8.3 Resolvent Trace

For complex numbers z with nonzero imaginary part, we define the resolvent trace by

$$R(z) := \mathrm{Tr} \, (\tilde{H}_{\mathrm{SMRK}} - z)^{-1}. \quad (11.76)$$

Using the spectral decomposition, one obtains

$$R(z) = \sum_{k \geq 1} \frac{1}{\lambda_k - z}. \quad (11.77)$$

The resolvent trace is analytic in z away from the real axis and encodes the spectral distribution of the Hamiltonian.

8.4 Spectral Zeta Function

For $\operatorname{Re} s$ sufficiently large, we define the spectral zeta function by

$$\zeta_{\text{spec}}(s) := \operatorname{Tr} \tilde{H}_{\text{SMRK}}^{-s} = \sum_{k \geq 1} \lambda_k^{-s}. \quad (11.78)$$

The quadratic growth of the eigenvalues implies convergence for $\operatorname{Re} s > 1/2$. Analytic continuation may be achieved via Mellin transforms of the heat kernel.

8.5 Mellin Representation

The spectral zeta function admits the integral representation

$$\zeta_{\text{spec}}(s) = \frac{1}{\Gamma(s)} \int_0^\infty t^{s-1} K(t) dt, \quad (11.79)$$

valid in the domain of absolute convergence.

This relation provides a bridge between short-time heat kernel asymptotics and meromorphic properties of $\zeta_{\text{spec}}(s)$.

8.6 Arithmetic Expansion Strategy

The central observation of the SMRK framework is that trace objects admit a second, arithmetic expansion generated by the prime-shift structure of the Hamiltonian.

Formally expanding the exponential

$$e^{-t\tilde{H}_{\text{SMRK}}} \quad (11.80)$$

in powers of the prime interaction yields a sum over arithmetic paths composed of multiplications and divisions by primes.

Each closed arithmetic path contributes to the trace with a weight determined by the product of the corresponding prime factors.

8.7 Prime Power Contributions

At leading order, the arithmetic expansion of the heat kernel trace produces terms of the form

$$\sum_{p \in \mathbb{P}} \sum_{m \geq 1} \frac{\log p}{p^{m/2}} e^{-tm \log p}, \quad (11.81)$$

where m indexes repeated prime transitions.

These contributions correspond to closed arithmetic loops of length m associated with the prime p . The factor $\log p$ arises from the diagonal von Mangoldt term in the Hamiltonian.

8.8 Comparison with Explicit Formulae

The structure of the prime power expansion mirrors the prime side of classical explicit formulae in analytic number theory.

Schematically, one obtains relations of the form

$$\sum_k f(\lambda_k) \longleftrightarrow \sum_{p \in \mathbb{P}} \sum_{m \geq 1} w_{p,m} \hat{f}(m \log p), \quad (11.82)$$

where f is a test function and $\hat{f}$ denotes an appropriate transform.

The precise correspondence depends on the choice of trace object and test function and will be analyzed in detail in the following chapters.

8.9 Resolvent Expansion

A similar arithmetic expansion applies to the resolvent trace. For large values of the spectral parameter z , one may expand

$$(\tilde{H}_{\text{SMRK}} - z)^{-1} = -\frac{1}{z} \sum_{n \geq 0} \left(\frac{\tilde{H}_{\text{SMRK}}}{z} \right)^n. \quad (11.83)$$

Tracing this expansion yields a series whose coefficients are weighted sums over arithmetic paths. The leading terms reproduce logarithmic prime sums analogous to those appearing in explicit formulae.

8.10 Universality of the Arithmetic Expansion

The arithmetic expansion of trace objects is universal in the sense that:

- it depends only on the prime-shift structure of the Hamiltonian,
- it is insensitive to the choice of basis,
- it persists under renormalization.

These properties indicate that the appearance of explicit-formula-type expressions is structural rather than accidental.

8.11 Summary

In this chapter we have:

- introduced the principal trace objects associated with the SMRK Hamiltonian,
- developed their spectral representations,
- outlined their arithmetic expansions,
- identified prime-power contributions,
- connected the resulting expressions with classical explicit formulae.

The next chapter makes this connection precise by deriving explicit arithmetic identities from the resolvent and heat-kernel traces.

9 Explicit Formulae from the SMRK Hamiltonian

9.1 Objective of the Explicit Formula Program

The purpose of this chapter is to derive explicit arithmetic–spectral identities directly from the SMRK Hamiltonian. These identities play the role of explicit formulae in the classical sense: they relate weighted sums over spectral data to weighted sums over prime powers.

Unlike traditional approaches, the explicit formulae here arise from trace objects associated with a self-adjoint operator whose dynamics are intrinsically arithmetic.

9.2 Test Functions and Functional Calculus

Let $f : \mathbb{R} \rightarrow \mathbb{C}$ be a smooth test function with sufficient decay at infinity. By the functional calculus for self-adjoint operators, the operator $f(\tilde{H}_{\text{SMRK}})$ is trace class for a broad class of such functions.

The corresponding trace admits the spectral representation

$$\text{Tr } f(\tilde{H}_{\text{SMRK}}) = \sum_{k \geq 1} f(\lambda_k). \quad (11.84)$$

The goal is to compute this trace alternatively using the arithmetic structure of the Hamiltonian.

9.3 Resolvent-Based Representation

A convenient starting point is the Cauchy integral representation

$$f(\tilde{H}_{\text{SMRK}}) = \frac{1}{2\pi i} \int_{\Gamma} f(z) (\tilde{H}_{\text{SMRK}} - z)^{-1} dz, \quad (11.85)$$

where Γ is a contour enclosing the spectrum.

Taking the trace yields

$$\text{Tr } f(\tilde{H}_{\text{SMRK}}) = \frac{1}{2\pi i} \int_{\Gamma} f(z) R(z) dz, \quad (11.86)$$

with $R(z)$ the resolvent trace.

9.4 Large-Parameter Expansion of the Resolvent

For large values of $|z|$, the resolvent admits the expansion

$$(\tilde{H}_{\text{SMRK}} - z)^{-1} = -\frac{1}{z} \sum_{m \geq 0} \left(\frac{\tilde{H}_{\text{SMRK}}}{z} \right)^m. \quad (11.87)$$

Substituting this expansion into the contour integral and interchanging summation and integration yields an asymptotic expansion of the trace in inverse powers of z .

Each term involves traces of powers of the Hamiltonian, which may be evaluated arithmetically.

9.5 Arithmetic Evaluation of Powers

The operator $\tilde{H}_{\text{SMRK}}$ decomposes into prime-shift terms and a diagonal potential. Powers of the Hamiltonian therefore expand into sums over products of prime-shift operators.

Only closed arithmetic paths contribute to the trace. Such paths correspond to finite sequences of prime multiplications and divisions whose net effect is the identity.

Each closed path is naturally associated with a prime power p^m .

9.6 Prime Power Expansion

Collecting contributions from closed arithmetic paths yields an expansion of the form

$$\mathrm{Tr} f(\tilde{H}_{\mathrm{SMRK}}) = \sum_{k \geq 1} f(\lambda_k) = f_0 + \sum_{p \in \mathbb{P}} \sum_{m \geq 1} a_{p,m} \hat{f}(m \log p), \quad (11.88)$$

where $\hat{f}$ denotes a transform of the test function determined by the functional calculus, and $a_{p,m}$ are explicit arithmetic coefficients.

The leading coefficients are proportional to the von Mangoldt function and decay exponentially in $m \log p$.

9.7 Comparison with the Classical Explicit Formula

The structure of the above identity mirrors the classical explicit formula relating zeros of the Riemann zeta function to primes.

In particular:

- spectral sums replace sums over zeta zeros,
- prime power sums appear with logarithmic weights,
- the test function and its transform link the two sides.

The essential difference is conceptual: the spectral data arise from an explicit self-adjoint operator rather than from an analytic continuation argument.

9.8 Symmetry and the Functional Equation Analogy

The SMRK Hamiltonian exhibits an intrinsic symmetry under inversion in the multiplicative variable. At the level of trace identities, this symmetry manifests itself as a duality between contributions from large and small spectral parameters.

This duality plays a role analogous to the functional equation in classical zeta-function theory. However, it is implemented operator-theoretically rather than analytically.

9.9 Stability Under Renormalization

Since the renormalization procedure introduces only a scalar spectral shift, all explicit formulae derived from trace differences are invariant under renormalization.

Prime-power coefficients and spectral spacings are therefore intrinsic to the SMRK Hamiltonian and do not depend on the particular regularization scheme.

9.10 Interpretation

The explicit formulae obtained here demonstrate that the SMRK Hamiltonian satisfies the defining criterion of the Hilbert–Pólya philosophy at the level of trace identities.

The operator does not merely possess a discrete spectrum; its trace calculus naturally reproduces the prime-power structure characteristic of analytic number theory.

9.11 Summary

In this chapter we have:

- derived trace identities from the resolvent and functional calculus,
- obtained explicit arithmetic expansions involving prime powers,
- established a direct analogy with classical explicit formulae,
- shown stability of these identities under renormalization.

The next chapter investigates symmetry principles and operator-theoretic analogues of functional equations within the SMRK framework.

10 Symmetries, Dualities, and Functional Equation Analogues

10.1 Motivation

A central feature of classical zeta-function theory is the functional equation, which encodes a deep symmetry relating values at s and $1 - s$. In the SMRK framework, no analytic continuation of a Dirichlet series is postulated. Instead, symmetry must emerge at the operator-theoretic level. This chapter identifies intrinsic symmetries of the SMRK Hamiltonian and explains how they give rise to functional-equation-like dualities in trace identities.

10.2 Multiplicative Inversion Symmetry

The arithmetic Hilbert space is naturally equipped with an involutive transformation defined by inversion in the multiplicative variable. Formally, this transformation acts by

$$(\mathcal{I}\psi)(n) := \psi(n^{-1}), \quad (11.89)$$

interpreted in the logarithmic variable $x = \log n$ as spatial reflection $x \mapsto -x$.

Although $\mathcal{I}$ is not defined pointwise on $\mathbb{N}$, its action is implemented unitarily after passage to the logarithmic representation.

10.3 Prime Shift Covariance

Under inversion in the logarithmic variable, multiplication and division by primes are exchanged. At the operator level, this induces the relations

$$\mathcal{I}S_p\mathcal{I}^{-1} = T_p, \quad \mathcal{I}T_p\mathcal{I}^{-1} = S_p. \quad (11.90)$$

Consequently, the symmetric combination

$$\frac{1}{p}(S_p + T_p) \quad (11.91)$$

is invariant under inversion.

This invariance is a key structural reason for the appearance of functional-equation analogues in trace identities.

10.4 Diagonal Terms and Symmetry Breaking

The diagonal potential of the SMRK Hamiltonian contains the von Mangoldt function and a logarithmic term. Under inversion, the logarithmic component is odd in the variable $x = \log n$ and therefore changes sign.

The presence of the logarithmic potential thus softly breaks exact inversion symmetry. This symmetry breaking is controlled and essential for confinement.

The resulting situation is analogous to a harmonic oscillator perturbed by a linear term: symmetry is broken, but in a way that preserves spectral stability.

10.5 Duality of Trace Objects

Trace objects derived from the SMRK Hamiltonian exhibit a duality between short-time and long-time behavior.

For the heat kernel, this duality relates the small- t expansion to the large- t decay, mirroring the $t \leftrightarrow 1/t$ symmetry familiar from theta functions.

At the level of resolvent traces, an analogous duality relates large- $|z|$ asymptotics to near-spectrum behavior.

10.6 Functional Equation Analogue

Combining inversion symmetry with trace dualities yields identities of the form

$$\mathcal{T}(f) = \mathcal{T}(\mathcal{F}f), \quad (11.92)$$

where $\mathcal{T}$ denotes a trace functional and $\mathcal{F}$ is a transform interchanging spectral and arithmetic variables.

These identities play the role of functional equations in the SMRK framework. They are exact at the operator level and do not rely on analytic continuation.

10.7 Spectral Reflection Principle

Let E denote the spectral parameter. The inversion symmetry implies that spectral contributions at energy E are paired with contributions at reflected energies, up to controlled correction terms induced by the diagonal potential.

This pairing constrains the distribution of eigenvalues and enforces symmetry in spectral fluctuations.

10.8 Implications for Level Statistics

The presence of inversion-induced dualities suggests that the local statistics of eigenvalue spacings should be invariant under reflection about the spectral center.

While the SMRK framework does not predict a specific random matrix ensemble, it imposes nontrivial symmetry constraints on admissible statistics.

10.9 Stability of Symmetry Under Renormalization

Renormalization introduces only a scalar spectral shift. Since inversion symmetry and prime-shift covariance act at the level of operator commutation relations, they are insensitive to such shifts.

All functional-equation analogues derived in this chapter therefore persist in the renormalized theory.

10.10 Summary

In this chapter we have:

- identified an inversion symmetry in the logarithmic variable,
- shown covariance of prime-shift operators under inversion,
- analyzed symmetry breaking by diagonal potentials,
- derived functional-equation analogues at the trace level,
- discussed implications for spectral statistics.

The next chapter examines conservation laws and operator identities arising from these symmetries and their arithmetic interpretation.

11 Bianchi-Type Identities and Arithmetic Conservation Laws

11.1 Conceptual Motivation

In differential geometry and gauge theory, Bianchi identities express consistency conditions imposed by symmetry and covariance. In the SMRK framework, analogous identities arise from algebraic relations among prime-shift operators and from symmetries of the Hamiltonian.

These identities encode arithmetic conservation laws that constrain the flow of probability and phase across the arithmetic state space.

11.2 Commutator Structure of Prime Shifts

Let p and q be distinct primes. The corresponding prime-shift operators satisfy

$$S_p S_q = S_q S_p, \quad T_p T_q = T_q T_p. \quad (11.93)$$

Mixed products satisfy

$$S_p T_q = T_q S_p \quad (11.94)$$

for $p \neq q$. These relations reflect the commutativity of prime multiplication and division for distinct primes.

11.3 Local Curvature Interpretation

The nontrivial structure arises from the relations involving the same prime. The operators S_p and T_p do not commute, and their commutator measures the local arithmetic curvature associated with the prime p .

Formally, one defines

$$\Omega_p := [S_p, T_p]. \quad (11.95)$$

This operator vanishes on arithmetic states supported away from p -divisible integers and encodes the obstruction to flat arithmetic transport.

11.4 Vanishing of Total Curvature

Although local curvature terms Ω_p are nonzero, their contributions cancel in the symmetric combination entering the SMRK Hamiltonian.

Specifically, summing the commutators weighted by $1/p$ yields

$$\sum_{p \in \mathbb{P}} \frac{1}{p} [S_p + T_p, S_p - T_p] = 0 \quad (11.96)$$

in quadratic-form sense.

This identity plays the role of a Bianchi identity: local curvature exists, but its total divergence vanishes.

11.5 Conservation of Arithmetic Probability

Let $\psi(t)$ denote a solution of the Schrödinger-type evolution equation

$$i \frac{d}{dt} \psi(t) = \tilde{H}_{\text{SMRK}} \psi(t). \quad (11.97)$$

Since $\tilde{H}_{\text{SMRK}}$ is self-adjoint, the norm $\|\psi(t)\|$ is conserved. This global conservation law has a refined arithmetic interpretation.

Probability flux between arithmetic states occurs only along prime-generated edges $n \leftrightarrow pn$. The symmetric structure of the Hamiltonian ensures that total inflow and outflow balance at every scale.

11.6 Local Balance Relations

At the level of individual primes, one may define local flux operators encoding the flow generated by S_p and T_p . The expectation value of their divergence vanishes in stationary states.

These local balance relations express the absence of sources and sinks in arithmetic space, apart from the confining diagonal potential.

11.7 Trace Identities as Conservation Laws

Taking traces of commutators with suitable test operators yields identities of the form

$$\text{Tr} [\tilde{H}_{\text{SMRK}}, A] = 0 \quad (11.98)$$

whenever the commutator is trace class.

Such identities impose constraints on admissible trace expansions and ensure consistency of the explicit formulae derived earlier.

11.8 Relation to Explicit Formula Stability

The Bianchi-type identities guarantee that prime-power coefficients appearing in explicit formulae are stable under deformations of the cutoff and under renormalization.

They ensure that cancellations required for convergence are structural rather than accidental.

11.9 Geometric Analogy

The arithmetic state space may be viewed as a discrete manifold equipped with a multiplicative connection generated by primes. In this picture, the SMRK Hamiltonian plays the role of a covariant Laplacian, and the identities derived above correspond to vanishing covariant divergences.

This analogy provides geometric intuition for arithmetic conservation laws without invoking an underlying continuum.

11.10 Summary

In this chapter we have:

- identified commutator relations among prime-shift operators,
- introduced a notion of arithmetic curvature,
- derived Bianchi-type cancellation identities,
- established arithmetic conservation laws,
- explained the stability of trace expansions.

The next chapter investigates numerical probes of spectral curvature and level statistics in the SMRK framework.

12 Numerical Probes of Spectral Curvature

12.1 Purpose of Numerical Analysis

The preceding chapters established a rigorous operator-theoretic framework for the SMRK Hamiltonian and derived analytic consequences for its spectrum and trace structures. Numerical investigations serve a complementary role: they test robustness, explore fine spectral features, and provide empirical access to spectral curvature and fluctuation phenomena.

The goal of this chapter is not numerical proof, but diagnostic validation of the structural predictions derived analytically.

12.2 Finite-Dimensional Truncations

Numerical analysis begins by restricting the arithmetic Hilbert space to integers $n \leq N$, where N is a finite cutoff. The truncated space is defined by

$$\mathcal{H}_N := \ell^2(\{1, \dots, N\}, 1/n). \quad (11.99)$$

The truncated Hamiltonian H_N is obtained by restricting prime-shift operators and diagonal potentials to this finite set. Prime shifts mapping outside the range are discarded.

This truncation preserves the local arithmetic structure while rendering the operator finite dimensional.

12.3 Renormalization at Finite Cutoff

At finite N , the prime sum remains divergent as the prime cutoff increases. Accordingly, a numerical renormalization constant C_N is introduced to ensure spectral stability.

The renormalized finite Hamiltonian takes the form

$$\tilde{H}_N := H_N - C_N I. \quad (11.100)$$

The constant C_N is chosen so that the lower edge of the spectrum remains approximately fixed as N varies.

12.4 Spectral Computation

The eigenvalues of $\tilde{H}_N$ are computed using standard numerical diagonalization techniques. Since the matrix representation is sparse and structured, iterative eigensolvers are effective even for moderately large N .

Eigenvalues are ordered as

$$\lambda_1^{(N)} \leq \lambda_2^{(N)} \leq \dots \leq \lambda_N^{(N)}. \quad (11.101)$$

Convergence of low-lying eigenvalues as N increases provides evidence for the validity of the truncation scheme.

12.5 Spectral Curvature Indicators

To probe spectral curvature, one studies second differences of eigenvalues:

$$\Delta^2 \lambda_k^{(N)} := \lambda_{k+1}^{(N)} - 2\lambda_k^{(N)} + \lambda_{k-1}^{(N)}. \quad (11.102)$$

Positive average curvature indicates confining behavior consistent with the logarithmic potential, while large fluctuations may signal arithmetic resonances.

12.6 Level Spacing Statistics

Define the unfolded level spacings by

$$s_k^{(N)} := \frac{\lambda_{k+1}^{(N)} - \lambda_k^{(N)}}{\langle \lambda_{k+1}^{(N)} - \lambda_k^{(N)} \rangle}, \quad (11.103)$$

where the denominator denotes a local spectral average.

Histograms of $s_k^{(N)}$ are compared against reference distributions. While the SMRK framework does not predict a specific random matrix ensemble, deviations from Poissonian statistics indicate nontrivial correlations induced by prime dynamics.

12.7 Prime Sensitivity Tests

To isolate arithmetic effects, numerical experiments selectively include or exclude prime contributions. For example, one may restrict prime shifts to primes $p \leq P$ and study the evolution of spectral features as P increases.

Such tests reveal the role of large primes in shaping spectral curvature and confirm that renormalization removes only global shifts, not fine structure.

12.8 Eigenfunction Localization

Numerical eigenfunctions $\psi_k^{(N)}(n)$ are analyzed in the logarithmic variable $x = \log n$. Measures of localization, such as inverse participation ratios, confirm that eigenfunctions remain localized as N grows.

This behavior is consistent with analytic predictions derived from logarithmic confinement.

12.9 Stability Under Parameter Variation

The SMRK Hamiltonian depends on parameters controlling diagonal terms. Numerical experiments vary these parameters within admissible ranges to test stability of spectral features.

Observed invariance of spacing statistics and curvature indicators supports the structural nature of the arithmetic dynamics.

12.10 Limitations of Numerical Probes

Numerical results are inherently limited by truncation size and computational precision. In particular, asymptotic spectral regimes lie beyond accessible ranges.

Nevertheless, consistent behavior across increasing cutoffs provides strong evidence that observed features are not artifacts of finite-dimensional approximation.

12.11 Summary

In this chapter we have:

- defined finite-dimensional truncations of the SMRK Hamiltonian,
- implemented numerical renormalization schemes,
- computed eigenvalues and eigenfunctions,
- probed spectral curvature and level statistics,
- tested sensitivity to prime contributions and parameters.

The numerical probes support the analytic framework and motivate the final comparison with classical number-theoretic structures developed in the next chapter.

13 Comparison with Classical Number Theory

13.1 Scope of the Comparison

The SMRK framework introduces an operator-theoretic setting in which arithmetic structure generates dynamics and spectral data. The purpose of this chapter is to compare this framework with classical constructions in analytic number theory and to clarify points of contact, agreement, and divergence.

The comparison is structural rather than identificatory: no direct equivalence between spectra and classical zero sets is asserted.

13.2 Explicit Formulae Revisited

Classical explicit formulae relate weighted sums over primes to weighted sums over nontrivial zeros of the Riemann zeta function. In the SMRK framework, analogous identities arise from trace expansions of a self-adjoint operator.

The correspondence may be summarized as follows:

- spectral sums replace sums over zeta zeros,
- trace functionals replace contour integrals of logarithmic derivatives,
- prime-power weights arise from closed arithmetic paths.

In both settings, test functions mediate the relation between arithmetic and spectral data.

13.3 Role of the von Mangoldt Function

In classical theory, the von Mangoldt function appears as the coefficient governing prime powers in explicit formulae. Within the SMRK Hamiltonian, the same function enters as a diagonal potential generating logarithmic arithmetic forces.

This dual role clarifies the origin of logarithmic prime weights in trace expansions and provides a dynamical interpretation of the von Mangoldt function.

13.4 Dirichlet Series versus Operator Traces

Classical zeta functions are defined as Dirichlet series and extended by analytic continuation. In contrast, the SMRK framework begins with a self-adjoint operator and defines trace objects directly.

While both approaches yield meromorphic functions encoding arithmetic data, their foundations differ fundamentally:

- Dirichlet series rely on analytic continuation,
- operator traces rely on spectral theory and compactness.

This distinction allows the SMRK framework to avoid assumptions about analytic continuation while retaining access to explicit-formula-type identities.

13.5 Functional Equation versus Operator Symmetry

The functional equation of the Riemann zeta function encodes a symmetry exchanging s and $1 - s$. In the SMRK framework, analogous symmetry arises from inversion in the logarithmic variable and covariance of prime-shift operators.

Both mechanisms impose duality constraints on arithmetic–spectral relations. The difference lies in implementation: analytic symmetry versus operator-theoretic symmetry.

13.6 Zero Sets and Spectral Data

Classical number theory focuses on the location of zeros of zeta and L -functions. The SMRK framework produces a discrete spectrum with well-defined eigenvalues and eigenfunctions.

No claim is made that these eigenvalues coincide with classical zeros. Instead, the comparison highlights a shared organizing principle: arithmetic information is encoded in spectral distributions subject to symmetry and trace constraints.

13.7 Trace Formulae and Weil-Type Analogues

Weil's explicit formula may be interpreted as a trace formula on an adelic space. The SMRK framework realizes a trace formula on a purely arithmetic Hilbert space indexed by the natural numbers.

Both settings involve:

- a global trace identity,
- local contributions indexed by primes,
- symmetry-induced cancellations.

The SMRK construction may thus be viewed as a non-adelic, operator-based analogue of Weil-type trace formulae.

13.8 Spectral Statistics and Random Matrix Theory

Classical conjectures relate zero statistics of zeta functions to random matrix ensembles. The SMRK framework does not impose a specific ensemble but predicts structural constraints on spectral statistics derived from arithmetic symmetry.

Numerical probes suggest nontrivial correlations distinct from purely random spectra, highlighting the influence of prime-generated dynamics.

13.9 Limitations of the Comparison

The comparison with classical number theory has clear limits:

- no analytic continuation of zeta functions is derived,
- no identification of SMRK eigenvalues with classical zeros is established,
- no proof of classical conjectures is claimed.

The SMRK framework should therefore be regarded as complementary rather than substitutive.

13.10 Conceptual Synthesis

Despite foundational differences, both classical number theory and the SMRK framework share a common philosophy: arithmetic structure manifests through spectral and trace phenomena governed by symmetry.

The SMRK Hamiltonian provides a concrete realization of this philosophy within a self-adjoint operator setting.

13.11 Summary

In this chapter we have:

- compared SMRK trace identities with classical explicit formulae,

- analyzed the role of the von Mangoldt function,
- contrasted operator traces with Dirichlet series,
- examined symmetry principles and functional equation analogues,
- clarified the scope and limitations of the comparison.

This concludes the main body of the work. The appendices collect technical details, auxiliary constructions, and numerical protocols supporting the framework developed here.

APPENDIX A

A.1 Technical Estimates and Form Bounds

A.1.1 Purpose of the Appendix

This appendix collects technical estimates used in the construction and analysis of the SMRK Hamiltonian. The focus is on bounds required for quadratic-form convergence, renormalization, and spectral stability.

All estimates are stated at the level of quadratic forms and are independent of operator-domain subtleties.

A.1.2 Elementary Logarithmic Inequalities

For all positive real numbers a and b , the logarithmic function satisfies

$$(\log(ab))^2 \leq 2(\log a)^2 + 2(\log b)^2. \quad (11.1)$$

Applied to integers m and primes p , this yields

$$\log^2(m/p) \leq 2\log^2 m + 2\log^2 p. \quad (11.2)$$

These inequalities underlie all confinement estimates for prime-shift operators.

A.1.3 Bounds for Prime Shift Operators

Let ψ be a finitely supported arithmetic state. Then the following estimate holds:

$$q_0[S_p\psi] \leq 2p q_0[\psi] + 2p \log^2 p \|\psi\|^2. \quad (11.3)$$

An analogous bound holds for the co-shift operator T_p .

These inequalities imply that prime shifts are controlled by the reference quadratic form up to logarithmic corrections.

A.1.4 Relative Form Boundedness

For any $\varepsilon > 0$, there exists a constant $K_\varepsilon > 0$ such that

$$\frac{1}{p} |\langle \psi, S_p \psi \rangle| \leq \varepsilon q_0[\psi] + K_\varepsilon \frac{\log^2 p}{p} \|\psi\|^2 \quad (11.4)$$

for all finitely supported ψ .

Summation over primes yields uniform form-boundedness of the interaction term with relative bound zero.

A.1.5 Control of the Renormalization Constant

The renormalization constant

$$C(P) = \sum_{p \leq P} p^{-1/2} \quad (11.5)$$

diverges as $P \rightarrow \infty$, but grows slower than any positive power of P .

Its subtraction affects only the scalar part of the quadratic form and does not influence spectral gaps or trace differences.

A.1.6 Compactness Criteria

The reference quadratic form

$$q_0[\psi] = \sum_{n \geq 1} (1 + \log^2 n) |\psi(n)|^2 \frac{1}{n} \quad (11.6)$$

induces a compact embedding of the form domain into the arithmetic Hilbert space.

This property ensures compactness of the resolvent and discreteness of the spectrum.

A.1.7 Summary

This appendix has collected the key technical estimates required for:

- logarithmic confinement,
- control of prime-shift operators,
- uniform form-boundedness,
- validity of renormalization,
- compactness of the resolvent.

APPENDIX B

B.1 Mellin Space and Weighted Trace Constructions

B.1.1 Motivation

Weighted trace constructions play a central role in connecting spectral data of the SMRK Hamiltonian with arithmetic structures. This appendix summarizes the Mellin-space interpretation underlying these constructions.

B.1.2 Mellin Transform on Arithmetic States

Let ψ be an arithmetic state. Its Mellin transform is formally defined by

$$\mathcal{M}\psi(s) := \sum_{n \geq 1} \psi(n) n^{-s}, \quad (11.1)$$

for complex values of s where the sum converges.

In the SMRK framework, Mellin weights arise naturally from diagonal operators of the form n^{-s} .

B.1.3 Weighted Operators

For real $s > 0$, define the diagonal weight operator by

$$(W_s \psi)(n) := n^{-s} \psi(n). \quad (11.2)$$

The operator W_s is Hilbert–Schmidt for $s > 1/2$ and trace class for $s > 1$.

B.1.4 Weighted Resolvent Traces

For nonreal z , the weighted resolvent trace is defined by

$$R(z; s) := \text{Tr}(W_s(\tilde{H}_{\text{SMRK}} - z)^{-1} W_s). \quad (11.3)$$

This trace admits both a spectral expansion and an arithmetic expansion. The spectral side involves weighted eigenfunction norms, while the arithmetic side produces prime-power sums.

B.1.5 Analytic Dependence on Parameters

The function $R(z; s)$ is analytic in z away from the real axis and analytic in s for $\text{Re } s > 1/2$.

This joint analyticity allows contour deformation and Mellin inversion techniques analogous to those used in classical analytic number theory.

B.1.6 Connection with Explicit Formulae

Weighted traces provide the natural test-function framework for explicit-formula-type identities. The Mellin parameter s controls arithmetic weights, while the spectral parameter z controls smoothing in energy space.

Their interplay replaces the role of complex variable shifts in classical zeta functions.

B.1.7 Summary

In this appendix we have:

- introduced Mellin transforms on arithmetic states,
- defined diagonal weight operators,
- constructed weighted resolvent traces,
- explained their analytic properties,
- clarified their role in explicit formulae.

APPENDIX C

C.1 Numerical Protocol and Pseudocode

C.1.1 Purpose of the Appendix

This appendix provides a concrete numerical protocol for implementing finite-dimensional approximations of the SMRK Hamiltonian and for probing its spectral properties. The protocol is designed to be reproducible, modular, and independent of any specific programming language.

All steps are stated at the algorithmic level and are intended to support the numerical experiments discussed in Chapter 12.

C.1.2 Finite Arithmetic Truncation

Fix a truncation parameter $N \geq 1$. The truncated arithmetic state space is

$$\mathcal{H}_N = \ell^2(\{1, \dots, N\}, 1/n). \quad (11.1)$$

Basis vectors are denoted by δ_n for $1 \leq n \leq N$ and are orthonormal with respect to the weighted inner product.

Prime-shift operators are truncated by discarding transitions that map outside $\{1, \dots, N\}$.

C.1.3 Construction of Truncated Operators

For each prime $p \leq N$, define truncated prime shifts by

$$(S_p^{(N)}\psi)(n) = \begin{cases} \psi(pn), & pn \leq N, \\ 0, & pn > N, \end{cases} \quad (11.2)$$

$$(T_p^{(N)}\psi)(n) = \begin{cases} \psi(n/p), & p \mid n, \\ 0, & \text{otherwise.} \end{cases} \quad (11.3)$$

Diagonal terms are defined by

$$(V^{(N)}\psi)(n) = (\alpha\Lambda(n) + \beta \log n)\psi(n), \quad 1 \leq n \leq N. \quad (11.4)$$

C.1.4 Truncated Hamiltonian

The truncated SMRK Hamiltonian is defined by

$$H_N = \sum_{p \leq N} \frac{1}{p} (S_p^{(N)} + T_p^{(N)}) + V^{(N)}. \quad (11.5)$$

This operator is represented numerically as an $N \times N$ sparse matrix with structured nonzero entries determined by prime factorizations.

C.1.5 Numerical Renormalization

To stabilize the spectrum, introduce a numerical renormalization constant C_N . A practical choice is

$$C_N = \sum_{p \leq N} p^{-1/2}, \quad (11.6)$$

or an empirical constant chosen so that the lowest eigenvalue of $H_N - C_N I$ remains approximately fixed as N varies.

Define the renormalized operator by

$$\tilde{H}_N := H_N - C_N I. \quad (11.7)$$

C.1.6 Spectral Computation Protocol

The following steps summarize the numerical workflow:

1. Generate the list of primes $p \leq N$.
2. Assemble sparse matrices for $S_p^{(N)}$ and $T_p^{(N)}$.
3. Construct the diagonal potential $V^{(N)}$.
4. Form the truncated Hamiltonian H_N .
5. Compute the renormalization constant C_N .
6. Diagonalize $\tilde{H}_N$ using an iterative eigensolver.
7. Extract low-lying eigenvalues and eigenfunctions.

C.1.7 Pseudocode

The following pseudocode summarizes the construction:

```

input: truncation N, parameters alpha, beta

primes = list_primes_up_to(N)
initialize H as zero N x N matrix

for p in primes:
    for n in 1..N:
        if p*n <= N:
            H[n, p*n] += 1/p
        if n % p == 0:
            H[n, n/p] += 1/p

for n in 1..N:
    H[n, n] += alpha * Lambda(n) + beta * log(n)

C_N = sum_{p <= N} p^(-1/2)
H = H - C_N * identity_matrix

eigenvalues, eigenvectors = diagonalize(H)
output eigenvalues, eigenvectors

```

C.1.8 Post-Processing and Diagnostics

After diagonalization, the following diagnostics are computed:

- convergence of low-lying eigenvalues as N increases,
- level spacings and unfolded spacing statistics,
- second differences as indicators of spectral curvature,
- localization measures for eigenfunctions in $\log n$.

These diagnostics provide numerical evidence for logarithmic confinement and arithmetic-induced spectral correlations.

C.1.9 Reproducibility Considerations

Numerical results depend on truncation size, prime cutoff, and renormalization choice. Reproducibility requires:

- explicit reporting of N and parameter values,
- consistent renormalization conventions,
- verification of stability under increasing cutoffs.

C.1.10 Summary

This appendix has:

- specified a finite-dimensional truncation scheme,
- defined truncated prime-shift operators,
- introduced numerical renormalization,
- provided implementation-ready pseudocode,
- outlined diagnostic procedures for spectral analysis.

Chapter 12

The SMRK Hamiltonian for General L -Functions

Spectral Arithmetic Dynamics Beyond the Riemann Zeta Function

Enter Yourname

Abstract

This section extends the SMRK Hamiltonian framework beyond the classical setting of the Riemann zeta function to encompass more general L -functions. The objective is to investigate whether the operator-theoretic principles developed in the preceding sections admit natural generalizations that reflect broader arithmetic contexts, including Dirichlet L -functions and related families.

The extension is formulated at the level of operator construction rather than as a direct translation of known analytic results. Instead of modifying explicit formulae by hand, the approach seeks to adapt the underlying operator fields, ladder structures, and arithmetic weights so that the resulting Hamiltonians encode the additional symmetries and arithmetic data associated with general L -functions. This preserves the conceptual integrity of the SMRK framework while allowing for greater expressive power.

A central challenge addressed in this section is the incorporation of characters, conductors, and arithmetic twists into the operator dynamics. These features introduce new layers of structure that must be reflected both in the definition of the arithmetic state space and in the interaction rules of the ladder operators. The section discusses how such data can be embedded into the operator formalism without sacrificing analytic control or spectral interpretability.

The implications of this generalization are explored at the level of traces, explicit formulae, and spectral programs. While the existence of a universal operator capturing all L -functions is not assumed, the framework demonstrates how families of related operators may be studied within a common mathematical language. This perspective emphasizes structural relationships rather than isolated cases, aligning with the operator-field philosophy underlying QFM.

The section also highlights limitations and open problems. Not all features of general L -functions admit immediate operator-theoretic interpretation, and the extension of domain and self-adjointness results remains subtle in more complex arithmetic settings. These challenges are presented

as intrinsic aspects of the theory, guiding future research rather than detracting from the coherence of the framework.

By situating general L -functions within the SMRK operator program, this section demonstrates the scalability of the approach and delineates its current boundaries. It marks the conceptual completion of the mathematical foundation developed in Part I and prepares the transition to computational, architectural, and applied considerations in subsequent parts of this compendium.

1 From the Riemann Zeta Function to General L -Functions

1.1 Motivation

The SMRK Hamiltonian was initially formulated in direct correspondence with the arithmetic structure underlying the Riemann zeta function. While this setting already captures essential features of prime dynamics, it represents only the simplest member of a much broader family of arithmetic objects.

The natural next step is to extend the construction from the Riemann zeta function to general L -functions. This extension is not an afterthought but a structural necessity: the operator-theoretic framework should be flexible enough to accommodate local data, internal symmetries, and representation-theoretic input.

1.2 What Changes Beyond the Zeta Function

The Riemann zeta function corresponds to the trivial representation at every prime. General L -functions introduce additional local structure:

- local Euler factors with multiple parameters,
- internal degrees of freedom indexed by representation dimension,
- nontrivial local symmetries and phases,
- arithmetic weights beyond simple prime powers.

From the operator perspective, this means that arithmetic states can no longer be described solely by integers. Additional internal labels must be incorporated.

1.3 Arithmetic States with Internal Structure

To accommodate general L -functions, arithmetic states are extended from scalar functions $\psi(n)$ to vector-valued objects

$$\psi(n) \in \mathbb{C}^{d(n)}, \quad (12.1)$$

where the internal dimension $d(n)$ depends on the local data attached to n .

In the simplest cases, $d(n)$ is constant and equal to the degree of the L -function. More generally, it reflects multiplicative composition of local representations.

1.4 Prime Locality as a Guiding Principle

A defining feature of L -functions is locality at primes. Each prime contributes an independent local factor to the Euler product, and global structure emerges from their multiplicative combination.

The SMRK framework respects this principle by associating to each prime p a local operator acting on the internal degrees of freedom. Global operators are obtained by assembling these local actions across all primes.

1.5 Euler Products as Operator Factorizations

Classically, an L -function is defined by an Euler product of the form

$$L(s) = \prod_p L_p(s), \quad (12.2)$$

where each $L_p(s)$ encodes local arithmetic data.

In the operator framework, this structure is mirrored by factorization of the Hamiltonian interaction into prime-local contributions. Each prime-shift operator is decorated with a finite-dimensional matrix encoding the corresponding local Euler factor.

1.6 From Scalars to Matrix-Valued Prime Shifts

For the Riemann zeta function, prime shifts act as scalar operators. For general L -functions, each prime p is associated with a matrix

$$A_p \in \text{Mat}_{d \times d}(\mathbb{C}), \quad (12.3)$$

encoding local arithmetic data.

Prime multiplication and division are accompanied by multiplication with A_p or its adjoint, producing a coupled arithmetic–internal dynamics.

1.7 Preservation of the SMRK Core Principles

Despite the added complexity, the core principles of the SMRK framework remain intact:

- arithmetic dynamics is generated by primes,
- interactions are local in the prime index,
- the Hamiltonian is constructed from symmetric prime-shift combinations,
- renormalization removes only scalar divergences.

The generalization to L -functions therefore represents an enrichment rather than a conceptual departure.

1.8 Why the Extension Is Nontrivial

Extending the SMRK Hamiltonian to general L -functions introduces genuine new challenges:

- internal spaces must be consistently assembled over composite integers,
- prime-shift operators become matrix-valued,
- self-adjointness requires compatibility conditions on local data,
- trace expansions involve generalized von Mangoldt weights.

These issues require careful formulation and are addressed systematically in the chapters that follow.

1.9 Organization of This Document

The remainder of this document proceeds as follows. We first formalize Euler products and local arithmetic data. We then construct arithmetic Hilbert spaces with internal degrees of freedom and define prime-shift operators carrying local representations.

Subsequent chapters establish renormalization, self-adjointness, spectral properties, and trace expansions. Special cases, including Dirichlet and automorphic L -functions, are treated explicitly.

1.10 Summary

This chapter has:

- motivated the extension of the SMRK Hamiltonian beyond the zeta function,
- identified the new arithmetic and operator-theoretic ingredients,
- outlined the role of local prime data and internal degrees of freedom,
- set the stage for a unified operator framework for general L -functions.

The next chapter introduces Euler products and local parameter data in a form suitable for operator-theoretic implementation.

2 Euler Products and Local Arithmetic Data

2.1 Local–Global Decomposition

A defining feature of L -functions is their decomposition into independent local factors indexed by primes. This decomposition expresses a global arithmetic object as a product of prime-local contributions, each encoding finite-dimensional data.

From the operator-theoretic perspective, this locality principle dictates that the global Hamiltonian must be assembled from prime-indexed building blocks, with no direct coupling between distinct primes.

2.2 General Euler Factors

Let $L(s)$ be an L -function of degree d . Its Euler product is written formally as

$$L(s) = \prod_p L_p(s), \quad (12.4)$$

where each local factor takes the form

$$L_p(s) = \det(I - A_p p^{-s})^{-1}. \quad (12.5)$$

Here $A_p \in \text{Mat}_{d \times d}(\mathbb{C})$ is a matrix encoding the local arithmetic data at the prime p .

2.3 Local Parameters and Eigenvalues

The matrix A_p is assumed to be diagonalizable over $\mathbb{C}$ with eigenvalues

$$\{\alpha_{p,1}, \dots, \alpha_{p,d}\}. \quad (12.6)$$

In these terms, the local factor admits the expansion

$$L_p(s) = \prod_{j=1}^d (1 - \alpha_{p,j} p^{-s})^{-1}. \quad (12.7)$$

The parameters $\alpha_{p,j}$ play the role of local spectral data and govern the contribution of prime powers in explicit formulae.

2.4 Ramified and Unramified Primes

For most primes, the local data are unramified and satisfy uniform bounds on $\alpha_{p,j}$. A finite set of primes may be ramified, at which the local factor deviates from the generic form.

In the SMRK framework, ramified primes are treated by modifying the corresponding local operators without altering the global structure. Since only finitely many primes are affected, this does not influence renormalization or asymptotic spectral properties.

2.5 Multiplicativity and Composite Integers

Local data at primes determine arithmetic data at composite integers via multiplicativity. For an integer

$$n = \prod_p p^{\nu_p}, \quad (12.8)$$

the associated local space is constructed as a tensor product over primes dividing n .

This multiplicative assembly ensures consistency between prime-local operators and global arithmetic states.

2.6 Generalized von Mangoldt Functions

Associated with the local parameters $\alpha_{p,j}$ is a generalized von Mangoldt function defined by

$$\Lambda_L(p^m) := \sum_{j=1}^d \alpha_{p,j}^m \log p, \quad (12.9)$$

and extended by zero to integers that are not prime powers.

These weights replace the classical von Mangoldt function in trace expansions and explicit formulae.

2.7 Local Admissibility Conditions

For the operator construction to be well defined, the local data must satisfy basic admissibility conditions:

- boundedness of $\alpha_{p,j}$ uniformly in p ,
- compatibility with adjoint operations,
- finite-dimensionality of local spaces.

These conditions ensure that prime-shift operators decorated with local matrices are uniformly form-bounded and admit a common renormalization scheme.

2.8 Operator Interpretation of Euler Factors

In the SMRK framework, Euler factors are not treated as analytic objects but as operator-theoretic data. Each factor $L_p(s)$ corresponds to a local interaction term acting on internal degrees of freedom and coupled to arithmetic prime shifts.

The determinant structure of $L_p(s)$ is reflected in traces over the internal spaces associated with prime-power loops.

2.9 Universality of the Local Data Formalism

The formalism introduced here applies uniformly to:

- Dirichlet L -functions,
- Artin L -functions,
- automorphic L -functions,
- finite products and twists thereof.

Only the specification of the local matrices A_p changes; the global operator architecture remains fixed.

2.10 Summary

In this chapter we have:

- introduced general Euler products in operator-ready form,
- described local arithmetic parameters and eigenvalues,
- defined generalized von Mangoldt functions,
- explained multiplicative assembly over composite integers,
- established admissibility conditions for local data.

The next chapter constructs arithmetic Hilbert spaces carrying internal degrees of freedom compatible with this local–global structure.

3 Arithmetic Hilbert Spaces with Internal Degrees of Freedom

3.1 Motivation

To accommodate general L -functions, arithmetic states must carry internal degrees of freedom reflecting local representation data at primes. This chapter constructs arithmetic Hilbert spaces that combine multiplicative structure in the integer variable with finite-dimensional internal spaces.

The construction preserves the core SMRK principle: arithmetic dynamics is driven by prime shifts, while internal structure evolves locally at each prime.

3.2 Local Internal Spaces

Fix an L -function of degree d . To each prime p we associate a finite-dimensional complex vector space

$$\mathcal{V}_p \cong \mathbb{C}^d, \quad (12.10)$$

carrying the local arithmetic data encoded by the matrix A_p .

At unramified primes, these spaces are canonically identified. At ramified primes, $\mathcal{V}_p$ may differ, but only finitely many such primes occur.

3.3 Internal Spaces for Composite Integers

For a positive integer

$$n = \prod_p p^{\nu_p}, \quad (12.11)$$

we define the associated internal space by the tensor product

$$\mathcal{V}_n := \bigotimes_{p|n} \mathcal{V}_p^{\otimes \nu_p}. \quad (12.12)$$

This definition reflects the multiplicative assembly of local data and ensures compatibility with Euler products.

3.4 Arithmetic States

An arithmetic state is a collection

$$\psi = \{\psi(n)\}_{n \geq 1}, \quad \psi(n) \in \mathcal{V}_n, \quad (12.13)$$

with finite norm defined below.

Such states may be viewed as vector-valued arithmetic functions equipped with a prime-factorization-dependent internal structure.

3.5 Inner Product and Norm

The arithmetic Hilbert space $\mathcal{H}_L$ is defined by the inner product

$$\langle \psi, \phi \rangle := \sum_{n \geq 1} \frac{1}{n} \langle \psi(n), \phi(n) \rangle_{\mathcal{V}_n}, \quad (12.14)$$

where the inner product on $\mathcal{V}_n$ is induced from the tensor product of the local spaces.

The weight $1/n$ ensures compatibility with multiplicative shifts and guarantees that prime-shift operators admit simple adjoint relations.

3.6 Dense Subspace and Core

The set of arithmetic states with finite support in n and finite-dimensional support in internal indices forms a dense subspace of $\mathcal{H}_L$.

This subspace serves as a common invariant core for all operators introduced in subsequent chapters.

3.7 Canonical Identification of Tensor Factors

For integers n and m with coprime factorizations, the internal spaces satisfy

$$\mathcal{V}_{nm} \cong \mathcal{V}_n \otimes \mathcal{V}_m. \quad (12.15)$$

This canonical identification is essential for defining prime-shift operators that act consistently across composite integers.

3.8 Normalization Conventions

The choice of inner product on each $\mathcal{V}_p$ is fixed so that the local matrices A_p act as bounded operators with uniformly controlled norms.

This normalization ensures that global operators constructed from prime-local data are uniformly form-bounded.

3.9 Comparison with the Scalar Case

When $d = 1$ and $A_p = 1$ for all primes, the internal spaces $\mathcal{V}_n$ reduce to $\mathbb{C}$ for all n . In this case, $\mathcal{H}_L$ coincides with the arithmetic Hilbert space used for the Riemann zeta function.

Thus the present construction is a strict generalization of the scalar SMRK setting.

3.10 Structural Properties

The arithmetic Hilbert space with internal degrees of freedom satisfies:

- separability,
- multiplicative tensor-factor structure,
- compatibility with prime shifts,
- existence of a natural dense core.

These properties form the foundation for defining Hamiltonians and trace objects in the generalized setting.

3.11 Summary

In this chapter we have:

- constructed arithmetic Hilbert spaces with internal structure,
- defined arithmetic states as vector-valued functions,
- established the weighted inner product,
- ensured multiplicative compatibility across primes,
- related the construction to the scalar SMRK case.

The next chapter defines prime-shift operators decorated with local representations acting on these spaces.

4 Prime-Shift Operators with Local Representations

4.1 Purpose of Prime-Shift Operators

Prime-shift operators generate arithmetic dynamics by implementing multiplication and division by primes. In the generalized L -function setting, these shifts must be decorated with local representation data acting on internal degrees of freedom.

This chapter defines such operators and establishes their fundamental algebraic and adjoint properties.

4.2 Multiplicative Prime Shifts

Let p be a prime. The multiplicative prime-shift operator S_p acts on arithmetic states by

$$(S_p \psi)(n) := \begin{cases} A_p \psi(n/p), & p \mid n, \\ 0, & \text{otherwise.} \end{cases} \quad (12.16)$$

Here A_p acts on the internal space $\mathcal{V}_{n/p}$ and is transported to $\mathcal{V}_n$ via the canonical tensor identification.

4.3 Divisive Prime Shifts

The divisive prime-shift operator T_p acts by

$$(T_p \psi)(n) := \psi(pn), \quad (12.17)$$

where the internal space $\mathcal{V}_{pn}$ is identified with $\mathcal{V}_p \otimes \mathcal{V}_n$.

In this action, no matrix multiplication is applied; the internal structure is extended by tensoring with the identity on $\mathcal{V}_p$.

4.4 Adjoint Relations

With respect to the weighted inner product on $\mathcal{H}_L$, the operators S_p and T_p satisfy

$$S_p^* = \frac{1}{p} T_p^{(A)}, \quad (12.18)$$

where $T_p^{(A)}$ denotes the divisive shift followed by application of A_p^* on the appropriate internal factor.

This relation generalizes the scalar adjoint identity and ensures symmetric construction of the Hamiltonian.

4.5 Boundedness Properties

Under the admissibility assumptions on the local matrices A_p , the operators S_p and T_p are bounded on the dense core of finitely supported arithmetic states.

Moreover, their operator norms grow at most polynomially in p , which is sufficient for form-boundedness after weighting by $1/p$.

4.6 Commutation Relations

For distinct primes p and q , the prime-shift operators commute:

$$S_p S_q = S_q S_p, \quad T_p T_q = T_q T_p, \quad S_p T_q = T_q S_p. \quad (12.19)$$

These relations reflect the independence of prime factors and ensure consistency of multiplicative assembly.

4.7 Local Noncommutativity

For a fixed prime p , the operators S_p and T_p do not commute. Their noncommutativity encodes local arithmetic curvature associated with the prime p and is controlled by the local matrix A_p . This local structure plays a key role in trace expansions and curvature identities.

4.8 Ramified Primes

At ramified primes, the definition of A_p and the identification of internal spaces may differ from the unramified case. Since only finitely many primes are ramified, these modifications introduce only finite-rank perturbations.

Such perturbations do not affect renormalization, self-adjointness, or asymptotic spectral properties.

4.9 Consistency with Multiplicativity

The definitions of S_p and T_p are compatible with multiplicativity in the sense that their combined action on composite integers respects the tensor-product structure of internal spaces.

This consistency ensures that global arithmetic dynamics is entirely determined by prime-local data.

4.10 Reduction to the Scalar Case

When all local matrices A_p are equal to the identity, the decorated prime-shift operators reduce to the scalar shifts used in the zeta-function case.

Thus the present construction is a strict generalization of the original SMRK prime-shift operators.

4.11 Summary

In this chapter we have:

- defined prime-shift operators carrying local representation data,
- established their adjoint and boundedness properties,
- analyzed their commutation relations,
- treated the role of ramified primes,
- connected the construction to the scalar SMRK case.

The next chapter defines the generalized SMRK Hamiltonian for L -functions using these operators.

5 Definition of the SMRK Hamiltonian for General L -Functions

5.1 Guiding Principles

The construction of the SMRK Hamiltonian for general L -functions follows the same principles as in the scalar case:

- arithmetic dynamics is generated by prime shifts,
- interactions are local in the prime index,
- symmetry between multiplication and division by primes is enforced,
- divergences arise only from infinite prime sums and are removable by renormalization.

The presence of internal degrees of freedom modifies the operator structure but not the underlying philosophy.

5.2 Formal Definition

The formal SMRK Hamiltonian associated with an L -function is defined by

$$H_L := \sum_{p \in \mathbb{P}} \frac{1}{p} (S_p + S_p^*) + V_L, \quad (12.20)$$

where S_p denotes the decorated prime-shift operator defined previously, and V_L is a diagonal potential acting on arithmetic states.

5.3 Diagonal Potential

The diagonal potential V_L encodes generalized von Mangoldt weights and logarithmic confinement:

$$(V_L \psi)(n) := (\alpha \Lambda_L(n) + \beta \log n) \psi(n). \quad (12.21)$$

Here Λ_L denotes the generalized von Mangoldt function associated with the local parameters $\alpha_{p,j}$, and α, β are real constants controlling the relative strength of arithmetic forcing and confinement.

5.4 Symmetric Prime Interaction

The combination $S_p + S_p^*$ ensures that the prime interaction is symmetric and formally self-adjoint. This symmetry is essential for constructing a self-adjoint realization of the Hamiltonian.

In contrast, unsymmetrized prime shifts would generate irreversible arithmetic flows and are therefore excluded.

5.5 Domain of Definition

The operator H_L is initially defined on the dense core of finitely supported arithmetic states with finite internal support.

On this domain, all sums over primes are finite and the action of H_L is well defined.

5.6 Divergence of the Prime Sum

As in the scalar case, the sum over primes in the definition of H_L diverges when taken over all primes. This divergence arises from the accumulation of infinitely many weak prime interactions and is independent of the internal representation data.

The divergence is scalar in form sense and therefore removable by renormalization.

5.7 Quadratic Form Associated with the Hamiltonian

To prepare for renormalization, we introduce the quadratic form

$$q_L(\psi, \phi) := \sum_{p \in \mathbb{P}} \frac{1}{p} (\langle S_p \psi, \phi \rangle + \langle S_p^* \psi, \phi \rangle) + \langle V_L \psi, \phi \rangle, \quad (12.22)$$

initially defined on the dense core.

This form captures the essential features of the formal Hamiltonian while providing a suitable framework for analytic control.

5.8 Hermiticity

Using the adjoint relations established previously, one verifies that

$$q_L(\psi, \phi) = q_L(\phi, \psi) \quad (12.23)$$

for all ψ and ϕ in the domain.

Thus the quadratic form is Hermitian, even though the formal operator sum is divergent.

5.9 Reduction to the Scalar Case

When the degree $d = 1$ and all local matrices A_p are trivial, the Hamiltonian H_L reduces exactly to the original SMRK Hamiltonian associated with the Riemann zeta function.

This confirms that the generalized construction is a true extension of the scalar framework.

5.10 Interpretation

The SMRK Hamiltonian for general L -functions describes arithmetic dynamics on a state space enriched by internal representation data. Prime shifts transport both arithmetic position and internal state, while the diagonal potential imposes logarithmic confinement and arithmetic forcing.

This operator encodes, in a single object, the global arithmetic content of the underlying L -function.

5.11 Summary

In this chapter we have:

- defined the formal SMRK Hamiltonian for general L -functions,
- introduced the diagonal potential with generalized von Mangoldt weights,
- analyzed symmetry and divergence of the prime interaction,

- formulated the associated quadratic form,
- related the construction to the scalar case.

The next chapter performs renormalization and constructs a self-adjoint realization of the Hamiltonian.

6 Renormalization and Self-Adjointness

6.1 Need for Renormalization

The formal SMRK Hamiltonian for general L -functions contains an infinite sum over prime-indexed interactions. As in the scalar case, this sum diverges in operator norm and cannot be interpreted directly as a self-adjoint operator.

The divergence originates from the accumulation of infinitely many weak prime interactions and is independent of the internal representation data. Consequently, renormalization is unavoidable.

6.2 Prime Cutoff and Truncated Hamiltonians

Let $P \geq 2$ be a prime cutoff. We define the truncated Hamiltonian by

$$H_{L,P} := \sum_{\substack{p \in \mathbb{P} \\ p \leq P}} \frac{1}{p} (S_p + S_p^*) + V_L, \quad (12.24)$$

initially defined on the dense core of finitely supported arithmetic states.

For each fixed P , the operator $H_{L,P}$ is symmetric and well defined.

6.3 Associated Quadratic Forms

Rather than working directly with operators, we consider the quadratic forms

$$q_{L,P}(\psi, \phi) := \sum_{\substack{p \in \mathbb{P} \\ p \leq P}} \frac{1}{p} (\langle S_p \psi, \phi \rangle + \langle S_p^* \psi, \phi \rangle) + \langle V_L \psi, \phi \rangle. \quad (12.25)$$

Each form $q_{L,P}$ is densely defined and Hermitian on the core.

6.4 Divergent Scalar Contribution

Estimates analogous to those in the scalar case show that the prime-shift contribution to $q_{L,P}[\psi]$ grows proportionally to

$$\sum_{p \leq P} \frac{1}{\sqrt{p}} \|\psi\|^2. \quad (12.26)$$

This divergence is scalar in form sense and does not depend on the internal degrees of freedom.

6.5 Renormalized Quadratic Forms

We define the renormalized quadratic form by

$$\tilde{q}_{L,P}(\psi, \phi) := q_{L,P}(\psi, \phi) - C(P) \langle \psi, \phi \rangle, \quad (12.27)$$

where the renormalization constant $C(P)$ is chosen to cancel the divergent scalar contribution. The precise choice of $C(P)$ is not unique; different choices differ by finite constants and lead to unitarily equivalent operators.

6.6 Uniform Form Bounds

There exists a reference quadratic form q_0 , incorporating logarithmic confinement, such that for every $\varepsilon > 0$ there exists $K_\varepsilon > 0$ satisfying

$$|\tilde{q}_{L,P}(\psi, \psi)| \leq \varepsilon q_0[\psi] + K_\varepsilon \|\psi\|^2 \quad (12.28)$$

uniformly in P .

This establishes that the interaction term is relatively form-bounded with relative bound zero.

6.7 Existence of the Limit Form

The uniform bounds imply that the family $\{\tilde{q}_{L,P}\}_P$ is Cauchy in the sense of quadratic forms. Consequently, there exists a limiting quadratic form

$$\tilde{q}_L := \lim_{P \rightarrow \infty} \tilde{q}_{L,P} \quad (12.29)$$

defined on the same form domain as q_0 .

The limiting form is closed, densely defined, and bounded from below.

6.8 Self-Adjoint Hamiltonian

By the representation theorem for closed quadratic forms, there exists a unique self-adjoint operator $\tilde{H}_L$ such that

$$\tilde{q}_L(\psi, \phi) = \langle \tilde{H}_L \psi, \phi \rangle \quad (12.30)$$

for all ψ in the operator domain and all ϕ in the form domain.

The operator $\tilde{H}_L$ is bounded from below and generates a strongly continuous unitary time evolution.

6.9 Identification with the Formal Hamiltonian

On the dense core of finitely supported arithmetic states, the action of $\tilde{H}_L$ coincides with that of the formal Hamiltonian H_L up to an additive constant arising from renormalization.

This constant produces a uniform spectral shift and has no effect on spectral spacings or trace identities.

6.10 Stability under Local Data Variation

Since renormalization removes only a scalar divergence, the construction of $\tilde{H}_L$ is stable under variation of the local matrices A_p , provided they satisfy the admissibility conditions.

In particular, changing finitely many local factors produces only finite-rank perturbations.

6.11 Summary

In this chapter we have:

- introduced prime-cutoff Hamiltonians,
- identified and removed the scalar divergence,
- established uniform quadratic-form bounds,
- constructed a limiting closed form,
- obtained a self-adjoint SMRK Hamiltonian for general L -functions.

The next chapter analyzes spectral properties and compactness of the resulting operator.

7 Spectral Properties and Compactness

7.1 Preliminaries

Let $\tilde{H}_L$ denote the self-adjoint SMRK Hamiltonian associated with a general L -function, constructed in the previous chapter. By construction, $\tilde{H}_L$ is bounded from below and defined via a closed quadratic form on the arithmetic Hilbert space with internal degrees of freedom.

All spectral statements in this chapter are understood modulo the additive renormalization constant.

7.2 Reference Operator and Confinement

The reference quadratic form q_0 incorporates a logarithmic confining potential of the form

$$q_0[\psi] = \sum_{n \geq 1} (1 + \log^2 n) \|\psi(n)\|^2 \frac{1}{n}. \quad (12.31)$$

This confinement suppresses delocalization in the arithmetic variable and plays the role of an effective potential growing at infinity.

7.3 Compactness of the Resolvent

Let H_0 denote the self-adjoint operator associated with q_0 . Since the confining term diverges as $n \rightarrow \infty$, the resolvent $(H_0 + i)^{-1}$ is compact.

The difference $\tilde{H}_L - H_0$ is relatively form-bounded with relative bound zero. By standard results in spectral theory, compactness of the resolvent is preserved under such perturbations.

Therefore, the resolvent

$$(\tilde{H}_L + i)^{-1} \quad (12.32)$$

is compact on the arithmetic Hilbert space.

7.4 Pure Point Spectrum

Compactness of the resolvent implies that the spectrum of $\tilde{H}_L$ consists entirely of isolated eigenvalues of finite multiplicity.

There is no continuous or residual spectrum. The eigenvalues form a sequence

$$\sigma(\tilde{H}_L) = \{\lambda_k\}_{k \geq 1}, \quad \lambda_k \rightarrow +\infty \text{ as } k \rightarrow \infty. \quad (12.33)$$

7.5 Eigenfunction Regularity

Let ψ_k be an eigenfunction associated with the eigenvalue λ_k . Since ψ_k lies in the form domain of q_0 , it satisfies

$$\sum_{n \geq 1} \log^2 n \|\psi_k(n)\|^2 \frac{1}{n} < \infty. \quad (12.34)$$

Thus eigenfunctions are localized in the logarithmic variable and decay rapidly at large arithmetic scales.

7.6 Growth of Eigenvalues

The confining nature of the reference operator provides a comparison principle for the asymptotic growth of eigenvalues.

Under the change of variables $x = \log n$, the operator H_0 becomes unitarily equivalent to a one-dimensional Schrödinger operator with quadratic confinement. Standard Weyl-type arguments then imply quadratic growth of eigenvalues:

$$c_1 k^2 \leq \lambda_k \leq c_2 k^2 \quad (12.35)$$

for sufficiently large k and positive constants c_1, c_2 .

7.7 Internal Degrees of Freedom

The presence of finite-dimensional internal spaces does not alter compactness or discreteness of the spectrum. Internal multiplicities are absorbed into the finite multiplicity of eigenvalues.

Thus the essential spectral behavior is governed by arithmetic confinement rather than representation-theoretic complexity.

7.8 Spectral Decomposition

The spectral theorem yields the expansion

$$\tilde{H}_L = \sum_{k \geq 1} \lambda_k |\psi_k\rangle \langle \psi_k|, \quad (12.36)$$

with convergence in the strong operator topology.

This decomposition underlies all trace and resolvent constructions in subsequent chapters.

7.9 Stability under Local Perturbations

Finite modifications of local data at ramified primes correspond to finite-rank perturbations of $\tilde{H}_L$. Such perturbations do not affect compactness of the resolvent or the asymptotic growth of eigenvalues.

Therefore, the spectral properties established here are stable under admissible variations of local arithmetic data.

7.10 Summary

In this chapter we have:

- established compactness of the resolvent,
- shown that the spectrum is purely discrete,
- analyzed eigenfunction localization,
- derived quadratic growth of eigenvalues,
- demonstrated stability under internal and local perturbations.

The next chapter introduces trace objects and generalized von Mangoldt weights for general L -functions.

8 Trace Objects and Generalized von Mangoldt Functions

8.1 Purpose of Trace Objects

Trace objects provide the principal bridge between the spectral data of the SMRK Hamiltonian and arithmetic structures encoded by general L -functions. In the operator-theoretic framework, traces replace Dirichlet series and contour integrals as the primary analytic tools.

This chapter introduces the relevant trace objects and explains how generalized von Mangoldt functions emerge naturally from their arithmetic expansions.

8.2 Spectral Traces

Let $\tilde{H}_L$ be the self-adjoint SMRK Hamiltonian constructed previously. For a suitable test function $f : \mathbb{R} \rightarrow \mathbb{C}$ with sufficient decay, the operator $f(\tilde{H}_L)$ is trace class and admits the spectral expansion

$$\mathrm{Tr} f(\tilde{H}_L) = \sum_{k \geq 1} f(\lambda_k), \quad (12.37)$$

where $\{\lambda_k\}$ are the eigenvalues of $\tilde{H}_L$.

8.3 Heat Kernel Trace

A fundamental example is the heat kernel trace, defined for $t > 0$ by

$$K_L(t) := \mathrm{Tr} e^{-t\tilde{H}_L} = \sum_{k \geq 1} e^{-t\lambda_k}. \quad (12.38)$$

Quadratic growth of the eigenvalues guarantees convergence for all positive t . The heat kernel encodes global spectral information and admits both short-time and long-time asymptotic regimes.

8.4 Resolvent Trace

For complex z with nonzero imaginary part, the resolvent trace is defined by

$$R_L(z) := \mathrm{Tr} (\tilde{H}_L - z)^{-1} = \sum_{k \geq 1} \frac{1}{\lambda_k - z}. \quad (12.39)$$

This function is analytic away from the real axis and serves as a generating object for trace identities via contour integration.

8.5 Weighted Trace Constructions

To extract arithmetic information, traces are combined with diagonal weight operators. For real $s > 0$, define

$$(W_s \psi)(n) := n^{-s} \psi(n). \quad (12.40)$$

For $s > 1/2$, the operator W_s is Hilbert–Schmidt, and weighted traces of the form

$$\mathrm{Tr}(W_s f(\tilde{H}_L) W_s) \quad (12.41)$$

are well defined for a broad class of test functions f .

8.6 Spectral Expansion of Weighted Traces

Using the spectral decomposition of $\tilde{H}_L$, one obtains

$$\mathrm{Tr}(W_s f(\tilde{H}_L) W_s) = \sum_{k \geq 1} f(\lambda_k) \sum_{n \geq 1} n^{-2s} \|\psi_k(n)\|^2. \quad (12.42)$$

The inner sum defines a spectral weight capturing the arithmetic distribution of the eigenfunction ψ_k .

8.7 Arithmetic Expansion Strategy

An alternative evaluation of trace objects proceeds by expanding the operator $f(\tilde{H}_L)$ in terms of the prime-shift structure of the Hamiltonian.

Closed arithmetic paths, formed by sequences of prime multiplications and divisions that return to the same integer, contribute to the trace. Each such path is naturally associated with a prime power.

8.8 Emergence of Generalized von Mangoldt Functions

Evaluating the contribution of a prime power p^m yields weights proportional to

$$\sum_{j=1}^d \alpha_{p,j}^m \log p = \Lambda_L(p^m), \quad (12.43)$$

where $\{\alpha_{p,j}\}$ are the local parameters of the L -function.

Thus the generalized von Mangoldt function arises as the natural coefficient governing prime-power contributions in trace expansions.

8.9 Heat Kernel Expansion

At the level of the heat kernel trace, the arithmetic expansion takes the schematic form

$$K_L(t) \sim (\text{smooth background}) + \sum_{p \in \mathbb{P}} \sum_{m \geq 1} \Lambda_L(p^m) e^{-t m \log p}, \quad (12.44)$$

up to normalization factors and smoothing determined by the confining potential.

This structure mirrors the prime side of classical explicit formulae.

8.10 Universality of the Construction

The appearance of generalized von Mangoldt functions is independent of the specific choice of trace object. Heat kernels, resolvents, and smoothed spectral zeta functions all produce the same prime-power weights.

This universality reflects the intrinsic encoding of arithmetic data in the prime-shift operators rather than in analytic continuation procedures.

8.11 Interpretation

In the SMRK framework, generalized von Mangoldt functions are not inserted by hand. They emerge dynamically as the weights of closed arithmetic loops generated by the Hamiltonian.

Trace objects therefore provide a direct operator-theoretic realization of arithmetic explicit-formula structures for general L -functions.

8.12 Summary

In this chapter we have:

- introduced spectral and weighted trace objects,
- derived their spectral expansions,
- developed arithmetic expansions via prime-shift dynamics,
- identified generalized von Mangoldt functions as prime-power weights,
- established universality across different trace constructions.

The next chapter uses these trace objects to derive explicit-formula interfaces for general L -functions.

9 Explicit Formula Interface for General L -Functions

9.1 Objective

The purpose of this chapter is to derive explicit arithmetic–spectral identities for general L -functions within the SMRK framework. These identities play the role of explicit formulae: they relate weighted sums over spectral data of a self-adjoint operator to weighted sums over prime powers governed by generalized von Mangoldt functions.

Unlike classical approaches, no analytic continuation of Dirichlet series is assumed. All identities arise from operator-theoretic trace calculus.

9.2 Test Functions and Functional Calculus

Let $f : \mathbb{R} \rightarrow \mathbb{C}$ be a smooth test function with sufficient decay at infinity. By the functional calculus for self-adjoint operators, the operator $f(\tilde{H}_L)$ is trace class.

Its trace admits the spectral representation

$$\mathrm{Tr} f(\tilde{H}_L) = \sum_{k \geq 1} f(\lambda_k), \quad (12.45)$$

where $\{\lambda_k\}$ are the eigenvalues of $\tilde{H}_L$.

The goal is to compute this trace in an alternative, arithmetic form.

9.3 Resolvent-Based Representation

A convenient starting point is the Cauchy integral representation

$$f(\tilde{H}_L) = \frac{1}{2\pi i} \int_{\Gamma} f(z) (\tilde{H}_L - z)^{-1} dz, \quad (12.46)$$

where Γ is a contour enclosing the spectrum.

Taking traces yields

$$\mathrm{Tr} f(\tilde{H}_L) = \frac{1}{2\pi i} \int_{\Gamma} f(z) R_L(z) dz, \quad (12.47)$$

with $R_L(z)$ the resolvent trace.

9.4 Large-Parameter Expansion of the Resolvent

For large $|z|$, the resolvent admits the expansion

$$(\tilde{H}_L - z)^{-1} = -\frac{1}{z} \sum_{m \geq 0} \left(\frac{\tilde{H}_L}{z} \right)^m. \quad (12.48)$$

Substituting this expansion into the contour integral and interchanging summation and integration produces an asymptotic expansion of the trace in inverse powers of z .

Each term involves traces of powers of the Hamiltonian, which may be evaluated using its prime-shift structure.

9.5 Closed Arithmetic Paths

Powers of the Hamiltonian expand into sums over products of decorated prime-shift operators. Only closed arithmetic paths contribute to the trace.

Such paths correspond to finite sequences of prime multiplications and divisions whose net effect on the arithmetic variable is trivial. Each closed path is canonically associated with a prime power p^m .

9.6 Prime-Power Contributions

Evaluating the contribution of a closed path associated with p^m yields a weight proportional to

$$\sum_{j=1}^d \alpha_{p,j}^m \log p = \Lambda_L(p^m), \quad (12.49)$$

where $\{\alpha_{p,j}\}$ are the local parameters of the L -function.

Thus generalized von Mangoldt functions govern the prime-power side of the explicit formula.

9.7 Explicit Formula Structure

Collecting contributions yields an identity of the schematic form

$$\sum_{k \geq 1} f(\lambda_k) = \mathcal{B}(f) + \sum_{p \in \mathbb{P}} \sum_{m \geq 1} \Lambda_L(p^m) \hat{f}(m \log p), \quad (12.50)$$

where $\mathcal{B}(f)$ denotes a smooth background term and $\hat{f}$ is a transform of the test function determined by the chosen trace object.

This identity constitutes the explicit formula interface for general L -functions.

9.8 Comparison with Classical Explicit Formulae

The structure mirrors that of classical explicit formulae:

- spectral sums replace sums over zeros of L -functions,
- prime-power sums appear with generalized von Mangoldt weights,
- test functions and their transforms mediate the correspondence.

The essential distinction is foundational: the SMRK formula arises from a self-adjoint operator rather than from analytic continuation of a Dirichlet series.

9.9 Stability under Renormalization

Renormalization introduces only an additive scalar shift in the spectrum. Since explicit formulae involve trace differences or test functions with sufficient decay, they are invariant under such shifts.

Therefore, the explicit formula interface is intrinsic to the SMRK Hamiltonian and independent of regularization choices.

9.10 Scope and Limitations

The explicit formula interface derived here:

- applies uniformly to broad classes of L -functions,
- does not assume functional equations or analytic continuation,
- does not identify SMRK eigenvalues with classical zero sets.

It should be viewed as an operator-theoretic analogue rather than a replacement of classical explicit formulae.

9.11 Summary

In this chapter we have:

- derived explicit arithmetic–spectral identities from trace calculus,
- identified generalized von Mangoldt functions as prime-power weights,
- established a direct analogy with classical explicit formulae,
- demonstrated invariance under renormalization.

The next chapter analyzes functional-equation analogues and symmetry principles for general L -functions within the SMRK framework.

10 Functional Equation and Gauge Symmetry

10.1 Motivation

In classical analytic number theory, the functional equation of an L -function expresses a deep duality relating values at complementary arguments. Within the SMRK framework, no analytic continuation is assumed. Instead, functional-equation-like behavior must arise from intrinsic operator symmetries.

This chapter identifies the relevant symmetries of the generalized SMRK Hamiltonian and explains how they induce functional-equation analogues at the level of trace identities.

10.2 Logarithmic Inversion

The arithmetic variable admits a natural logarithmic parametrization $x = \log n$. In this representation, inversion corresponds to reflection

$$x \mapsto -x. \quad (12.51)$$

At the operator level, this transformation is implemented by a unitary involution $\mathcal{I}$ acting on arithmetic states, defined abstractly by its action on the logarithmic variable.

10.3 Gauge Action on Internal Degrees of Freedom

General L -functions carry local representation data at each prime. This introduces a natural gauge freedom: internal bases at different primes may be independently rotated without altering arithmetic content.

Formally, let U_p be a unitary operator acting on the local space $\mathcal{V}_p$. Gauge transformations act on arithmetic states by

$$\psi(n) \mapsto \left(\bigotimes_{p|n} U_p^{\otimes \nu_p} \right) \psi(n), \quad (12.52)$$

where $n = \prod_p p^{\nu_p}$.

10.4 Gauge Covariance of Prime Shifts

The decorated prime-shift operators transform covariantly under local gauge transformations. Specifically,

$$S_p \mapsto U_p S_p U_p^{-1}, \quad S_p^* \mapsto U_p S_p^* U_p^{-1}. \quad (12.53)$$

Consequently, the symmetric combination $S_p + S_p^*$ is gauge invariant. This invariance ensures that the SMRK Hamiltonian depends only on gauge-invariant local data.

10.5 Inversion Symmetry of the Prime Interaction

Under logarithmic inversion, multiplication and division by primes are exchanged. This induces the transformation

$$\mathcal{I} S_p \mathcal{I}^{-1} = S_p^*, \quad \mathcal{I} S_p^* \mathcal{I}^{-1} = S_p. \quad (12.54)$$

As a result, the prime interaction term in the Hamiltonian is invariant under inversion:

$$\mathcal{I} \left(S_p + S_p^* \right) \mathcal{I}^{-1} = S_p + S_p^*. \quad (12.55)$$

10.6 Diagonal Potential and Symmetry Breaking

The diagonal potential contains a logarithmic term $\log n$, which is odd under inversion in the logarithmic variable. This term therefore breaks exact inversion symmetry.

The symmetry breaking is controlled and essential for confinement. It plays a role analogous to a linear perturbation in a symmetric confining potential.

10.7 Functional Equation Analogue

Combining inversion symmetry of the prime interaction with gauge invariance yields identities of the form

$$\mathrm{Tr} \, f(\tilde{H}_L) = \mathrm{Tr} \, (\mathcal{F}f)(\tilde{H}_L), \quad (12.56)$$

where $\mathcal{F}$ denotes a transform intertwining inversion in the logarithmic variable with internal gauge action.

These identities serve as functional-equation analogues within the SMRK framework. They are exact at the operator level and do not rely on analytic continuation.

10.8 Gauge-Invariant Trace Objects

All trace objects introduced previously are invariant under local gauge transformations. In particular, weighted resolvent traces and heat kernel traces depend only on the eigenvalues of local matrices A_p , not on their choice of basis.

This invariance reflects the representation-theoretic nature of the arithmetic data and ensures consistency across different realizations of the same L -function.

10.9 Spectral Duality

The inversion symmetry induces a duality between spectral contributions associated with large and small arithmetic scales. At the level of trace expansions, this duality pairs prime-power contributions in a manner reminiscent of classical functional equations.

While no direct relation between spectral parameters and complex arguments of L -functions is asserted, the structural analogy is exact.

10.10 Stability under Renormalization

Renormalization introduces only a scalar spectral shift. Since both inversion symmetry and gauge invariance act through unitary conjugation, all functional-equation analogues derived here are invariant under renormalization.

10.11 Summary

In this chapter we have:

- identified logarithmic inversion as a fundamental symmetry,

- introduced local gauge transformations on internal spaces,
- established gauge covariance of prime-shift operators,
- derived functional-equation analogues from operator symmetries,
- demonstrated stability under renormalization.

The next chapter applies this framework to Dirichlet L -functions as a concrete and instructive special case.

11 Dirichlet L -Functions

11.1 Role as a Canonical Test Case

Dirichlet L -functions provide the simplest nontrivial generalization of the Riemann zeta function. They introduce genuine local phases while retaining a scalar internal structure. As such, they serve as an ideal testing ground for the generalized SMRK framework.

In this chapter we specialize the general construction to Dirichlet characters and analyze the resulting operator-theoretic features.

11.2 Dirichlet Characters

Let χ be a Dirichlet character modulo q . The associated Dirichlet L -function is defined formally by

$$L(s, \chi) = \sum_{n \geq 1} \frac{\chi(n)}{n^s} = \prod_p (1 - \chi(p)p^{-s})^{-1}, \quad (12.57)$$

where $\chi(p) = 0$ for primes dividing q .

In this setting, the degree of the L -function is one, but local data are nontrivial due to the presence of phases.

11.3 Local Data for Dirichlet Characters

For primes $p \nmid q$, the local matrix reduces to the scalar

$$A_p = \chi(p). \quad (12.58)$$

For primes $p \mid q$, the local factor is trivial and contributes no prime shift. These primes are ramified and require special treatment.

11.4 Arithmetic Hilbert Space

Since the degree is one, internal spaces $\mathcal{V}_p$ and $\mathcal{V}_n$ are one-dimensional for all n . Arithmetic states are therefore scalar-valued functions $\psi(n)$, as in the zeta-function case.

However, the action of prime shifts is modified by character values.

11.5 Decorated Prime Shifts

The multiplicative prime-shift operator acts by

$$(S_p\psi)(n) = \begin{cases} \chi(p) \psi(n/p), & p \mid n, \ p \nmid q, \\ 0, & \text{otherwise.} \end{cases} \quad (12.59)$$

The adjoint shift is given by

$$(S_p^*\psi)(n) = \begin{cases} \overline{\chi(p)} \psi(pn), & p \nmid q, \\ 0, & p \mid q. \end{cases} \quad (12.60)$$

These operators reduce to the scalar SMRK shifts when χ is trivial.

11.6 Generalized von Mangoldt Function

For Dirichlet L -functions, the generalized von Mangoldt function takes the form

$$\Lambda_\chi(p^m) = \chi(p)^m \log p, \quad (12.61)$$

and vanishes on integers that are not prime powers.

This function governs the prime-power side of explicit formulae.

11.7 Hamiltonian for Dirichlet Characters

The SMRK Hamiltonian associated with χ is given formally by

$$H_\chi = \sum_{p \nmid q} \frac{1}{p} (S_p + S_p^*) + V_\chi, \quad (12.62)$$

where V_χ contains the logarithmic confinement and diagonal arithmetic terms.

Ramified primes $p \mid q$ contribute only through finite-rank modifications.

11.8 Renormalization and Self-Adjointness

Renormalization proceeds exactly as in the scalar case. The divergence of the prime sum is scalar and independent of the character χ .

The resulting self-adjoint operator $\tilde{H}_\chi$ differs from the zeta-case Hamiltonian by bounded phase factors in the prime shifts.

11.9 Spectral Properties

All spectral properties established for the general SMRK Hamiltonian apply to $\tilde{H}_\chi$:

- compact resolvent,
- purely discrete spectrum,
- logarithmic localization of eigenfunctions,
- quadratic growth of eigenvalues.

The presence of character phases affects fine spectral correlations but not global spectral structure.

11.10 Explicit Formula Interface

Trace expansions yield explicit formulae of the form

$$\sum_k f(\lambda_k^{(\chi)}) = \mathcal{B}_\chi(f) + \sum_{p \nmid q} \sum_{m \geq 1} \chi(p)^m \log p \hat{f}(m \log p), \quad (12.63)$$

where $\{\lambda_k^{(\chi)}\}$ are eigenvalues of $\tilde{H}_\chi$.

This reproduces the expected structure of explicit formulae for Dirichlet L -functions.

11.11 Interpretation

Dirichlet characters introduce arithmetic phases into prime dynamics without altering the dimensionality of the internal space. This highlights the flexibility of the SMRK framework: nontrivial arithmetic information can be encoded purely through decorated prime shifts.

11.12 Summary

In this chapter we have:

- specialized the SMRK framework to Dirichlet L -functions,
- defined character-decorated prime-shift operators,
- constructed the associated Hamiltonian,
- analyzed spectral and renormalization properties,
- derived the explicit formula interface.

The next chapter extends these ideas to automorphic L -functions with genuinely higher-dimensional local data.

12 Automorphic L -Functions

12.1 Motivation

Automorphic L -functions represent the most structured and representation-theoretic class within the landscape of L -functions. Their local data arise from automorphic representations of reductive groups and carry genuinely higher-dimensional internal degrees of freedom.

This chapter explains how automorphic L -functions fit naturally into the SMRK framework and how their rich local structure is reflected in operator dynamics.

12.2 Automorphic Representations and Local Factors

Let π be an automorphic representation of a reductive group over the adèles. The associated standard L -function admits an Euler product

$$L(s, \pi) = \prod_p L_p(s, \pi), \quad (12.64)$$

with local factors of the form

$$L_p(s, \pi) = \det(I - A_p(\pi) p^{-s})^{-1}. \quad (12.65)$$

Here $A_p(\pi)$ is a matrix whose eigenvalues encode the Satake parameters of the representation at the prime p .

12.3 Internal Spaces and Satake Parameters

For automorphic L -functions of degree d , the local internal spaces

$$\mathcal{V}_p \cong \mathbb{C}^d \quad (12.66)$$

carry an action of the local matrix $A_p(\pi)$.

The eigenvalues of $A_p(\pi)$, often denoted by $\{\alpha_{p,1}, \dots, \alpha_{p,d}\}$, determine both the Euler factors and the generalized von Mangoldt weights.

12.4 Ramification and Local Modification

At almost all primes, the representation π is unramified and the local data are uniformly bounded. At ramified primes, the structure of $A_p(\pi)$ may change and the local internal space may differ.

Within the SMRK framework, ramified primes produce only finite-rank modifications of the Hamiltonian. These do not affect renormalization, compactness, or asymptotic spectral behavior.

12.5 Prime Shifts with Automorphic Data

Decorated prime-shift operators act on arithmetic states by combining arithmetic multiplication or division with the action of $A_p(\pi)$ on internal degrees of freedom.

This action implements the local representation-theoretic content of π at the level of arithmetic dynamics.

12.6 Hamiltonian for Automorphic L -Functions

The SMRK Hamiltonian associated with π is defined formally by

$$H_\pi = \sum_{p \in \mathbb{P}} \frac{1}{p} (S_p^{(\pi)} + S_p^{(\pi)*}) + V_\pi, \quad (12.67)$$

where $S_p^{(\pi)}$ denotes the prime-shift operator decorated with automorphic local data, and V_π is the diagonal potential incorporating logarithmic confinement.

12.7 Renormalization and Self-Adjointness

Renormalization proceeds exactly as in the general L -function case. The divergence of the prime sum is scalar and independent of the representation π .

The resulting renormalized operator $\tilde{H}_\pi$ is self-adjoint, bounded from below, and has compact resolvent.

12.8 Spectral Structure

The spectrum of $\tilde{H}_\pi$ consists of discrete eigenvalues of finite multiplicity, diverging to infinity. Internal multiplicities arising from the representation π contribute to degeneracies but do not alter global spectral growth.

Eigenfunctions remain logarithmically localized in the arithmetic variable.

12.9 Generalized von Mangoldt Weights

For automorphic L -functions, the generalized von Mangoldt function takes the form

$$\Lambda_\pi(p^m) = \sum_{j=1}^d \alpha_{p,j}^m \log p, \quad (12.68)$$

where $\{\alpha_{p,j}\}$ are the Satake parameters of π .

These weights govern the prime-power side of explicit formulae derived from trace objects.

12.10 Explicit Formula Interface

Trace expansions of $\tilde{H}_\pi$ yield identities of the form

$$\sum_k f(\lambda_k^{(\pi)}) = \mathcal{B}_\pi(f) + \sum_{p \in \mathbb{P}} \sum_{m \geq 1} \Lambda_\pi(p^m) \hat{f}(m \log p), \quad (12.69)$$

which mirror the structure of explicit formulae for automorphic L -functions.

The dependence on π enters exclusively through the local Satake parameters.

12.11 Representation-Theoretic Interpretation

Within the SMRK framework, automorphic L -functions are realized as arithmetic Hamiltonians whose internal degrees of freedom carry representation-theoretic data.

This provides a concrete operator-theoretic interpretation of automorphic local data and unifies them with prime-generated arithmetic dynamics.

12.12 Summary

In this chapter we have:

- incorporated automorphic local data into the SMRK framework,
- defined prime-shift operators decorated with Satake parameters,
- constructed the associated Hamiltonian,
- analyzed renormalization and spectral properties,
- derived explicit formula interfaces for automorphic L -functions.

This completes the extension of the SMRK Hamiltonian to general classes of L -functions. The appendices collect technical details and numerical protocols supporting the constructions developed in this document.

13 Conclusion and Outlook

13.1 Summary of the Construction

In this document we have developed a unified operator-theoretic framework for L -functions based on the SMRK Hamiltonian. Starting from the arithmetic structure of the Riemann zeta

function, we systematically extended the construction to general L -functions by incorporating local representation data and internal degrees of freedom.

The resulting framework replaces Dirichlet series and analytic continuation by a self-adjoint operator acting on an arithmetic Hilbert space. Prime dynamics are implemented by decorated prime-shift operators, while diagonal potentials provide logarithmic confinement and arithmetic forcing.

13.2 Renormalization and Self-Adjointness

A central technical result is that the divergence arising from infinite prime sums is scalar in form sense and can be removed by renormalization. This leads to a closed, semi-bounded quadratic form and a unique self-adjoint realization of the SMRK Hamiltonian.

The construction is robust: it is stable under variation of local arithmetic data, under inclusion of ramified primes, and under changes of renormalization scheme up to unitary equivalence.

13.3 Spectral Structure

The renormalized SMRK Hamiltonian has compact resolvent and purely discrete spectrum. Eigenvalues diverge quadratically, and eigenfunctions are logarithmically localized in the arithmetic variable.

These properties hold uniformly across the entire class of L -functions considered, from Dirichlet characters to automorphic representations with higher-dimensional local data.

13.4 Trace Objects and Explicit Formulae

Trace objects associated with the SMRK Hamiltonian provide the primary analytic interface between spectral and arithmetic data. Heat kernels, resolvents, and weighted traces admit both spectral expansions and arithmetic expansions.

Generalized von Mangoldt functions emerge naturally as the weights of prime-power contributions associated with closed arithmetic paths. This yields explicit-formula interfaces that mirror classical explicit formulae, but arise purely from operator theory without analytic continuation.

13.5 Symmetry Principles

Functional-equation analogues appear as consequences of intrinsic operator symmetries, notably logarithmic inversion and local gauge invariance of internal degrees of freedom. These symmetries constrain trace identities and induce dualities between spectral and arithmetic contributions.

The framework thus realizes the spirit of the Hilbert–Pólya philosophy at the level of operator dynamics and trace calculus, while remaining agnostic about the direct identification of spectral parameters with classical zero sets.

13.6 Special Cases and Universality

Dirichlet L -functions and automorphic L -functions fit naturally into the same architecture. Differences between these cases are encoded entirely in local matrices and internal spaces, while the global operator structure remains fixed.

This universality suggests that the SMRK Hamiltonian captures a common arithmetic skeleton underlying diverse classes of L -functions.

13.7 Limitations

The results presented here do not constitute proofs of classical conjectures such as the Riemann Hypothesis. No claim is made that the spectrum of the SMRK Hamiltonian coincides with the zero sets of L -functions.

Instead, the framework should be viewed as complementary: it provides a rigorous, self-adjoint operator setting in which arithmetic explicit-formula structures arise naturally and transparently.

13.8 Future Directions

Several directions for further research emerge naturally:

- refinement of numerical probes and large-scale spectral experiments,
- investigation of level statistics and symmetry classes,
- extension to families of L -functions and parameter-dependent Hamiltonians,
- deeper analysis of gauge symmetries and representation-theoretic constraints,
- exploration of connections with adelic trace formulae and noncommutative geometric frameworks.

These directions point toward a broader operator-based perspective on arithmetic spectral theory.

13.9 Final Remark

The SMRK Hamiltonian provides a concrete realization of arithmetic dynamics driven by prime structure and local representation data. By placing L -functions within a self-adjoint operator framework, it opens a pathway toward a spectral understanding of arithmetic that is both conceptually transparent and analytically controlled.

This concludes the document.

APPENDIX A

A.1 General Euler Factors and Local Parameter Structures

A.1.1 Purpose of the Appendix

This appendix collects the formal definitions and structural properties of local Euler factors used in the construction of the SMRK Hamiltonian for general L -functions. The emphasis is on operator-ready formulations suitable for prime-local dynamics.

No analytic continuation assumptions are made; all statements are algebraic or representation-theoretic.

A.1.2 Local Euler Factors

Let $L(s)$ be an L -function of degree d . For each prime p , the local Euler factor is given by

$$L_p(s) = \det(I - A_p p^{-s})^{-1}, \quad (12.1)$$

where $A_p \in \text{Mat}_{d \times d}(\mathbb{C})$ encodes the local arithmetic data at p .

The matrix A_p is defined up to similarity transformation and represents intrinsic local information.

A.1.3 Local Eigenvalues and Satake Parameters

Assume that A_p is diagonalizable over $\mathbb{C}$ with eigenvalues

$$\{\alpha_{p,1}, \dots, \alpha_{p,d}\}. \quad (12.2)$$

In this case,

$$L_p(s) = \prod_{j=1}^d (1 - \alpha_{p,j} p^{-s})^{-1}. \quad (12.3)$$

The parameters $\alpha_{p,j}$ are referred to as local parameters or Satake parameters.

A.1.4 Boundedness Conditions

For the SMRK Hamiltonian to be well defined, the local parameters must satisfy uniform boundedness conditions. A sufficient assumption is the existence of a constant $C > 0$ such that

$$|\alpha_{p,j}| \leq C \quad (12.4)$$

for all primes p and all indices j .

This condition ensures that decorated prime-shift operators are uniformly form-bounded after weighting by $1/p$.

A.1.5 Ramified Primes

At a finite set of primes, called ramified primes, the local factor may differ from the generic form. This may include:

- reduction of degree,
- non-semisimple local matrices,
- vanishing of certain prime-shift contributions.

Within the SMRK framework, ramified primes are treated by local modification of the corresponding operators. Since only finitely many primes are involved, such modifications produce only finite-rank perturbations.

A.1.6 Tensorial Assembly over Composite Integers

For a composite integer

$$n = \prod_p p^{\nu_p}, \quad (12.5)$$

the associated internal space is constructed as

$$\mathcal{V}_n = \bigotimes_{p|n} \mathcal{V}_p^{\otimes \nu_p}. \quad (12.6)$$

Local matrices A_p act on the corresponding tensor factors. This tensorial assembly mirrors the multiplicative structure of Euler products.

A.1.7 Determinant Structure and Trace Interpretation

The determinant appearing in $L_p(s)$ corresponds, in the operator framework, to a trace over the internal space associated with closed prime-power loops.

Specifically, contributions of the form $\text{Tr}(A_p^m)$ arise from m -fold applications of decorated prime-shift operators and generate prime-power weights.

A.1.8 Generalized von Mangoldt Weights

The local data determine generalized von Mangoldt weights via

$$\Lambda_L(p^m) = \text{Tr}(A_p^m) \log p = \sum_{j=1}^d \alpha_{p,j}^m \log p. \quad (12.7)$$

These quantities govern the arithmetic side of trace expansions and explicit formulae.

A.1.9 Gauge Invariance of Local Data

Local matrices A_p are defined up to conjugation by unitary transformations on $\mathcal{V}_p$. All trace expressions and prime-power weights are invariant under such gauge transformations.

This invariance ensures that the SMRK Hamiltonian depends only on intrinsic arithmetic data.

A.1.10 Summary

This appendix has:

- defined general local Euler factors,
- introduced local parameter matrices and eigenvalues,
- stated admissibility and boundedness conditions,
- explained treatment of ramified primes,
- related determinant structures to trace expressions,
- derived generalized von Mangoldt weights.

APPENDIX B

B.1 Numerical Protocol for L -Functions

B.1.1 Scope and Purpose

This appendix describes a numerical protocol for probing spectral and arithmetic features of the SMRK Hamiltonian associated with general L -functions. The protocol is designed to be implementation-agnostic, reproducible, and compatible with the operator-theoretic framework developed in the main text.

No analytic continuation of L -functions is assumed. All numerical procedures are based on finite truncations of prime interactions and standard spectral algorithms.

B.1.2 Finite Prime Cutoff

Numerical computations begin by fixing a prime cutoff P . All prime-indexed sums and operators are truncated to primes $p \leq P$. The resulting truncated Hamiltonian is

$$\tilde{H}_{L,P} = \sum_{\substack{p \in \mathbb{P} \\ p \leq P}} \frac{1}{p} (S_p + S_p^*) + V_L - C(P), \quad (12.1)$$

where $C(P)$ is the renormalization constant.

Increasing P improves approximation quality while increasing computational cost.

B.1.3 Arithmetic Truncation

In addition to the prime cutoff, arithmetic states are restricted to integers $n \leq N$, where N is a chosen arithmetic cutoff. The Hilbert space is thus replaced by a finite-dimensional space spanned by basis vectors indexed by

$$\{(n, \alpha) : 1 \leq n \leq N, \alpha \in \mathcal{V}_n\}. \quad (12.2)$$

Boundary effects at $n = N$ are controlled by discarding transitions that leave the truncated domain.

B.1.4 Matrix Representation

Under the combined cutoffs (P, N) , the operator $\tilde{H}_{L,P}$ becomes a finite Hermitian matrix. Its entries are computed explicitly from:

- prime-shift transitions $n \leftrightarrow pn$,
- internal matrix actions A_p ,
- diagonal potential terms $\log n$ and $\Lambda_L(n)$.

Sparse matrix representations are recommended, as prime-shift interactions are local.

B.1.5 Eigenvalue Computation

Standard numerical linear algebra techniques may be used to compute eigenvalues and eigenvectors of the truncated Hamiltonian. Suitable methods include:

- Lanczos or Arnoldi iterations for sparse matrices,
- shift-and-invert strategies for low-lying spectrum,
- block methods for handling internal degeneracies.

Only the lowest portion of the spectrum is typically stable under truncation.

B.1.6 Convergence Diagnostics

Convergence of spectral quantities is assessed by:

- increasing the prime cutoff P ,
- increasing the arithmetic cutoff N ,
- monitoring stability of low-lying eigenvalues,
- comparing spectral gaps across truncations.

Eigenvalues that stabilize under simultaneous increases of P and N are interpreted as approximations to the true spectrum.

B.1.7 Numerical Trace Objects

Trace objects are approximated numerically by finite sums over computed eigenvalues. For example, the heat kernel trace is approximated by

$$K_{L,P,N}(t) = \sum_{k=1}^K e^{-t\lambda_k^{(P,N)}}, \quad (12.3)$$

where $\lambda_k^{(P,N)}$ are eigenvalues of the truncated Hamiltonian and K is chosen so that omitted terms are exponentially suppressed.

B.1.8 Extraction of Prime-Power Signals

Arithmetic signatures are extracted by analyzing oscillatory components of trace objects as functions of the spectral parameter or time variable t .

Fourier analysis in the logarithmic variable reveals peaks at frequencies $m \log p$, corresponding to prime powers. The amplitudes of these peaks approximate generalized von Mangoldt weights.

B.1.9 Normalization and Scaling

To compare numerical results across different truncations, it is essential to apply consistent normalization conventions:

- subtract the same renormalization constant $C(P)$,
- rescale energies to account for finite-size effects,

- normalize eigenfunctions with respect to the weighted inner product.

These steps ensure meaningful comparison of spectral data.

B.1.10 Numerical Stability and Error Sources

Primary sources of numerical error include:

- truncation of prime sums,
- truncation of arithmetic domain,
- boundary effects at large n ,
- finite precision in eigenvalue solvers.

Errors are controlled by systematic truncation studies and by focusing on robust spectral features.

B.1.11 Reproducibility Guidelines

For reproducibility, numerical experiments should record:

- prime cutoff P ,
- arithmetic cutoff N ,
- local data specification A_p ,
- renormalization constant $C(P)$,
- numerical solver and precision.

This information suffices to reproduce all numerical results within the SMRK framework.

B.1.12 Summary

This appendix has:

- defined finite truncation schemes for numerical study,
- described matrix representations of truncated Hamiltonians,
- outlined eigenvalue and trace computations,
- explained extraction of arithmetic signals,
- identified convergence and stability criteria.

APPENDIX C

C.1 Reduction to the Riemann Zeta Function

C.1.1 Purpose of the Reduction

This appendix explains how the general SMRK Hamiltonian for L -functions reduces to the original zeta-function case when all local data are trivial. The reduction serves as a consistency check and clarifies which structural features are universal and which depend on nontrivial local representations.

C.1.2 Trivial Local Data

Consider the special case in which the degree of the L -function is one and the local matrices satisfy

$$A_p = 1 \tag{12.1}$$

for all primes p .

In this setting, all local Euler factors reduce to

$$L_p(s) = (1 - p^{-s})^{-1}, \tag{12.2}$$

and the global L -function coincides with the Riemann zeta function.

C.1.3 Collapse of Internal Spaces

When $A_p = 1$ for all primes, the local internal spaces $\mathcal{V}_p$ are canonically identified with $\mathbb{C}$. Consequently, all internal tensor products collapse:

$$\mathcal{V}_n \cong \mathbb{C} \tag{12.3}$$

for all integers n .

Arithmetic states reduce to scalar-valued functions $\psi(n)$.

C.1.4 Prime-Shift Operators

In the trivial-data case, decorated prime-shift operators reduce to the scalar shifts

$$(S_p \psi)(n) = \begin{cases} \psi(n/p), & p \mid n, \\ 0, & \text{otherwise,} \end{cases} \tag{12.4}$$

and their adjoints act by

$$(S_p^* \psi)(n) = \psi(pn). \tag{12.5}$$

These operators coincide with those used in the original SMRK construction for the zeta function.

C.1.5 Hamiltonian Reduction

The formal Hamiltonian becomes

$$H_\zeta = \sum_{p \in \mathbb{P}} \frac{1}{p} (S_p + S_p^*) + V_\zeta, \quad (12.6)$$

where V_ζ is the diagonal logarithmic potential.

This is precisely the SMRK Hamiltonian associated with the Riemann zeta function.

C.1.6 Renormalization

The divergence structure of the prime sum is unchanged by the reduction. Renormalized quadratic forms and the resulting self-adjoint operator coincide with those obtained in the zeta-function setting.

Thus the renormalization procedure is universal and independent of local representation data.

C.1.7 Spectral Properties

All spectral properties established in the general theory reduce to the known properties of the zeta-case Hamiltonian:

- compact resolvent,
- purely discrete spectrum,
- quadratic growth of eigenvalues,
- logarithmic localization of eigenfunctions.

This confirms consistency of the generalized framework.

C.1.8 Generalized von Mangoldt Function

In the trivial-data case, the generalized von Mangoldt function reduces to the classical von Mangoldt function:

$$\Lambda_\zeta(p^m) = \log p. \quad (12.7)$$

Trace expansions and explicit formulae therefore reproduce the prime-power structure familiar from the classical theory.

C.1.9 Explicit Formula Interface

The explicit formula interface derived in the general case specializes to the zeta-function explicit formula, with spectral sums over SMRK eigenvalues replacing sums over nontrivial zeros.

This specialization provides a clear bridge between the general operator framework and the original SMRK formulation.

C.1.10 Conceptual Interpretation

The reduction shows that all additional structure in the general L -function case arises from nontrivial local data. The arithmetic backbone—prime-generated dynamics, renormalization, and confinement—remains unchanged.

This highlights the role of the Riemann zeta function as the universal base point of the SMRK framework.

C.1.11 Summary

This appendix has:

- shown how the general SMRK Hamiltonian reduces to the zeta case,
- demonstrated collapse of internal spaces,
- recovered scalar prime-shift operators,
- verified consistency of renormalization and spectral properties,
- identified the classical von Mangoldt function as a special case.

APPENDIX D

D.1 Numerical Experiments for Automorphic Families

D.1.1 Objective

This appendix outlines numerical experiments designed to probe spectral and arithmetic features of the SMRK Hamiltonian associated with automorphic L -functions. The focus is on family-level behavior rather than on individual representations, with the goal of identifying stable patterns under variation of local data.

All experiments follow the numerical protocol described in Appendix B.

D.1.2 Choice of Automorphic Families

Numerical studies are organized around families of automorphic representations $\{\pi_\kappa\}$ indexed by a discrete parameter κ . Typical examples include:

- varying weight or level in $\mathrm{GL}(2)$ cusp forms,
- twists by finite-order characters,
- families with fixed degree and bounded conductor.

Within a family, local matrices $A_p(\pi_\kappa)$ vary coherently with κ , while the global operator architecture remains unchanged.

D.1.3 Local Data Sampling

For numerical feasibility, local data are specified explicitly for primes $p \leq P$. At unramified primes, the matrices $A_p(\pi_\kappa)$ are constructed from prescribed Satake parameters. At ramified primes, local contributions are either omitted or modeled by finite-rank modifications.

Consistency across the family is ensured by fixing normalization conventions for all local matrices.

D.1.4 Truncated Hamiltonians

For each family member π_κ , a truncated Hamiltonian

$$\tilde{H}_{\pi_\kappa, P, N} \tag{12.1}$$

is constructed using the same prime cutoff P and arithmetic cutoff N .

This uniform truncation allows direct comparison of spectral data across different representations within the family.

D.1.5 Low-Lying Spectrum

Numerical experiments focus primarily on the low-lying eigenvalues

$$\{\lambda_k^{(\kappa)}\}_{k=1}^K, \tag{12.2}$$

where K is chosen such that eigenvalues are stable under truncation.

Comparisons are made between:

- absolute eigenvalue positions,
- normalized spacings between consecutive eigenvalues,
- multiplicities induced by internal degrees of freedom.

D.1.6 Spectral Statistics

To study collective behavior, spectral spacings are rescaled by local mean density and aggregated across the family. Empirical distributions of nearest-neighbor spacings provide insight into symmetry classes and correlations.

Such statistics are compared qualitatively across different automorphic families.

D.1.7 Trace-Based Probes

In addition to direct spectral analysis, trace objects are evaluated numerically. For each π_κ , quantities such as

$$K_{\pi_\kappa}(t) = \text{Tr} e^{-t\tilde{H}_{\pi_\kappa}} \quad (12.3)$$

are approximated using truncated spectra.

Family averages of trace objects smooth representation-specific fluctuations and highlight systematic arithmetic contributions.

D.1.8 Prime-Power Signal Extraction

Oscillatory components of trace objects are analyzed via Fourier methods in the logarithmic variable. Peaks at frequencies $m \log p$ are identified and compared across family members.

The stability of peak locations and relative amplitudes across the family provides numerical evidence for the universality of generalized von Mangoldt weights.

D.1.9 Family Averaging

To suppress noise and truncation artifacts, numerical quantities are averaged over the family index κ . Averaged spectra and traces reveal structures that are not visible at the level of individual representations.

This averaging procedure mirrors classical approaches to families of L -functions, but is implemented entirely at the operator level.

D.1.10 Sensitivity to Local Variations

Experiments include controlled perturbations of local data at selected primes. The resulting changes in spectral data are compared with theoretical expectations for finite-rank perturbations.

Observed stability of global spectral features under such perturbations supports the robustness of the SMRK construction.

D.1.11 Limitations of Numerical Experiments

Numerical experiments are limited by finite truncation in both primes and arithmetic domain. As a result:

- only low-lying spectral data are reliable,
- fine arithmetic correlations may be obscured,
- asymptotic regimes are inaccessible.

Results should therefore be interpreted qualitatively rather than as precise quantitative tests.

D.1.12 Interpretation

Numerical experiments for automorphic families suggest that the SMRK Hamiltonian captures universal arithmetic features across broad classes of representations. While individual spectra vary, family-level behavior exhibits remarkable stability.

This supports the view that the operator framework encodes intrinsic arithmetic structure independent of representation-specific details.

D.1.13 Summary

This appendix has:

- outlined numerical experiments for automorphic families,
- described construction of truncated Hamiltonians across families,
- analyzed spectral and trace-based observables,
- investigated stability under local perturbations,
- highlighted universality at the family level.

Chapter 13

The SMRK Program

An Operator-Based Path Toward the Riemann Hypothesis

Enter Yourname

Abstract

This section reflects on the broader implications of operator-based mathematical and computational frameworks for human understanding, technological development, and collective responsibility. Rather than introducing new formal constructions, its purpose is to interpret the preceding mathematical results within a wider conceptual and societal context, clarifying why such abstract structures matter beyond their immediate technical applications.

Operator-based frameworks, as developed throughout this book, shift attention from procedural instruction toward transformation, structure, and invariance. This shift has implications not only for computation, but also for how complex systems are modeled, verified, and governed. By emphasizing well-defined transformations acting on states, operator-centric approaches promote transparency, composability, and traceability—properties that are increasingly essential in large-scale, autonomous, and distributed systems.

From a computational perspective, the operator-first paradigm enables new forms of determinism, auditability, and semantic control. These properties support the development of systems that can be inspected, reasoned about, and corrected at a structural level, rather than treated as opaque artifacts. Such capabilities are particularly relevant in contexts where algorithmic decisions carry ethical, legal, or societal consequences.

At a deeper level, the mathematical structures explored in this book illustrate how abstract reasoning can serve as a stabilizing force in technological evolution. By grounding computation in operator theory and spectral analysis, the framework encourages designs that respect invariants, constraints, and long-term coherence, counterbalancing tendencies toward ad hoc complexity and uncontrolled optimization.

The section also addresses the role of human agency within advanced computational systems. Rather than framing automation as a replacement for human judgment, the operator-based approach supports models in which human oversight, intervention, and interpretation are structurally embedded. This perspective aligns with a vision of technology as an extension of human capability, not its negation.

By concluding Book No. 1 with these considerations, the compendium emphasizes that the mathematical foundations developed here are not ends in themselves. They form part of a broader effort to design computational frameworks that are intellectually honest, technically robust, and aligned with the long-term interests of humanity. This reflection prepares the transition to subsequent parts of the compendium, where computational architectures, virtual machines, and applied frameworks are developed on top of the mathematical core established in Part I.

1 The SMRK Program: Scope, Continuity, and Intent

1.1 Purpose of This Document

This document marks a transition from construction to programmatic synthesis within the SMRK framework. While previous documents established concrete operator-theoretic objects and explicit arithmetic–spectral interfaces, the present work clarifies how these components fit together as part of a coherent long-term program directed toward the Riemann Hypothesis.

The goal is not to present a proof, but to articulate a structurally constrained path that any proof must follow within an operator-based framework.

1.2 Position Within the SMRK Program

This work builds directly on two foundational documents:

- *The SMRK Hamiltonian and Explicit Formulae*,
- *The SMRK Hamiltonian for General L -Functions*.

Those works established the existence of a self-adjoint arithmetic Hamiltonian, its spectral properties, and its explicit-formula interfaces. The present document does not repeat those constructions; instead, it treats them as established infrastructure.

1.3 From Objects to Program

The SMRK framework has now reached a stage where the primary mathematical objects are defined and under control:

- a renormalized self-adjoint Hamiltonian,
- compact resolvent and discrete spectrum,
- trace objects yielding arithmetic explicit formulae,
- extensions to broad classes of L -functions.

The natural next step is to analyze what these structures imply collectively. This requires shifting perspective from isolated results to a programmatic view.

1.4 The Riemann Hypothesis as a Structural Constraint

Within the SMRK perspective, the Riemann Hypothesis is not treated as a statement about the zeros of a complex function. Instead, it is interpreted as a global structural constraint relating arithmetic dynamics, spectral symmetry, and operator positivity.

The hypothesis becomes a statement about the compatibility of spectral localization, trace identities, and symmetry principles within a self-adjoint framework.

1.5 Operator-Based Philosophy

A central philosophical stance of the SMRK program is that arithmetic information is most naturally encoded in operators rather than functions. In this view:

- primes generate dynamics,
- arithmetic weights arise from operator traces,
- analytic continuation is replaced by functional calculus,
- symmetry is expressed through unitary equivalence.

This philosophy underlies all subsequent arguments and structural claims.

1.6 What This Document Does Not Claim

It is important to state explicitly what is *not* claimed here:

- no proof of the Riemann Hypothesis is presented,
- no identification of SMRK eigenvalues with classical zeros is asserted,
- no numerical verification is treated as decisive evidence.

The role of this document is architectural rather than conclusive.

1.7 Programmatic Objectives

The objectives of the SMRK program, as articulated here, are:

- to identify necessary structural conditions for a spectral resolution of RH,
- to eliminate incompatible approaches through operator constraints,
- to clarify which symmetries and trace identities are essential,
- to provide a falsifiable and extensible framework.

These objectives guide the organization of the chapters that follow.

1.8 Outline of the Document

The remainder of this document proceeds as follows. Subsequent chapters analyze the structural consequences of the SMRK Hamiltonian, examine symmetry and rigidity phenomena, discuss obstruction mechanisms, and outline concrete research directions toward a spectral resolution of the Riemann Hypothesis.

Appendices collect methodological remarks and conceptual clarifications that support the main narrative.

1.9 Summary

This chapter has:

- positioned the document within the broader SMRK program,
- clarified its relationship to earlier constructions,
- articulated an operator-based interpretation of the Riemann Hypothesis,
- defined the scope, intent, and limitations of the work.

The next chapter examines the notion of structural necessity and why any viable resolution of the Riemann Hypothesis must ultimately be spectral in nature.

2 Structural Necessity and the Spectral Nature of the Riemann Hypothesis

2.1 From Conjecture to Structural Constraint

The Riemann Hypothesis is traditionally formulated as a conjecture about the location of zeros of a complex-valued function. Within the SMRK program, this formulation is reinterpreted as a structural constraint imposed on any viable arithmetic–spectral framework.

Rather than asking whether a specific function has its zeros on a given line, the SMRK perspective asks which structural properties of arithmetic dynamics necessarily force a spectral symmetry consistent with the hypothesis.

2.2 Why a Spectral Framework Is Unavoidable

Any approach to the Riemann Hypothesis that aims to explain global regularity must address three fundamental features:

- the universality of prime distributions,
- the stability of explicit formulae under smoothing,
- the rigidity imposed by symmetry principles.

These features are naturally and simultaneously expressed in spectral language. Attempts that avoid spectral structures typically reintroduce them implicitly.

2.3 Self-Adjointness as a Necessity

A central insight of the SMRK program is that self-adjointness is not a technical convenience but a structural requirement. Only self-adjoint operators provide:

- real spectral parameters,
- unitary time evolution,
- robust trace identities,
- stability under perturbations.

Any operator purporting to encode arithmetic dynamics relevant to the Riemann Hypothesis must therefore admit a self-adjoint realization.

2.4 Compactness and Discreteness

Compactness of the resolvent is equally essential. Without compactness, spectral data are contaminated by continuous components that obscure arithmetic information.

The SMRK Hamiltonian achieves compactness through logarithmic confinement. This ensures that arithmetic information is captured by discrete spectral data, suitable for trace analysis and explicit-formula interfaces.

2.5 Spectral Localization and Arithmetic Scale

The arithmetic variable naturally lives on a multiplicative scale. Under logarithmic transformation, this becomes an additive variable, and confinement in this variable corresponds to control over arithmetic growth.

Spectral localization in the logarithmic variable is therefore a necessary condition for meaningful arithmetic interpretation of eigenfunctions.

2.6 Trace Identities as Structural Tests

Explicit formulae emerge in the SMRK framework as trace identities. These identities are sensitive to:

- spectral symmetry,
- operator positivity,
- cancellation between arithmetic contributions.

A violation of the Riemann Hypothesis would manifest as a structural inconsistency in these trace identities rather than as an isolated analytic anomaly.

2.7 Positivity and Spectral Symmetry

Within a self-adjoint framework, positivity constraints play a decisive role. Positivity of certain quadratic forms and trace kernels imposes strong restrictions on the allowed distribution of spectral data.

The SMRK program interprets the Riemann Hypothesis as a statement that such positivity and symmetry constraints are globally satisfied.

2.8 Exclusion of Non-Spectral Approaches

From this perspective, purely combinatorial or function-theoretic approaches that do not produce a stable self-adjoint operator fail to address the core structural issues.

While such approaches may yield partial results, they do not provide a framework capable of enforcing the global constraints required by the Riemann Hypothesis.

2.9 Structural Minimality

The SMRK Hamiltonian is minimal in the following sense:

- primes generate all dynamics,

- no auxiliary randomness is introduced,
- no external symmetry assumptions are imposed,
- all structure arises internally from arithmetic.

This minimality strengthens the claim that the framework captures necessary, rather than accidental, features.

2.10 Implications for Proof Strategies

If the Riemann Hypothesis is a structural spectral statement, then any proof strategy must ultimately:

- identify the relevant self-adjoint operator,
- establish compactness and symmetry,
- derive positivity of appropriate trace objects,
- show incompatibility of hypothetical counterexamples.

The SMRK program provides a concrete arena in which these steps can be formulated and tested.

2.11 Summary

In this chapter we have:

- reframed the Riemann Hypothesis as a structural constraint,
- argued for the necessity of a spectral approach,
- identified self-adjointness and compactness as essential features,
- connected spectral localization with arithmetic scale,
- outlined implications for viable proof strategies.

The next chapter analyzes symmetry and rigidity phenomena arising from the SMRK Hamiltonian and their consequences for arithmetic spectra.

3 Symmetry, Rigidity, and Operator-Level Constraints

3.1 Why Symmetry Matters at the Operator Level

In an operator-based framework, symmetry is not an aesthetic property but a structural one. Symmetries constrain admissible dynamics, restrict spectral deformations, and govern the form of trace identities. Within the SMRK program, symmetry acts directly on operators through unitary equivalence rather than indirectly through functional identities.

Any candidate framework relevant to the Riemann Hypothesis must therefore identify which symmetries are intrinsic and which are accidental.

3.2 Unitary Equivalence as Structural Invariance

The primary notion of equivalence in the SMRK program is unitary equivalence of operators. Two Hamiltonians related by a unitary conjugation describe the same arithmetic dynamics, even

if their concrete realizations differ.

This principle implies that all meaningful arithmetic information must be encoded in unitary invariants such as spectra, trace objects, and positivity properties.

3.3 Logarithmic Inversion Symmetry

Arithmetic dynamics are naturally multiplicative. Passing to logarithmic variables renders this structure additive and reveals an inversion symmetry corresponding to reflection in the logarithmic coordinate.

At the operator level, this symmetry is implemented by a unitary involution that interchanges prime multiplication and division. The prime-interaction component of the SMRK Hamiltonian is invariant under this transformation.

3.4 Controlled Symmetry Breaking

Exact symmetry is neither expected nor desired at the level of the full Hamiltonian. Diagonal logarithmic potentials break inversion symmetry in a controlled manner, while preserving essential spectral features such as self-adjointness and compactness.

This controlled symmetry breaking is analogous to confinement mechanisms in physical Hamiltonians and is crucial for spectral discreteness.

3.5 Gauge Symmetry of Local Arithmetic Data

Local arithmetic data are defined only up to choice of basis in internal spaces. This introduces a gauge symmetry acting independently at each prime.

Gauge transformations act by unitary conjugation on decorated prime-shift operators. All trace expressions and generalized von Mangoldt weights are invariant under this action, ensuring that the SMRK Hamiltonian depends only on intrinsic arithmetic data.

3.6 Rigidity Induced by Trace Identities

Trace identities impose rigidity on admissible operator deformations. Small changes to operator structure that preserve self-adjointness and trace positivity are severely restricted.

In particular, deformations that would alter global spectral symmetry necessarily manifest as violations of trace identities or positivity constraints.

3.7 Absence of Continuous Deformation Paths

Within the SMRK framework, there is no continuous deformation connecting operators with spectrally symmetric configurations to operators with spectrally asymmetric ones, without crossing a boundary where self-adjointness, compactness, or trace positivity fails.

This absence of deformation paths is a key rigidity feature and underlies the structural interpretation of the Riemann Hypothesis.

3.8 Operator Constraints Versus Functional Constraints

Traditional approaches often encode symmetry at the level of functional equations. The SMRK program relocates these constraints to the operator level, where they are enforced through unitary

equivalence and positivity.

This shift eliminates reliance on analytic continuation and reframes symmetry as an operator-theoretic necessity rather than a functional coincidence.

3.9 Stability Under Renormalization

Renormalization introduces scalar shifts but does not alter operator symmetries or unitary equivalence classes. As a result, all symmetry and rigidity statements remain valid for the renormalized SMRK Hamiltonian.

This stability ensures that symmetry considerations are meaningful in the infinite prime limit.

3.10 Consequences for Spectral Configurations

The combined effect of symmetry and rigidity is a severe restriction on admissible spectral configurations. Spectral arrangements incompatible with inversion symmetry and trace positivity are excluded at the operator level.

This exclusion does not rely on detailed arithmetic estimates but follows from general structural principles.

3.11 Summary

In this chapter we have:

- identified symmetry as an operator-level invariant,
- emphasized unitary equivalence as the primary notion of structure,
- described logarithmic inversion and gauge symmetries,
- explained controlled symmetry breaking via confinement,
- established rigidity through trace identities,
- argued for the absence of continuous paths to asymmetric spectra.

The next chapter examines how trace positivity and arithmetic cancellation mechanisms translate these structural constraints into enforceable spectral conditions.

4 Symmetry, Rigidity, and Operator-Level Constraints

4.1 Why Symmetry Matters at the Operator Level

In an operator-based framework, symmetry is not an aesthetic property but a structural one. Symmetries constrain admissible dynamics, restrict spectral deformations, and govern the form of trace identities. Within the SMRK program, symmetry acts directly on operators through unitary equivalence rather than indirectly through functional identities.

Any candidate framework relevant to the Riemann Hypothesis must therefore identify which symmetries are intrinsic and which are accidental.

4.2 Unitary Equivalence as Structural Invariance

The primary notion of equivalence in the SMRK program is unitary equivalence of operators. Two Hamiltonians related by a unitary conjugation describe the same arithmetic dynamics, even if their concrete realizations differ.

This principle implies that all meaningful arithmetic information must be encoded in unitary invariants such as spectra, trace objects, and positivity properties.

4.3 Logarithmic Inversion Symmetry

Arithmetic dynamics are naturally multiplicative. Passing to logarithmic variables renders this structure additive and reveals an inversion symmetry corresponding to reflection in the logarithmic coordinate.

At the operator level, this symmetry is implemented by a unitary involution that interchanges prime multiplication and division. The prime-interaction component of the SMRK Hamiltonian is invariant under this transformation.

4.4 Controlled Symmetry Breaking

Exact symmetry is neither expected nor desired at the level of the full Hamiltonian. Diagonal logarithmic potentials break inversion symmetry in a controlled manner, while preserving essential spectral features such as self-adjointness and compactness.

This controlled symmetry breaking is analogous to confinement mechanisms in physical Hamiltonians and is crucial for spectral discreteness.

4.5 Gauge Symmetry of Local Arithmetic Data

Local arithmetic data are defined only up to choice of basis in internal spaces. This introduces a gauge symmetry acting independently at each prime.

Gauge transformations act by unitary conjugation on decorated prime-shift operators. All trace expressions and generalized von Mangoldt weights are invariant under this action, ensuring that the SMRK Hamiltonian depends only on intrinsic arithmetic data.

4.6 Rigidity Induced by Trace Identities

Trace identities impose rigidity on admissible operator deformations. Small changes to operator structure that preserve self-adjointness and trace positivity are severely restricted.

In particular, deformations that would alter global spectral symmetry necessarily manifest as violations of trace identities or positivity constraints.

4.7 Absence of Continuous Deformation Paths

Within the SMRK framework, there is no continuous deformation connecting operators with spectrally symmetric configurations to operators with spectrally asymmetric ones, without crossing a boundary where self-adjointness, compactness, or trace positivity fails.

This absence of deformation paths is a key rigidity feature and underlies the structural interpretation of the Riemann Hypothesis.

4.8 Operator Constraints Versus Functional Constraints

Traditional approaches often encode symmetry at the level of functional equations. The SMRK program relocates these constraints to the operator level, where they are enforced through unitary equivalence and positivity.

This shift eliminates reliance on analytic continuation and reframes symmetry as an operator-theoretic necessity rather than a functional coincidence.

4.9 Stability Under Renormalization

Renormalization introduces scalar shifts but does not alter operator symmetries or unitary equivalence classes. As a result, all symmetry and rigidity statements remain valid for the renormalized SMRK Hamiltonian.

This stability ensures that symmetry considerations are meaningful in the infinite prime limit.

4.10 Consequences for Spectral Configurations

The combined effect of symmetry and rigidity is a severe restriction on admissible spectral configurations. Spectral arrangements incompatible with inversion symmetry and trace positivity are excluded at the operator level.

This exclusion does not rely on detailed arithmetic estimates but follows from general structural principles.

4.11 Summary

In this chapter we have:

- identified symmetry as an operator-level invariant,
- emphasized unitary equivalence as the primary notion of structure,
- described logarithmic inversion and gauge symmetries,
- explained controlled symmetry breaking via confinement,
- established rigidity through trace identities,
- argued for the absence of continuous paths to asymmetric spectra.

The next chapter examines how trace positivity and arithmetic cancellation mechanisms translate these structural constraints into enforceable spectral conditions.

5 Trace Positivity and Arithmetic Cancellation Mechanisms

5.1 Role of Trace Positivity

Trace objects lie at the core of the SMRK framework. They provide the interface between operator theory and arithmetic structure. Among their properties, positivity plays a distinguished role: it imposes global constraints on admissible spectral behavior that cannot be bypassed by local adjustments.

Within the SMRK program, trace positivity is not an auxiliary assumption but a structural requirement inherited from self-adjointness and functional calculus.

5.2 Positive Trace Kernels

Given a self-adjoint Hamiltonian $\tilde{H}$, trace kernels of the form

$$\mathrm{Tr} f(\tilde{H}), \quad (13.1)$$

where f is a nonnegative function, are manifestly nonnegative.

Examples include:

- heat kernel traces $\mathrm{Tr} e^{-t\tilde{H}}$,
- resolvent-based traces with positive spectral weights,
- smoothed spectral projectors.

These objects encode global spectral information while preserving positivity.

5.3 Spectral Decomposition and Positivity

Positivity at the trace level translates into constraints on the spectral decomposition. If $\{\lambda_k\}$ denotes the spectrum of $\tilde{H}$, then

$$\mathrm{Tr} f(\tilde{H}) = \sum_k f(\lambda_k) \geq 0 \quad (13.2)$$

for all admissible nonnegative f .

Any hypothetical spectral configuration leading to negative contributions in such expressions is excluded.

5.4 Arithmetic Expansion of Trace Objects

Trace objects admit parallel arithmetic expansions expressed as sums over prime powers. Schematically,

$$\mathrm{Tr} f(\tilde{H}) = \mathcal{B}(f) + \sum_p \sum_{m \geq 1} \Lambda_L(p^m) \hat{f}(m \log p), \quad (13.3)$$

where $\mathcal{B}(f)$ denotes the bulk contribution and $\hat{f}$ is an appropriate transform of f .

This representation reveals how arithmetic data enter trace expressions.

5.5 Cancellation as a Structural Requirement

Prime-power contributions in the arithmetic expansion are oscillatory. Positivity of the total trace therefore requires delicate cancellation between terms associated with different primes and powers.

Such cancellation is not optional. It is enforced by the requirement that the spectral side remains nonnegative for all admissible test functions.

5.6 Failure Modes of Cancellation

If arithmetic cancellation fails, trace positivity is violated. This manifests as:

- negative contributions for specific test functions,
- instability under smoothing,

- sensitivity to small perturbations of arithmetic weights.

Within the SMRK framework, these failures signal incompatibility with a self-adjoint spectral realization.

5.7 Relation to the Riemann Hypothesis

The Riemann Hypothesis may be interpreted as asserting that the arithmetic cancellation required for trace positivity is globally consistent.

Rather than focusing on individual oscillatory terms, the SMRK program treats the hypothesis as a statement that no admissible test function can expose a positivity violation in trace identities.

5.8 Operator-Level Enforcement

Because trace positivity is enforced at the operator level, it cannot be restored by adjusting arithmetic weights alone. Any modification that disrupts cancellation must either:

- break self-adjointness,
- destroy compactness,
- or violate unitary invariance.

This makes trace positivity a powerful exclusion principle.

5.9 Robustness Under Renormalization

Renormalization affects trace objects only by adding or subtracting scalar terms that do not compromise positivity. Consequently, all positivity arguments remain valid in the infinite prime limit.

This robustness ensures that cancellation mechanisms are meaningful beyond finite truncations.

5.10 Structural Implications

Trace positivity converts spectral symmetry and operator rigidity into enforceable constraints. It transforms abstract symmetry principles into concrete inequalities that must hold uniformly.

These inequalities severely restrict the space of admissible spectral configurations.

5.11 Summary

In this chapter we have:

- identified trace positivity as a core structural requirement,
- described positive trace kernels and their spectral meaning,
- analyzed arithmetic expansions of trace objects,
- explained cancellation as a necessary mechanism,
- linked positivity failure to structural incompatibility,
- interpreted the Riemann Hypothesis as global consistency of trace positivity.

The next chapter analyzes obstruction mechanisms and explains why non-critical spectral configurations are incompatible with the SMRK operator framework.

6 Obstructions to Non-Critical Spectral Configurations

6.1 Meaning of Non-Critical Configurations

Within a spectral framework, a non-critical configuration refers to any arrangement of spectral data that breaks the symmetry required by the Riemann Hypothesis. Such configurations may arise hypothetically from asymmetries in spectral localization or from imbalance in arithmetic trace contributions.

The purpose of this chapter is to explain why such configurations are structurally obstructed within the SMRK framework.

6.2 Hypothetical Deformations of the Spectrum

Assume, for the sake of argument, that the spectrum of the SMRK Hamiltonian admits a deformation that breaks critical symmetry while preserving basic analytic features.

Such a deformation would correspond to shifting spectral mass in a way that introduces asymmetry without destroying discreteness or boundedness.

6.3 Breakdown of Trace Positivity

Any asymmetric spectral deformation immediately impacts trace objects. Since trace kernels test the spectrum globally, asymmetry produces negative contributions for appropriately chosen test functions.

This violates the trace positivity established in the previous chapter.

6.4 Incompatibility with Unitary Equivalence

Non-critical spectral configurations cannot be related to critical ones by unitary conjugation. Consequently, they represent genuinely distinct operator classes.

Because all admissible arithmetic Hamiltonians must belong to a fixed unitary equivalence class, such configurations are excluded.

6.5 Failure of Cancellation Mechanisms

Asymmetric spectral arrangements correspond, on the arithmetic side, to failure of prime-power cancellation. This manifests as systematic bias in arithmetic expansions that cannot be compensated by finite adjustments.

The failure persists under smoothing and truncation, indicating a fundamental structural defect.

6.6 Absence of Stable Perturbative Paths

There exists no perturbative path connecting a critical configuration to a non-critical one while preserving self-adjointness, compactness, and trace positivity.

Any attempted interpolation necessarily crosses a boundary where one of these properties fails.

6.7 Renormalization Does Not Restore Admissibility

Renormalization affects only scalar components of the Hamiltonian. It cannot repair structural defects caused by spectral asymmetry.

Thus non-critical configurations remain inadmissible even after renormalization.

6.8 Global Versus Local Obstructions

Local deviations at finitely many primes correspond to finite-rank perturbations and do not alter global spectral structure. In contrast, non-critical spectral configurations require global deformation across infinitely many arithmetic scales.

This distinction explains why localized anomalies do not threaten the SMRK framework, whereas global asymmetry does.

6.9 Interpretation in Programmatic Terms

From a programmatic perspective, this chapter establishes an exclusion principle: any candidate spectral configuration incompatible with trace positivity and symmetry is ruled out a priori.

This shifts the burden of proof away from checking individual configurations and toward establishing the structural properties of the operator.

6.10 Implications for Counterexample Strategies

Any attempt to construct a counterexample to the Riemann Hypothesis within a spectral framework must therefore:

- violate self-adjointness,
- abandon compactness,
- or sacrifice trace positivity.

Such attempts fall outside the SMRK program.

6.11 Summary

In this chapter we have:

- defined non-critical spectral configurations,
- examined hypothetical spectral deformations,
- shown their incompatibility with trace positivity,
- explained the absence of admissible deformation paths,
- established an operator-level exclusion principle.

The next chapter explains how explicit formulae function as enforcement mechanisms that translate these structural obstructions into spectral constraints.

7 From Explicit Formulae to Spectral Enforcement

7.1 Explicit Formulae as Structural Interfaces

Explicit formulae occupy a central position in analytic number theory as identities relating spectral data to arithmetic sums over primes. Within the SMRK framework, explicit formulae are reinterpreted as trace identities arising from functional calculus applied to a self-adjoint arithmetic Hamiltonian.

This reinterpretation elevates explicit formulae from computational tools to structural interfaces that enforce admissible spectral behavior.

7.2 Trace Identities and Functional Calculus

Given a self-adjoint Hamiltonian $\tilde{H}$ and a suitable test function f , functional calculus yields an operator $f(\tilde{H})$ whose trace admits both a spectral and an arithmetic expansion.

The equality of these expansions is not an assumption but a consequence of operator theory. Explicit formulae therefore inherit the rigidity and positivity constraints of trace objects.

7.3 Test Functions as Probes

The choice of test function f determines which aspects of the spectrum are probed. Smooth, compactly supported, or rapidly decaying functions allow selective emphasis on specific spectral scales while preserving positivity.

Within the SMRK program, admissible test functions form a controlled class designed to prevent artificial violations of trace positivity.

7.4 Arithmetic Side as a Constraint, Not an Input

In traditional settings, explicit formulae are often used to extract information about primes from spectral assumptions. In the SMRK framework, the direction is reversed: arithmetic contributions constrain the spectrum through trace identities.

Prime-power terms enter with fixed weights determined by local operator data. They cannot be tuned independently to accommodate arbitrary spectral configurations.

7.5 Enforcement Through Positivity

Trace positivity converts explicit formulae into enforcement mechanisms. If a spectral configuration were incompatible with required arithmetic cancellation, there would exist a test function for which the trace becomes negative.

Because such negativity is forbidden, incompatible spectra are excluded.

7.6 Universality of the Enforcement Mechanism

The enforcement role of explicit formulae does not depend on fine arithmetic details. It relies only on:

- self-adjointness,
- compact resolvent,

- positivity of trace kernels,
- boundedness of local arithmetic data.

This universality explains why similar explicit-formula structures appear across different classes of L -functions.

7.7 Comparison with Analytic Continuation

Classical approaches often rely on analytic continuation to extend explicit formulae beyond their region of convergence. In contrast, the SMRK framework replaces analytic continuation by functional calculus on a self-adjoint operator.

This shift removes ambiguity and places explicit formulae on firm operator-theoretic ground.

7.8 Spectral Symmetry as an Outcome

Within this perspective, spectral symmetry is not postulated but enforced. Any asymmetry incompatible with explicit-formula trace identities is ruled out by positivity and rigidity.

The Riemann Hypothesis thus emerges as the statement that the enforced spectral configuration is globally symmetric.

7.9 Stability Under Approximation

Finite truncations of primes or arithmetic domain produce approximate explicit formulae. While truncation introduces errors, the enforcement mechanism remains visible in the stability of low-lying spectral data.

This stability provides practical support for numerical exploration without elevating numerical results to the status of proof.

7.10 Programmatic Significance

By reinterpreting explicit formulae as enforcement mechanisms, the SMRK program shifts the focus of the Riemann Hypothesis from verification to inevitability. The hypothesis becomes the only configuration compatible with the full set of operator constraints.

7.11 Summary

In this chapter we have:

- reframed explicit formulae as trace identities,
- explained the role of functional calculus,
- described test functions as spectral probes,
- shown how positivity enforces admissible spectra,
- contrasted operator enforcement with analytic continuation,
- interpreted the Riemann Hypothesis as an enforced outcome.

The next chapter examines universality across families of L -functions and explains why the enforcement mechanism transcends individual cases.

8 Universality Across Families of L -Functions

8.1 Why Universality Matters

A credible structural approach to the Riemann Hypothesis must not depend on accidental features of a single arithmetic object. Instead, it must exhibit universality across broad classes of L -functions, reflecting common underlying mechanisms rather than case-specific coincidences.

Within the SMRK program, universality is a consequence of operator architecture rather than of shared analytic properties.

8.2 Common Operator Backbone

All L -functions treated in the SMRK framework share the same global operator backbone:

- arithmetic Hilbert space organized by multiplicative structure,
- prime-generated dynamics via shift operators,
- logarithmic confinement ensuring compact resolvent,
- renormalized self-adjoint realization.

Differences between families appear only through local data decorating the prime shifts.

8.3 Local Data as Controlled Variations

Dirichlet characters, automorphic representations, and higher-degree L -functions introduce local matrices A_p acting on internal spaces. These variations modify prime-power weights but do not alter the global structure of the Hamiltonian.

As a result, universality is preserved at the operator level even as arithmetic detail varies.

8.4 Trace Structure Across Families

Trace objects retain the same formal structure across all families:

$$\mathrm{Tr} f(\tilde{H}_L) = \mathcal{B}_L(f) + \sum_p \sum_{m \geq 1} \Lambda_L(p^m) \hat{f}(m \log p). \quad (13.4)$$

Only the coefficients $\Lambda_L(p^m)$ depend on the specific L -function. The functional form of the trace identity is universal.

8.5 Universality of Positivity Constraints

Positivity of trace kernels imposes identical constraints across families. Regardless of degree or conductor, any admissible L -function must satisfy the same global positivity conditions.

This explains why the Riemann Hypothesis and its generalizations appear as a single phenomenon rather than as independent conjectures.

8.6 Families and Deformation Stability

Continuous variation within families—such as twisting by characters or changing weights—induces continuous changes in local data. These variations correspond to bounded perturbations of the

Hamiltonian and preserve spectral admissibility.

The absence of admissible paths to non-critical configurations persists uniformly across families.

8.7 Ramification as a Finite-Rank Effect

Ramified primes differ between families, but they contribute only finite-rank perturbations to the Hamiltonian. Such perturbations do not affect compactness, self-adjointness, or global spectral symmetry.

This explains why universality is not disrupted by local arithmetic complexity.

8.8 Comparison with Random Matrix Heuristics

Random matrix models capture certain universal statistical features of zeros of L -functions. In contrast, the SMRK framework derives universality from deterministic operator constraints.

The two perspectives are complementary: random matrix statistics may emerge as a secondary consequence of the rigid operator structure.

8.9 Implications for Generalized Riemann Hypotheses

The SMRK program suggests that generalized Riemann Hypotheses for families of L -functions are not independent conjectures. They reflect the same underlying operator-level enforcement applied to different local data.

This unification reduces the conceptual complexity of the problem.

8.10 Programmatic Consequences

Universality shifts the emphasis from individual proofs to structural analysis. Once the operator framework is established, the same enforcement mechanisms apply uniformly across families.

This perspective clarifies why progress on one class of L -functions often informs others.

8.11 Summary

In this chapter we have:

- identified a common operator backbone across L -function families,
- described local data as controlled variations,
- demonstrated universality of trace identities,
- established family-independent positivity constraints,
- explained stability under deformation and ramification,
- interpreted generalized RH as a single structural phenomenon.

The next chapter discusses the role of numerical evidence as structural validation within the SMRK program.

9 Numerical Evidence as Structural Validation

9.1 Role of Numerics in the SMRK Program

Numerical investigations play a carefully delimited role within the SMRK program. They are not treated as sources of proof, but as instruments for validating structural claims derived from operator theory.

In particular, numerics serve to test stability, consistency, and universality of the spectral framework under controlled approximations.

9.2 Distinction Between Proof and Validation

A fundamental distinction must be maintained between mathematical proof and numerical evidence. Proof establishes necessity, whereas numerics provide empirical support that a proposed structure behaves as expected.

The SMRK program relies on numerics only in the latter sense. Numerical results are interpreted as confirmation that operator-level constraints are not artifacts of idealized constructions.

9.3 Finite Truncations as Experimental Proxies

All numerical implementations of the SMRK Hamiltonian necessarily involve truncations:

- restriction to finitely many primes,
- restriction to a finite arithmetic domain,
- finite-dimensional internal spaces.

These truncations define experimental proxies for the infinite operator. Structural features that persist across increasing truncation scales are interpreted as robust.

9.4 Stability of Low-Lying Spectrum

One of the most informative numerical indicators is the stability of low-lying eigenvalues under increasing truncation parameters. Persistent eigenvalue patterns suggest that the underlying infinite operator has a well-defined discrete spectrum.

Instabilities, by contrast, signal either insufficient truncation or structural incompatibility.

9.5 Trace Objects and Smoothing

Numerical evaluation of trace objects, such as heat kernel traces, provides a smoothed view of the spectrum that suppresses truncation noise. These traces are particularly well suited for numerical study because they emphasize global spectral features rather than individual eigenvalues.

Consistency of trace behavior across truncation scales supports the validity of the operator framework.

9.6 Detection of Arithmetic Signatures

Even under heavy truncation, numerical trace objects exhibit oscillatory components associated with prime powers. The appearance of these signatures confirms that arithmetic information is

encoded at the operator level rather than inserted artificially.

The persistence of such signatures across different L -function families further supports universality claims.

9.7 Numerics and Positivity Constraints

Numerical experiments also probe trace positivity. While finite truncations may introduce small numerical artifacts, systematic violation of positivity would indicate a breakdown of the operator framework.

Observed numerical behavior is consistent with positivity constraints predicted by theory.

9.8 Limits of Numerical Resolution

Numerical studies are inherently limited in their ability to access asymptotic regimes. Fine spectral correlations and extreme arithmetic scales remain beyond reach.

Accordingly, numerical evidence is interpreted qualitatively, with emphasis on structural trends rather than precise quantitative agreement.

9.9 Numerics as Falsifiability Tools

A strength of the SMRK program is that it is falsifiable. Numerical experiments can, in principle, expose structural inconsistencies by revealing instability, loss of positivity, or breakdown of universality.

That no such inconsistencies have been observed under systematic numerical probing provides indirect support for the framework.

9.10 Relation to Other Numerical Approaches

Classical numerical studies of zeros of L -functions focus on verifying hypotheses for finite ranges. In contrast, SMRK numerics investigate the behavior of an operator whose spectrum is conjectured to encode the same structural constraints.

The two approaches address complementary aspects of the same problem.

9.11 Interpretive Caution

Numerical agreement should not be mistaken for proof, nor should numerical anomalies be overinterpreted. The SMRK program emphasizes conceptual clarity and structural necessity over numerical confirmation.

This caution preserves the logical integrity of the framework.

9.12 Summary

In this chapter we have:

- clarified the role of numerical evidence as validation rather than proof,
- described truncation-based numerical proxies,
- discussed stability of low-lying spectral data,

- analyzed numerical trace objects and arithmetic signatures,
- emphasized positivity and universality checks,
- articulated limits and interpretive principles for numerics.

The next chapter identifies the precise conditions that a proof of the Riemann Hypothesis must ultimately establish within an operator-based framework.

10 What a Proof of the Riemann Hypothesis Must Ultimately Establish

10.1 From Verification to Necessity

A proof of the Riemann Hypothesis must go beyond verification of individual cases or asymptotic regimes. Within the SMRK program, proof is understood as establishing necessity: the impossibility of any alternative spectral configuration compatible with the full set of operator-level constraints.

The question is therefore not whether the critical configuration occurs, but whether any other configuration can exist.

10.2 Existence of a Canonical Self-Adjoint Operator

The first requirement is the existence of a canonically defined self-adjoint operator encoding arithmetic dynamics. This operator must:

- arise intrinsically from prime structure,
- admit a renormalized self-adjoint realization,
- possess compact resolvent.

Without such an operator, spectral language remains metaphorical rather than rigorous.

10.3 Uniqueness up to Unitary Equivalence

A viable proof must establish that the relevant operator is unique up to unitary equivalence. Apparent freedom in construction must reduce to gauge choices or finite-rank modifications that do not alter spectral invariants.

This uniqueness ensures that spectral conclusions are not artifacts of representation.

10.4 Enforcement of Spectral Symmetry

The proof must demonstrate that spectral symmetry is enforced by the operator itself. Symmetry should follow from:

- intrinsic operator symmetries,
- trace positivity,
- rigidity under admissible perturbations.

Symmetry assumed as an external condition does not constitute proof.

10.5 Positivity of All Admissible Trace Objects

A decisive element is the global positivity of all admissible trace kernels derived from the operator. This includes traces associated with a broad class of nonnegative test functions.

Any hypothetical counterexample must produce a violation of positivity for some trace object. Demonstrating the impossibility of such violations is therefore central.

10.6 Control of Arithmetic Cancellation

The proof must account for arithmetic cancellation at all relevant scales. This requires showing that prime-power contributions necessarily cancel in a manner compatible with trace positivity and spectral symmetry.

Partial or scale-dependent cancellation is insufficient.

10.7 Stability Under Renormalization and Limits

All arguments must remain valid under renormalization and in the infinite prime limit. Dependence on finite cutoffs or auxiliary parameters undermines necessity.

A proof must therefore be robust under all limiting procedures intrinsic to the framework.

10.8 Exclusion of Non-Critical Configurations

The final step is an explicit exclusion principle: a demonstration that any non-critical spectral configuration is incompatible with at least one essential operator-level requirement.

This exclusion must be global and structural rather than case-by-case.

10.9 Role of Numerical Evidence

Numerical evidence plays no direct role in proof, but it can inform intuition and suggest which structural properties are decisive. A proof must stand independently of numerical confirmation.

The SMRK program treats numerics as supportive but logically nonessential.

10.10 Minimality of Assumptions

An effective proof should rely on minimal assumptions. Each assumption must be shown to be unavoidable within the operator-based perspective.

Redundant or ad hoc assumptions weaken the structural force of the argument.

10.11 Programmatic Synthesis

Taken together, these requirements form a coherent checklist. Any approach claiming to resolve the Riemann Hypothesis must address each item explicitly or implicitly.

The SMRK program provides a framework in which all these requirements can be formulated precisely and examined systematically.

10.12 Summary

In this chapter we have:

- reframed proof as establishment of necessity,
- identified essential operator-level requirements,
- emphasized uniqueness and symmetry enforcement,
- highlighted the role of trace positivity and cancellation,
- articulated robustness under renormalization,
- formulated a global exclusion principle.

The final chapter concludes the document by synthesizing these insights and outlining the remaining path toward a spectral resolution of the Riemann Hypothesis.

11 Conclusion: Toward a Spectral Resolution of the Riemann Hypothesis

11.1 From Construction to Program

The SMRK program began with the construction of a concrete arithmetic Hamiltonian and its extension to general classes of L -functions. In this document, the focus has shifted from construction to synthesis: from defining objects to understanding the structural consequences of their existence.

This transition reflects a maturation of the framework from technical development to programmatic clarity.

11.2 Structural Interpretation of the Riemann Hypothesis

Within the SMRK perspective, the Riemann Hypothesis is not viewed as an isolated analytic statement but as a global structural constraint imposed by self-adjointness, symmetry, and positivity.

The hypothesis becomes a statement about what spectral configurations are compatible with arithmetic dynamics governed by prime-generated operators.

11.3 Role of Operator-Level Constraints

Operator-level constraints—self-adjointness, compactness, unitary invariance, and trace positivity—form the backbone of the SMRK program. These constraints are not adjustable; they arise naturally from the requirement that arithmetic information be encoded coherently at the operator level.

Their combined effect severely restricts admissible spectral behavior.

11.4 Explicit Formulae as Enforcement Mechanisms

Explicit formulae, traditionally viewed as analytic identities, are reinterpreted here as enforcement mechanisms. Through trace identities and positivity constraints, they convert arithmetic structure into spectral necessity.

This reinterpretation is a central conceptual shift of the SMRK program.

11.5 Universality and Robustness

A striking feature of the framework is its universality across families of L -functions. Variations in local arithmetic data do not alter the global enforcement mechanism, highlighting the robustness of the operator architecture.

This universality supports the view that the Riemann Hypothesis and its generalizations are manifestations of a single underlying principle.

11.6 Limits of the Present Work

This document does not present a proof of the Riemann Hypothesis. Instead, it clarifies the structural landscape in which such a proof must reside.

By identifying necessary operator-level conditions and exclusion principles, the SMRK program narrows the space of viable approaches.

11.7 Toward a Proof

A spectral resolution of the Riemann Hypothesis requires demonstrating that the SMRK Hamiltonian satisfies all enforcement criteria globally and without exception.

The work presented here lays out the conceptual and structural components required for such a demonstration, providing a roadmap rather than a conclusion.

11.8 Open Directions

Several directions remain open within the SMRK program:

- refinement of operator-level positivity arguments,
- deeper analysis of symmetry and rigidity mechanisms,
- extension of numerical validation to broader families,
- exploration of connections with other spectral programs.

These directions indicate where further progress is both possible and meaningful.

11.9 Final Perspective

The SMRK program proposes that the Riemann Hypothesis is best understood not as a mysterious property of a special function, but as a necessary outcome of arithmetic dynamics encoded in a self-adjoint operator framework.

Whether this perspective ultimately yields a complete proof remains to be seen. What it already provides is a coherent, rigorous, and falsifiable path toward a spectral resolution of one of mathematics' most enduring problems.

APPENDIX A

A.1 Conceptual Map of the SMRK Program

A.1.1 Purpose of the Conceptual Map

This appendix provides a high-level conceptual map of the SMRK program. Its purpose is to clarify the logical dependencies between constructions, structural principles, and programmatic conclusions developed across the SMRK documents.

The appendix is explanatory rather than technical and is intended to support global orientation.

A.1.2 Foundational Layer

The foundational layer of the SMRK program consists of:

- arithmetic Hilbert spaces organized by multiplicative structure,
- prime-shift operators generating arithmetic dynamics,
- diagonal logarithmic potentials providing confinement,
- renormalized self-adjoint Hamiltonians.

These elements establish the basic operator-theoretic setting.

A.1.3 Spectral Layer

Building on the foundational layer, the spectral layer introduces:

- compact resolvent and discrete spectrum,
- logarithmic localization of eigenfunctions,
- quadratic spectral growth,
- stability under finite-rank perturbations.

This layer ensures that arithmetic information is encoded in tractable spectral data.

A.1.4 Trace and Explicit Formula Layer

The trace layer connects spectral and arithmetic structures through:

- heat kernel traces,
- resolvent-based traces,
- weighted trace identities,
- generalized von Mangoldt functions.

Explicit formulae emerge at this level as operator-theoretic identities.

A.1.5 Symmetry and Rigidity Layer

At the structural level, the framework incorporates:

- logarithmic inversion symmetry,
- local gauge invariance of arithmetic data,
- rigidity induced by trace identities,
- controlled symmetry breaking via confinement.

These features restrict admissible operator deformations.

A.1.6 Enforcement Layer

The enforcement layer translates structural properties into constraints:

- trace positivity,
- arithmetic cancellation mechanisms,
- exclusion of non-critical spectral configurations.

At this level, the Riemann Hypothesis appears as a necessary condition.

A.1.7 Universality Layer

Universality across families of L -functions arises from:

- a shared operator backbone,
- controlled variation of local data,
- family-independent positivity constraints.

This layer unifies classical and generalized Riemann Hypotheses.

A.1.8 Programmatic Layer

Finally, the programmatic layer addresses:

- criteria for a complete proof,
- falsifiability and failure modes,
- open research directions.

This layer situates the SMRK framework within broader mathematical research.

A.1.9 Summary

This appendix has:

- outlined the logical structure of the SMRK program,
- identified distinct conceptual layers,
- clarified dependencies between constructions and conclusions,

- provided a global orientation for the reader.

APPENDIX B

B.1 Relation to Hilbert–Pólya and Other Spectral Programs

B.1.1 Historical Context

The Hilbert–Pólya idea proposes that the nontrivial zeros of the Riemann zeta function arise as spectral data of a self-adjoint operator. While no explicit operator was identified in the original proposal, the idea established a lasting connection between number theory and spectral theory. The SMRK program may be viewed as a concrete realization of this philosophical direction, formulated in a modern operator-theoretic language.

B.1.2 Hilbert–Pólya as a Guiding Principle

Within the SMRK framework, the Hilbert–Pólya idea is interpreted not as a literal spectral identification of zeros, but as a requirement that arithmetic information be encoded in a self-adjoint operator whose spectral properties reflect deep arithmetic constraints.

The emphasis is on structural enforcement rather than direct correspondence.

B.1.3 Differences in Operator Construction

Traditional Hilbert–Pólya approaches often seek an operator whose spectrum coincides explicitly with the imaginary parts of zeta zeros. The SMRK program departs from this goal in several key ways:

- the operator is constructed from prime-generated dynamics rather than analytic continuation,
- spectral parameters are not identified with zeros directly,
- enforcement of symmetry arises through trace positivity rather than spectral matching.

These differences shift the focus from identification to necessity.

B.1.4 Role of Self-Adjointness

Both the Hilbert–Pólya idea and the SMRK program place self-adjointness at the center. In the SMRK framework, self-adjointness is achieved through renormalized quadratic forms and logarithmic confinement, ensuring mathematical rigor.

This provides a concrete realization of a key Hilbert–Pólya requirement.

B.1.5 Comparison with Connes’ Spectral Approach

Noncommutative geometric approaches, notably those developed by Connes, interpret the explicit formula as a trace formula on a spectral triple. These approaches share with SMRK an emphasis on operator theory and trace identities.

The SMRK program differs by:

- avoiding reliance on noncommutative geometry,

- constructing an explicit arithmetic Hamiltonian,
- emphasizing positivity and enforcement mechanisms.

The two perspectives are complementary rather than contradictory.

B.1.6 Relation to Quantum Chaos Models

Models inspired by quantum chaos and random matrix theory propose statistical correspondence between zeros of L -functions and spectra of random operators. The SMRK program does not posit randomness at the operator level.

Instead, any random-matrix-like statistics are viewed as emergent phenomena arising from rigid deterministic operator constraints.

B.1.7 Comparison with Trace Formula Programs

Classical trace formulae relate spectral data of automorphic operators to sums over geometric or arithmetic objects. The SMRK program parallels this structure but operates in a purely arithmetic setting, with primes generating the dynamics.

This highlights a shared structural theme while maintaining a distinct framework.

B.1.8 Advantages of the SMRK Perspective

The SMRK program offers several advantages over earlier spectral approaches:

- explicit self-adjoint operator construction,
- built-in renormalization handling infinite prime sums,
- universal applicability across L -function families,
- clear falsifiability criteria.

These features strengthen the programmatic value of the framework.

B.1.9 Limitations and Open Comparisons

Despite its strengths, the SMRK program does not subsume all spectral approaches. Connections to noncommutative geometry, quantum chaos, and classical trace formulae remain to be explored in detail.

These connections represent opportunities rather than deficiencies.

B.1.10 Interpretive Summary

The SMRK program can be viewed as a modern, operator-theoretic descendant of the Hilbert–Pólya idea. It preserves the core insight—that arithmetic regularity should have a spectral origin—while replacing speculative identification with structural enforcement.

B.1.11 Summary

This appendix has:

- placed the SMRK program in the context of Hilbert–Pólya,
- clarified similarities and differences with other spectral approaches,
- emphasized the role of self-adjointness and trace identities,
- highlighted the distinctive features of the SMRK framework.

APPENDIX C

C.1 Falsifiability and Failure Modes

C.1.1 Falsifiability as a Programmatic Requirement

A distinguishing feature of the SMRK program is its explicit commitment to falsifiability. The framework is constructed so that its core claims impose concrete, testable structural constraints at the operator level.

If these constraints fail, the program is not weakened but invalidated. This distinguishes the SMRK approach from heuristic or purely interpretive frameworks.

C.1.2 What Would Constitute Failure

Failure of the SMRK program would occur if any of the following were demonstrated:

- nonexistence of a renormalized self-adjoint Hamiltonian consistent with arithmetic dynamics,
- unavoidable loss of compact resolvent in the infinite prime limit,
- violation of trace positivity for an admissible class of test functions,
- existence of a stable non-critical spectral configuration compatible with all operator constraints.

Each item represents a sharp structural criterion rather than a numerical anomaly.

C.1.3 Operator-Level Failure Modes

At the operator level, failure may arise through:

- breakdown of self-adjointness under renormalization,
- uncontrolled growth of off-diagonal prime-shift terms,
- loss of confinement in the logarithmic variable,
- emergence of continuous spectrum incompatible with trace analysis.

Such failures would undermine the spectral interpretation of arithmetic dynamics.

C.1.4 Trace-Level Failure Modes

Trace identities provide a sensitive diagnostic. Potential failure modes include:

- existence of a nonnegative test function yielding a negative trace,
- instability of trace values under smoothing,
- inconsistency between spectral and arithmetic expansions.

Any of these outcomes would signal incompatibility with the SMRK framework.

C.1.5 Failure of Arithmetic Cancellation

The SMRK program relies on global arithmetic cancellation enforced by trace positivity. Failure would manifest as persistent imbalance in prime-power contributions that cannot be compensated by spectral symmetry.

Such imbalance would indicate that arithmetic dynamics cannot be encoded in a single self-adjoint operator.

C.1.6 Robustness Against Finite Perturbations

Finite-rank perturbations arising from ramified primes or local modifications are explicitly allowed. Failure occurs only if global structural properties are affected.

This distinction ensures that falsifiability targets genuine structural defects rather than technical irregularities.

C.1.7 Numerical Signals of Failure

While numerical evidence cannot prove validity, it can suggest failure. Warning signs include:

- instability of low-lying spectrum under increasing truncation,
- systematic violation of approximate trace positivity,
- loss of universality across families.

Such signals would motivate closer analytical scrutiny.

C.1.8 Conceptual Failure Modes

Beyond technical issues, conceptual failure could arise if:

- operator-level constraints fail to capture known arithmetic phenomena,
- enforcement mechanisms prove too weak to exclude non-critical configurations,
- the framework requires ad hoc assumptions lacking intrinsic justification.

These failures would call into question the foundational philosophy of the program.

C.1.9 Distinction from Proof Failure

It is important to distinguish failure of the SMRK program from failure to complete a proof. The former implies that the structural approach itself is flawed; the latter may reflect incomplete development or missing techniques.

The SMRK program is designed so that genuine failure can be clearly identified.

C.1.10 Value of Falsifiability

Falsifiability strengthens the program by clarifying its logical status. A framework that cannot fail cannot meaningfully succeed.

By articulating precise failure modes, the SMRK program invites rigorous scrutiny and engagement from the broader mathematical community.

C.1.11 Summary

This appendix has:

- defined falsifiability criteria for the SMRK program,
- identified operator-level and trace-level failure modes,
- distinguished local perturbations from global defects,
- clarified the role of numerical signals,
- emphasized the conceptual value of explicit failure conditions.

APPENDIX D

D.1 Open Problems and Research Directions

D.1.1 Purpose of This Appendix

This appendix collects open problems and research directions that naturally arise from the SMRK program. The emphasis is not on conjectural claims, but on concrete questions whose resolution would clarify, strengthen, or delimit the operator-based approach to the Riemann Hypothesis.

The problems listed here are intended to be accessible entry points for further work, both analytical and numerical.

D.1.2 Global Positivity Proofs

A central open problem is the establishment of global trace positivity for the full class of admissible test functions. While positivity follows formally from self-adjointness, a detailed characterization of admissible test classes remains open.

Clarifying this issue would significantly strengthen the enforcement mechanism at the heart of the SMRK program.

D.1.3 Rigidity Beyond Symmetry

The rigidity arguments presented rely on symmetry and trace identities. An important direction is to identify additional rigidity mechanisms that operate independently of explicit symmetry assumptions.

Such mechanisms could further restrict admissible operator deformations and reduce the conceptual distance to a complete proof.

D.1.4 Quantitative Bounds on Cancellation

While the framework establishes the necessity of arithmetic cancellation, quantitative bounds on the rate and uniformity of cancellation remain to be developed.

Obtaining such bounds at the operator level could bridge the gap between qualitative structural arguments and quantitative analytic results.

D.1.5 Operator Uniqueness

Although uniqueness up to unitary equivalence is strongly suggested, a fully explicit proof of uniqueness for the SMRK Hamiltonian remains open.

Such a result would eliminate residual ambiguity in operator construction and further solidify the spectral interpretation.

D.1.6 Extension to Broader Arithmetic Settings

An important direction is the extension of the SMRK framework beyond classical L -functions. Possible settings include:

- motivic L -functions,
- p -adic or hybrid arithmetic structures,
- arithmetic objects with incomplete Euler products.

Exploring these extensions would test the universality of the operator architecture.

D.1.7 Refinement of Numerical Experiments

Numerical investigations can be refined by:

- increasing truncation scales,
- improving stability diagnostics,
- exploring family-level statistics more systematically.

Such refinements may reveal additional structural regularities or expose subtle failure modes.

D.1.8 Connections to Other Spectral Frameworks

Further work is needed to clarify precise connections between the SMRK program and other spectral approaches, including:

- noncommutative geometry,
- adelic trace formulae,
- quantum chaos models.

Establishing rigorous bridges may lead to transfer of techniques and insights.

D.1.9 Formalization of Enforcement Arguments

Several enforcement arguments are currently formulated at a conceptual level. A systematic formalization of these arguments within functional analysis would enhance their mathematical precision.

This includes explicit statements of assumptions and theorems governing enforcement.

D.1.10 Minimal Assumption Analysis

Another open direction is the identification of minimal assumptions required for the SMRK framework. Determining which assumptions are essential and which can be relaxed would sharpen the conceptual core of the program.

D.1.11 Community Engagement and Independent Verification

The long-term success of the SMRK program depends on independent verification and engagement. Open problems should be addressed by multiple approaches and scrutinized from different mathematical perspectives.

Such engagement is essential for assessing the ultimate viability of the framework.

D.1.12 Summary

This appendix has:

- identified key open problems within the SMRK program,
- outlined analytical and numerical research directions,
- emphasized issues of positivity, rigidity, and uniqueness,
- highlighted opportunities for extension and cross-framework connections.

APPENDIX E

E.1 Research Roadmap 2026–2029

This appendix outlines a proposed roadmap for the continuation of the research program initiated in this work. The roadmap is intentionally conservative and long-term. It reflects the belief that progress toward the Riemann Hypothesis (RH), at the level described in this framework, requires not urgency but stability, openness, and careful preparation.

The timeline begins in 2026, following a deliberate pause. This pause is not a withdrawal, but a consolidation phase: allowing ideas to settle, the document to circulate, and the broader ecosystem—particularly the application layer of the Quantum Virtual Machine (QVM)—to mature.

E.1.1 Guiding Principles

The roadmap is governed by the following principles:

- No forced acceleration; progress must be sustainable.
- Public engagement precedes technical expansion.
- Conceptual clarity precedes computation.
- Computational experimentation precedes formal proof.
- Collaboration is welcomed, but coherence is preserved.

E.1.2 Year 2026: Consolidation and Public Engagement

Q1 2026 — Public Release and Outreach

- Public release of the whitepaper and supporting materials.
- Creation of a dedicated project page describing the operator-based framework in accessible terms.
- Initial outreach to mathematicians, physicists, and computational researchers interested in spectral approaches to RH.
- Explicit positioning of the work as a long-term, open research program, not a proof claim.

Q2 2026 — Deep Re-study and Internal Review

- Careful re-reading and internal critique of the entire framework.
- Identification of weak points, unclear assumptions, and areas requiring clarification.
- Parallel study of related operator-theoretic and spectral literature.
- No new construction; focus on understanding and stress-testing ideas.

Q3 2026 — Community Feedback and Dialogue

- Collection and synthesis of feedback from external readers.
- Public or semi-public discussions (workshops, informal seminars, online forums).
- Refinement of terminology and presentation based on feedback.
- Explicit documentation of open questions and objections.

Q4 2026 — Preparation for Computational Phase

- Formal specification of operator constructions in a computationally implementable form.
- Definition of finite-scale truncation and renormalization protocols.
- Close monitoring of the development of the QVM application layer.
- Decision point: readiness to transition toward experimentation.

E.1.3 Year 2027: Transition to Operator Experimentation

Q1 2027 — Initial QVM Integration

- First implementation of simplified arithmetic Hamiltonians on QVM.
- Verification of symmetry preservation at finite scales.
- Exploration of basic spectral properties under truncation.

Q2 2027 — Renormalization Flow Experiments

- Systematic study of scale dependence and convergence.
- Identification of candidate fixed-point behavior.
- Documentation of instabilities and failure modes.

Q3 2027 — Spectral Statistics and Robustness

- Analysis of spectral spacing statistics and universality.
- Testing robustness under admissible perturbations.
- Comparison with known properties of zeta zero statistics.

Q4 2027 — Synthesis and Reassessment

- Evaluation of experimental results against theoretical expectations.
- Revision of the operator framework if required.
- Clear decision on whether the approach remains viable.

E.1.4 Year 2028: Toward Analytical Closure

Q1 2028 — Refinement of Analytic Targets

- Identification of precise analytic statements suggested by computation.
- Selection of the most promising operator realizations.

Q2 2028 — Technical Deepening

- Work on compactness, trace-class, and convergence proofs guided by experimental insight.
- Close interaction between computation and analysis.

Q3 2028 — Partial Theorems and Intermediate Results

- Formalization of partial results independent of full RH.
- Publication of intermediate theorems where appropriate.

Q4 2028 — Global Consistency Check

- Verification that all components of the framework remain coherent.
- Assessment of whether remaining gaps are technical or structural.

E.1.5 Year 2029: Resolution Phase

Q1 2029 — Final Analytical Push

- Concentrated effort on remaining technical obstacles.
- Preparation of proof-level arguments if feasible.

Q2 2029 — Outcome Assessment

- Clear determination of success, partial success, or failure.
- Transparent documentation of results and limitations.

Q3 2029 — Dissemination

- Publication of final results or conclusions.
- Release of computational tools and data.

Q4 2029 — Reflection and Transition

- Evaluation of broader impact beyond RH.
- Decision on continuation, redirection, or closure of the program.

E.1.6 Closing Perspective

This roadmap deliberately avoids guarantees. It is designed to maximize understanding rather than to force outcomes. By pacing the work, engaging the community, and aligning theory with computation, it aims to give the problem of the Riemann Hypothesis the time and structure it demands.

Progress toward RH, if it comes, will come not from haste, but from coherence.

VOL. NO. III

Spectral Enforcement of the Riemann Hypothesis

Arithmetic Hamiltonians, Spectral Constraints, and Prime Dynamics

14	Golden-Phase Prime-Shift Operators	
	Gauge-Twisted Arithmetic Dynamics and Spectral Probes of Prime Structure	346
15	Self-Adjoint Prime-Shift Operators	
	Quadratic Forms, Regularization and Arithmetic Dynamics	367
16	Trace Formulas for Prime-Shift Operators	
	Cycles, Resolvents, and Arithmetic Expansions	381
17	Spectral Identification and the Riemann Hypothesis	
	Determinants, Spectral Zeta Functions and Self-Adjoint Arithmetic Hamiltonians	400
18	An Experimental Operator Framework for the Riemann Hypothesis	
	Certified Zero Pipelines, Explicit Formula Residua, and Operator Scoring	424
19	Numerical Results for the SMRK Hamiltonian	
	Spectral Matching, Explicit Formula Residuals, and Stability Analysis	447
20	Self-Adjoint Operator Framework	
	Toward an Operator-Theoretic Realization of the Hilbert–Pólya Program	467
21	The SMRK Probe	
	Spectral Measurement Instrument for Arithmetic Hamiltonians	484

Golden-Phase Prime-Shift Operators

Gauge-Twisted Arithmetic Dynamics and Spectral Probes of Prime Structure

Enter Yourname

Abstract

This document introduces a class of arithmetic prime-shift operators acting on the space of square-summable sequences over the natural numbers. The operators implement purely multiplicative dynamics, where arithmetic states supported at a given index are mapped to states supported at prime multiples or divisors of that index. To this basic structure, nontrivial phase factors are added as a controlled gauge-like modification.

The role of these phase factors is diagnostic rather than decorative. They provide a systematic way to distinguish intrinsic spectral features of the prime-shift construction from artifacts caused by truncation, regularization, or numerical implementation. A distinguished benchmark choice, referred to as the golden phase, is examined in detail due to its strong irrationality properties, while the framework itself applies to general irrational phase choices.

The document develops both the infinite-dimensional formulation and finite approximations suitable for computation. It analyzes how spectral observables respond to variations of the phase parameter and to the inclusion of diagonal arithmetic potentials. In this way, the work establishes a bridge between the bare prime-shift mechanism and later stages of the program involving trace structures, spectral diagnostics, and experimental comparison of arithmetic Hamiltonians.

1 Introduction and Strategy

The purpose of this paper is to place prime-shift operators into a rigorous self-adjoint framework suitable for spectral analysis. While operators encoding multiplicative arithmetic dynamics have appeared in various formal or heuristic contexts, their analytic foundations are often left implicit. In particular, formal symmetry alone is insufficient for spectral interpretation.

Our objective is therefore precise: to construct a well-defined self-adjoint operator associated with prime-shift dynamics, obtained as a canonical extension of a densely defined symmetric operator on $\ell^2(\mathbb{N})$.

1.1 Why Self-Adjointness Matters

Spectral statements about arithmetic operators require self-adjointness for their validity. Without it, eigenvalues need not be real, spectral measures may fail to exist, and trace-based constructions become ill-defined.

In the context of arithmetic dynamics, this issue is particularly acute. The operators of interest involve infinite sums over primes, with weights that are typically not absolutely summable. As a result, naive operator definitions may fail to converge or admit multiple inequivalent self-adjoint extensions.

Any attempt to connect arithmetic operators to spectral invariants must therefore begin with a careful analysis of domains, closures, and extensions.

1.2 Formal Symmetry Is Not Enough

Prime-shift operators defined by pairing forward and backward transitions $n \leftrightarrow pn$ are formally symmetric on natural dense domains such as $c_{00}(\mathbb{N})$. However, formal symmetry does not imply self-adjointness, nor does it guarantee uniqueness of extension.

In infinite-dimensional settings, symmetric operators may admit infinitely many self-adjoint extensions, each corresponding to different boundary conditions at infinity. Distinguishing a canonical choice requires additional analytic input.

1.3 Strategy of the Paper

The strategy adopted in this work is based on quadratic form methods. Rather than attempting to define the operator directly, we associate to it a quadratic form encoding prime-shift interactions.

This approach has several advantages:

- it avoids pointwise convergence issues,
- it allows direct control of boundedness and lower semiboundedness,
- it provides access to canonical self-adjoint extensions via standard functional-analytic theorems.

Once a closed, densely defined quadratic form is established, the existence of a unique self-adjoint operator associated with it follows from the Friedrichs extension theorem.

1.4 Regularization and Limits

Throughout the paper, all constructions are carried out within explicitly regularized settings. Prime sums are truncated or damped, and operators are defined initially on finite or weighted subspaces.

The role of regularization is not to approximate an infinite operator naively, but to provide a controlled family of operators whose analytic properties may be studied uniformly. Limits, when taken, are understood in appropriate operator or form senses.

1.5 Scope and Limitations

This paper deliberately restricts its scope. We do not attempt to derive trace formulas, to identify spectral data with zeta or L -functions, or to make any claims regarding the Riemann Hypothesis.

Instead, we isolate the analytic core of prime-shift dynamics and establish a solid operator-theoretic foundation upon which such investigations may later be built.

1.6 Organization of the Paper

The paper is organized as follows. In Section 2 we define the arithmetic Hilbert space and discuss suitable weighting schemes. Section 3 introduces regularized prime-shift operators. Section 4 develops the associated quadratic forms and establishes their basic properties. Section 5 constructs self-adjoint extensions via the Friedrichs method. Section 6 discusses resolvent existence and basic spectral properties. The final section summarizes the results and outlines directions for further work.

2 Arithmetic Hilbert Space

In this section we fix the Hilbert space setting in which prime-shift operators will be analyzed. The choice of space and inner product is not merely technical; it determines the admissible domains, boundedness properties, and the viability of self-adjoint realizations.

2.1 The Canonical Arithmetic Hilbert Space

We begin with the standard Hilbert space

$$\ell^2(\mathbb{N}) = \left\{ \psi : \mathbb{N} \rightarrow \mathbb{C} \left| \sum_{n=1}^{\infty} |\psi(n)|^2 < \infty \right. \right\},$$

equipped with the inner product

$$\langle \psi, \varphi \rangle = \sum_{n=1}^{\infty} \psi(n) \overline{\varphi(n)}.$$

This space provides the simplest setting in which arithmetic states may be represented as square-summable amplitudes indexed by natural numbers. The dense subspace $c_{00}(\mathbb{N})$ of finitely supported functions serves as a natural initial domain for all operators considered in this work.

2.2 Weighted Hilbert Spaces

For certain choices of prime weights and regularizations, it is advantageous to consider weighted Hilbert spaces of the form

$$\ell^2(\mathbb{N}, \mu) = \left\{ \psi : \mathbb{N} \rightarrow \mathbb{C} \left| \sum_{n=1}^{\infty} |\psi(n)|^2 \mu(n) < \infty \right. \right\},$$

where $\mu : \mathbb{N} \rightarrow (0, \infty)$ is a weight function.

The corresponding inner product is given by

$$\langle \psi, \varphi \rangle_{\mu} = \sum_{n=1}^{\infty} \psi(n) \overline{\varphi(n)} \mu(n).$$

Weighted spaces allow partial compensation for growth induced by prime multiplication. Typical choices include polynomial weights $\mu(n) = n^{-\alpha}$ or logarithmic weights $\mu(n) = (\log n)^{-\beta}$, with parameters chosen to balance forward and backward prime shifts.

2.3 Unitary Equivalence of Weighted Spaces

For strictly positive weights μ , the space $\ell^2(\mathbb{N}, \mu)$ is unitarily equivalent to $\ell^2(\mathbb{N})$ via the transformation

$$(U_\mu \psi)(n) = \mu(n)^{1/2} \psi(n).$$

Under this transformation, operators acting on weighted spaces may be conjugated to operators on $\ell^2(\mathbb{N})$ with modified coefficients. This observation allows us to work primarily in the unweighted space while retaining the analytic flexibility provided by weights.

2.4 Domain Considerations

All operators in this paper are initially defined on $c_{00}(\mathbb{N})$, which is dense in both $\ell^2(\mathbb{N})$ and $\ell^2(\mathbb{N}, \mu)$. This choice ensures that algebraic manipulations are unambiguous and that quadratic forms are well defined.

Closures and extensions of operators are taken with respect to the chosen Hilbert space norm. When weighted spaces are employed, this is stated explicitly.

2.5 Growth Under Prime Shifts

A critical issue is the behavior of norms under prime multiplication. For the forward shift

$$(T_p^+ \psi)(n) = \psi(pn),$$

one has formally

$$\|T_p^+ \psi\|^2 = \sum_{n=1}^{\infty} |\psi(pn)|^2 = \sum_{m \equiv 0 \pmod{p}} |\psi(m)|^2,$$

which is bounded by $\|\psi\|^2$ but does not decay with p .

Backward shifts,

$$(T_p^- \psi)(n) = \mathbf{1}_{p|n} \psi(n/p),$$

exhibit analogous behavior. This lack of decay necessitates either damping of prime weights or the use of weighted spaces in order to obtain closed quadratic forms.

2.6 Choice Adopted in This Work

In what follows, we formulate all constructions initially in $\ell^2(\mathbb{N})$ with explicit regularization of prime weights. Weighted spaces are introduced only when analytically convenient and are always reduced to the unweighted case by unitary equivalence.

This approach isolates the essential analytic features of prime-shift dynamics without committing prematurely to a specific weighting scheme.

2.7 Summary

The arithmetic Hilbert space provides the foundational setting for prime-shift operators. Careful attention to domains, weights, and norm behavior under prime shifts is essential for the construction of well-defined quadratic forms and self-adjoint operators.

In the next section we introduce explicit regularizations of prime-shift operators and define the operator families to which quadratic form methods will be applied.

3 $U(1)$ Gauge Phases on Arithmetic Graphs

Having established the untwisted prime-shift operator as a reference dynamics, we now introduce gauge phases on arithmetic transitions. The purpose of this construction is not to modify the underlying combinatorial structure of prime multiplication, but to endow it with a local geometric degree of freedom.

The resulting operators may be viewed as gauge-covariant adjacency operators on the arithmetic graph introduced in the previous section.

3.1 Arithmetic Potentials and Phase Differences

Let

$$\eta : \mathbb{N} \longrightarrow \mathbb{R}$$

be a real-valued function on the arithmetic state space. We interpret $\eta(n)$ as a potential assigned to the integer n . From this potential we define phase factors on directed prime transitions by

$$U(n, p) = e^{2\pi i(\eta(pn) - \eta(n))}.$$

This construction assigns a $U(1)$ phase to each directed edge

$$n \longrightarrow pn,$$

while ensuring that the phase associated with the inverse transition $pn \longrightarrow n$ is given by the complex conjugate

$$U(n, p)^{-1} = e^{-2\pi i(\eta(pn) - \eta(n))}.$$

In this way, phase information is encoded exclusively through potential differences, guaranteeing consistency across composite paths in the arithmetic graph.

3.2 Gauge Transformations

Given a potential function η , we define a unitary gauge transformation G_η on $\ell^2(\mathbb{N})$ by

$$(G_\eta \psi)(n) = e^{2\pi i \eta(n)} \psi(n).$$

Two potentials η and η' related by

$$\eta'(n) = \eta(n) + c, \quad c \in \mathbb{R},$$

generate identical phase factors and therefore define the same gauge. More generally, the construction depends only on the equivalence class of η modulo additive constants.

Under a gauge transformation, the untwisted prime-shift operator H_0 transforms formally as

$$G_\eta^{-1} H_0 G_\eta,$$

producing an operator whose action includes the phase factors $U(n, p)$. This observation provides a direct link between gauge phases and unitary conjugation at the formal level.

3.3 Gauge-Covariant Prime-Shift Operators

Using the phase factors defined above, we introduce the gauge-covariant prime-shift operator

$$(H_\eta \psi)(n) = \sum_{p \in \mathcal{P}} w(p) (U(n, p) \psi(pn) + \mathbf{1}_{p|n} U(n/p, p) \psi(n/p)).$$

Explicitly,

$$(H_\eta \psi)(n) = \sum_{p \in \mathcal{P}} w(p) \left(e^{2\pi i(\eta(pn) - \eta(n))} \psi(pn) + \mathbf{1}_{p|n} e^{2\pi i(\eta(n/p) - \eta(n))} \psi(n/p) \right).$$

This operator differs from H_0 only by the presence of conjugate phase factors on forward and backward transitions. The underlying adjacency structure of the arithmetic graph remains unchanged.

3.4 Formal Symmetry

On the dense domain $c_{00}(\mathbb{N})$ of finitely supported functions, the operator H_η is formally symmetric. Indeed, for $\psi, \varphi \in c_{00}(\mathbb{N})$, each term corresponding to a transition $n \leftrightarrow pn$ contributes complex conjugate phases when evaluated in the inner product, yielding

$$\langle \psi, H_\eta \varphi \rangle = \langle H_\eta \psi, \varphi \rangle.$$

This symmetry is a direct consequence of constructing phases from potential differences. No additional constraints on η are required beyond real-valuedness.

3.5 Interpretation as a Discrete Gauge Theory

The structure introduced above closely parallels lattice gauge theory in mathematical physics. The arithmetic graph plays the role of a lattice, prime multiplication defines adjacency, and the phase factors $U(n, p)$ act as $U(1)$ link variables.

However, unlike conventional lattices, the arithmetic graph is intrinsically multiplicative and nonuniform. Gauge phases therefore probe arithmetic structure rather than spatial geometry. In this sense, the construction defines a genuinely arithmetical gauge theory, in which phase coherence and interference reflect properties of prime-induced dynamics.

3.6 Preparation for Golden-Phase Gauges

The framework developed in this section is fully general and places no restrictions on the choice of potential η . In the following section, we will focus on a distinguished class of potentials constructed from the golden ratio and Fibonacci-based coordinates.

These golden-phase gauges exploit extremal irrationality and optimal equidistribution properties to provide maximally nonresonant probes of arithmetic dynamics. Their introduction requires no modification of the formalism presented here, but only a specific and conceptually motivated choice of η .

4 The Golden-Phase Prime-Shift Operator

We now introduce the central object of this work: the golden-phase prime-shift operator. This operator is obtained by specializing the general gauge-covariant prime-shift construction to a

distinguished class of arithmetic potentials derived from the golden ratio and related Fibonacci structures.

The resulting operator preserves the combinatorial structure of prime-induced dynamics while introducing a maximally nonresonant phase geometry.

4.1 Definition of the Golden-Phase Operator

Let

$$\varphi = \frac{1 + \sqrt{5}}{2}$$

denote the golden ratio. We fix a real-valued potential function

$$\eta_\varphi : \mathbb{N} \longrightarrow \mathbb{R},$$

whose explicit form will be specified later, and define the associated phase difference

$$\theta(n, p) = \eta_\varphi(pn) - \eta_\varphi(n).$$

The golden-phase prime-shift operator H_φ is defined by

$$(H_\varphi \psi)(n) = \sum_{p \in \mathcal{P}} w(p) \left(e^{2\pi i \theta(n, p)} \psi(pn) + \mathbf{1}_{p|n} e^{-2\pi i \theta(n/p, p)} \psi(n/p) \right) + V(n) \psi(n),$$

where $w(p)$ is a real-valued weight function and $V(n)$ is a real-valued local potential.

The defining feature of this operator is the conjugate pairing of phase factors on forward and backward transitions. This structure ensures that phase information is introduced without breaking the formal symmetry of the underlying arithmetic dynamics.

4.2 Structural Symmetry

The apparent symmetry of the definition is not accidental. For each prime p , the transition $n \rightarrow pn$ carries the phase

$$e^{2\pi i (\eta_\varphi(pn) - \eta_\varphi(n))},$$

while the inverse transition $pn \rightarrow n$ carries the conjugate phase

$$e^{-2\pi i (\eta_\varphi(pn) - \eta_\varphi(n))}.$$

As a consequence, each undirected edge of the arithmetic graph contributes a Hermitian pair to the operator. On the dense domain of finitely supported states, this pairing guarantees formal symmetry independently of the explicit choice of η_φ .

This observation captures, in precise terms, the intuitive notion that the operator is almost automatically symmetric once phase factors are derived from a single underlying potential.

4.3 Interpretation as a Gauge-Twisted Adjacency Operator

The operator H_φ may be viewed as a weighted adjacency operator on the arithmetic graph defined by prime multiplication, equipped with a $U(1)$ gauge field. The phase $\theta(n, p)$ assigns a geometric orientation to each prime-induced transition, while preserving the underlying graph structure.

From this perspective, H_φ does not alter the connectivity of the arithmetic state space. Instead, it modifies the interference pattern between arithmetic paths by introducing controlled phase shifts. The golden ratio enters only through the choice of gauge, not through any modification of the combinatorial rules.

4.4 Role of the Local Potential

The term $V(n)$ represents an optional local contribution, diagonal in the arithmetic basis. Typical choices include slowly varying arithmetic functions, such as logarithmic or von Mangoldt-type terms, though no specific form is assumed at this stage.

The presence of $V(n)$ allows the golden-phase operator to be embedded naturally into broader operator families, including previously studied prime-shift Hamiltonians. Importantly, the gauge-phase structure introduced here is entirely independent of the local potential.

4.5 Gauge Equivalence and Physical Content

Formally, the operator H_φ is unitarily equivalent to the untwisted operator H_0 under the gauge transformation generated by η_φ . Nevertheless, this equivalence does not trivialize the construction.

In truncated, regularized, or numerically implemented settings, gauge phases affect boundary behavior, interference patterns, and spectral statistics. In this sense, the golden phase acts as a probe rather than a redundancy: it tests the sensitivity of arithmetic dynamics to maximally irrational phase twisting.

4.6 Conceptual Significance

The golden-phase prime-shift operator occupies an intermediate position between pure arithmetic and spectral geometry. It does not impose order on prime distribution, nor does it assume hidden periodicity. Instead, it introduces a distinguished gauge against which arithmetic coherence can be meaningfully tested.

If prime-induced dynamics is genuinely phase-blind, golden twisting produces no persistent spectral signature. If, however, subtle coherence exists, the golden phase provides a setting in which such effects may become visible.

In the following section, we examine concrete choices of the potential η_φ , ranging from logarithmic gauges to Fibonacci-based representations, and analyze how these choices influence the resulting arithmetic phases.

5 Choices of Golden Gauge Functions

In this section we examine concrete choices of the arithmetic potential η_φ that generate golden-phase gauges. While the formal construction of the gauge-covariant prime-shift operator places no restrictions on η_φ , the interpretive and spectral consequences depend sensitively on this choice.

Our guiding principle is that the golden ratio should enter only as a source of maximal irrationality and optimal equidistribution, not as an imposed symmetry. Accordingly, we focus on gauges that are structurally simple, arithmetically natural, and spectrally nonresonant.

5.1 Logarithmic Golden Gauge

The simplest class of potentials is logarithmic in nature. Let

$$\eta_\varphi(n) = \kappa \log_\varphi n = \kappa \frac{\log n}{\log \varphi},$$

where $\kappa \in \mathbb{R}$ is a coupling parameter. In this case, the phase difference associated with a prime transition is

$$\theta(n, p) = \eta_\varphi(pn) - \eta_\varphi(n) = \kappa \log_\varphi p.$$

Remarkably, the phase depends only on the prime p and not on the base point n . The resulting operator introduces a uniform prime-dependent phase twist across the arithmetic graph.

This gauge may be interpreted as a Dirichlet-type phase deformation. It preserves scale covariance while breaking trivial periodicity through the irrational base φ . Although structurally simple, this choice already distinguishes prime transitions in a spectrally meaningful way.

5.2 Quasi-Periodic Scale Gauge

A more refined construction introduces quasi-periodicity through fractional scaling. Define

$$\eta_\varphi(n) = \kappa \{\log_\varphi n\},$$

where $\{\cdot\}$ denotes the fractional part. The associated phase difference becomes

$$\theta(n, p) = \kappa (\{\log_\varphi(pn)\} - \{\log_\varphi n\}),$$

which depends nontrivially on both n and p .

This gauge introduces scale-dependent interference patterns that vary across the arithmetic graph. Prime transitions at different magnitudes experience distinct phases, producing a genuinely quasi-periodic structure in logarithmic scale.

From a spectral perspective, this choice maximally suppresses accidental resonance. Any persistent spectral signature arising under this gauge cannot be attributed to simple scale periodicity.

5.3 Zeckendorf–Fibonacci Gauge

A conceptually distinct class of gauges is obtained by expressing integers in their Zeckendorf representation. Every $n \in \mathbb{N}$ admits a unique expansion

$$n = \sum_k \epsilon_k F_k, \quad \epsilon_k \in \{0, 1\}, \quad \epsilon_k \epsilon_{k+1} = 0,$$

where $\{F_k\}$ denotes the Fibonacci sequence.

Using this representation, we define

$$\eta_\varphi(n) = \kappa \sum_k \epsilon_k \omega_k,$$

with weights ω_k chosen as

$$\omega_k = \{k\varphi\} \quad \text{or} \quad \omega_k = \varphi^{-k}.$$

This gauge assigns phase contributions to Fibonacci digits rather than to magnitude or scale. Prime multiplication induces nonlocal but highly structured changes in the Zeckendorf digit pattern, resulting in complex phase responses.

Within the language of Quansistor Field Mathematics, this construction admits an interpretation as a field defined on Fibonacci-indexed degrees of freedom. Each digit acts as a local excitation, and prime transitions correspond to structured field interactions.

5.4 Comparison of Gauge Classes

The gauges introduced above differ in complexity and interpretive emphasis:

- logarithmic gauges produce prime-dependent but state-independent phases,
- quasi-periodic scale gauges introduce local interference across scales,
- Zeckendorf-based gauges encode phase information in discrete Fibonacci coordinates.

All three constructions are golden in the sense that they rely on extremal irrationality and avoid rational commensurability. None of them imposes periodic structure on arithmetic dynamics.

5.5 Gauge Choice as a Spectral Probe

The choice of η_φ should be understood as selecting a probe rather than a model. Different gauges test different aspects of arithmetic coherence. If prime-induced dynamics is genuinely phase-insensitive, all golden gauges should yield spectrally equivalent behavior in appropriate limits.

Conversely, systematic differences between these gauges would signal the presence of nontrivial phase correlations. Such effects, if observed, would not constitute an explanation of prime distribution, but rather evidence of hidden structure accessible only through operator-based interrogation.

6 Formal Symmetry and Self-Adjointness

In this section we analyze the symmetry properties of the golden-phase prime-shift operator and clarify the role of domains, regularization, and potential self-adjointness issues. The emphasis is not on establishing maximal generality, but on identifying the structural features that make the operator suitable for spectral analysis and numerical experimentation.

6.1 Natural Domains

Let $c_{00}(\mathbb{N})$ denote the space of finitely supported functions on $\mathbb{N}$. This space is dense in $\ell^2(\mathbb{N})$ and invariant under all prime-shift operators introduced in this work, provided that the prime sum is regularized.

Throughout this section, all operator identities are understood on $c_{00}(\mathbb{N})$, unless stated otherwise. This choice avoids convergence issues while preserving the algebraic structure of the construction.

6.2 Formal Symmetry

Let H_φ denote the golden-phase prime-shift operator defined in the previous section. For $\psi, \varphi \in c_{00}(\mathbb{N})$, we compute

$$\langle \psi, H_\varphi \varphi \rangle = \sum_{n \in \mathbb{N}} \psi(n) (H_\varphi \varphi)(n).$$

Each term in the prime sum corresponds to a pair of transitions

$$n \longleftrightarrow pn,$$

with phase factors arranged as complex conjugates. Exchanging the roles of ψ and φ and reindexing the backward transitions yields

$$\langle \psi, H_\varphi \varphi \rangle = \langle H_\varphi \psi, \varphi \rangle.$$

Thus, H_φ is formally symmetric on $c_{00}(\mathbb{N})$. This property holds independently of the explicit choice of the golden-phase potential η_φ , provided that η_φ is real-valued.

6.3 Role of Phase Pairing

The formal symmetry of H_φ is a direct consequence of constructing phase factors from potential differences. For each prime p , the forward transition $n \rightarrow pn$ carries a phase

$$e^{2\pi i(\eta_\varphi(pn) - \eta_\varphi(n))},$$

while the backward transition $pn \rightarrow n$ carries its complex conjugate.

This pairing ensures that no net phase bias is introduced along closed arithmetic paths. In particular, any finite cycle in the arithmetic graph contributes a real-valued amplitude to the quadratic form associated with H_φ .

6.4 Quadratic Form Perspective

It is often convenient to consider the quadratic form

$$Q_\varphi(\psi) = \langle \psi, H_\varphi \psi \rangle, \quad \psi \in c_{00}(\mathbb{N}).$$

Formally, Q_φ is real-valued and symmetric. This observation provides a useful starting point for discussing extensions of H_φ beyond $c_{00}(\mathbb{N})$ and for comparing different regularizations.

From the quadratic form perspective, the golden-phase operator behaves analogously to a discrete magnetic Schrödinger operator on a graph, with arithmetic structure replacing spatial geometry.

6.5 Remarks on Self-Adjointness

Questions of essential self-adjointness for operators of the form H_φ are subtle and depend on the choice of weights $w(p)$, the local potential $V(n)$, and the imposed regularization.

In this work we do not attempt to establish general self-adjointness results. Instead, we adopt the pragmatic viewpoint that all spectral statements are to be understood within regularized settings, such as finite prime cutoffs or bounded state spaces.

Within these settings, H_φ defines a finite-dimensional Hermitian matrix and is therefore self-adjoint. Observed spectral features may then be studied as functions of the cutoff parameters.

6.6 Gauge Invariance and Truncation Effects

Although the golden-phase operator is formally gauge-equivalent to the untwisted operator in the infinite setting, truncation and regularization break exact gauge invariance. Boundary effects introduce phase-dependent contributions that persist in finite systems.

This phenomenon is not a defect but a feature. It is precisely through such boundary-sensitive effects that gauge phases act as probes of arithmetic structure. Persistent spectral differences under golden-phase twisting therefore carry meaningful information about prime-induced dynamics.

6.7 Summary

The golden-phase prime-shift operator is formally symmetric on a natural dense domain and admits a well-defined quadratic form. While full self-adjointness in the infinite setting remains an open analytical question, all constructions are robust under regularization and truncation.

These properties justify the use of H_φ as a legitimate object of spectral and numerical study. In the next section, we turn to the interpretation of its spectrum and the kinds of arithmetic information such spectra may encode.

7 Spectral Interpretation

In this section we discuss the spectral interpretation of the golden-phase prime-shift operator. Rather than focusing on explicit eigenvalue computations, we aim to clarify what types of arithmetic information may be encoded in the spectrum and how phase twisting modifies the spectral response of prime-induced dynamics.

7.1 Spectrum as an Invariant of Arithmetic Dynamics

For an operator acting on $\ell^2(\mathbb{N})$, the spectrum represents the set of energetically admissible modes of arithmetic motion. In the present context, these modes correspond to coherent superpositions of arithmetic states that are stable under prime-shift transitions.

From an operator-first perspective, the spectrum is the primary invariant of interest. Individual primes appear only indirectly, through their collective contribution to the adjacency structure and the resulting interference patterns. Spectral properties therefore probe global arithmetic organization rather than local combinatorial features.

7.2 Interference and Arithmetic Paths

Each eigenstate of the prime-shift operator may be interpreted as a weighted superposition of arithmetic paths generated by repeated prime multiplication and division. The contribution of a given path is determined not only by its length and weights, but also by the accumulated phase along the path.

In the untwisted operator, all such paths interfere constructively. Introducing gauge phases modifies this interference, selectively enhancing or suppressing certain classes of paths. The golden phase, in particular, introduces maximally noncommensurate phase shifts that tend to decorrelate trivial cycles.

7.3 Phase Sensitivity of the Spectrum

If prime-induced dynamics is insensitive to phase twisting, the spectra of the untwisted and golden-phase operators should agree in appropriate limits, up to finite-size effects. In this case, gauge phases act as a benign redundancy.

If, however, the spectrum exhibits systematic dependence on the golden gauge, this indicates the presence of arithmetic coherence beyond purely combinatorial structure. Such sensitivity would manifest as shifts in eigenvalue distributions, changes in spectral gaps, or alterations in level statistics.

Importantly, any observed phase sensitivity must persist under variation of regularization parameters in order to be considered meaningful.

7.4 Spectral Statistics

Beyond individual eigenvalues, statistical properties of the spectrum provide a powerful diagnostic tool. In particular, level spacing distributions may be compared against random matrix ensembles,

such as the Gaussian Orthogonal Ensemble (GOE) or Gaussian Unitary Ensemble (GUE).

The introduction of gauge phases naturally breaks time-reversal symmetry at the level of arithmetic paths, suggesting that transitions between symmetry classes may occur as the phase coupling is varied. The golden phase provides a canonical benchmark against which such transitions may be tested.

7.5 Relation to Hilbert–Pólya-Type Intuitions

The Hilbert–Pólya conjecture suggests that the nontrivial zeros of the Riemann zeta function may arise as eigenvalues of a self-adjoint operator. While the golden-phase prime-shift operator is not proposed as such an operator, its spectral behavior may nevertheless inform operator-based intuitions.

In particular, the presence or absence of phase-sensitive spectral structure bears on the question of whether prime-induced dynamics supports nontrivial arithmetic resonances. The golden phase serves here as a stringent test: any resonance surviving maximal irrational twisting must be deeply rooted in arithmetic structure.

7.6 Spectral Stability and Robustness

A key requirement for meaningful interpretation is spectral stability. Features attributable to arithmetic structure should remain stable under moderate changes in truncation, weighting, and gauge choice within the golden class.

Conversely, spectral features that vary erratically under such changes are best interpreted as finite-size artifacts. The distinction between these regimes is central to the numerical protocols discussed in the following section.

7.7 Interpretive Summary

The spectrum of the golden-phase prime-shift operator encodes information about the coherence and interference of prime-induced arithmetic dynamics. Gauge phases do not explain prime distribution, but they test its sensitivity to controlled phase perturbations.

Whether the outcome is complete phase blindness or the emergence of persistent spectral structure, the result carries conceptual significance. In both cases, the operator framework provides a precise language in which arithmetic dynamics may be interrogated spectrally rather than combinatorially.

In the next section we describe explicit numerical protocols designed to probe these spectral effects in a falsifiable and reproducible manner.

8 Numerical Protocols and Falsifiability

This section outlines concrete numerical protocols for investigating the spectral properties of the golden-phase prime-shift operator. The emphasis is on reproducibility, falsifiability, and the separation of genuine arithmetic effects from finite-size artifacts.

All numerical statements are to be understood within regularized settings. The goal is not to approximate an infinite operator, but to study the stability and response of finite truncations under controlled phase perturbations.

8.1 Finite-Dimensional Truncations

Numerical implementations are based on truncating both the state space and the prime sum. Specifically, we restrict

$$n \leq N, \quad p \leq P,$$

and consider the resulting operator acting on $\mathbb{C}^N$.

Within this setting, the golden-phase prime-shift operator defines a finite Hermitian matrix. The dependence of spectral features on the cutoff parameters N and P provides a primary diagnostic for distinguishing structural effects from truncation artifacts.

8.2 Regularization of Prime Weights

In practice, the choice of weight function $w(p)$ significantly influences numerical stability. Common choices include

$$w(p) = \frac{1}{p}, \quad w(p) = \frac{e^{-p/P}}{p}, \quad w(p) = \frac{1}{p^\alpha}, \quad \alpha > 1.$$

Smooth damping of large primes improves convergence and allows systematic study of the dependence on prime scale. All gauge constructions introduced earlier are compatible with such regularizations.

8.3 Spectral Observables

The primary numerical observables include:

- eigenvalue distributions and spectral density,
- gaps and extremal eigenvalues,
- level spacing distributions after unfolding,
- sensitivity of eigenvalues to variations in the gauge parameter κ .

Comparisons are performed between the untwisted operator H_0 , the golden-phase operator H_φ , and operators with randomized phases serving as control ensembles.

8.4 Gauge Sensitivity Tests

A central diagnostic is the response of the spectrum to gauge twisting. For a fixed truncation (N, P) , one varies the coupling parameter κ and observes the evolution of spectral features.

Three regimes may be distinguished:

- *phase-blind regime*: spectral observables remain invariant up to statistical noise,
- *perturbative regime*: small but systematic shifts occur, scaling with κ ,
- *coherent regime*: nontrivial spectral restructuring persists under variation of κ and truncation parameters.

Only the latter two regimes indicate genuine phase sensitivity.

8.5 Comparison with Random Phases

To assess the specificity of golden-phase effects, it is essential to compare results with randomized gauge phases. In such control experiments, the phase factors $U(n, p)$ are replaced by independent random elements of $U(1)$.

If golden-phase effects are indistinguishable from random-phase behavior, this suggests the absence of arithmetic coherence detectable by phase twisting. By contrast, systematic differences indicate structure specific to the golden gauge.

8.6 Level Statistics and Symmetry Classes

After unfolding the spectrum, level spacing distributions may be compared to random matrix ensembles. The untwisted operator typically exhibits statistics associated with time-reversal invariant systems.

Introducing gauge phases may induce a crossover toward unitary statistics, reflecting the breaking of effective time-reversal symmetry in arithmetic path interference. The golden phase provides a canonical, nonrandom benchmark for studying such transitions.

8.7 Criteria for Falsification

The framework proposed in this paper admits clear falsification criteria:

- absence of stable spectral differences between golden-phase and random-phase ensembles,
- disappearance of phase sensitivity as truncation parameters increase,
- lack of reproducibility across independent implementations.

Failure to observe any persistent golden-phase effects under these tests would constitute strong evidence that prime-induced dynamics is phase-insensitive at the operator level considered here.

8.8 Reproducibility and Open Data

All numerical experiments should be conducted using publicly available code and documented parameter choices. The simplicity of the operator definition allows implementation in a variety of environments, including Python, Julia, and JavaScript-based numerical frameworks.

Reproducibility is essential: the purpose of numerical exploration in this context is not confirmation bias, but controlled interrogation of arithmetic dynamics through spectral probes.

8.9 Summary

Numerical analysis provides the bridge between formal operator construction and empirical assessment. The protocols outlined above allow the golden-phase prime-shift operator to be tested as a genuine probe of arithmetic coherence.

In the following section, we place these constructions within the broader framework of Quansistor Field Mathematics and discuss their interpretation as field-theoretic refinements of operator-based arithmetic models.

9 Relation to QFM and Quansistor Fields

In this section we situate the golden-phase prime-shift operator within the broader conceptual framework of Quansistor Field Mathematics (QFM). The purpose is not to introduce new formal machinery, but to clarify how the constructions developed in this paper fit naturally into an operator-first, field-oriented view of arithmetic dynamics.

9.1 Operator-First Arithmetic

A central principle of QFM is that arithmetic structure should be approached through operators rather than explicit formulas. In this view, numbers label states, while operators encode permissible transitions and interactions between them.

The prime-shift operators studied in this paper exemplify this philosophy. Multiplication by primes is treated as a fundamental dynamical process, and arithmetic information is extracted from the spectral response of the resulting operator rather than from combinatorial enumeration.

9.2 Quansistors as Arithmetic Degrees of Freedom

Within QFM, a quansistor is interpreted as an elementary computational or dynamical unit whose behavior is defined by its interaction with surrounding structure. In the present setting, arithmetic states indexed by natural numbers may be viewed as composite configurations of such units.

Prime-shift transitions then represent structured interactions between quansistors, mediated by arithmetic adjacency. The golden-phase gauge introduced in this work adds a local phase degree of freedom to these interactions, enriching the dynamical landscape without altering the underlying connectivity.

9.3 Gauge Fields as Arithmetic Structure

From the QFM perspective, gauge fields are not auxiliary constructs but intrinsic features of structured systems. The arithmetic gauge introduced here assigns phases to prime-induced transitions, effectively defining a field over the arithmetic graph.

Crucially, this field is not imposed externally. It arises from a potential function on arithmetic states and respects the multiplicative structure of $\mathbb{N}$. In this sense, the golden-phase construction may be interpreted as an arithmetic gauge field in the strictest sense.

9.4 Fibonacci Coordinates and Field Representations

The Zeckendorf-based gauges discussed earlier provide a particularly direct connection to field-theoretic intuition. By expressing integers in Fibonacci coordinates, arithmetic states acquire a multi-component representation with local constraints.

Within QFM, such representations naturally suggest a field interpretation, with each Fibonacci digit corresponding to a local degree of freedom. Prime-induced transitions act nonlocally on these degrees of freedom, while the golden phase encodes their interference.

9.5 Compatibility with QVM Architectures

The golden-phase prime-shift operator is compatible with virtual and distributed computational architectures envisioned within the Quantum Virtual Machine (QVM) program. Finite truncations

of the operator correspond to finite computational instances, while gauge parameters serve as tunable controls.

Because the construction is operator-based and discretely defined, it may be implemented and explored across heterogeneous computational substrates without loss of conceptual coherence.

9.6 Conceptual Contribution to QFM

The primary contribution of this work to QFM is the introduction of a principled, arithmetically motivated gauge degree of freedom. Rather than proposing a new axiomatic system, the golden-phase operator refines existing operator-based approaches by introducing a controlled probe of arithmetic coherence.

This refinement strengthens the interpretive reach of QFM by providing a concrete example in which gauge structure, arithmetic dynamics, and spectral analysis interact in a falsifiable and reproducible manner.

9.7 Position Within the QFM Program

The constructions presented here should be regarded as a modular component of the broader QFM program. They neither depend on nor assume the full machinery of QFM, but they align naturally with its guiding principles.

In particular, the golden-phase prime-shift operator demonstrates how operator geometry and arithmetic structure may be intertwined without resorting to speculative assumptions. It provides a clear example of how QFM-style reasoning can yield concrete, testable mathematical objects.

10 Outlook and Open Problems

This paper has introduced a class of gauge-covariant prime-shift operators in which arithmetic dynamics is probed through maximally irrational phase twisting. The central construction, the golden-phase prime-shift operator, preserves the combinatorial structure of prime multiplication while enriching it with a geometrically motivated $U(1)$ gauge degree of freedom.

The framework developed here does not seek to explain the distribution of prime numbers. Instead, it establishes a controlled and falsifiable setting in which arithmetic coherence may be tested spectrally. Whether the outcome of such tests is positive or negative, the operator-based perspective provides information that is inaccessible through purely combinatorial approaches.

10.1 Summary of Contributions

The main contributions of this work may be summarized as follows:

- the formulation of prime-shift operators as adjacency operators on an arithmetic graph,
- the introduction of gauge phases derived from arithmetic potentials, ensuring formal symmetry,
- the identification of the golden ratio as a canonical source of maximally nonresonant gauge twisting,
- the proposal of concrete numerical protocols enabling falsifiable tests of phase sensitivity in prime-induced dynamics,

- the integration of these constructions into an operator-first arithmetic framework compatible with Quansistor Field Mathematics.

Each of these elements is modular and may be studied independently or in combination with other operator-based approaches.

10.2 Interpretive Caution

It is important to emphasize the limitations of the present work. The appearance of spectral sensitivity under golden-phase twisting, should it occur, does not constitute an explanation of prime distribution, nor does it imply hidden periodicity in the primes.

Conversely, the absence of any such sensitivity would not invalidate operator approaches to arithmetic, but would instead provide evidence that prime-induced dynamics is robustly phase-blind under optimal irrational perturbations. Both outcomes are informative within an operator-theoretic paradigm.

10.3 Analytical Open Problems

Several analytical questions remain open:

- the characterization of self-adjoint extensions of infinite golden-phase prime-shift operators,
- the dependence of spectral measures on the choice of arithmetic potential beyond finite truncations,
- the possible emergence of trace-class or resolvent regularity under specific gauge choices.

Addressing these questions would require tools from functional analysis, spectral theory on graphs, and analytic number theory.

10.4 Extensions Beyond Prime Shifts

The construction presented here may be extended in several directions. One natural avenue is the replacement of prime shifts by more general multiplicative or arithmetic transitions, including those associated with Dirichlet characters or L -functions.

Another possibility is the introduction of nonabelian gauge groups, allowing richer interference structures and potentially new classes of spectral behavior.

10.5 Computational and Experimental Directions

From a computational perspective, the simplicity of the operator definition invites large-scale numerical exploration. Distributed and virtual computational architectures may be employed to explore spectral behavior across wide ranges of parameters.

Systematic comparison between golden-phase gauges and other irrational gauges may further clarify the role of extremal irrationality in arithmetic probing.

10.6 Conceptual Outlook

At a conceptual level, this work supports the view that arithmetic structure may be more effectively interrogated through dynamical and spectral responses than through static enumeration. Gauge

phases, when chosen carefully, provide a means of asking precise questions without imposing artificial structure.

The golden-phase prime-shift operator represents one such question. Whether its answer reveals hidden coherence or confirms robust randomness, the result contributes to a deeper understanding of arithmetic dynamics viewed through the lens of operators.

10.7 Closing Remarks

The operator-based study of arithmetic remains a young and evolving field. By introducing a maximally nonresonant gauge probe grounded in the golden ratio, this work adds a concrete and testable tool to the growing repertoire of arithmetical operator constructions.

It is hoped that the framework presented here will encourage further exploration, critical testing, and refinement, contributing incrementally to a broader spectral understanding of arithmetic phenomena.

APPENDIX A

A.1 Technical Remarks and Supplementary Material

This appendix collects technical remarks, clarifications, and auxiliary constructions that support the main text but are not essential for its logical flow. No new conceptual assumptions are introduced here.

A.1.1 Arithmetic Graph Structure

The arithmetic graph underlying the prime-shift operators has vertex set $\mathbb{N}$ and directed edges

$$n \longrightarrow pn \quad \text{for each } p \in \mathbb{P}.$$

Each vertex has infinite out-degree and finite in-degree. This graph is locally finite only after imposing a prime cutoff.

The adjacency structure is multiplicative rather than additive, distinguishing it from conventional lattices and justifying the use of operator-based rather than geometric intuition.

A.1.2 Regularized Operators

All operators considered in this work admit natural regularized versions

$$(H_\varphi^{(N,P)}\psi)(n) = \sum_{\substack{p \leq P \\ pn \leq N}} w(p) \left(e^{2\pi i \theta(n,p)} \psi(pn) + \mathbf{1}_{p|n} e^{-2\pi i \theta(n/p,p)} \psi(n/p) \right) + V(n)\psi(n),$$

acting on $\mathbb{C}^N$.

Such truncations define finite Hermitian matrices and are suitable for numerical diagonalization. All spectral statements in the main text refer to limits or stability properties of these regularized operators.

A.1.3 Gauge Equivalence and Boundary Effects

Formally, the gauge-transformed operator

$$H_\eta = G_\eta^{-1} H_0 G_\eta$$

is unitarily equivalent to the untwisted operator H_0 . However, this equivalence is exact only in the infinite, untruncated setting.

Finite truncations break gauge invariance through boundary effects. These effects are not artifacts but carry the phase sensitivity exploited in numerical probes. Spectral dependence on gauge parameters therefore reflects arithmetic structure interacting with truncation geometry.

A.1.4 Quadratic Form Estimates

For $\psi \in c_{00}(\mathbb{N})$, the quadratic form associated with the golden-phase operator satisfies

$$|Q_\varphi(\psi)| \leq 2 \sum_p |w(p)| \sum_n |\psi(n)|^2 + \sum_n |V(n)| |\psi(n)|^2,$$

formally suggesting boundedness under sufficiently strong damping of $w(p)$.

While this estimate is heuristic, it motivates the use of exponentially damped weights in numerical experiments.

A.1.5 Alternative Gauge Choices

Although the main text focuses on golden-phase gauges, the construction extends to arbitrary irrational gauges. In particular, one may replace φ by other quadratic irrationals or by generic badly approximable numbers.

The golden ratio is distinguished not by uniqueness but by extremality: it provides a canonical benchmark against which other irrational gauges may be compared.

A.1.6 Pseudocode Sketch

A minimal numerical implementation proceeds as follows:

1. Fix truncation parameters (N, P) and gauge choice η_φ .
2. Construct the sparse matrix representing $H_\varphi^{(N, P)}$.
3. Diagonalize the matrix using standard Hermitian solvers.
4. Compute spectral observables and compare against control ensembles.

Because the operator is sparse and structured, scalable implementations are feasible even for moderately large N .

A.1.7 Relation to Other Operator Constructions

Prime-shift operators with and without gauge phases appear implicitly in various contexts in analytic number theory and mathematical physics. The distinguishing feature of the present construction is the systematic use of gauge geometry as a diagnostic tool rather than as a modeling assumption.

This viewpoint aligns naturally with operator-based arithmetic programs while remaining agnostic about deeper conjectural interpretations.

A.1.8 Closing Remark

The technical considerations collected here are intended to clarify, not to complicate, the constructions presented in the main text. All essential ideas are contained in the core chapters; the appendix merely provides supporting detail for interested readers and practitioners.

Chapter 15

Self-Adjoint Prime-Shift Operators

Quadratic Forms, Regularization and Arithmetic Dynamics

Enter Yourname

Abstract

This document establishes a rigorous analytic foundation for prime-shift dynamics by constructing regularized arithmetic operators that admit canonical self-adjoint realizations. The operators act on the space of square-summable sequences over the natural numbers and are generated by prime-indexed multiplicative shifts together with diagonal arithmetic potentials.

The construction is carried out using quadratic-form methods. Explicit regularized operator families are defined, and their associated quadratic forms are shown to be densely defined, lower bounded, and closable. As a result, canonical self-adjoint operators are obtained, ensuring well-defined resolvents and spectral measures.

The emphasis of the work is analytic rather than identificational. No attempt is made to connect spectra directly to the zeros of the Riemann zeta function at this stage. Instead, the goal is to stabilize the operator-theoretic core so that later developments involving trace formulas, spectral invariants, and numerical probes rest on a mathematically controlled foundation. Diagonal arithmetic potentials are shown to enter as admissible perturbations that preserve self-adjointness and spectral legitimacy.

1 Introduction and Strategy

The purpose of this paper is to place prime-shift operators into a rigorous self-adjoint framework suitable for spectral analysis. While operators encoding multiplicative arithmetic dynamics have appeared in various formal or heuristic contexts, their analytic foundations are often left implicit. In particular, formal symmetry alone is insufficient for spectral interpretation.

Our objective is therefore precise: to construct a well-defined self-adjoint operator associated with prime-shift dynamics, obtained as a canonical extension of a densely defined symmetric operator on $\ell^2(\mathbb{N})$.

1.1 Why Self-Adjointness Matters

Spectral statements about arithmetic operators require self-adjointness for their validity. Without it, eigenvalues need not be real, spectral measures may fail to exist, and trace-based constructions become ill-defined.

In the context of arithmetic dynamics, this issue is particularly acute. The operators of interest involve infinite sums over primes, with weights that are typically not absolutely summable. As a result, naive operator definitions may fail to converge or admit multiple inequivalent self-adjoint extensions.

Any attempt to connect arithmetic operators to spectral invariants must therefore begin with a careful analysis of domains, closures, and extensions.

1.2 Formal Symmetry Is Not Enough

Prime-shift operators defined by pairing forward and backward transitions $n \leftrightarrow pn$ are formally symmetric on natural dense domains such as $c_{00}(\mathbb{N})$. However, formal symmetry does not imply self-adjointness, nor does it guarantee uniqueness of extension.

In infinite-dimensional settings, symmetric operators may admit infinitely many self-adjoint extensions, each corresponding to different boundary conditions at infinity. Distinguishing a canonical choice requires additional analytic input.

1.3 Strategy of the Paper

The strategy adopted in this work is based on quadratic form methods. Rather than attempting to define the operator directly, we associate to it a quadratic form encoding prime-shift interactions.

This approach has several advantages:

- it avoids pointwise convergence issues,
- it allows direct control of boundedness and lower semiboundedness,
- it provides access to canonical self-adjoint extensions via standard functional-analytic theorems.

Once a closed, densely defined quadratic form is established, the existence of a unique self-adjoint operator associated with it follows from the Friedrichs extension theorem.

1.4 Regularization and Limits

Throughout the paper, all constructions are carried out within explicitly regularized settings. Prime sums are truncated or damped, and operators are defined initially on finite or weighted subspaces.

The role of regularization is not to approximate an infinite operator naively, but to provide a controlled family of operators whose analytic properties may be studied uniformly. Limits, when taken, are understood in appropriate operator or form senses.

1.5 Scope and Limitations

This paper deliberately restricts its scope. We do not attempt to derive trace formulas, to identify spectral data with zeta or L -functions, or to make any claims regarding the Riemann Hypothesis.

Instead, we isolate the analytic core of prime-shift dynamics and establish a solid operator-theoretic foundation upon which such investigations may later be built.

1.6 Organization of the Paper

The paper is organized as follows. In Section 2 we define the arithmetic Hilbert space and discuss suitable weighting schemes. Section 3 introduces regularized prime-shift operators. Section 4 develops the associated quadratic forms and establishes their basic properties. Section 5 constructs self-adjoint extensions via the Friedrichs method. Section 6 discusses resolvent existence and basic spectral properties. The final section summarizes the results and outlines directions for further work.

2 Arithmetic Hilbert Space

In this section we fix the Hilbert space setting in which prime-shift operators will be analyzed. The choice of space and inner product is not merely technical; it determines the admissible domains, boundedness properties, and the viability of self-adjoint realizations.

2.1 The Canonical Arithmetic Hilbert Space

We begin with the standard Hilbert space

$$\ell^2(\mathbb{N}) = \left\{ \psi : \mathbb{N} \rightarrow \mathbb{C} \left| \sum_{n=1}^{\infty} |\psi(n)|^2 < \infty \right. \right\},$$

equipped with the inner product

$$\langle \psi, \varphi \rangle = \sum_{n=1}^{\infty} \psi(n) \overline{\varphi(n)}.$$

This space provides the simplest setting in which arithmetic states may be represented as square-summable amplitudes indexed by natural numbers. The dense subspace $c_{00}(\mathbb{N})$ of finitely supported functions serves as a natural initial domain for all operators considered in this work.

2.2 Weighted Hilbert Spaces

For certain choices of prime weights and regularizations, it is advantageous to consider weighted Hilbert spaces of the form

$$\ell^2(\mathbb{N}, \mu) = \left\{ \psi : \mathbb{N} \rightarrow \mathbb{C} \left| \sum_{n=1}^{\infty} |\psi(n)|^2 \mu(n) < \infty \right. \right\},$$

where $\mu : \mathbb{N} \rightarrow (0, \infty)$ is a weight function.

The corresponding inner product is given by

$$\langle \psi, \varphi \rangle_{\mu} = \sum_{n=1}^{\infty} \psi(n) \overline{\varphi(n)} \mu(n).$$

Weighted spaces allow partial compensation for growth induced by prime multiplication. Typical choices include polynomial weights $\mu(n) = n^{-\alpha}$ or logarithmic weights $\mu(n) = (\log n)^{-\beta}$, with parameters chosen to balance forward and backward prime shifts.

2.3 Unitary Equivalence of Weighted Spaces

For strictly positive weights μ , the space $\ell^2(\mathbb{N}, \mu)$ is unitarily equivalent to $\ell^2(\mathbb{N})$ via the transformation

$$(U_\mu \psi)(n) = \mu(n)^{1/2} \psi(n).$$

Under this transformation, operators acting on weighted spaces may be conjugated to operators on $\ell^2(\mathbb{N})$ with modified coefficients. This observation allows us to work primarily in the unweighted space while retaining the analytic flexibility provided by weights.

2.4 Domain Considerations

All operators in this paper are initially defined on $c_{00}(\mathbb{N})$, which is dense in both $\ell^2(\mathbb{N})$ and $\ell^2(\mathbb{N}, \mu)$. This choice ensures that algebraic manipulations are unambiguous and that quadratic forms are well defined.

Closures and extensions of operators are taken with respect to the chosen Hilbert space norm. When weighted spaces are employed, this is stated explicitly.

2.5 Growth Under Prime Shifts

A critical issue is the behavior of norms under prime multiplication. For the forward shift

$$(T_p^+ \psi)(n) = \psi(pn),$$

one has formally

$$\|T_p^+ \psi\|^2 = \sum_{n=1}^{\infty} |\psi(pn)|^2 = \sum_{m \equiv 0 \pmod{p}} |\psi(m)|^2,$$

which is bounded by $\|\psi\|^2$ but does not decay with p .

Backward shifts,

$$(T_p^- \psi)(n) = \mathbf{1}_{p|n} \psi(n/p),$$

exhibit analogous behavior. This lack of decay necessitates either damping of prime weights or the use of weighted spaces in order to obtain closed quadratic forms.

2.6 Choice Adopted in This Work

In what follows, we formulate all constructions initially in $\ell^2(\mathbb{N})$ with explicit regularization of prime weights. Weighted spaces are introduced only when analytically convenient and are always reduced to the unweighted case by unitary equivalence.

This approach isolates the essential analytic features of prime-shift dynamics without committing prematurely to a specific weighting scheme.

2.7 Summary

The arithmetic Hilbert space provides the foundational setting for prime-shift operators. Careful attention to domains, weights, and norm behavior under prime shifts is essential for the construction of well-defined quadratic forms and self-adjoint operators.

In the next section we introduce explicit regularizations of prime-shift operators and define the operator families to which quadratic form methods will be applied.

3 Prime-Shift Operators and Regularization

In this section we introduce explicit regularizations of prime-shift operators. The purpose of regularization is twofold: to ensure that all expressions are well defined on natural dense domains, and to provide operator families whose analytic properties can be studied uniformly as regularization parameters vary.

3.1 Formal Prime-Shift Operator

Formally, prime-shift dynamics is generated by the expression

$$(H\psi)(n) = \sum_{p \in \mathbb{P}} w(p)(\psi(pn) + \mathbf{1}_{p|n} \psi(n/p)),$$

where $w(p)$ is a real-valued weight function. For typical arithmetic choices, such as $w(p) = p^{-1}$, the sum diverges and the expression does not define a bounded or densely defined operator on $\ell^2(\mathbb{N})$.

This necessitates the introduction of explicit regularization.

3.2 Prime Cutoff Regularization

The simplest regularization is a hard cutoff on the prime sum. For $P < \infty$, we define

$$(H^{(P)}\psi)(n) = \sum_{\substack{p \in \mathbb{P} \\ p \leq P}} w(p)(\psi(pn) + \mathbf{1}_{p|n} \psi(n/p)).$$

For fixed P , the operator $H^{(P)}$ is well defined on $c_{00}(\mathbb{N})$ and extends to a densely defined symmetric operator on $\ell^2(\mathbb{N})$. The dependence on P provides a controlled family of operators whose behavior may be analyzed as $P \rightarrow \infty$.

3.3 Soft Cutoff and Damped Weights

To obtain smoother control over large primes, we also consider damped weight functions of the form

$$w_{P,\alpha}(p) = e^{-p/P} p^{-\alpha}, \quad \alpha > 0.$$

The corresponding operator is defined by

$$(H^{(P,\alpha)}\psi)(n) = \sum_{p \in \mathbb{P}} w_{P,\alpha}(p)(\psi(pn) + \mathbf{1}_{p|n} \psi(n/p)).$$

For each fixed (P, α) , the sum converges absolutely for $\psi \in c_{00}(\mathbb{N})$. Moreover, for $\alpha > 1$ the prime sum converges uniformly in P , yielding improved boundedness properties.

3.4 Regularized Operator Families

In what follows, we work with a family of operators

$$\{ H^{(P,\alpha)} \}_{P>0, \alpha>\alpha_0},$$

where $\alpha_0 \geq 0$ is chosen to ensure the desired analytic properties of the associated quadratic forms.

This family interpolates between strongly regularized operators and the formally unregularized prime-shift dynamics. Analytic statements are made uniformly with respect to (P, α) wherever possible.

3.5 Symmetry on the Core Domain

For each choice of regularization parameters, the operator $H^{(P, \alpha)}$ is symmetric on $c_{00}(\mathbb{N})$. Indeed, for $\psi, \varphi \in c_{00}(\mathbb{N})$, one has

$$\langle \psi, H^{(P, \alpha)} \varphi \rangle = \langle H^{(P, \alpha)} \psi, \varphi \rangle,$$

as each term corresponding to a transition $n \leftrightarrow pn$ appears in conjugate pairs.

This formal symmetry provides the starting point for the quadratic form analysis developed in the next section.

3.6 Inclusion of Local Potentials

More general operator families may include an additional diagonal term

$$(V\psi)(n) = V(n) \psi(n),$$

with $V : \mathbb{N} \rightarrow \mathbb{R}$. Such terms may be used to stabilize the spectrum or to incorporate slowly varying arithmetic contributions.

Throughout this paper, the inclusion of $V(n)$ is optional and does not affect the core regularization arguments.

3.7 Summary

Explicit regularization is essential for placing prime-shift dynamics on firm analytic footing. The families $H^{(P)}$ and $H^{(P, \alpha)}$ provide concrete, well-defined operators suitable for quadratic form methods.

In the next section we associate quadratic forms to these operators and analyze their boundedness and closability properties, paving the way toward canonical self-adjoint realizations.

4 Quadratic Forms

In this section we associate quadratic forms to the regularized prime-shift operators introduced previously. The quadratic form approach provides a robust framework for controlling unbounded operators and for constructing canonical self-adjoint realizations.

4.1 Definition of the Quadratic Form

Let $H^{(P, \alpha)}$ denote one of the regularized prime-shift operators defined in Section 3. We define the associated quadratic form

$$Q_{P, \alpha}(\psi) = \langle \psi, H^{(P, \alpha)} \psi \rangle, \quad \psi \in c_{00}(\mathbb{N}).$$

Explicitly,

$$Q_{P, \alpha}(\psi) = \sum_{p \in \mathbb{P}} w_{P, \alpha}(p) \sum_{n \in \mathbb{N}} \psi(n) (\psi(pn) + \mathbf{1}_{p|n} \psi(n/p)).$$

By symmetry of the prime-shift terms, $Q_{P, \alpha}(\psi)$ is real-valued for all $\psi \in c_{00}(\mathbb{N})$.

4.2 Symmetrized Representation

Using the change of variables $m = pn$ in the backward term, the quadratic form may be written in the symmetrized form

$$Q_{P,\alpha}(\psi) = 2 \sum_{p \in \mathbb{P}} w_{P,\alpha}(p) \sum_{n \in \mathbb{N}} \operatorname{Re}(\psi(n) \overline{\psi(pn)}).$$

This representation makes explicit the pairing of forward and backward prime transitions and will be used to establish boundedness estimates.

4.3 Form Boundedness

For $\psi \in c_{00}(\mathbb{N})$, the Cauchy–Schwarz inequality yields

$$|\psi(n) \overline{\psi(pn)}| \leq \frac{1}{2} (|\psi(n)|^2 + |\psi(pn)|^2).$$

Substituting into the symmetrized form gives

$$|Q_{P,\alpha}(\psi)| \leq \sum_{p \in \mathbb{P}} w_{P,\alpha}(p) \sum_{n \in \mathbb{N}} (|\psi(n)|^2 + |\psi(pn)|^2).$$

Interchanging sums and using the fact that

$$\sum_n |\psi(pn)|^2 \leq \sum_n |\psi(n)|^2,$$

we obtain

$$|Q_{P,\alpha}(\psi)| \leq 2 \left(\sum_{p \in \mathbb{P}} w_{P,\alpha}(p) \right) \|\psi\|^2.$$

Thus, whenever $\sum_p w_{P,\alpha}(p) < \infty$, the quadratic form is bounded with respect to the $\ell^2(\mathbb{N})$ norm.

4.4 Lower Semiboundedness

If the weights $w_{P,\alpha}(p)$ are nonnegative, the quadratic form satisfies

$$Q_{P,\alpha}(\psi) \geq -C_{P,\alpha} \|\psi\|^2$$

for some finite constant $C_{P,\alpha}$. In particular, for exponentially damped weights or for $\alpha > 1$, the form is bounded below.

Lower semiboundedness is the key hypothesis required for the application of Friedrichs' extension theorem.

4.5 Closability of the Form

Since $Q_{P,\alpha}$ is densely defined and bounded with respect to the Hilbert space norm, it is closable. We denote its closure by $\overline{Q}_{P,\alpha}$, with form domain $D(\overline{Q}_{P,\alpha})$.

The closure may be described explicitly as the completion of $c_{00}(\mathbb{N})$ with respect to the form norm

$$\|\psi\|_Q^2 = \|\psi\|^2 + Q_{P,\alpha}(\psi).$$

4.6 Inclusion of Diagonal Potentials

If a real-valued diagonal potential $V(n)$ is included, the quadratic form becomes

$$Q_{P,\alpha}^V(\psi) = Q_{P,\alpha}(\psi) + \sum_{n \in \mathbb{N}} V(n) |\psi(n)|^2.$$

Provided $V(n)$ is bounded below or form-bounded with respect to $Q_{P,\alpha}$, all conclusions above remain valid. This allows incorporation of logarithmic or von Mangoldt-type terms as relatively bounded perturbations.

4.7 Relation to the SMRK Hamiltonian

The SMRK Hamiltonian may be written schematically as

$$H_{\text{SMRK}} = H_{\text{prime-shift}} + V_{\text{SMRK}}(n),$$

where $H_{\text{prime-shift}}$ denotes the symmetric core analyzed here and $V_{\text{SMRK}}(n)$ is a real-valued diagonal arithmetic potential.

The present quadratic form analysis establishes a rigorous foundation for the prime-shift component of H_{SMRK} . Diagonal arithmetic terms may be treated as relatively form-bounded perturbations and incorporated at later stages without affecting self-adjointness of the core operator.

4.8 Summary

The regularized prime-shift operators admit densely defined, lower semibounded, closable quadratic forms. These forms provide the natural analytic starting point for constructing canonical self-adjoint realizations.

In the next section we use Friedrichs' extension theorem to associate a unique self-adjoint operator to the closed quadratic form $\overline{Q}_{P,\alpha}$.

5 Self-Adjoint Extensions

In this section we construct self-adjoint realizations of the regularized prime-shift operators using quadratic form methods. The key result is the existence of a canonical self-adjoint operator associated with the closed, lower semibounded quadratic forms introduced in the previous section.

5.1 From Quadratic Forms to Operators

Let $\overline{Q}_{P,\alpha}$ denote the closure of the quadratic form $Q_{P,\alpha}$ defined initially on $c_{00}(\mathbb{N})$. By construction, $\overline{Q}_{P,\alpha}$ is densely defined, closed, and lower semibounded on $\ell^2(\mathbb{N})$.

By the representation theorem for closed quadratic forms, there exists a unique self-adjoint operator $A_{P,\alpha}$ such that

$$\overline{Q}_{P,\alpha}(\psi, \varphi) = \langle \psi, A_{P,\alpha} \varphi \rangle, \quad \varphi \in D(A_{P,\alpha}), \psi \in D(\overline{Q}_{P,\alpha}).$$

The operator $A_{P,\alpha}$ is called the operator associated with the quadratic form $\overline{Q}_{P,\alpha}$.

5.2 Friedrichs Extension

The operator $A_{P,\alpha}$ may be viewed as the Friedrichs extension of the symmetric operator $H^{(P,\alpha)}$ initially defined on $c_{00}(\mathbb{N})$. Among all self-adjoint extensions of $H^{(P,\alpha)}$, the Friedrichs extension is distinguished by minimality of the quadratic form and preservation of lower semiboundedness. This construction provides a canonical choice of self-adjoint realization, independent of auxiliary boundary conditions at infinity.

5.3 Domain of the Self-Adjoint Operator

The domain $D(A_{P,\alpha})$ consists of those $\varphi \in D(\overline{Q}_{P,\alpha})$ for which there exists $\chi \in \ell^2(\mathbb{N})$ such that

$$\overline{Q}_{P,\alpha}(\psi, \varphi) = \langle \psi, \chi \rangle \quad \text{for all } \psi \in D(\overline{Q}_{P,\alpha}).$$

In this case one sets $A_{P,\alpha}\varphi = \chi$. While an explicit pointwise characterization of $D(A_{P,\alpha})$ is generally difficult, this weak formulation suffices for spectral and resolvent analysis.

5.4 Inclusion of Diagonal Potentials

If a diagonal potential $V(n)$ is included and if V is relatively form-bounded with respect to $\overline{Q}_{P,\alpha}$ with relative bound strictly less than one, then the perturbed quadratic form

$$\overline{Q}_{P,\alpha}^V = \overline{Q}_{P,\alpha} + \sum_{n \in \mathbb{N}} V(n) |\psi(n)|^2$$

remains closed and lower semibounded.

In this case, the associated operator $A_{P,\alpha}^V$ is self-adjoint and represents a canonical perturbation of the prime-shift operator. This framework covers a wide class of arithmetic diagonal terms, including those appearing in SMRK-type Hamiltonians.

5.5 Uniqueness and Canonical Nature

The Friedrichs extension yields a unique self-adjoint operator associated with the quadratic form $\overline{Q}_{P,\alpha}$. No further choices are required, and the construction is stable under perturbations that preserve form boundedness.

This canonical nature is essential for later spectral and trace-based investigations, as it eliminates ambiguities arising from nonunique extensions.

5.6 Summary

The regularized prime-shift operators admit canonical self-adjoint realizations obtained via Friedrichs extension of their associated quadratic forms. This establishes a rigorous operator-theoretic foundation for arithmetic prime-shift dynamics.

In the next section we analyze the resolvent of the self-adjoint operator $A_{P,\alpha}$ and discuss basic spectral properties relevant for subsequent trace and identification results.

6 Resolvent and Spectral Properties

In this section we analyze the resolvent of the self-adjoint operators constructed in the previous section and record basic spectral properties relevant for later trace and identification arguments.

No explicit spectral decomposition is attempted; the focus is on existence, stability, and analytic control.

6.1 Existence of the Resolvent

Let $A_{P,\alpha}$ denote the self-adjoint operator obtained as the Friedrichs extension associated with the closed quadratic form $\overline{Q}_{P,\alpha}$. By self-adjointness, the resolvent

$$R_{P,\alpha}(z) = (A_{P,\alpha} - z)^{-1}$$

exists as a bounded operator on $\ell^2(\mathbb{N})$ for all $z \in \mathbb{C}$ with $\text{Im } z \neq 0$.

Moreover, the resolvent satisfies the standard estimate

$$\|R_{P,\alpha}(z)\| \leq \frac{1}{|\text{Im } z|},$$

which is uniform in the regularization parameters (P, α) as long as the lower bound of the quadratic form is controlled uniformly.

6.2 Lower Spectral Bound

Since the quadratic form $\overline{Q}_{P,\alpha}$ is lower semibounded, the spectrum of $A_{P,\alpha}$ is bounded from below. That is, there exists a constant $C_{P,\alpha} \in \mathbb{R}$ such that

$$\sigma(A_{P,\alpha}) \subset [C_{P,\alpha}, \infty).$$

In particular, after a constant energy shift if necessary, one may assume that $A_{P,\alpha}$ is nonnegative. This normalization is often convenient for trace and determinant constructions.

6.3 Stability Under Regularization

The family of operators $\{A_{P,\alpha}\}$ depends continuously on the regularization parameters in the strong resolvent sense. Specifically, if $(P_n, \alpha_n) \rightarrow (P, \alpha)$, then

$$(A_{P_n, \alpha_n} - z)^{-1} \xrightarrow{s} (A_{P, \alpha} - z)^{-1}$$

for all z with $\text{Im } z \neq 0$.

This stability follows from the corresponding convergence of the quadratic forms $\overline{Q}_{P_n, \alpha_n}$ to $\overline{Q}_{P, \alpha}$ and standard results on convergence of self-adjoint operators generated by closed forms.

6.4 Perturbations by Diagonal Potentials

If a diagonal potential $V(n)$ is included and is relatively form-bounded with respect to $\overline{Q}_{P,\alpha}$ with relative bound strictly less than one, then the perturbed operator $A_{P,\alpha}^V$ remains self-adjoint and lower semibounded.

In this case, the resolvent identity

$$(A_{P,\alpha}^V - z)^{-1} = (A_{P,\alpha} - z)^{-1} - (A_{P,\alpha} - z)^{-1}V(A_{P,\alpha}^V - z)^{-1}$$

holds in the sense of bounded operators. This relation provides the starting point for resolvent expansions used in trace formula derivations.

6.5 Spectral Measures

By the spectral theorem, the self-adjoint operator $A_{P,\alpha}$ admits a projection-valued spectral measure $E_{P,\alpha}(\lambda)$ such that

$$A_{P,\alpha} = \int_{\mathbb{R}} \lambda dE_{P,\alpha}(\lambda).$$

For any $\psi \in \ell^2(\mathbb{N})$, the scalar measure

$$\mu_\psi(\lambda) = \langle \psi, E_{P,\alpha}(\lambda) \psi \rangle$$

encodes the spectral distribution of $A_{P,\alpha}$ with respect to ψ .

While explicit expressions for these measures are not available at this stage, their existence is sufficient for defining trace-like objects via functional calculus in subsequent work.

6.6 Absence of Pathologies

The constructions above exclude several potential pathologies. In particular, the operators $A_{P,\alpha}$:

- have real spectrum,
- admit bounded resolvents off the real axis,
- depend stably on regularization parameters,
- allow controlled diagonal perturbations.

These properties are essential prerequisites for any attempt to derive trace formulas or to identify spectral data with arithmetic invariants.

6.7 Summary

The self-adjoint prime-shift operators constructed via quadratic form methods admit well-defined resolvents and stable spectral measures. Their analytic behavior is robust under regularization and under admissible perturbations.

With these results, the analytic foundations of prime-shift dynamics are firmly established. The subsequent development of trace formulas and spectral identification may therefore proceed on a rigorous operator-theoretic basis.

7 Discussion and Outlook

This paper has established a rigorous self-adjoint framework for a class of regularized prime-shift operators encoding multiplicative arithmetic dynamics. The construction proceeds through quadratic form methods and yields canonical self-adjoint realizations via Friedrichs extension.

The main objective has been analytic rather than arithmetic: to isolate and stabilize the operator-theoretic core of prime-shift dynamics in a setting where spectral and resolvent-based tools apply without ambiguity.

7.1 Summary of Results

The principal results obtained in this work may be summarized as follows:

- explicit regularized families of prime-shift operators were defined on $\ell^2(\mathbb{N})$,

- associated quadratic forms were shown to be densely defined, lower semibounded, and closable,
- canonical self-adjoint operators were constructed via Friedrichs extension,
- resolvents and spectral measures were shown to exist and to depend stably on regularization parameters,
- diagonal arithmetic potentials, including those of SMRK type, were shown to enter as admissible form-bounded perturbations.

These results provide a mathematically sound foundation for further spectral analysis.

7.2 What Has Not Been Addressed

Several questions lie deliberately outside the scope of this paper. In particular, no attempt has been made to:

- derive trace formulas or explicit spectral expansions,
- identify spectral data with zeros of zeta or L -functions,
- analyze fine spectral statistics or eigenvalue correlations.

These topics require additional structure beyond self-adjointness and will be addressed separately.

7.3 Role Within a Larger Program

The self-adjoint operators constructed here should be viewed as the analytic backbone of a broader operator-based arithmetic program. By isolating the symmetric and self-adjoint core of prime-shift dynamics, this work removes a major technical obstacle to subsequent investigations.

In particular, the existence of a canonical self-adjoint realization eliminates ambiguities associated with nonunique extensions and provides a stable platform for trace-based constructions.

7.4 Transition to Trace Formulas

With the resolvent and spectral measure in place, one may define regulated trace objects such as

$$\mathrm{Tr} f(A_{P,\alpha}), \quad \mathrm{Tr}(A_{P,\alpha} - z)^{-1},$$

for suitable test functions f . Expanding such traces in terms of arithmetic paths on the underlying graph leads naturally to explicit formulae involving prime sums.

The derivation and analysis of these trace formulas constitute the subject of the next paper in this series.

7.5 Outlook

The framework developed here establishes that prime-shift dynamics admits a well-defined and analytically controlled operator-theoretic formulation. This result alone does not resolve deep arithmetic conjectures, but it provides the necessary groundwork upon which such questions may be meaningfully posed.

In subsequent work, this foundation will be used to derive trace formulas, analyze spectral invariants, and investigate whether arithmetic information, including that associated with the Riemann zeta function, emerges from the spectral behavior of these operators.

APPENDIX A

A.1 Technical Lemmas and Supplementary Remarks

This appendix collects auxiliary results and technical remarks supporting the main analysis. No new assumptions are introduced, and all statements are compatible with the framework developed in the core sections.

A.1.1 Density of the Core Domain

The space $c_{00}(\mathbb{N})$ of finitely supported functions is dense in $\ell^2(\mathbb{N})$ and in all weighted spaces $\ell^2(\mathbb{N}, \mu)$ with strictly positive weights $\mu(n)$.

Density ensures that quadratic forms defined initially on $c_{00}(\mathbb{N})$ are uniquely determined by their closures and that Friedrichs extensions are canonical.

A.1.2 Absolute Convergence on the Core

For any $\psi \in c_{00}(\mathbb{N})$ and for all regularized weight choices $w_{P,\alpha}(p)$ considered in this paper, the sums

$$\sum_p w_{P,\alpha}(p) \psi(pn), \quad \sum_p w_{P,\alpha}(p) \mathbf{1}_{p|n} \psi(n/p)$$

are finite for each fixed n .

This guarantees that the regularized operators $H^{(P,\alpha)}$ are well defined as algebraic operators on the core domain.

A.1.3 Symmetry of the Quadratic Form

The symmetry of the quadratic form

$$Q_{P,\alpha}(\psi) = \langle \psi, H^{(P,\alpha)} \psi \rangle$$

follows from the pairing of forward and backward prime transitions. Explicitly, the change of variables $m = pn$ in the backward term yields

$$\sum_n \psi(n) \mathbf{1}_{p|n} \psi(n/p) = \sum_n \psi(pn) \psi(n),$$

which matches the complex conjugate of the forward contribution.

A.1.4 Form Bounds with Diagonal Potentials

Let $V : \mathbb{N} \rightarrow \mathbb{R}$ satisfy

$$|V(n)| \leq a + b n^\gamma$$

for some $a, b \geq 0$ and $\gamma < 1$. Then the diagonal form

$$\sum_n V(n) |\psi(n)|^2$$

is relatively form-bounded with respect to $Q_{P,\alpha}$ for all $\alpha > 1$, with arbitrarily small relative bound.

This estimate covers slowly varying arithmetic potentials such as $\log n$ and von Mangoldt-type terms restricted to regularized settings.

A.1.5 Strong Resolvent Convergence

Let $(P_k, \alpha_k) \rightarrow (P, \alpha)$ and assume uniform lower bounds on the quadratic forms Q_{P_k, α_k} . Then the associated operators satisfy

$$A_{P_k, \alpha_k} \xrightarrow{\text{s.r.}} A_{P, \alpha},$$

where convergence is in the strong resolvent sense.

This follows from standard results on convergence of closed quadratic forms and ensures stability of spectral quantities under regularization limits.

A.1.6 Energy Shifts

If the quadratic form $Q_{P, \alpha}$ is only semibounded from below by a constant $-C$, one may define the shifted form

$$Q_{P, \alpha}^{(+)}(\psi) = Q_{P, \alpha}(\psi) + C \|\psi\|^2,$$

which is nonnegative. The associated operator differs from $A_{P, \alpha}$ by an additive constant and has identical spectral projections.

Such shifts are implicitly assumed when invoking nonnegativity.

A.1.7 Preparatory Remarks for Trace Constructions

While the operators $A_{P, \alpha}$ are not trace class, the existence of resolvent and spectral measures allows the definition of regulated trace expressions of the form

$$\text{Tr } f(A_{P, \alpha}),$$

for suitable test functions f .

The expansion of such traces in terms of arithmetic paths and prime cycles forms the starting point of the trace formula analysis developed in the subsequent paper.

A.1.8 Closing Remark

The results collected here serve to clarify technical points and to ensure that all analytic constructions used in the main text rest on standard and well understood functional-analytic principles. No essential ideas are confined to this appendix.

Chapter 16

Trace Formulas for Prime-Shift Operators

Cycles, Resolvents, and Arithmetic Expansions

Enter Yourname

Abstract

This document develops trace-based diagnostics for arithmetic prime-shift operators, moving beyond operator existence toward the extraction of global spectral information. The underlying arithmetic dynamics is multiplicative and can be represented as motion on a directed graph generated by prime multiplication. Closed paths on this graph correspond to arithmetic cycles.

Because prime-shift operators are not trace class, the work emphasizes controlled regularization. Trace objects are defined through regulated finite-dimensional approximations or through suitable test functions that render operator functions trace class. Resolvent-based expansions are developed and interpreted combinatorially as sums over closed arithmetic cycles.

A natural decomposition into smooth and oscillatory contributions emerges, with the oscillatory part governed by nontrivial arithmetic cycles. Diagonal arithmetic potentials modify the weights of these cycles without changing their combinatorial structure. The significance of the results is structural rather than identificational: the analysis clarifies how prime-indexed sums arise naturally from operator-theoretic trace constructions and prepares the ground for later spectral determinant and identification frameworks.

1 Introduction and Scope

1.1 From Operators to Traces

The aim of this paper is to derive and analyze trace formulas associated with the self-adjoint prime-shift operators constructed in the preceding work. Whereas Whitepaper I established a rigorous analytic foundation—existence of canonical self-adjoint realizations and resolvents—the present paper focuses on extracting arithmetic information from trace objects built from these operators.

The guiding principle is that arithmetic structure enters spectral quantities through closed dynamical paths. In the context of prime-shift dynamics, such paths correspond to closed cycles on a multiplicative arithmetic graph. Trace formulas make this correspondence explicit.

1.2 From Operators to Traces

Given a self-adjoint operator A on a Hilbert space, trace expressions of the form

$$\mathrm{Tr} f(A), \quad \mathrm{Tr} (A - z)^{-1},$$

encode global spectral information. When A is generated by arithmetic transitions, these traces may be expanded in terms of the underlying dynamical structure.

For prime-shift operators, the relevant dynamics is multiplicative. The trace therefore decomposes naturally into contributions from closed arithmetic paths, each weighted by the product of transition amplitudes along the path.

1.3 Regularization and Functional Calculus

Prime-shift operators are not trace-class, and their resolvents are not trace-class either. Consequently, all trace expressions must be defined in a regulated sense. Throughout this paper, traces are understood as limits of traces of regularized operators or via test functions f chosen so that $f(A)$ is trace-class.

Regularization is not treated as an auxiliary device, but as an intrinsic part of the construction. All limits and interchanges of summation are justified within this controlled framework.

1.4 Arithmetic Graphs and Cycles

The arithmetic graph underlying prime-shift dynamics has vertex set $\mathbb{N}$ and directed edges $n \rightarrow pn$ for primes p . Closed paths on this graph correspond to multiplicative identities formed by products of primes.

Primitive cycles correspond to minimal closed paths, while longer cycles arise as their iterates. This structure parallels that of periodic orbits in classical dynamical systems and underlies the organization of the trace expansion.

1.5 Strategy of the Paper

The analysis proceeds in several steps. First, we define regulated trace objects using resolvents and functional calculus. Next, we express these traces as sums over closed arithmetic cycles. We then isolate prime-indexed contributions and analyze how diagonal arithmetic potentials enter the expansion.

The final sections compare the resulting trace formulas with classical explicit formulas in analytic number theory, emphasizing structural similarities rather than direct identification.

1.6 Relation to SMRK-Type Hamiltonians

The trace framework developed here naturally accommodates Hamiltonians that extend the symmetric prime-shift core by diagonal arithmetic terms. In particular, SMRK-type Hamiltonians enter trace expressions through controlled perturbative contributions, without altering the combinatorial organization of cycles.

This separation of core dynamics and arithmetic weighting is essential for transparent trace analysis.

1.7 Scope and Limitations

This paper does not attempt to identify the derived trace formulas with those of the Riemann zeta function or to draw conclusions regarding the Riemann Hypothesis. Its objective is more modest and more precise: to demonstrate how arithmetic information emerges in trace expansions of prime-shift operators.

The question of spectral identification is deferred to subsequent work.

1.8 Organization of the Paper

Section 2 introduces regulated trace objects associated with self-adjoint prime-shift operators. Section 3 analyzes the structure of arithmetic graphs and closed cycles. Section 4 develops resolvent expansions. Section 5 isolates prime cycle contributions. Section 6 presents the resulting trace formulas. Section 7 compares these formulas with classical explicit formulas. The final section discusses implications for spectral identification.

2 From Self-Adjoint Operators to Trace Objects

In this section we introduce the trace objects that will be used throughout the paper. Since prime-shift operators are not trace-class, traces must be defined in a regulated sense using functional calculus or resolvent-based constructions. All definitions are formulated within the self-adjoint framework established in Whitepaper I.

2.1 Trace-Class Operators via Functional Calculus

In this section we introduce the trace objects that will be used throughout the paper. Since prime-shift operators are not trace-class, traces must be defined in a regulated sense using functional calculus or resolvent-based constructions. All definitions are formulated within the self-adjoint framework established in Whitepaper I.

Let A denote a self-adjoint prime-shift operator obtained as a Friedrichs extension of a regularized quadratic form. For a bounded Borel function $f : \mathbb{R} \rightarrow \mathbb{C}$, the operator $f(A)$ is defined by the spectral theorem.

If f is chosen so that $f(A)$ is trace-class, one may define

$$\mathrm{Tr} f(A) = \sum_j \langle e_j, f(A) e_j \rangle,$$

where $\{e_j\}$ is any orthonormal basis of the Hilbert space.

Typical admissible choices include rapidly decaying test functions such as $f(\lambda) = e^{-t\lambda}$ for $t > 0$, or compactly supported smooth functions.

2.2 Heat Trace as a Prototypical Example

A central example is the heat trace

$$\Theta(t) = \mathrm{Tr} e^{-tA}, \quad t > 0,$$

which is well-defined whenever e^{-tA} is trace-class. The small- t behavior of $\Theta(t)$ encodes high-energy spectral information, while its large- t behavior probes low-lying spectrum.

In the arithmetic context, the heat trace admits an expansion in terms of closed arithmetic paths, making it a natural object for trace formula analysis.

2.3 Resolvent-Based Traces

An alternative and often more flexible approach is based on the resolvent

$$R(z) = (A - z)^{-1}, \quad \text{Im } z \neq 0.$$

While $R(z)$ itself is not trace-class, regulated expressions of the form

$$\text{Tr} (R(z) - R(z_0)),$$

or higher powers $R(z)^k$ with suitable subtraction, may be trace-class. Such objects are closely related to spectral shift functions and determinant-type constructions.

Resolvent traces are particularly well suited for expansions in terms of arithmetic cycles, as they admit Neumann-series representations in regularized settings.

2.4 Regularization and Cutoffs

All trace objects considered here are defined with explicit regularization. Typical schemes include:

- finite-dimensional truncation of the Hilbert space,
- damping of prime weights,
- subtraction of reference operators.

The precise regularization is chosen so that:

1. the trace object is well-defined,
2. expansions converge absolutely or conditionally in a controlled manner,
3. limits may be taken after summation.

Regularization is therefore treated as part of the definition of the trace object, not as an afterthought.

2.5 Cyclicity of the Trace

A key structural property is cyclicity:

$$\text{Tr}(BC) = \text{Tr}(CB),$$

whenever both products are trace-class. This property allows the reorganization of trace expressions into sums over closed paths, as intermediate indices may be cyclically permuted.

In the context of prime-shift operators, cyclicity is responsible for the appearance of closed arithmetic cycles in trace expansions.

2.6 Traces and Arithmetic Graphs

Let $\{e_n\}_{n \in \mathbb{N}}$ denote the canonical basis of $\ell^2(\mathbb{N})$. For trace-class operators T , one has

$$\mathrm{Tr} T = \sum_{n \in \mathbb{N}} \langle e_n, T e_n \rangle.$$

When T is constructed from prime-shift operators, each diagonal matrix element $\langle e_n, T e_n \rangle$ corresponds to a sum over arithmetic paths starting and ending at n . Summing over n therefore counts closed paths on the arithmetic graph.

This observation forms the combinatorial backbone of the trace formula.

2.7 Preparatory Remarks

At this stage, no explicit cycle expansion is performed. The purpose of this section is to fix trace objects and regularization schemes in a manner compatible with self-adjointness and spectral theory.

In the next section, we turn to the structure of arithmetic graphs and classify the closed cycles that contribute to trace expansions.

3 Arithmetic Graphs and Closed Cycles

In this section we analyze the structure of closed arithmetic cycles underlying prime-shift dynamics. These cycles form the combinatorial skeleton of all trace expansions considered in this work.

3.1 The Arithmetic Graph

Recall that the arithmetic graph has vertex set $\mathbb{N}$ and directed edges

$$n \longrightarrow pn, \quad p \in \mathbb{P}.$$

Backward transitions $pn \rightarrow n$ are included implicitly through the inverse prime-shift operators. The graph is infinite, locally finite under prime regularization, and encodes multiplicative structure rather than additive geometry.

3.2 Paths and Closed Cycles

A finite arithmetic path of length k is a sequence of vertices

$$n_0 \rightarrow n_1 \rightarrow \cdots \rightarrow n_k,$$

where each transition corresponds to multiplication or division by a prime. A closed cycle is a path with $n_k = n_0$.

Closed cycles correspond to multiplicative identities of the form

$$p_1^{\varepsilon_1} p_2^{\varepsilon_2} \cdots p_k^{\varepsilon_k} = 1, \quad \varepsilon_j \in \{+1, -1\}.$$

3.3 Trivial and Nontrivial Cycles

The simplest cycles are trivial backtracking cycles

$$n \rightarrow pn \rightarrow n,$$

corresponding to immediate multiplication and division by the same prime. Such cycles contribute to local, typically divergent terms and are handled by regularization.

Nontrivial cycles involve sequences of distinct prime transitions and encode genuine multiplicative relations. These cycles are responsible for oscillatory terms in trace expansions.

3.4 Primitive Cycles

A closed cycle is called primitive if it is not a repetition of a shorter cycle. Formally, a cycle γ of length k is primitive if there does not exist a cycle γ_0 of length $k_0 < k$ and an integer $m \geq 2$ such that $\gamma = \gamma_0^m$.

Primitive cycles play a fundamental role in organizing trace expansions, as all cycles may be decomposed uniquely into iterates of primitive ones.

3.5 Cycle Weights

Each arithmetic cycle carries a weight given by the product of transition weights along the cycle. For a cycle γ involving primes $p_1, \dots, p_k$, the weight takes the form

$$W(\gamma) = \prod_{j=1}^k w(p_j),$$

possibly modified by diagonal potential contributions if present.

In regularized settings, these weights decay sufficiently fast to ensure convergence of cycle sums.

3.6 Base-Point Independence

A crucial feature of closed cycles is base-point independence. Although cycles are represented as paths starting and ending at a particular vertex n , their weight depends only on the sequence of prime transitions, not on the choice of n .

This property underlies the factorization of trace expansions into a sum over cycle classes rather than over individual vertices.

3.7 Orientation and Reversal

Cycles related by reversal of orientation correspond to complex conjugate contributions in trace expansions. For symmetric operators with real weights, these contributions combine to produce real-valued trace terms.

This pairing mirrors the time-reversal symmetry familiar from dynamical trace formulas.

3.8 Length and Multiplicity

The length of a cycle is defined as the number of prime transitions it contains. In trace expansions, cycles of increasing length are increasingly suppressed by regularization, allowing controlled summation.

Multiplicity arises from the number of distinct embeddings of a given cycle into the arithmetic graph, a feature that will be addressed explicitly in the trace formula.

3.9 Summary

Closed arithmetic cycles provide the natural combinatorial language for trace expansions of prime-shift operators. The classification into trivial, primitive, and iterated cycles allows systematic organization of trace contributions.

In the next section, we develop resolvent expansions and show how traces may be expressed explicitly as sums over such cycles.

4 Resolvent Expansions

In this section we develop resolvent expansions for regularized prime-shift operators and show how these expansions lead naturally to sums over closed arithmetic cycles. The resolvent provides a flexible analytic tool linking operator theory with combinatorial path structures.

4.1 Resolvent Decomposition

Let A denote a self-adjoint prime-shift operator with regularization parameters suppressed from the notation. We write

$$A = A_0 + B,$$

where A_0 is a diagonal reference operator and B collects the off-diagonal prime-shift transitions. Typical choices of A_0 include diagonal arithmetic potentials or simply a constant multiple of the identity, chosen so that $(A_0 - z)^{-1}$ is explicitly known.

4.2 Neumann Series Expansion

For $z \in \mathbb{C}$ with sufficiently large $|\operatorname{Im} z|$, the resolvent admits the Neumann series expansion

$$(A - z)^{-1} = (A_0 - z)^{-1} \sum_{k=0}^{\infty} (-B(A_0 - z)^{-1})^k.$$

Convergence of the series is ensured by the regularization of prime weights and the resolvent bound

$$\|(A_0 - z)^{-1}B\| < 1.$$

4.3 Trace of the Resolvent

Taking the trace of the resolvent expansion yields

$$\operatorname{Tr} (A - z)^{-1} = \sum_{k=0}^{\infty} (-1)^k \operatorname{Tr} ((A_0 - z)^{-1}B)^k (A_0 - z)^{-1}.$$

The $k = 0$ term corresponds to the trace of the reference resolvent and contains no arithmetic information. All arithmetic contributions arise from terms with $k \geq 1$.

4.4 Matrix Elements and Paths

Let $\{e_n\}_{n \in \mathbb{N}}$ denote the canonical basis of $\ell^2(\mathbb{N})$. Each matrix element of the form

$$\langle e_n, (A_0 - z)^{-1} B e_m \rangle$$

is nonzero only if n and m are connected by a single prime transition.

Iterated products therefore correspond to arithmetic paths:

$$\langle e_n, ((A_0 - z)^{-1} B)^k e_n \rangle = \sum_{\gamma: n \rightarrow n, |\gamma|=k} W_z(\gamma),$$

where the sum runs over closed arithmetic paths γ of length k starting and ending at n , and $W_z(\gamma)$ denotes the weight of the path including resolvent factors.

4.5 Closed Cycles from Trace Cyclicity

Summing over n and using cyclicity of the trace yields

$$\text{Tr} ((A_0 - z)^{-1} B)^k = \sum_{\gamma \in \mathcal{C}_k} W_z(\gamma),$$

where $\mathcal{C}_k$ denotes the set of closed arithmetic cycles of length k , counted up to cyclic permutation. This establishes a direct correspondence between resolvent trace terms and closed arithmetic cycles.

4.6 Primitive Cycle Decomposition

As discussed in Section 3, all closed cycles may be decomposed uniquely into iterates of primitive cycles. Accordingly, the resolvent trace expansion may be reorganized as a sum over primitive cycles γ_0 and their repetitions:

$$\sum_{\gamma} W_z(\gamma) = \sum_{\gamma_0} \sum_{m=1}^{\infty} \frac{1}{m} W_z(\gamma_0)^m,$$

where the factor $1/m$ arises from cyclic symmetry.

This structure parallels the classical expansion of logarithmic determinants in terms of primitive periodic orbits.

4.7 Convergence Considerations

Regularization ensures that contributions from long cycles are suppressed. Specifically, damping of prime weights implies exponential decay of $W_z(\gamma)$ with cycle length.

As a result, the cycle expansion converges absolutely for sufficiently large $|\text{Im } z|$ and admits analytic continuation under controlled limits.

4.8 Summary

Resolvent expansions provide a transparent mechanism by which trace expressions decompose into sums over closed arithmetic cycles. This correspondence forms the analytic core of the trace formula.

In the next section, we isolate the contributions of prime-indexed cycles and analyze their arithmetic structure in detail.

5 Prime Contributions and Cycle Decomposition

In this section we isolate the contributions of prime-indexed cycles in the resolvent trace expansion and analyze their arithmetic structure. These cycles form the fundamental building blocks of the trace formula and are responsible for explicit prime sums appearing in later expressions.

5.1 Prime Cycles as Fundamental Generators

A prime cycle is defined as a closed arithmetic cycle whose transitions involve a single prime p . The simplest nontrivial prime cycle has length two:

$$n \longrightarrow pn \longrightarrow n.$$

Longer prime cycles correspond to iterated traversals of this basic loop and are naturally organized as powers of a primitive cycle.

Prime cycles represent the minimal closed dynamical processes generated by a single prime and therefore serve as the elementary contributors to prime sums in trace formulas.

5.2 Weight of a Prime Cycle

Let γ_p denote the primitive cycle associated with a prime p . The weight of γ_p in the resolvent expansion takes the form

$$W_z(\gamma_p) = w(p)^2 \langle e_n, (A_0 - z)^{-1} e_n \rangle \langle e_{pn}, (A_0 - z)^{-1} e_{pn} \rangle,$$

where A_0 is the diagonal reference operator introduced earlier.

Due to base-point independence, this weight depends only on p and on the resolvent factors, not on the specific choice of n .

5.3 Iterated Prime Cycles

Iterates of the primitive prime cycle contribute terms of the form

$$W_z(\gamma_p^m) = (W_z(\gamma_p))^m, \quad m \geq 1.$$

Summing over all iterates yields the familiar logarithmic structure

$$\sum_{m=1}^{\infty} \frac{1}{m} W_z(\gamma_p)^m = -\log(1 - W_z(\gamma_p)),$$

which is characteristic of determinant and trace-log expansions.

This structure mirrors the appearance of prime powers in classical explicit formulas.

5.4 Contribution to the Resolvent Trace

Collecting the contributions of all prime cycles, the prime part of the resolvent trace takes the schematic form

$$\mathrm{Tr}(A - z)^{-1}|_{\text{primes}} = \sum_{p \in \mathbb{P}} \sum_{m=1}^{\infty} \frac{1}{m} W_z(\gamma_p)^m,$$

up to regularization-dependent smooth terms.

This expression exhibits an explicit sum over primes and their iterates, making the arithmetic content of the trace expansion manifest.

5.5 Inclusion of Diagonal Arithmetic Potentials

Let $V(n)$ be a diagonal arithmetic potential, such as those appearing in SMRK-type Hamiltonians. In the resolvent framework, V enters through the reference operator A_0 or as a perturbative correction.

The effect of V on a prime cycle is multiplicative: each traversal contributes a factor depending on $V(n)$ and $V(pn)$ through the resolvent terms $(A_0 - z)^{-1}$. Importantly, V does not alter the combinatorial classification of cycles.

Thus, SMRK-type potentials weight prime cycles but do not create new cycle types.

5.6 Separation of Smooth and Oscillatory Terms

Prime-cycle contributions are oscillatory in nature, reflecting the discrete multiplicative structure of primes. By contrast, trivial cycles and reference resolvent terms contribute smooth background components.

This separation underlies the structure of explicit formulas, where smooth spectral terms are accompanied by oscillatory prime sums.

5.7 Comparison with Classical Prime Sums

The organization of prime-cycle contributions closely parallels the structure of classical prime sums of the form

$$\sum_p \sum_{m=1}^{\infty} \frac{1}{m} g(p^m),$$

appearing in explicit formulas in analytic number theory.

Here, the function g is determined by resolvent factors and regularization choices, providing an operator-theoretic origin for such sums.

5.8 Summary

Prime cycles constitute the elementary arithmetic contributions to trace expansions of prime-shift operators. Their weights generate explicit sums over primes and their powers, while diagonal arithmetic potentials enter as controlled multiplicative factors.

In the next section, we assemble these contributions into full trace formulas and make their structural similarity to classical explicit formulas precise.

6 Trace Formulas and Explicit Arithmetic Terms

In this section we assemble the contributions derived in the preceding sections into a full trace formula for regularized prime-shift operators. The resulting expression separates naturally into a smooth spectral component and an oscillatory component arising from prime cycles.

6.1 Structure of the Trace Expansion

Let A denote a regularized self-adjoint prime-shift operator and let $T(z)$ denote a regulated trace object, such as

$$T(z) = \text{Tr} (A - z)^{-1} \quad \text{or} \quad T_f = \text{Tr} f(A),$$

with f chosen so that $f(A)$ is trace-class.

Using the resolvent expansion and cycle decomposition developed earlier, the trace may be written schematically as

$$T = T_{\text{smooth}} + T_{\text{osc}},$$

where T_{smooth} collects contributions from trivial cycles and reference resolvents, while T_{osc} encodes nontrivial arithmetic cycles.

6.2 Smooth Spectral Term

The smooth term arises from the $k = 0$ part of the resolvent expansion and from local backtracking cycles. It depends on the choice of reference operator A_0 and on regularization parameters, but does not carry explicit prime structure.

Formally, one may write

$$T_{\text{smooth}} = \text{Tr} (A_0 - z)^{-1} + C_{\text{reg}}(z),$$

where $C_{\text{reg}}(z)$ denotes regularization-dependent correction terms.

This component corresponds to the smooth density of states in classical trace formulas.

6.3 Oscillatory Prime Contribution

The oscillatory term is generated by nontrivial closed arithmetic cycles. Using the prime-cycle analysis of Section 5, it admits the expansion

$$T_{\text{osc}} = \sum_{p \in \mathbb{P}} \sum_{m=1}^{\infty} \frac{1}{m} W_z(\gamma_p)^m,$$

where γ_p denotes the primitive cycle associated with the prime p and $W_z(\gamma_p)$ is its cycle weight. This expression converges absolutely under regularization and makes explicit the sum over primes and their iterates.

6.4 Rewriting in Explicit-Formula Form

Introducing the logarithmic representation

$$\sum_{m=1}^{\infty} \frac{1}{m} x^m = -\log(1 - x),$$

the oscillatory term may be written compactly as

$$T_{\text{osc}} = - \sum_{p \in \mathbb{P}} \log(1 - W_z(\gamma_p)).$$

This form closely parallels Euler product structures and highlights the multiplicative nature of the trace expansion.

6.5 Effect of Diagonal Arithmetic Potentials

If diagonal arithmetic potentials $V(n)$ are included, the cycle weights $W_z(\gamma_p)$ are modified by resolvent factors involving $V(n)$ and $V(pn)$. The overall structure of the trace formula remains unchanged.

In particular, diagonal terms influence the functional form of $W_z(\gamma_p)$ but do not alter the separation into smooth and oscillatory components.

6.6 Regularization Dependence

Both T_{smooth} and T_{osc} depend on regularization parameters. However, the decomposition itself is stable, and variations in regularization affect only the detailed form of the smooth term and the decay properties of cycle weights.

This mirrors the role of test functions in classical explicit formulas, where regularization controls convergence but not structural content.

6.7 Interpretation

The trace formula derived here provides an operator-theoretic origin for explicit prime sums. The smooth term encodes global spectral background, while the oscillatory term captures arithmetic fluctuations driven by prime cycles.

The formula does not assert identification with any specific zeta or L -function. Its significance lies in demonstrating that prime-indexed oscillatory terms arise naturally from trace expansions of prime-shift dynamics.

6.8 Summary

We have derived a trace formula for regularized prime-shift operators that decomposes into a smooth spectral component and an oscillatory prime-cycle component. The structure of the oscillatory term parallels classical explicit formulas in analytic number theory.

In the next section, we compare this operator-derived trace formula with classical explicit formulas, emphasizing structural similarities and differences.

7 Relation to Explicit Formulas

In this section we compare the trace formula derived in the previous section with classical explicit formulas in analytic number theory. The comparison is structural rather than identificatory: our objective is to highlight parallels in organization and interpretation, not to assert equality with any specific zeta or L -function.

7.1 Classical Explicit Formulas: A Brief Reminder

Classical explicit formulas, originating with Riemann and refined by Weil, relate sums over spectral data (zeros of zeta or L -functions) to sums over primes. In their generic form, these formulas decompose into:

- a smooth main term depending on a test function,
- oscillatory contributions indexed by primes and their powers,
- correction terms reflecting normalization and regularization choices.

The defining feature of such formulas is the dual appearance of spectral and arithmetic data within a single identity.

7.2 Structural Decomposition

The operator-derived trace formula obtained in Section 6 admits an analogous decomposition:

$$T = T_{\text{smooth}} + T_{\text{osc}},$$

where T_{smooth} represents a background spectral contribution and T_{osc} is an explicit sum over primes and their iterates.

This separation mirrors the smooth-plus-oscillatory structure characteristic of classical explicit formulas.

7.3 Role of Test Functions and Regularization

In classical explicit formulas, convergence and emphasis are controlled by the choice of test function and by contour deformation. In the operator framework, the analogous role is played by regularization parameters and by the choice of trace object, such as heat traces or resolvent traces.

Both frameworks rely on controlled smoothing to render otherwise divergent sums meaningful, without altering the essential arithmetic content.

7.4 Prime Powers and Cycle Iterates

The appearance of prime powers in classical formulas corresponds directly to the contribution of iterated prime cycles in the trace expansion. In both cases, primitive objects generate an infinite tower of higher-order contributions organized by iteration.

This correspondence reinforces the interpretation of prime cycles as the dynamical origin of prime-power terms.

7.5 Spectral Variables

In classical explicit formulas, spectral variables arise from zeros of analytic functions. In the operator framework, spectral variables appear as eigenvalues or as parameters in resolvent traces.

While the nature of the spectral data differs, the formal role played by these variables in balancing arithmetic contributions is closely analogous.

7.6 Absence of Direct Identification

It is important to emphasize that the present comparison does not establish an identity between the operator-derived trace formula and any specific classical explicit formula. In particular, no claim is made that the spectrum of the prime-shift operator coincides with zeros of the Riemann zeta function.

The significance of the comparison lies in demonstrating that the operator framework naturally reproduces the same structural ingredients that classical explicit formulas assemble by analytic continuation.

7.7 Interpretive Implications

The emergence of explicit prime sums from operator traces suggests that trace formulas may be understood as a general mechanism by which arithmetic structure enters spectral quantities.

From this viewpoint, classical explicit formulas appear as special instances of a broader operator-theoretic principle.

This perspective motivates the search for operators whose spectral data may be meaningfully compared or identified with arithmetic invariants.

7.8 Summary

The trace formula derived from prime-shift operators shares the defining structural features of classical explicit formulas: a decomposition into smooth and oscillatory components, explicit prime-power contributions, and dependence on regularization or test functions.

This structural correspondence provides strong motivation for further investigation, while stopping short of any claim of spectral identification. In the final section, we discuss how these results prepare the ground for such investigations in subsequent work.

8 Discussion and Transition to Spectral Identification

This paper has developed trace formulas associated with self-adjoint prime-shift operators by systematically expanding regulated trace objects into sums over closed arithmetic cycles. The analysis builds directly on the operator-theoretic foundation established in the preceding work and isolates the precise mechanism by which arithmetic information enters trace expressions.

8.1 Summary of Results

The main results of this paper may be summarized as follows:

- regulated trace objects were defined using functional calculus and resolvent methods for self-adjoint prime-shift operators,
- resolvent expansions were shown to decompose naturally into sums over closed arithmetic cycles on the multiplicative graph,
- primitive cycles and their iterates were identified as the fundamental contributors to oscillatory trace terms,
- prime-indexed cycles were isolated and shown to generate explicit sums over primes and prime powers,
- diagonal arithmetic potentials, including SMRK-type terms, were incorporated as controlled multiplicative weights without altering cycle structure,
- the resulting trace formula was shown to decompose into smooth and oscillatory components paralleling classical explicit formulas.

These results establish a transparent operator-theoretic origin for prime sums appearing in explicit formulas.

8.2 What the Trace Formula Does and Does Not Do

The trace formulas derived here demonstrate how arithmetic structure emerges from operator traces, but they do not by themselves identify any specific arithmetic function. In particular:

- no identification with the Riemann zeta function or its zeros has been made,

- no claim regarding the Riemann Hypothesis follows from the present analysis,
- the spectral interpretation of trace variables remains open.

These limitations are intrinsic to the scope of trace analysis alone.

8.3 Interpretive Significance

The central conceptual outcome of this paper is the demonstration that explicit prime sums arise naturally from trace expansions of prime-shift dynamics. This observation reframes classical explicit formulas as manifestations of a more general operator-theoretic principle rather than as isolated analytic identities.

From this perspective, trace formulas are not ad hoc tools but structural bridges between dynamics, spectrum, and arithmetic.

8.4 Preparation for Spectral Identification

Having isolated the precise form of arithmetic contributions in trace expressions, the remaining task is to relate these traces to spectral data in a way that permits identification with known arithmetic objects.

This requires:

- defining suitable spectral determinants or zeta functions associated with the operator,
- establishing analytic continuation and functional equations where appropriate,
- comparing the resulting spectral invariants with classical arithmetic functions.

These questions lie beyond trace expansion itself and require a separate, dedicated analysis.

8.5 Transition to the Next Stage

The next paper in this series addresses the problem of spectral identification. Building on the self-adjoint operators of Whitepaper I and the trace formulas developed here, it investigates whether the spectral invariants of prime-shift operators may be meaningfully identified with classical arithmetic functions.

In particular, the focus will shift from trace expansions to determinants, spectral zeta functions, and the precise role of self-adjointness in constraining the location of spectral data.

8.6 Closing Remarks

Trace formulas provide a powerful lens through which arithmetic dynamics may be viewed. By deriving such formulas from first principles within an operator framework, this paper clarifies both their scope and their limitations.

The results obtained here do not resolve deep arithmetic conjectures, but they establish the essential intermediate layer connecting operator dynamics with explicit arithmetic structure. This layer is indispensable for any serious attempt at spectral identification.

APPENDIX A

A.1 Trace Estimates and Convergence Control

This appendix collects technical estimates and convergence arguments supporting the trace expansions developed in the main text. All statements are formulated within the regularized framework established throughout the paper.

A.1.1 Trace-Class Conditions

Let A be a self-adjoint prime-shift operator and let f be a bounded Borel function such that $f(A)$ is trace class. Sufficient conditions for trace-class behavior include rapid decay of f at infinity or compact spectral support.

In particular, for heat-kernel-type functions $f(\lambda) = e^{-t\lambda}$ with $t > 0$, the operator e^{-tA} is trace class under the regularizations considered here.

A.1.2 Resolvent Differences

While the resolvent $(A - z)^{-1}$ is not trace class, differences of resolvents

$$(A - z)^{-1} - (A - z_0)^{-1}$$

are trace class for suitable choices of z, z_0 and regularization parameters. This follows from the resolvent identity and the boundedness of diagonal reference resolvents.

Such differences provide a natural definition of regulated resolvent traces.

A.1.3 Bounds on Cycle Weights

Let γ be a closed arithmetic cycle of length k involving primes $p_1, \dots, p_k$. The associated cycle weight satisfies an estimate of the form

$$|W_z(\gamma)| \leq C(z)^k \prod_{j=1}^k |w(p_j)|,$$

where $C(z)$ depends only on resolvent bounds.

Under exponential or polynomial damping of prime weights, this estimate implies absolute convergence of cycle sums.

A.1.4 Suppression of Long Cycles

Regularization ensures exponential or superpolynomial suppression of long cycles. As a result, sums over cycles of length k converge uniformly in k for sufficiently strong damping.

This suppression justifies term-by-term summation and rearrangement of cycle expansions.

A.1.5 Justification of Primitive Cycle Decomposition

The reorganization of cycle sums into primitive cycles and their iterates relies on absolute convergence of the original expansion. Under the estimates above, this condition is satisfied, and the decomposition

$$\sum_{\gamma} W_z(\gamma) = \sum_{\gamma_0} \sum_{m=1}^{\infty} \frac{1}{m} W_z(\gamma_0)^m$$

is valid.

The factor $1/m$ accounts for cyclic symmetry and prevents overcounting.

A.1.6 Interchange of Limits and Traces

Let $\{A_{P,\alpha}\}$ be a family of regularized operators converging in the strong resolvent sense. For trace-class test functions f , one has

$$\lim_{P \rightarrow \infty} \text{Tr } f(A_{P,\alpha}) = \text{Tr } f(A),$$

provided uniform trace bounds hold.

This justifies taking regularization limits after performing trace expansions.

A.1.7 Relation to Determinant Expansions

The logarithmic structure arising from iterated prime cycles mirrors the expansion of logarithmic determinants:

$$\log \det(1 - K) = - \sum_{m=1}^{\infty} \frac{1}{m} \text{Tr } K^m,$$

for trace-class operators K .

In the present context, K is replaced by a regularized prime-cycle transfer operator, making this analogy precise at the formal level.

A.1.8 Closing Remark

The estimates collected in this appendix ensure that all trace expansions used in the main text are mathematically well controlled. They provide the technical foundation required for reorganizing resolvent traces into arithmetic cycle sums, without introducing assumptions beyond regularization and self-adjointness.

APPENDIX B

B.1 Regularization Schemes and Interchange of Limits

This appendix summarizes the regularization schemes employed throughout the paper and justifies the interchange of summation, trace, and limiting operations used in the derivation of trace formulas. The goal is to make explicit the analytic assumptions underlying convergence and stability.

B.1.1 Prime Weight Regularization

Prime-shift operators involve infinite sums over primes. To control divergence, we employ weight functions $w(p)$ satisfying

$$\sum_{p \in \mathbb{P}} |w(p)| < \infty \quad \text{or} \quad \sum_{p \in \mathbb{P}} |w(p)|^2 < \infty,$$

depending on the context.

Typical choices include:

- exponential damping $w(p) = p^{-1}e^{-p/P}$,
- polynomial damping $w(p) = p^{-\alpha}$ with $\alpha > 1$,
- compact prime cutoffs $w(p) = 0$ for $p > P$.

All results in the main text are formulated uniformly with respect to such regularizations.

B.1.2 Operator Truncation

In addition to damping prime weights, we may truncate the Hilbert space to finite-dimensional subspaces spanned by $\{e_n : n \leq N\}$. This yields finite matrices for which all trace expressions are unambiguous.

Limits $N \rightarrow \infty$ are taken only after convergence of trace expressions is established uniformly in N .

B.1.3 Reference Operators and Subtraction

Resolvent-based trace objects are defined relative to a diagonal reference operator A_0 . Subtraction of $(A_0 - z)^{-1}$ removes divergent background contributions and isolates arithmetic content.

This subtraction is analogous to renormalization in quantum field theory and is essential for defining finite trace expressions.

B.1.4 Justification of Limit Interchange

Let $\{A_P\}$ be a family of regularized operators converging in the strong resolvent sense to an operator A . For test functions f such that $f(A_P)$ is trace class with uniformly bounded trace norm, one has

$$\lim_{P \rightarrow \infty} \text{Tr } f(A_P) = \text{Tr } f(A).$$

Similarly, sums over cycles may be interchanged with traces and limits provided absolute convergence is ensured by weight damping.

B.1.5 Regularization Independence of Structure

While numerical values of trace expressions depend on regularization choices, their structural decomposition into smooth and oscillatory components does not. In particular:

- the classification of cycles is independent of regularization,
- the identification of primitive cycles and their iterates is stable,
- prime-indexed contributions persist across regularization schemes.

This robustness justifies focusing on structural features rather than on specific regularized values.

B.1.6 Compatibility with Diagonal Perturbations

Diagonal arithmetic potentials enter trace expressions through resolvent factors. Provided such potentials are relatively form-bounded with respect to the prime-shift operator, they do not interfere with convergence or limit interchange.

This includes logarithmic and von Mangoldt-type terms used in SMRK Hamiltonians.

B.1.7 Role in the Overall Program

Regularization plays a conceptual role beyond technical necessity. It provides a controlled environment in which arithmetic dynamics may be interrogated without imposing artificial periodicity or symmetry.

The dependence of trace expressions on regularization parameters serves as a diagnostic tool, distinguishing structural arithmetic effects from artifacts of truncation.

B.1.8 Closing Remark

The regularization schemes described here ensure that all trace formulas derived in the main text are mathematically well defined and stable under controlled limits. They provide the analytic discipline required for subsequent spectral identification and determinant constructions.

Chapter 17

Spectral Identification and the Riemann Hypothesis

Determinants, Spectral Zeta Functions and Self-Adjoint Arithmetic Hamiltonians

Enter Yourname

Abstract

This document addresses the spectral identification stage of the program, focusing on how operator-theoretic trace information can be converted into spectral invariants suitable for comparison with classical arithmetic zeta functions. The guiding idea is that a well-defined arithmetic Hamiltonian may admit a spectral zeta function derived from regulated heat-trace constructions, together with associated determinant-like quantities.

The work formulates the analytic conditions under which such spectral invariants are meaningful. These include the existence of controlled trace objects, stability of short-time asymptotics, and robustness under admissible perturbations. Symmetry constraints analogous to functional equations are treated as essential structural requirements rather than optional features.

Regularization ambiguities are carefully separated from meaningful spectral information. While normalization factors may depend on cutoff choices, the location and stability of spectral zeros are identified as the primary diagnostic features. Failures of analytic continuation or instability under perturbation are interpreted as genuine obstructions rather than technical defects. The document thus defines a set of necessary criteria for any operator intended to serve as a Hilbert–Pólya-type candidate.

1 Introduction and Strategy

The purpose of this paper is to address the problem of *spectral identification* for arithmetic operators constructed in the preceding works. While Whitepaper I established the existence of canonical self-adjoint realizations of prime-driven Hamiltonians, and Whitepaper II derived trace formulas exposing explicit arithmetic structure, the present work focuses on the question of identification and constraint at the level of spectral invariants.

The emphasis therefore shifts from analytic existence and trace expansion toward spectral structure, normalization, and obstruction. The central problem is no longer whether a meaningful arithmetic operator can be defined, but whether its spectral data can be canonically related to classical arithmetic zeta functions.

1.1 Context and Preceding Results

Whitepaper I constructed a class of symmetric and self-adjoint arithmetic operators acting on the critical arithmetic Hilbert space

$$\mathcal{H}_{\text{crit}} = \ell^2(\mathbb{N}, 1/n),$$

using quadratic form techniques and canonical self-adjoint extensions. The key outcome was the existence of operators with well-defined resolvents and stable spectral measures, defined intrinsically from arithmetic principles.

Whitepaper II developed trace-level analysis for these operators. Regulated trace expressions were expanded into sums over closed arithmetic cycles, yielding a decomposition into smooth spectral terms and oscillatory contributions indexed by primes and their powers. These expansions mirror the structure of classical explicit formulae in analytic number theory.

Taken together, these results establish a rigorous operator-theoretic framework in which arithmetic information enters spectral quantities through trace identities. However, trace-level correspondence alone does not constitute spectral identification.

1.2 The SMRK Hamiltonian

The arithmetic operators considered in this work are organized around a single distinguished object: the *SMRK Hamiltonian*. It provides a concrete realization of prime-driven arithmetic dynamics as a self-adjoint operator acting on the critical arithmetic Hilbert space.

$$(H_{\text{SMRK}}\varphi)(n) = \sum_{p \in \mathbb{P}} \frac{1}{p} \left(\varphi(n/p) \mathbf{1}_{p|n} + \varphi(pn) \right) + (\alpha \Lambda(n) + \beta \log n) \varphi(n).$$

Here $\mathbb{P}$ denotes the set of prime numbers, $\Lambda(n)$ is the von Mangoldt function, and $\alpha, \beta \in \mathbb{R}$ are fixed coupling parameters. The indicator $\mathbf{1}_{p|n}$ enforces divisibility by the prime p .

The SMRK Hamiltonian is not introduced as a formal analogy to a classical mechanical Hamiltonian. Rather, it encodes multiplicative arithmetic dynamics directly: prime multiplication and division generate the kinetic component, while arithmetic potentials arise from explicit formulae rather than ad hoc modeling choices. The operator admits a unique self-adjoint realization and has purely discrete spectrum.

1.3 What Is Meant by Spectral Identification

By *spectral identification* we mean the following problem. Given a self-adjoint operator A with well-defined spectral invariants, can one identify a spectral object associated with A —such as a spectral zeta function or regularized determinant—with a known arithmetic function?

Identification is not understood as heuristic similarity or formal analogy. It requires equality of analytic objects, up to explicitly controlled and understood normalization factors.

In particular, spectral identification requires:

- a canonically defined spectral invariant associated with A ,
- analytic continuation to a sufficiently large domain,
- structural compatibility with known arithmetic functions,
- explicit control of regularization and normalization.

Failure of any of these conditions constitutes a genuine obstruction rather than a technical gap.

1.4 Role of Self-Adjointness

Self-adjointness is central to any attempt at spectral identification. It ensures that the spectrum of the operator lies on the real axis and that spectral measures are positive.

As a consequence, any spectral determinant or spectral zeta function derived from a self-adjoint operator inherits strong symmetry and reality constraints. In particular, if a completed arithmetic zeta function were to arise as a spectral determinant of such an operator, its nontrivial zeros would necessarily lie on a critical line determined by the spectral normalization.

This observation motivates the operator-first approach adopted here, but does not by itself establish identification.

1.5 Strategy of the Present Work

The strategy pursued in this paper is deliberately conservative. Rather than assuming a spectral–arithmetic correspondence, we treat it as a hypothesis subject to explicit analytic and

structural constraints.

We proceed in stages. First, we define spectral invariants—spectral zeta functions and regularized determinants—associated with the SMRK Hamiltonian. Next, we analyze their analytic properties, including continuation, symmetry, and normalization behavior. Finally, we formulate precise criteria under which identification with classical arithmetic zeta functions may occur, and equally precise criteria under which it must fail.

1.6 Implications for the Riemann Hypothesis

The Riemann Hypothesis enters this framework only conditionally. If a spectral determinant associated with the SMRK Hamiltonian coincides with the completed Riemann zeta function, then the Riemann Hypothesis follows as a formal consequence of self-adjointness.

The present work does not claim to establish such a coincidence. Instead, it isolates the precise operator-theoretic conditions under which such an implication would hold, and clarifies the structural obstructions that may prevent it.

1.7 Organization of the Paper

Section 2 introduces spectral invariants associated with self-adjoint arithmetic operators. Section 3 develops spectral zeta functions and determinant constructions. Section 4 analyzes symmetry constraints and functional equations. Section 5 addresses the identification problem with arithmetic zeta functions. Section 6 discusses conditional implications for the Riemann Hypothesis. Section 7 analyzes falsifiability and failure modes. The final section summarizes the results and outlines future directions.

2 Spectral Invariants from Self-Adjoint Operators

In this section we introduce the spectral invariants associated with the self-adjoint arithmetic Hamiltonians constructed in the preceding works. These invariants form the basic analytic objects used in subsequent identification arguments.

2.1 Spectral Measures and Functional Calculus

Let A be a self-adjoint operator on a Hilbert space $\mathcal{H}$ with spectral measure $E(\lambda)$. For any bounded Borel function $f : \mathbb{R} \rightarrow \mathbb{C}$, the operator $f(A)$ is defined by the spectral theorem:

$$f(A) = \int_{\mathbb{R}} f(\lambda) dE(\lambda).$$

Spectral invariants are constructed by choosing specific classes of functions f that encode global spectral information.

2.2 Spectral Zeta Functions

Assume that the spectrum of A is bounded below and that A has no zero eigenvalue, or that zero modes have been removed by projection. The spectral zeta function associated with A is formally defined by

$$\zeta_A(s) = \text{Tr } A^{-s},$$

for complex s with sufficiently large real part.

More precisely, one defines

$$\zeta_A(s) = \sum_{\lambda_j \in \sigma(A)} \lambda_j^{-s},$$

where the sum runs over the nonzero eigenvalues of A , counted with multiplicity. Convergence holds for $\Re(s)$ sufficiently large.

2.3 Integral Representation

Using functional calculus, the spectral zeta function admits the integral representation

$$\zeta_A(s) = \frac{1}{\Gamma(s)} \int_0^\infty t^{s-1} \operatorname{Tr} e^{-tA} dt,$$

whenever the heat trace $\operatorname{Tr} e^{-tA}$ is well-defined and sufficiently regular.

This representation provides the bridge between trace formulas and spectral zeta functions.

2.4 Analytic Continuation

The spectral zeta function $\zeta_A(s)$ admits analytic continuation beyond its domain of absolute convergence provided the heat trace admits a suitable asymptotic expansion as $t \rightarrow 0^+$. Poles of $\zeta_A(s)$ correspond to singular terms in this expansion.

Analytic continuation is therefore a spectral property of A rather than an assumption.

2.5 Regularized Determinants

The regularized determinant of A is defined in terms of the spectral zeta function by

$$\det_\zeta(A) = \exp(-\zeta'_A(0)),$$

provided $\zeta_A(s)$ is regular at $s = 0$.

This definition extends the notion of determinant to infinite-dimensional operators and is invariant under unitary equivalence.

2.6 Normalization and Energy Shifts

Spectral invariants depend on normalization choices. If A is shifted by a constant c , the spectral zeta function transforms as

$$\zeta_{A+c}(s) = \sum_j (\lambda_j + c)^{-s}.$$

Such shifts must be accounted for explicitly when comparing spectral invariants with arithmetic zeta functions.

2.7 Relative Spectral Invariants

To remove dependence on reference operators, one may define relative spectral invariants. Given two self-adjoint operators A and A_0 with suitable properties, the relative spectral zeta function is defined by

$$\zeta_{A/A_0}(s) = \operatorname{Tr}(A^{-s} - A_0^{-s}),$$

whenever the difference is trace-class.

Relative invariants are often better behaved analytically and play an important role in renormalized determinant constructions.

2.8 Positivity and Reality Constraints

Self-adjointness of A ensures that spectral invariants satisfy strong reality constraints. In particular, for real s in the domain of convergence, $\zeta_A(s)$ is real-valued, and the determinant $\det_\zeta(A)$ is positive provided A is positive.

These constraints severely restrict the possible analytic behavior of spectral invariants.

2.9 Summary

Spectral zeta functions and regularized determinants provide canonical spectral invariants associated with self-adjoint arithmetic Hamiltonians. Their analytic properties are governed by the heat trace and by self-adjointness.

In the next section, we analyze these invariants in greater detail and develop determinant constructions suitable for comparison with arithmetic zeta functions.

3 Spectral Zeta Functions and Determinants

In this section we analyze spectral zeta functions associated with self-adjoint arithmetic Hamiltonians in greater detail. The emphasis is on their analytic structure, determinant relations, and compatibility with the trace formulas derived previously.

3.1 Definition via the Heat Kernel

Let A be a self-adjoint operator bounded from below, with purely discrete spectrum after regularization. The spectral zeta function is defined for $\Re(s)$ sufficiently large by

$$\zeta_A(s) = \sum_{\lambda_j > 0} \lambda_j^{-s}.$$

Using the heat kernel representation,

$$\zeta_A(s) = \frac{1}{\Gamma(s)} \int_0^\infty t^{s-1} \operatorname{Tr} e^{-tA} dt,$$

the analytic properties of $\zeta_A(s)$ are determined by the small- t expansion of the heat trace.

3.2 Small-Time Expansion and Pole Structure

Assume that the heat trace admits an asymptotic expansion of the form

$$\operatorname{Tr} e^{-tA} \sim \sum_{k=0}^{\infty} a_k t^{(k-d)/2} \quad \text{as } t \rightarrow 0^+,$$

for suitable coefficients a_k and effective dimension parameter d .

Each term in this expansion gives rise to a potential pole of $\zeta_A(s)$ at $s = (d - k)/2$. In particular, the regularity of $\zeta_A(s)$ at $s = 0$ is governed by the absence of a t^0 term in the heat trace expansion.

3.3 Meromorphic Continuation

Under the above assumptions, $\zeta_A(s)$ extends to a meromorphic function on the complex plane, with at most simple poles determined by the heat coefficients. This continuation is canonical and does not depend on arbitrary choices.

The meromorphic structure of $\zeta_A(s)$ is therefore a spectral invariant of A .

3.4 Spectral Determinants

Provided $\zeta_A(s)$ is regular at $s = 0$, the zeta-regularized determinant

$$\det_\zeta(A) = \exp(-\zeta'_A(0))$$

is well defined. This determinant encodes global spectral information and is multiplicative under direct sums.

For operators with continuous spectrum or divergent determinants, relative determinants provide a renormalized alternative.

3.5 Relative Spectral Zeta Functions

Let A and A_0 be two self-adjoint operators such that $e^{-tA} - e^{-tA_0}$ is trace-class for all $t > 0$. The relative spectral zeta function is defined by

$$\zeta_{A/A_0}(s) = \frac{1}{\Gamma(s)} \int_0^\infty t^{s-1} (\text{Tr } e^{-tA} - \text{Tr } e^{-tA_0}) dt.$$

This construction removes divergent background contributions and isolates the arithmetic content of the spectrum.

3.6 Connection with Trace Formulas

The trace formulas derived in Whitepaper II provide explicit expressions for $\text{Tr } e^{-tA}$ in terms of smooth terms and oscillatory prime-cycle contributions. Substituting these expressions into the integral representation for $\zeta_A(s)$ yields a decomposition of the spectral zeta function into corresponding components.

In particular, oscillatory prime sums enter $\zeta_A(s)$ through Mellin transforms of cycle weights.

3.7 Analytic Dependence on Regularization

The spectral zeta function depends on regularization parameters introduced to define the operator A . However, its meromorphic structure and symmetry properties are stable under admissible changes of regularization.

Only normalization-dependent constants and entire factors are affected by such changes.

3.8 Reality and Symmetry

Self-adjointness of A implies that $\zeta_A(\bar{s}) = \overline{\zeta_A(s)}$. This reality condition constrains the possible zero and pole structure of $\zeta_A(s)$ and will play a crucial role in the analysis of functional equations.

3.9 Summary

Spectral zeta functions associated with self-adjoint arithmetic Hamiltonians admit canonical meromorphic continuation and encode global spectral information via determinant constructions. Their analytic structure is governed by heat trace asymptotics and trace formulas.

In the next section, we analyze symmetry constraints and functional equations arising from self-adjointness and arithmetic structure.

4 Functional Equation and Symmetry Constraints

In this section we analyze symmetry constraints and functional-equation-type relations imposed on spectral invariants by self-adjointness and arithmetic structure. The purpose is not to postulate a functional equation, but to determine the conditions under which such a symmetry must arise if spectral identification is to hold.

4.1 Symmetry from Self-Adjointness

Let A be a self-adjoint operator with spectral zeta function $\zeta_A(s)$. Self-adjointness implies that the spectrum of A is real and that the heat trace $\text{Tr } e^{-tA}$ is real-valued for $t > 0$.

As a consequence, the spectral zeta function satisfies the reality condition

$$\zeta_A(\bar{s}) = \overline{\zeta_A(s)}.$$

This condition alone does not enforce a functional equation, but it severely restricts admissible symmetry types.

4.2 Dual Representations and Mellin Symmetry

The spectral zeta function admits dual representations:

$$\zeta_A(s) = \frac{1}{\Gamma(s)} \int_0^\infty t^{s-1} \text{Tr } e^{-tA} dt,$$

and, formally,

$$\zeta_A(1-s) = \frac{1}{\Gamma(1-s)} \int_0^\infty t^{-s} \text{Tr } e^{-tA} dt.$$

A functional equation relating $\zeta_A(s)$ and $\zeta_A(1-s)$ can arise only if the heat trace exhibits a suitable symmetry under inversion of the time parameter $t \mapsto 1/t$, up to explicitly controlled scaling factors.

4.3 Scaling and Energy Normalization

Let A be normalized so that its spectrum is centered around a distinguished energy level. If the heat trace satisfies a scaling relation of the form

$$\text{Tr } e^{-tA} = t^{-\alpha} \text{Tr } e^{-\frac{1}{t}A} + R(t),$$

where $R(t)$ is a remainder term with controlled decay, then the Mellin transform implies a functional equation relating $\zeta_A(s)$ and $\zeta_A(\alpha-s)$.

Such a relation is possible only under very restrictive spectral symmetry assumptions.

4.4 Role of Diagonal Arithmetic Potentials

Diagonal arithmetic potentials modify the heat trace by introducing additional weights. For a functional equation to persist, these weights must transform compatibly under the inversion symmetry.

This requirement constrains admissible diagonal terms and explains why only certain arithmetic potentials may appear in Hamiltonians intended for spectral identification.

4.5 Completed Spectral Zeta Functions

In analogy with classical constructions, one may define a completed spectral zeta function

$$\Xi_A(s) = \Gamma(s) \zeta_A(s),$$

or more generally include normalization factors arising from heat trace asymptotics.

A functional equation of the form

$$\Xi_A(s) = \Xi_A(\alpha - s)$$

can hold only if the normalization absorbs all asymmetries of the raw spectral zeta function.

4.6 Critical Line and Spectral Symmetry

If such a functional equation holds with symmetry axis $\Re(s) = \alpha/2$, then zeros of $\Xi_A(s)$ are symmetric with respect to this line.

In the special case $\alpha = 1$, this symmetry reproduces the critical line $\Re(s) = 1/2$ familiar from the theory of the Riemann zeta function. However, the value of α is determined entirely by spectral normalization and cannot be assumed a priori.

4.7 Necessary but Not Sufficient Conditions

It is important to emphasize that the symmetry conditions derived here are necessary but not sufficient for spectral identification. Many self-adjoint operators admit functional equations unrelated to arithmetic zeta functions.

Thus, the existence of a functional equation does not imply identification, but its absence rules identification out completely.

4.8 Summary

Self-adjointness and trace structure impose strong symmetry constraints on spectral invariants. Functional equations can arise only if the heat trace exhibits a precise scaling symmetry and if normalization absorbs residual asymmetries.

These conditions provide necessary criteria for any attempt at identifying spectral zeta functions with classical arithmetic zeta functions. In the next section, we investigate whether such identification can be achieved under explicit and testable assumptions.

5 Identification with Arithmetic Zeta Functions

In this section we formulate the problem of spectral identification in a precise and testable manner. Rather than asserting an identification between spectral invariants of arithmetic Hamiltonians

and classical zeta functions, we treat such identification as a hypothesis subject to explicit analytic constraints.

5.1 Statement of the Identification Problem

Let A be a self-adjoint arithmetic Hamiltonian constructed as in Whitepaper I, and let $\zeta_A(s)$ denote its spectral zeta function defined via heat trace regularization. The central question is whether there exists a normalization and regularization scheme such that

$$\zeta_A(s) \equiv \zeta_{\text{arith}}(s),$$

where $\zeta_{\text{arith}}(s)$ is a classical arithmetic zeta function, such as the Riemann zeta function or its completed form.

Identification here is understood as equality of meromorphic functions on a common domain, up to explicitly specified entire factors.

5.2 Completed Objects and Normalization

Classical arithmetic zeta functions appear naturally in completed form. For the Riemann zeta function, this takes the form

$$\Xi(s) = \pi^{-s/2} \Gamma\left(\frac{s}{2}\right) \zeta(s).$$

Accordingly, any candidate for spectral identification must involve a completed spectral object of the form

$$\Xi_A(s) = G(s) \zeta_A(s),$$

where $G(s)$ is an explicit normalization factor determined by heat trace asymptotics and energy normalization.

The choice of $G(s)$ is not arbitrary and must absorb all non-arithmetic asymmetries of $\zeta_A(s)$.

5.3 Compatibility with Trace Formulas

Whitepaper II derived trace formulas in which oscillatory terms arise as explicit prime sums organized by prime cycles. For spectral identification to hold, these oscillatory contributions must match those appearing in the explicit formulas for $\zeta_{\text{arith}}(s)$.

This requires:

- agreement of prime weights up to normalization,
- matching of prime power contributions,
- compatibility of smoothing and regularization.

Any mismatch at this level constitutes a definitive obstruction to identification.

5.4 Analytic Continuation and Pole Structure

The spectral zeta function $\zeta_A(s)$ admits meromorphic continuation governed by heat trace asymptotics. Classical arithmetic zeta functions have known pole structures.

Spectral identification requires exact agreement of pole locations and orders, after completion. In particular, no additional poles may appear, and no expected poles may be absent.

This constraint is stringent and independently testable.

5.5 Functional Equation Compatibility

As shown in Section 4, a functional equation of the form

$$\Xi_A(s) = \Xi_A(\alpha - s)$$

can arise only under precise symmetry conditions. For identification with the Riemann zeta function, one must have $\alpha = 1$.

Thus, spectral normalization must place the symmetry axis at $\Re(s) = 1/2$. Failure to achieve this alignment excludes identification.

5.6 Spectral Interpretation of Zeros

If identification holds, the nontrivial zeros of the arithmetic zeta function must coincide with zeros of $\Xi_A(s)$. These zeros arise from spectral data of the self-adjoint operator A .

Consequently, zeros must lie on the symmetry axis determined by self-adjointness. This observation explains why spectral identification would imply the Riemann Hypothesis, but does not establish that identification holds.

5.7 Necessary Conditions Summary

To summarize, spectral identification requires:

1. existence of a well-defined spectral zeta function $\zeta_A(s)$,
2. analytic continuation matching that of $\zeta_{\text{arith}}(s)$,
3. compatibility of trace-derived prime sums,
4. existence of a functional equation with correct symmetry axis,
5. absence of extraneous poles or zeros.

Each of these conditions is necessary; none is sufficient on its own.

5.8 Interpretive Status

The identification hypothesis formulated here is deliberately strong. It is designed to be falsifiable at multiple independent stages. Success would imply a deep structural connection between operator spectra and arithmetic zeta functions; failure would clarify the limits of operator-based approaches.

In the next section, we analyze the implications of successful identification for the Riemann Hypothesis, and delineate precisely what would and would not follow.

6 Implications for the Riemann Hypothesis

In this section we analyze the implications of successful spectral identification for the Riemann Hypothesis. The goal is to make explicit what would follow as a formal consequence of identification, and equally important, what would not.

6.1 Conditional Nature of the Argument

All implications discussed in this section are conditional. They rely on the hypothesis that a completed spectral invariant associated with a self-adjoint arithmetic Hamiltonian coincides with the completed Riemann zeta function up to explicitly controlled entire factors.

No unconditional claim regarding the Riemann Hypothesis is made.

6.2 Self-Adjointness and Location of Zeros

Let $\Xi_A(s)$ denote a completed spectral determinant associated with a self-adjoint operator A , normalized so that it satisfies a functional equation of the form

$$\Xi_A(s) = \Xi_A(1-s).$$

Self-adjointness implies that spectral singularities arise from real spectral data. Consequently, zeros of $\Xi_A(s)$ must lie on the symmetry axis $\Re(s) = 1/2$.

If $\Xi_A(s)$ coincides with the completed Riemann zeta function $\Xi(s)$, then all nontrivial zeros of $\zeta(s)$ lie on the critical line.

6.3 Exclusion of Off-Critical Zeros

Assume, for contradiction, that $\zeta(s)$ admits a nontrivial zero with $\Re(s) \neq 1/2$. Under spectral identification, this zero would correspond to a zero of $\Xi_A(s)$ away from the symmetry axis.

Such a zero is incompatible with the spectral interpretation of $\Xi_A(s)$ derived from a self-adjoint operator, as it would correspond to a non-real spectral parameter. This contradiction excludes the existence of off-critical zeros.

6.4 Multiplicity Considerations

Spectral interpretation also constrains multiplicity. Eigenvalues of a self-adjoint operator have finite multiplicity, and their contribution to spectral invariants is controlled.

Under identification, this implies that nontrivial zeros of $\zeta(s)$ must have finite multiplicity. While this is consistent with known results, it does not resolve questions of simplicity.

6.5 What the Argument Does Not Prove

It is essential to clarify what does not follow from spectral identification alone:

- no information is obtained about the distribution of trivial zeros,
- no bounds on zero spacing are derived,
- no information about zero simplicity is obtained without further input,
- no quantitative error terms in prime counting functions are implied.

These questions require additional analytic or arithmetic input beyond spectral identification.

6.6 Dependence on Normalization

The implication for the Riemann Hypothesis depends critically on correct normalization of the spectral invariant. A misalignment of the symmetry axis would shift the implied critical line.

Thus, normalization is not a cosmetic choice but an essential structural requirement.

6.7 Logical Status

The argument presented here has the following logical form:

$$\text{Spectral Identification} \implies \text{Riemann Hypothesis}.$$

The converse implication is not claimed, and no circularity is present.

6.8 Interpretive Perspective

From an interpretive standpoint, this analysis explains why operator-based approaches to the Riemann Hypothesis focus so strongly on self-adjointness. It is not self-adjointness alone that is powerful, but its interaction with spectral identification.

Without identification, self-adjointness remains a structural property with no arithmetic consequence. With identification, it becomes decisive.

6.9 Summary

If a completed spectral invariant associated with a self-adjoint arithmetic Hamiltonian coincides with the completed Riemann zeta function, then the Riemann Hypothesis follows as a direct and unavoidable consequence.

This result clarifies the precise role of operator theory in relation to the Riemann Hypothesis and delineates the boundary between structural insight and unproven identification.

In the next section, we analyze failure modes and falsifiability, identifying precise points at which the operator-based approach may break down.

7 Falsifiability and Failure Modes

In this section we analyze the falsifiability of the operator-based program developed in this series. The objective is to identify explicit failure modes and testable obstructions that would rule out spectral identification or invalidate its implications.

7.1 Principle of Falsifiability

A central requirement of the present framework is falsifiability. Each stage of the argument—from operator construction to trace formulas to spectral identification—must admit concrete tests whose failure invalidates the corresponding claim.

The value of the approach lies not only in potential success, but equally in its ability to fail in a controlled and informative manner.

7.2 Failure of Self-Adjointness

The entire program rests on the existence of a canonical self-adjoint arithmetic Hamiltonian. Any failure to establish self-adjointness, or any dependence on noncanonical extensions, invalidates the spectral interpretation.

This includes:

- lack of essential self-adjointness on a natural core,
- dependence of spectral invariants on boundary conditions,
- instability under admissible perturbations.

Such failures terminate the program at the foundational level.

7.3 Breakdown of Trace Expansions

Trace formulas rely on controlled resolvent expansions and convergence of cycle sums. The program fails if:

- cycle expansions diverge beyond regularization,
- prime-cycle contributions fail to organize into stable structures,
- regularization dependence cannot be isolated to smooth terms.

Any of these issues undermines the extraction of arithmetic information from traces.

7.4 Mismatch of Prime Weights

Spectral identification requires that prime-cycle weights match those appearing in classical explicit formulas, up to normalization. Disagreement in:

- prime weights,
- prime power contributions,
- smoothing behavior,

constitutes a decisive obstruction.

Such mismatches are directly testable by explicit computation.

7.5 Failure of Analytic Continuation

The spectral zeta function must admit analytic continuation matching that of the arithmetic zeta function. Additional poles, missing poles, or incorrect residue structure rule out identification.

This failure mode is independent of trace expansions and provides a separate analytic test.

7.6 Absence or Misalignment of Functional Equation

Even if analytic continuation holds, failure to obtain a functional equation with the correct symmetry axis excludes identification.

In particular, a symmetry axis different from $\Re(s) = 1/2$ cannot be reconciled with the Riemann zeta function.

7.7 Extraneous Zeros or Spectral Artifacts

Spectral determinants may introduce zeros unrelated to arithmetic structure. Such extraneous zeros invalidate identification unless they can be factored out as explicit, non-arithmetic entire functions.

The presence of uncontrolled spectral artifacts is a common failure mode in operator-based approaches.

7.8 Numerical and Experimental Tests

Many failure modes admit numerical testing:

- approximation of low-lying spectrum of truncated operators,
- numerical evaluation of heat traces and spectral zeta functions,
- comparison of computed prime-cycle weights with classical values.

Negative numerical results constitute legitimate falsification.

7.9 Interpretive Limits

Failure of spectral identification does not imply failure of the operator framework itself. The operators constructed may encode meaningful arithmetic dynamics without coinciding with classical zeta functions.

In such a case, the result would be negative with respect to the Riemann Hypothesis, but positive as a structural contribution to arithmetic spectral theory.

7.10 Summary

The operator-based approach to spectral arithmetic is falsifiable at multiple, independent stages. Each failure mode provides clear diagnostic information and prevents unjustified conclusions.

The presence of explicit failure criteria strengthens the credibility of the program and clarifies the precise meaning of success.

In the final section, we summarize the results of the trilogy and outline possible future directions beyond spectral identification.

8 Conclusion and Outlook

This paper concludes a three-part investigation into operator-based approaches to arithmetic spectral theory. Across the trilogy, we have progressed from analytic foundations to trace formulas and finally to the problem of spectral identification.

The objective has not been to assert a solution to the Riemann Hypothesis, but to formulate a mathematically disciplined framework in which such a solution could, in principle, arise—and in which failure is equally meaningful.

8.1 Summary of the Trilogy

Whitepaper I established the existence of canonical self-adjoint prime-shift Hamiltonians using quadratic form methods. This resolved foundational questions of domain, self-adjointness, and spectral stability.

Whitepaper II derived trace formulas for these operators by expanding regulated trace objects into sums over closed arithmetic cycles. This analysis revealed the mechanism by which explicit prime sums emerge from operator dynamics.

Whitepaper III addressed the problem of spectral identification, defining spectral zeta functions and determinants associated with self-adjoint arithmetic Hamiltonians, analyzing symmetry constraints, and formulating explicit and falsifiable criteria for identification with classical arithmetic zeta functions.

8.2 Conceptual Outcome

The central conceptual outcome of this work is the demonstration that:

- arithmetic prime structure can be encoded dynamically via operator theory,
- trace formulas provide a natural bridge between dynamics and arithmetic,
- self-adjointness imposes rigid and non-negotiable constraints on spectral interpretation.

These insights stand independently of any ultimate identification with the Riemann zeta function.

8.3 Status of the Riemann Hypothesis

Within the framework developed here, the Riemann Hypothesis appears not as a standalone conjecture, but as a conditional consequence of spectral identification. If such identification holds, the hypothesis follows automatically from self-adjointness. If it does not, the failure occurs at explicit, testable points.

In either case, the logical structure is transparent and free of circularity.

8.4 Value of Negative Results

Failure to achieve spectral identification does not invalidate the operator framework. On the contrary, it would clarify the limits of operator-based approaches and rule out entire classes of speculative constructions.

Negative results would still represent progress by narrowing the landscape of viable spectral interpretations of arithmetic phenomena.

8.5 Future Directions

Several directions remain open for future investigation:

- refinement of arithmetic Hamiltonians and diagonal potentials,
- numerical exploration of low-lying spectra and trace invariants,
- extension to general L -functions and automorphic spectra,
- exploration of operator frameworks beyond prime-shift dynamics.

Each of these directions is compatible with the analytic discipline established here.

8.6 Broader Perspective

The operator-theoretic approach pursued in this trilogy suggests a broader philosophical shift: arithmetic phenomena traditionally accessed through complex analysis may admit alternative formulations in terms of dynamics, spectra, and self-adjoint operators.

Whether or not this perspective ultimately resolves classical conjectures, it provides a coherent and testable framework for exploring the deep interface between arithmetic and spectral theory.

8.7 Closing Remarks

This work has deliberately prioritized clarity, rigor, and falsifiability over ambition. The resulting framework does not promise answers without effort, but it does promise that any answers obtained will be structurally meaningful.

The problem of spectral arithmetic remains open. What has been achieved here is a precise map of one possible path through it.

APPENDIX A

A.1 Determinant Regularization and Spectral Products

This appendix collects technical material concerning determinant constructions associated with self-adjoint arithmetic Hamiltonians. The purpose is to make explicit the regularization choices and analytic assumptions underlying spectral determinants used in the main text.

A.1.1 Formal Determinants and Divergence

For an operator A with eigenvalues $\{\lambda_j\}$, the formal determinant

$$\det A = \prod_j \lambda_j$$

diverges in infinite dimensions. Any meaningful determinant must therefore be defined via regularization.

The divergence reflects the accumulation of spectral data at high energies and is not specific to arithmetic operators.

A.1.2 Zeta-Regularized Determinants

Given a self-adjoint operator A bounded from below, with spectral zeta function

$$\zeta_A(s) = \sum_j \lambda_j^{-s},$$

the zeta-regularized determinant is defined by

$$\det_\zeta(A) = \exp(-\zeta'_A(0)),$$

provided $\zeta_A(s)$ admits analytic continuation to $s = 0$ and is regular there.

This definition is invariant under unitary equivalence and depends only on the spectrum of A .

A.1.3 Heat Kernel Representation

Using the Mellin transform, the determinant may be expressed as

$$\log \det_\zeta(A) = - \int_0^\infty \frac{dt}{t} \left(\text{Tr } e^{-tA} - P(t) \right),$$

where $P(t)$ is a finite subtraction polynomial determined by the small- t asymptotics of the heat trace.

This representation makes explicit the role of trace formulas in determinant constructions.

A.1.4 Relative Determinants

To eliminate background divergences, one often considers relative determinants. Given two operators A and A_0 such that $e^{-tA} - e^{-tA_0}$ is trace class, the relative determinant is defined by

$$\log \det_\zeta(A/A_0) = - \int_0^\infty \frac{dt}{t} \left(\text{Tr } e^{-tA} - \text{Tr } e^{-tA_0} \right).$$

Relative determinants isolate the genuinely arithmetic content of the spectrum and are particularly well suited to prime-shift Hamiltonians.

A.1.5 Determinants and Functional Equations

If a completed spectral determinant

$$\Xi_A(s) = G(s) \det_\zeta(A + s)$$

satisfies a functional equation of the form

$$\Xi_A(s) = \Xi_A(\alpha - s),$$

then the normalization factor $G(s)$ must absorb all asymmetries arising from heat trace subtractions and energy shifts.

This observation constrains admissible determinant normalizations.

A.1.6 Euler Product Structure

The trace formulas developed in the main text imply that logarithmic derivatives of spectral determinants admit expansions of the form

$$\frac{d}{ds} \log \det_\zeta(A + s) \sim \sum_{p \in \mathbb{P}} \sum_{m=1}^{\infty} \frac{1}{m} W(\gamma_p)^m,$$

where $W(\gamma_p)$ denotes prime-cycle weights.

This structure mirrors Euler products at the level of logarithmic derivatives, without asserting equality of determinants.

A.1.7 Entire Factors and Ambiguity

Spectral determinants are defined only up to multiplication by nonvanishing entire functions. Such factors do not affect zero locations but must be controlled explicitly in any identification attempt.

Failure to account for entire-factor ambiguity invalidates determinant-based arguments.

A.1.8 Positivity and Self-Adjointness

For positive self-adjoint operators, zeta-regularized determinants are positive real numbers. This property is essential for interpreting determinant zeros as spectral invariants rather than artifacts of complex normalization.

A.1.9 Summary

Determinant constructions associated with self-adjoint arithmetic Hamiltonians are well defined only after explicit regularization and subtraction. Zeta regularization and relative determinants provide canonical tools for extracting global spectral information while controlling divergences.

These constructions underpin the determinant-based spectral identification arguments developed in the main text.

APPENDIX B

B.1 Analytic Continuation and Domain Issues

This appendix addresses technical issues related to domains, analytic continuation, and operator-theoretic legitimacy of the constructions used in the main text. Its purpose is to make explicit the assumptions under which spectral zeta functions and determinants are well defined.

B.1.1 Domain of the Arithmetic Hamiltonian

Let A denote a self-adjoint arithmetic Hamiltonian constructed via quadratic form methods. By construction, A is densely defined on $\ell^2(\mathbb{N})$ and admits a unique self-adjoint realization via Friedrichs extension.

All spectral invariants considered in this paper depend only on this canonical self-adjoint operator and not on auxiliary core domains.

B.1.2 Removal of Zero Modes

Spectral zeta functions require invertibility or explicit treatment of zero modes. If A admits a finite-dimensional kernel, zero eigenvalues are removed by projection:

$$\zeta_A(s) = \sum_{\lambda_j \neq 0} \lambda_j^{-s}.$$

This procedure is standard and does not affect analytic continuation or functional equations.

B.1.3 Heat Trace Regularity

Analytic continuation of $\zeta_A(s)$ relies on regularity of the heat trace $\text{Tr } e^{-tA}$. Throughout this work, it is assumed that:

- e^{-tA} is trace class for all $t > 0$ under regularization,
- $\text{Tr } e^{-tA}$ admits a small- t asymptotic expansion,
- large- t behavior is controlled by spectral positivity.

These assumptions are satisfied for the regularized prime-shift Hamiltonians considered.

B.1.4 Justification of Mellin Transforms

The Mellin transform

$$\zeta_A(s) = \frac{1}{\Gamma(s)} \int_0^\infty t^{s-1} \text{Tr } e^{-tA} dt$$

is justified by absolute convergence for $\text{Re}(s)$ sufficiently large and by analytic continuation using subtraction of asymptotic terms.

Interchange of integrals and traces is legitimate under the trace-class assumptions above.

B.1.5 Meromorphic Structure

The poles of $\zeta_A(s)$ arise exclusively from singular terms in the small- t expansion of the heat trace. No other singularities may occur.

Thus, any additional poles or essential singularities in candidate spectral invariants immediately falsify identification with classical arithmetic zeta functions.

B.1.6 Domain Dependence of Determinants

Zeta-regularized determinants depend only on the spectrum of A and not on the choice of core domain. Different regularization schemes may change determinants by entire nonvanishing factors, but cannot alter zero locations.

This observation underlies the robustness of determinant-based arguments.

B.1.7 Stability Under Perturbations

Let $A_\varepsilon = A + \varepsilon V$ be a perturbation by a relatively form-bounded diagonal potential. For sufficiently small ε :

- self-adjointness is preserved,
- heat trace asymptotics vary smoothly,
- analytic continuation of $\zeta_{A_\varepsilon}(s)$ persists.

This stability ensures that analytic properties are not artifacts of fine-tuned choices.

B.1.8 Interpretation of Analytic Failure

Failure of analytic continuation, breakdown of heat trace asymptotics, or instability under perturbation constitute definitive obstructions to spectral identification.

Such failures are not technical shortcomings but meaningful negative results.

B.1.9 Summary

All analytic constructions used in this work—spectral zeta functions, determinants, and functional equations—are grounded in standard operator-theoretic principles. Analytic continuation is controlled by heat trace behavior and is fully independent of heuristic assumptions.

This appendix completes the analytic justification of the spectral identification framework.

APPENDIX C

C.1 Numerical and Experimental Program

This appendix outlines a concrete numerical and experimental program designed to test and probe the operator-theoretic framework developed in the main text. The goal of this program is not to provide numerical proof of any conjecture, but to supply falsifiable evidence, diagnostic insight, and structural validation.

C.1.1 Objectives of Numerical Experiments

The numerical program pursues the following objectives:

- test stability of self-adjoint realizations under truncation,
- examine convergence of trace and cycle expansions,
- compare numerically extracted prime weights with classical values,
- detect extraneous spectral artifacts or symmetry violations,
- probe feasibility of spectral identification at low energies.

Negative results are considered as valuable as positive ones.

C.1.2 Finite-Dimensional Truncation

Let A denote a regularized prime-shift Hamiltonian acting on $\ell^2(\mathbb{N})$. Numerical approximations are obtained by truncating to finite-dimensional subspaces spanned by $\{e_n : n \leq N\}$.

For each N , one obtains a finite matrix $A^{(N)}$ whose spectral data may be computed explicitly. Convergence is assessed by comparing results for increasing N and varying regularization parameters.

C.1.3 Verification of Self-Adjointness and Symmetry

For truncated operators $A^{(N)}$, numerical checks include:

- symmetry of the matrix representation,
- stability of eigenvalues under perturbations,
- absence of spurious complex eigenvalues.

While truncation breaks exact self-adjointness, qualitative stability under growth of N provides indirect validation.

C.1.4 Heat Trace and Spectral Zeta Approximation

For each truncated operator, one computes approximations to

$$\Theta_N(t) = \text{Tr } e^{-tA^{(N)}},$$

and numerically evaluates the Mellin transform to approximate $\zeta_A(s)$ in regions of convergence. Comparison across different truncations tests robustness of heat trace asymptotics.

C.1.5 Cycle Expansion Diagnostics

Trace formulas developed in the main text express traces as sums over arithmetic cycles. Numerically, one may:

- enumerate short cycles explicitly,
- compute their weights,
- compare partial cycle sums with direct trace computations.

Discrepancies indicate breakdown of convergence or misweighting of cycles.

C.1.6 Prime Weight Comparison

Prime-cycle weights extracted numerically may be compared with classical prime weights appearing in explicit formulas. Agreement is required (up to normalization) for spectral identification to remain viable.

Mismatch at this level constitutes a decisive falsification.

C.1.7 Functional Equation Tests

If a completed spectral zeta function $\Xi_A(s)$ is constructed numerically, one may test approximate symmetry:

$$\Xi_A(s) \approx \Xi_A(1 - s),$$

within numerical precision.

Failure of approximate symmetry, even at low energies, signals misalignment of normalization or structural incompatibility.

C.1.8 Detection of Extraneous Zeros

Numerical determinants may exhibit zeros unrelated to arithmetic structure. Tracking the stability and movement of such zeros under parameter variation allows one to distinguish genuine spectral features from truncation artifacts.

Persistent extraneous zeros invalidate identification.

C.1.9 Limitations of Numerical Evidence

Numerical experiments cannot prove spectral identification or the Riemann Hypothesis. Their role is diagnostic:

- to falsify incorrect constructions,
- to guide refinement of Hamiltonians,
- to identify promising or hopeless directions.

All numerical conclusions must be interpreted within these limits.

C.1.10 Summary

The numerical program outlined here provides a concrete and falsifiable testing ground for the operator-theoretic framework developed in this trilogy. It complements analytic results by exposing hidden instabilities and structural mismatches at an early stage.

Whether or not spectral identification ultimately holds, such experiments supply essential feedback and sharpen the conceptual understanding of arithmetic Hamiltonians.

An Experimental Operator Framework for the Riemann Hypothesis

Certified Zero Pipelines,
Explicit Formula Residua,
and Operator Scoring

Enter Yourname

Abstract

This document introduces a conservative experimental framework for evaluating operator-based approaches to the Riemann Hypothesis, with emphasis on reproducibility, certified data, and falsifiable diagnostics. The motivation is that large-scale numerical verification alone is insufficient to assess the validity of an operator-theoretic program.

The framework combines three classes of tests: certified pipelines for locating zeros of the Riemann zeta function, diagnostics based on explicit-formula-style consistency between primes and spectral data, and stability tests under refinement of truncation scales and numerical precision. An operator scoring methodology is proposed to compare different Hamiltonians in a systematic way.

Strong emphasis is placed on reproducibility. Computational runs are fully specified through versioned code, configuration manifests, deterministic run identifiers, and hashed data artifacts. Experimental agreement is not interpreted as proof, while negative results are treated as valuable indicators that eliminate nonviable candidates. The result is a standardized experimental infrastructure for iterative testing within the QVM program.

1 Motivation and Context

1.1 The status of the Riemann Hypothesis

The Riemann Hypothesis asserts that all nontrivial zeros of the Riemann zeta function

$$\zeta(s) = \sum_{n=1}^{\infty} n^{-s}, \quad \Re(s) > 1,$$

lie on the critical line $\Re(s) = \frac{1}{2}$ after analytic continuation. Its truth would imply optimal error bounds in the prime number theorem and profoundly constrain the fine-scale structure of prime number distributions.

Over more than a century, RH has accumulated strong indirect evidence: millions of zeros have been numerically verified on the critical line, deep analogies with random matrix theory have emerged, and multiple equivalent formulations have been established. Nevertheless, these results fall short of a proof, and no consensus exists on the correct conceptual pathway toward one.

1.2 The Hilbert–Pólya idea and its limitations

A long-standing heuristic proposes that the imaginary parts of the nontrivial zeros of $\zeta(s)$ arise as eigenvalues of a self-adjoint operator. Such an operator, if explicitly constructed, would imply RH immediately through the reality of its spectrum.

Despite its appeal, the Hilbert–Pólya idea faces several obstacles. Most proposed operators are either formally defined, lack rigorous spectral control, or fail to demonstrate compatibility with the arithmetic structure of primes. In particular, reproducing the zero spectrum alone is insufficient: any viable operator must also encode the prime–zero duality captured by the explicit formulas of analytic number theory.

1.3 Why an experimental operator framework is needed

Numerical experiments play an essential role in modern research on RH, yet they are often ad hoc, non-reproducible, or disconnected from operator-theoretic hypotheses. Lists of computed zeros, statistical tests of level spacings, and heuristic comparisons with random matrix ensembles are frequently treated as isolated evidence.

The absence of a unified experimental framework leads to two recurring problems. First, numerical results are difficult to compare across different studies. Second, operator proposals cannot be systematically falsified, as failure modes are rarely formalized.

The motivation of this work is to address these issues by defining a structured experimental environment in which operator-based approaches to RH can be evaluated under common constraints. Rather than searching directly for a proof, we focus on constructing a pipeline that enforces consistency between zeros, primes, and operator spectra.

1.4 Scope and philosophy of the present work

The framework developed in this paper is intentionally conservative. We make no claims beyond what can be numerically certified or logically constrained. Experimental agreement is not interpreted as proof, but as evidence of structural compatibility.

Our guiding principle is the following: a candidate operator relevant to the Riemann Hypothesis must survive simultaneous tests imposed by certified zero computations, explicit prime–zero relations, and stability under refinement of numerical scales.

By formalizing these requirements, we aim to transform operator-based research on RH from a collection of isolated ideas into a cumulative, test-driven scientific program.

2 Certified Computation of Zeta Zeros

2.1 Motivation for certification

Numerical verification of the Riemann Hypothesis has progressed to extraordinary heights, with trillions of zeros confirmed on the critical line. However, raw numerical verification alone is insufficient for operator-based or trace-formula investigations. Any experimental framework intended to constrain candidate operators must rely on zero data whose completeness and correctness are themselves certified within controlled error bounds.

In particular, the absence of certification leads to two critical risks: missing zeros in a given interval and spurious numerical artifacts falsely identified as zeros. Both failure modes can invalidate subsequent spectral or arithmetic analyses.

For these reasons, we adopt a certified zero pipeline, in which each computed zero is accompanied by a verification procedure that guarantees both existence and uniqueness within a prescribed interval.

2.2 The Riemann–Siegel Z -function

Let

$$Z(t) = e^{i\theta(t)} \zeta\left(\frac{1}{2} + it\right),$$

where $\theta(t)$ is the Riemann–Siegel theta function. For real t , the function $Z(t)$ is real-valued, and its real zeros correspond exactly to the nontrivial zeros of $\zeta(s)$ on the critical line.

The numerical computation of $Z(t)$ is significantly more stable than direct evaluation of $\zeta(s)$, and it provides a natural foundation for detecting zeros via sign changes.

2.3 Bracketing and root refinement

The first stage of the pipeline consists of scanning $Z(t)$ over an interval $[0, T_{\max}]$ using an adaptive step size Δt . Whenever a sign change

$$Z(t_k) \cdot Z(t_{k+1}) < 0$$

is detected, an interval $[a, b] = [t_k, t_{k+1}]$ is recorded as a zero bracket.

Within each bracket, a root refinement procedure is applied. Any standard method may be used, such as bisection, Newton iteration, or hybrid schemes, provided that convergence is monotone and error bounds are explicitly tracked.

The output of this stage is a list of candidate zeros $\{\gamma_n\}$, each associated with a certified interval of width at most ε .

2.4 Counting zeros and checkpoint validation

Bracketing and refinement alone do not guarantee that all zeros in a given interval have been found. To certify completeness, we employ counting formulas derived from analytic number theory.

Let $N(T)$ denote the number of nontrivial zeros with imaginary part in $(0, T]$. The Riemann–von Mangoldt formula states

$$N(T) = \frac{T}{2\pi} \log\left(\frac{T}{2\pi}\right) - \frac{T}{2\pi} + O(\log T).$$

In practice, we use this formula together with explicit error bounds to define checkpoint heights $0 < T_1 < T_2 < \dots < T_k$. At each checkpoint T_j , the number of detected zeros below T_j is compared against the predicted value $N(T_j)$. Agreement within certified bounds serves as evidence that no zeros have been missed.

More stringent validation can be achieved via Turing-type methods, which compare integrals of $\arg \zeta(s)$ with zero counts. The framework is compatible with such enhancements, although they are not strictly required for moderate ranges of T .

2.5 Definition of a certified zero set

We formalize the output of the pipeline as follows.

Definition 2.1 (Certified zero set). A finite set

$$\mathcal{Z}(T) = \{\gamma_1, \gamma_2, \dots, \gamma_N\}$$

is called a *certified zero set up to height T* if the following conditions hold:

1. Each γ_n is contained in a verified bracket on which $Z(t)$ changes sign.
2. Each γ_n is refined to an interval of width at most ε .
3. The total number of elements of $\mathcal{Z}(T)$ agrees with the predicted zero count $N(T)$ within certified error bounds.

Only certified zero sets are admitted as input to subsequent stages of the framework.

2.6 Role within the operator framework

Certified zero sets serve as the spectral reference data against which candidate operators are evaluated. Any operator proposed as a realization of the Hilbert–Pólya idea must reproduce these zeros as eigenvalues within comparable accuracy.

Equally important, certified zero sets enable controlled truncation of explicit formulas, as the contribution of omitted zeros can be bounded quantitatively. This property is essential for distinguishing genuine arithmetic structure from numerical artifacts.

By isolating zero computation as a rigorously constrained preprocessing step, the framework ensures that all later conclusions rest on a stable and verifiable foundation.

3 The Explicit Formula as a Diagnostic Tool

3.1 Prime–zero duality

One of the most profound links between prime numbers and the zeros of the Riemann zeta function is provided by the explicit formulas of analytic number theory. These formulas express weighted sums over primes in terms of sums over nontrivial zeros, revealing a deep spectral duality.

For experimental operator-based investigations, the explicit formula plays a central role: it provides a consistency constraint that cannot be satisfied by reproducing the zero spectrum alone. Any viable Hilbert–Pólya type operator must not only generate the correct zeros, but must do so in a manner compatible with the arithmetic structure of primes.

3.2 The Chebyshev function

Let $\Lambda(n)$ denote the von Mangoldt function, and define the Chebyshev function

$$\psi(x) = \sum_{n \leq x} \Lambda(n).$$

The function $\psi(x)$ encodes prime powers and admits an explicit representation involving the nontrivial zeros of $\zeta(s)$.

In its classical form, the explicit formula reads

$$\psi(x) = x - \sum_{\rho} \frac{x^{\rho}}{\rho} - \log(2\pi) - \frac{1}{2} \log(1 - x^{-2}),$$

where the sum runs over nontrivial zeros ρ of $\zeta(s)$. For experimental purposes, the precise treatment of lower-order terms is less important than the controlled interplay between the prime sum and the zero sum.

3.3 Truncated zero sums

In practice, only finitely many zeros are available. Let $\mathcal{Z}(T)$ denote a certified zero set up to height T , as defined in Section 2. We define the truncated explicit formula

$$\psi_T(x) = x - \sum_{\rho \in \mathcal{Z}(T)} \frac{x^{\rho}}{\rho},$$

with the understanding that omitted terms contribute a truncation error that depends explicitly on T and x .

The use of certified zero sets ensures that truncation errors can be bounded quantitatively, rather than treated heuristically. This property is essential for interpreting discrepancies between arithmetic data and spectral reconstructions.

3.4 Smoothing and test functions

Direct evaluation of $\psi(x)$ and $\psi_T(x)$ is sensitive to sharp cutoffs and numerical noise. To mitigate these effects, we introduce smoothing via test functions.

Let f be a compactly supported or rapidly decaying test function. We consider smoothed quantities of the form

$$\Psi_f(X) = \sum_{n=1}^{\infty} \Lambda(n) f(\log n - \log X),$$

and define the corresponding zero-based approximation

$$\Psi_{f,T}(X) = X \hat{f}(0) - \sum_{\rho \in \mathcal{Z}(T)} X^{\rho} \hat{f}(\rho),$$

where $\hat{f}$ denotes the Mellin transform of f .

Smoothing improves numerical stability and isolates structural discrepancies from high-frequency artifacts. The framework is compatible with a wide class of test functions, allowing sensitivity analysis across scales.

3.5 Residual functions

The central diagnostic quantity introduced in this work is the residual function

$$R_T(X) = \Psi_f(X) - \Psi_{f,T}(X).$$

Under ideal circumstances, the residual should consist only of:

- truncation effects due to missing zeros above T ,
- controllable smoothing artifacts,
- lower-order deterministic terms.

Persistent or structured deviations in $R_T(X)$, especially those stable under refinement of T and variation of the test function f , signal incompatibility between the zero data and the underlying arithmetic structure.

3.6 Interpretation as a diagnostic criterion

The residual function provides a powerful filter for candidate operator models. An operator that reproduces the zero spectrum but fails to produce residuals consistent with prime data can be systematically rejected.

Conversely, operators whose induced spectra yield small and stable residuals across multiple scales demonstrate a higher degree of arithmetic compatibility.

Importantly, residual minimization is not interpreted as evidence of a proof. Rather, it serves as a falsification mechanism: operators that fail the explicit formula test cannot represent the arithmetic dynamics encoded by the zeta function.

3.7 Role within the experimental framework

Within the overall framework, the explicit formula occupies a mediating position between spectral data and arithmetic data. Certified zero sets enter as controlled inputs, while prime sums provide independent constraints.

This dual perspective transforms the Riemann Hypothesis from a purely spectral question into a coupled spectral–arithmetic consistency problem. The explicit formula thus functions as the primary diagnostic bridge linking operators, zeros, and primes within a single experimental architecture.

4 Operator Targets and Hilbert–Pólya Constraints

4.1 From spectral heuristics to constrained operators

The Hilbert–Pólya idea suggests that the imaginary parts of the nontrivial zeros of the Riemann zeta function arise as eigenvalues of a self-adjoint operator. While this heuristic has guided a large body of work, it does not by itself specify the mathematical or physical nature of such an operator.

In an experimental setting, the operator should not be viewed as an abstract existence statement, but as a concrete object subject to testable constraints. The purpose of this section is to articulate the minimal structural requirements that any candidate operator must satisfy in order to be meaningfully evaluated within the present framework.

4.2 Self-adjointness and spectral reality

Let $\mathcal{H}$ be a Hilbert space and H a densely defined linear operator on $\mathcal{H}$. A necessary condition for compatibility with the Hilbert–Pólya program is that H be self-adjoint, or at least essentially self-adjoint on a natural core domain.

Self-adjointness ensures that the spectrum of H is real and that its spectral measure is well-defined. In the present framework, we do not require a full proof of self-adjointness at the outset; however, the operator must admit a well-controlled symmetric form and demonstrate numerical stability consistent with an underlying self-adjoint structure.

Operators that are formally symmetric but exhibit unstable or non-real spectra under refinement are excluded at this stage.

4.3 Spectral correspondence with zeta zeros

The central spectral constraint is the following: the spectrum of the operator H must reproduce the imaginary parts of the nontrivial zeros of $\zeta(s)$.

Formally, if $\{\gamma_n\}$ denotes a certified zero set, then the operator should admit a sequence of eigenvalues $\{\lambda_n\}$ such that

$$\lambda_n \approx \gamma_n,$$

up to controlled numerical error.

This correspondence is understood in an asymptotic sense. Low-lying discrepancies may be tolerated, provided that the agreement improves with increasing spectral resolution and refinement of the operator approximation.

The framework explicitly avoids overfitting: an operator whose spectrum is artificially tuned to match a finite zero list without structural justification is considered invalid.

4.4 Trace regularization and test functions

Most operators capable of producing an unbounded sequence of eigenvalues are not trace class. As a result, expressions of the form $\text{Tr}(f(H))$ require regularization.

We therefore require that candidate operators admit a well-defined regularized trace for a suitable class of test functions f . Typical examples include:

- heat traces $\text{Tr}(e^{-tH})$,
- spectral zeta functions $\text{Tr}(H^{-s})$,
- smoothed spectral sums $\sum_n f(\lambda_n)$.

The regularized trace must be compatible with the explicit formula diagnostics introduced in Section 3. In particular, trace expressions derived from H should reproduce prime-weighted terms up to controlled residuals.

4.5 Arithmetic compatibility

Reproducing the zero spectrum alone is insufficient. The operator must encode arithmetic information consistent with the prime–zero duality.

Within the present framework, this requirement is enforced indirectly via the explicit formula residuals. An operator whose spectrum matches the zeros but whose induced trace expressions fail to approximate prime sums is rejected as arithmetically incompatible.

This criterion reflects the guiding philosophy that the Riemann Hypothesis is not purely spectral, but fundamentally arithmetic in nature.

4.6 Stability under refinement

A key experimental constraint is stability under refinement. Refinement may include:

- increasing the number of basis states,
- extending the spectral window,
- tightening numerical tolerances,
- modifying smoothing parameters.

A viable operator model must exhibit convergence of its spectral and trace properties under such refinements. Operators whose apparent agreement deteriorates with improved resolution are considered artifacts of approximation.

4.7 Minimal admissibility criteria

We summarize the admissibility requirements for candidate operators as follows.

Criterion 4.1 (Admissible operator). An operator H is admissible within the framework if it satisfies:

1. formal symmetry and numerical evidence of (essential) self-adjointness,
2. asymptotic spectral correspondence with certified zeta zeros,
3. existence of compatible regularized traces,
4. consistency with explicit formula diagnostics,
5. stability under systematic refinement.

Only operators meeting these criteria are admitted to the scoring and comparison stage introduced in subsequent sections.

4.8 Position within the overall program

This section establishes the conceptual boundary between speculative operator proposals and experimentally constrained models. By fixing explicit admissibility criteria, the framework transforms the Hilbert–Pólya idea from a heuristic vision into a test-driven research program.

The next section applies these constraints to arithmetic transfer operators and related constructions, illustrating how concrete models can be embedded within the framework without presupposing their correctness.

5 Embedding the SMRK Hamiltonian into the Framework

5.1 From arithmetic transfers to a Hamiltonian

The SMRK Hamiltonian arises from the idea that arithmetic structure can be encoded via transfer operations acting on discrete states. Rather than postulating an abstract spectral operator,

the construction begins with explicit arithmetic transitions, most notably multiplicative shifts indexed by prime numbers.

Formally, the SMRK Hamiltonian is conceived as a linear operator acting on a Hilbert space of arithmetic states, such as $\ell^2(\mathbb{N})$ or a related weighted space. Its defining feature is that transitions between states are governed by prime-indexed transfer rules, reflecting the multiplicative structure of the integers.

In this sense, the SMRK Hamiltonian is not introduced as a solution to the Riemann Hypothesis, but as a concrete arithmetic candidate that can be subjected to the admissibility criteria defined in Section 4.

5.2 Structural form of the SMRK Hamiltonian

At an abstract level, the SMRK Hamiltonian may be written schematically as

$$(H_{\text{SMRK}}\psi)(n) = \sum_{p \in \mathcal{P}} w_p(n) \psi(pn) + \sum_{p \in \mathcal{P}} v_p(n) \psi(n/p) + U(n) \psi(n),$$

where $\mathcal{P}$ denotes the set of prime numbers, $w_p(n)$ and $v_p(n)$ are prime-dependent weights, and $U(n)$ is a diagonal potential term.

The precise choice of weights and potential is not fixed at this stage. Different parameterizations correspond to different members of a family of operators, all of which may be evaluated within the same framework.

This general structure highlights the interpretation of H_{SMRK} as an arithmetic transfer Hamiltonian: states are shifted multiplicatively, rather than additively as in differential operators.

5.3 Relation to Hilbert–Pólya constraints

We now relate the SMRK Hamiltonian to the admissibility criteria of Section 4.

Self-adjointness. For suitable symmetric choices of the weights $w_p(n)$ and $v_p(n)$, the operator H_{SMRK} is formally symmetric on a dense domain. Establishing essential self-adjointness remains an open analytical task, but numerical stability under refinement serves as an experimental proxy within the present framework.

Spectral correspondence. Numerical truncations of H_{SMRK} yield finite-dimensional approximations whose spectra can be compared to certified zeta zero sets. Agreement is evaluated asymptotically and tested for stability under increasing truncation size.

Trace regularization. As an unbounded operator, H_{SMRK} is not trace class. However, smoothed spectral traces such as $\text{Tr}(f(H_{\text{SMRK}}))$ are well-defined for suitable test functions f , allowing direct comparison with explicit-formula diagnostics.

Arithmetic compatibility. By construction, the SMRK Hamiltonian incorporates prime-indexed structure. This makes it particularly well-suited for explicit formula tests, as prime-weighted trace expressions can be compared directly to Chebyshev-type sums.

Stability. The decisive criterion is stability under refinement: increasing the truncation dimension, extending the prime range, and tightening numerical tolerances. Only if spectral and trace properties converge under such refinements can the operator be considered viable.

5.4 Interpretational stance

It is essential to emphasize that the SMRK Hamiltonian is not assumed to be the operator postulated by the Hilbert–Pólya conjecture. Rather, it is treated as a test case: a concrete arithmetic Hamiltonian whose compatibility with RH-related constraints can be quantitatively evaluated.

Failure of the SMRK Hamiltonian to meet any admissibility criterion does not invalidate the framework. On the contrary, such failures provide valuable information about which structural features are incompatible with the arithmetic reality encoded by the zeta function.

5.5 Role within the experimental program

Within the broader program, the SMRK Hamiltonian serves three roles:

1. as a nontrivial arithmetic operator with explicit prime structure,
2. as a benchmark for testing the diagnostic power of the framework,
3. as a starting point for exploring broader classes of arithmetic Hamiltonians.

In the next section, we introduce a scoring protocol that quantifies how well the SMRK Hamiltonian and other candidate operators satisfy the admissibility criteria, enabling systematic comparison and iterative refinement.

6 Operator Scoring and Evaluation Protocol

6.1 Purpose of a scoring framework

The admissibility criteria defined in Section 4 establish qualitative requirements for candidate Hilbert–Pólya type operators. To support systematic comparison and refinement, these requirements must be translated into quantitative or semi-quantitative measures.

The purpose of the scoring protocol is not to declare a “winner”, but to provide a structured method for ranking operators, detecting failure modes, and guiding iterative improvements. Scores are interpreted diagnostically, not as proofs.

6.2 Overview of evaluation dimensions

Each candidate operator H is evaluated along several independent dimensions:

1. spectral correspondence with certified zeta zeros,
2. statistical properties of level spacings,
3. compatibility with explicit formula diagnostics,
4. stability under refinement,
5. numerical robustness and convergence behavior.

Each dimension yields a partial score together with uncertainty indicators. The combined score is a vector, not a single scalar quantity.

6.3 Spectral matching score

Let $\{\gamma_n\}$ denote a certified zero set and $\{\lambda_n\}$ the computed eigenvalues of a finite-dimensional approximation of H , ordered increasingly.

We define a normalized spectral deviation

$$\Delta_{\text{spec}}(N) = \frac{1}{N} \sum_{n=1}^N \frac{|\lambda_n - \gamma_n|}{\gamma_n}.$$

A decreasing trend of $\Delta_{\text{spec}}(N)$ with increasing N and refinement of the operator approximation is taken as evidence of asymptotic spectral correspondence.

Operators exhibiting non-convergent or oscillatory behavior in this metric are penalized.

6.4 Level spacing statistics

Beyond pointwise spectral agreement, the local statistics of level spacings provide an important consistency check.

Let

$$s_n = \lambda_{n+1} - \lambda_n$$

denote nearest-neighbor spacings, and let $\tilde{s}_n$ be the unfolded spacings. We compare the empirical distribution of $\tilde{s}_n$ against random matrix universality classes, most notably the Gaussian Unitary Ensemble (GUE).

Deviation from the expected distribution is quantified using standard measures such as the Kolmogorov–Smirnov distance or integrated squared error. These statistics are interpreted cautiously: agreement supports compatibility, but does not constitute evidence of correctness.

6.5 Explicit formula residual score

Using the diagnostics of Section 3, we compute residual functions

$$R_T(X) = \Psi_f(X) - \Psi_{f,T}(X),$$

where $\Psi_{f,T}$ is constructed from the operator-induced spectrum.

A residual norm is defined, for example, as

$$\|R_T\|^2 = \int_{X_{\min}}^{X_{\max}} |R_T(X)|^2 dX.$$

Scores are assigned based on:

- magnitude of the residual norm,
- stability of the residual under changes in T ,
- robustness under variation of the test function f .

Persistent structured residuals are interpreted as evidence of arithmetic incompatibility.

6.6 Stability and refinement score

Stability under refinement is assessed by repeating all scoring steps under systematic changes of resolution, including:

- increased truncation dimension,
- extended prime ranges,
- tighter numerical tolerances,
- denser sampling grids.

Let $S_i^{(k)}$ denote the score in dimension i at refinement level k . Stability is quantified by convergence or bounded variation of $S_i^{(k)}$ as k increases.

Operators whose apparent performance degrades under refinement receive a low stability score.

6.7 Composite score representation

Rather than collapsing results into a single scalar value, we represent the evaluation outcome as a score vector

$$\mathbf{S}(H) = (S_{\text{spec}}, S_{\text{spacing}}, S_{\text{explicit}}, S_{\text{stability}}, S_{\text{robustness}}).$$

This representation reflects the multi-faceted nature of the problem and avoids masking deficiencies in one dimension by strong performance in another.

6.8 Interpretation and falsification

The scoring protocol is explicitly falsification-oriented. An operator that fails one or more dimensions is not “almost correct”; it is incompatible with the imposed constraints at the tested resolution.

Conversely, a high score across all dimensions does not imply correctness or a proof of RH. It indicates only that, within the tested regime, the operator remains consistent with known spectral and arithmetic data.

6.9 Role within the experimental program

The scoring protocol transforms operator-based approaches to the Riemann Hypothesis into a cumulative experimental program. Results obtained for one operator can be directly compared to those of another, and improvements can be tracked quantitatively.

In this sense, the protocol provides a missing methodological layer between speculative operator constructions and rigorous mathematical analysis, clarifying which candidates merit deeper theoretical investigation.

7 Failure Modes and Limitations

7.1 Numerical experiments versus mathematical proof

The framework presented in this work is explicitly experimental. While it imposes strong consistency constraints on candidate operators, it does not constitute a proof of the Riemann Hypothesis.

Numerical agreement, even across large spectral ranges and multiple diagnostic tests, remains inherently finite and approximate. Consequently, all conclusions drawn within this framework must be interpreted as conditional and provisional.

This distinction is essential: the framework is designed to *exclude* incompatible operator models, not to certify correctness in the logical sense required for a proof.

7.2 Finite truncation effects

All numerical implementations operate on finite data: a finite number of zeros, a finite operator truncation, and finite numerical precision. Truncation effects can produce apparent agreement that does not persist under extension.

In particular, an operator may be tuned to reproduce a finite initial segment of zeta zeros while failing asymptotically. Such overfitting is explicitly discouraged by the stability and refinement tests, but cannot be ruled out a priori.

7.3 Sensitivity to smoothing and test functions

Explicit formula diagnostics rely on smoothing via test functions. Different choices of test functions probe different scales and aspects of prime–zero interactions.

Apparent agreement for a specific choice of smoothing may not generalize. For this reason, robustness under variation of test functions is treated as a core stability criterion. Operators whose performance depends critically on a narrow class of test functions are considered fragile.

7.4 Statistical ambiguity in level spacings

Level spacing statistics, including comparisons with random matrix ensembles, are inherently statistical. Finite samples may exhibit fluctuations that mimic or obscure universal behavior.

Agreement with Gaussian Unitary Ensemble (GUE) statistics supports compatibility with known phenomenology, but does not uniquely characterize the zeta zeros. Conversely, deviations from GUE do not necessarily invalidate an operator model, especially at low spectral heights.

These tests must therefore be interpreted as supportive diagnostics, not decisive criteria.

7.5 Operator domain and self-adjointness issues

Formal symmetry of an operator does not guarantee self-adjointness. Subtle domain issues may arise, particularly for unbounded operators defined via arithmetic transfers.

Numerical stability alone cannot substitute for rigorous analysis of essential self-adjointness. An operator may appear well-behaved numerically while lacking a mathematically well-defined spectral theory.

The framework highlights such issues, but resolving them ultimately requires analytic techniques beyond its scope.

7.6 Dependence on certified zero data

The reliability of all subsequent diagnostics depends on the correctness and completeness of the certified zero sets. Errors or omissions at this stage can propagate through the entire pipeline.

While checkpoint validation and optional Turing-type methods reduce this risk, they do not eliminate it entirely. Future extensions of the framework may incorporate stronger certification techniques as they become available.

7.7 Computational limitations

Large-scale experiments require substantial computational resources. As spectral windows and truncation dimensions increase, memory and runtime constraints may limit achievable resolutions. These practical limitations necessitate trade-offs between depth and breadth of exploration. The framework is therefore best viewed as an exploratory tool rather than an exhaustive verifier.

7.8 Scope of conclusions

The conclusions supported by this framework are necessarily limited in scope. A high-scoring operator is best interpreted as *consistent with current experimental constraints*, not as confirmed or unique.

Likewise, failure of a particular operator, including the SMRK Hamiltonian, does not imply that the Hilbert–Pólya program itself is misguided. Rather, it refines our understanding of which structural features are incompatible with known arithmetic data.

7.9 Summary

By explicitly acknowledging its limitations, the framework avoids overinterpretation of numerical evidence. Its strength lies not in claiming resolution, but in providing a disciplined environment for eliminating incorrect ideas and focusing analytical effort on the most promising candidates.

8 Outlook and Future Directions

8.1 From experimental consistency to analytic structure

The framework developed in this work establishes a disciplined experimental environment for operator-based investigations of the Riemann Hypothesis. By enforcing simultaneous consistency with certified zero data, explicit prime–zero relations, and stability under refinement, it narrows the space of viable operator candidates.

A natural next step is to translate experimentally successful structures into objects amenable to rigorous analysis. In particular, operators that score highly across all dimensions warrant detailed investigation of their domains, closures, and self-adjoint extensions. Such analysis lies beyond the scope of the present work, but the framework provides a principled filter for selecting promising targets.

8.2 Toward rigor in operator analysis

The ultimate goal of the Hilbert–Pólya program is not numerical agreement, but a mathematically precise correspondence between arithmetic structure and spectral theory. For candidate operators identified by the framework, future work may address:

- proofs of essential self-adjointness,
- characterization of spectral measures,
- derivation of trace formulas from first principles,
- control of error terms and asymptotics.

The experimental framework can inform these efforts by highlighting which features appear structurally necessary and which are likely artifacts of approximation.

8.3 Extension to families of L -functions

Although this work focuses on the Riemann zeta function, the framework is not intrinsically limited to it. Many components, including certified zero pipelines and explicit formula diagnostics, extend naturally to families of L -functions.

Exploring operator models that simultaneously encode multiple L -functions may reveal deeper structural principles governing arithmetic spectra. Such extensions offer a pathway from isolated conjectures toward a unified operator-theoretic view of analytic number theory.

8.4 Computational scaling and distributed architectures

As numerical resolution increases, computational demands become significant. Future implementations may benefit from distributed or parallel architectures, allowing larger spectral windows and higher truncation dimensions to be explored.

The framework is compatible with such extensions, provided reproducibility and data integrity are preserved. Distributed computation may thus serve as an enabling technology, not as a conceptual dependency.

8.5 Interaction with broader operator frameworks

The experimental methodology outlined here is not restricted to a single operator construction. Arithmetic transfer Hamiltonians, differential operators, and hybrid models can all be evaluated within the same structure.

This flexibility encourages cross-fertilization between different approaches to the Riemann Hypothesis and reduces fragmentation in operator-based research. Negative results are as informative as positive ones, clarifying which conceptual directions deserve further attention.

8.6 Final perspective

The Riemann Hypothesis occupies a unique position at the intersection of arithmetic, analysis, and spectral theory. While a proof remains elusive, progress need not be confined to isolated theoretical advances.

By treating operator proposals as experimentally testable objects, the framework presented here offers a path toward cumulative understanding. Its value lies not in resolving the conjecture directly, but in structuring inquiry, eliminating incompatible ideas, and guiding analytical effort toward the most coherent and constrained models.

In this sense, the framework aims to serve as an intermediate layer between numerical exploration and rigorous mathematics, supporting steady progress on one of the deepest problems in mathematics.

APPENDIX A

A.1 Numerical Protocols (Pseudocode)

This appendix records the core numerical protocols referenced in Sections 2–3. The purpose is not to prescribe a single implementation, but to fix a reproducible logical structure: *zero discovery* $\rightarrow$ *root refinement* $\rightarrow$ *counting / checkpoints* $\rightarrow$ *certified zero set*.

A.1.1 Configuration parameters

We assume a configuration object `cfg` containing numerical parameters. Typical fields include:

```
cfg.T_max           # maximum height
cfg.dt_init         # initial scan step
cfg.dt_min          # minimal step for adaptive scan
cfg.dt_max          # maximal step for adaptive scan
cfg.eps_root        # target bracket width for refined roots
cfg.max_refine_iter  # max iterations in root refinement
cfg.checkpoints      # list of checkpoint heights [T1, T2, ..., Tk]
cfg.count_tolerance  # allowed discrepancy in checkpoint counts
cfg.smoothing_params # parameters for smoothed explicit-formula tests
```

The framework does not depend on specific values, but requires that all values are logged and version-controlled to ensure reproducibility.

A.1.2 Protocol P0: Certified zero computation

P0.1: Scan and bracket zeros via sign changes

We use the Riemann–Siegel Z -function $Z(t)$ on the critical line. The scan step may be adaptive.

```
function scan_brackets(T_max, cfg):
    t = 0
    dt = cfg.dt_init
    brackets = []

    Z_prev = Z(t)

    while t < T_max:
        t_next = min(t + dt, T_max)
        Z_next = Z(t_next)

        if Z_prev == 0:
            # Rare in practice; treat as degenerate bracket
            brackets.append([t, t])
        else if Z_prev * Z_next < 0:
            brackets.append([t, t_next])
```

```

# Optional adaptive stepping heuristic:
# if |Z_next| is small or oscillatory, reduce dt; otherwise increase.
dt = adapt_step(dt, Z_prev, Z_next, cfg)

t = t_next
Z_prev = Z_next

return brackets

```

P0.2: Refine each bracket to a certified root interval

Any monotone method is acceptable (bisection / hybrid). We record both the refined interval and the approximate root.

```

function refine_root_interval(a, b, cfg):
    Za = Z(a)
    Zb = Z(b)

    if a == b:
        return (a, b)    # degenerate bracket case

    assert Za * Zb <= 0

    for iter in 1..cfg.max_refine_iter:
        if (b - a) <= cfg.eps_root:
            break

        m = 0.5 * (a + b)
        Zm = Z(m)

        if Zm == 0:
            a = m
            b = m
            break
        else if Za * Zm < 0:
            b = m
            Zb = Zm
        else:
            a = m
            Za = Zm

    return (a, b)

```

P0.3: Build the certified zero set up to each checkpoint

We maintain an ordered list of refined root intervals. We then enforce checkpoint validation against a theoretical count model.

```

function compute_certified_zeros(cfg):
    brackets = scan_brackets(cfg.T_max, cfg)

```



```

refined = []
for [a,b] in brackets:
    (ra, rb) = refine_root_interval(a, b, cfg)
    refined.append([ra, rb])

# Sort by interval center
refined.sort_by_center()

# Checkpoints: validate completeness
for T in cfg.checkpoints:
    count_found = number_of_intervals_with_center_leq(refined, T)
    count_pred = predicted_zero_count_N(T)

    if abs(count_found - count_pred) > cfg.count_tolerance:
        raise Error("Checkpoint validation failed at T = " + str(T))

# Output as certified zero set:
zeros = []
for [ra, rb] in refined:
    gamma = 0.5*(ra + rb) # representative value
    zeros.append(gamma)

return zeros, refined

```

A.1.3 Protocol P1: Data products and logging

To prevent silent drift between experiments, the following artifacts should be logged:

- cfg used for the run (serialized)
- list of refined root intervals [a_n, b_n]
- representative zeros gamma_n
- checkpoint table: (T_j, count_found, count_pred, delta)
- hash / checksum of the produced zero set file

A minimal table format for checkpoint output is:

```

T_j | found | predicted | delta
-----+-----+-----+-----
... | ...   | ...       | ...

```

A.1.4 Protocol P2: Explicit formula residuals (high-level)

Section 3 introduces smoothed prime-zero diagnostics. We record the minimal interface here.

P2.1: Prime-side evaluation

```

function compute_Psi_prime_side(X_grid, f, cfg):
    # Psi_f(X) = sum_{n>=1} Lambda(n) * f(log n - log X)
    # In practice truncate n up to N_max(X) determined by f and cfg.
    Psi = []

```

```

for X in X_grid:
    val = 0
    for n in relevant_n(X, f, cfg):
        val += Lambda(n) * f(log(n) - log(X))
    Psi.append(val)
return Psi

```

P2.2: Zero-side evaluation (truncated)

```

function compute_Psi_zero_side(X_grid, f_hat, zeros, cfg):
    # Psi_{f,T}(X) = X * f_hat(0) - sum_{rho in Z(T)} X^{rho} * f_hat(rho)
    # Here rho = 1/2 + i*gamma for critical-line zeros.
    PsiT = []
    for X in X_grid:
        val = X * f_hat(0)
        for gamma in zeros:
            rho = 0.5 + i*gamma
            val -= X**rho * f_hat(rho)
        PsiT.append(real(val)) # after pairing +/-gamma, result is real
    return PsiT

```

P2.3: Residual construction and stability checks

```

function compute_residuals(X_grid, Psi, PsiT):
    R = []
    for k in 1..len(X_grid):
        R.append(Psi[k] - PsiT[k])
    return R

function stability_sweep(cfg_list):
    # Vary T, smoothing, grid density, etc.
    # Track whether residual structure persists under refinement.
    results = []
    for cfg in cfg_list:
        zeros, intervals = compute_certified_zeros(cfg)
        Psi = compute_Psi_prime_side(cfg.X_grid, cfg.f, cfg)
        PsiT = compute_Psi_zero_side(cfg.X_grid, cfg.f_hat, zeros, cfg)
        R = compute_residuals(cfg.X_grid, Psi, PsiT)
        results.append(summary_statistics(R, cfg))
    return results

```

A.1.5 Notes on certification strength

The above protocols define the *logical* structure of certification. For increased rigor, checkpoint validation may be strengthened by Turing-type methods and explicit bounds on $\arg \zeta(s)$ integrals. The framework is compatible with such enhancements; the minimal requirement is that the resulting zero set is treated as certified input for subsequent spectral and explicit-formula computations.

APPENDIX B

B.1 Reproducibility and Data Integrity

This appendix specifies the minimal reproducibility standard for the experimental framework introduced in Sections 2–3. The goal is to ensure that numerical claims (e.g. “certified zeros up to height T ” or “residual norms across scales”) can be independently re-run, verified, and compared.

B.1.1 Principles

We adopt the following principles:

1. **Determinism where possible.** If an experiment uses randomness (e.g. randomized grids, Monte-Carlo probes), the random seed must be fixed and recorded.
2. **Full provenance.** Every produced dataset must carry enough metadata to uniquely identify the code version, configuration, and environment.
3. **Immutable artifacts.** Primary outputs (zero lists, checkpoint tables, residual arrays) are treated as immutable once published. Any change creates a new artifact with a new hash.
4. **Comparability.** Artifacts must be stored in stable formats with documented schemas, so results can be compared across versions and implementations.

B.1.2 Version control requirements

All code used to generate numerical artifacts must be version controlled. A minimal record for each run includes:

- repository URL / identifier
- git commit hash (or equivalent immutable revision id)
- dirty flag (whether local changes existed)
- branch / tag name (optional)

If the experiment is performed outside a version-controlled repository, a snapshot archive of the code must be produced and hashed alongside the outputs.

B.1.3 Environment capture

The computational environment can affect numerical results. For each run, record at minimum:

- OS name and version
- CPU architecture (and optional instruction set flags)
- language/runtime version (Python/Julia/C++ compiler etc.)
- numerical library versions (mpmath, Arb, MPFR, FFT libs, etc.)
- floating-point precision settings (binary64, mp.dps, Arb prec, etc.)

If containerization is used (Docker/Podman), the container image hash should be recorded and preferred as a canonical environment reference.

B.1.4 Configuration logging

All experiment parameters must be serialized and stored as part of the run. We recommend using a plain text format such as JSON or YAML. A minimal configuration must include:

- `T_max`
- scan parameters: `dt_init`, `dt_min`, `dt_max`
- root refinement parameters: `eps_root`, `max_refine_iter`
- checkpoint heights and count tolerances
- smoothing parameters for explicit-formula tests
- grids used: `X_grid` specification, `spacing`, `bounds`

The configuration file itself must be hashed and referenced by that hash in every output file.

B.1.5 Artifact formats and schemas

To support long-term comparability, we recommend the following stable formats:

- **Zeros:** a line-oriented text file (CSV/TSV) or JSONL containing one row per zero.
- **Root intervals:** CSV/TSV with columns (a_n, b_n, γ_n) where $\gamma_n = \frac{1}{2}(a_n + b_n)$.
- **Checkpoint table:** CSV/TSV with columns $(T_j, \text{found}, \text{predicted}, \Delta)$.
- **Residual arrays:** CSV/TSV or a binary format (Parquet/NPY) with explicit column names and units.

If binary formats are used, a text manifest must accompany them to document the schema and the software required to read the file.

B.1.6 Cryptographic hashing and manifests

Every primary artifact produced by the framework must be hashed. Let $H(\cdot)$ denote a cryptographic hash function (e.g. SHA-256). For each run, produce a manifest file containing:

- artifact filename
- artifact byte size
- SHA-256 hash
- creation timestamp (UTC)
- config hash reference
- code revision reference

A recommended minimal manifest structure is:

```
run_id: <string>
created_utc: <timestamp>
code_revision: <commit-hash>
config_sha256: <hash>
artifacts:
- name: zeros_Tmax_<...>.tsv
bytes: <...>
sha256: <...>
```

```
- name: checkpoints_<...>.tsv
bytes: <...>
sha256: <...>
- name: residuals_<...>.tsv
bytes: <...>
sha256: <...>
```

B.1.7 Run identifiers and naming conventions

We recommend a deterministic run identifier:

$$\text{run_id} := H(\text{code_revision} \parallel \text{config_hash} \parallel \text{environment_summary}),$$

where $\parallel$ denotes concatenation of canonicalized strings.

Artifact filenames should include at least:

- the run identifier (or a prefix of it),
- the maximum height $T_{\max}$ for zero computations,
- the smoothing/test-function label for explicit-formula runs.

Example:

```
zeros_Tmax_1e6_run_ab12cd34.tsv
checkpoints_Tmax_1e6_run_ab12cd34.tsv
residuals_gauss_sigma0p2_Tmax_1e6_run_ab12cd34.tsv
manifest_run_ab12cd34.yaml
```

B.1.8 Minimal comparability tests

To confirm that two runs are comparable, the following must match:

1. identical code revision (or documented differences),
2. identical configuration hash,
3. compatible numerical precision settings,
4. identical artifact schema versions.

If any of these differ, results may still be scientifically comparable, but should not be treated as byte-identical reproductions. Such cases must be documented explicitly.

B.1.9 Integrity notes on timestamping

If third-party timestamping is used (e.g. OpenTimestamps), it should be applied to the manifest file rather than to individual artifacts. This anchors the full run state (code revision, configuration, and all output hashes) under a single immutable timestamp commitment.

Timestamping strengthens provenance claims, but does not replace reproducibility requirements: the experiment must remain runnable from the recorded code and configuration.

B.1.10 Summary

The reproducibility protocol above defines the minimum standard for treating numerical results in this work as verifiable scientific artifacts. The intent is to support cumulative progress: future operator models and refinements can be compared against prior results without ambiguity about computational provenance or data integrity.

Chapter 19

Numerical Results for the SMRK Hamiltonian

Spectral Matching, Explicit Formula Residuals, and Stability Analysis

Enter Yourname

Abstract

This document reports a comprehensive set of numerical results for the SMRK Hamiltonian formulated as an arithmetic transfer operator on the space of square-summable sequences over the natural numbers. The goal is not numerical proof of the Riemann Hypothesis, but the construction of reproducible and scalable diagnostics.

The analysis focuses on spectral alignment between computed eigenvalues and reference zero data, consistency with explicit-formula-style prime diagnostics, and stability under variations of truncation dimension, prime cutoff, and potential parameters. Normalized error measures and the identification of stable parameter regimes are emphasized.

A central criterion for validity is robustness: meaningful signals persist under refinement, respect expected symmetries, and reproduce under identical configurations. The document specifies configuration standards, solver choices, and data formats, providing a baseline dataset and methodology for future iterations of the SMRK program.

1 Experimental Setup

1.1 Purpose and scope

This section specifies the precise numerical setup used to generate the results reported in this work. All methodological principles, including certified zero computation and explicit formula diagnostics, are inherited from the experimental framework defined in Whitepaper IV and are not repeated here.

The present section focuses exclusively on fixing the concrete realization of the SMRK Hamiltonian, its numerical truncation, and the procedures used to extract spectral and trace data. All results in subsequent sections should be interpreted relative to this setup.

1.2 Hilbert space and basis

The SMRK Hamiltonian is realized as an operator acting on the Hilbert space

$$\mathcal{H} = \ell^2(\mathbb{N}),$$

with the canonical basis $\{|n\rangle\}_{n \geq 1}$.

For numerical purposes, we introduce a finite-dimensional truncation

$$\mathcal{H}_N = \text{span}\{|1\rangle, |2\rangle, \dots, |N\rangle\},$$

where N denotes the truncation dimension. All reported spectra are obtained from operators restricted to $\mathcal{H}_N$, with N varied systematically to assess stability under refinement.

1.3 Definition of the SMRK Hamiltonian

In the experiments reported here, the SMRK Hamiltonian is defined as an arithmetic transfer operator of the form

$$(H_{\text{SMRK}}\psi)(n) = \sum_{p \leq P_{\max}} \frac{1}{p} \left(\psi(pn) + \mathbf{1}_{p|n} \psi(n/p) \right) + (\alpha \Lambda(n) + \beta \log n) \psi(n),$$

where:

- the sum runs over prime numbers p ,
- $P_{\max}$ is a prime cutoff parameter,
- $\Lambda(n)$ denotes the von Mangoldt function,
- $\mathbf{1}_{p|n}$ is the indicator function of divisibility,
- $\alpha, \beta \in \mathbb{R}$ are scalar parameters.

This form represents a symmetric prime-indexed transfer Hamiltonian with a diagonal arithmetic potential. The choice of weights $1/p$ and the inclusion of the logarithmic term are motivated by compatibility with explicit formula structures, but are not assumed to be optimal or unique.

1.4 Matrix representation and truncation

On the truncated space $\mathcal{H}_N$, the operator H_{SMRK} is represented as an $N \times N$ real symmetric matrix. Entries corresponding to transitions outside the truncated index range are discarded.

Explicitly, matrix elements are given by

$$\langle m | H_{\text{SMRK}} | n \rangle = \sum_{p \leq P_{\max}} \frac{1}{p} \left(\delta_{m,pn} + \delta_{m,n/p} \mathbf{1}_{p|n} \right) + \delta_{m,n} (\alpha \Lambda(n) + \beta \log n),$$

for $1 \leq m, n \leq N$.

The truncation introduces boundary effects, particularly for large n near N . These effects are monitored by varying both N and $P_{\max}$ and by excluding edge regions from sensitive diagnostics where appropriate.

1.5 Numerical diagonalization

Eigenvalues of the truncated Hamiltonian are computed using standard dense or sparse symmetric eigensolvers, depending on the truncation size.

All computations are performed with explicitly recorded numerical precision. For large N , only the lowest portion of the spectrum is computed, as higher eigenvalues are more sensitive to truncation artifacts.

Eigenvalues are ordered increasingly and denoted by

$$\lambda_1^{(N)} \leq \lambda_2^{(N)} \leq \dots.$$

1.6 Reference zeta zero data

Spectral comparisons are performed against certified nontrivial zeros of the Riemann zeta function on the critical line.

Let $\{\gamma_n\}$ denote the ordered set of imaginary parts of zeros used as reference data. All zero sets employed in this work satisfy the certification criteria defined in Whitepaper IV, including checkpoint validation.

The number of reference zeros used in each comparison is matched to the number of reliably computed eigenvalues to avoid extrapolation bias.

1.7 Parameter ranges and refinement strategy

The following parameters are varied to assess stability:

- truncation dimension N ,
- prime cutoff $P_{\max}$,
- potential coefficients α and β ,
- numerical precision and solver tolerances.

Parameter sweeps are performed systematically, with each configuration recorded as a separate reproducible run. Only trends that persist across multiple refinement levels are interpreted as meaningful.

1.8 Summary

This section fixes the concrete numerical realization of the SMRK Hamiltonian used throughout this paper. By explicitly specifying the operator form, truncation scheme, and reference data, it ensures that all reported results are reproducible and interpretable within a well-defined context.

2 Spectral Matching Results

2.1 Objective of spectral comparison

The primary numerical test for the SMRK Hamiltonian is the comparison of its spectrum with the imaginary parts of the nontrivial zeros of the Riemann zeta function.

This section reports direct spectral matching results between the eigenvalues of truncated SMRK Hamiltonians and certified zeta zero data. The purpose is to quantify the degree of agreement, its dependence on truncation parameters, and its stability under refinement.

2.2 Eigenvalue–zero alignment

Let

$$\lambda_1^{(N)} \leq \lambda_2^{(N)} \leq \dots$$

denote the ordered eigenvalues of the truncated SMRK Hamiltonian acting on $\mathcal{H}_N$, and let

$$\gamma_1 \leq \gamma_2 \leq \dots$$

denote the ordered imaginary parts of certified nontrivial zeros of the Riemann zeta function.

Spectral alignment is assessed by comparing $\lambda_n^{(N)}$ with γ_n for indices n within a stable spectral window. The comparison is restricted to ranges where boundary effects and numerical instabilities are negligible.

2.3 Normalized spectral deviation

To quantify spectral agreement, we define the normalized deviation

$$\Delta_n^{(N)} = \frac{|\lambda_n^{(N)} - \gamma_n|}{\gamma_n}.$$

This quantity allows comparison across different spectral heights and truncation dimensions. Aggregate measures such as

$$\overline{\Delta}^{(N)} = \frac{1}{M} \sum_{n=1}^M \Delta_n^{(N)}$$

are reported for fixed M and varied N , where M denotes the number of reliably resolved eigenvalues.

2.4 Dependence on truncation dimension

Spectral deviation is examined as a function of the truncation dimension N . As N increases, one of the following behaviors is observed:

- monotone decrease of $\overline{\Delta}^{(N)}$,
- saturation at a finite error level,
- growth or oscillation indicating instability.

Only the first behavior is interpreted as evidence of asymptotic spectral correspondence. The other behaviors signal truncation artifacts or structural incompatibility.

2.5 Low-lying versus higher eigenvalues

Low-lying eigenvalues often exhibit larger relative deviations, due to stronger sensitivity to boundary conditions and diagonal potential terms.

For this reason, spectral agreement is evaluated separately for:

- low-index eigenvalues,
- intermediate spectral ranges,
- the highest reliably computed eigenvalues.

This stratification helps distinguish localized discrepancies from global spectral trends.

2.6 Robustness under parameter variation

The spectral comparison is repeated for multiple values of the parameters α , β , $P_{\max}$, and N . Parameter regimes yielding qualitatively similar spectral alignment are identified.

Conversely, parameter choices that produce spurious agreement only within narrow tuning windows are flagged as unstable. Such regimes are documented but not treated as evidence of structural validity.

2.7 Summary of spectral findings

The results of this section demonstrate that the SMRK Hamiltonian can exhibit partial spectral alignment with the nontrivial zeros of the Riemann zeta function over finite ranges and resolutions.

However, spectral matching alone is insufficient to establish relevance to the Riemann Hypothesis. Accordingly, these results are interpreted as preliminary consistency checks, to be complemented by statistical, arithmetic, and stability diagnostics in subsequent sections.

3 Level Spacing Statistics

3.1 Motivation for spacing analysis

While pointwise spectral matching tests whether individual eigenvalues approximate the imaginary parts of zeta zeros, it does not capture the local statistical structure of the spectrum.

Level spacing statistics probe short-range correlations between neighboring eigenvalues. For the nontrivial zeros of the Riemann zeta function, extensive numerical evidence indicates agreement with the Gaussian Unitary Ensemble (GUE) of random matrix theory.

In this section, we examine whether the spectrum of the SMRK Hamiltonian exhibits compatible local statistics, independently of absolute alignment.

3.2 Nearest-neighbor spacings

Let

$$\lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_M$$

denote a sequence of reliably computed eigenvalues of the truncated SMRK Hamiltonian.

The nearest-neighbor spacings are defined as

$$s_n = \lambda_{n+1} - \lambda_n, \quad n = 1, \dots, M-1.$$

Raw spacings depend on the local density of eigenvalues. To allow comparison across spectral regions, the spectrum must be unfolded.

3.3 Spectral unfolding

Unfolding transforms the eigenvalues to a new sequence with approximately unit mean spacing. Let $N(\lambda)$ denote the integrated spectral density, estimated either empirically or via a smooth fit. The unfolded eigenvalues are given by

$$\tilde{\lambda}_n = N(\lambda_n),$$

and the unfolded spacings by

$$\tilde{s}_n = \tilde{\lambda}_{n+1} - \tilde{\lambda}_n.$$

Care is taken to ensure that the unfolding procedure does not artificially impose random-matrix-like behavior. Multiple unfolding methods are tested to assess robustness.

3.4 Comparison with random matrix ensembles

The empirical distribution of unfolded spacings $\tilde{s}_n$ is compared with standard random matrix ensembles. The primary reference is the Gaussian Unitary Ensemble (GUE), whose spacing distribution is

$$P_{\text{GUE}}(s) = \frac{32}{\pi^2} s^2 e^{-4s^2/\pi}.$$

Alternative ensembles, such as the Gaussian Orthogonal Ensemble (GOE) and Poisson statistics, are used as controls.

Deviation between the empirical distribution and reference distributions is quantified using the Kolmogorov–Smirnov distance and related measures.

3.5 Finite-size and boundary effects

Spacing statistics are sensitive to finite-size effects, particularly for low-lying eigenvalues and near truncation boundaries.

To mitigate these effects, statistics are computed only on interior spectral windows, excluding regions strongly affected by truncation or diagonal dominance.

Results are reported for multiple window sizes to assess consistency.

3.6 Results and observed trends

Across tested parameter regimes, the spacing statistics of the SMRK Hamiltonian exhibit mixed behavior.

In certain configurations, the unfolded spacings show partial agreement with GUE predictions over limited spectral ranges. In other regimes, significant deviations are observed, including excess level clustering or near-Poisson behavior.

These observations indicate that local spectral correlations are highly sensitive to parameter choices and truncation details.

3.7 Interpretation and limitations

Agreement with GUE-like statistics is interpreted as supportive evidence of compatibility with known zeta zero phenomenology, but not as confirmation of arithmetic relevance.

Conversely, failure to reproduce GUE statistics does not necessarily invalidate the SMRK Hamiltonian. Finite-size effects, incomplete unfolding, or unresolved truncation artifacts may obscure universal behavior.

Accordingly, level spacing statistics are treated as a diagnostic tool, to be considered alongside spectral matching, explicit formula residuals, and stability tests.

3.8 Summary

Level spacing analysis provides an important statistical perspective on the SMRK Hamiltonian spectrum. While partial agreement with random matrix predictions can be observed in selected regimes, the results emphasize the necessity of complementary diagnostics and careful interpretation.

4 Explicit Formula Residual Results

4.1 Purpose of the explicit formula test

Spectral agreement and level spacing statistics do not by themselves guarantee arithmetic relevance. The explicit formula provides a direct bridge between the distribution of primes and the spectral data.

In this section, we evaluate the SMRK Hamiltonian using explicit formula diagnostics, comparing prime-side quantities with zero-side reconstructions derived from the operator spectrum. The resulting residuals serve as a stringent test of arithmetic compatibility.

4.2 Prime-side reference quantities

Let $\Lambda(n)$ denote the von Mangoldt function. We consider smoothed prime-weighted sums of the form

$$\Psi_f(X) = \sum_{n=1}^{\infty} \Lambda(n) f(\log n - \log X),$$

where f is a smooth test function with compact support or rapid decay.

In numerical implementations, the sum is truncated according to the support of f and recorded with controlled error bounds. These quantities serve as independent arithmetic reference data.

4.3 Zero-side reconstruction from the SMRK spectrum

Given a finite set of eigenvalues $\{\lambda_n\}$ of the truncated SMRK Hamiltonian, we construct the zero-side approximation

$$\Psi_{f,H}(X) = X \hat{f}(0) - \sum_n X^{\rho_n} \hat{f}(\rho_n), \quad \rho_n = \frac{1}{2} + i\lambda_n,$$

where $\hat{f}$ denotes the Mellin transform of f .

This construction mirrors the truncated explicit formula used for certified zeta zeros, with operator eigenvalues playing the role of imaginary parts of nontrivial zeros.

4.4 Definition of residuals

The explicit formula residual associated with the SMRK Hamiltonian is defined as

$$R_H(X) = \Psi_f(X) - \Psi_{f,H}(X).$$

Under ideal circumstances, $R_H(X)$ should consist primarily of truncation effects, smoothing artifacts, and controlled lower-order terms. Persistent structured deviations indicate incompatibility between the operator-induced spectrum and the arithmetic data.

4.5 Residual magnitude and scaling

Residuals are evaluated over a logarithmic grid of X values. Their magnitude is quantified using norms such as

$$\|R_H\|^2 = \int_{X_{\min}}^{X_{\max}} |R_H(X)|^2 dX,$$

as well as pointwise statistics.

The dependence of residual norms on truncation dimension, prime cutoff, and smoothing parameters is systematically recorded.

4.6 Stability under refinement

A central diagnostic is the behavior of residuals under refinement. Specifically, we examine whether:

- residual magnitudes decrease as truncation dimension increases,
- residual structure remains stable under changes in the test function f ,
- oscillatory features persist across refinement levels.

Residual patterns that remain stable under refinement are interpreted as structural. Those that diminish or change qualitatively are attributed to truncation artifacts.

4.7 Observed behavior

Numerical experiments reveal that explicit formula residuals for the SMRK Hamiltonian exhibit nontrivial structure.

In certain parameter regimes, residual magnitudes decrease modestly with increasing resolution, suggesting partial arithmetic compatibility. In other regimes, persistent oscillatory residuals remain, indicating a mismatch between the operator spectrum and prime-weighted sums.

These findings demonstrate that explicit formula diagnostics are more discriminating than spectral or spacing tests alone.

4.8 Interpretation and implications

The residual analysis confirms that arithmetic compatibility is a strong and restrictive constraint. Even operators that perform well in spectral matching may fail explicit formula tests.

The SMRK Hamiltonian does not uniformly pass the explicit formula diagnostics across all tested regimes. This outcome does not invalidate the operator-based approach, but highlights the sensitivity of arithmetic structure to operator details.

4.9 Summary

Explicit formula residuals provide the most stringent numerical test applied in this study. They reveal both promising behavior in selected regimes and clear failure modes in others.

These results underscore the necessity of combining spectral, statistical, and arithmetic diagnostics when evaluating candidate operators for relevance to the Riemann Hypothesis.

5 Stability Under Refinement

5.1 Motivation

Numerical agreement observed at a fixed resolution may arise from truncation artifacts, parameter tuning, or accidental cancellations. To distinguish such effects from structurally meaningful behavior, it is essential to study stability under systematic refinement.

In this section, we examine how the numerical properties of the SMRK Hamiltonian evolve as resolution parameters are varied. Only features that persist across multiple refinement levels are treated as significant.

5.2 Refinement dimensions

Refinement is performed along several independent axes:

- truncation dimension N of the Hilbert space,
- prime cutoff parameter $P_{\max}$,
- numerical precision and solver tolerances,
- spectral window size used for diagnostics,
- smoothing parameters in explicit formula tests.

Each refinement dimension is varied independently and in selected coupled sweeps, allowing interactions between parameters to be identified.

5.3 Spectral stability

Spectral stability is assessed by monitoring the behavior of eigenvalues $\lambda_n^{(N)}$ as N increases.

For fixed index n , a stable spectral feature is characterized by convergence

$$\lambda_n^{(N)} \longrightarrow \lambda_n \quad \text{as } N \rightarrow \infty,$$

up to controlled numerical error.

In practice, convergence is evaluated by tracking relative changes

$$\frac{|\lambda_n^{(N_2)} - \lambda_n^{(N_1)}|}{|\lambda_n^{(N_1)}|}$$

for increasing truncation levels $N_1 < N_2$.

Eigenvalues exhibiting erratic drift, splitting, or reordering under refinement are classified as unstable and excluded from sensitive diagnostics.

5.4 Stability of spectral matching

The normalized spectral deviation defined in Section 2 is examined across refinement levels. A stable matching regime is indicated by:

- decreasing or bounded deviation as N and $P_{\max}$ increase,
- consistent ordering of matched eigenvalues,
- absence of abrupt error growth near truncation boundaries.

Failure of these conditions signals that apparent agreement at low resolution does not extrapolate.

5.5 Stability of level spacing statistics

Level spacing statistics are recomputed for increasing truncation dimensions and for different interior spectral windows.

Stability is indicated by:

- similar unfolded spacing distributions across refinement levels,
- comparable goodness-of-fit metrics with respect to reference ensembles,
- robustness to moderate changes in unfolding procedures.

Strong dependence of spacing statistics on truncation size or window selection is interpreted as evidence of finite-size dominance.

5.6 Stability of explicit formula residuals

Explicit formula residuals provide the most sensitive refinement test. Residual functions $R_H(X)$ are examined for:

- reduction in overall magnitude with increasing resolution,
- persistence or decay of structured oscillatory components,
- qualitative similarity across smoothing parameter choices.

Residual features that remain stable under simultaneous refinement of truncation dimension, prime cutoff, and smoothing scale are interpreted as intrinsic to the operator structure.

5.7 Cross-parameter interactions

Refinement along one axis may amplify or suppress instability along another. For example, increasing $P_{\max}$ without a corresponding increase in N can introduce boundary artifacts, while increased N with insufficient prime coverage may underrepresent arithmetic structure.

Such interactions are explicitly monitored, and only parameter regimes exhibiting coherent behavior across multiple axes are considered reliable.

5.8 Summary of refinement behavior

The refinement analysis reveals that the SMRK Hamiltonian exhibits a mixture of stable and unstable features.

Certain spectral and statistical properties persist under refinement, suggesting genuine structural behavior. Other apparent agreements degrade as resolution increases, indicating truncation-induced artifacts.

These findings reinforce the necessity of refinement-based validation and motivate the careful interpretation of numerical agreement as provisional rather than definitive.

6 Negative Results and Failure Cases

6.1 Role of negative results

Negative results play a central role in the experimental framework adopted in this work. Because the objective is falsification rather than confirmation, documenting failure modes is as important as reporting partial successes.

This section records parameter regimes, numerical configurations, and operator variants for which the SMRK Hamiltonian fails to meet one or more diagnostic criteria. These results delimit the space of plausible operator constructions and help prevent overinterpretation of isolated positive findings.

6.2 Spectral overfitting at low truncation

In several parameter configurations, good spectral alignment with zeta zeros was observed only at low truncation dimensions. As the truncation dimension N increased, spectral deviations grew or eigenvalue ordering became unstable.

Such behavior is interpreted as overfitting to finite spectral data. Operators exhibiting this pattern were excluded from further analysis, even if initial spectral comparisons appeared favorable.

6.3 Instability under prime cutoff variation

The prime cutoff parameter $P_{\max}$ plays a critical role in shaping the arithmetic structure of the SMRK Hamiltonian.

In some regimes, increasing $P_{\max}$ without proportional increase in N led to pronounced boundary effects, manifesting as spectral distortion and increased explicit formula residuals.

Conversely, overly restrictive prime cutoffs failed to capture sufficient arithmetic structure, resulting in near-Poisson spectral statistics. These regimes were classified as inadequate.

6.4 Failure of explicit formula compatibility

Several operator variants exhibited acceptable spectral matching and level spacing statistics, yet failed explicit formula diagnostics.

In these cases, explicit formula residuals remained large or structurally unstable under refinement, indicating a fundamental mismatch between the operator-induced spectrum and prime-weighted sums.

These failures demonstrate that explicit formula tests are strictly stronger than purely spectral diagnostics.

6.5 Parameter sensitivity and fine tuning

Some configurations produced apparent agreement with diagnostics only within narrow parameter windows. Small perturbations of parameters α , β , or $P_{\max}$ resulted in rapid degradation of spectral or arithmetic scores.

Such fine-tuned regimes are not considered structurally meaningful. The framework treats robustness to moderate parameter variation as a necessary condition for relevance.

6.6 Numerical pathologies

Additional failure modes arose from numerical issues, including:

- loss of orthogonality in eigenvectors at large N ,
- sensitivity to solver tolerances,
- accumulation of round-off errors in explicit formula sums.

While these issues can sometimes be mitigated through increased precision or alternative algorithms, they nonetheless impose practical limits on achievable resolution.

6.7 Implications for operator design

The documented failures highlight several lessons for future operator constructions:

- spectral matching alone is insufficient,
- arithmetic structure must be encoded robustly,
- stability under refinement is a non-negotiable requirement,
- fine tuning should be viewed with skepticism.

These observations inform the design of modified or alternative operators and motivate the exploration of broader operator families.

6.8 Summary

The negative results reported here demonstrate the discriminating power of the experimental framework. By explicitly recording failure cases, the study narrows the landscape of plausible operator models and strengthens the credibility of any surviving candidates.

7 Discussion and Interpretation

7.1 Summary of numerical findings

The numerical experiments reported in this work provide a multifaceted assessment of the SMRK Hamiltonian as a candidate operator within the Hilbert–Pólya program.

Across spectral matching, level spacing statistics, explicit formula diagnostics, and refinement analysis, the results reveal a nuanced picture. The SMRK Hamiltonian exhibits partial compatibility with known properties of the nontrivial zeros of the Riemann zeta function, but also displays clear limitations and failure modes.

These findings underscore the importance of evaluating operator proposals along multiple independent dimensions, rather than relying on a single diagnostic.

7.2 Interpretation of spectral agreement

The observed spectral alignment between SMRK eigenvalues and zeta zeros over finite ranges suggests that arithmetic transfer operators can encode nontrivial spectral structure.

However, the lack of uniform convergence under refinement indicates that spectral agreement alone does not establish relevance to the Riemann Hypothesis. Instead, such agreement should be viewed as an initial filter: necessary, but far from sufficient.

This interpretation aligns with the broader understanding that many constructions can reproduce finite spectral data without capturing the underlying arithmetic dynamics.

7.3 Statistical structure versus arithmetic content

Level spacing statistics provide insight into local spectral correlations. Partial agreement with GUE-like behavior in selected regimes is consistent with known phenomenology of zeta zeros.

At the same time, the sensitivity of spacing statistics to truncation and parameter choices highlights their limited discriminating power. Statistical compatibility does not uniquely characterize arithmetic origin and must be interpreted in conjunction with explicit arithmetic tests.

7.4 Explicit formula diagnostics as a decisive filter

Among all diagnostics employed, explicit formula residuals emerge as the most restrictive. They directly test whether the operator-induced spectrum can reproduce prime-weighted sums, rather than merely mimic spectral patterns.

The persistence of structured residuals in many parameter regimes demonstrates that encoding arithmetic content is substantially more demanding than achieving spectral resemblance. This observation reinforces the view that arithmetic compatibility is the defining constraint for any operator relevant to the Riemann Hypothesis.

7.5 Stability and structural significance

Stability under refinement serves as a practical proxy for structural validity. Features that degrade as resolution increases are unlikely to reflect intrinsic properties of the underlying operator.

The mixed stability behavior observed for the SMRK Hamiltonian suggests that certain aspects of its construction capture meaningful structure, while others remain sensitive to truncation artifacts or parameter tuning.

This distinction provides guidance for future refinements and modifications of arithmetic transfer operators.

7.6 Implications for the Hilbert–Pólya program

The results presented here do not support the conclusion that the SMRK Hamiltonian realizes the Hilbert–Pólya conjecture. They do, however, demonstrate that the conjecture can be approached as a falsifiable experimental program.

By embedding candidate operators within a disciplined numerical framework, it becomes possible to systematically exclude incompatible models and focus analytical effort on constructions that survive stringent tests.

This shift from speculative construction to constrained evaluation represents a methodological advance in operator-based research on RH.

7.7 Lessons for future operator constructions

Several general lessons emerge:

- arithmetic structure must be encoded robustly,
- spectral agreement must persist under refinement,
- explicit formula compatibility is essential,
- fine-tuned parameter regimes are suspect.

These lessons apply not only to the SMRK Hamiltonian, but to any future operator proposal aimed at explaining zeta zero statistics.

7.8 Concluding perspective

The numerical study presented in this work illustrates both the promise and the limitations of arithmetic Hamiltonian approaches to the Riemann Hypothesis.

While no definitive conclusions can be drawn regarding RH itself, the framework succeeds in transforming operator proposals into testable scientific objects. In doing so, it helps clarify which directions merit deeper analytical investigation and which are unlikely to succeed.

The value of this work lies not in resolving the Riemann Hypothesis, but in advancing a structured pathway toward understanding the interplay between arithmetic, operators, and spectral phenomena.

APPENDIX A

A.1 Numerical Parameters and Configurations

This appendix records the numerical parameters used in the experiments reported in this work. All results in Sections 2–6 should be interpreted relative to these configurations.

Unless stated otherwise, each parameter set corresponds to an independent reproducible run, as defined by the reproducibility protocol referenced in Whitepaper IV.

A.1.1 Truncation Parameters

The SMRK Hamiltonian is truncated to finite-dimensional subspaces $\mathcal{H}_N \subset \ell^2(\mathbb{N})$. The truncation dimension N is varied to assess stability.

Typical values used in the experiments include:

`N` in {200, 400, 800, 1200, 2000}

Only interior spectral regions, sufficiently separated from truncation boundaries, are used for sensitive diagnostics.

A.1.2 Prime Cutoff

The prime cutoff parameter $P_{\max}$ limits the range of primes included in the transfer terms.

Representative values include:

`P_max` in {11, 23, 47, 97, 197}

For each truncation dimension N , $P_{\max}$ is chosen such that prime-induced transitions do not dominate boundary effects.

A.1.3 Diagonal Potential Coefficients

The diagonal arithmetic potential is parameterized by real coefficients α and β :

$$U(n) = \alpha \Lambda(n) + \beta \log n.$$

Explored parameter ranges include:

`alpha` in [0.0, 1.0]

`beta` in [-1.0, 1.0]

Parameter sweeps are performed with step sizes chosen to detect sensitivity without fine tuning.

A.1.4 Numerical Precision and Solvers

Eigenvalue computations are performed using symmetric eigensolvers, with numerical precision adjusted according to truncation size.

Typical settings include:

- double precision for $N \leq 800$
- extended precision for $N > 800$
- solver tolerance $\leq 1e-10$

Precision settings are logged for each run and included in the reproducibility manifest.

A.1.5 Reference Zero Data

Certified nontrivial zeros of the Riemann zeta function are used as reference data.

For each truncation level, only as many zeros are used as can be reliably matched to stable eigenvalues. Zero sets satisfy the certification criteria described in Whitepaper IV, including checkpoint validation.

A.1.6 Explicit Formula Parameters

Explicit formula diagnostics use smoothed test functions f with compact support or rapid decay.

Representative choices include:

- Gaussian kernels with sigma in $\{0.1, 0.2, 0.3\}$
- compact bump functions with fixed width

Residuals are evaluated on logarithmic grids spanning multiple scales in X .

A.1.7 Run Identification

Each numerical run is assigned a deterministic run identifier, computed as a hash of:

- code revision identifier,
- serialized configuration parameters,
- environment summary.

This identifier is referenced by all output artifacts and manifests associated with the run.

A.1.8 Notes on Extensibility

The parameter ranges reported here are not intended to be exhaustive. They represent a baseline configuration sufficient to reveal both partial agreement and failure modes.

Future studies may extend these ranges or introduce additional parameters, provided that reproducibility and stability analysis are maintained.

APPENDIX B

B.1 Reproducibility and Data Integrity

This appendix specifies the minimal reproducibility standard for the experimental framework introduced in Sections 2–3. The goal is to ensure that numerical claims (e.g. “certified zeros up to height T ” or “residual norms across scales”) can be independently re-run, verified, and compared.

B.1.1 Principles

We adopt the following principles:

1. **Determinism where possible.** If an experiment uses randomness (e.g. randomized grids or Monte Carlo probes), the random seed must be fixed and recorded.
2. **Full provenance.** Every produced dataset must carry enough metadata to uniquely identify the code version, configuration, and environment.
3. **Immutable artifacts.** Primary outputs (zero lists, checkpoint tables, residual arrays) are treated as immutable once published. Any change creates a new artifact with a new hash.
4. **Comparability.** Artifacts must be stored in stable formats with documented schemas, so results can be compared across versions and implementations.

B.1.2 Version Control Requirements

All code used to generate numerical artifacts must be version controlled. A minimal record for each run includes:

- repository URL / identifier
- git commit hash (or equivalent immutable revision id)
- dirty flag (whether local changes existed)
- branch / tag name (optional)

If the experiment is performed outside a version-controlled repository, a snapshot archive of the code must be produced and hashed alongside the outputs.

B.1.3 Environment Capture

The computational environment can affect numerical results. For each run, record at minimum:

- OS name and version
- CPU architecture (and optional instruction set flags)
- language/runtime version (Python/Julia/C++ compiler, etc.)
- numerical library versions (mpmath, Arb, MPFR, FFT libraries, etc.)
- floating-point precision settings (binary64, mp.dps, Arb precision, etc.)

If containerization is used (Docker/Podman), the container image hash should be recorded and preferred as a canonical environment reference.

B.1.4 Configuration Logging

All experiment parameters must be serialized and stored as part of the run. We recommend using a plain text format such as JSON or YAML. A minimal configuration must include:

- T_max
- scan parameters: dt_init, dt_min, dt_max
- root refinement parameters: eps_root, max_refine_iter
- checkpoint heights and count tolerances
- smoothing parameters for explicit-formula tests
- grids used: X_grid specification, spacing, bounds

The configuration file itself must be hashed and referenced by that hash in every output file.

B.1.5 Artifact Formats and Schemas

To support long-term comparability, we recommend the following stable formats:

- **Zeros:** line-oriented text (CSV/TSV) or JSONL with one row per zero.
- **Root intervals:** CSV/TSV with columns (a_n, b_n, γ_n) where $\gamma_n = \frac{1}{2}(a_n + b_n)$.
- **Checkpoint table:** CSV/TSV with columns $(T_j, \text{found}, \text{predicted}, \Delta)$.
- **Residual arrays:** CSV/TSV or binary formats (Parquet/NPY) with explicit column names and units.

If binary formats are used, a text manifest must accompany them to document the schema and the software required to read the file.

B.1.6 Cryptographic Hashing and Manifests

Every primary artifact produced by the framework must be hashed. Let $H(\cdot)$ denote a cryptographic hash function (e.g. SHA-256). For each run, produce a manifest file containing:

- artifact filename
- artifact byte size
- SHA-256 hash
- creation timestamp (UTC)
- config hash reference
- code revision reference

A recommended minimal manifest structure is:

```
run_id: <string>
created_utc: <timestamp>
code_revision: <commit-hash>
config_sha256: <hash>
artifacts:
- name: zeros_Tmax_<...>.tsv
  bytes: <...>
  sha256: <...>
- name: checkpoints_<...>.tsv
  bytes: <...>
```



```
sha256: <...>
- name: residuals_<...>.tsv
bytes: <...>
sha256: <...>
```

B.1.7 Run Identifiers and Naming Conventions

We recommend a deterministic run identifier:

$$\text{run_id} := H(\text{code_revision} \parallel \text{config_hash} \parallel \text{environment_summary}),$$

where $\parallel$ denotes concatenation of canonicalized strings.

Artifact filenames should include at least:

- the run identifier (or a prefix of it),
- the maximum height $T_{\max}$ for zero computations,
- the smoothing or test-function label for explicit-formula runs.

Example:

```
zeros_Tmax_1e6_run_ab12cd34.tsv
checkpoints_Tmax_1e6_run_ab12cd34.tsv
residuals_gauss_sigma0p2_Tmax_1e6_run_ab12cd34.tsv
manifest_run_ab12cd34.yaml
```

B.1.8 Minimal Comparability Tests

To confirm that two runs are comparable, the following must match:

1. identical code revision (or documented differences),
2. identical configuration hash,
3. compatible numerical precision settings,
4. identical artifact schema versions.

If any of these differ, results may still be scientifically comparable, but should not be treated as byte-identical reproductions. Such cases must be documented explicitly.

B.1.9 Integrity Notes on Timestamping

If third-party timestamping is used (e.g. OpenTimestamps), it should be applied to the manifest file rather than to individual artifacts. This anchors the full run state (code revision, configuration, and all output hashes) under a single immutable timestamp commitment.

Timestamping strengthens provenance claims but does not replace reproducibility requirements: the experiment must remain runnable from the recorded code and configuration.

B.1.10 Summary

The reproducibility protocol above defines the minimum standard for treating numerical results in this work as verifiable scientific artifacts. The intent is to support cumulative progress: future

operator models and refinements can be compared against prior results without ambiguity about computational provenance or data integrity.

Chapter 20

Self-Adjoint Operator Framework

Toward an Operator-Theoretic Realization of the Hilbert–Pólya Program

Enter Yourname

Abstract

This document formulates a strictly operator-theoretic framework for arithmetic Hamiltonians of Hilbert–Pólya type and serves as the formal entry point of the identification program. The construction begins with a careful specification of the underlying Hilbert space, operator domains, and elementary building blocks, prior to any discussion of symmetry, closure, or spectral interpretation.

The state space is taken to be the space of square-summable sequences over the natural numbers, equipped with its standard basis indexed by n . On a dense core of finitely supported sequences, prime-indexed shift operators are introduced. These operators implement multiplicative arithmetic dynamics by mapping a state supported at n to a state supported at pn , together with a corresponding inverse action when divisibility allows. In addition, diagonal multiplication operators defined by real-valued arithmetic functions are included as potential terms.

The principal contribution of the document is methodological. It demonstrates how such arithmetic shift and potential operators can be assembled into candidate Hamiltonians while systematically addressing the domain problems inherent to unbounded operators. Conditions for symmetry, closability, and canonical self-adjoint realization are discussed, providing a rigorous foundation for subsequent analytic and numerical work.

At this stage, no claims are made regarding spectral identification or the distribution of zeros of the Riemann zeta function. The objective is instead to eliminate a central weakness of earlier Hilbert–Pólya-type constructions, namely the reliance on formally defined operators lacking controlled domains and well-defined self-adjoint status. The resulting framework provides a clean and auditable operator foundation upon which later developments of the program are built.

1 Mathematical Setting and Definitions

This section establishes the precise mathematical framework in which the operator studied in this work is defined. No spectral, arithmetic, or conjectural claims are made here. The sole

purpose of this section is to specify the underlying Hilbert space, notation, and basic objects with complete rigor.

1.1 Underlying Hilbert Space

Let $\mathbb{N} = \{1, 2, 3, \dots\}$ denote the set of positive integers. We consider the Hilbert space

$$\mathcal{H} := \ell^2(\mathbb{N}),$$

consisting of all complex-valued sequences

$$\psi : \mathbb{N} \rightarrow \mathbb{C}$$

such that

$$\|\psi\|_{\mathcal{H}}^2 = \sum_{n=1}^{\infty} |\psi(n)|^2 < \infty.$$

The inner product on $\mathcal{H}$ is defined by

$$\langle \psi, \phi \rangle = \sum_{n=1}^{\infty} \overline{\psi(n)} \phi(n), \quad \psi, \phi \in \mathcal{H}.$$

This Hilbert space is separable and admits the standard orthonormal basis

$$\{e_n\}_{n \in \mathbb{N}}, \quad e_n(m) = \delta_{n,m}.$$

1.2 Arithmetic Notation

We use the following standard arithmetic functions throughout:

- $\mathcal{P}$ denotes the set of prime numbers.
- $\Lambda(n)$ denotes the von Mangoldt function:

$$\Lambda(n) = \begin{cases} \log p, & \text{if } n = p^k \text{ for some prime } p \text{ and } k \geq 1, \\ 0, & \text{otherwise.} \end{cases}$$

All sums over primes are taken over $\mathcal{P}$ and are understood in the sense specified explicitly when introduced.

1.3 Dense Subspace

Let $\mathcal{D} \subset \mathcal{H}$ denote the vector space of finitely supported sequences:

$$\mathcal{D} = \{\psi \in \mathcal{H} \mid \exists N \in \mathbb{N} \text{ such that } \psi(n) = 0 \text{ for all } n > N\}.$$

Lemma 1.1. The subspace $\mathcal{D}$ is dense in $\mathcal{H}$.

Proof. Every element of $\ell^2(\mathbb{N})$ can be approximated in norm by its finite truncations. □

All operators introduced in this work will be defined initially on $\mathcal{D}$.

1.4 Multiplicative Shift Operators

For each prime $p \in \mathcal{P}$, we define the forward and backward multiplicative shift operators

$$(T_p^+ \psi)(n) := \psi(pn), \quad (T_p^- \psi)(n) := \begin{cases} \psi(n/p), & p \mid n, \\ 0, & p \nmid n. \end{cases}$$

These operators map $\mathcal{D}$ into itself.

Lemma 1.2. For each prime p , the operators T_p^+ and T_p^- are well-defined linear operators on $\mathcal{D}$.

Proof. If ψ has finite support, then both pn and n/p range over finite sets. Hence the resulting sequences are also finitely supported. $\square$

1.5 Diagonal Arithmetic Operators

Let $V : \mathbb{N} \rightarrow \mathbb{R}$ be a real-valued function. We define the associated multiplication operator

$$(M_V \psi)(n) := V(n) \psi(n), \quad \psi \in \mathcal{D}.$$

In particular, we will consider diagonal operators generated by combinations of $\Lambda(n)$ and $\log n$. No assumptions on boundedness are made at this stage.

1.6 Operator Framework

The operator studied in subsequent sections will be constructed as a linear combination of:

- multiplicative shift operators $T_p^\pm$,
- diagonal multiplication operators M_V ,

defined on the common dense domain $\mathcal{D}$.

At this stage, no claims are made regarding:

- symmetry,
- closability,
- self-adjointness,
- spectral properties.

These questions are addressed systematically in the following sections.

1.7 Scope of This Section

This section is purely definitional. All statements made here are elementary and verifiable directly from the definitions. No conjectural or heuristic assumptions are used.

2 Domain and Symmetry of the Operator

In this section we define the operator studied in this work on a common dense domain and establish its symmetry. No claims regarding self-adjointness or spectral properties are made here. The goal is to verify that the operator is a legitimate symmetric operator on a Hilbert space, thereby qualifying as a candidate for a Hilbert–Pólya type construction.

2.1 Definition of the Operator

Let $\alpha, \beta \in \mathbb{R}$ be fixed real parameters. We define a linear operator

$$H : \mathcal{D} \rightarrow \mathcal{H}$$

by the action

$$(H\psi)(n) = \sum_{p \in \mathcal{P}} \frac{1}{p} \left(\psi(pn) + \mathbf{1}_{p|n} \psi(n/p) \right) + (\alpha \Lambda(n) + \beta \log n) \psi(n), \quad (20.1)$$

for all $\psi \in \mathcal{D}$ and $n \in \mathbb{N}$. Here $\mathbf{1}_{p|n}$ denotes the indicator function of the divisibility relation.

Lemma 2.1. For every $\psi \in \mathcal{D}$, the sum in (20.1) is finite for each n , and $H\psi \in \mathcal{D}$.

Proof. Since ψ has finite support, $\psi(pn) \neq 0$ for only finitely many primes p . Likewise, for fixed n , the condition $p \mid n$ holds for only finitely many primes. Thus the sum over p is finite. Moreover, the support of $H\psi$ is contained in a finite set determined by the support of ψ . $\square$

2.2 Domain Considerations

Throughout this work, the operator H is initially defined on the dense subspace $\mathcal{D} \subset \mathcal{H}$. All algebraic manipulations in this section take place on $\mathcal{D}$.

No extension beyond $\mathcal{D}$ is assumed at this stage. Questions of closability and self-adjointness are deferred to the next section.

2.3 Adjoint Relations of the Shift Operators

Let $p \in \mathcal{P}$ be fixed. For $\psi, \phi \in \mathcal{D}$, we compute

$$\langle T_p^+ \psi, \phi \rangle = \sum_{n=1}^{\infty} \overline{\psi(pn)} \phi(n) = \sum_{m=1}^{\infty} \overline{\psi(m)} \mathbf{1}_{p|m} \phi(m/p) = \langle \psi, T_p^- \phi \rangle.$$

Lemma 2.2. On the domain $\mathcal{D}$, the operators T_p^+ and T_p^- are formal adjoints:

$$(T_p^+)^* = T_p^-, \quad (T_p^-)^* = T_p^+.$$

2.4 Symmetry of the Diagonal Part

Let $V(n) = \alpha \Lambda(n) + \beta \log n$. Since $V(n) \in \mathbb{R}$ for all n , the associated multiplication operator M_V satisfies

$$\langle M_V \psi, \phi \rangle = \langle \psi, M_V \phi \rangle \quad \forall \psi, \phi \in \mathcal{D}.$$

2.5 Symmetry of the Full Operator

We now establish the main result of this section.

Theorem 2.1. The operator H defined in (20.1) is symmetric on $\mathcal{D}$, i.e.

$$\langle H\psi, \phi \rangle = \langle \psi, H\phi \rangle \quad \forall \psi, \phi \in \mathcal{D}.$$

Proof. By linearity, it suffices to verify symmetry separately for each component.

The prime-shift part satisfies

$$\left\langle \sum_{p \in \mathcal{P}} \frac{1}{p} (T_p^+ + T_p^-) \psi, \phi \right\rangle = \left\langle \psi, \sum_{p \in \mathcal{P}} \frac{1}{p} (T_p^+ + T_p^-) \phi \right\rangle,$$

by the adjoint relations established above.

The diagonal part is symmetric by construction. Combining these observations yields the claim. $\square$

2.6 Remarks

- Symmetry alone does not imply self-adjointness.
- At this stage, the operator H may admit multiple self-adjoint extensions, or none.
- The purpose of this section is solely to establish H as a well-defined symmetric operator on a dense domain.

The question of essential self-adjointness is addressed in the next section.

3 Essential Self-Adjointness

This section addresses the central structural question of the operator H : whether the symmetric operator defined on the dense domain $\mathcal{D}$ admits a unique self-adjoint extension. This property is a necessary condition for any Hilbert–Pólya type interpretation.

3.1 Preliminaries

Let H denote the symmetric operator introduced in Section 2, with domain $\mathcal{D} \subset \mathcal{H} = \ell^2(\mathbb{N})$.

Recall that a symmetric operator H is called *essentially self-adjoint* if its closure $\overline{H}$ is self-adjoint, or equivalently, if H admits a unique self-adjoint extension.

We emphasize that symmetry alone does not guarantee this property. The analysis below is therefore indispensable.

3.2 Quadratic Form Associated to H

For $\psi \in \mathcal{D}$, define the quadratic form

$$Q[\psi] := \langle \psi, H\psi \rangle. \quad (20.2)$$

By symmetry of H , the form Q is real-valued. Explicitly,

$$\begin{aligned} Q[\psi] &= \sum_{p \in \mathcal{P}} \frac{1}{p} \left(\sum_{n=1}^{\infty} \overline{\psi(n)} \psi(pn) + \sum_{n=1}^{\infty} \overline{\psi(pn)} \psi(n) \right) \\ &\quad + \sum_{n=1}^{\infty} (\alpha \Lambda(n) + \beta \log n) |\psi(n)|^2. \end{aligned} \quad (20.3)$$

All sums are finite for $\psi \in \mathcal{D}$.

3.3 Lower Semiboundedness

Lemma 3.1. There exists a constant $C \in \mathbb{R}$ such that

$$Q[\psi] \geq -C \|\psi\|_{\mathcal{H}}^2 \quad \forall \psi \in \mathcal{D}.$$

Proof. The prime-shift contribution satisfies the estimate

$$\left| \sum_{p \in \mathcal{P}} \frac{1}{p} \sum_{n=1}^{\infty} \overline{\psi(n)} \psi(pn) \right| \leq \left(\sum_{p \in \mathcal{P}} \frac{1}{p^2} \right)^{1/2} \|\psi\|_{\mathcal{H}}^2,$$

by Cauchy–Schwarz and the square-summability of $(1/p)$.

The diagonal terms involving $\Lambda(n)$ and $\log n$ are real-valued. Since $\Lambda(n) \geq 0$ and $\log n \geq 0$ for $n \geq 2$, any possible negative contribution can be absorbed into a global constant depending on α and β .

Combining these estimates yields the claim. $\square$

3.4 Closability and Friedrichs Extension

By Lemma 3.1, the quadratic form Q is lower semibounded. It follows that Q is closable and admits a unique closed extension $\overline{Q}$.

Theorem 3.1. The operator H admits a self-adjoint Friedrichs extension H_F associated with the closed form $\overline{Q}$.

Proof. This is a standard consequence of the representation theorem for closed, densely defined, lower semibounded quadratic forms. $\square$

3.5 Essential Self-Adjointness Criterion

The existence of a Friedrichs extension alone does not imply essential self-adjointness. To establish uniqueness, one must show that no other self-adjoint extensions exist.

We therefore consider the deficiency spaces

$$\mathcal{N}_{\pm} := \ker(H^* \mp i).$$

Theorem 3.2 (Gatekeeper Theorem). If the deficiency indices of H satisfy

$$\dim \mathcal{N}_+ = \dim \mathcal{N}_- = 0,$$

then H is essentially self-adjoint on $\mathcal{D}$.

This theorem is standard; see, e.g., Reed and Simon, Vol. II.

3.6 Reduction of the Problem

At this point, the question of essential self-adjointness is reduced to the absence of nontrivial $\ell^2(\mathbb{N})$ solutions of

$$H^* \psi = \pm i \psi.$$

Equivalently, one must exclude square-summable solutions of the difference equation

$$\sum_{p \in \mathcal{P}} \frac{1}{p} \left(\psi(pn) + \mathbf{1}_{p|n} \psi(n/p) \right) + (\alpha \Lambda(n) + \beta \log n) \psi(n) = \pm i \psi(n). \quad (20.4)$$

3.7 Status of the Result

- The operator H is symmetric and lower semibounded.
- A canonical self-adjoint (Friedrichs) extension exists.
- Essential self-adjointness is reduced to the nonexistence of ℓ^2 solutions of (20.4).

A proof of the vanishing of the deficiency spaces would establish essential self-adjointness and thereby legitimize the operator as a Hilbert–Pólya candidate.

This analysis is deferred to future work.

4 Spectral Framework

This section outlines the spectral-theoretic setting relevant to the operator H , independently of any arithmetic interpretation. No claims are made here regarding the location of eigenvalues or their relation to zeros of zeta or L -functions.

4.1 Self-Adjoint Extensions and Spectra

Let H be the symmetric operator defined on $\mathcal{D} \subset \mathcal{H}$. Whenever a self-adjoint extension $\tilde{H}$ of H exists, the spectral theorem applies and yields a projection-valued measure $E_{\tilde{H}}(\lambda)$ such that

$$\tilde{H} = \int_{\mathbb{R}} \lambda dE_{\tilde{H}}(\lambda).$$

The spectrum $\sigma(\tilde{H})$ decomposes into:

$$\sigma(\tilde{H}) = \sigma_{\text{pp}} \cup \sigma_{\text{ac}} \cup \sigma_{\text{sc}},$$

corresponding to pure point, absolutely continuous, and singular continuous parts.

At this stage, no assumption is made regarding which components are present.

4.2 Resolvent and Spectral Measures

For $z \in \mathbb{C} \setminus \mathbb{R}$, the resolvent operator

$$R(z) := (\tilde{H} - z)^{-1}$$

is bounded and analytic. Spectral information is encoded in matrix elements of the resolvent,

$$\langle \psi, R(z)\phi \rangle = \int_{\mathbb{R}} \frac{1}{\lambda - z} d\mu_{\psi, \phi}(\lambda),$$

where $\mu_{\psi, \phi}$ is the complex spectral measure associated with $\psi, \phi \in \mathcal{H}$.

4.3 Trace and Determinant Structures

If suitable regularization conditions are satisfied, one may formally consider trace expressions of the form

$$\text{Tr } f(\tilde{H}) = \int_{\mathbb{R}} f(\lambda) dN(\lambda),$$

where $N(\lambda)$ denotes the spectral counting function.

In particular, zeta-regularized determinants may be introduced via

$$\det(\tilde{H} - z) := \exp\left(-\frac{d}{ds}\Big|_{s=0} \operatorname{Tr}(\tilde{H} - z)^{-s}\right),$$

whenever the trace is well-defined.

No claim is made here that such determinants exist or converge for H ; this section merely establishes the formal framework.

4.4 Conditional Spectral Interpretation

Should H admit a unique self-adjoint extension, any identification between spectral data of $\tilde{H}$ and arithmetic objects must proceed through explicit trace or resolvent identities.

The logical direction emphasized throughout this work is:

$$\text{operator-theoretic structure} \Rightarrow \text{spectral identities} \Rightarrow \text{arithmetic consequences},$$

never the reverse.

4.5 Scope

This section is intentionally abstract. It introduces no arithmetic assumptions and does not rely on conjectures. Its role is to fix the spectral language required for subsequent analysis.

5 Numerical Experiments

This section reports numerical experiments designed to probe structural features of the operator H . These results are not presented as evidence for the Riemann Hypothesis, but as consistency checks of the operator framework.

5.1 Finite Truncations

The operator H is approximated by finite-dimensional truncations acting on $\ell^2(\{1, \dots, N\})$. Prime sums are truncated to $p \leq P(N)$.

Such truncations break exact symmetry and self-adjointness, but provide insight into qualitative behavior.

5.2 Spectral Observations

Numerical spectra of truncated operators exhibit:

- predominantly real eigenvalues,
- increasing spectral density with N ,
- sensitivity to parameter choices (α, β) .

These observations are consistent with, but do not imply, the existence of a self-adjoint infinite-dimensional limit.

5.3 Stability Tests

Eigenvalue distributions were tested against:

- changes in truncation size,
- variations in prime cutoffs,
- perturbations of diagonal terms.

Qualitative features persist under moderate perturbations.

5.4 Limitations

Finite truncations cannot resolve:

- essential self-adjointness,
- deficiency indices,
- true spectral type.

Numerical experiments serve only as supporting evidence and motivation for further analysis.

5.5 Reproducibility

All numerical experiments are reproducible using deterministic algorithms and fixed parameter choices. Implementation details are documented separately.

6 Riemann Hypothesis as a Corollary

This section clarifies the precise logical relationship between the operator-theoretic framework developed in this work and the Riemann Hypothesis. No new analytical results are introduced. The purpose is solely to state conditional implications in a transparent, non-circular manner.

6.1 Statement of the Riemann Hypothesis

The Riemann Hypothesis asserts that all nontrivial zeros ρ of the Riemann zeta function $\zeta(s)$ satisfy

$$\Re(\rho) = \frac{1}{2}.$$

Equivalently, all nontrivial zeros may be written as

$$\rho = \frac{1}{2} + i\gamma, \quad \gamma \in \mathbb{R}.$$

6.2 Conditional Hilbert–Pólya Principle

The Hilbert–Pólya principle may be formulated abstractly as follows:

If there exists a self-adjoint operator whose spectral data encode the nontrivial zeros of $\zeta(s)$, then the Riemann Hypothesis holds.

The validity of this principle rests entirely on two independent requirements:

1. the existence of a canonical self-adjoint operator,
2. a rigorous identification between its spectral data and the zeros of $\zeta(s)$.

6.3 Logical Structure of the Present Framework

Within the framework developed in this work, the following conditional chain is identified:

- (A) The symmetric operator H defined on $\mathcal{D}$ is essentially self-adjoint.
- (B) The spectrum (or suitably regularized spectral determinant) of the unique self-adjoint extension $\tilde{H}$ coincides with the set of imaginary parts of nontrivial zeros of $\zeta(s)$.

Theorem 6.1 (Conditional Corollary). If conditions (A) and (B) hold, then the Riemann Hypothesis is true.

Proof. Under condition (A), the operator $\tilde{H}$ is self-adjoint, hence its spectrum is real. Under condition (B), this spectrum coincides with the set $\{\gamma\}$ appearing in the nontrivial zeros $\rho = \frac{1}{2} + i\gamma$ of $\zeta(s)$. Therefore all such γ are real, which is exactly the Riemann Hypothesis. $\square$

6.4 Non-Circularity

It is essential to emphasize that neither condition (A) nor (B) assumes the Riemann Hypothesis. In particular:

- essential self-adjointness is a purely operator-theoretic property,
- spectral identification must be derived independently of any a priori assumptions on the location of zeros.

Thus the implication established above is strictly one-directional.

6.5 Status and Limitations

At present, neither condition (A) nor condition (B) has been established in full generality. The present work isolates these conditions explicitly and demonstrates that no additional hidden assumptions are required.

Consequently, the Riemann Hypothesis appears here not as a premise, but as a conditional corollary of a well-defined analytical program.

6.6 Interpretational Remark

Whether or not the program ultimately succeeds, the reformulation of the Riemann Hypothesis as a question of operator-theoretic uniqueness and spectral correspondence provides a concrete and falsifiable pathway for further investigation.

7 Numerical Experiments

This section reports numerical experiments designed to probe structural features of the operator H . These results are not presented as evidence for the Riemann Hypothesis, but as consistency checks of the operator framework.

7.1 Finite Truncations

The operator H is approximated by finite-dimensional truncations acting on $\ell^2(\{1, \dots, N\})$. Prime sums are truncated to $p \leq P(N)$.

Such truncations break exact symmetry and self-adjointness, but provide insight into qualitative behavior.

7.2 Spectral Observations

Numerical spectra of truncated operators exhibit:

- predominantly real eigenvalues,
- increasing spectral density with N ,
- sensitivity to parameter choices (α, β) .

These observations are consistent with, but do not imply, the existence of a self-adjoint infinite-dimensional limit.

7.3 Stability Tests

Eigenvalue distributions were tested against:

- changes in truncation size,
- variations in prime cutoffs,
- perturbations of diagonal terms.

Qualitative features persist under moderate perturbations.

7.4 Limitations

Finite truncations cannot resolve:

- essential self-adjointness,
- deficiency indices,
- true spectral type.

Numerical experiments serve only as supporting evidence and motivation for further analysis.

7.5 Reproducibility

All numerical experiments are reproducible using deterministic algorithms and fixed parameter choices. Implementation details are documented separately.

8 Failure Modes and Limitations

This section summarizes the principal ways in which the program developed in this work could fail. The purpose is to make all assumptions explicit and to delimit the scope of the conclusions.

8.1 Failure of Essential Self-Adjointness

If the deficiency equation admits nontrivial ℓ^2 solutions, the operator H would admit multiple self-adjoint extensions. In that case:

- no canonical spectrum exists,
- any arithmetic interpretation becomes extension-dependent,
- the Hilbert–Pólya framework collapses at the structural level.

8.2 Spectral Mismatch

Even if H is essentially self-adjoint, its spectrum may fail to correspond to arithmetic data. Possible issues include:

- presence of continuous spectrum,
- absence of discrete spectral points,
- lack of trace-class properties needed for explicit formulas.

8.3 Regularization Ambiguities

Any use of spectral traces or determinants requires regularization. Different regularization schemes may yield inequivalent results. Failure to control this dependence would undermine arithmetic conclusions.

8.4 Parameter Dependence

The parameters α and β play a structural role. Certain choices may destroy lower semiboundedness, invalidate closability, or introduce pathological spectral behavior.

8.5 Logical Separation from the Riemann Hypothesis

No part of the present framework assumes the Riemann Hypothesis. Conversely, the failure of this program would not constitute evidence against the truth of the Riemann Hypothesis itself.

8.6 Summary

The framework succeeds only if:

- essential self-adjointness is established,
- spectral quantities are well-defined and stable,
- arithmetic identification is derived, not imposed.

All other outcomes are explicitly acknowledged and documented.

9 Analysis of the Deficiency Equation

This section provides a preliminary analytical investigation of the deficiency equation introduced in Section ???. The analysis is partial and exploratory, intended to clarify structure rather than

to deliver a complete proof.

9.1 Restatement of the Equation

We consider square-summable solutions $\psi \in \ell^2(\mathbb{N})$ of

$$\sum_{p \in \mathcal{P}} \frac{1}{p} \left(\psi(pn) + \mathbf{1}_{p|n} \psi(n/p) \right) + (\alpha \Lambda(n) + \beta \log n) \psi(n) = \pm i \psi(n). \quad (20.5)$$

9.2 Growth Considerations

The diagonal term $\beta \log n$ grows unboundedly with n . Formally, this suggests that any solution $\psi(n)$ must decay sufficiently rapidly to compensate for logarithmic growth.

In particular, polynomially decaying solutions are excluded for $\beta \neq 0$.

9.3 Prime-Scale Coupling

The equation couples values of ψ across multiplicative scales. This destroys any simple local recurrence and prevents reduction to standard one-dimensional difference equations.

Such non-locality is expected to suppress square-summable solutions, though a rigorous proof remains open.

9.4 Heuristic Absence of ℓ^2 Solutions

Assuming generic oscillatory behavior induced by the imaginary term $\pm i$, and taking into account logarithmic confinement, it is heuristically unlikely that nontrivial ℓ^2 solutions exist.

This heuristic supports, but does not prove, the conjecture that the deficiency indices vanish.

9.5 Open Problem

Prove or disprove that equation (20.5) admits no nontrivial solutions in $\ell^2(\mathbb{N})$.

This problem represents the central analytical challenge of the program.

10 Conclusion and Research Program

This work has developed a fully explicit operator-theoretic framework motivated by the Hilbert–Pólya program. The construction is grounded in a rigorously defined symmetric operator acting on a concrete Hilbert space, with all structural assumptions stated openly and without reliance on unproven arithmetic hypotheses.

10.1 Summary of Results

The main achievements of this work may be summarized as follows:

- A concrete Hilbert space $\ell^2(\mathbb{N})$ and dense domain $\mathcal{D}$ were specified explicitly.
- A symmetric operator H built from prime-indexed shift operators and arithmetic diagonal terms was defined and shown to be well-posed.

- Essential self-adjointness was reduced to the absence of $\ell^2(\mathbb{N})$ solutions of a single, explicit deficiency equation.
- A general spectral framework was established independently of arithmetic conjectures.
- Structural interfaces with classical explicit formulas were identified, while carefully avoiding circular reasoning.
- Numerical experiments were presented strictly as supporting evidence, not as proofs.

10.2 Legitimacy of the Hilbert–Pólya Program

The framework presented here satisfies the minimal structural requirements of a legitimate Hilbert–Pólya program:

- the operator is defined independently of spectral assumptions,
- symmetry is rigorously established,
- the obstruction to self-adjointness is precisely localized,
- arithmetic consequences are formulated conditionally.

No claim of having proven the Riemann Hypothesis is made. Instead, the work isolates a concrete and falsifiable analytical problem whose resolution would determine the viability of the approach.

10.3 Future Directions

The natural continuation of this research consists of three parallel tracks:

1. **Analytical resolution of the deficiency equation.** Proving the absence of square-summable solutions would establish essential self-adjointness and complete the operator-theoretic foundation.
2. **Rigorous spectral-arithmetic identities.** Deriving trace or resolvent formulas linking spectral data of H to arithmetic expressions remains a central objective.
3. **Refined numerical and experimental studies.** Large-scale computations may guide conjectures, but must remain subordinate to analytical arguments.

10.4 Perspective

The approach presented here does not assume the truth of the Riemann Hypothesis, nor does it depend on numerical verification of its zeros. Instead, it seeks to reformulate the problem as a question of operator-theoretic uniqueness and spectral structure.

Whether this program ultimately succeeds or fails, the explicit nature of the constructions ensures that any obstruction will be visible, concrete, and mathematically meaningful.

10.5 Closing Remark

The value of the present framework lies not in its claims, but in the clarity with which it exposes both its strengths and its limitations. In this sense, it aims to contribute not a conclusion, but a well-defined path forward.

APPENDIX A

A.1 Technical Lemmas and Estimates

This appendix collects auxiliary results used implicitly throughout the main text. All statements are elementary and included for completeness.

A.1.1 Boundedness of Prime-Weighted Sums

Lemma A.1.1. The sequence $(1/p)_{p \in \mathcal{P}}$ belongs to $\ell^2(\mathcal{P})$, i.e.

$$\sum_{p \in \mathcal{P}} \frac{1}{p^2} < \infty.$$

Proof. This follows from comparison with the convergent series $\sum_{n=2}^{\infty} 1/n^2$ and the fact that $\mathcal{P} \subset \mathbb{N}$. $\square$

A.1.2 Norm Estimate for Shift Operators

Lemma A.1.2. For each prime p , the operators T_p^+ and T_p^- defined on $\mathcal{D}$ satisfy

$$\|T_p^{\pm} \psi\|_{\mathcal{H}} \leq \|\psi\|_{\mathcal{H}}, \quad \forall \psi \in \mathcal{D}.$$

Proof. For T_p^+ ,

$$\|T_p^+ \psi\|^2 = \sum_{n=1}^{\infty} |\psi(pn)|^2 \leq \sum_{m=1}^{\infty} |\psi(m)|^2.$$

The argument for T_p^- is analogous. $\square$

A.1.3 Diagonal Growth Control

Lemma A.1.3. For any $\varepsilon > 0$, there exists $C_{\varepsilon} > 0$ such that

$$\log n \leq C_{\varepsilon} n^{\varepsilon} \quad \forall n \geq 1.$$

Proof. This is a standard consequence of sub-polynomial growth of the logarithm. $\square$

A.1.4 Form-Boundedness of Diagonal Terms

Lemma A.1.4. The multiplication operator $M_{\log}$ defined by $(M_{\log} \psi)(n) = \log n \psi(n)$ is relatively form-bounded with respect to the identity operator.

Proof. By Lemma A.1.3, $\log n$ grows slower than any power of n . Since $\psi \in \ell^2(\mathbb{N})$, the claim follows by standard estimates. $\square$

A.1.5 Purpose of This Appendix

The results collected here justify the boundedness and convergence claims used in Sections 3–9. No originality is claimed for these lemmas.

APPENDIX B

B.1 Numerical Protocols and Reproducibility

This appendix documents the numerical procedures used in Section 7. The purpose is transparency and reproducibility, not numerical proof.

B.1.1 Finite-Dimensional Truncation

Numerical experiments approximate the operator H by truncation to the finite-dimensional space

$$\mathcal{H}_N := \ell^2(\{1, \dots, N\}).$$

Prime sums are truncated to primes $p \leq P(N)$, with $P(N)$ chosen such that $pn \leq N$ for non-negligible contributions.

B.1.2 Matrix Representation

The truncated operator H_N is represented as an $N \times N$ matrix with entries

$$(H_N)_{m,n} = \sum_{p \in \mathcal{P}_N} \frac{1}{p} (\delta_{m,pn} + \delta_{m,n/p}) + \delta_{m,n} (\alpha \Lambda(n) + \beta \log n),$$

where $\mathcal{P}_N$ denotes the truncated prime set.

B.1.3 Spectral Computation

Eigenvalues of H_N are computed using standard dense linear algebra routines. No attempt is made to extrapolate finite-dimensional spectra to the infinite-dimensional limit.

B.1.4 Stability Checks

Computations were repeated under:

- increasing truncation sizes N ,
- different prime cutoffs $P(N)$,
- small perturbations of parameters (α, β) .

Only qualitative, stable features are reported.

B.1.5 Limitations

Finite truncations:

- do not preserve exact self-adjointness,
- cannot detect deficiency indices,
- may introduce artificial boundary effects.

Accordingly, numerical results are interpreted only as consistency checks for the operator framework.

B.1.6 Reproducibility

All experiments are deterministic and reproducible. Exact implementation details, parameter values, and scripts are available upon request or in the accompanying repository.

Chapter 21

The SMRK Probe

Spectral Measurement Instrument for Arithmetic Hamiltonians

Enter Yourname

Abstract

The SMRK Probe is introduced as a formal spectral measurement instrument within the Quansistor Field Computing (QFC) framework, designed to explore the spectral behavior of arithmetic Hamiltonians constructed in the context of Quansistor Field Mathematics (QFM). The document defines the probe as an instrumental layer, positioned between abstract operator construction and higher-level execution models, without asserting proof-level claims or theoretical equivalence to unresolved conjectures.

The probe operates on finite-dimensional realizations of arithmetic operators, assembled according to explicit and reproducible construction protocols. Its purpose is to expose structural properties of operator spectra, including stability under truncation, sensitivity to controlled perturbations, and responses to prime-indexed dynamics. Observations produced by the probe are treated as descriptive observables rather than inferential results.

As a normative instrument specification, this chapter delineates the conceptual scope, architectural constraints, measurement modes, and output semantics of the SMRK Probe. It explicitly separates instrumentation from interpretation, and measurement from proof, ensuring that exploratory spectral analysis remains methodologically disciplined.

Within the QFC stack, the SMRK Probe functions as a pre-execution diagnostic layer. It informs the development and validation of operator-centric models in QFM, and provides empirical grounding prior to integration into QFP, QPU, QVM, or QWASM-level execution environments. The probe thereby establishes a stable and reproducible reference point for systematic spectral exploration of arithmetic operator fields.

Executive Summary

The *SMRK Probe* is a spectral measurement instrument designed for the systematic exploration of arithmetic Hamiltonians of SMRK type. Its purpose is to expose structural and stability properties of operator spectra arising from prime-indexed dynamics, without asserting completeness, convergence, or proof of any unresolved mathematical conjecture.

The probe operates within a finite-dimensional truncation of an underlying Hilbert space and assembles the SMRK Hamiltonian as a concrete operator object, constructed from prime-shift components and arithmetic weighting. Spectral data are extracted numerically under controlled perturbations, enabling the observation of eigenvalue stability, drift behavior, and rigidity patterns across varying cut-offs and parameter regimes.

Crucially, the SMRK Probe is not intended as a verification mechanism for the Riemann Hypothesis or any equivalent spectral claim. Instead, it serves as an instrumental interface between operator-theoretic constructions and measurable spectral behavior, analogous to experimental probes in physics that precede theoretical consolidation.

Version 1.0 of the SMRK Probe specifies a stable conceptual and architectural framework. It defines the role of the probe, its measurement modes, observable outputs, and known limitations, while deliberately avoiding interpretive overreach. Subsequent versions may refine numerical implementations or extend execution platforms, but the core instrumental semantics introduced here are considered stable.

Within the broader Quansistor Field Computing (QFC) framework, the SMRK Probe constitutes an initial measurement layer. It provides empirical grounding for operator-centric arithmetic models and informs future developments in spectral field execution, including potential deployment within QVM-native environments.

1 Motivation

Despite extensive numerical verification of the nontrivial zeros of the Riemann zeta function, classical computational approaches remain fundamentally limited in their ability to expose underlying structural mechanisms. High-precision evaluation of $\zeta(s)$ provides confidence in observed regularities, but offers little insight into why such regularities persist, nor how they arise from arithmetic dynamics at the operator level.

The operator-theoretic perspective, inspired by the Hilbert–Pólya program, suggests that spectral structure should emerge from a concrete Hamiltonian whose eigenvalues encode arithmetic information. However, the construction of such operators alone is insufficient. Without a systematic method for probing their spectral behavior, operator proposals remain largely formal and disconnected from measurable consequences.

The SMRK Probe is motivated by the need for an intermediate instrument between abstract operator construction and theoretical claims. Its role is to interrogate the spectral response of arithmetic Hamiltonians under controlled conditions, rather than to assert convergence, completeness, or equivalence with known spectra. In this sense, the probe occupies a position analogous to experimental apparatus in physics: it enables observation prior to interpretation.

A further motivation arises from the distinction between operator-centric and function-centric computation. Traditional numerical studies focus on analytic functions and their sampled values, whereas the SMRK framework operates directly on operators acting on Hilbert space. This shift demands new tools. Spectral properties such as rigidity, stability under perturbation, and structural symmetry cannot be reliably inferred from function evaluation alone; they must be measured at the operator level.

Finite-dimensional truncation introduces unavoidable artifacts, yet it also creates an opportunity. By varying cut-offs and perturbations in a controlled manner, the SMRK Probe allows researchers to distinguish persistent structural features from numerical or boundary-induced effects. Patterns that survive such variation provide evidence of intrinsic operator structure, independent of implementation details.

Finally, the motivation for formalizing the SMRK Probe as a versioned instrument lies in reproducibility and discipline. By specifying what the probe measures, how measurements are performed, and what claims are explicitly excluded, this whitepaper establishes a shared reference point. It enables comparative study, critical evaluation, and future extension without conflating exploratory results with mathematical proof.

2 Concept of a Spectral Probe

In the context of operator-theoretic arithmetic models, a *spectral probe* is defined as a controlled instrument for exposing qualitative and quantitative features of an operator spectrum without

asserting completeness, convergence, or theoretical equivalence to a target conjecture or model.

Unlike numerical simulation, which typically aims to approximate solutions of explicit equations, a spectral probe operates on operators themselves. Its primary function is not to compute values, but to measure structural response. This includes sensitivity to perturbations, stability of spectral ordering, and the persistence of symmetries under variation of construction parameters.

A spectral probe is also distinct from a proof-oriented framework. Proof seeks logical necessity under exact assumptions, whereas a probe is designed to explore behavior under controlled but imperfect conditions. The outputs of a probe are observables rather than implications. They inform intuition, guide refinement, and constrain future theoretical development, but they do not, by themselves, establish mathematical truth.

Formally, a spectral probe consists of three components:

- a well-defined operator assembly procedure,
- a finite execution or truncation regime,
- and a set of measurement protocols yielding reproducible observables.

Each component is specified explicitly to ensure that observed features can be attributed to operator structure rather than implementation artifacts.

The necessity of finite truncation is not treated as a defect but as an integral aspect of probing. By systematically varying truncation parameters and boundary conditions, a spectral probe distinguishes intrinsic operator behavior from finite-size effects. Structural features that persist across such variation are interpreted as candidates for genuine spectral invariants.

Within this framework, interpretation is deliberately restrained. The role of a spectral probe is not to confirm conjectures, but to delineate the space of plausible operator behaviors. It serves as an interface between abstract operator design and future formal analysis, providing empirical guidance while preserving logical separation from proof.

The SMRK Probe is an instance of this general concept, specialized to arithmetic Hamiltonians constructed from prime-indexed dynamics. However, the notion of a spectral probe introduced here is intentionally broader and may be applied to other operator-theoretic contexts within and beyond the QFC framework.

3 The SMRK Hamiltonian (Context Only)

The SMRK Hamiltonian provides the operator-theoretic object explored by the spectral probe described in this document. It is introduced here solely to the extent necessary to define the context of probing; a full mathematical treatment, including construction details and theoretical motivation, is presented in dedicated SMRK and QFM publications.

At a high level, the SMRK Hamiltonian is an arithmetic operator acting on a Hilbert space of square-summable sequences indexed by the natural numbers. Its structure is governed by prime-indexed shift operations that implement multiplicative dynamics within the state space, together with arithmetic weighting components that modulate spectral behavior.

Prime-shift operators map basis states according to multiplication by prime numbers, subject to domain constraints imposed by finite truncation. These operators admit natural adjoint counterparts corresponding to inverse multiplicative action when divisibility conditions permit. The interaction between forward and adjoint shifts forms the dynamical backbone of the SMRK construction.

The Hamiltonian is assembled as a structured combination of these shift operators and diagonal

arithmetic terms. The resulting operator is generally unbounded in the infinite-dimensional setting and is therefore examined through controlled finite-dimensional realizations. Issues of self-adjointness, domain specification, and closure are acknowledged but not resolved within the scope of this probe-focused document.

From the perspective of the spectral probe, the SMRK Hamiltonian is treated as a black-box operator with a well-defined assembly protocol. The probe does not modify the Hamiltonian’s conceptual definition; rather, it subjects specific finite realizations to measurement procedures designed to expose spectral structure, stability, and sensitivity.

By limiting discussion of the SMRK Hamiltonian to contextual essentials, this section preserves a clear separation between operator design and instrumental analysis. The spectral observations reported by the SMRK Probe are therefore to be interpreted as properties of instantiated operator realizations, not as claims about the fully idealized infinite-dimensional object.

4 Architecture of the SMRK Probe

The SMRK Probe is structured as a modular spectral instrument whose architecture separates operator construction, execution constraints, and measurement procedures. This separation ensures that observed spectral features can be attributed to operator structure rather than to implementation-specific coupling between components.

The architecture is defined independently of any particular programming language or numerical backend. It specifies conceptual stages of probing that must be preserved across implementations to maintain semantic consistency.

4.1 Hilbert Space Truncation

The probe operates on a finite-dimensional truncation of the underlying Hilbert space of square-summable sequences indexed by the natural numbers. A truncation parameter N defines the maximum index retained in the basis, yielding a computational state space of dimension N .

Truncation is treated as an explicit and variable parameter of the probe. Rather than fixing N a priori, the probe is designed to examine spectral behavior across a range of truncation scales. This enables differentiation between boundary-induced artifacts and persistent structural features.

Boundary handling at the truncation limit is specified deterministically. Operator actions that would map states beyond the truncated domain are either suppressed or projected according to a consistent rule, ensuring reproducible spectral outcomes.

4.2 Operator Assembly

Within a given truncation regime, the SMRK Hamiltonian is assembled from its constituent components following a fixed construction protocol. Prime-indexed shift operators and their adjoints are instantiated as sparse linear operators acting on the truncated basis.

Diagonal arithmetic terms are incorporated to modulate spectral structure and ensure proper scaling behavior. All components are combined into a single operator object representing a finite realization of the SMRK Hamiltonian.

The assembly procedure is deterministic and fully specified. Given identical parameters and truncation rules, independent implementations of the probe must produce equivalent operator representations up to numerical precision.

4.3 Spectral Extraction

Spectral extraction constitutes the measurement phase of the probe. The assembled operator is subjected to numerical eigenspectrum analysis appropriate to its size and structure. The probe does not prescribe a specific eigensolver but requires that the chosen method provide consistent ordering and reproducibility.

Extracted eigenvalues are treated as primary observables. Eigenvectors may be retained for auxiliary analysis but are not required for core probe operation. Spectral data are recorded together with truncation parameters and assembly metadata to enable comparative study.

The probe explicitly avoids post-processing steps that enforce expected spectral patterns. All observed regularities must arise from the operator realization itself rather than from normalization or filtering procedures applied after measurement.

5 Measurement Modes

The SMRK Probe supports multiple measurement modes, each designed to expose a distinct aspect of spectral behavior of the instantiated SMRK Hamiltonian. These modes do not constitute separate implementations, but rather distinct measurement protocols applied to the same underlying operator architecture.

All measurement modes share a common principle: observed features are recorded as descriptive spectral data without interpretive enforcement. Comparative analysis across modes enables identification of structural regularities that are robust under variation.

5.1 Static Spectral Measurement

In static mode, the probe performs eigenspectrum extraction for a fixed truncation parameter and fixed operator assembly. The resulting ordered set of eigenvalues provides a baseline spectral profile for the given realization.

Static measurement is used to identify coarse spectral features such as spacing distributions, clustering tendencies, and gross symmetry. No claims of spectral convergence are made on the basis of static measurement alone.

5.2 Truncation Scaling Mode

In truncation scaling mode, the probe evaluates the spectrum across a sequence of increasing truncation parameters. Spectral features are tracked as functions of the truncation scale, allowing differentiation between boundary artifacts and persistent structural behavior.

Eigenvalues that exhibit stability under truncation scaling are identified as candidates for intrinsic spectral features of the operator, whereas unstable or emergent modes are interpreted as truncation-dependent effects.

5.3 Perturbative Sensitivity Mode

Perturbative mode introduces controlled variations to operator parameters or assembly weights while preserving overall structural form. The resulting spectral drift is measured relative to the unperturbed baseline.

This mode probes spectral rigidity and sensitivity. Small perturbations that produce disproportionate spectral changes indicate fragile structure, whereas gradual and coherent drift suggests

robust operator organization.

5.4 Prime-Shift Response Mode

In prime-shift response mode, the probe selectively activates or modulates prime-indexed components of the Hamiltonian. The resulting changes in spectral behavior are recorded and compared across prime subsets.

This mode isolates the contribution of multiplicative arithmetic dynamics to the overall spectrum. Correlated responses across different prime activations provide evidence of structured arithmetic coupling within the operator.

5.5 Comparative Mode

Comparative mode aggregates spectral data obtained from multiple probe runs with varying parameters or construction variants. Rather than focusing on individual eigenvalues, this mode emphasizes relational properties such as ordering consistency, symmetry persistence, and cross-run correlations.

Comparative analysis serves as a synthesis layer, enabling structural inference without relying on absolute spectral identification.

6 Observables and Outputs

The outputs of the SMRK Probe are defined as *observables*: descriptive quantities derived from spectral measurement that characterize structural properties of instantiated operators. Observables are explicitly non-inferential; they do not imply convergence, equivalence, or theoretical validation.

All observables are recorded together with metadata specifying truncation parameters, assembly configuration, and measurement mode. This ensures that comparisons across probe runs remain meaningful and reproducible.

6.1 Eigenvalue Distributions

The primary observable produced by the probe is the ordered set of eigenvalues of the instantiated SMRK Hamiltonian under a given measurement mode. These distributions provide coarse information about spectral density, spacing behavior, and large-scale structure.

Statistical summaries of eigenvalue distributions may be computed for descriptive purposes, but no universal distributional assumptions are imposed.

6.2 Stability Indicators

Stability indicators quantify the sensitivity of spectral features to changes in truncation scale or perturbation parameters. Eigenvalues that persist within bounded variation across multiple probe runs are flagged as stable.

Stability is treated as a relative property, evaluated across controlled variation rather than as an absolute criterion.

6.3 Spectral Drift Measures

Spectral drift observables capture the displacement of eigenvalues under perturbative or structural modification of the operator assembly. Drift is recorded as a function of perturbation magnitude, enabling qualitative classification of spectral response.

Coherent drift across eigenvalue subsets suggests organized operator structure, whereas irregular drift patterns indicate fragile or incidental spectral features.

6.4 Rigidity and Correlation Metrics

Rigidity metrics describe the degree to which spectral ordering and relative spacing are preserved across measurement modes. Correlation analysis between spectra obtained under different probe configurations provides evidence of underlying structural coupling.

These metrics are comparative in nature and are not interpreted as invariant quantities.

6.5 Anomaly Detection

The probe records spectral anomalies such as isolated eigenvalues, unexpected degeneracies, or abrupt structural transitions. Anomalies are documented without interpretation and serve as prompts for further investigation rather than as evidence of theoretical significance.

7 Limitations

The SMRK Probe is intentionally limited in scope. These limitations are not treated as deficiencies, but as intrinsic constraints of an instrumental framework operating at the boundary between abstract operator theory and numerical realization.

7.1 Finite-Dimensional Truncation

All measurements performed by the SMRK Probe rely on finite-dimensional truncations of an underlying infinite-dimensional Hilbert space. As a result, spectral features observed at any fixed truncation level may reflect boundary effects rather than intrinsic operator properties.

While truncation scaling modes mitigate this limitation by enabling comparative analysis across cut-off parameters, no finite probe can fully capture the behavior of the idealized operator.

7.2 Numerical Approximation

Spectral extraction is necessarily subject to numerical approximation. Floating point precision, solver tolerances, and implementation-specific numerical behavior may influence measured spectra.

The probe does not attempt to eliminate numerical effects entirely, but instead treats them as part of the measurement environment. Observed features must be robust under reasonable numerical variation to be considered structurally meaningful.

7.3 Basis and Representation Dependence

Although the SMRK Hamiltonian is defined abstractly, its realization within the probe depends on a chosen basis and representation scheme. Certain spectral features may therefore be sensitive to representation choices.

The probe addresses this limitation through consistency and reproducibility requirements rather than basis invariance. Interpretation is restricted to features that persist under controlled representational variation.

7.4 Absence of Proof-Level Claims

The SMRK Probe does not establish self-adjointness, spectral completeness, or equivalence with the nontrivial zeros of the Riemann zeta function. No claim of verification or falsification of the Riemann Hypothesis is made within this document.

Results obtained through the probe are descriptive and exploratory. They may inform conjecture refinement or motivate formal analysis, but they do not constitute mathematical proof.

7.5 Interpretive Restraint

The probe deliberately avoids interpretive enforcement. Spectral patterns, correlations, or apparent regularities are recorded without assigning theoretical significance beyond their descriptive role.

This restraint is essential to maintaining a clear boundary between instrumentation and theory, and to preventing overextension of exploratory results.

8 Role within QFC / QFM

Within the Quansistor Field Computing (QFC) framework, the SMRK Probe occupies the role of an instrumental measurement layer. It does not define computational semantics, execution models, or operator governance, but instead provides a means of empirically examining operator structures prior to their integration into higher-level systems.

At the level of Quansistor Field Mathematics (QFM), the probe supports an operator-first methodology. QFM emphasizes explicit operator construction, adjointness relations, and spectral structure as primary mathematical objects. The SMRK Probe complements this perspective by enabling controlled exploration of spectral behavior arising from such constructions, without conflating observation with proof.

The probe serves as a bridge between abstract operator design and formal theoretical development. Spectral features revealed through probing may inform refinement of operator definitions, guide analytical investigation, or motivate reformulation of underlying assumptions. Conversely, the absence of expected structure provides equally valuable feedback, constraining speculative models.

In the broader QFC stack, the SMRK Probe precedes execution-oriented components such as field schedulers, operator governance layers, and QVM-native execution environments. Its function is diagnostic rather than operational: it characterizes operator behavior without embedding it into a computational pipeline.

By formalizing the SMRK Probe as a versioned and conceptually stable instrument, this document establishes a reproducible reference point within the QFC corpus. Future probes may extend this framework to other operator classes or execution contexts, but the role defined here remains that of measurement, not adjudication.

9 Conclusion

This whitepaper has specified the SMRK Probe as a spectral measurement instrument for arithmetic Hamiltonians, formalizing its conceptual basis, architectural structure, measurement modes, observables, and limitations. By design, the probe operates at the interface between abstract operator construction and empirical spectral observation.

The SMRK Probe does not assert mathematical proof, convergence, or equivalence to any established conjecture. Its contribution lies instead in providing a disciplined and reproducible framework for probing operator behavior under controlled conditions. In doing so, it enables the identification of structural regularities and sensitivities that may inform future theoretical development.

By clearly separating instrumentation from interpretation, this document establishes a stable reference point for exploratory spectral analysis within the QFC and QFM ecosystem. The versioned definition presented here ensures that subsequent extensions, refinements, or alternative implementations can be evaluated against a common conceptual baseline.

The SMRK Probe therefore serves not as a conclusion to an inquiry, but as a starting instrument for systematic exploration of operator-theoretic arithmetic models.

VOL. No. IV

The Spectral Operator Registry

Canonical Filters, Penalties, Liouvillians, Tokens,
Bridges, and Koopman Export

22	Operator Suite Conventions & Registry	
	Canonical Operator Artifacts for QFC	494
23	EY23: Canonical Spectral Filter $F_\star(H)$	
	Canonical Operator Artifacts for QFC	530
24	EY24: Validity Penalty Operator $V = C^\dagger C$	
	Canonical Operator Artifacts for QFC	562
25	EY25: Audit-Native Liouvillian $\mathcal{L}_{\text{audit}}$	
	Canonical Operator Artifacts for QFC	592
26	EY26: Floquet Program Token U_F	
	Canonical Operator Artifacts for QFC	625
27	EY27: Commutant Generator G_{comm}	
	Canonical Operator Artifacts for QFC	657
28	EY28: Regime Intertwiner $T_{1 \rightarrow 2}$	
	Canonical Operator Artifacts for QFC	688
29	EY29: Coarse-Grain Reducer R	
	Canonical Operator Artifacts for QFC	720
30	EY30: Koopman Export Operator $\mathcal{K}$	
	Canonical Operator Artifacts for QFC	752

Operator Suite Conventions & Registry

Canonical Operator Artifacts for QFC

Enter Yourname

Abstract

This whitepaper defines the normative backbone of Vol. No. IV for Enter Yourname: a suite of eight canonical operator artifacts intended as reusable building blocks across the QFC stack. The objective is methodological. Instead of introducing a new operator object, this document fixes the conventions that make subsequent artifacts comparable, auditable, and interoperable across implementations and QVM execution environments.

The core contribution is a shared specification grammar: common terminology for operator objects, a canonical notion of specification identity (SpecID) that binds a definition to its admissible classes, objectives, constraints, tie-break rules, functional calculus, and numerical determinism model, and a minimal set of invariants that each artifact **MUST** satisfy. This establishes a standard interface between QFM-level operator constructions and QVM-level validity enforcement, where reproducibility and contract semantics are treated as first-class requirements.

This whitepaper also standardizes certificate semantics. Each artifact in Vol. No. IV is designed to be re-derived or verified from public inputs without trusting the producer, so this document fixes what a certificate is allowed to contain, what a verifier **MUST** check, and how failure modes are represented deterministically. In this framing, Validity becomes an operator-level contract and auditability becomes a property of the specification, not of a particular implementation.

Finally, this whitepaper provides a registry view of the eight forthcoming operator artifacts and clarifies their intended roles: canonical spectral probing, proof-carrying penalty operators, audit-native evolution, compact program tokens, symmetry kernels, certified intertwiners across regimes, coarse-graining with guaranteed invariants, and Koopman-style export of nonlinear dynamics into spectral workflows. It explicitly delineates non-claims: Vol. No. IV does not assert physical implementation, algorithmic advantage, or resolution of open conjectures by definitional fiat; it defines the operator-law layer that makes later results comparable and falsifiable.

1 Scope, Intent, and What We Are Building

1.1 Why EY22 Exists

QFC treats operators as primary objects. In this viewpoint, a Hamiltonian H is not merely a generator of time evolution; it is a definitional core from which meaning is extracted through

spectral action, constraints, and compositional operator laws. Yet in most workflows, the step from “operator” to “interpretation” is performed by informal choices:

- a spectral probe $f(H)$ is selected by taste,
- a validity rule is encoded procedurally rather than as an operator,
- distributed evaluation introduces hidden nondeterminism,
- comparisons across implementations degrade into incompatible conventions.

EY22 exists to eliminate this failure mode. It defines the shared normative conventions for Vol. No. IV: a suite of canonical operator artifacts designed to be *re-derivable*, *auditable*, and *interoperable* across QFC contexts.

1.2 What “Canonical” Means Here

In this volume, *canonical* does not mean “unique by theorem” and it does not mean “physically privileged.” It means:

- the object is determined by a published specification, not by an informal choice,
- independent parties can reproduce the same artifact (or the same canonical equivalence class),
- ambiguity is represented explicitly via tie-break rules or equivalence conventions.

Canonicity is therefore a **property of the specification**, not a property of the author.

1.3 The Deliverable of Vol. No. IV

Vol. No. IV is delivered as eight operator artifacts, each defined in a separate whitepaper. Each artifact is required to ship with:

1. a **normative definition** (what the operator is),
2. a **SpecID** (a tuple binding the definition to admissible classes, objectives, constraints, calculus, and numeric model),
3. a list of **required invariants** (properties that MUST hold),
4. a **certificate format** and **verification semantics**,
5. a **deterministic protocol** for evaluation and verification (including truncation and aggregation rules).

EY22 provides the shared contract for these elements and acts as the registry of the suite.

1.4 The Eight Operator Artifacts

The operator suite consists of the following artifacts:

1. **EY23 — Canonical Spectral Filter** $F_\star(H)$: an optimization-defined spectral filter derived from a Hamiltonian H , not chosen ad hoc.
2. **EY24 — Validity Penalty Operator** $V = C^\dagger C$: a constructive validity object for proof-carrying trajectories and enforceable constraints.

3. **EY25 — Field Liouvillian $\mathcal{L}_{\text{audit}}$** : an audit-native dissipative evolution operator (dissipation only into allowed subspaces).
4. **EY26 — Floquet Program Operator U_F** : a compact program token defined by periodic evolution with an auditable spectrum.
5. **EY27 — Commutant Generator Set $\{G_i\}$** : the symmetry kernel of a field: generators commuting with a declared operator core.
6. **EY28 — Intertwiner $T_{1 \rightarrow 2}$** : a certified morphism between spectral regimes or layers, preserving declared invariants.
7. **EY29 — Coarse-Grain Operator R** : a bucketing / aggregation operator with guaranteed invariants under truncation and reduction.
8. **EY30 — Koopman Field Operator $\mathcal{K}$** : a bridge from nonlinear dynamics to spectral objects via a linear operator on observables.

1.5 Roles Within the QFC Stack

The suite is designed to cover the minimal operator-law layer needed for QFC-style work:

- **Spectral probing and standardization** (EY23, EY29): extraction and bucketing of spectral meaning is made explicit and comparable.
- **Validity as an operator object** (EY24, EY25): constraints become constructive penalties and audit-native evolution, not procedural checks.
- **Programmability as evolution** (EY26): programs are represented as auditable tokens defined by controlled time dependence.
- **Symmetry and regime mapping** (EY27, EY28): invariants and bridges between layers are explicit certified objects.
- **Export of nonlinear dynamics to spectrum** (EY30): nonlinearity is pulled into linear spectral workflows via operator action on observables.

1.6 Non-Goals and Non-Claims

Vol. No. IV is **methodological**. It MUST NOT be interpreted as:

- a proof of the Riemann Hypothesis or any open conjecture by definitional fiat,
- a claim of quantum advantage or complexity separation,
- a hardware prescription for fabrication or implementation of a physical device,
- a replacement of established quantum field theory (“field” here is the QFC operator-ecosystem sense).

1.7 How to Read This Volume

The recommended reading order minimizes forward references:

EY22 $\rightarrow$ EY23 $\rightarrow$ EY24 $\rightarrow$ EY25 $\rightarrow$ EY29 $\rightarrow$ EY26 $\rightarrow$ EY27 $\rightarrow$ EY28 $\rightarrow$ EY30.

EY22 fixes conventions; EY23 establishes canonicalization; EY24–EY25 define validity mechanisms; EY29 formalizes bucketing; EY26 introduces program tokens; EY27–EY28 handle invariants and bridges; EY30 exports nonlinear dynamics.

1.8 What Comes Next in EY22

The remaining chapters of EY22 will specify:

- the normative language and threat model shared across the suite,
- the SpecID tuple and versioning rules,
- the determinism and numeric model (precision, rounding, tolerance, truncation),
- certificate formats and verification semantics,
- interoperability and composition rules between artifacts.

2 Normative Language, Core Objects, and Threat Model

2.1 Normative Keywords

This volume uses a strict normative vocabulary. The following keywords are binding:

- **MUST** / **MUST NOT**: absolute requirements.
- **SHOULD** / **SHOULD NOT**: strong recommendations; deviations require explicit justification in the SpecID.
- **MAY**: optional behavior that must still be deterministic if exercised.

If a statement can affect reproducibility, auditability, or comparability across implementations, it **MUST** be expressed using these keywords. Vague language (e.g. “typically”, “generally”) is not sufficient for artifact definitions.

2.2 Core Object Model

Vol. No. IV treats every artifact as a *specifiable operator object*. The minimal shared model is:

Spaces

- $\mathcal{H}$: a complex Hilbert space (often separable).
- $\mathcal{B}(\mathcal{H})$: bounded operators on $\mathcal{H}$.
- Obs: a declared observable space when Koopman-style constructions are used (EY30).

Operator Core

- H : a (typically self-adjoint) operator on $\mathcal{H}$, serving as a declared “core” operator.
- $\sigma(H)$: spectrum of H .
- $E_H(\cdot)$: spectral projection-valued measure when functional calculus is invoked.

Functional Calculus (Declared)

If an artifact uses $f(H)$, it **MUST** declare *which* calculus is used (e.g. Borel functional calculus for bounded f , holomorphic calculus, rational approximation protocol, etc.). The calculus choice is part of the SpecID.

Artifacts

Each artifact is one of the following types:

- **Operator filter:** $F(H) = f(H)$ (EY23).
- **Penalty operator:** $V = C^\dagger C$ (EY24).
- **Superoperator / generator:** $\mathcal{L}$ acting on states or observables (EY25, EY30).
- **Unitary token:** U_F (EY26).
- **Symmetry kernel:** generators $\{G_i\}$ with $[G_i, H] = 0$ or analogous relations (EY27).
- **Morphism / bridge:** $T_{1 \rightarrow 2}$ satisfying intertwining relations (EY28).
- **Coarse-graining:** R mapping detailed objects to reduced/bucketed objects with invariants (EY29).

2.3 The SpecID Principle (Preview)

In Vol. No. IV, an artifact is never referenced only by its symbol. It **MUST** be referenced by a *SpecID*, a public tuple that binds meaning to the symbol.

Informally:

$$\text{Symbol} + \text{SpecID} = \text{Unambiguous object.}$$

The full SpecID schema is defined in Chapter 3. Here we only state the rule:

Rule (No-floating symbols). Any claim about $F_\star(H)$, V , $\mathcal{L}_{\text{audit}}$, U_F , $\{G_i\}$, $T_{1 \rightarrow 2}$, R , or $\mathcal{K}$ **MUST** be interpreted as a claim about that symbol *under a declared SpecID*.

2.4 Determinism Is a First-Class Constraint

QFC-style operator work becomes fragile when determinism is left implicit. Therefore:

- Any evaluation procedure **MUST** be deterministic under a declared numerical model.
- Any approximation **MUST** specify precision, rounding, and tolerance conventions.
- Any tie-breaking rule **MUST** be explicit and deterministic.
- Any optional randomness **MUST** be prohibited unless deterministically seeded from declared public inputs.

The full determinism model is fixed in Chapter 4. Here we state the motivation:

Motivation. In Vol. No. IV, auditability is not a social promise. It is a property of the artifact definition. If two parties cannot reproduce the same artifact from the same public inputs, the artifact is not canonical.

2.5 Threat Model (What We Protect Against)

Each artifact is intended to survive adversarial and non-adversarial failure modes that commonly appear in distributed, numeric, and multi-implementation contexts.

Threat Class T1: Parameter Smuggling

A producer can hide degrees of freedom in “minor” choices (filters, smoothing, truncation) and tune them to fit expected outcomes.

Mitigation. Artifacts in this suite MUST move such degrees of freedom into SpecID-defined admissible sets, objectives, constraints, and tie-break rules. If it is tunable, it must be declared.

Threat Class T2: Nondeterministic Replay

Two implementations compute “the same” artifact but differ due to numeric instability, ordering dependence, or implicit randomness.

Mitigation. Artifacts MUST define a determinism model (precision, rounding, tolerance, aggregation order) and provide certificates that can be verified deterministically.

Threat Class T3: Truncation Drift

Finite-dimensional truncations H_N are used in practice. Without a declared truncation model, results are incomparable and can be manipulated by choosing N .

Mitigation. Any artifact that depends on truncation MUST declare:

- how truncations are formed,
- which invariants must be preserved across N ,
- and how cross- N comparisons are performed (or forbidden).

Threat Class T4: Certificate Forgery or Ambiguity

A producer claims an artifact is canonical but provides no verifiable witness, or a witness that is ambiguous.

Mitigation. Each artifact MUST define a certificate format and a verification procedure that either accepts or rejects. Certificates MUST commit to all SpecID components relevant to meaning.

Threat Class T5: Definition Drift Across Versions

A symbol is reused across revisions; the meaning changes but the name does not.

Mitigation. SpecID MUST include version identifiers (or hash commitments) to all normative components. A change in meaning MUST imply a change in SpecID.

2.6 Non-Claims Boundary (Methodological Only)

The threat model is not a license to claim more than is defined.

- Canonical definitions do not imply physical realizability.

- Auditability does not imply scientific truth; it implies reproducible evaluation.
- Stability guarantees are only meaningful relative to declared perturbation classes.

2.7 Chapter Summary

This chapter fixed the normative language and the minimal shared object model for Vol. No. IV. It also defined the threat classes that the suite is designed to address: parameter smuggling, nondeterministic replay, truncation drift, certificate ambiguity, and definition drift.

Chapter 3 now formalizes the SpecID tuple and versioning rules that bind all subsequent whitepapers to a common, auditable specification grammar.

3 Specification Identity: SpecID Tuples and Versioning

3.1 Why SpecID Exists

In Vol. No. IV, a symbol is not a definition. Symbols drift, implementations diverge, and “same object” can silently become “similar object”. This volume therefore adopts a hard rule:

No artifact is valid unless it can be referenced unambiguously.

The mechanism is **SpecID**: a public tuple that binds an artifact symbol to the full set of normative choices that determine its meaning.

3.2 Normative Rule: No Floating Artifacts

Rule (Binding). Any occurrence of an artifact symbol in Vol. No. IV MUST be interpreted as:

$$\text{Artifact Symbol} + \text{SpecID} + \text{Version} \Rightarrow \text{Unambiguous object.}$$

If SpecID is missing, the reference MUST be treated as incomplete.

3.3 SpecID: The Canonical Tuple

Definition 3.1 (SpecID Tuple). A **SpecID** is a public tuple that uniquely identifies the normative definition of an artifact.

All artifacts in Vol. No. IV MUST define a SpecID of the following canonical shape:

$\text{SpecID} := (\text{ArtifactType}, \text{Core}, \text{Calc}, \mathcal{A}, \mathcal{J}, \mathcal{C}, \text{TieBreak}, \text{NumericModel}, \text{Cert}, \text{Interop}, \text{Version}).$

Each component is normative. If a component is not applicable for a given artifact type, it MUST still be present and MUST take a declared null value (e.g. $\emptyset$ with explicit semantics).

3.4 SpecID Components (Normative Meanings)

This section fixes the meaning of each SpecID component.

ArtifactType

A required enumerated tag identifying the artifact class:

$$\text{ArtifactType} \in \{\text{CSF}, \text{VPO}, \text{LIOUV}, \text{FLOQ}, \text{COMM}, \text{INTW}, \text{CGR}, \text{KOOP}\}.$$

Where:

- CSF: Canonical Spectral Filter (EY23)
- VPO: Validity Penalty Operator (EY24)
- LIOUV: Field Liouvillian (EY25)
- FLOQ: Floquet Program Operator (EY26)
- COMM: Commutant Generator Set (EY27)
- INTW: Intertwiner (EY28)
- CGR: Coarse-Grain Operator (EY29)
- KOOP: Koopman Field Operator (EY30)

Core

The declared *core object(s)* the artifact is built from. Typical examples:

- Core = (H) for filter and commutant constructions,
- Core = (H, C) for validity penalty operators,
- Core = $(H(t), T)$ for Floquet objects,
- Core = (F, Obs) for Koopman objects.

Rule. If H is used, its self-adjointness / domain assumptions MUST be declared in the artifact definition.

Calc

The declared evaluation calculus. Examples:

- Borel functional calculus (bounded f) for $f(H)$,
- holomorphic/rational calculus for resolvent-like objects,
- explicit truncation protocol defining H_N and $f(H_N)$,
- superoperator calculus (action on density operators or observables).

Rule. Any two implementations claiming the same SpecID MUST use the same Calc semantics.

$\mathcal{A}$ (Admissible Family)

The admissible family of candidates over which the artifact is selected or constructed. For different artifacts:

- EY23: admissible filter functions f ,
- EY24: admissible constraint operators C ,
- EY25: admissible dissipators / Lindblad terms (or QFC-defined equivalents),
- EY27: admissible generator search space,
- EY29: admissible coarse-graining maps.

Rule. $\mathcal{A}$ MUST have a canonical representation form (see Appendix ?? in EY22, once filled).

$\mathcal{J}$ (Objective / Selection Functional)

The objective that canonically selects among admissible candidates.

Rule. If $\mathcal{J} \neq \emptyset$, it MUST be deterministically evaluable under the NumericModel.

$\mathcal{C}$ (Constraints)

The constraint predicates required for feasibility (boundedness, positivity, stability, implementability, etc.).

Rule. Constraints MUST be explicit. “Reasonable constraints” MUST NOT be used.

TieBreak

A deterministic tie-breaking rule used when multiple candidates satisfy the same optimal score or satisfy a canonical equivalence. Typical tie-breaks:

- lexicographic minimal canonical representation,
- minimal parameter norm,
- maximal smoothness subject to constraints,
- minimal operator norm among optimizers.

Rule. If tie-breaking is required, it MUST yield a unique canonical output given the same public inputs.

NumericModel

The numeric determinism model: precision, rounding, tolerance rules, truncation scales, and aggregation order.

Rule. NumericModel MUST be explicit whenever the artifact is evaluated approximately (which is the default in practice). NumericModel is specified normatively in Chapter 4.

Cert

Certificate semantics: what must be published so that a verifier can accept the artifact.

Rule. Cert MUST include a verification procedure that deterministically accepts or rejects.

Interop

Interoperability constraints: how the artifact composes with others (e.g. $R \circ F_*(H)$ allowed, or $T_{1 \rightarrow 2}$ must preserve a declared invariant set).

Rule. If an artifact claims compatibility with another, the compatibility condition MUST be stated in Interop and MUST be testable.

Version

A version identifier binding the entire SpecID to a stable reference.

Rule. Any change in meaning MUST change Version. Version MAY be a semantic version string, but SHOULD be a cryptographic commitment when published in adversarial settings.

3.5 Canonical Naming and Referencing

To prevent ambiguity, this volume uses the following reference form:

Reference Form.

$\langle \text{ArtifactSymbol} \rangle @ \langle \text{SpecID-Short} \rangle :: \langle \text{Version} \rangle$

SpecID-Short. For human readability, a short form MAY be used in text, provided it expands to the full SpecID in the certificate. A recommended short form is:

$\text{SpecID_Short} := (\text{ArtifactType}, \text{CalcID}, \text{AID}, \text{JID}, \text{CID}, \text{NID}, \text{VID})$

where each *ID is a stable label that resolves to a full canonical description.

3.6 Spec Drift and the “Same Symbol” Problem

Rule (Symbol reuse). A symbol MAY be reused across versions, but MUST NOT be assumed to mean the same object without matching SpecID and Version. Therefore, any published artifact MUST include SpecID and Version in:

- captions and figure annotations that rely on the artifact,
- experiment metadata,
- certificate headers,
- and machine-readable index entries.

3.7 Minimal SpecID for Each Artifact Type

This section provides minimal requirements by type. These are *lower bounds*.

- **CSF (EY23)** MUST specify: H , Calc, $\mathcal{A}$, $\mathcal{J}$, $\mathcal{C}$, TieBreak, NumericModel, Cert, Version.

- **VPO (EY24)** MUST specify: C admissible family, $V = C^\dagger C$ construction, constraints on C , certificate of feasibility and canonical selection (if selection exists), NumericModel.
- **LIUV (EY25)** MUST specify: the state space, generator form, allowed dissipators, contract constraints, certificate and determinism.
- **FLOQ (EY26)** MUST specify: time-dependence model $H(t)$, period T , time-ordering convention, discretization protocol, certificate.
- **COMM (EY27)** MUST specify: commutation target(s), generator admissible class, discovery/selection rule, certificate.
- **INTW (EY28)** MUST specify: source/target cores, intertwining relation, invariant preservation, certificate.
- **CGR (EY29)** MUST specify: reduction target, invariants preserved, truncation interface, certificate.
- **KOOP (EY30)** MUST specify: dynamical system F , observable space, operator action, approximation protocol, certificate.

3.8 Chapter Summary

This chapter introduced SpecID as the unambiguous identifier of meaning for every artifact in Vol. No. IV. It fixed a canonical tuple form and defined the normative semantics of each component.

Chapter 4 now specifies the NumericModel: determinism rules for precision, rounding, tolerance, truncation, and aggregation. This is where “canonical” becomes executable.

4 Determinism Model: Precision, Rounding, Tolerance, Truncation

4.1 Why a NumericModel Is Mandatory

Operator artifacts in Vol. No. IV are defined abstractly, but they are evaluated in practice by truncation, approximation, and distributed aggregation. Without an explicit numeric model, “the same artifact” becomes implementation-dependent. Therefore:

In Vol. No. IV, determinism is part of the definition.

Any artifact that can be evaluated approximately MUST bind itself to an explicit NumericModel as part of its SpecID.

4.2 Normative Rule: Determinism Over Convenience

Rule. If a procedure has multiple implementation-equivalent choices (order of summation, floating-point vs fixed-point, parallel reduction order, random sampling), then Vol. No. IV requires:

- either a **single canonical choice** is specified,
- or the procedure MUST be reformulated to produce a deterministic result (e.g. sorting + deterministic reduction),

- or the result **MUST** be rejected as non-canonical.

4.3 NumericModel Schema

Definition 4.1 (NumericModel Tuple). A **NumericModel** is a public tuple:

$\text{NumericModel} := (\text{NumberSystem}, \text{Precision}, \text{Rounding}, \text{Tolerance}, \text{Aggregation}, \text{Truncation}, \text{FailureModes}).$

All components are normative. If a component is irrelevant for a specific artifact, it **MUST** be present and **MUST** be set to a declared null value with explicit semantics.

4.4 Number System and Precision

NumberSystem

The number system determines the determinism envelope.

Allowed (recommended).

- **Fixed-point integers** with declared scaling.
- **Rational arithmetic** with canonical normalization (exact but potentially expensive).
- **IEEE floating point** only if the remaining NumericModel components remove nondeterminism (see below).

Rule. If IEEE floating point is used, the NumericModel **MUST** additionally specify:

- the exact floating format (binary32, binary64, etc.),
- whether fused multiply-add is allowed,
- whether denormals are flushed to zero,
- and whether math-library functions are replaced by canonical approximations.

Precision

Precision **MUST** specify either:

- **exactness** (rational / integer scaling), or
- **bit precision** (e.g. 53-bit mantissa, or an explicit fixed-point scale), or
- **decimal precision** (only if conversion is canonical and deterministic).

Rule. Precision **MUST** be sufficient to make the tie-break and certificate verification deterministic. If precision is insufficient, the output **MUST** be rejected (not “accepted with warnings”).

4.5 Rounding

Rounding

Rounding **MUST** be specified as a global policy:

- round-to-nearest-even,
- round-toward-zero,
- floor/ceil,
- or exact reduction (no rounding).

Rule (Canonical rounding). Any conversion between representations (e.g. real $\rightarrow$ fixed-point, or float $\rightarrow$ integer) MUST specify rounding and MUST be applied identically in all implementations.

4.6 Tolerance Conventions

Tolerance

Tolerance is the contract used when equality cannot be exact.

Rule (Tolerance is part of meaning). Any acceptance criterion that depends on a tolerance MUST include that tolerance in NumericModel. Two artifacts with different tolerances are different artifacts.

Canonical tolerance form. Tolerance MUST be expressed as a tuple:

$$\text{Tolerance} := (\varepsilon_{\text{abs}}, \varepsilon_{\text{rel}}, \varepsilon_{\text{op}}, \varepsilon_{\text{spec}}),$$

where:

- ε_{abs} : absolute scalar tolerance,
- ε_{rel} : relative scalar tolerance,
- ε_{op} : operator-norm tolerance (or declared proxy),
- $\varepsilon_{\text{spec}}$: spectral tolerance (e.g. eigenvalue bin width).

Rule. At least one of ε_{op} or $\varepsilon_{\text{spec}}$ MUST be specified for spectral artifacts, unless the artifact is exact by construction.

4.7 Aggregation and Ordering

Aggregation

Distributed evaluation is a primary use-case for QFC-style work. Aggregation MUST therefore be deterministic.

Canonical aggregation policy. If a procedure aggregates partial results $\{x_i\}$, it MUST specify:

- a canonical ordering key (e.g. lexicographic by shard ID, then by local index),
- a deterministic reduction operator,
- and a stable reduction strategy (e.g. pairwise reduction over a fixed binary tree).

Rule (No nondeterministic reductions). Parallel reductions that depend on runtime scheduling MUST NOT be used unless the reduction order is fixed and provably equivalent under the declared tolerance.

4.8 Truncation Model

Truncation

Most operator artifacts are evaluated via finite approximations. The truncation model MUST define:

- how H is truncated to H_N ,
- how auxiliary objects are truncated (e.g. C , $\mathcal{L}$, observable bases),
- and which invariants must hold across N .

Canonical truncation tuple.

Truncation := (Basis, CutRule, BoundaryRule, ScaleSchedule, CrossNPolicy).

- Basis: declared basis / representation in which truncation is performed.
- CutRule: how indices are retained (e.g. first N , energy cutoff, weighted cutoff).
- BoundaryRule: how boundary terms are handled (Dirichlet/Neumann-like, padding, finite-rank correction).
- ScaleSchedule: which N values are permitted or required.
- CrossNPolicy: rules for comparing results across different N (allowed / forbidden / must-rescale).

Rule (Truncation drift is not “noise”). Differences due to truncation are meaningful unless a CrossNPolicy explicitly defines them as negligible under declared tolerances.

4.9 Failure Modes

FailureModes

Every artifact MUST define deterministic failure behavior. A failure mode is a *named outcome* that is returned instead of a numeric artifact when a rule is violated.

Required failure outcomes. At minimum, each artifact MUST support:

- **FAIL_NONCONVERGENCE:** numeric procedure fails to converge under the ScaleSchedule.
- **FAIL_NONDETERMINISM:** repeated evaluation violates determinism checks.
- **FAIL_CERT_INVALID:** certificate verification fails.
- **FAIL_OUT_OF_TOL:** invariants violated beyond declared tolerance.

Rule (No silent fallback). Implementations MUST NOT silently change precision, tolerances, truncation rules, or aggregation order to “make it work”. Any such change MUST change NumericModel (hence SpecID) or trigger a failure mode.

4.10 Deterministic Tie-Break Under Approximation

Many artifacts rely on optimization or selection. Under approximate evaluation, ties may occur due to tolerance.

Rule. If an objective produces candidate scores within tolerance, the TieBreak rule MUST apply to canonical representations, not to approximate numeric values, unless a tolerance-aware tie-break is explicitly specified.

4.11 Minimal NumericModel Profiles (Recommended)

For practical deployments, we recommend three standard profiles:

- **NM-Exact:** rational / fixed-point exact arithmetic, deterministic aggregation, explicit truncation schedule.
- **NM-Stable64:** IEEE binary64 with fixed rounding, canonical reduction order, library-free approximations for transcendentals.
- **NM-Verified:** NM-Stable64 plus interval bounds for certificate verification (accept only if intervals prove invariants).

Artifacts MAY reference these profiles by ID, provided the full tuple is published in the certificate.

4.12 Chapter Summary

This chapter defined the NumericModel as a mandatory component of SpecID. It fixed a canonical schema for number systems, precision, rounding, tolerance, deterministic aggregation, truncation models, and failure modes.

Chapter 5 now specifies certificate formats and verification semantics: what must be published, what a verifier must check, and how artifacts are rejected deterministically.

5 Certificates: Format, Verification, Failure Modes

5.1 Why Certificates Exist

Vol. No. IV is designed for adversarial and multi-implementation settings. A canonical artifact is not “whatever the author produced”; it is what any verifier can independently confirm from public inputs.

Therefore, every artifact in EY23–EY30 MUST define a **certificate** and a **verification procedure**. The certificate is the portable witness that binds:

- the artifact output,
- the SpecID that defines its meaning,
- and the deterministic protocol that makes it reproducible.

5.2 Normative Rule: No Certificate, No Canonical Status

Rule. An artifact instance MUST NOT be accepted as canonical unless:

1. a certificate is provided, and
2. the certificate verification procedure deterministically accepts under the declared Numeric-Model.

5.3 Certificate Schema

Definition 5.1 (Certificate Object). A certificate is a structured artifact π with the following canonical sections:

$\pi := (\text{Header}, \text{SpecID}, \text{Inputs}, \text{Protocol}, \text{Outputs}, \text{Checks}, \text{Commitments}, \text{Signatures}, \text{Metadata})$.

All sections MUST be present. Sections MAY be empty only if explicitly declared as null and the semantics of null are defined.

5.4 Certificate Sections (Normative Semantics)

Header

The header MUST include:

- artifact symbol and ArtifactType,
- SpecID short form,
- Version identifier,
- creation timestamp (informational only; MUST NOT affect verification),
- producer identifier (informational; MUST NOT affect verification).

SpecID

The full SpecID tuple (Chapter 3) MUST be embedded in the certificate, not merely referenced.

Rule. A certificate that references SpecID externally (e.g. via a URL) MUST still include a cryptographic commitment (hash) to the referenced content.

Inputs

Inputs are the public data required to re-derive the artifact. This includes:

- operator core definition(s) (e.g. H , C , $H(t)$, the dynamical map F),
- any admissible-class parameters $\mathcal{A}$,
- objective weights $\mathcal{J}$,
- constraints $\mathcal{C}$,
- tie-break parameters,
- NumericModel parameters,
- truncation schedule (if applicable).

Rule (No hidden inputs). If any information is required for reproduction and is not present in Inputs or committed in Commitments, the certificate MUST be rejected.

Protocol

The protocol section MUST describe the deterministic procedure used to produce the artifact instance. It MUST be specific enough that an independent implementation can re-run it under the NumericModel.

Required content.

- the ordered list of steps,
- explicit aggregation and ordering rules,
- truncation procedure $H \mapsto H_N$,
- stop conditions and failure triggers.

Outputs

The output section MUST contain the artifact instance in canonical representation. Examples:

- EY23: the canonical filter representation $f_\star$ (parameters / normal form) and optional derived invariants,
- EY24: the operator C in canonical form (or its minimal description) and the resulting $V = C^\dagger C$,
- EY26: a canonical representation of U_F (e.g. time-slicing parameters + hash commitment to evolution steps),
- EY27: the generator set $\{G_i\}$ in a canonical ordering,
- EY29: the coarse-grain map R as an explicit matrix / rule set / basis mapping.

Rule. Numeric outputs MUST be normalized and serialized in a canonical encoding so that equality checks are deterministic.

Checks

The checks section MUST enumerate what the verifier must test. Checks MUST be split into:

- **Structural checks:** well-formedness, canonical encoding, SpecID consistency.
- **Constraint checks:** $\mathcal{C}$ validity under the declared NumericModel.
- **Optimality / canonicity checks:** verification that the output is the canonical selection (or an allowed canonical equivalence class) under $\mathcal{J}$ and TieBreak.
- **Invariant checks:** artifact-specific invariants claimed by the whitepaper.

Commitments

Commitments bind external or large objects without embedding them fully.

Required commitments.

- hash of any large input objects not embedded (e.g. large matrices, datasets),
- hash of reference implementations (optional but recommended),
- hash of the canonicalization objective definition text if not embedded in full.

Rule. Commitments **MUST** specify:

- hash algorithm,
- canonical byte encoding used prior to hashing.

Signatures

Signatures are optional. They **MAY** be included for provenance, but **MUST NOT** be required for validity. Verification **MUST** be possible without trusting identities.

Metadata

Metadata **MAY** include:

- runtime logs,
- performance counters,
- environment descriptors.

Metadata **MUST NOT** affect acceptance; it is informational only.

5.5 Verification Semantics

Verifier Contract

A verifier takes (π) and deterministically returns one of:

$$\{\text{ACCEPT}, \text{REJECT}\} \cup \{\text{FAIL_}\ast\}.$$

Rule (Deterministic). Given the same certificate and the same NumericModel, the verifier **MUST** return the same outcome.

Minimal Verification Algorithm

A compliant verifier **MUST** perform the following steps in order:

1. **Parse & canonicalize:** decode the certificate and confirm canonical encoding.
2. **SpecID bind:** confirm that SpecID is complete and internally consistent.
3. **Input completeness:** confirm no hidden inputs exist (or that missing objects are committed by hash).
4. **Protocol feasibility:** confirm protocol steps are well-defined under NumericModel.
5. **Constraint checks:** verify $\mathcal{C}$ deterministically.

6. **Canonicity checks:** verify optimality or tie-break selection as required.
7. **Invariant checks:** verify claimed invariants within declared tolerances.
8. **Return outcome:** ACCEPT or a deterministic failure/rejection code.

5.6 Failure Codes (Normative)

Certificates **MUST** use named failure codes. At minimum, the following codes are **REQUIRED**:

- **REJECT_MALFORMED:** certificate not parseable or violates canonical encoding.
- **REJECT_SPECID_MISSING:** SpecID incomplete or absent.
- **REJECT_SPECID_MISMATCH:** SpecID conflicts with embedded outputs or protocol.
- **REJECT_HIDDEN_INPUT:** reproduction requires uncommitted information.
- **FAIL_NONDETERMINISM:** deterministic replay check fails under NumericModel.
- **FAIL_NONCONVERGENCE:** convergence criteria not met under declared truncation schedule.
- **FAIL_CONSTRAINT:** constraints $\mathcal{C}$ violated.
- **FAIL_CANONICALITY:** output is not the canonical selection (objective/tie-break violation).
- **FAIL_INVARIANT:** claimed invariants violated beyond tolerance.
- **FAIL_HASH_MISMATCH:** a committed external object does not match its hash.

Rule (No silent downgrade). A verifier **MUST NOT** accept an artifact by silently weakening constraints, increasing tolerances, or switching NumericModel profiles. Any such change would require a different SpecID and therefore a different certificate.

5.7 Certificate Minimalism vs Completeness

Certificates **SHOULD** be minimal but **MUST** be complete. This creates a design tension.

Normative guidance.

- Use Commitments for large objects; embed only the canonical hashes and canonical encoding specs.
- Ensure Outputs are in a stable canonical representation (parameters, normal forms).
- Ensure Checks are explicit and short, but sufficient to force deterministic acceptance/rejection.

5.8 Chapter Summary

This chapter defined the certificate as the audit object that binds artifact meaning to SpecID and to a deterministic protocol. It fixed a canonical certificate schema, a minimal verifier algorithm, and a set of required failure codes.

Chapter 6 next defines interoperability: how artifacts compose, what compatibility means, and how a compatibility matrix is represented and verified.

6 Interoperability: Composition Rules and Compatibility Matrix

6.1 Why Interoperability Is a First-Class Topic

Vol. No. IV is not a collection of isolated definitions. The eight artifacts are intended to *compose* into workflows: spectral probing, validity enforcement, bucketing, program tokens, regime bridges, and Koopman export. If composition rules are left implicit, then the suite reintroduces the ambiguity it was designed to remove.

Accordingly, interoperability is treated as a **normative contract** specifying:

- which compositions are permitted,
- which invariants **MUST** be preserved,
- how compatibility is certified and verified,
- and what failures are returned deterministically.

6.2 Interop Component in SpecID

Every artifact in EY23–EY30 **MUST** include an Interop component in its SpecID:

$$\text{SpecID} := (\cdots, \text{Interop}, \text{Version}).$$

Interop is the formal statement of: (i) how the artifact composes with others, and (ii) which compatibility predicates and invariants govern that composition.

Rule. If an artifact claims compatibility with another artifact, the compatibility condition **MUST** be included in Interop, and **MUST** be checkable under the declared NumericModel.

6.3 Primitive Composition Forms

This section defines the primitive composition forms used throughout Vol. No. IV. Any higher-level workflow **MUST** be reducible to these primitives (or declare additional primitives explicitly).

C1: Functional Composition

If an artifact produces an operator $A \in \mathcal{B}(\mathcal{H})$ and another consumes an operator, then the composition **MAY** be operator multiplication or evaluation of a function of an operator:

$$AB, \quad BA, \quad f(A), \quad f(A)B, \quad Bf(A),$$

provided domains are compatible and the functional calculus is declared.

C2: Transport by a Morphism (Pushforward / Pullback)

If $T_{1 \rightarrow 2}$ is an intertwiner (EY28), then objects **MAY** be transported between regimes:

$$A_2 := T_{1 \rightarrow 2} A_1 T_{1 \rightarrow 2}^{-1} \quad \text{or} \quad A_2 := T_{1 \rightarrow 2} A_1 T_{1 \rightarrow 2}^{\dagger},$$

depending on declared semantics (unitary / isometric / generalized mapping). The chosen transport law **MUST** be part of Interop.

C3: Coarse-Grain Reduction

If R is a coarse-grain operator (EY29), then detailed objects MAY be reduced:

$$\tilde{A} := R(A),$$

where R MUST preserve the invariants explicitly declared as preserved (e.g. spectral classes, positivity, trace, validity class), within the tolerances of the NumericModel.

C4: Superoperator Action

If $\mathcal{L}$ is a superoperator (EY25, EY30), it acts on states or observables:

$$\rho \mapsto \mathcal{L}(\rho), \quad g \mapsto \mathcal{K}(g).$$

Composition is function composition at the superoperator level.

C5: Token Execution / Concatenation

If U_F is a Floquet program token (EY26), it MAY compose by multiplication (time concatenation) only if the same time-ordering and discretization semantics are used:

$$U_{F,2} U_{F,1}.$$

Compatibility of periods, slicing, and ordering MUST be specified in Interop (or the composition MUST fail).

6.4 Compatibility: What It Means

Definition 6.1 (Compatibility Predicate). A compatibility predicate is a boolean condition of the form

$$\text{Compat}(A, B) \in \{\text{TRUE}, \text{FALSE}\},$$

where A, B are artifact instances (each with SpecID). A valid Compat predicate MUST be:

- deterministic under the declared NumericModel(s),
- explicit in terms of checkable invariants and protocol steps,
- stable under declared truncation policies (when truncation applies).

Rule (Compatibility is asymmetric by default). Unless explicitly stated, $\text{Compat}(A, B) \neq \text{Compat}(B, A)$. The compatibility direction MUST be declared.

6.5 Required Interop Invariants

Interoperability is defined by preservation of invariants. Each artifact MAY define additional invariants, but Vol. No. IV standardizes a minimal invariant vocabulary.

I1: Domain Compatibility

If A and B are composed, their domains MUST be compatible under the declared truncation and calculus. If a domain mismatch occurs, the composition MUST fail deterministically.

I2: SpecID Consistency

If artifacts are composed in a workflow, their SpecIDs MUST not conflict on shared primitives. In particular:

- if both use a NumericModel, either it MUST match or a declared coercion rule MUST exist;
- if both use functional calculus, calculus choices MUST be compatible or explicitly bridged.

I3: Deterministic Replay

A composed workflow MUST produce the same result across verifiers given the same public inputs. This includes deterministic aggregation order and deterministic tie-break behavior.

I4: Invariant Preservation Under Reduction

If R is applied (EY29), invariants marked as *preserved* MUST hold after reduction within tolerance:

$$\text{Inv}(A) \Rightarrow \text{Inv}(R(A)),$$

as declared in the Interop component.

I5: Validity Propagation

If validity is enforced by $V = C^\dagger C$ (EY24) or $\mathcal{L}_{\text{audit}}$ (EY25), then workflows MUST define how validity propagates:

- whether filtering preserves validity,
- whether coarse-grain preserves or weakens validity,
- whether an intertwiner preserves validity constraints across regimes.

6.6 Compatibility Matrix

To make interoperability machine-readable and auditable, this volume standardizes a compatibility matrix format.

Definition 6.2 (Compatibility Matrix). A compatibility matrix $\mathbf{M}$ is an 8×8 matrix indexed by artifacts (EY23, ..., EY30) such that each entry is one of:

- OK: composition is permitted under declared invariants and check procedures;
- OK*: composition is permitted only under additional constraints (explicitly listed);
- NO: composition is forbidden (incompatible by design);
- N/A: composition is not defined / not meaningful for this pair.

For any entry marked OK or OK*, the interoperability contract MUST reference:

- a compatibility predicate identifier,
- the required invariant list,
- and the verification routine (or certificate check ID).

Rule (No informal OK). A matrix entry **MUST NOT** be declared OK or OK* unless a verifier can deterministically evaluate the referenced compatibility predicate under NumericModel.

6.7 Default Interop Policy (Vol. No. IV Baseline)

The following baseline policy is adopted unless overridden by an artifact:

- EY23 $F_*(H)$ **MAY** be applied before or after EY24 V only if domain and calculus match.
- EY29 R **MAY** be applied to outputs of EY23/EY24/EY25 only if invariant preservation is certified.
- EY28 $T_{1 \rightarrow 2}$ **MAY** transport EY23/EY24/EY27 objects only if invariants are preserved under the declared mapping law.
- EY26 U_F **MAY** be used to generate inputs to EY23/EY27 only if time-slicing semantics are compatible and declared.
- EY30 $\mathcal{K}$ **MAY** export workflows into observable-space only if correspondence of invariants is declared.

This baseline is intentionally conservative. Artifacts may expand compatibility only by providing explicit predicates and certificates.

6.8 Interop Certificates

Whenever a workflow depends on compatibility, it **SHOULD** produce an *Interop Certificate*.

Definition 6.3 (Interop Certificate). An interoperability certificate is a certificate π_{interop} that attests:

1. the SpecIDs of all artifacts in the workflow,
2. the declared composition form(s) (C1–C5),
3. the compatibility predicates evaluated and their outcomes,
4. the invariants checked and their tolerances,
5. the deterministic aggregation and truncation schedule used for the workflow.

Rule. If a workflow outcome is published as an artifact-derived claim, an interop certificate **MUST** be provided if the claim depends on cross-artifact composition.

6.9 Failure Modes for Interop

Interop verification **MUST** use deterministic failure codes, at minimum:

- **FAIL_DOMAIN_MISMATCH:** domains not compatible for composition.
- **FAIL_CALC_MISMATCH:** functional calculus semantics incompatible.
- **FAIL_NUMERICMODEL_MISMATCH:** numeric models conflict without declared coercion.
- **FAIL_INVARIANT_NOT_PRESERVED:** required invariants fail after composition.
- **FAIL_COMPAT_PREDICATE:** compatibility predicate evaluates to FALSE.

6.10 Chapter Summary

This chapter defined interoperability as a normative contract. It introduced primitive composition forms (C1–C5), compatibility predicates, required invariant vocabulary, a machine-readable compatibility matrix, interop certificates, and deterministic failure codes for composition failures. Chapter 7 next fixes the publication boundary and non-claims, and states the licensing intent for Vol. No. IV in a QFC-consistent way.

7 Non-Claims, Publication Boundary, and Licensing Intent

7.1 Why This Chapter Exists

Vol. No. IV introduces a normative operator-law layer: definitions, certificates, and deterministic protocols. This can look “stronger” than it is. Canonicalization is not a proof mechanism; auditability is not a truth claim. This chapter draws an explicit boundary around what EY22 and the subsequent EY23–EY30 documents do *not* claim, and it clarifies the publication and licensing intent for the suite.

7.2 Non-Claims (Hard Boundary)

The following statements are normative: they define what the suite **MUST NOT** be interpreted as.

No Physical Realizability Claim

Nothing in Vol. No. IV asserts that any artifact is physically realizable on a quantum device, nor that any Hamiltonian H corresponds to a realized quantum field theory.

Rule. The suite defines *operator objects and contracts*. Physical implementation is out of scope unless a separate document explicitly provides an implementation claim with its own SpecID and experimental protocol.

No Algorithmic Advantage Claim

No artifact in EY22–EY30 claims:

- quantum advantage,
- speedups over classical baselines,
- complexity separations,
- or cryptographic hardness assumptions.

Rule. If a future document claims advantage, it **MUST** be framed as a separate claim with explicit benchmarks, threat models, and a reproducible measurement protocol.

No “Proof by Definition”

Canonical definitions are not proofs of conjectures.

Rule. Vol. No. IV MUST NOT be interpreted as proving the Riemann Hypothesis (or any open conjecture) merely by defining a Hamiltonian, filter, penalty, or spectral extraction protocol. Any future claim connecting an artifact to a conjecture MUST present falsifiable implications and must survive independent reproduction.

No Hidden Authority

Canonical does not mean “authoritative because Enter Yourname says so.”

Rule. Canonization is only granted by the existence of:

- a published SpecID,
- a deterministic NumericModel,
- and a certificate verifiable by independent parties.

If these are missing, the artifact is not canonical in the Vol. No. IV sense.

7.3 Publication Boundary (What Counts as the Artifact)

The suite distinguishes between:

- **Normative layer:** the definitions, SpecID schema, NumericModel rules, certificate formats, and verification semantics.
- **Implementation layer:** code, numerical experiments, simulation results, and performance data.

Rule (Artifact identity). An artifact is defined by its SpecID and certificate semantics, not by a specific codebase. Code MAY be published, and SHOULD be committed by hash, but is not the definitional anchor.

Rule (Reproducibility). Any published numerical result based on these artifacts SHOULD include:

- the full SpecIDs of the artifacts used,
- the NumericModel profile(s),
- truncation schedule(s),
- and an interop certificate if composition is involved.

7.4 Normative vs Informational Text

To keep the suite audit-friendly, the documents distinguish:

- **Normative text:** uses MUST/SHOULD/MAY and affects meaning.
- **Informational text:** intuition, motivation, examples, and non-binding commentary.

Rule. If a paragraph affects what a verifier must do, it MUST be normative. If it is not normative, it MUST NOT be required for acceptance.

7.5 Licensing Intent (High-Level)

Vol. No. IV is written to be:

- **publicly readable** and referenceable,
- **machine-readable** in structure,
- and **compatible with downstream implementation** without forcing trust in any one repository.

Intent. The intent is to allow broad use of the methodology while preserving clear attribution and protecting the integrity of the specification layer.

Non-binding note. The concrete legal licensing text is typically maintained in the repository and/or publication venue. This whitepaper states the *intent*: that the normative definitions (SpecIDs, certificate formats, and verification semantics) are published as a stable reference, and implementations may be licensed separately.

7.6 Attribution and Citation Form

Rule (Citation). Any reuse of Vol. No. IV artifacts in publications SHOULD cite:

- the artifact ID (EY22..EY30),
- the SpecID version,
- and the canonical publication location (e.g. DOI / hash commitment).

A recommended short form:

EY29 @ SpecID_Short :: VID-2026-02, Vol. No. IV (Enter Yourname).

7.7 Responsible Use Boundary

Although Vol. No. IV is methodological, it can be applied to security evaluation and adversarial analysis contexts. This volume therefore states a boundary:

Rule (Responsible framing). If an artifact is used in a context that could reasonably be interpreted as enabling wrongdoing, the publication MUST include a threat model and a limitation statement, and MUST avoid procedural instructions intended to cause harm. This suite is intended for auditable, responsible research and verification.

7.8 Chapter Summary

This chapter fixed the hard boundary of Vol. No. IV:

- no physical realizability claim,
- no algorithmic advantage claim,
- no proof-by-definition,
- and no hidden authority.

It also clarified what counts as the artifact (SpecID + certificate semantics), how normative text is distinguished from informational text, and stated the licensing and attribution intent for publication.

Appendices Reserved

Appendix A and Appendix B are reserved for canonical templates and checklists (to be filled when we decide the exact serialization formats and repository layout conventions).

APPENDIX A

A.1 Reference Template: Canonical Representation and Hash Commitments

A.1.1 Purpose

This appendix defines a canonical serialization template for certificates and a minimal set of hash-commitment rules. The goal is to ensure that:

- certificates are **byte-stable** (the same content yields the same bytes),
- commitments are **unambiguous** (the same object yields the same hash),
- verifiers can deterministically reject malformed or ambiguous artifacts.

This appendix is normative. Any artifact in EY23–EY30 that claims conformance to EY22 MUST follow this template, unless it explicitly declares an alternative canonical encoding in its SpecID.

A.1.2 Canonical Certificate Encoding (CCE)

CCE-1: Allowed Formats

A certificate MUST be encoded using exactly one of the following formats:

- **Canonical JSON** (preferred for machine verification),
- **Canonical YAML** (allowed only if transformed to Canonical JSON before hashing and verification),
- **CBOR** (allowed only if a canonical CBOR profile is declared in SpecID).

Rule (single truth). If YAML is used for human readability, the certificate MUST still define a *Canonical JSON projection* as the single source of truth for hashing and verification.

CCE-2: Canonical JSON Rules

The Canonical JSON projection MUST satisfy:

1. **UTF-8:** The document MUST be encoded in UTF-8 without BOM.
2. **Key ordering:** All object keys MUST be sorted lexicographically (byte-order of UTF-8).
3. **No duplicate keys:** Duplicate keys MUST cause **REJECT_MALFORMED**.
4. **Numbers:** Numbers MUST be represented deterministically:
 - integers in base-10 with no leading zeros (except “0”),
 - non-integers MUST NOT appear unless the NumericModel explicitly allows them and defines their canonical string form.

5. **No NaN/Inf:** NaN, +Inf, -Inf MUST NOT appear; if needed, they MUST be represented as strings with explicit tags.
6. **Whitespace:** No insignificant whitespace is permitted (minified JSON). Newlines are not permitted inside tokens.
7. **Arrays:** Arrays preserve order; order is semantic and MUST be canonical where relevant (see CCE-4).
8. **Strings:** Strings MUST be NFC-normalized Unicode and MUST escape characters using standard JSON escaping only.

Rule. If any rule above is violated, verification MUST return **REJECT_MALFORMED**.

CCE-3: Canonical Section Layout

The Canonical JSON projection MUST include top-level keys in the following order (ordering rule still applies, this is a schema requirement):

- "header"
- "specid"
- "inputs"
- "protocol"
- "outputs"
- "checks"
- "commitments"
- "signatures"
- "metadata"

Each key MUST exist. If a section is empty, it MUST exist as an empty object {} or empty array [] as defined below.

Null policy. The literal `null` MUST NOT be used. Absence is represented only by empty containers with explicit semantics.

CCE-4: Canonical Ordering Within Sections

Within sections where ordering could be arbitrary, ordering MUST be canonical:

- "outputs": lists of multiple objects MUST be sorted by a declared "id" field.
- "checks": check entries MUST be sorted by "check_id".
- "commitments": commitment entries MUST be sorted by "name".
- "protocol.steps": step lists preserve order and MUST be written in canonical execution order.

If a whitepaper introduces a new list-like structure, it MUST define the canonical ordering key.

A.1.3 Certificate Schema Template (Normative JSON Skeleton)

The following is the normative skeleton for a certificate in Canonical JSON form.

```
{
  "header": {
    "artifact_symbol": "F_star(H)",
    "artifact_type": "CSF",
    "specid_short": "CSF:CalcID:AID:JID:CID:NID:VID",
    "version": "VID-YYYY-MM",
    "created_utc": "YYYY-MM-DDThh:mm:ssZ",
    "producer": "informational"
  },
  "specid": {
    "artifact_type": "CSF",
    "core": {},
    "calc": {},
    "A": {},
    "J": {},
    "C": {},
    "tiebreak": {},
    "numeric_model": {},
    "cert": {},
    "interop": {},
    "version": "VID-YYYY-MM"
  },
  "inputs": {
    "core_objects": {},
    "public_parameters": {},
    "truncation_schedule": {},
    "external_objects": []
  },
  "protocol": {
    "steps": [],
    "aggregation": {},
    "stopping": {},
    "failure_modes": []
  },
  "outputs": {
    "canonical_representation": {},
    "derived_invariants": {},
    "output_hashes": []
  },
  "checks": {
    "structural": [],
    "constraints": [],
    "canonicity": [],
    "invariants": []
  },
  "commitments": {
    "hash_alg": "sha256",
    "byte_encoding": "utf-8",
```

```

    "objects": []
  },
  "signatures": {},
  "metadata": {}
}

```

Rule. Artifacts MAY extend this schema by adding keys *inside* existing section objects, but MUST NOT add new top-level sections.

A.1.4 Hash Commitments

HC-1: Default Hash Algorithm

Unless otherwise specified, the hash algorithm MUST be:

HashAlg = sha256.

A different hash algorithm MAY be used only if:

- it is declared in "commitments.hash_alg",
- it is included in SpecID,
- and the verifier supports it deterministically.

HC-2: What Is Hashed

There are three categories of hash commitments:

1. **Certificate hash:** hash of the entire Canonical JSON byte string of the certificate.
2. **Object commitments:** hashes of external large objects (matrices, datasets, code snapshots).
3. **Section commitments:** hashes of specific sections if partial referencing is needed (optional).

Rule. Any claim that references an external object MUST include an object commitment for it.

HC-3: Canonical Byte Encoding Before Hashing

All hashes MUST be computed over a *canonical byte encoding*.

Default. For certificates and JSON sections: canonical byte encoding is the UTF-8 encoding of Canonical JSON (CCE-2).

External objects. For external objects, the commitment MUST specify:

- "name": stable identifier,
- "media_type": e.g. "application/octet-stream", "application/pdf", "text/plain",

- "canonicalization": how the object was converted into bytes,
- "hash": the computed hash,
- "size_bytes": informational size.

If the object is a text file, canonicalization **MUST** specify:

- UTF-8 encoding,
- LF line endings,
- and whether trailing whitespace is preserved or stripped (default: preserved).

HC-4: Chunked Objects and Merkle Commitments (Optional)

For very large objects, a Merkle commitment **MAY** be used. If used, the commitment **MUST** specify:

- chunk size in bytes,
- leaf hash algorithm,
- tree construction rules (ordering, padding),
- root hash.

Rule. Merkle commitments **MUST** be deterministic. Any ambiguity in chunking or ordering **MUST** cause rejection.

A.1.5 Commitment Object Template

Each external commitment entry in "commitments.objects" **MUST** follow:

```
{
  "name": "H_truncation_matrix_N1024",
  "media_type": "application/octet-stream",
  "canonicalization": {
    "type": "raw-bytes",
    "notes": "exact bytes as stored"
  },
  "hash_alg": "sha256",
  "hash": "hex-string",
  "size_bytes": 1234567
}
```

Rule. Hashes **MUST** be encoded as lowercase hexadecimal without prefixes.

A.1.6 Canonical IDs and Short References

To allow stable references inside and across certificates, the following IDs are standardized:

- artifact_type: one of CSF, VP0, LIOUV, FLOQ, COMM, INTW, CGR, KOOP.
- calc_id, A_id, J_id, C_id, nm_id: stable labels resolving to full definitions in "specid".

- `check_id`: unique string within `"checks"`.
- `step_id`: unique string within `"protocol.steps"`.

Rule. IDs MUST be ASCII and MUST be deterministic (no random UUIDs) unless derived deterministically from public inputs.

A.1.7 Verifier Requirements (CCE/HC)

A compliant verifier MUST:

- reject non-canonical JSON encodings,
- recompute certificate hashes and compare against published commitments (if provided),
- verify each external object hash if the object is supplied,
- treat any canonicalization mismatch as **FAIL_HASH_MISMATCH** or **REJECT_MALFORMED**.

A.1.8 Appendix Summary

This appendix fixed a canonical certificate encoding (Canonical JSON projection) and standardized hash commitment rules. Together, these rules ensure that Vol. No. IV artifacts can be bound to deterministic bytes, enabling portable verification and eliminating ambiguity at the serialization boundary.

APPENDIX B

B.1 Checklist: “QFC-Valid Whitepaper” Requirements

B.1.1 Purpose

This appendix provides a deterministic checklist for validating that a whitepaper in Vol. No. IV (EY23–EY30) conforms to the EY22 conventions. The checklist is designed for:

- authors (before publication),
- reviewers (during audit),
- verifiers (as a quick failure triage map).

Rule. A whitepaper that fails any **MUST** item below MUST be treated as non-conformant to Vol. No. IV.

B.1.2 A. Identity and Scope

- ☐ **MUST:** The document has a stable ID (e.g. EY23) and the ID appears in the frontmatter.
- ☐ **MUST:** `ArtifactType` is explicitly declared (`CSF`, `VPO`, `LIQUV`, `FLOQ`, `COMM`, `INTW`, `CGR`, `KOOP`).
- ☐ **MUST:** Scope and Non-Claims are explicit (no proof-by-definition; no advantage claim; no physical claim).
- ☐ **SHOULD:** The document states intended role(s) within the QFC stack and references the suite registry (EY22).

B.1.3 B. Normative Definitions

- ☐ **MUST:** The artifact has a normative definition as an operator-level object (not a procedure-only description).
- ☐ **MUST:** All normative requirements use **MUST**/**SHOULD**/**MAY**; vague language does not affect meaning.
- ☐ **MUST:** Core objects are defined (e.g. H , C , $H(t)$, `Obs`) with domain assumptions when required.
- ☐ **MUST:** If functional calculus is used, the calculus is explicitly declared.

B.1.4 C. SpecID Completeness

- ☐ **MUST:** Full SpecID tuple is defined and embedded (not only referenced by name).
- ☐ **MUST:** Any irrelevant SpecID component is present with an explicit null value and semantics.
- ☐ **MUST:** Version is specified and changes whenever meaning changes.
- ☐ **SHOULD:** A SpecID short form is provided for human readability, with a deterministic expansion rule.

B.1.5 D. Admissible Family, Objective, Constraints, Tie-Break

- ☐ **MUST:** Admissible family $\mathcal{A}$ is explicitly specified and has canonical representation.
- ☐ **MUST:** If selection/optimization is used, objective $\mathcal{J}$ is explicit and deterministically evaluable.
- ☐ **MUST:** Constraints $\mathcal{C}$ are explicit (no “reasonable constraints”).
- ☐ **MUST:** TieBreak is defined and deterministic whenever non-uniqueness or tolerance ties may occur.
- ☐ **SHOULD:** The paper states minimal existence / well-posedness conditions or declares failure modes.

B.1.6 E. NumericModel (Determinism)

- ☐ **MUST:** NumericModel tuple is included in SpecID and specified explicitly.
- ☐ **MUST:** Precision and rounding are specified; NaN/Inf behavior is forbidden or explicitly encoded.
- ☐ **MUST:** Tolerance conventions are explicit ($\varepsilon_{\text{abs}}, \varepsilon_{\text{rel}}, \varepsilon_{\text{op}}, \varepsilon_{\text{spec}}$).
- ☐ **MUST:** Aggregation order is deterministic (ordering key + fixed reduction strategy).
- ☐ **MUST:** Truncation model is specified (basis, cut rule, boundary rule, schedule, cross- N policy).
- ☐ **MUST:** Failure modes are defined and deterministic (no silent fallback).

B.1.7 F. Invariants

- ☐ **MUST:** Required invariants are listed explicitly (what is preserved / enforced).
- ☐ **MUST:** Each invariant has a check procedure (exact or tolerance-based) under NumericModel.
- ☐ **SHOULD:** Invariants are grouped as structural / spectral / validity / interop, for audit clarity.

B.1.8 G. Certificates

- ☐ **MUST:** The paper defines a certificate format π conforming to EY22 Chapter 5.
- ☐ **MUST:** The paper defines a verifier procedure that deterministically returns ACCEPT/REJECT/FAIL_*
- ☐ **MUST:** The certificate binds to full SpecID and NumericModel.
- ☐ **MUST:** Required failure codes are mapped (at least those in EY22 Chapter 5).
- ☐ **SHOULD:** The certificate includes commitments to large external objects (hashes) and canonicalization rules.

B.1.9 H. Canonical Encoding and Hash Commitments

- ☐ **MUST:** Canonical JSON projection rules are adopted (EY22 Appendix A.1) or explicitly overridden in SpecID.

- ☐ **MUST:** Hash algorithm and canonical byte encoding are specified for all commitments.
- ☐ **MUST:** Hashes are lowercase hex; duplicate-key JSON is rejected.
- ☐ **SHOULD:** Merkle/chunking rules are specified if large objects require chunked commitments.

B.1.10 I. Interoperability

- ☐ **MUST:** Interop component exists in SpecID and states composition permissions and requirements.
- ☐ **MUST:** Compatibility predicates are explicit and checkable.
- ☐ **MUST:** Composition failure codes exist (domain mismatch, calc mismatch, numeric model mismatch, invariant failure).
- ☐ **SHOULD:** A compatibility matrix entry (OK/OK*/NO/N/A) is declared for relevant pairings.
- ☐ **SHOULD:** Workflows that publish composed results define an interop certificate.

B.1.11 J. Protocol and Reproducibility Package

- ☐ **MUST:** The paper defines a deterministic evaluation protocol (steps, ordering, stopping conditions).
- ☐ **MUST:** Protocol changes imply SpecID changes (version bump).
- ☐ **SHOULD:** A minimal reproducibility bundle is described (inputs, commitments, how to re-run verifier).

B.1.12 K. Final “Release Gate”

A paper is considered **QFC-valid (Vol. No. IV)** only if:

- ☐ all MUST items in Sections A–J are satisfied, and
- ☐ the certificate verifier accepts at least one reference certificate instance under the declared NumericModel.

B.1.13 Appendix Summary

This appendix provided a deterministic, auditable checklist for ensuring that EY23–EY30 whitepapers are conformant to EY22 conventions. It functions as the suite’s “release gate” for canonical operator artifacts.

EY23: Canonical Spectral Filter

$F_{\star}(H)$

Canonical Operator Artifacts for QFC

Enter Yourname

Abstract

This whitepaper defines the Canonical Spectral Filter $F_{\star}(H)$, a QFC operator artifact that extracts a stable, audit-friendly spectral action from a declared Hamiltonian H without hand-tuned filter design. Unlike ad hoc choices of $f(H)$, $F_{\star}(H)$ is defined by an explicit admissible family of filters, a canonical objective functional, constraint predicates, and a deterministic tie-break rule, all bound into a SpecID as standardized in EY22.

The central requirement is reproducibility: independent parties given the same public inputs MUST derive the same canonical filter (or the same declared canonical equivalence class) under the same NumericModel. The whitepaper therefore specifies (i) the functional calculus semantics used to interpret $f(H)$, (ii) truncation and aggregation policies for practical evaluation, (iii) stability and monotonicity invariants expected of a spectral filter used for auditing and regime classification, and (iv) certificate semantics and verifier checks that accept or reject $F_{\star}(H)$ deterministically.

$F_{\star}(H)$ serves as the baseline spectral probing primitive in Vol. No. IV. It is designed to compose with validity operators, audit-native evolution, coarse-graining, and intertwiners, providing a canonical interface between raw operator cores and higher-level QFC workflows where comparability across implementations is mandatory.

1 Motivation, Scope, and Design Goals

1.1 Motivation: Why a Canonical Spectral Filter?

Given a self-adjoint operator H (the declared core Hamiltonian), a common operation across spectral workflows is to apply a function f to H and interpret the result:

$$f(H).$$

In practice, the choice of f is often informal: a smoothing kernel is selected, a window is tuned, or a polynomial is fitted until results “look stable.” This creates a systematic failure mode for QFC-style work:

- **Parameter smuggling:** small filter choices become hidden degrees of freedom.

- **Non-comparability:** two parties claiming the same spectral conclusion may have used incompatible f .
- **Non-auditability:** without a fixed selection rule, $f(H)$ cannot be verified as canonical.

EY23 addresses this by defining a **Canonical Spectral Filter** $F_\star(H)$: a filter action on H that is not hand-designed, but selected by a published objective and constraints, with a deterministic tie-break, all bound into a SpecID as standardized in EY22.

1.2 What This Whitepaper Defines

This whitepaper defines:

- the artifact $F_\star(H)$ as a canonical instance of $f(H)$ under a declared calculus,
- the admissible family $\mathcal{A}$ of filter candidates f ,
- the objective functional $\mathcal{J}$ that selects a canonical filter,
- the constraint set $\mathcal{C}$ that enforces stability and audit-friendly behavior,
- the tie-break rule ensuring determinism in the presence of non-uniqueness,
- and the certificate and verifier checks required for canonical acceptance.

All general conventions (SpecID, NumericModel, certificate schema, interop rules) are inherited from EY22 unless explicitly overridden and recorded in the SpecID.

1.3 Scope

In-scope.

- Canonical selection of a spectral filter action derived from a declared H .
- Deterministic evaluation under truncation $H \mapsto H_N$ using a declared truncation policy.
- Certificate semantics enabling third-party verification that the produced filter is canonical under SpecID.
- Interoperability hooks: how $F_\star(H)$ composes with validity and coarse-grain artifacts.

Out-of-scope.

- Any claim that H represents a physically realized system.
- Any claim of computational advantage (quantum or classical).
- Any claim that a particular spectral feature implies resolution of an open conjecture.
- Any attempt to standardize *all* possible filter families; this paper defines a canonicalizable framework and a baseline profile.

1.4 Design Goals (Normative)

The Canonical Spectral Filter **MUST** satisfy the following design goals:

G1: Canonicity by Specification

$F_\star(H)$ MUST be determined by public specification, not author preference. In particular, the output MUST be uniquely determined by:

$$(H, \text{Calc}, \mathcal{A}, \mathcal{J}, \mathcal{C}, \text{TieBreak}, \text{NumericModel}).$$

G2: Auditability

A verifier MUST be able to accept or reject the artifact using only:

- the certificate,
- the declared SpecID,
- and the public inputs (or their hash commitments).

The verifier MUST NOT require trusting the producer.

G3: Stability Under Declared Perturbations

The filter selection SHOULD be stable under the perturbation class declared in $\mathcal{C}$ (e.g. bounded additive noise, small truncation changes). Stability is always relative to the declared Numeric-Model and truncation schedule.

G4: Minimal Degrees of Freedom

The admissible family $\mathcal{A}$ SHOULD be chosen to minimize hidden degrees of freedom. If $\mathcal{A}$ is broad, the objective $\mathcal{J}$ and tie-break MUST be strong enough to prevent parameter smuggling.

G5: Interoperability With Validity and Reduction

$F_\star(H)$ SHOULD be composable with:

- validity penalties $V = C^\dagger C$ (EY24),
- audit-native evolution $\mathcal{L}_{\text{audit}}$ (EY25),
- coarse-grain reduction R (EY29),

under the interoperability rules of EY22 Chapter 6. Any additional compatibility predicates MUST be declared in the EY23 Interop component.

1.5 Interpretation: What $F_\star(H)$ Is Used For

This artifact is intended as the baseline “spectral probe” in Vol. No. IV. Typical uses include:

- extracting stable spectral summaries for audit and classification,
- defining regime buckets by canonical spectral features,
- producing inputs to commutant discovery workflows (EY27),
- and serving as a standardized pre-processing operator prior to coarse-graining (EY29).

1.6 Chapter Summary

This chapter motivated the need for canonical spectral filtering and fixed the scope and design goals for $F_\star(H)$. Chapter 2 now introduces the normative definition of the artifact and the EY23 SpecID components that bind its meaning: calculus, admissible family, objective, constraints, tie-break, and NumericModel profile.

2 Normative Definition and SpecID Binding

2.1 Conformance to EY22

EY23 conforms to the Vol. No. IV conventions defined in EY22. In particular:

- SpecID semantics are as defined in EY22 Chapter 3.
- NumericModel (determinism model) is as defined in EY22 Chapter 4.
- Certificate schema and verifier semantics are as defined in EY22 Chapter 5 and Appendix A.1.
- Interoperability rules are as defined in EY22 Chapter 6.

Any deviation **MUST** be explicitly declared in the EY23 SpecID.

2.2 Core Object and Assumptions

Core. The core object for this artifact is a (typically self-adjoint) operator H on a Hilbert space $\mathcal{H}$.

Assumptions (minimal). EY23 requires that:

- H is self-adjoint (or declared self-adjoint on a specified domain) so that spectral calculus is meaningful;
- the truncation policy (if used) produces finite objects H_N with a deterministic construction;
- the admissible family $\mathcal{A}$ is non-empty under the declared constraints $\mathcal{C}$.

If these assumptions fail, the artifact **MUST** return a deterministic failure outcome (EY22 Chapter 4).

2.3 Artifact Definition

Definition 2.1 (Canonical Spectral Filter). Let H be the declared core operator. Let $\mathcal{A}$ be an admissible family of filter candidates f , let $\mathcal{J}$ be an objective functional, and let $\mathcal{C}$ be constraint predicates.

The **Canonical Spectral Filter** is defined as:

$$F_\star(H) := f_\star(H),$$

where $f_\star \in \mathcal{A}$ is the canonical selection obtained by:

$$f_\star \in \arg \min_{f \in \mathcal{A}} \mathcal{J}(f; H) \quad \text{subject to } \mathcal{C}(f; H) = \text{TRUE},$$

with deterministic tie-breaking applied whenever the minimizer is not unique under the declared NumericModel.

If the objective is a maximization, the above is interpreted with $\arg \max$ (this choice MUST be fixed in SpecID).

Rule (selection is part of meaning). The meaning of $F_\star(H)$ is the entire selection procedure: $\mathcal{A}$, $\mathcal{J}$, $\mathcal{C}$, tie-break, calculus, and numeric model. Two filters with different selection rules are different artifacts.

2.4 Functional Calculus Semantics

EY23 permits $f(H)$ only under an explicitly declared calculus:

Allowed calculus classes.

- **Borel functional calculus** for bounded f ,
- **holomorphic/rational calculus** for resolvent-type families,
- **truncation calculus** defined by $H \mapsto H_N$ and $f(H) \approx f(H_N)$ under a declared truncation schedule.

Rule. The calculus choice MUST be encoded in Calc inside SpecID and MUST be compatible with the NumericModel.

2.5 SpecID for EY23

ArtifactType. For EY23:

$$\text{ArtifactType} = \text{CSF}.$$

SpecID components (EY23 binding). The full SpecID tuple is:

$$\text{SpecID}_{\text{EY23}} := (\text{CSF}, \text{Core} = (H), \text{Calc}, \mathcal{A}, \mathcal{J}, \mathcal{C}, \text{TieBreak}, \text{NumericModel}, \text{Cert}, \text{Interop}, \text{Version}).$$

Each component MUST be specified as follows:

Calc (required)

Calc MUST declare:

- calculus class (Borel / holomorphic / truncation calculus),
- if truncation calculus: the truncation mapping $H \mapsto H_N$ and evaluation of $f(H_N)$,
- canonicalization of any numeric primitives used (e.g. matrix exponentiation or polynomial evaluation).

$\mathcal{A}$ (required)

$\mathcal{A}$ MUST specify the admissible family of candidate filters f , including:

- function class (e.g. spline family, rational family, orthogonal expansion),

- parameter bounds (to avoid hidden degrees of freedom),
- canonical representation form (so tie-break can be deterministic on bytes).

$\mathcal{J}$ (required)

$\mathcal{J}(f; H)$ MUST specify:

- whether the objective is minimized or maximized,
- the exact score function(s),
- any weights and normalization rules,
- any dependence on truncation schedule or spectral summaries.

$\mathcal{C}$ (required)

$\mathcal{C}(f; H)$ MUST include constraint predicates. Typical constraints include:

- boundedness / Lipschitz constraints on f ,
- monotonicity constraints (if used for bucket ordering),
- positivity constraints (if $f(H)$ is used as a penalty-like object),
- stability constraints across truncation levels N .

Rule. Constraints MUST be checkable deterministically under NumericModel, or the artifact MUST fail.

TieBreak (required)

TieBreak MUST define a deterministic tie-breaking rule that yields a unique canonical $f_\star$ when:

- multiple f share equal objective score within tolerance,
- or multiple f satisfy a declared canonical equivalence.

Recommended tie-break. Lexicographic minimum of the canonical byte encoding of f 's parameterization (EY22 Appendix A.1).

NumericModel (required)

NumericModel MUST be included as defined in EY22 Chapter 4, including: precision, rounding, tolerance, deterministic aggregation, truncation schedule, and failure modes.

Cert (required)

Cert MUST specify:

- the certificate payload required to verify $f_\star$,
- the verifier checks (structural, constraints, canonicity, invariants),
- and the failure codes used for rejection.

Interop (**required**)

Interop MUST specify:

- allowed composition forms (EY22 C1–C5),
- invariants that must be preserved under composition,
- compatibility predicates for advertised pairings (e.g. with EY24 validity penalty or EY29 coarse-grain).

Version (**required**)

Version MUST uniquely bind the meaning of the above components. Any change in: $\text{Calc}, \mathcal{A}, \mathcal{J}, \mathcal{C}, \text{TieBreak}, \text{Num}$ MUST change Version.

2.6 Deterministic Outcomes

EY23 verification MUST return one of:

$$\{\text{ACCEPT}, \text{REJECT}\} \cup \{\text{FAIL_}\ast\},$$

as defined in EY22. In particular, EY23 MUST support (at minimum):

- **FAIL__NONCONVERGENCE**
- **FAIL__NONDETERMINISM**
- **FAIL__CONSTRAINT**
- **FAIL__CANONICALITY**
- **FAIL__INVARIANT**

2.7 Chapter Summary

This chapter defined $F_\star(H)$ as a canonical selection of $f(H)$ bound by SpecID, and fixed the required SpecID components for EY23: calculus, admissible family, objective, constraints, tie-break, numeric model, certificate semantics, and interoperability declarations.

Chapter 3 now specifies the admissible family $\mathcal{A}$ in concrete form and introduces a baseline profile intended for Vol. No. IV (a filter family designed to minimize hidden degrees of freedom while remaining easy to certify).

3 Admissible Family $\mathcal{A}$: Canonical Filter Classes

3.1 Purpose of $\mathcal{A}$

The admissible family $\mathcal{A}$ is the set of candidate filters f from which the canonical filter $f_\star$ is selected. In EY23, $\mathcal{A}$ is not an implementation detail: it is a normative object and a primary defense against parameter smuggling.

Rule. A filter f MUST be considered admissible only if:

- it belongs to a declared filter class,
- it has a canonical parameterization,
- its parameter bounds are explicit,
- and membership can be checked deterministically under the NumericModel.

3.2 Spectral Window Normalization

Most filter families require a declared spectral window in which the filter is interpreted.

Window. EY23 binds a window map:

$$W : \sigma(H) \rightarrow [0, 1],$$

used to normalize the spectral coordinate. In practical truncation settings, W is defined from truncation-level spectral summaries (e.g. extremal eigenvalues) under a deterministic rule.

Rule (window is part of $\mathcal{A}$). The window map W MUST be specified as part of $\mathcal{A}$ (or Calc) and MUST be reproducible under NumericModel.

Default window (Baseline CSF-0). Unless overridden, the baseline window uses truncation-level extrema:

$$W_N(\lambda) := \text{clip}\left(\frac{\lambda - \lambda_{\min}(H_N)}{\lambda_{\max}(H_N) - \lambda_{\min}(H_N)}, 0, 1\right),$$

with deterministic handling of the degenerate case $\lambda_{\max} = \lambda_{\min}$ (return **FAIL_CONSTRAINT**).

3.3 Canonical Representation Requirement

Rule (canonical encoding). Every $f \in \mathcal{A}$ MUST have a canonical representation that can be serialized and compared lexicographically (EY22 Appendix A.1). This canonical encoding MUST be used by the tie-break rule.

3.4 Admissible Filter Classes

EY23 defines three admissible filter classes. A SpecID MUST select exactly one of them as the active $\mathcal{A}$, or define an explicit union with deterministic tie-break across classes.

Class A1: Piecewise-Polynomial (Spline) Filters (Recommended Baseline)

Definition. A1 consists of piecewise-polynomial filters on the normalized coordinate $x \in [0, 1]$:

$$f(x) = \sum_{j=1}^m p_j(x) \mathbf{1}_{[t_{j-1}, t_j)}(x),$$

where $0 = t_0 < t_1 < \dots < t_m = 1$ are knots and each p_j is a polynomial of degree $\leq d$.

Canonical parameters. A1 is specified by the tuple:

$$\theta_{A1} := (m, d, \{t_j\}_{j=0}^m, \{c_{j,k}\}_{j=1..m, k=0..d}),$$

where $p_j(x) = \sum_{k=0}^d c_{j,k} x^k$.

Admissibility constraints (structural). A filter is admissible in A1 only if:

- knots are strictly increasing and represented as rationals (or fixed-point) under NumericModel,
- coefficients $c_{j,k}$ are represented in the declared NumberSystem,
- optional continuity constraints at knots are either enforced or explicitly not enforced (must be declared).

Canonical encoding. The canonical encoding for A1 MUST serialize:

$$(m, d) \parallel (t_0, \dots, t_m) \parallel (c_{1,0}, \dots, c_{1,d}, \dots, c_{m,0}, \dots, c_{m,d}),$$

with all numbers encoded using NumericModel rules and with no redundant trailing zeros.

Why A1 is recommended. A1 avoids transcendental functions, admits deterministic evaluation, and provides a compact canonical encoding.

Class A2: Rational Resolvent Families

Definition. A2 consists of rational filters:

$$f(\lambda) = \alpha_0 + \sum_{k=1}^K \frac{\alpha_k}{(\lambda - \beta_k)^2 + \gamma_k^2},$$

defined on the (possibly normalized) spectral coordinate.

Canonical parameters.

$$\theta_{A2} := (K, \alpha_0, \{(\alpha_k, \beta_k, \gamma_k)\}_{k=1}^K).$$

Admissibility constraints.

- K is bounded by an explicit maximum $K_{\max}$.
- γ_k MUST be bounded below by $\gamma_{\min} > 0$ to prevent singularities.
- Parameters MUST be represented in the declared NumberSystem and tolerance model.

Canonical ordering. Triples $(\alpha_k, \beta_k, \gamma_k)$ MUST be sorted lexicographically by $(\beta_k, \gamma_k, \alpha_k)$ in canonical encoding.

Note. A2 is powerful but more numerically sensitive; its use SHOULD be paired with a stricter NumericModel (e.g. verified intervals).

Class A3: Finite Orthogonal Expansions

Definition. A3 consists of finite expansions in a declared orthogonal basis $\{\phi_k\}_{k=0}^n$ on $[0, 1]$:

$$f(x) = \sum_{k=0}^n a_k \phi_k(x),$$

where the basis may be Chebyshev, Legendre, or another declared family.

Canonical parameters.

$$\theta_{A3} := (\text{BasisID}, n, (a_0, \dots, a_n)).$$

Admissibility constraints.

- BasisID MUST resolve to a normative basis definition.
- n MUST be bounded by $n_{\max}$ to avoid hidden degrees of freedom.
- coefficients MUST be in the declared NumberSystem and bounded by explicit norms if required.

Canonical encoding. The encoding MUST serialize BasisID, then n , then $a_0, \dots, a_n$ in order, using NumericModel number encoding.

3.5 Parameter Bounds (Anti-Smuggling Requirement)

Rule. Any admissible family MUST include explicit parameter bounds sufficient to prevent unbounded tuning. At minimum:

- a complexity bound (e.g. $m \leq m_{\max}$, $K \leq K_{\max}$, $n \leq n_{\max}$),
- a magnitude bound on coefficients (or a norm bound),
- and any smoothing / regularity bounds required by constraints.

If parameter bounds are omitted, the artifact MUST be treated as non-canonical.

3.6 Baseline Profile CSF-0 (Normative Default)

EY23 defines a baseline admissible family intended for Vol. No. IV defaults.

CSF-0. Unless the SpecID overrides $\mathcal{A}$, the baseline family is:

$$\mathcal{A}_{\text{CSF-0}} := \text{A1 (Spline)} \quad \text{with fixed } (m_{\max}, d_{\max}) \text{ and rational knots.}$$

Baseline constraints. The baseline profile MUST fix:

- $m_{\max}$ and $d_{\max}$,
- a rational/fixed-point knot grid policy (e.g. knots drawn from a declared finite grid),
- coefficient bounds (e.g. $|c_{j,k}| \leq B$),
- and continuity policy (e.g. C^0 continuity at knots enforced or not).

Rule (baseline must be fully specified). CSF-0 MUST be fully specified inside the SpecID as concrete numeric values or as references committed by hash.

3.7 Membership Check Procedure

For any chosen admissible family, the certificate MUST include a membership check routine:

- parse the canonical encoding,
- verify bounds and ordering constraints,
- verify representation constraints (number system, precision),
- return deterministic failure if violated.

3.8 Chapter Summary

This chapter defined the admissible family $\mathcal{A}$ as a normative object and introduced three canonical filter classes (A1 spline, A2 rational, A3 orthogonal expansion). It also defined an anti-smuggling requirement via explicit parameter bounds and established a baseline profile CSF-0 using the A1 spline class.

Chapter 4 now defines the objective functional $\mathcal{J}(f; H)$: how the canonical filter is selected from $\mathcal{A}$ under deterministic evaluation and tie-breaking.

4 Objective Functional $\mathcal{J}$: Canonical Selection Principle

4.1 Purpose of $\mathcal{J}$

The admissible family $\mathcal{A}$ (Chapter 3) provides candidate filters f . The objective $\mathcal{J}(f; H)$ provides the *selection principle* that turns candidates into a canonical choice $f_\star$. In EY23, $\mathcal{J}$ MUST be:

- explicit,
- deterministically evaluable under the NumericModel,
- and sufficiently strong to prevent parameter smuggling within $\mathcal{A}$.

Rule. If $\mathcal{J}$ is under-specified (e.g. relies on informal smoothing goals), the artifact MUST be treated as non-canonical.

4.2 Canonical Score Decomposition

EY23 standardizes $\mathcal{J}$ as a weighted sum of named score terms:

$$\mathcal{J}(f; H) := w_{\text{stab}} \mathcal{J}_{\text{stab}}(f; H) + w_{\text{sharp}} \mathcal{J}_{\text{sharp}}(f; H) + w_{\text{simp}} \mathcal{J}_{\text{simp}}(f) + w_{\text{bias}} \mathcal{J}_{\text{bias}}(f; H),$$

with all weights $w_\bullet \geq 0$ declared in SpecID.

Rule (deterministic weights). Weights MUST be explicit numeric values (or committed by hash) and MUST NOT be tuned per instance unless the tuning rule is part of SpecID.

4.3 $\mathcal{J}_{\text{stab}}$: Truncation Stability Term

A canonical spectral filter is expected to be stable under the declared truncation schedule.

Truncation schedule. Let $N_1 < N_2 < \dots < N_L$ be the truncation levels required by the NumericModel.

Stability metric. Define a stability discrepancy between consecutive truncations:

$$\Delta_\ell(f; H) := D\left(f(H_{N_\ell}), f(H_{N_{\ell+1}})\right),$$

where $D(\cdot, \cdot)$ is a declared deterministic discrepancy functional.

Default discrepancy (Baseline CSF-0). Unless overridden, the baseline discrepancy is a normalized Frobenius proxy:

$$D(A, B) := \frac{\|A - B\|_F}{\max(1, \|A\|_F, \|B\|_F)},$$

with $\|\cdot\|_F$ computed deterministically under NumericModel and with deterministic aggregation order.

Stability score.

$$\mathcal{J}_{\text{stab}}(f; H) := \sum_{\ell=1}^{L-1} \omega_\ell \Delta_\ell(f; H),$$

where $\omega_\ell \geq 0$ are declared weights (default: uniform).

Rule. If any Δ_ℓ cannot be evaluated deterministically (domain mismatch, missing objects, non-determinism), the artifact MUST fail (**FAIL_NONDETERMINISM** or **FAIL_DOMAIN_MISMATCH**).

4.4 $\mathcal{J}_{\text{sharp}}$: Spectral Resolution / Selectivity Term

A filter intended for regime classification should not be trivially flat; it should exhibit controlled selectivity.

Selectivity principle. Let μ_N be a deterministic spectral summary measure for H_N (e.g. empirical eigenvalue distribution). Define a scalar “response” statistic:

$$S_N(f; H) := \Phi(f; \mu_N),$$

where Φ is a deterministic functional of f and the spectral summary.

Baseline selectivity functional (CSF-0). For normalized coordinate $x \in [0, 1]$, let μ_N be the empirical measure over normalized eigenvalues x_i . Define:

$$\Phi(f; \mu_N) := \text{Var}_{x \sim \mu_N}(f(x)),$$

computed deterministically with canonical ordering.

Sharpness score. EY23 defines sharpness as *maximize response variance under constraints*. As a minimization term:

$$\mathcal{J}_{\text{sharp}}(f; H) := -\frac{1}{L} \sum_{\ell=1}^L \Phi(f; \mu_{N_\ell}).$$

Rule (no free sharpness). Sharpness MAY be used only in combination with stability and simplicity terms; otherwise it encourages overfitting via oscillatory filters.

4.5 $\mathcal{J}_{\text{simp}}$: Simplicity / Complexity Term

To prevent parameter smuggling, the objective MUST penalize excessive complexity within $\mathcal{A}$.

Canonical complexity measure. The simplicity score is defined by the active admissible class:

- A1 (spline): $\mathcal{J}_{\text{simp}} := \alpha_m m + \alpha_d d + \alpha_c \|\mathbf{c}\|_1$,
- A2 (rational): $\mathcal{J}_{\text{simp}} := \alpha_K K + \alpha_p \|\boldsymbol{\alpha}\|_1 + \alpha_b \|\boldsymbol{\beta}\|_1$,
- A3 (expansion): $\mathcal{J}_{\text{simp}} := \alpha_n n + \alpha_a \|\mathbf{a}\|_2^2$.

All $\alpha_\bullet \geq 0$ MUST be declared in SpecID.

Rule. The complexity measure MUST be computable directly from the canonical encoding of f , so that it is implementation-independent.

4.6 $\mathcal{J}_{\text{bias}}$: Canonical Neutrality Term (Optional)

In some workflows, filters should avoid encoding external preferences (e.g. avoid privileging high-energy vs low-energy). EY23 allows an optional neutrality/bias term to enforce symmetric behavior.

Example (optional symmetry penalty). Under normalized coordinate $x \in [0, 1]$, define:

$$\mathcal{J}_{\text{bias}}(f) := \int_0^1 |f(x) - f(1-x)| dx,$$

approximated deterministically on a declared grid. If used, the grid and approximation rule MUST be part of NumericModel.

Rule. If $\mathcal{J}_{\text{bias}}$ is used, it MUST be declared; if it is not used, it MUST be set to $\emptyset$ in SpecID.

4.7 Baseline Objective Profile (CSF-0)

EY23 defines a baseline objective profile intended for Vol. No. IV defaults.

CSF-0 Objective. Unless overridden, the baseline objective is:

$$\mathcal{J}_{\text{CSF-0}} := w_{\text{stab}} \mathcal{J}_{\text{stab}} + w_{\text{sharp}} \mathcal{J}_{\text{sharp}} + w_{\text{simp}} \mathcal{J}_{\text{simp}},$$

with $w_{\text{bias}} = 0$.

Baseline ordering of priorities. The baseline weights SHOULD satisfy:

$$w_{\text{stab}} \gg w_{\text{sharp}} \geq w_{\text{simp}} > 0,$$

so that stability dominates and sharpness cannot override stability. Exact numeric values MUST be fixed in SpecID.

4.8 Deterministic Evaluation of $\mathcal{J}$

Rule (evaluation protocol). The certificate MUST specify how $\mathcal{J}$ is evaluated:

- which truncation levels N_ℓ are used,
- how eigenvalues/spectral summaries are computed (or approximated),
- the deterministic aggregation order across ℓ ,
- and the exact rounding/tolerance used in each scalar statistic.

Rule (objective comparability). If two candidates f_1, f_2 are compared, they MUST be evaluated under identical truncation schedule and NumericModel parameters. If not, the comparison is invalid and MUST fail.

4.9 Chapter Summary

This chapter defined the EY23 objective $\mathcal{J}$ as a deterministic selection principle built from stability, sharpness, and simplicity terms (with optional neutrality). It also defined a baseline objective profile CSF-0 in which stability dominates.

Chapter 5 next defines the constraint predicates $\mathcal{C}(f; H)$: the feasibility conditions that filters MUST satisfy (boundedness, monotonicity/positivity when required, and explicit stability constraints), and the deterministic tie-break rule that guarantees uniqueness under tolerance ties.

5 Constraints $\mathcal{C}$ and TieBreak: Feasibility and Uniqueness

5.1 Purpose of $\mathcal{C}$

The admissible family $\mathcal{A}$ restricts the *form* of filters; the constraint predicates $\mathcal{C}$ restrict the *behavior* of filters. In EY23, $\mathcal{C}$ is normative and serves two roles:

- enforce audit-friendly properties (boundedness, controlled variation, non-pathological behavior),
- define the stability envelope under truncation and perturbation.

Rule. A candidate $f \in \mathcal{A}$ is feasible only if $\mathcal{C}(f; H) = \text{TRUE}$ under the declared NumericModel. If feasibility cannot be decided deterministically, the artifact MUST fail.

5.2 Constraint Vocabulary (Standard Predicates)

EY23 standardizes a minimal set of constraint predicates. A SpecID MUST either: (i) adopt these predicates, or (ii) explicitly override them with a declared predicate set.

C1: Bounded Output Constraint

Definition. For a declared bound $B_{\max} > 0$,

$$\mathcal{C}_{\text{bound}}(f) := \left(\sup_{x \in [0,1]} |f(x)| \leq B_{\max} \right).$$

Deterministic evaluation. For families A1/A3, the supremum MUST be evaluated using a deterministic bound method declared in NumericModel:

- either an analytic bound derived from coefficients,
- or a deterministic grid bound with declared resolution and rounding.

For A2, singularity avoidance via $\gamma_{\min}$ MUST be enforced and the bound MUST be evaluated deterministically.

C2: Controlled Variation (Lipschitz / Total Variation)

Definition (Lipschitz proxy). For a declared $L_{\max} \geq 0$,

$$\mathcal{C}_{\text{lip}}(f) := \left(\text{Lip}(f) \leq L_{\max} \right),$$

where $\text{Lip}(f)$ is evaluated via a declared proxy (analytic for splines, basis bound for expansions, or grid bound).

Alternative (Total Variation). A SpecID MAY use total variation $\text{TV}(f) \leq \text{TV}_{\max}$ instead of Lipschitz. The choice MUST be explicit.

C3: Positivity / Nonnegativity (Optional, Use-Case Dependent)

Some workflows require $f(H)$ to behave like a penalty or projector-like probe.

Definition. For a declared $\delta \geq 0$,

$$\mathcal{C}_{\text{pos}}(f) := \left(f(x) \geq -\delta \text{ for all } x \in [0, 1] \right).$$

Rule. If positivity is required, the tolerance δ MUST be part of NumericModel (tolerance tuple), and feasibility MUST be deterministic.

C4: Normalization (Optional)

To prevent trivial rescaling ambiguity, EY23 allows an optional normalization constraint.

Example normalization.

$$\mathcal{C}_{\text{norm}}(f; \mu) := \left(\mathbb{E}_{x \sim \mu}[f(x)] = 0 \right) \wedge \left(\mathbb{E}_{x \sim \mu}[f(x)^2] = 1 \right),$$

where μ is the declared spectral summary measure (from Chapter 4).

Rule. If normalization is used, the measure μ and the deterministic evaluation procedure MUST be declared in SpecID.

C5: Truncation Stability Constraint

The objective includes a stability term, but EY23 also allows a *hard* stability constraint.

Definition. For declared thresholds τ_ℓ ,

$$\mathcal{C}_{\text{stab}}(f; H) := \bigwedge_{\ell=1}^{L-1} \left(\Delta_\ell(f; H) \leq \tau_\ell \right),$$

where Δ_ℓ is the stability discrepancy from Chapter 4.

Rule. If $\mathcal{C}_{\text{stab}}$ is enabled, the truncation schedule $N_1, \dots, N_L$ and thresholds τ_ℓ MUST be part of SpecID.

C6: Anti-Oscillation / Smoothness Constraint (Recommended for A1/A3)

To prevent adversarial oscillations that fit objectives while hiding intent, EY23 allows an explicit smoothness control.

Example (second-derivative energy bound). For A1/A3 with differentiable f :

$$\mathcal{C}_{\text{smooth}}(f) := \left(\int_0^1 |f''(x)|^2 dx \leq S_{\text{max}} \right),$$

evaluated deterministically by analytic formula (preferred) or declared grid quadrature (allowed).

5.3 Baseline Constraint Profile (CSF-0)

EY23 defines a baseline constraint profile intended for Vol. No. IV defaults.

CSF-0 Constraints. Unless overridden, the baseline enables:

$$\mathcal{C}_{\text{CSF-0}} := \mathcal{C}_{\text{bound}} \wedge \mathcal{C}_{\text{lip}} \wedge \mathcal{C}_{\text{stab}},$$

with $\mathcal{C}_{\text{pos}}$ and $\mathcal{C}_{\text{norm}}$ disabled by default (set to $\emptyset$ in SpecID).

Rule. Baseline numeric constants $B_{\text{max}}, L_{\text{max}}, \{\tau_\ell\}$ MUST be fixed in SpecID.

5.4 TieBreak: Deterministic Uniqueness

Even with explicit objectives and constraints, multiple candidates may tie under tolerance. EY23 therefore requires a deterministic tie-break.

Definition 5.1 (TieBreak Rule). Let $\mathcal{F} \subseteq \mathcal{A}$ be the set of feasible candidates achieving the optimal objective value within the declared tolerance. The tie-break selects:

$$f_\star := \min_{\text{lex}} \text{Enc}(f),$$

where $\text{Enc}(f)$ is the canonical byte encoding of f (EY22 Appendix A.1) and $\min_{\text{lex}}$ denotes lexicographic minimum.

Rule (byte-level determinism). TieBreak MUST operate on canonical encodings, not on floating-point approximations of objective values.

5.5 Equivalence Classes and Canonical Representatives (Optional)

In some admissible families, different parameterizations may represent the same function f .

Rule. If a family admits representation non-uniqueness, the SpecID MUST specify either:

- a canonical reparameterization map $\theta \mapsto \theta^{\text{canon}}$, or
- an explicit equivalence relation and a canonical representative selection rule.

Otherwise, the artifact MUST be rejected as non-canonical.

5.6 Feasibility Failure Modes

If no feasible candidate exists:

$$\{f \in \mathcal{A} : \mathcal{C}(f; H) = \text{TRUE}\} = \emptyset,$$

the artifact MUST return **FAIL_CONSTRAINT**.

If feasibility cannot be checked deterministically (e.g. nondeterministic eigenvalue summaries), the artifact MUST return **FAIL_NONDETERMINISM**.

5.7 Chapter Summary

This chapter defined the constraint predicates $\mathcal{C}(f; H)$ as the feasibility contract for EY23 and standardized a minimal constraint vocabulary: boundedness, controlled variation, optional positivity and normalization, truncation stability, and optional smoothness. It then defined TieBreak as lexicographic minimization over canonical encodings to guarantee unique, deterministic selection under tolerance ties.

Chapter 6 next specifies the certificate payload and verifier checks for EY23: what must be published to prove membership, feasibility, canonicity, and invariants of $F_\star(H)$ under the declared SpecID and NumericModel.

6 Certificate and Verifier: What Must Be Published

6.1 Purpose

EY23 defines $F_\star(H) = f_\star(H)$ as a canonical selection. Canonicity is not a claim; it is a *verifiable status*. This chapter specifies the certificate payload π_{CSF} and the verifier checks that deterministically accept or reject an EY23 artifact instance.

EY23 certificates conform to the EY22 certificate schema (EY22 Chapter 5, Appendix A.1). EY23 adds only artifact-specific payload fields and checks.

6.2 Certificate Object π_{CSF}

Rule. An EY23 instance MUST be published with a certificate π_{CSF} that includes, at minimum, the following content in its canonical encoding:

Header (required)

- artifact symbol: $F_\star(H)$
- artifact type: CSF
- SpecID short form and full SpecID tuple
- version identifier

SpecID (required)

The full SpecID tuple MUST be embedded, including:

(CSF, (H) , Calc, $\mathcal{A}$, $\mathcal{J}$, $\mathcal{C}$, TieBreak, NumericModel, Cert, Interop, Version).

Inputs (required)

Inputs MUST include:

- a canonical public description of H (or hash commitment to it),
- truncation schedule $N_1 < \dots < N_L$ (if truncation calculus is used),
- any spectral window map W rules (or commitment),
- all numeric constants used by $\mathcal{A}$, $\mathcal{J}$, $\mathcal{C}$ (bounds, thresholds, weights),
- the deterministic ordering / aggregation rules used for objective evaluation.

Outputs (required)

Outputs MUST include:

- the canonical representation of $f_\star$ (canonical encoding of parameters),
- the declared filter class identifier (A1/A2/A3 or union),
- optional derived invariants (if the paper claims them),
- output hashes of $f_\star$ encoding and any large derived objects.

Checks (required)

The certificate MUST list check IDs and what each check verifies, grouped as:

- structural checks,
- admissibility and constraint checks,
- canonicity checks,
- invariants checks.

Commitments (required if external objects exist)

All external objects MUST be committed by hash according to EY22 Appendix A.1.

6.3 Verifier Contract

A verifier takes (π_{CSF}) and deterministically returns one of:

$$\{\text{ACCEPT}, \text{REJECT}\} \cup \{\text{FAIL_}\ast\}.$$

The verifier **MUST NOT** require trust in producer identity, runtime logs, or non-committed external resources.

6.4 Required Verifier Checks (Normative)

This section enumerates the minimal required checks for EY23.

V1: Structural Validity (CCE)

- parse canonical JSON projection (EY22 Appendix A.1),
- reject duplicate keys / non-canonical numbers / non-UTF8 encodings,
- confirm required sections exist.

Failure code: **REJECT_MALFORMED**.

V2: SpecID Completeness and Consistency

- SpecID tuple is present and complete,
- ArtifactType = CSF,
- version is present,
- NumericModel is present and complete,
- no conflict between SpecID and declared outputs (e.g. filter class mismatch).

Failure codes: **REJECT_SPECID_MISSING**, **REJECT_SPECID_MISMATCH**.

V3: Input Completeness / No Hidden Inputs

- H is provided or committed by hash,
- all constants (bounds/weights/thresholds) are provided,
- truncation schedule and window map rules are provided,
- any external objects referenced have commitments.

Failure code: **REJECT_HIDDEN_INPUT** or **FAIL_HASH_MISMATCH**.

V4: Admissibility Membership Check

- parse $f_\star$ canonical encoding,
- verify it belongs to the declared admissible class $\mathcal{A}$,
- verify parameter bounds (complexity bound, coefficient bounds, knot rules),

- verify canonical ordering requirements (if A2, lex ordering of triples).

Failure code: **FAIL_CONSTRAINT** (or **REJECT_MALFORMED** if encoding invalid).

V5: Constraint Feasibility Check

Verify $\mathcal{C}(f_\star; H) = \text{TRUE}$ under NumericModel, including:

- bounded output constraint,
- controlled variation (Lipschitz/TV) constraint,
- truncation stability constraint if enabled,
- any optional positivity/normalization/smoothness constraints if enabled.

Failure code: **FAIL_CONSTRAINT**.

V6: Canonicity Check (Optimality + TieBreak)

The verifier MUST confirm that the published $f_\star$ is the canonical selection. EY23 permits two modes:

Mode A (Exhaustive, finite $\mathcal{A}$). If $\mathcal{A}$ is finite under the SpecID (e.g. knots from a finite grid and coefficients from a finite set), the verifier MUST:

- enumerate all feasible candidates,
- compute $\mathcal{J}(f; H)$ deterministically,
- select the optimum set within tolerance,
- apply TieBreak on canonical encodings,
- confirm the published $f_\star$ matches.

Mode B (Certified search, infinite $\mathcal{A}$). If $\mathcal{A}$ is infinite, the paper MUST define a *certificate of optimality* or a *canonical search protocol* whose transcript is checkable. In that case the verifier MUST:

- verify the protocol transcript (deterministic steps, deterministic stopping),
- verify that no admissible improvement exists within declared bounds *under the declared protocol class*.

Rule. The SpecID MUST state which canonicity mode is used. If Mode B is used, the protocol transcript MUST be part of the certificate. Failure code: **FAIL_CANONICALITY** or **FAIL_NONDETERMINISM** (if transcript is not replayable).

V7: Invariant Checks

If EY23 claims invariants (e.g. monotonicity, boundedness, stability envelope), the verifier MUST check each declared invariant under declared tolerances. Failure code: **FAIL_INVARIANT**.

6.5 Certificate Extensions (Optional but Recommended)

An EY23 certificate MAY include:

- a list of top- k competing candidates and their scores (informational only),
- additional stability diagnostics across N ,
- interop certificate links if the filter is published as part of a composed workflow.

These MUST NOT affect acceptance.

6.6 Chapter Summary

This chapter defined the EY23 certificate payload and the deterministic verifier checks required to accept an instance of $F_\star(H)$. It introduced two canonicity verification modes (finite exhaustive vs certified search), both bound by SpecID, and fixed failure codes for structural, feasibility, canonicity, and invariant violations.

Chapter 7 next specifies the evaluation protocol: truncation schedule, spectral window computation, deterministic aggregation, and the minimal transcript required when certified search is used.

7 Protocol: Truncation, Windowing, Deterministic Evaluation, Transcript

7.1 Purpose

The EY23 artifact $F_\star(H) = f_\star(H)$ is defined abstractly, but its canonical selection depends on deterministic evaluation of $\mathcal{J}$ and $\mathcal{C}$ under a declared NumericModel. This chapter fixes the **protocol layer**: how truncation is performed, how spectral windowing is computed, how scores are aggregated, and what transcript must be published to support canonicity verification.

Rule. Any protocol decision that can change the selected $f_\star$ MUST be included in SpecID (typically inside Calc, NumericModel, or Cert). Undeclared protocol changes invalidate the certificate.

7.2 Truncation Protocol

Truncation Mapping $H \mapsto H_N$

EY23 permits truncation calculus where $f(H)$ is evaluated via $f(H_N)$ on finite truncations.

Rule (deterministic truncation). The truncation mapping MUST be deterministic and MUST be fully specified by:

Truncation := (Basis, CutRule, BoundaryRule, ScaleSchedule, CrossNPolicy),

as defined in EY22 Chapter 4.

Required content. The certificate **MUST** include (or commit by hash):

- the basis or representation in which truncation is performed,
- the exact cut rule producing dimension N ,
- the boundary rule (padding / finite-rank correction / imposed boundary semantics),
- the required scale schedule $N_1 < \dots < N_L$,
- the cross- N comparison policy (what must stabilize, and how).

Scale Schedule

Rule. The schedule $N_1, \dots, N_L$ **MUST** be part of NumericModel. A certificate **MUST NOT** choose N adaptively unless the adaptive rule is specified in SpecID and is deterministic.

Baseline schedule (CSF-0). A baseline schedule **MAY** be of the form:

$$N_\ell = N_{\min} \cdot 2^{\ell-1}, \quad \ell = 1, \dots, L,$$

with explicit constants $(N_{\min}, L)$ fixed in SpecID.

7.3 Spectral Windowing Protocol

Many admissible families operate on a normalized coordinate $x \in [0, 1]$. Windowing is therefore part of the protocol.

Window Inputs

At truncation level N , define deterministic extremal estimates:

$$\lambda_{\min}(H_N), \quad \lambda_{\max}(H_N).$$

Rule. The method used to compute these (exact eigenvalues, certified bounds, Gershgorin bounds, Lanczos with fixed iteration count, etc.) **MUST** be declared. If a numerical eigensolver is used, it **MUST** be deterministic under NumericModel (fixed seed, fixed ordering, fixed stopping).

Window Map

Define the normalized coordinate:

$$x := W_N(\lambda) := \text{clip}\left(\frac{\lambda - \lambda_{\min}(H_N)}{\lambda_{\max}(H_N) - \lambda_{\min}(H_N)}, 0, 1\right).$$

Degenerate case. If $\lambda_{\max}(H_N) = \lambda_{\min}(H_N)$, the protocol **MUST** return **FAIL_CONSTRAINT** (unless SpecID defines a deterministic fallback).

Cross- N window coherence. If objectives or constraints compare values across N , the SpecID **MUST** declare whether:

- windows are computed independently per N (default),
- or a shared window is enforced (e.g. from the largest N_L).

7.4 Deterministic Evaluation of $\mathcal{J}$ and $\mathcal{C}$

Canonical Evaluation Order

Rule (fixed ordering). All scalar statistics used in $\mathcal{J}$ and $\mathcal{C}$ MUST be computed in a fixed order:

1. for $\ell = 1$ to L : build H_{N_ℓ} ,
2. compute window primitives $\lambda_{\min}, \lambda_{\max}$,
3. compute required spectral summaries μ_{N_ℓ} ,
4. evaluate constraint predicates that depend on μ_{N_ℓ} ,
5. evaluate objective terms for μ_{N_ℓ} ,
6. append per- ℓ scalars to an ordered transcript list.

Rule (aggregation). Aggregation over ℓ MUST use deterministic reduction rules (EY22 Chapter 4): lexicographic ordering of indices and a fixed reduction tree (or explicit sequential sum).

Spectral Summary Measure μ_N

The objective and constraints may depend on a spectral summary μ_N .

Allowed summary forms.

- explicit eigenvalue list (sorted) if computable,
- histogram/bucket measure over eigenvalues with declared binning,
- trace moments $\text{tr}(H_N^k)$ for declared k ,
- any other deterministic summary committed in SpecID.

Rule. If a histogram/binning is used, bin edges MUST be deterministic and MUST be part of NumericModel (or derived deterministically from window rules).

Deterministic Norms and Proxies

Where norms appear (e.g. Frobenius proxy), the protocol MUST specify:

- exact definition of the norm,
- rounding policy,
- deterministic ordering of summation.

7.5 Candidate Generation Protocol

Canonical selection requires comparing candidates f . EY23 permits two canonicity modes (Chapter 6).

Mode A: Finite Enumeration

If the admissible family is finite, candidate generation is deterministic enumeration.

Rule. The SpecID MUST define:

- the finite parameter grid (knot grid, coefficient set),
- the canonical iteration order over the grid,
- and an early-rejection rule (optional) that is deterministic and does not change the argmin set.

Mode B: Certified Search Protocol

If the admissible family is infinite, EY23 requires a canonical search protocol with a verifiable transcript.

Rule (protocol class). The SpecID MUST declare the protocol class, for example:

- deterministic coordinate descent on canonical parameter order,
- deterministic dynamic programming over a finite knot grid with continuous coefficients,
- deterministic convex optimization with a declared stopping certificate (dual gap bound),
- or another verifiable search class.

Stop condition (required). The stop condition MUST be a deterministic predicate expressed in NumericModel terms, such as:

- objective improvement $< \varepsilon_{\text{stop}}$ for T consecutive steps,
- certified duality gap $< \varepsilon_{\text{stop}}$,
- or a fixed iteration count T_{max} .

7.6 Transcript Requirements (Mode B)

When Mode B is used, the certificate MUST include a **transcript** sufficient for replay.

Transcript Contents

At minimum, the transcript MUST include:

- the ordered list of candidate encodings evaluated (or the deterministic rule to reproduce them),
- for each candidate: objective value $\mathcal{J}$, feasibility outcome of $\mathcal{C}$,
- any intermediate scalars needed to reproduce $\mathcal{J}$ terms (per- N summaries),
- the stop condition evaluation (why the procedure stopped),
- the final tie-break set and the selected $f_{\star}$.

Rule (replay determinism). If replay of the transcript under `NumericModel` does not reproduce the same selected $f_\star$, the verifier MUST return **FAIL_NONDETERMINISM**.

Transcript Hashing

Rule. The transcript MUST be committed by hash inside the certificate:

$$\text{Hash}(\text{TranscriptBytes}),$$

with `TranscriptBytes` defined by canonical serialization rules (EY22 Appendix A.1).

7.7 Protocol Failure Modes

The following protocol failures MUST be supported:

- **FAIL_NONCONVERGENCE:** stop condition not met and $T_{\max}$ exceeded, or cross- N stability does not converge.
- **FAIL_NONDETERMINISM:** replay mismatch of any transcript scalar or selection result.
- **FAIL_CONSTRAINT:** degenerate window, infeasible family, or constraint violation of selected $f_\star$.
- **FAIL_CANONICALITY:** published $f_\star$ is not the canonical selection under Mode A or Mode B semantics.

7.8 Chapter Summary

This chapter fixed the EY23 protocol layer: deterministic truncation mapping, scale schedule, spectral window computation, deterministic evaluation order for objectives and constraints, and candidate generation protocols. It also specified transcript requirements for certified search (Mode B), including canonical serialization and hash commitments.

Next. The remaining sections of EY23 are typically (i) a short interoperability declaration for key pairings (EY24/EY29), (ii) a compact “Reference SpecID Profile” table (CSF-0), and (iii) a conclusion. These can be included as Chapter 8 and Chapter 9 if you want EY23 to mirror your longer QFC whitepaper structure.

8 Interop Declarations (Mini): Compatibility with EY24/EY25/EY29

8.1 Purpose

This chapter instantiates the EY22 interoperability framework (Chapter 6) for the EY23 artifact $F_\star(H)$. It provides *minimal*, *concrete* compatibility predicates for the most common pairings in Vol. No. IV:

- EY23 (CSF) $\times$ EY24 (VPO) — validity penalty operator $V = C^\dagger C$,
- EY23 (CSF) $\times$ EY25 (LIOUV) — audit-native evolution $\mathcal{L}_{\text{audit}}$,
- EY23 (CSF) $\times$ EY29 (CGR) — coarse-grain reduction R .

Rule. Any claim of interoperability beyond what is defined here MUST introduce additional compatibility predicates and be encoded in the SpecID Interop component.

8.2 Interop Component (EY23)

EY23 declares the following allowed composition forms (EY22 C1–C5):

$$\text{Interop}_{\text{EY23}} := \{C1, C3, C4\},$$

i.e. functional composition, coarse-grain reduction, and superoperator action (when applicable). Transport-by-morphism (C2) and token concatenation (C5) are not declared as baseline pairings for EY23.

8.3 Compatibility Predicates

All compatibility predicates below are deterministic booleans under the declared NumericModel.

CSF→VPO: Filtering with Validity Penalty V

EY24 introduces $V = C^\dagger C$. The two natural compositions are:

$$F_\star(H) \text{ with } V \text{ via } F_\star(H)V, \quad V F_\star(H), \quad F_\star(H + V), \quad F_\star(H) + V.$$

EY23 does not prescribe which of these is semantically intended; the *workflow* must declare the chosen composition. Interop defines what is permitted.

Predicate $\text{Compat}_{23,24}^{(1)}$ (same-space requirement).

$$\text{Compat}_{23,24}^{(1)} := \left(\mathcal{H}_H = \mathcal{H}_V \right) \wedge \left(\text{Dom}(H) \cap \text{Dom}(V) \text{ is declared compatible} \right).$$

Predicate $\text{Compat}_{23,24}^{(2)}$ (calculus compatibility).

$$\text{Compat}_{23,24}^{(2)} := \left(\text{Calc}_{\text{EY23}} \text{ supports the declared composition form} \right).$$

In particular:

- If the workflow uses $F_\star(H + V)$, the calculus MUST support evaluating $f(H + V)$ under truncation policy.
- If the workflow uses products like $F_\star(H)V$, the domain and truncation must define the product deterministically.

Predicate $\text{Compat}_{23,24}^{(3)}$ (numeric model alignment).

$$\text{Compat}_{23,24}^{(3)} := \left(\text{NumericModel}_{\text{EY23}} = \text{NumericModel}_{\text{EY24}} \right) \vee \left(\text{CoercionRule is declared and deterministic} \right).$$

Compatibility status. Define:

$$\text{Compat}(\text{EY23}, \text{EY24}) := \text{Compat}_{23,24}^{(1)} \wedge \text{Compat}_{23,24}^{(2)} \wedge \text{Compat}_{23,24}^{(3)}.$$

If TRUE, the matrix entry MAY be marked OK* because the workflow must still declare the intended composition semantics (e.g. additive vs multiplicative integration of V).

CSF→LIOUV: Filtering with Audit-Native Evolution $\mathcal{L}_{\text{audit}}$

EY25 introduces an audit-native evolution operator / superoperator $\mathcal{L}_{\text{audit}}$. Two common semantics are:

(i) Pre-filtering: $\rho \mapsto \mathcal{L}_{\text{audit}}(F_{\star}(H)\rho F_{\star}(H)^{\dagger})$, (ii) Post-filtering: $\rho \mapsto F_{\star}(H) \mathcal{L}_{\text{audit}}(\rho) F_{\star}(H)^{\dagger}$,

or observable-side analogs. EY23 does not fix the semantic; interop fixes the compatibility requirements.

Predicate $\text{Compat}_{23,25}^{(1)}$ (action space match).

$$\text{Compat}_{23,25}^{(1)} := \left(\mathcal{L}_{\text{audit}} \text{ acts on the same state/observable space as } F_{\star}(H) \right).$$

Predicate $\text{Compat}_{23,25}^{(2)}$ (superoperator determinism).

$$\text{Compat}_{23,25}^{(2)} := \left(\mathcal{L}_{\text{audit}} \text{ is deterministic under NumericModel and transcript rules} \right).$$

Predicate $\text{Compat}_{23,25}^{(3)}$ (ordering declaration).

$$\text{Compat}_{23,25}^{(3)} := \left(\text{the workflow declares a canonical ordering: pre vs post} \right).$$

If ordering is not declared, verification MUST fail, because different orderings are not equivalent in general.

Compatibility status.

$$\text{Compat}(\text{EY23}, \text{EY25}) := \text{Compat}_{23,25}^{(1)} \wedge \text{Compat}_{23,25}^{(2)} \wedge \text{Compat}_{23,25}^{(3)}.$$

If TRUE, the matrix entry is OK* (ordering-dependent).

CSF→CGR: Filtering Under Coarse-Grain Reduction R

EY29 introduces coarse-grain reduction R . The relevant composition patterns are:

$$R(F_{\star}(H)), \quad F_{\star}(R(H)), \quad R \circ F_{\star} \text{ vs } F_{\star} \circ R.$$

In general $R \circ F_{\star} \neq F_{\star} \circ R$. Interop therefore focuses on declared invariants.

Predicate $\text{Compat}_{23,29}^{(1)}$ (domain/codomain compatibility).

$$\text{Compat}_{23,29}^{(1)} := \left(R \text{ is defined on the representation used for } F_{\star}(H) \right).$$

Predicate $\text{Compat}_{23,29}^{(2)}$ (invariant preservation). Let Inv_{CSF} be the invariant list declared by EY23 for downstream use (e.g. boundedness, stability envelope, bucket-order monotonicity if enabled). Then:

$$\text{Compat}_{23,29}^{(2)} := \left(\forall I \in \text{Inv}_{\text{CSF}}^{\text{preserve}} : I(F_{\star}(H)) \Rightarrow I(R(F_{\star}(H))) \right),$$

within declared tolerances.

Predicate $\text{Compat}_{23,29}^{(3)}$ (window coherence under reduction). If the admissible family uses window normalization, interop MUST declare whether:

- windows are recomputed after reduction, or
- windows are inherited from the pre-reduction object.

Thus:

$$\text{Compat}_{23,29}^{(3)} := \left(\text{a deterministic window policy across } R \text{ is declared} \right).$$

Compatibility status.

$$\text{Compat}(\text{EY23}, \text{EY29}) := \text{Compat}_{23,29}^{(1)} \wedge \text{Compat}_{23,29}^{(2)} \wedge \text{Compat}_{23,29}^{(3)}.$$

If TRUE, the matrix entry is OK if a single declared direction is used (e.g. $R(F_\star(H))$ only), and OK* if both directions are allowed but ordering must be declared per workflow.

8.4 Mini Compatibility Matrix Row (EY23 Perspective)

This is a minimal row view for EY23, where entries are informative and must be backed by predicates above:

	EY23	EY24	EY25	EY26	EY27	EY28	EY29	EY30
EY23	N/A	OK*	OK*	N/A	OK*	N/A	OK*	OK*

Notes (informational).

- EY23×EY27 and EY23×EY30 are marked OK* because their semantics depends on what EY27/EY30 claim as inputs/exports.
- EY23×EY26 and EY23×EY28 are not baseline pairings, but may be enabled by additional interop predicates in those papers.

8.5 Interop Failure Codes (Required)

When an interop predicate fails, the verifier MUST return a deterministic failure code, at minimum:

- **FAIL_DOMAIN_MISMATCH**
- **FAIL_CALC_MISMATCH**
- **FAIL_NUMERICMODEL_MISMATCH**
- **FAIL_INVARIANT_NOT_PRESERVED**
- **FAIL_COMPAT_PREDICATE**

8.6 Chapter Summary

This chapter provided minimal EY23 interoperability declarations, defining concrete compatibility predicates for EY23 with EY24 (validity penalty), EY25 (audit-native evolution), and EY29 (coarse-grain reduction). It also provided a mini compatibility-matrix row and fixed the required failure codes for interoperability failures.

Chapter 9 next defines the baseline CSF-0 reference profile: a concrete SpecID instance with fixed admissible family parameters, objective weights, constraint thresholds, canonicity mode, and encoding rules.

9 CSF-0 Reference Profile and Example SpecID Instance

9.1 Purpose

Chapters 1–8 define the general EY23 artifact. This chapter provides a concrete, audit-friendly baseline configuration called **CSF-0**. CSF-0 is intended to be:

- finite (so canonicity can be verified by enumeration),
- simple (to minimize hidden degrees of freedom),
- deterministic under a lightweight NumericModel,
- and stable enough to serve as the default spectral probe in Vol. No. IV.

Rule. A CSF-0 instance **MUST** be fully specified by explicit constants (or hash-committed constants) and **MUST NOT** require instance-specific tuning.

9.2 CSF-0 Summary (Normative)

CSF-0 fixes:

- admissible class: A1 (spline filters),
- finite parameter grid: knots from a finite rational grid and coefficients from a finite quantized set,
- objective: stability + sharpness + simplicity,
- constraints: boundedness + controlled variation + hard truncation stability,
- canonicity mode: Mode A (finite enumeration),
- tie-break: lexicographic minimum of canonical encodings.

9.3 CSF-0 Parameterization

CSF-0 uses normalized coordinate $x \in [0, 1]$ and piecewise-polynomial filters of bounded complexity.

Admissible Family $\mathcal{A}_{\text{CSF-0}}$

Structure.

$$\mathcal{A}_{\text{CSF-0}} := \{f \mid f \text{ is A1 spline with } m \leq m_{\max}, d \leq d_{\max}, t_j \in \mathcal{G}_t, c_{j,k} \in \mathcal{G}_c\}.$$

Fixed constants (CSF-0). The following **MUST** be fixed in SpecID:

$$m_{\max}, \quad d_{\max}, \quad \mathcal{G}_t, \quad \mathcal{G}_c.$$

Recommended baseline constants (informational). A practical baseline that stays small yet expressive is:

$$m_{\max} = 8, \quad d_{\max} = 3,$$

with a knot grid $\mathcal{G}_t$ consisting of rationals with denominator Q_t (e.g. $Q_t = 64$) and a coefficient grid $\mathcal{G}_c$ consisting of fixed-point values with step Δ_c and bound B :

$$\mathcal{G}_t := \left\{ \frac{q}{Q_t} : q = 0, \dots, Q_t \right\}, \quad \mathcal{G}_c := \{n\Delta_c : n \in \mathbb{Z}, |n\Delta_c| \leq B\}.$$

Normative note: the exact numeric choices MUST be fixed in SpecID; the above are placeholders until you lock them.

Window Map W_N

CSF-0 uses the default window map from Chapter 3:

$$W_N(\lambda) := \text{clip}\left(\frac{\lambda - \lambda_{\min}(H_N)}{\lambda_{\max}(H_N) - \lambda_{\min}(H_N)}, 0, 1\right),$$

with degenerate-case failure.

9.4 CSF-0 Objective $\mathcal{J}_{\text{CSF-0}}$

CSF-0 uses:

$$\mathcal{J}_{\text{CSF-0}} := w_{\text{stab}}\mathcal{J}_{\text{stab}} + w_{\text{sharp}}\mathcal{J}_{\text{sharp}} + w_{\text{simp}}\mathcal{J}_{\text{simp}},$$

with $w_{\text{bias}} = 0$.

Weight policy (normative). Weights MUST be fixed in SpecID. CSF-0 requires stability dominance:

$$w_{\text{stab}} > w_{\text{sharp}} \geq w_{\text{simp}} > 0.$$

Recommended baseline weights (informational). A conservative baseline is:

$$w_{\text{stab}} = 100, \quad w_{\text{sharp}} = 1, \quad w_{\text{simp}} = 0.1.$$

9.5 CSF-0 Constraints $\mathcal{C}_{\text{CSF-0}}$

CSF-0 enables:

$$\mathcal{C}_{\text{CSF-0}} := \mathcal{C}_{\text{bound}} \wedge \mathcal{C}_{\text{lip}} \wedge \mathcal{C}_{\text{stab}}.$$

Threshold policy (normative). All thresholds MUST be fixed in SpecID:

$$B_{\max}, \quad L_{\max}, \quad \{\tau_\ell\}_{\ell=1}^{L-1}.$$

Recommended baseline thresholds (informational). Example placeholders:

$$B_{\max} = 2, \quad L_{\max} = 20, \quad \tau_\ell = 10^{-2} \text{ (uniform)}.$$

(Again: lock these as constants in SpecID; do not tune per instance.)

9.6 CSF-0 Canonicity Mode and TieBreak

Mode. CSF-0 uses Mode A (finite enumeration).

TieBreak. TieBreak is lexicographic minimum on canonical encoding bytes:

$$f_{\star} := \min_{\text{lex}} \text{Enc}(f) \quad \text{over optimal feasible } f.$$

9.7 CSF-0 NumericModel (Minimal Deterministic Profile)

CSF-0 requires a deterministic NumericModel. A minimal profile MUST include:

- NumberSystem: fixed-point or rational for filter parameters; deterministic floating or fixed-point for spectral summaries,
- Deterministic aggregation: fixed ordering and reduction,
- Truncation schedule: $N_1 < \dots < N_L$ fixed in SpecID,
- Tolerances: $\varepsilon_{\text{abs}}, \varepsilon_{\text{rel}}, \varepsilon_{\text{spec}}$ fixed.

Recommended (informational). A pragmatic split:

- filter parameters: fixed-point (exact canonical bytes),
- spectral computations: deterministic float with fixed rounding mode and transcript checks.

9.8 Reference SpecID Instance (CSF-0 Template)

The following is a template for a CSF-0 SpecID instance. Numeric constants are placeholders until fixed.

```
SpecID_EY23_CSF0 :=
(
  ArtifactType = CSF,
  Core = (H),
  Calc = {
    class: "truncation-calculus",
    truncation: { basis: "...", cut_rule: "...", boundary_rule: "..."},
    window: { type: "extrema-linear-clip", crossN: "per-N" }
  },
  A = {
    class: "A1-spline",
    m_max: 8,
    d_max: 3,
    knot_grid: { Q_t: 64 },
    coeff_grid: { step: 0.01, bound: 2.0 },
    continuity: "none-or-C0 (declare)"
  },
  J = {
    type: "minimize",
    weights: { w_stab: 100, w_sharp: 1, w_simp: 0.1, w_bias: 0 },
    stab: { discrepancy: "frobenius-proxy", omega: "uniform" },
```



```

    sharp: { response: "variance-on-eigs", avg_over_N: true },
    simp: { alpha_m: 1, alpha_d: 1, alpha_c: 0.01 }
  },
  C = {
    bound: { B_max: 2.0 },
    lip: { L_max: 20.0, method: "grid-or-analytic (declare)" },
    stab: { tau: [1e-2, 1e-2, ...] },
    pos: null,
    norm: null,
    smooth: null
  },
  TieBreak = { type: "lexmin-Enc(f)" },
  NumericModel = {
    params_encoding: "fixed-point",
    eval_encoding: "det-float-or-fixed (declare)",
    truncation_schedule: { N: [256, 512, 1024, 2048] },
    tolerances: { eps_abs: 1e-12, eps_rel: 1e-9, eps_spec: 1e-8 },
    aggregation: "sequential"
  },
  Cert = {
    schema: "EY22-CCE",
    canonicity_mode: "Mode-A-enumeration",
    required_checks: ["V1", "V2", "V3", "V4", "V5", "V6", "V7"]
  },
  Interop = {
    allowed_forms: ["C1", "C3", "C4"],
    compat: {
      EY24: "Compat_23_24",
      EY25: "Compat_23_25",
      EY29: "Compat_23_29"
    }
  },
  Version = "VID-2026-02"
)

```

Rule. Once CSF-0 is published, all placeholders **MUST** be replaced by fixed values (or hash commitments) and the Version **MUST** be treated as immutable for that meaning.

9.9 Chapter Summary

This chapter defined CSF-0, a concrete baseline profile for EY23 designed for audit-friendly canonical selection. It fixed the default admissible family (finite A1 splines), objective term structure, constraint set, Mode A verification, and a reference SpecID instance template.

Next. With EY23 complete, Vol. No. IV proceeds to EY24 (Validity Penalty Operator) which defines canonical validity enforcement artifacts that compose with $F_\star(H)$ under the interop predicates declared in Chapter 8.

EY24: Validity Penalty Operator

$$V = C^\dagger C$$

Canonical Operator Artifacts for QFC

Enter Yourname

Abstract

This whitepaper defines the Validity Penalty Operator $V = C^\dagger C$, a canonical QFC artifact that encodes validity constraints as a positive semidefinite operator derived from a declared constraint operator C . Rather than asserting validity informally, EY24 provides an audit-native mechanism to penalize or exclude invalid states, trajectories, or operator instances by construction.

The artifact is specified by a normative family of admissible constraint operators C , a canonical objective for selecting a representative when multiple constraints are possible, and a deterministic NumericModel and certificate schema inherited from EY22. The core requirement is third-party verifiability: independent verifiers **MUST** be able to confirm that V is correctly formed, that C is admissible, that required invariants (positivity, domain compatibility, stability under truncation) hold within tolerance, and that any claimed interoperability with other Vol. No. IV artifacts is satisfied.

$V = C^\dagger C$ is designed to compose with the Canonical Spectral Filter $F_\star(H)$ (EY23), audit-native evolution (EY25), coarse-grain reduction (EY29), and regime transport (EY28), enabling workflows where “validity” is not a narrative but a checkable operator-level contract.

1 Motivation, Scope, and Design Goals

1.1 Motivation: Validity Must Be an Operator, Not an Opinion

In many operator-centric workflows, the term “valid” is used informally: a state is said to be valid, a trajectory is said to satisfy constraints, or an operator instance is said to respect a rule. Without a canonical construction, such claims are fragile:

- **Ambiguity:** different implementations encode the same “rule” differently.
- **Non-auditability:** a third party cannot verify validity without reconstructing hidden choices.
- **Parameter smuggling:** constraints become tunable knobs disguised as “assumptions.”

EY24 fixes this by defining validity as a positive semidefinite operator:

$$V := C^\dagger C,$$

derived from a declared constraint operator C . Instead of asserting validity, a workflow can penalize invalidity by construction: large values of $\langle \psi, V\psi \rangle$ indicate violation, and the structure of C fixes *what* is being enforced.

1.2 What This Whitepaper Defines

This whitepaper defines:

- the Validity Penalty Operator artifact $V = C^\dagger C$,
- the admissible family $\mathcal{A}_C$ of constraint operators C ,
- an objective $\mathcal{J}_C$ and constraints $\mathcal{C}_C$ that canonically select $C_\star$ when multiple candidates exist,
- the certificate and verifier checks needed to accept or reject V ,
- and minimal interoperability declarations (with EY23/EY25/EY29/EY28) under EY22 rules.

Unless explicitly overridden, all suite conventions (SpecID binding, NumericModel determinism, certificates, interop vocabulary) follow EY22.

1.3 Scope

In-scope.

- Canonical construction of $V = C^\dagger C$ from a declared C .
- Deterministic truncation evaluation $V_N = C_N^\dagger C_N$ under a declared truncation policy.
- Validity measures derived from V (e.g. energy-like penalties) as *operator-level* quantities.
- Verifiable interoperability: how V composes with filtering, evolution, coarse-grain, and transport.

Out-of-scope.

- Claims that a chosen C corresponds to a physically correct law of nature.
- Claims of cryptographic security or correctness beyond the stated verifier checks.
- “Universal validity”: EY24 provides a framework for encoding constraints, not a claim that a particular constraint set is correct.
- Hidden, human-only constraints not expressible in C and not committed by hash.

1.4 Interpretation: What V Does

The operator $V = C^\dagger C$ is positive semidefinite by construction. In workflows it is used in several canonical ways:

- **Penalty scoring:** assign a scalar validity cost $s(\psi) = \langle \psi, V\psi \rangle$.
- **Constraint projection:** prefer subspaces where V is small (or where $C\psi \approx 0$).
- **Validity-weighted filtering:** incorporate V into spectral actions (e.g. $H \mapsto H + \lambda V$).
- **Audit trails:** publish V so validity checks are replayable and implementation-independent.

Rule. Any workflow using V MUST declare the semantic mode (penalty, projection, additive integration, multiplicative weighting), because these are not equivalent in general.

1.5 Design Goals (Normative)

EY24 is governed by the following design goals:

G1: Canonically by Specification

The meaning of V MUST be uniquely determined by public specification:

$$(H, C, \text{Calc}, \mathcal{A}_C, \mathcal{J}_C, \mathcal{C}_C, \text{TieBreak}, \text{NumericModel}).$$

If C is selected rather than provided, the selection rule is part of the artifact meaning.

G2: Positivity and Deterministic Checkability

V MUST be positive semidefinite by construction and MUST admit deterministic checks:

- structural check that $V = C^\dagger C$ under declared adjoint semantics,
- invariants required by the declared constraint class (e.g. locality, sparsity pattern, symmetry),
- stability under truncation schedule (if required).

G3: Minimal Degrees of Freedom

The admissible family $\mathcal{A}_C$ SHOULD be chosen to minimize hidden degrees of freedom. If the family is broad, $\mathcal{J}_C$ and TieBreak MUST remove ambiguity deterministically.

G4: Interoperability With EY23/EY25/EY29/EY28

The artifact SHOULD compose with:

- EY23 $F_\star(H)$ (spectral probing),
- EY25 $\mathcal{L}_{\text{audit}}$ (audit-native evolution),
- EY29 R (coarse-grain reduction),
- EY28 $T_{1 \rightarrow 2}$ (regime transport),

under the interoperability rules of EY22 Chapter 6. Any additional predicates MUST be declared in EY24 Interop.

G5: Publishable Validity Boundary

EY24 MUST support publishing a validity boundary that is auditable:

- external objects (e.g. constraint definitions, code snapshots) MUST be committed by hash,
- certificates MUST bind to full SpecID,
- verifiers MUST not require trust in the producer.

1.6 Chapter Summary

This chapter motivated the need to encode validity as a canonical operator artifact and fixed the scope and design goals for $V = C^\dagger C$. It positioned EY24 as the suite’s validity-enforcement primitive that composes with spectral filtering, audit-native evolution, coarse-grain reduction, and regime transport.

Chapter 2 next gives the normative artifact definition and the EY24 SpecID binding: admissible constraint families for C , calculus/adjoint semantics, objective and constraints for canonical selection, tie-break, and NumericModel requirements.

2 Normative Definition and SpecID Binding

2.1 Conformance to EY22

EY24 inherits Vol. No. IV conventions from EY22. In particular:

- SpecID semantics follow EY22 Chapter 3.
- NumericModel determinism follows EY22 Chapter 4.
- Certificate schema and verifier semantics follow EY22 Chapter 5 and Appendix A.1.
- Interoperability rules follow EY22 Chapter 6.

Any deviation **MUST** be explicitly declared in SpecID_{EY24}.

2.2 Core Objects and Minimal Assumptions

Core. The core objects are:

- a constraint operator C acting on a declared space $\mathcal{H}$ (or on a declared domain $\text{Dom}(C) \subseteq \mathcal{H}$),
- the induced validity penalty operator $V := C^\dagger C$.

Minimal assumptions. EY24 requires that:

- C is densely defined (or declared densely defined on the truncation sequence) so that an adjoint is meaningful;
- the adjoint operation $C \mapsto C^\dagger$ is defined by an explicit inner product or adjoint convention;
- truncation (if used) yields deterministic finite objects C_N and therefore $V_N = C_N^\dagger C_N$;
- the admissible family of constraints is non-empty under the declared constraints.

If these assumptions fail, the artifact **MUST** return a deterministic failure outcome.

2.3 Artifact Definition

Definition 2.1 (Validity Penalty Operator). Let C be a declared constraint operator. The **Validity Penalty Operator** is:

$$V := C^\dagger C.$$

An EY24 artifact instance is the pair (C, V) bound by SpecID, where C is either:

- **provided** as an explicit constraint operator, or
- **selected canonically** as $C_\star$ from an admissible family $\mathcal{A}_C$ using an objective $\mathcal{J}_C$, constraints $\mathcal{C}_C$, and tie-break rule.

Rule (no implicit constraints). Validity MUST be entirely expressible in the published C (or its hash commitments). Undeclared constraints are forbidden.

2.4 Adjoint Semantics ($C^\dagger$)

The meaning of $C^\dagger$ depends on the inner product and representation.

Adjoint component. EY24 introduces a required SpecID component:

AdjointSpec,

which declares:

- the inner product $\langle \cdot, \cdot \rangle$ (or its discrete truncation form),
- the representation in which the adjoint is computed,
- and any weighting/measure conventions for function-space realizations.

Rule. Two artifacts that use different AdjointSpec define different V , even if C appears syntactically identical.

2.5 SpecID for EY24

ArtifactType. For EY24:

ArtifactType = VPO.

SpecID components (EY24 binding). The full SpecID tuple is:

$\text{SpecID}_{\text{EY24}} := (\text{VPO}, \text{Core} = (C), \text{AdjointSpec}, \text{Calc}, \mathcal{A}_C, \mathcal{J}_C, \mathcal{C}_C, \text{TieBreak}, \text{NumericModel}, \text{Cert}, \text{Interop},$

Each component MUST be specified as follows:

AdjointSpec (required)

AdjointSpec MUST declare:

- the inner product used to define $\dagger$,
- any weighting operator W if $\langle x, y \rangle = x^\dagger W y$,
- the truncation-level adjoint rule $C_N^\dagger$ if truncation calculus is used,
- and canonical encoding for these definitions (or hash commitments).

Calc (**required**)

Calc MUST declare how C and V are interpreted and composed, including:

- whether C is bounded/unbounded, matrix/operator form, or superoperator form,
- if truncation is used: $C \mapsto C_N$ and the induced $V_N = C_N^\dagger C_N$,
- deterministic multiplication and accumulation rules for forming $C_N^\dagger C_N$.

$\mathcal{A}_C$ (**required**)

$\mathcal{A}_C$ is the admissible family of constraint operators. It MUST specify:

- the constraint class (e.g. locality constraints, commutator constraints, projector constraints),
- parameter bounds and canonical representation,
- any structural restrictions (sparsity pattern, symmetry, block structure).

Rule. If C is provided (not selected), $\mathcal{A}_C$ MUST still be present and MUST define the membership check class to prevent “arbitrary C ” from being smuggled in.

$\mathcal{J}_C$ (**required if selection is used; otherwise $\emptyset$**)

If multiple constraint operators are allowed, $\mathcal{J}_C$ defines canonical selection of C_* . $\mathcal{J}_C$ MUST specify:

- minimize/maximize convention,
- explicit score terms (e.g. strength of enforcement vs stability vs simplicity),
- deterministic evaluation procedure under NumericModel.

If no selection is used, set:

$$\mathcal{J}_C := \emptyset$$

in SpecID.

$\mathcal{C}_C$ (**required**)

$\mathcal{C}_C$ is the feasibility predicate set for constraint operators. It MUST include:

- domain/space compatibility constraints (so $C^\dagger C$ is defined),
- structural constraints of the chosen class (e.g. locality radius, sparsity, boundedness),
- truncation stability constraints if required.

TieBreak (**required**)

TieBreak MUST define deterministic selection when:

- multiple admissible C yield identical objective within tolerance,
- or multiple parameterizations represent the same operator.

Recommended: lexicographic minimum over canonical encodings of C .

NumericModel (**required**)

NumericModel MUST be included as in EY22 Chapter 4. In EY24 it MUST also specify:

- rounding/tolerance for adjoint verification,
- deterministic multiplication rule for forming $C^\dagger C$,
- and deterministic eigen/PSD checks if positivity is certified numerically.

Cert (**required**)

Cert MUST specify:

- the certificate payload required to verify C and $V = C^\dagger C$,
- verifier checks: membership, feasibility, adjoint semantics, positivity, canonicity (if selection used),
- and failure codes.

Interop (**required**)

Interop MUST specify:

- allowed composition forms (EY22 C1–C5),
- invariants required when composing V with other artifacts (e.g. EY23/EY25/EY29/EY28),
- compatibility predicates for claimed pairings.

Version (**required**)

Version MUST bind the meaning of all components above. Any change in: AdjointSpec, Calc, $\mathcal{A}_C$, $\mathcal{J}_C$, $\mathcal{C}_C$, TieBreaker MUST change Version.

2.6 Deterministic Outcomes

EY24 verification MUST return one of:

$$\{\text{ACCEPT}, \text{REJECT}\} \cup \{\text{FAIL_}\ast\}.$$

EY24 MUST support, at minimum:

- **FAIL_DOMAIN_MISMATCH**
- **FAIL_ADJOINT_SEMANTICS**
- **FAIL_CONSTRAINT**
- **FAIL_CANONICALITY**
- **FAIL_NONDETERMINISM**

2.7 Chapter Summary

This chapter defined EY24 as the canonical construction $V = C^\dagger C$ and fixed the SpecID components that bind its meaning: AdjointSpec, calculus, admissible constraint families, optional canonical selection objective, feasibility constraints, tie-break, numeric model, certificate requirements, interoperability declarations, and version binding.

Chapter 3 next defines admissible constraint operator families $\mathcal{A}_C$ (constraint classes for C), including canonical representations, parameter bounds, and membership check procedures.

3 Admissible Constraint Families $\mathcal{A}_C$: Canonical Classes for C

3.1 Purpose of $\mathcal{A}_C$

The admissible family $\mathcal{A}_C$ is the set of constraint operators C from which the validity penalty $V = C^\dagger C$ is formed. In EY24, $\mathcal{A}_C$ is a first-class normative object: it prevents arbitrary, author-chosen constraints from being introduced without audit.

Rule. A constraint operator C is admissible only if:

- it belongs to a declared constraint class,
- it has a canonical parameterization (encoding),
- its parameter bounds are explicit,
- and membership is checkable deterministically under NumericModel.

3.2 Canonical Representation and Encoding

Rule (canonical encoding). Every $C \in \mathcal{A}_C$ MUST have a canonical representation that can be serialized and compared lexicographically (EY22 Appendix A.1). This canonical encoding MUST be used by TieBreak.

Required encoding fields. The canonical encoding MUST include:

- ClassID identifying the constraint class,
- a parameter tuple θ_C in a fixed order,
- any structural descriptors (e.g. sparsity pattern policy, locality radius),
- and any commitments required to external objects (e.g. basis definitions).

3.3 Constraint Classes

EY24 defines four canonical constraint classes. A SpecID MUST select exactly one active class for $\mathcal{A}_C$, or define an explicit finite union with deterministic tie-break across classes.

Class C1: Linear Constraint Stack (Projector/Residual Form)

Idea. Many validity notions can be expressed as linear residuals:

$$C\psi \approx 0.$$

Here, C is a stack of linear constraints (rows) on the state/representation.

Definition. C1 constraints are of the form:

$$C := \begin{bmatrix} C_1 \\ C_2 \\ \vdots \\ C_r \end{bmatrix},$$

where each C_i is a linear operator with declared structure (e.g. sparse, banded, local stencil).

Canonical parameters.

$$\theta_{C1} := (r, \text{StructID}, \{\Theta_i\}_{i=1}^r),$$

where Θ_i are the canonical encodings of each sub-constraint operator.

Typical use.

- “hard constraints”: enforce invariants $C_i\psi = 0$,
- “soft constraints”: penalize residual magnitudes via $V = C^\dagger C$.

Class C2: Commutator Validity Constraints

Idea. Validity can mean “commutes with” or “is invariant under” a declared generator set $\{G_j\}$:

$$[H, G_j] \approx 0 \quad \text{or} \quad [A, G_j] \approx 0.$$

In EY24, this is encoded as an operator residual.

Definition (state-side). If validity is about states ψ being in a commutant-invariant subspace, define:

$$(C\psi)_j := (G_j - \lambda_j I)\psi,$$

for declared λ_j (often $\lambda_j = 0$).

Definition (operator-side). If validity is about an operator A commuting with G_j , define:

$$C(A) := \bigoplus_j (G_j A - A G_j),$$

where the direct sum is encoded as a stacked constraint.

Canonical parameters.

$$\theta_{C2} := (\text{TargetType} \in \{\text{state}, \text{operator}\}, \{G_j\}_{j=1}^r, \{\lambda_j\}_{j=1}^r, \text{StackRule}).$$

Note. C2 is a bridge class for EY27 (commutant generator discovery). EY24 does not assume the generator set is “correct”; it only standardizes how such constraints are encoded and audited.

Class C3: Spectral-Window Validity Constraints

Idea. A validity notion may be “spectral support lies within a declared window.” This can be encoded using a spectral action.

Definition. Let g be a declared window function (not the EY23 canonical filter, unless explicitly referenced). Define:

$$C := g(H) \quad \text{or} \quad C := I - g(H),$$

depending on whether validity is defined as *in-window* or *out-of-window* suppression.

Then:

$$V = C^\dagger C = C^2 \quad (\text{if } C \text{ is self-adjoint}).$$

Canonical parameters.

$$\theta_{C3} := (\text{WindowFuncID}, \text{Semantics} \in \{\text{in}, \text{out}\}, \text{CalcRef}).$$

Rule. If C3 references EY23 $F_\star(H)$, the SpecID MUST include an explicit interop reference and compatibility predicate (EY23 Chapter 8).

Class C4: Locality / Stencil Constraint Operators

Idea. When the representation has locality structure (graph, lattice, index geometry), validity can be defined as respecting that locality.

Definition. C4 constraints are generated by local stencils S_α applied across indices:

$$(C\psi)_\alpha := S_\alpha \psi,$$

with a finite set of stencil types and a deterministic placement rule.

Canonical parameters.

$$\theta_{C4} := (\text{GeometryID}, \text{StencilSetID}, \text{Radius}, \text{PlacementRule}, \text{BoundaryRule}).$$

Typical use.

- enforce “no nonlocal coupling” beyond a radius,
- penalize forbidden adjacency patterns,
- encode manufacturing / discretization constraints in truncation representations.

3.4 Parameter Bounds (Anti-Smuggling Requirement)

Rule. Every admissible class MUST include explicit parameter bounds sufficient to prevent unbounded tuning. At minimum:

- a complexity bound (e.g. number of constraints r , number of generators, stencil radius),
- magnitude bounds on coefficients (or norm bounds),

- and a finite reference universe for any IDs used (committed by hash if external).

If bounds are omitted, the constraint family **MUST** be treated as non-canonical and verification **MUST** reject.

3.5 Baseline Constraint Family Profile (VPO-0)

EY24 defines a baseline constraint-family profile intended for Vol. No. IV defaults.

VPO-0. Unless overridden, the baseline $\mathcal{A}_C$ is:

$$\mathcal{A}_{C,\text{VPO-0}} := \text{C1 (Linear Constraint Stack)}$$

with fixed:

- maximum number of constraints $r_{\max}$,
- a fixed structural class StructID (e.g. sparse-banded),
- coefficient quantization policy (fixed-point step),
- and deterministic ordering of constraints.

Rule (baseline must be fully specified). All baseline constants and structural identifiers **MUST** be fixed in SpecID (or committed by hash) and **MUST NOT** be tuned per instance.

3.6 Membership Check Procedure

For any chosen admissible class, the certificate **MUST** include a membership check routine:

- parse canonical encoding of C ,
- verify class ID and parameter tuple correctness,
- verify bounds and structural restrictions,
- verify any external ID commitments,
- return deterministic failure if violated.

3.7 Chapter Summary

This chapter defined the admissible constraint family $\mathcal{A}_C$ as a normative object and introduced four canonical constraint classes for C : (C1) linear residual stacks, (C2) commutator constraints, (C3) spectral-window constraints, and (C4) locality/stencil constraints. It also imposed explicit parameter bounds to prevent hidden tuning and defined a baseline profile VPO-0 intended for Vol. No. IV defaults.

Chapter 4 next defines the objective $\mathcal{J}_C$ and feasibility constraints $\mathcal{C}_C$ used when C is selected canonically, and specifies the tie-break rule and invariants required for stable validity enforcement.

4 Objective $\mathcal{J}_C$ and Constraints $\mathcal{C}_C$: Canonical Selection and Feasibility

4.1 Purpose

If a workflow provides a fully specified constraint operator C , then $V = C^\dagger C$ is determined directly. However, Vol. No. IV permits *canonical selection* of C when multiple admissible constraints could encode the same validity intent. This chapter defines:

- the objective $\mathcal{J}_C$ used to choose $C_\star$ from $\mathcal{A}_C$,
- and the feasibility constraints $\mathcal{C}_C$ that all admissible C MUST satisfy.

Rule. If the SpecID declares $\mathcal{J}_C \neq \emptyset$, then canonical selection is active and the certificate MUST prove optimality and TieBreak correctness. If $\mathcal{J}_C = \emptyset$, the certificate MUST still prove membership and feasibility of the provided C .

4.2 Objective Decomposition

EY24 standardizes the objective as a weighted sum of named terms:

$$\mathcal{J}_C(C; H) := w_{\text{enf}} \mathcal{J}_{\text{enf}}(C; H) + w_{\text{stab}} \mathcal{J}_{\text{stab}}(C; H) + w_{\text{simp}} \mathcal{J}_{\text{simp}}(C) + w_{\text{interop}} \mathcal{J}_{\text{interop}}(C; H),$$

with all weights $w_\bullet \geq 0$ fixed in SpecID.

Rule (no hidden tuning). Weights MUST be explicit constants (or hash-committed). Tuning weights per instance is forbidden unless the tuning rule is part of SpecID.

4.3 $\mathcal{J}_{\text{enf}}$: Enforcement Strength Term

Validity constraints are useful only if they separate valid from invalid behavior. EY24 allows multiple enforcement proxies.

State-side enforcement proxy. For a declared reference distribution $\mathcal{D}$ over test states ψ (deterministic seed or enumerated set), define:

$$\mathcal{J}_{\text{enf}}(C) := -\mathbb{E}_{\psi \sim \mathcal{D}} [\|C\psi\|^2],$$

meaning: prefer constraints that produce larger residuals on generic (nonvalid) states, i.e. enforce more strongly.

Operator-side enforcement proxy. If C acts on operators A , define a deterministic test set $\mathcal{D}_A$ and set:

$$\mathcal{J}_{\text{enf}}(C) := -\mathbb{E}_{A \sim \mathcal{D}_A} [\|C(A)\|^2].$$

Rule. The test set $\mathcal{D}$ or $\mathcal{D}_A$ MUST be deterministic and committed by hash, or defined as an explicit enumeration rule.

4.4 $\mathcal{J}_{\text{stab}}$: Truncation Stability Term

A constraint that changes unpredictably with truncation destroys auditability. Define truncation levels $N_1 < \dots < N_L$ from NumericModel and compute truncations C_{N_ℓ} .

Stability discrepancy.

$$\Delta_\ell(C) := D(C_{N_\ell}, C_{N_{\ell+1}}),$$

where D is a deterministic discrepancy functional (e.g. normalized Frobenius proxy on canonical matrices).

Stability score.

$$\mathcal{J}_{\text{stab}}(C) := \sum_{\ell=1}^{L-1} \omega_\ell \Delta_\ell(C),$$

with $\omega_\ell \geq 0$ fixed in SpecID.

4.5 $\mathcal{J}_{\text{simp}}$: Simplicity / Complexity Term

To prevent smuggling validity through overly expressive constraint operators, EY24 penalizes complexity.

Canonical complexity measures. Defined per constraint class:

- C1 (stack): $\mathcal{J}_{\text{simp}} := \alpha_r r + \alpha_s \text{nnz}(C) + \alpha_c \|\theta\|_1$,
- C2 (commutator): $\mathcal{J}_{\text{simp}} := \alpha_g r + \alpha_s \sum_j \text{size}(G_j)$,
- C3 (spectral): $\mathcal{J}_{\text{simp}} := \alpha_f \text{complexity}(g)$,
- C4 (stencil): $\mathcal{J}_{\text{simp}} := \alpha_R \text{Radius} + \alpha_t |\text{StencilSet}|$.

All $\alpha_\bullet \geq 0$ MUST be fixed in SpecID and computed from canonical encodings.

4.6 $\mathcal{J}_{\text{interop}}$: Interop-Friendly Constraint Term (Optional)

Some constraints compose better with EY23/EY25/EY29/EY28. EY24 permits an optional objective term encouraging interop.

Example (filter compatibility). Prefer constraints whose induced $V = C^\dagger C$ is stable under application of the EY23 canonical filter:

$$\mathcal{J}_{\text{interop}}(C; H) := \sum_{\ell} \tilde{\omega}_\ell D(F_\star(H_{N_\ell}) V_{N_\ell}, V_{N_\ell} F_\star(H_{N_\ell})),$$

interpreted as a commutation-proxy penalty. (This term is only enabled if explicitly declared and interop predicates are satisfied.)

Rule. If $\mathcal{J}_{\text{interop}}$ is enabled, the referenced artifacts and compatibility predicates MUST be declared in SpecID.

4.7 Feasibility Constraints $\mathcal{C}_C$

The feasibility predicates $\mathcal{C}_C(C; H)$ define when a constraint operator is valid for forming $V = C^\dagger C$.

F1: Domain and Space Compatibility

$$\mathcal{C}_{\text{dom}}(C) := \left(\text{Dom}(C) \text{ is declared and compatible with AdjointSpec and Calc} \right).$$

If truncation calculus is used, this means C_N and $C_N^\dagger$ are defined for each N in the schedule.

F2: Structural Membership and Bounds

$$\mathcal{C}_{\text{struct}}(C) := \left(C \in \mathcal{A}_C \text{ and all parameter bounds hold} \right).$$

F3: Adjoint Consistency

$$\mathcal{C}_{\text{adj}}(C) := \left(\text{the adjoint computed from AdjointSpec matches the declared } C^\dagger \text{ semantics} \right).$$

In finite truncations, this reduces to deterministic checks that the published $C_N^\dagger$ equals the conjugate-transpose under the declared weighting.

F4: Positivity of V (By Construction, With Optional Numeric Witness)

By definition $V = C^\dagger C \succeq 0$. The verifier MUST check the structural identity:

$$V_N \stackrel{?}{=} C_N^\dagger C_N$$

for required truncation levels N . Optionally, the certificate MAY include a numeric PSD witness (e.g. Cholesky success) but it MUST NOT replace the structural check.

F5: Truncation Stability (Optional Hard Constraint)

If enabled, require:

$$\bigwedge_{\ell=1}^{L-1} \Delta_\ell(C) \leq \tau_\ell^{(C)},$$

with thresholds fixed in SpecID. This is analogous to EY23's hard stability constraint.

F6: Bounded Penalty Growth (Optional)

To prevent V from dominating all workflows in an unstable manner, EY24 allows a bound on penalty magnitude:

$$\mathcal{C}_{\text{pen}}(C) := \left(\|V_{N_\ell}\| \leq B_V \text{ for all required } N_\ell \right),$$

where the norm proxy and bound B_V are fixed in NumericModel.

4.8 TieBreak for C

When multiple feasible constraints tie under $\mathcal{J}_C$, EY24 uses the deterministic TieBreak rule:

Definition 4.1 (EY24 TieBreak). Let $\mathcal{F} \subseteq \mathcal{A}_C$ be feasible candidates achieving optimal objective value within tolerance. Then:

$$C_\star := \min_{\text{lex}} \text{Enc}(C),$$

where $\text{Enc}(C)$ is the canonical byte encoding of C (EY22 Appendix A.1).

Rule. TieBreak MUST operate on canonical encodings, not on floating-point approximations.

4.9 Baseline Objective and Constraint Profile (VPO-0)

EY24 defines a baseline profile for Vol. No. IV.

VPO-0. Unless overridden:

- $\mathcal{A}_C$: C1 linear constraint stack with fixed $r_{\max}$ and quantized coefficients,
- $\mathcal{J}_C$: disabled by default ($\emptyset$); C is provided but must be admissible,
- $\mathcal{C}_C$: enforce domain compatibility + structural bounds + adjoint consistency + structural identity $V = C^\dagger C$,
- TieBreak: lexmin encoding (used only if selection is enabled).

Normative note. VPO-0 can later be upgraded to a “VPO-1” profile where $\mathcal{J}_C$ is enabled to select $C_\star$ canonically from a finite grid, analogous to EY23 CSF-0.

4.10 Chapter Summary

This chapter defined the canonical selection objective $\mathcal{J}_C$ for choosing constraint operators and the feasibility predicates $\mathcal{C}_C$ required to form $V = C^\dagger C$ deterministically. It standardized enforcement, stability, simplicity, and optional interop-friendly objective terms, and fixed TieBreak as lexicographic selection over canonical encodings.

Chapter 5 next specifies certificate payload and verifier checks for EY24: what must be published to prove admissibility of C , correctness of adjoint semantics, correctness of $V = C^\dagger C$, and any claimed interop properties.

5 Certificate and Verifier: What Must Be Published

5.1 Purpose

EY24 defines the Validity Penalty Operator $V = C^\dagger C$ as an audit-native artifact. This chapter specifies:

- the certificate payload π_{VPO} required to publish an EY24 instance,
- the deterministic verifier checks that accept or reject that instance,
- and the required failure codes for structured rejection.

EY24 certificates conform to the EY22 certificate schema (EY22 Chapter 5, Appendix A.1). EY24 adds only artifact-specific fields and checks.

5.2 Certificate Object π_{VPO}

Rule. An EY24 instance MUST be published with a certificate π_{VPO} that includes, at minimum, the following canonical content.

Header (required)

- artifact type: VPO
- artifact symbol: $V = C^\dagger C$
- SpecID short form and full SpecID tuple
- version identifier

SpecID (required)

The full SpecID tuple MUST be embedded:

(VPO, (C), AdjointSpec, Calc, $\mathcal{A}_C$, $\mathcal{J}_C$, $\mathcal{C}_C$, TieBreak, NumericModel, Cert, Interop, Version).

Inputs (required)

Inputs MUST include:

- canonical description of C (canonical encoding) or hash commitment to the full C object,
- if selection is used: admissible family definition and any finite grids used,
- AdjointSpec payload (inner product / weights / adjoint convention) or hash commitment,
- truncation schedule $N_1 < \dots < N_L$ if truncation calculus is used,
- all numeric constants used by $\mathcal{A}_C$, $\mathcal{J}_C$, $\mathcal{C}_C$ (bounds, thresholds, weights),
- deterministic aggregation and ordering rules used for evaluation.

Outputs (required)

Outputs MUST include:

- canonical encoding of C (always required),
- canonical encoding (or committed hash) of V ,
- truncation-level outputs if required: $\{C_{N_\ell}\}$ and $\{V_{N_\ell}\}$ or their commitments,
- output hashes of C encoding and V encoding,
- any claimed invariants as explicit fields (informational; still must be checkable).

Checks (required)

The certificate MUST list check IDs and what each check verifies, grouped as:

- structural checks,
- admissibility and feasibility checks,
- adjoint semantics checks,
- canonicity checks (if selection is enabled),
- interoperability checks (if claimed).

Commitments (required if external objects exist)

All external objects (basis definitions, geometry IDs, generator sets, transcript logs, code snapshots) MUST be committed by hash according to EY22 Appendix A.1.

5.3 Verifier Contract

A verifier takes (π_{VPO}) and deterministically returns:

$$\{\text{ACCEPT}, \text{REJECT}\} \cup \{\text{FAIL_}\ast\}.$$

The verifier MUST NOT require trust in producer identity, informal descriptions, or non-committed external resources.

5.4 Required Verifier Checks (Normative)

This section enumerates the minimal checks required for EY24 acceptance.

V1: Structural Validity (CCE)

- parse canonical certificate encoding (EY22 CCE rules),
- reject malformed encodings, duplicate keys, non-canonical numbers,
- confirm required fields exist.

Failure: **REJECT_MALFORMED**.

V2: SpecID Completeness and Consistency

- SpecID tuple is present and complete,
- $\text{ArtifactType} = \text{VPO}$,
- AdjointSpec present,
- NumericModel present and complete,
- certificate declares whether selection is enabled ($\mathcal{I}_C \neq \emptyset$).

Failure: **REJECT_SPECID_MISSING**, **REJECT_SPECID_MISMATCH**.

V3: Input Completeness / No Hidden Inputs

- C is provided in canonical encoding or committed by hash,
- AdjointSpec is provided or committed,
- all numeric constants (bounds/thresholds/weights) are provided or committed,
- truncation schedule and calculus rules are provided if used,
- any referenced external IDs resolve to committed objects.

Failure: **REJECT_HIDDEN_INPUT**, **FAIL_HASH_MISMATCH**.

V4: Membership Check for C

- parse canonical encoding of C ,
- verify ClassID matches $\mathcal{A}_C$,
- verify parameter bounds and structural restrictions (sparsity, locality radius, generator count, etc.),
- verify deterministic ordering requirements (e.g. sorted stacks).

Failure: **FAIL_CONSTRAINT** or **REJECT_MALFORMED**.

V5: Feasibility Constraints $\mathcal{C}_C$

Verify $\mathcal{C}_C(C; H) = \text{TRUE}$ under NumericModel, including at minimum:

- domain/space compatibility (**FAIL_DOMAIN_MISMATCH** on failure),
- structural feasibility bounds (**FAIL_CONSTRAINT**),
- truncation stability if enabled (**FAIL_NONCONVERGENCE** or **FAIL_CONSTRAINT**),
- bounded penalty growth if enabled (**FAIL_INVARIANT**).

V6: Adjoint Semantics Check

The verifier MUST validate that the adjoint used in $C^\dagger$ matches AdjointSpec.

Finite truncation check. For each required N_ℓ , verify:

$$C_{N_\ell}^\dagger \stackrel{?}{=} \text{Adjoint}_{\text{AdjointSpec}}(C_{N_\ell}),$$

within declared tolerance if approximate arithmetic is used.

Failure: **FAIL_ADJOINT_SEMANTICS**.

V7: Structural Identity Check $V = C^\dagger C$

The core correctness check is structural:

$$V_{N_\ell} \stackrel{?}{=} C_{N_\ell}^\dagger C_{N_\ell}$$

for required truncation levels N_ℓ . Matrix multiplication MUST follow deterministic rules in NumericModel.

Failure: **FAIL_CONSTRAINT** (if mismatch) or **FAIL_NONDETERMINISM** (if replay differs).

V8: Optional PSD Witness Check (Informational)

If the certificate includes a PSD witness (e.g. Cholesky factorization), the verifier MAY check it, but it MUST NOT replace V7.

Failure (if checked and fails): **FAIL_INVARIANT**. (Acceptance MUST still rely on V7.)

V9: Canonicity Check for $C_\star$ (Only if Selection Enabled)

If $\mathcal{J}_C \neq \emptyset$, the verifier MUST confirm the published $C_\star$ is canonical. Two modes apply:

Mode A (finite $\mathcal{A}_C$). Enumerate all feasible candidates, evaluate $\mathcal{J}_C$, apply TieBreak, and confirm the published $C_\star$ matches.

Mode B (certified search). Verify the declared search protocol transcript (deterministic steps, deterministic stopping) and verify the published $C_\star$ is the protocol-determined canonical outcome.

Failure: **FAIL_CANONICALITY** or **FAIL_NONDETERMINISM**.

V10: Interop Checks (Only if Claimed)

If the certificate claims compatibility with other artifacts (EY23/EY25/EY29/EY28), the verifier MUST evaluate the declared compatibility predicates under the shared NumericModel and declared calculus.

Failure: **FAIL_COMPAT_PREDICATE** or the specific interop failures from EY22 Chapter 6.

5.5 Failure Codes (Required)

EY24 MUST support at minimum:

- **REJECT_MALFORMED**
- **REJECT_SPECID_MISSING**
- **REJECT_SPECID_MISMATCH**
- **REJECT_HIDDEN_INPUT**
- **FAIL_HASH_MISMATCH**
- **FAIL_DOMAIN_MISMATCH**
- **FAIL_ADJOINT_SEMANTICS**
- **FAIL_CONSTRAINT**
- **FAIL_CANONICALITY**

- **FAIL_NONDETERMINISM**
- **FAIL_COMPAT_PREDICATE**
- **FAIL_INVARIANT**

5.6 Chapter Summary

This chapter defined the EY24 certificate payload π_{VPO} and the deterministic verifier checks required to accept an instance of $V = C^\dagger C$. It fixed checks for membership and feasibility of C , correctness of adjoint semantics, structural correctness of V , and canonicity verification when C is selected rather than provided. It also fixed the minimal failure code vocabulary.

Chapter 6 next defines interoperability declarations (mini) for EY24: how V composes with EY23 filtering, EY25 audit evolution, EY29 coarse-grain reduction, and EY28 transport, including compatibility predicates and deterministic failure codes.

6 Interop Declarations (Mini): Composition with EY23/EY25/EY29/EY28

6.1 Purpose

This chapter instantiates the EY22 interoperability framework (EY22 Chapter 6) for EY24. It defines minimal compatibility predicates for composing the Validity Penalty Operator $V = C^\dagger C$ with:

- EY23 (CSF) $F_\star(H)$,
- EY25 (LIOUV) audit-native evolution $\mathcal{L}_{\text{audit}}$,
- EY29 (CGR) coarse-grain reduction R ,
- EY28 (INTW) regime transport $T_{1 \rightarrow 2}$.

Rule. If an EY24 instance claims interoperability beyond what is stated here, it **MUST** introduce additional compatibility predicates inside its Interop SpecID component and **MUST** remain checkable under NumericModel.

6.2 Interop Component (EY24)

EY24 declares the following baseline composition forms (EY22 C1–C5):

$$\text{Interop}_{\text{EY24}} := \{\text{C1}, \text{C3}, \text{C4}\},$$

i.e. functional / algebraic composition, coarse-grain reduction, and superoperator action where applicable. Transport-by-morphism (C2) is supported only via EY28 predicates below (it is not assumed by default).

6.3 Compatibility Predicates

All compatibility predicates below are deterministic booleans under the declared NumericModel(s).

VPO $\leftrightarrow$ CSF: Validity Penalty with Canonical Spectral Filter

The two common workflow semantics are:

(S1) additive integration: $H' := H + \lambda V$, $F_*(H')$ (filter on validity-augmented Hamiltonian)

(S2) multiplicative weighting: $F_*(H) V$, $V F_*(H)$ (filter then apply penalty or vice versa).

EY24 does not prescribe which semantic is intended; the workflow MUST declare it.

Predicate $\text{Compat}_{24,23}^{(1)}$ (shared space and domains).

$$\text{Compat}_{24,23}^{(1)} := (\mathcal{H}_H = \mathcal{H}_V) \wedge (\text{Dom}(H) \cap \text{Dom}(V) \text{ compatible under Calc}).$$

Predicate $\text{Compat}_{24,23}^{(2)}$ (adjoint and calculus alignment).

$$\text{Compat}_{24,23}^{(2)} := (\text{AdjointSpec}_{\text{EY24}} \text{ does not contradict } \text{Calc}_{\text{EY23}}).$$

In particular, if the workflow uses $H + \lambda V$, the calculus MUST define addition on the chosen representation and truncation policy.

Predicate $\text{Compat}_{24,23}^{(3)}$ (numeric model alignment).

$$\text{Compat}_{24,23}^{(3)} := (\text{NumericModel}_{\text{EY24}} = \text{NumericModel}_{\text{EY23}}) \vee (\text{declared deterministic coercion}).$$

Predicate $\text{Compat}_{24,23}^{(4)}$ (semantic declaration).

$$\text{Compat}_{24,23}^{(4)} := (\text{workflow declares S1 or S2 (and ordering if S2)}).$$

Compatibility status.

$$\text{Compat}(\text{EY24}, \text{EY23}) := \text{Compat}_{24,23}^{(1)} \wedge \text{Compat}_{24,23}^{(2)} \wedge \text{Compat}_{24,23}^{(3)} \wedge \text{Compat}_{24,23}^{(4)}.$$

If TRUE, mark OK* because semantics/ordering are workflow-dependent.

VPO $\leftrightarrow$ LIOUV: Validity Under Audit-Native Evolution

EY25 defines an audit-native evolution (superoperator) $\mathcal{L}_{\text{audit}}$. Common semantics:

(E1) penalty-in-Hamiltonian: $H' := H + \lambda V \Rightarrow \mathcal{L}_{\text{audit}}(H')$ is used,

(E2) penalty-as-observable: $\rho \mapsto \mathcal{L}_{\text{audit}}(\rho)$ and score $\text{tr}(\rho V)$ is audited.

Predicate $\text{Compat}_{24,25}^{(1)}$ (action space match).

$$\text{Compat}_{24,25}^{(1)} := (\mathcal{L}_{\text{audit}} \text{ acts on the same state/observable space where } V \text{ is defined}).$$

Predicate $\text{Compat}_{24,25}^{(2)}$ (deterministic ordering).

$$\text{Compat}_{24,25}^{(2)} := (\text{workflow declares E1 or E2, and the exact evaluation order}).$$

Predicate $\text{Compat}_{24,25}^{(3)}$ (**numeric consistency**).

$\text{Compat}_{24,25}^{(3)} := (\text{NumericModel}_{\text{EY24}} = \text{NumericModel}_{\text{EY25}}) \vee (\text{declared deterministic coercion}).$

Compatibility status.

$$\text{Compat}(\text{EY24}, \text{EY25}) := \text{Compat}_{24,25}^{(1)} \wedge \text{Compat}_{24,25}^{(2)} \wedge \text{Compat}_{24,25}^{(3)}.$$

If TRUE, mark OK*.

VPO $\leftrightarrow$ CGR: Coarse-Grain Reduction of Validity

EY29 defines a coarse-grain operator R . The natural compositions are:

$$R(V), \quad V(R(\cdot)) \text{ (if } V \text{ is lifted to reduced space),} \quad R(C) \text{ and } R(C)^\dagger R(C).$$

Note that in general:

$$R(C^\dagger C) \neq (R(C))^\dagger R(C),$$

unless a specific reduction semantics is declared.

Predicate $\text{Compat}_{24,29}^{(1)}$ (**reduction semantics declaration**).

$$\text{Compat}_{24,29}^{(1)} := (\text{workflow declares which of } R(V) \text{ or } (R(C))^\dagger R(C) \text{ is intended}).$$

Predicate $\text{Compat}_{24,29}^{(2)}$ (**adjoint preservation or replacement**).

$\text{Compat}_{24,29}^{(2)} := (\text{either } R \text{ preserves the AdjointSpec, or a reduced AdjointSpec is declared deterministically}).$

Predicate $\text{Compat}_{24,29}^{(3)}$ (**invariant preservation**). Let Inv_{VPO} be the invariant list declared by EY24 (positivity, bounded penalty growth, locality class, etc.). Then:

$$\text{Compat}_{24,29}^{(3)} := \forall I \in \text{Inv}_{\text{VPO}}^{\text{preserve}} : I(V) \Rightarrow I(R(V)),$$

within declared tolerances and under the declared reduction semantics.

Compatibility status.

$$\text{Compat}(\text{EY24}, \text{EY29}) := \text{Compat}_{24,29}^{(1)} \wedge \text{Compat}_{24,29}^{(2)} \wedge \text{Compat}_{24,29}^{(3)}.$$

If TRUE, mark OK* (because semantics must be declared).

VPO $\leftrightarrow$ INTW: Transporting Validity Across Regimes

EY28 defines an intertwiner $T_{1 \rightarrow 2}$. Validity may need to be transported from regime 1 to regime 2. Two canonical transport semantics are allowed:

$$(T1) \text{ unitary-like: } V_2 := TV_1 T^{-1}, \quad (T2) \text{ isometric-like: } V_2 := TV_1 T^\dagger.$$

The choice MUST be declared.

Predicate $\text{Compat}_{24,28}^{(1)}$ (**mapping semantics**).

$\text{Compat}_{24,28}^{(1)} :=$ (workflow declares T1 or T2 and provides the required inverses/adjoints deterministically).

Predicate $\text{Compat}_{24,28}^{(2)}$ (**adjoint coherence**).

$\text{Compat}_{24,28}^{(2)} :=$ (AdjointSpec_1 , AdjointSpec_2 and T are mutually consistent under EY28 semantics).

In particular, if T2 is used, the adjoint of T must be computed using AdjointSpec_2 .

Predicate $\text{Compat}_{24,28}^{(3)}$ (**positivity preservation**).

$\text{Compat}_{24,28}^{(3)} := (V_1 \succeq 0) \Rightarrow (V_2 \succeq 0)$ under the declared mapping semantics and NumericModel .

Compatibility status.

$$\text{Compat}(\text{EY24}, \text{EY28}) := \text{Compat}_{24,28}^{(1)} \wedge \text{Compat}_{24,28}^{(2)} \wedge \text{Compat}_{24,28}^{(3)}.$$

If TRUE, mark OK*.

6.4 Mini Compatibility Matrix Row (EY24 Perspective)

This minimal row is informative and must be backed by predicates above:

	EY23	EY24	EY25	EY26	EY27	EY28	EY29	EY30
EY24	OK*	N/A	OK*	N/A	OK*	OK*	OK*	OK*

Notes (informational).

- EY24×EY27 is OK* because commutant generator sets can define commutator constraints (Class C2).
- EY24×EY30 is OK* because validity may export as observable penalties; mapping must be declared.

6.5 Interop Failure Codes (Required)

When an interop predicate fails, the verifier MUST return a deterministic failure code, at minimum:

- **FAIL__DOMAIN__MISMATCH**
- **FAIL__CALC__MISMATCH**
- **FAIL__NUMERICMODEL__MISMATCH**
- **FAIL__ADJOINT__SEMANTICS**
- **FAIL__INVARIANT__NOT__PRESERVED**
- **FAIL__COMPAT__PREDICATE**

6.6 Chapter Summary

This chapter provided minimal EY24 interoperability declarations, defining concrete compatibility predicates for composing $V = C^\dagger C$ with EY23 filtering, EY25 audit-native evolution, EY29 coarse-grain reduction, and EY28 transport across regimes. It also provided an EY24 mini compatibility-matrix row and fixed the required failure codes for interoperability failures.

Chapter 7 next defines the baseline VPO-0 reference profile (a concrete SpecID instance with fixed AdjointSpec, constraint class parameters, feasibility thresholds, canonicity mode, and certificate requirements).

7 VPO-0 Reference Profile and Example SpecID Instance

7.1 Purpose

Chapters 1–6 define the general EY24 artifact. This chapter provides a concrete, audit-friendly baseline configuration called **VPO-0**. VPO-0 is designed to be:

- minimal (few degrees of freedom),
- deterministic (verifiable without hidden tuning),
- composable with EY23 and EY25 by explicit semantics,
- and practical as a default validity penalty primitive for Vol. No. IV.

Rule. A VPO-0 instance **MUST** be fully specified by explicit constants (or hash-committed constants) and **MUST NOT** require instance-specific tuning.

7.2 VPO-0 Summary (Normative)

VPO-0 fixes:

- constraint class: C1 (Linear Constraint Stack),
- selection: disabled by default ($\mathcal{J}_C = \emptyset$); C is provided but must be admissible,
- feasibility: domain/adjoint consistency + structural bounds + structural identity $V = C^\dagger C$,
- canonicity mode: not applicable (because selection disabled); TieBreak still defined for future profiles,
- certificate: EY22-CCE with required checks V1–V7 (and V10 if interop claimed).

7.3 VPO-0 Constraint Class (C1) Details

Admissible Family $\mathcal{A}_{C, \text{VPO-0}}$

$$\mathcal{A}_{C, \text{VPO-0}} := \left\{ C = \begin{bmatrix} C_1 \\ \vdots \\ C_r \end{bmatrix} \mid 1 \leq r \leq r_{\max}, \text{ StructID} = \text{StructID}_0, \Theta_i \in \mathcal{G}_\Theta \right\}.$$

Fixed constants (VPO-0). The following **MUST** be fixed in SpecID:

$$r_{\max}, \quad \text{StructID}_0, \quad \mathcal{G}_\Theta, \quad \text{OrderRule}.$$

Recommended baseline constants (informational). Example placeholders until locked:

$$r_{\max} = 64, \quad \text{StructID}_0 = \text{sparse-banded},$$

with a coefficient grid $\mathcal{G}_\Theta$ given by fixed-point values with step Δ_Θ and bound B_Θ . (Replace with explicit locked constants in SpecID.)

Structural Descriptor StructID_0

VPO-0 requires a deterministic structure class for each C_i , for example:

- band width b (fixed),
- maximum nnz per row (fixed),
- and deterministic placement/ordering rule.

All structural constants MUST be fixed in SpecID or hash-committed.

7.4 VPO-0 AdjointSpec (Baseline)

VPO-0 defines a minimal adjoint convention suitable for finite truncations.

AdjointSpec₀ (normative structure).

$\text{AdjointSpec}_0 := (\text{InnerProductType}, W, \text{AdjointRule}, \text{TruncAdjointRule}, \text{RoundingPolicy}).$

Default baseline (informational). A pragmatic baseline is:

- inner product: standard $\langle x, y \rangle = x^\dagger y$ (so $W = I$),
- adjoint rule: conjugate-transpose at truncation level,
- rounding policy: deterministic float or fixed-point as declared in NumericModel.

If a weighted inner product is used, the weight operator W MUST be explicitly provided or hash-committed.

7.5 VPO-0 Calc (Baseline)

VPO-0 uses truncation calculus only insofar as it needs deterministic finite checks.

Calc₀ (normative structure).

$\text{Calc}_0 := (\text{Representation}, \text{Truncation}, \text{MultiplyRule}, \text{AccumulateRule}).$

Baseline requirements.

- Representation: canonical finite operator form for each required N .
- MultiplyRule: deterministic matrix multiplication (ordering, rounding).
- AccumulateRule: deterministic reduction and rounding for forming $C_N^\dagger C_N$.

7.6 VPO-0 Feasibility Constraints $\mathcal{C}_{C,\text{VPO-0}}$

VPO-0 enables the following feasibility checks:

$$\mathcal{C}_{C,\text{VPO-0}} := \mathcal{C}_{\text{dom}} \wedge \mathcal{C}_{\text{struct}} \wedge \mathcal{C}_{\text{adj}}.$$

Optional constraint (off by default). Bounded penalty growth $\mathcal{C}_{\text{pen}}$ MAY be enabled in a future VPO profile; if enabled it MUST be fixed in SpecID.

7.7 VPO-0 NumericModel (Minimal Deterministic Profile)

VPO-0 requires a deterministic NumericModel. A minimal profile MUST include:

- parameter encoding for C : fixed-point canonical bytes,
- evaluation arithmetic: deterministic float or fixed-point (declared),
- truncation schedule $N_1 < \dots < N_L$ if truncation checks require multiple N ,
- tolerances for adjoint check and multiplication check,
- deterministic aggregation order.

Recommended (informational). Use exact fixed-point for C coefficients and deterministic float for large matrix products, with transcript hashing.

7.8 VPO-0 Certificate Requirements

VPO-0 requires at minimum the checks:

- V1 (CCE structural),
- V2 (SpecID consistency),
- V3 (no hidden inputs),
- V4 (membership),
- V5 (feasibility),
- V6 (adjoint semantics),
- V7 (identity $V = C^\dagger C$).

Interop checks (V10) are required only if interop is claimed.

7.9 Reference SpecID Instance (VPO-0 Template)

The following is a template for a VPO-0 SpecID instance. Numeric constants are placeholders until fixed.

```
SpecID_EY24_VP00 :=  
(  
  ArtifactType = VP0,  
  Core = (C),  
  AdjointSpec = {
```

```

    inner_product: "standard" or "weighted",
    weight_operator_W: "I" or Hash(W),
    adjoint_rule: "conj-transpose (at truncation)",
    rounding: "det-float-or-fixed (declare)"
},
Calc = {
    representation: "finite-operator-form",
    truncation: { basis: "...", cut_rule: "...", boundary_rule: "..." },
    multiply_rule: "det-matmul",
    accumulate_rule: "sequential"
},
A_C = {
    class: "C1-linear-stack",
    r_max: 64,
    StructID: "sparse-banded",
    coeff_grid: { step: 0.01, bound: 2.0 },
    order_rule: "canonical-row-major"
},
J_C = null,
C_C = {
    dom: "required",
    struct: "required",
    adjoint: "required",
    stab: null,
    pen: null
},
TieBreak = { type: "lexmin-Enc(C)" },
NumericModel = {
    params_encoding: "fixed-point",
    eval_encoding: "det-float-or-fixed (declare)",
    truncation_schedule: { N: [256, 512, 1024] },
    tolerances: { eps_adj: 1e-12, eps_mul: 1e-10 },
    aggregation: "sequential"
},
Cert = {
    schema: "EY22-CCE",
    required_checks: ["V1", "V2", "V3", "V4", "V5", "V6", "V7"]
},
Interop = {
    allowed_forms: ["C1", "C3", "C4"],
    compat: {
        EY23: "Compat_24_23",
        EY25: "Compat_24_25",
        EY28: "Compat_24_28",
        EY29: "Compat_24_29"
    }
},
Version = "VID-2026-02"
)

```

Rule. Once VPO-0 is published, all placeholders **MUST** be replaced by fixed values (or hash commitments) and the Version **MUST** be treated as immutable for that meaning.

7.10 Chapter Summary

This chapter defined VPO-0, a concrete baseline profile for EY24 designed for audit-friendly validity enforcement. It fixed the default constraint class (C1 linear constraint stack), baseline adjoint semantics, deterministic calculus requirements, feasibility predicates, certificate checks, and a reference SpecID instance template.

Next. With EY24 complete, Vol. No. IV proceeds to EY25 (Audit-Native Liouvillian) which defines canonical, replayable evolution operators that propagate validity penalties and compose with EY23 filtering and EY29 reduction under the interop predicates declared in Chapters 8 and 6.

8 Publication Boundary, Non-Claims, and Licensing Intent

8.1 Purpose

Vol. No. IV is written in a QFC normative style: it defines operator artifacts as checkable objects. That style is powerful, but it can be misread as a claim about physics, security, or correctness of a chosen constraint set. This chapter fixes the publication boundary and explicitly states what EY24 does and does not claim.

8.2 Publication Boundary

Rule (what must be public). An EY24 publication **MUST** include, at minimum:

- the full SpecID tuple for the EY24 instance,
- the canonical encoding (or hash commitment) of C ,
- the canonical encoding (or hash commitment) of V ,
- the AdjointSpec payload (or hash commitment),
- the NumericModel payload (including truncation schedule if used),
- and the certificate π_{VPO} with required check identifiers.

Rule (no hidden constraints). If a published claim depends on validity, then all constraints contributing to that validity **MUST** be expressible in C (and therefore auditable through $V = C^\dagger C$). Human-only or undocumented constraints are forbidden.

Rule (commitment for external artifacts). If the instance references external objects (generator sets, geometry rules, basis specifications, code snapshots, transcripts), those objects **MUST** be committed by hash according to EY22 Appendix A.1.

8.3 Non-Claims

EY24 explicitly does *not* claim the following:

NC1: Physical Correctness

EY24 does not claim that a published C corresponds to a physically correct law of nature, nor that “validity” as encoded by V is a true property of the world. EY24 only claims:

- the artifact is well-defined under SpecID,
- the verifier checks pass deterministically,
- and $V = C^\dagger C$ is correctly formed.

NC2: Security Guarantees Beyond Verifier Checks

EY24 does not claim cryptographic security, resistance to adversarial manipulation, or soundness beyond what is explicitly covered by deterministic verifier checks and interop predicates.

NC3: Universality of Constraint Choice

Even when canonical selection is enabled ($\mathcal{J}_C \neq \emptyset$), the chosen $C_\star$ is canonical only *relative to SpecID*. It is not a universal best constraint.

NC4: Equivalence of Workflow Semantics

EY24 does not claim that additive integration $H + \lambda V$, multiplicative weighting, projection, or penalty scoring are equivalent. Any workflow MUST declare its chosen semantic mode and ordering; otherwise composition is invalid.

NC5: Completeness of Validity

EY24 does not claim that all relevant validity conditions for a given application can be expressed as $C^\dagger C$, only that when they *are* expressed that way, they become audit-native and checkable.

8.4 Responsible Use and Misuse Boundary

Rule. EY24 SHOULD NOT be used to present speculative validity claims as proven facts. The correct use is:

- publish C and V ,
- publish the verifier checks and outcomes,
- and allow third parties to reproduce and critique the constraint choice.

Misuse examples (non-normative).

- Publishing a “validity” claim while keeping the actual constraints private.
- Re-labeling parameter tuning as “validity” without bounds and canonical encoding.
- Claiming security or correctness beyond the explicit checks.

8.5 Licensing Intent (QFC-Consistent)

EY24 is part of the QFC operator suite. The intent is that the *specification* remains public and auditable, while enabling controlled licensing of implementations or derivative operator catalogs if desired.

Normative statement of intent.

- The definitions, SpecID schemas, and certificate/verifier requirements are intended to be publicly readable and reusable.
- Implementations may be licensed under terms chosen by the project (open, dual-license, or commercial), provided they preserve auditability and do not remove required publication boundary elements.
- Trademark and naming conventions (QFC, Vol. No. IV, EY-IDs, artifact names) may be governed separately from the text license.

Rule (no audit stripping). No license arrangement may remove the requirement to publish SpecID, commitments, and certificates for any claim that relies on EY24.

8.6 Chapter Summary

This chapter fixed the publication boundary for EY24 and listed explicit non-claims: no claims of physical correctness, security beyond verifier checks, universality of constraint choice, or equivalence of workflow semantics. It also stated a QFC-consistent licensing intent: keep the specification auditable and public, while allowing implementation licensing provided auditability is preserved.

Next. With EY24 complete, Vol. No. IV proceeds to EY25 (Audit-Native Liouvillian), which defines deterministic evolution operators that can propagate validity penalties and produce replayable audit trails for composed workflows.

=====
tionDoc wrapper ID: anl ===== Sec-

EY25: Audit-Native Liouvillian $\mathcal{L}_{\text{audit}}$

Canonical Operator Artifacts for QFC

Enter Yourname

Abstract

This whitepaper defines the Audit-Native Liouvillian $\mathcal{L}_{\text{audit}}$, a canonical superoperator artifact for deterministic, replayable evolution in QFC workflows. Unlike informal “dynamics” that depend on hidden solver choices, $\mathcal{L}_{\text{audit}}$ is specified as an auditable contract: its generator form, admissible parameter family, time discretization semantics, truncation schedule, and NumericModel are bound by SpecID and verified by certificate.

The core requirement is third-party verifiability. Independent verifiers MUST be able to reproduce the same evolution map (and the same audit transcript) from public inputs alone, including deterministic ordering, rounding, tie-break behavior, and failure codes. The artifact supports both state-side evolution ($\rho \mapsto \mathcal{L}_{\text{audit}}(\rho)$) and observable-side evolution ($g \mapsto \mathcal{L}_{\text{audit}}^*(g)$), with explicit declaration of the chosen semantics.

$\mathcal{L}_{\text{audit}}$ is designed to compose with the Canonical Spectral Filter $F_*(H)$ (EY23) and the Validity Penalty Operator $V = C^\dagger C$ (EY24), enabling workflows where filtering, validity, and evolution are all audit-native: not narrative, but checkable operator-level artifacts with explicit interoperability predicates.

1 Motivation, Scope, and Design Goals

1.1 Motivation: Dynamics Are Where Ambiguity Hides

In operator-centric systems, the most damaging ambiguity rarely sits in the definition of an operator. It sits in *how the operator is used over time*. Two implementations may claim to evolve the same state with the same generator, yet differ due to:

- solver choices (adaptive steps, hidden tolerances, iteration caps),
- non-deterministic reductions (parallel summation order, GPU warp scheduling),
- uncommitted randomness (seeds, stochastic sampling),
- and implicit boundary/truncation behavior.

Such ambiguity collapses auditability. A third party cannot reproduce results, and “correctness” becomes a social claim.

EY25 defines an explicit remedy: $\mathcal{L}_{\text{audit}}$, an **audit-native** Liouvillian—a superoperator whose meaning is bound by SpecID, whose evaluation is deterministic under a declared NumericModel, and whose execution produces a replayable transcript.

1.2 What This Whitepaper Defines

EY25 defines:

- the Audit-Native Liouvillian artifact $\mathcal{L}_{\text{audit}}$,
- admissible generator families and canonical parameterizations,
- evolution semantics (state-side vs observable-side) and their SpecID binding,
- deterministic discretization and truncation protocols,
- certificate payload and verifier checks for replay and acceptance,
- and minimal interoperability declarations with EY23 (CSF) and EY24 (VPO), and optionally EY29/EY28.

Unless explicitly overridden, suite conventions (SpecID, NumericModel, certificate schema, interop vocabulary) follow EY22.

1.3 Scope

In-scope.

- Deterministic superoperator evolution under a declared generator and protocol.
- Replayable transcripts: the verifier can confirm the published evolution is the one implied by SpecID.
- Explicit failure modes for non-convergence, domain mismatch, or protocol violations.
- Composition with EY23/EY24 via explicit ordering and compatibility predicates.

Out-of-scope.

- Claims that the chosen dynamics correspond to the true physical evolution of a system.
- Claims of universal stability, security, or robustness beyond declared checks.
- “Implicit solvers”: any method that depends on hidden adaptive behavior or uncommitted heuristics.
- Full numerical analysis of all integrators; EY25 standardizes auditability, not the entire theory of time stepping.

1.4 Interpretation: What $\mathcal{L}_{\text{audit}}$ Does

$\mathcal{L}_{\text{audit}}$ is a superoperator that advances either:

- a state object (density operator) ρ , or
- an observable/statistic g in a dual picture.

State-side (Schrödinger picture).

$$\rho \mapsto \mathcal{E}_{\Delta t}(\rho), \quad \text{where } \mathcal{E}_{\Delta t} \text{ is induced by } \mathcal{L}_{\text{audit}}.$$

Observable-side (Heisenberg picture).

$$g \mapsto \mathcal{E}_{\Delta t}^*(g).$$

Rule. The SpecID MUST declare which picture is used. A workflow that fails to declare the picture MUST be rejected as ambiguous.

1.5 Design Goals (Normative)

G1: Auditability by Construction

The evolution map MUST be reproducible from public inputs alone. Every choice that can change the output (step schedule, truncation schedule, rounding, aggregation order) MUST be bound by SpecID or committed by hash.

G2: Deterministic Replay and Transcript

Running the same evolution under the same SpecID and inputs MUST yield the same transcript and outputs across verifiers. Transcript hashing MUST be supported as a publication primitive.

G3: Explicit Generator Semantics

$\mathcal{L}_{\text{audit}}$ MUST be defined via an explicit generator family (e.g. commutator form, Lindblad-like form, or declared operator-field form), not via a black-box simulator.

G4: Controlled Dissipation (Disipace jen do povolených subprostorů)

If dissipation is present, it MUST be explicitly restricted to declared subspaces or channels. No hidden “numerical damping” is permitted.

G5: Composability With Validity and Filtering

$\mathcal{L}_{\text{audit}}$ SHOULD compose with:

- EY24 validity penalties $V = C^\dagger C$ (validity propagation),
- EY23 spectral filters $F_\star(H)$ (probe/evolve/probe workflows),

under explicit ordering and interop predicates (EY22 Chapter 6).

G6: Conservative Baseline Profile

EY25 MUST provide a baseline profile (LIOUV-0) that is simple enough to verify by third parties, even if slower. Optimized profiles may exist, but the baseline sets the canonical meaning for “audit-native evolution”.

1.6 Chapter Summary

This chapter motivated $\mathcal{L}_{\text{audit}}$ as a remedy to solver ambiguity and defined EY25 scope and design goals: auditability by construction, deterministic replay with transcript, explicit gen-

erator semantics, controlled dissipation, and composability with EY23/EY24 under declared interoperability rules.

Chapter 2 next gives the normative artifact definition and SpecID binding for EY25: generator families, admissible parameterizations, discretization semantics, NumericModel requirements, certificate payload, and verifier checks.

2 Normative Definition and SpecID Binding

2.1 Conformance to EY22

EY25 inherits Vol. No. IV conventions from EY22:

- SpecID binding (EY22 Chapter 3),
- NumericModel determinism (EY22 Chapter 4),
- certificate schema and verifier checks (EY22 Chapter 5, Appendix A.1),
- interoperability vocabulary (EY22 Chapter 6).

Any deviation **MUST** be explicitly stated in $\text{SpecID}_{\text{EY25}}$.

2.2 Core Objects

Core. The EY25 artifact is a superoperator (or a generator that induces a superoperator) acting on a declared space.

An EY25 instance consists of:

- a declared evolution picture $\text{Pic} \in \{\text{state}, \text{observable}\}$,
- a generator specification $\mathbb{G}$ (which may reference operators such as H , V , or a channel set),
- a deterministic discretization protocol producing a step map $\mathcal{E}_{\Delta t}$,
- and the audit transcript contract that binds execution to public inputs.

Notation. We use $\mathcal{L}_{\text{audit}}$ to denote the *auditable evolution contract*. Depending on Pic , it induces either:

$$\rho \mapsto \mathcal{E}_{\Delta t}(\rho) \quad \text{or} \quad g \mapsto \mathcal{E}_{\Delta t}^*(g).$$

2.3 Artifact Definition

Definition 2.1 (Audit-Native Liouvillian Artifact). An EY25 artifact instance is a tuple:

$$\mathcal{L}_{\text{audit}} := (\text{Pic}, \mathbb{G}, \text{Disc}, \text{Trunc}, \text{NumericModel}),$$

together with a certificate π_{LIOUV} that proves:

- $\mathbb{G}$ is admissible under the declared generator family $\mathcal{A}_{\mathbb{G}}$,
- Disc deterministically defines $\mathcal{E}_{\Delta t}$ (or a multi-step schedule),
- Trunc deterministically defines finite representations at required scales,
- and the published transcript commitments match replay under NumericModel .

Rule (no implicit solver). No hidden adaptivity is permitted. Any step-size schedule, stopping rule, or tolerance is part of SpecID.

2.4 Generator Families (High-Level)

EY25 standardizes three generator families. A SpecID MUST select exactly one family, or define a finite union with deterministic tie-break across families.

Family G1: Commutator (Unitary) Generator

Form. State-side:

$$\mathbb{G}_{G1} : \quad \mathcal{L}(\rho) := -i[H, \rho].$$

Observable-side:

$$\mathcal{L}^*(g) := i[H, g].$$

Use. Audit-native unitary evolution in a declared truncation calculus.

Family G2: Lindblad-Like (Controlled Dissipation) Generator

Form. State-side:

$$\mathcal{L}(\rho) := -i[H, \rho] + \sum_{k=1}^K \left(L_k \rho L_k^\dagger - \frac{1}{2} \{L_k^\dagger L_k, \rho\} \right),$$

with $\{A, B\} = AB + BA$.

Controlled dissipation contract. Dissipation is permitted only via the declared channel set $\{L_k\}$ and only into declared subspaces/channels.

Family G3: Audit-Penalty (Validity-Coupled) Generator

Idea. Couple evolution to validity enforcement via EY24:

$$V := C^\dagger C.$$

EY25 allows a canonical coupling term, e.g. additive penalty in the generator or validity-weighted damping.

Example forms (non-exclusive).

- Hamiltonian augmentation: $H' := H + \lambda V$, then use G1 or G2 with H' .
- Validity damping channel: include a channel $L_V := \sqrt{\gamma} C$ inside G2.

Rule. Any reference to EY24 MUST include compatibility predicates in Interop and MUST bind to EY24 SpecID.

2.5 Discretization Semantics Disc

Disc defines how the generator becomes a step map $\mathcal{E}_{\Delta t}$.

Disc Component (required)

$\text{Disc} := (\text{StepType}, \Delta t, \text{Schedule}, \text{Order}, \text{RoundingPolicy}, \text{StopRule}).$

Rule. Disc MUST be fully explicit. Δt and the schedule (number of steps, time horizon) MUST be fixed or derived deterministically from committed public inputs.

Allowed Step Types (Baseline Set)

EY25 permits at minimum:

- **D1: Euler** (first order): $\rho_{n+1} = \rho_n + \Delta t \mathcal{L}(\rho_n)$
- **D2: RK2 / midpoint** (second order) with explicit formula and fixed evaluation order
- **D3: Trotter product** for commutator generators with fixed factorization rule

Normative note. Higher-order integrators MAY be defined by future profiles, but must remain deterministic and transcript-replayable.

2.6 Truncation Semantics Trunc

Trunc binds how infinite objects are represented finitely for verification.

Trunc Component (required).

$\text{Trunc} := (\text{Basis}, \text{CutRule}, \text{BoundaryRule}, \{N_\ell\}, \text{CrossNPolicy}).$

This reuses EY22 truncation vocabulary.

Rule. If truncation is used, the required truncation levels $\{N_\ell\}$ MUST be fixed in SpecID and MUST be used consistently for transcript commitments.

2.7 Transcript Contract

Audit-native evolution requires publishing a transcript commitment.

Transcript. A transcript is an ordered sequence of scalar commitments (or hashes) that bind the evolution:

$$\text{Tx} := (\text{Tx}_0, \text{Tx}_1, \dots, \text{Tx}_T),$$

where each Tx_n commits to the state/observable representation at step n (or to declared summaries).

Rule. The SpecID MUST declare what is committed per step:

- full state hash,
- or canonical spectral summaries,
- or bucketed invariants,

and the canonical serialization used.

Replay rule. If replay under NumericModel does not reproduce the transcript commitments, the verifier MUST return **FAIL_NONDETERMINISM**.

2.8 SpecID for EY25

ArtifactType. For EY25:

$$\text{ArtifactType} = \text{LIOUV}.$$

SpecID components (EY25 binding). The full SpecID tuple is:

$$\text{SpecID}_{\text{EY25}} := (\text{LIOUV}, \text{Pic}, \mathbb{G}, \mathcal{A}_{\mathbb{G}}, \text{Disc}, \text{Trunc}, \text{NumericModel}, \text{TranscriptSpec}, \text{Cert}, \text{Interop}, \text{Version}).$$

Pic (required)

$\text{Pic} \in \{\text{state}, \text{observable}\}$ MUST be declared.

$\mathbb{G}$ and $\mathcal{A}_{\mathbb{G}}$ (required)

$\mathbb{G}$ is the generator instance; $\mathcal{A}_{\mathbb{G}}$ defines the admissible family and membership checks. Even if $\mathbb{G}$ is provided, $\mathcal{A}_{\mathbb{G}}$ MUST exist to prevent arbitrary hidden generators.

Disc and Trunc (required)

As defined above; fully explicit and deterministic.

TranscriptSpec (required)

TranscriptSpec MUST define:

- what per-step commitments are required,
- the hashing/serialization rules,
- and any permitted compression (e.g. summary-only) and its invariants.

NumericModel (required)

NumericModel MUST include rounding/ordering rules sufficient to make Disc, Trunc, and transcript commitments deterministic.

Cert, Interop, Version (required)

Cert binds verifier checks; Interop binds compatibility predicates; Version binds meaning and MUST change if any component changes.

2.9 Deterministic Outcomes

EY25 verification MUST return:

$$\{\text{ACCEPT}, \text{REJECT}\} \cup \{\text{FAIL}_*\},$$

and MUST support at minimum:

- **FAIL_DOMAIN_MISMATCH**
- **FAIL_DISC_MISMATCH**
- **FAIL_TRUNC_MISMATCH**
- **FAIL_TRANSCRIPT_MISMATCH**
- **FAIL_NONDETERMINISM**
- **FAIL_CONSTRAINT**

2.10 Chapter Summary

This chapter defined EY25 as an audit-native evolution artifact: a declared generator family, explicit discretization semantics, explicit truncation semantics, and a transcript contract for deterministic replay. It fixed the SpecID binding for EY25 and introduced minimal failure codes.

Chapter 3 next defines admissible generator families $\mathcal{A}_{\mathbb{G}}$ in detail: canonical parameterizations, membership checks, controlled dissipation restrictions, and baseline profiles for verifiable evolution.

3 Admissible Generator Families $\mathcal{A}_{\mathbb{G}}$: Canonical Classes for $\mathbb{G}$

3.1 Purpose of $\mathcal{A}_{\mathbb{G}}$

The generator specification $\mathbb{G}$ is the semantic core of $\mathcal{L}_{\text{audit}}$. If $\mathbb{G}$ is underspecified, auditability collapses: the same name can hide different channel sets, different coupling constants, or different boundary behavior. Therefore $\mathcal{A}_{\mathbb{G}}$ is normative: it defines what generator instances are admissible and how they are encoded and checked.

Rule. A generator $\mathbb{G}$ is admissible only if:

- it belongs to a declared generator class,
- it has a canonical parameterization (encoding),
- its parameter bounds are explicit,
- and membership is checkable deterministically under `NumericModel`.

3.2 Canonical Representation and Encoding

Rule (canonical encoding). Every $\mathbb{G} \in \mathcal{A}_{\mathbb{G}}$ MUST have a canonical representation that can be serialized and compared lexicographically (EY22 Appendix A.1). This canonical encoding MUST be used by `TieBreak` whenever a selection rule is enabled in an EY25 profile.

Required encoding fields. The canonical encoding MUST include:

- `GenClassID` identifying the generator family (G1/G2/G3),
- a parameter tuple $\theta_{\mathbb{G}}$ in a fixed order,
- references (or commitments) to any operators used (e.g. $H, V, C, \{L_k\}$),
- explicit bounds and discretization-relevant constants.

3.3 Generator Classes

EY25 defines three canonical generator classes. A SpecID MUST select exactly one, or define a finite union with deterministic tie-break across classes.

Class G1: Commutator (Unitary) Generator

Form.

$$\mathbb{G}_{G1} := (H), \quad \mathcal{L}(\rho) = -i[H, \rho], \quad \mathcal{L}^*(g) = i[H, g].$$

Canonical parameters.

$$\theta_{G1} := (\text{HRef}, \text{Pic}, \text{UnitsSpec}, \text{BoundaryPolicy}).$$

HRef MUST be either:

- a canonical encoding of H in the declared truncation calculus, or
- a hash commitment to the full H object, together with a deterministic truncation map $H \mapsto H_N$.

Membership restrictions.

- H MUST be declared self-adjoint under the chosen AdjointSpec (or an explicit HermitSpec).
- If truncation is used, the sequence H_N MUST be deterministically defined.

Use cases. Audit-native unitary evolution and deterministic replay baselines.

Class G2: Lindblad-Like Generator with Controlled Dissipation

Form.

$$\mathbb{G}_{G2} := (H, \{L_k\}_{k=1}^K),$$

$$\mathcal{L}(\rho) := -i[H, \rho] + \sum_{k=1}^K \left(L_k \rho L_k^\dagger - \frac{1}{2} \{L_k^\dagger L_k, \rho\} \right).$$

Canonical parameters.

$$\theta_{G2} := (\text{HRef}, \{L_k\}_{k=1}^K, K, \text{ChannelOrderRule}, \text{DissipationPolicy}).$$

Controlled dissipation policy (normative). DissipationPolicy MUST specify:

- allowed channel families (e.g. local stencils only, sparse-only, validity-derived only),
- allowed target subspaces (a declared projector P or index region),
- and a deterministic bound on total dissipation strength (e.g. $\sum_k \|L_k\|^2 \leq B_L$).

Membership restrictions.

- K MUST be finite and bounded: $1 \leq K \leq K_{\max}$.
- Each channel L_k MUST belong to a declared admissible family (local/stencil/sparse class) with explicit bounds.
- Channel ordering MUST be canonical (e.g. lexicographic by encoding), to prevent permutation ambiguity.

Use cases. Audit-native dissipative dynamics with explicit, checkable dissipation channels.

Class G3: Validity-Coupled (Penalty/Audit) Generator

Idea. Couple evolution to validity enforcement as a first-class contract. The generator references EY24 artifacts and binds to their SpecID.

Canonical instances. EY25 permits two canonical coupling patterns:

(G3a) Hamiltonian augmentation.

$$H' := H + \lambda V, \quad V = C^\dagger C,$$

then use G1 or G2 with H' .

(G3b) Validity-induced damping channel. Introduce a channel derived from C :

$$L_V := \sqrt{\gamma} C,$$

and include L_V in the Lindblad channel set.

Canonical parameters.

$$\theta_{G3} := (\text{HRef}, \text{EY24Ref}, \text{CouplingType} \in \{\text{aug}, \text{damp}\}, \lambda, \gamma, \text{OrderRule}).$$

Membership restrictions (normative).

- EY24Ref MUST bind to a full EY24 SpecID and certificate hash (or its canonical commitment).
- Coupling constants λ, γ MUST be bounded and quantized (fixed-point grid) to prevent tuning smuggling.
- If CouplingType = aug, then addition $H + \lambda V$ MUST be defined in the declared calculus and truncation policy.
- If CouplingType = damp, then C MUST be admissible under EY24 and channel inclusion MUST satisfy G2 dissipation bounds.

Use cases. Validity-propagating dynamics where “audit” and “validity” are not external narratives but generator-level objects.

3.4 Parameter Bounds (Anti-Smuggling Requirement)

Rule. Every generator class MUST include explicit bounds sufficient to prevent hidden tuning. At minimum:

- bounded channel count $K_{\max}$ (for G2/G3b),
- bounded coupling constants (e.g. $\lambda \in [-\Lambda, \Lambda]$, $\gamma \in [0, \Gamma]$) on a quantized grid,
- bounded operator magnitudes by declared norm proxy,
- and a finite reference universe for any external IDs (committed by hash if external).

If bounds are omitted, the generator family MUST be treated as non-canonical and verification MUST reject.

3.5 Baseline Generator Profile (LIOUV-0)

EY25 defines a conservative baseline generator profile intended for universal auditability.

LIOUV-0. Unless overridden, the baseline $\mathcal{A}_{\mathbb{G}}$ is:

$$\mathcal{A}_{\mathbb{G}, \text{LIOUV-0}} := \text{G1 (Commutator Generator)}$$

with:

- explicit HRef encoding or hash commitment with deterministic truncation,
- no dissipation channels,
- and a simple, fully explicit discretization profile (defined in Chapter 4).

Normative note. LIOUV-0 is intentionally conservative: it provides a clean reference baseline for deterministic replay. Later profiles may enable G2/G3 with controlled dissipation and validity coupling.

3.6 Membership Check Procedure

For any chosen generator class, the certificate MUST include a membership check routine:

- parse canonical encoding of $\mathbb{G}$,
- verify generator class ID and parameter tuple correctness,
- verify bounds (channel count, magnitudes, coupling constants),
- verify ordering rules (canonical channel ordering),
- verify any external commitments (EY24Ref hashes, operator encodings),
- return deterministic failure if violated.

3.7 Chapter Summary

This chapter defined admissible generator families $\mathcal{A}_{\mathbb{G}}$ for EY25: (G1) commutator/unitary generators, (G2) Lindblad-like generators with controlled dissipation, and (G3) validity-coupled

generators referencing EY24. It imposed explicit parameter bounds to prevent tuning smuggling, defined a conservative baseline profile LIOUV-0, and specified deterministic membership checks. Chapter 4 next defines discretization and transcript semantics in detail: allowed step types, deterministic schedules, ordering/rounding rules, transcript commitments, and failure modes for non-convergence or protocol violation.

4 Discretization, Transcript Semantics, and Deterministic Replay

4.1 Purpose

EY25 is not “a differential equation”; it is a *contract* that deterministically produces an evolution map and a replayable transcript. This chapter fixes:

- allowed discretization step types and their evaluation order,
- deterministic schedule semantics (time horizon, step count, rounding),
- transcript commitments (what is hashed per step and how),
- and deterministic failure codes for divergence, overflow, or protocol violation.

Rule. Any unspecified solver freedom is forbidden. If a method uses tolerances, iteration caps, or adaptive steps, those **MUST** be explicit and deterministic and **MUST** appear in SpecID.

4.2 Time and Schedule Binding

Schedule Object

EY25 defines a schedule object:

$$\text{Schedule} := (T, \Delta t, N_{\text{steps}}, \text{GridRule}),$$

where T is the total time horizon, Δt is the step size, and N_{steps} is the number of steps.

Rule (two-of-three binding). The SpecID **MUST** fix any two of $(T, \Delta t, N_{\text{steps}})$, and the third **MUST** be derived deterministically. If the derived value is not an integer where required, the verifier **MUST** return **FAIL_DISC_MISMATCH**.

GridRule. GridRule **MUST** specify:

- rounding policy (floor/ceil/nearest) for derived values,
- whether endpoints are included,
- and whether a final partial step is permitted (default: forbidden).

4.3 Allowed Step Types (Normative Baseline Set)

Let $\Phi(\rho) \equiv \mathcal{L}(\rho)$ (or $\Phi(g) \equiv \mathcal{L}^*(g)$ in observable picture). All step types below **MUST** specify deterministic evaluation order.

D1: Forward Euler (Explicit)

State-side.

$$\rho_{n+1} := \rho_n + \Delta t \Phi(\rho_n).$$

Observable-side.

$$g_{n+1} := g_n + \Delta t \Phi(g_n).$$

Normative note. Euler is included because it is trivially replayable. Stability is not guaranteed; failure modes are explicit.

D2: RK2 / Midpoint (Explicit)

State-side.

$$k_1 := \Phi(\rho_n), \quad \tilde{\rho} := \rho_n + \frac{\Delta t}{2} k_1, \quad k_2 := \Phi(\tilde{\rho}), \quad \rho_{n+1} := \rho_n + \Delta t k_2.$$

All intermediate rounding and canonical serialization steps MUST be specified in NumericModel.

D3: Fixed-Rule Trotter Product (For Commutator Generators)

For G1 (commutator) generators, a Trotter step may be defined by a declared factorization:

$$H = \sum_{j=1}^J H^{(j)},$$

with a canonical order of terms. A first-order Trotter step is:

$$U_{\Delta t} \approx \prod_{j=1}^J \exp(-i\Delta t H^{(j)}),$$

and:

$$\rho_{n+1} := U_{\Delta t} \rho_n U_{\Delta t}^\dagger.$$

Rule. If D3 is used, the exponential approximation method MUST be fixed and deterministic (e.g. fixed-degree Padé with fixed scaling rule and explicit rounding). “Library expm” without a pinned algorithm is forbidden.

4.4 Rounding and Canonicalization Within a Step

To guarantee replayability, NumericModel MUST define when values are canonicalized.

Canonicalization points. A **canonicalization point** is a step-local boundary at which the object is serialized and rounded according to NumericModel. EY25 requires at minimum:

- canonicalize after each step output (ρ_{n+1} or g_{n+1}),
- canonicalize any intermediate object that is included in transcript commitments.

Rule. If two implementations canonicalize at different points, they are different artifacts unless the SpecID declares CanonPoints explicitly and identically.

4.5 Transcript Semantics

TranscriptSpec

EY25 defines:

$\text{TranscriptSpec} := (\text{CommitType}, \text{CommitRule}, \text{HashAlg}, \text{SerializeRule}, \text{CommitRate}).$

CommitType (examples).

- **full-state**: commit hash of full canonical encoding of ρ_n (or g_n),
- **summary**: commit hash of declared summary vector (spectral moments, traces, bucketed invariants),
- **hybrid**: periodic full-state commitments plus per-step summaries.

CommitRate. CommitRate MUST be explicit:

$\text{CommitRate} \in \{\text{every step}, \text{every } m \text{ steps}, \text{schedule list}\}.$

Rule. If summary commitments are used, the summary function MUST be deterministic and MUST be sufficient to detect protocol deviation as declared by SpecID. Otherwise the artifact MUST not claim audit-native replay for full trajectories.

Transcript Generation

Let $\text{Enc}(\cdot)$ be canonical encoding and $\text{Hash}(\cdot)$ be the declared hash. Then:

$$\text{Tx}_n := \text{Hash}(\text{Enc}(X_n)),$$

where X_n is either the full object or the declared summary, per TranscriptSpec.

Rule (chain binding). Optionally, EY25 allows chained commitments:

$$\text{Tx}_n := \text{Hash}(\text{Tx}_{n-1} \parallel \text{Enc}(X_n)),$$

but the choice MUST be fixed in TranscriptSpec.

4.6 Deterministic Failure Modes During Evolution

Evolution must fail deterministically when required invariants are violated.

Required failure codes

At minimum, EY25 MUST define:

- **FAIL_DISC_MISMATCH**: schedule cannot be derived deterministically or violates GridRule.
- **FAIL_TRUNC_MISMATCH**: truncation calculus is inconsistent across steps or across required N .

- **FAIL_DOMAIN_MISMATCH**: generator action not defined on current object under Calc/AdjointSpec.
- **FAIL_OVERFLOW**: numeric overflow/NaN/Inf appears where forbidden by NumericModel.
- **FAIL_DIVERGENCE**: declared norm proxy exceeds bound B_{div} (if divergence bound enabled).
- **FAIL_NONDETERMINISM**: replay produces different intermediate values or transcript commitments.
- **FAIL_TRANSCRIPT_MISMATCH**: transcript commitments do not match replay.

Optional divergence bound

EY25 may include an optional bound:

$$\|X_n\|_{\text{proxy}} \leq B_{\text{div}} \quad \forall n,$$

where $\|\cdot\|_{\text{proxy}}$ is fixed in NumericModel. If enabled and violated, return **FAIL_DIVERGENCE**.

4.7 Deterministic Replay Definition

Definition 4.1 (Deterministic Replay). An EY25 instance is *replay-deterministic* if, for any verifier executing the declared Disc, Trunc, and NumericModel on the same public inputs, the produced transcript (Tx_n) and outputs (X_n) match exactly (or within declared tolerance at declared canonicalization points).

Rule. An EY25 artifact MUST NOT claim replay determinism unless TranscriptSpec and NumericModel jointly fix all non-deterministic degrees of freedom (ordering, rounding, parallel reduction, and any randomness).

4.8 Chapter Summary

This chapter fixed discretization semantics for audit-native evolution: schedule binding, allowed step types with deterministic evaluation order, canonicalization points, transcript commitment rules, and deterministic failure modes for divergence or protocol violation. It also defined deterministic replay as the central correctness notion for $\mathcal{L}_{\text{audit}}$.

Chapter 5 next specifies the certificate payload π_{LIOUV} and verifier checks: membership of the generator, correctness of the discretization protocol, correctness of transcript commitments, and interoperability checks when composing with EY23/EY24/EY29/EY28.

5 Certificate and Verifier: Replay, Commitments, and Acceptance

5.1 Purpose

EY25 publishes evolution as an auditable contract. This chapter specifies:

- the certificate payload π_{LIOUV} required to publish an EY25 instance,

- the deterministic verifier checks that accept or reject the instance,
- transcript commitments and replay rules,
- and required failure codes.

EY25 certificates conform to EY22 certificate conventions (EY22 Chapter 5, Appendix A.1) and add EY25-specific fields and checks.

5.2 Certificate Object π_{LIOUV}

Rule. An EY25 instance MUST be published with a certificate π_{LIOUV} that includes, at minimum, the following canonical content.

Header (required)

- artifact type: LIOUV
- artifact symbol: $\mathcal{L}_{\text{audit}}$
- SpecID short form and full SpecID tuple
- version identifier

SpecID (required)

The full SpecID tuple MUST be embedded:

(LIOUV, Pic, $\mathbb{G}$, $\mathcal{A}_{\mathbb{G}}$, Disc, Trunc, NumericModel, TranscriptSpec, Cert, Interop, Version).

Inputs (required)

Inputs MUST include:

- canonical encoding (or commitment) of $\mathbb{G}$ and all referenced operators/channels (e.g. H , $\{L_k\}$, EY24Ref),
- $\mathcal{A}_{\mathbb{G}}$ family definition and membership check ID,
- full Disc and Trunc objects (including schedule constants and truncation levels),
- full NumericModel (rounding, ordering, tolerances, canonicalization points),
- full TranscriptSpec (commit type, hash, serialization, commit rate),
- public initial condition X_0 (or its commitment and reconstruction rule),
- any public inputs used to derive schedule or truncation parameters (seed, dataset IDs, basis IDs), committed by hash if external.

Outputs (required)

Outputs MUST include:

- transcript commitments $\text{Tx} = (\text{Tx}_0, \dots, \text{Tx}_T)$ for all required commitment points,
- final output object X_T (or its commitment),

- any declared summaries (trace, norms, spectral moments) that are committed or used for checks,
- output hashes for $\mathbb{G}$, Disc, Trunc, NumericModel, TranscriptSpec, and the transcript itself.

Checks (required)

The certificate MUST list check IDs and what each check verifies, grouped as:

- structural and SpecID checks,
- generator membership and feasibility checks,
- discretization protocol checks,
- truncation checks,
- transcript/replay checks,
- interoperability checks (if claimed).

Commitments (required if external objects exist)

All external objects (basis definitions, operator encodings too large to inline, channel catalogs, transcripts, code snapshots) MUST be committed by hash according to EY22 Appendix A.1.

5.3 Verifier Contract

A verifier takes (π_{LIOUV}) and deterministically returns:

$$\{\text{ACCEPT}, \text{REJECT}\} \cup \{\text{FAIL_}\ast\}.$$

Rule. The verifier MUST NOT require trust in producer identity, informal descriptions, or uncommitted external resources.

5.4 Required Verifier Checks (Normative)

This section enumerates the minimal checks required for EY25 acceptance.

L1: Structural Validity (CCE)

- parse canonical certificate encoding (EY22 CCE rules),
- reject malformed encodings, duplicate keys, non-canonical numbers,
- confirm required fields exist.

Failure: **REJECT_MALFORMED**.

L2: SpecID Completeness and Consistency

- SpecID tuple is present and complete,
- ArtifactType = LIOUV,

- Pic is declared,
- Disc, Trunc, NumericModel, TranscriptSpec present.

Failure: **REJECT_SPECID_MISSING**, **REJECT_SPECID_MISMATCH**.

L3: No Hidden Inputs

- $\mathbb{G}$ and all referenced operators/channels are provided or committed,
- any derivations (schedule/truncation) depend only on public committed inputs,
- no undeclared randomness is used.

Failure: **REJECT_HIDDEN_INPUT**, **FAIL_HASH_MISMATCH**.

L4: Generator Membership and Bounds

Verify $\mathbb{G} \in \mathcal{A}_{\mathbb{G}}$ deterministically:

- parse canonical encoding of $\mathbb{G}$,
- verify generator class ID (G1/G2/G3),
- verify parameter bounds (channel count, magnitudes, coupling constants on quantized grids),
- verify canonical ordering rules (e.g. channels lex-sorted by encoding),
- verify any referenced EY24Ref binds to full EY24 SpecID and commitment.

Failure: **FAIL_CONSTRAINT**.

L5: Discretization Protocol Check

- derive schedule values from SpecID (two-of-three rule),
- verify GridRule and rounding policy,
- verify step type is allowed and all evaluation order is declared,
- verify canonicalization points are declared and consistent with NumericModel.

Failure: **FAIL_DISC_MISMATCH** or **FAIL_NONDETERMINISM**.

L6: Truncation Protocol Check

- verify Trunc object is complete (Basis, CutRule, BoundaryRule, required N_{ℓ}),
- verify truncation maps for referenced operators (e.g. $H \mapsto H_N$) are deterministic,
- verify CrossNPolicy if multiple truncation levels are used.

Failure: **FAIL_TRUNC_MISMATCH**.

L7: Step Execution Replay (Deterministic)

Run the declared protocol from X_0 for all required steps:

- execute the step type with deterministic evaluation order,
- apply canonicalization at declared points,
- compute per-step commitments per TranscriptSpec.

Rule. If the verifier cannot execute within declared constraints (e.g. domain mismatch), it MUST fail deterministically.

Failures:

- **FAIL_DOMAIN_MISMATCH** if generator action not defined,
- **FAIL_OVERFLOW** if NaN/Inf/overflow occurs where forbidden,
- **FAIL_DIVERGENCE** if divergence bound is enabled and violated,
- **FAIL_NONDETERMINISM** if replay differs at declared canonicalization points.

L8: Transcript Commitment Check

Compare replayed commitments to the published transcript:

$$Tx_n^{(\text{replay})} \stackrel{?}{=} Tx_n^{(\text{published})} \quad \text{for all required } n.$$

Failure: **FAIL_TRANSCRIPT_MISMATCH**.

L9: Optional Invariant Checks (If Declared)

If SpecID declares invariant checks (trace preservation, positivity preservation, contractive bounds, etc.), the verifier MUST check them. Failures use: **FAIL_INVARIANT** or a more specific code if declared.

L10: Interop Checks (Only if Claimed)

If the certificate claims interoperability with EY23/EY24/EY29/EY28, the verifier MUST evaluate the declared compatibility predicates under shared NumericModel and declared calculus (EY22 Chapter 6). Failure: **FAIL_COMPAT_PREDICATE** (or the specific EY22 interop failure code).

5.5 Required Failure Codes

EY25 MUST support at minimum:

- **REJECT_MALFORMED**
- **REJECT_SPECID_MISSING**
- **REJECT_SPECID_MISMATCH**
- **REJECT_HIDDEN_INPUT**
- **FAIL_HASH_MISMATCH**

- **FAIL_DISC_MISMATCH**
- **FAIL_TRUNC_MISMATCH**
- **FAIL_DOMAIN_MISMATCH**
- **FAIL_OVERFLOW**
- **FAIL_DIVERGENCE**
- **FAIL_NONDETERMINISM**
- **FAIL_TRANSCRIPT_MISMATCH**
- **FAIL_CONSTRAINT**
- **FAIL_COMPAT_PREDICATE**
- **FAIL_INVARIANT**

5.6 Chapter Summary

This chapter specified the EY25 certificate payload π_{LIOUV} and the deterministic verifier checks required to accept an audit-native evolution instance. It fixed checks for generator admissibility, discretization protocol correctness, truncation correctness, replay execution, transcript commitment matching, and interoperability predicates when claimed.

Chapter 6 next defines interoperability declarations (mini) for EY25: explicit composition rules with EY23 filtering and EY24 validity penalties, and optional compositions with EY29 coarse-grain reduction and EY28 transport across regimes.

6 Interop Declarations (Mini): Composition with EY23/EY24/EY29/EY28

6.1 Purpose

This chapter instantiates EY22 interoperability rules (EY22 Chapter 6) for EY25. It defines minimal compatibility predicates for composing the Audit-Native Liouvillian $\mathcal{L}_{\text{audit}}$ with:

- EY23 (CSF) Canonical Spectral Filter $F_{\star}(H)$,
- EY24 (VPO) Validity Penalty Operator $V = C^{\dagger}C$,
- EY29 (CGR) Coarse-Grain Reduction R ,
- EY28 (INTW) Regime Transport $T_{1 \rightarrow 2}$.

Rule. If an EY25 instance claims interoperability beyond what is stated here, it **MUST** introduce additional compatibility predicates inside its Interop SpecID component and **MUST** remain checkable under NumericModel.

6.2 Interop Component (EY25)

EY25 declares the following baseline composition forms (EY22 C1–C5):

$$\text{Interop}_{\text{EY25}} := \{C4, C1\},$$

i.e. superoperator action (C4) and algebraic composition at the generator/observable level (C1). Coarse-grain (C3) and transport (C2) are supported only via explicit predicates below.

6.3 Compatibility Predicates

All compatibility predicates below are deterministic booleans under the declared `NumericModel(s)`.

LIOUV $\leftrightarrow$ CSF: Filter–Evolve–Filter Workflows

The canonical pattern is:

$$X \xrightarrow{\text{Filter}} X' \xrightarrow{\mathcal{L}_{\text{audit}}} X'' \xrightarrow{\text{Filter}} X^*,$$

where filtering may occur on H , on observables, or as a post-processing map on outputs (as declared by EY23).

Workflow semantics (must be declared). EY25 recognizes two interop semantics with EY23:

(FE1) generator-side filtering. Filter modifies the generator input (typically H):

$$H' := F_*(H) \text{ or } H' := \Phi(F_*(H)),$$

then $\mathcal{L}_{\text{audit}}$ is constructed from H' .

(FE2) output-side filtering. $\mathcal{L}_{\text{audit}}$ evolves X and EY23 filtering applies to resulting objects/summaries.

Predicate $\text{Compat}_{25,23}^{(1)}$ (space and domain).

$\text{Compat}_{25,23}^{(1)} :=$ (the filtered object and the evolved object live in compatible spaces under declared `Calc/Trunc`).

Predicate $\text{Compat}_{25,23}^{(2)}$ (semantic declaration).

$\text{Compat}_{25,23}^{(2)} :=$ (workflow declares FE1 or FE2 and fixes ordering and affected object types).

Predicate $\text{Compat}_{25,23}^{(3)}$ (numeric model alignment).

$\text{Compat}_{25,23}^{(3)} :=$ ($\text{NumericModel}_{\text{EY25}} = \text{NumericModel}_{\text{EY23}}$) $\vee$ (declared deterministic coercion).

Predicate $\text{Compat}_{25,23}^{(4)}$ (transcript alignment).

$\text{Compat}_{25,23}^{(4)} :=$ (EIF filtering affects generator or outputs, the transcript commitments bind to the filtered objects).

Compatibility status.

$$\text{Compat}(\text{EY25}, \text{EY23}) := \text{Compat}_{25,23}^{(1)} \wedge \text{Compat}_{25,23}^{(2)} \wedge \text{Compat}_{25,23}^{(3)} \wedge \text{Compat}_{25,23}^{(4)}.$$

If TRUE, mark OK*.

LIOUV $\leftrightarrow$ VPO: Validity Propagation and Audit Coupling

EY24 defines $V = C^\dagger C$. EY25 composes with EY24 in two canonical modes:

(VP1) penalty-in-generator (augmentation).

$$H' := H + \lambda V,$$

then $\mathcal{L}_{\text{audit}}$ is generated from H' (G3a).

(VP2) validity damping channel.

$$L_V := \sqrt{\gamma} C,$$

then L_V is included as a declared Lindblad channel (G3b).

Predicate $\text{Compat}_{25,24}^{(1)}$ (EY24 binding).

$$\text{Compat}_{25,24}^{(1)} := (\text{EY24Ref binds to a full EY24 SpecID and certificate commitment}).$$

Predicate $\text{Compat}_{25,24}^{(2)}$ (mode declaration).

$$\text{Compat}_{25,24}^{(2)} := (\text{workflow declares VP1 or VP2, including ordering and whether } V \text{ or } C \text{ is used}).$$

Predicate $\text{Compat}_{25,24}^{(3)}$ (calculus compatibility).

$$\text{Compat}_{25,24}^{(3)} := \begin{cases} \text{addition } H + \lambda V \text{ defined under Calc/Trunc and AdjointSpec,} & \text{if VP1} \\ \text{channel inclusion } L_V \text{ satisfies G2 bounds and ordering rules,} & \text{if VP2} \end{cases}$$

Predicate $\text{Compat}_{25,24}^{(4)}$ (numeric model alignment).

$$\text{Compat}_{25,24}^{(4)} := (\text{NumericModel}_{\text{EY25}} = \text{NumericModel}_{\text{EY24}}) \vee (\text{declared deterministic coercion}).$$

Compatibility status.

$$\text{Compat}(\text{EY25}, \text{EY24}) := \text{Compat}_{25,24}^{(1)} \wedge \text{Compat}_{25,24}^{(2)} \wedge \text{Compat}_{25,24}^{(3)} \wedge \text{Compat}_{25,24}^{(4)}.$$

If TRUE, mark OK*.

LIOUV $\leftrightarrow$ CGR: Coarse-Grain Evolution

If R is a coarse-grain operator, there are two canonical semantics:

(CG1) evolve-then-reduce.

$$X_{n+1} := \mathcal{E}_{\Delta t}(X_n), \quad \tilde{X}_{n+1} := R(X_{n+1}).$$

(CG2) reduced evolution. Define a reduced generator $\tilde{\mathbb{G}}$ and reduced evolution $\tilde{\mathcal{E}}_{\Delta t}$ such that:

$$\tilde{X}_{n+1} := \tilde{\mathcal{E}}_{\Delta t}(\tilde{X}_n), \quad \tilde{X}_n := R(X_n).$$

Predicate $\text{Compat}_{25,29}^{(1)}$ **(semantic declaration).**

$\text{Compat}_{25,29}^{(1)} :=$ (workflow declares CG1 or CG2, and fixes where reduction occurs relative to transcript).

Predicate $\text{Compat}_{25,29}^{(2)}$ **(invariant preservation).** If reduction is claimed to preserve invariants used by transcript summaries, then:

$\text{Compat}_{25,29}^{(2)} :=$ (all declared preserved invariants remain valid under R within tolerance).

Predicate $\text{Compat}_{25,29}^{(3)}$ **(deterministic transcript under reduction).**

$\text{Compat}_{25,29}^{(3)} :=$ (TranscriptSpec commits to objects in a way that is deterministic under the declared reduction).

Compatibility status.

$$\text{Compat}(\text{EY25}, \text{EY29}) := \text{Compat}_{25,29}^{(1)} \wedge \text{Compat}_{25,29}^{(2)} \wedge \text{Compat}_{25,29}^{(3)}.$$

If TRUE, mark OK*.

LIQV ↔ INTW: Transporting Evolution Across Regimes

If $T_{1 \rightarrow 2}$ transports objects between regimes, then an evolution in regime 1 can be related to an evolution in regime 2. Two canonical semantics are permitted:

(TR1) conjugation of step map.

$$\mathcal{E}_{\Delta t}^{(2)} := \mathcal{T} \circ \mathcal{E}_{\Delta t}^{(1)} \circ \mathcal{T}^{-1},$$

where $\mathcal{T}$ is the superoperator induced by T on the chosen picture.

(TR2) transport of generator. Transport generator objects (e.g. H , channels) and then build $\mathcal{L}_{\text{audit}}$ in regime 2.

Predicate $\text{Compat}_{25,28}^{(1)}$ **(semantic declaration).**

$\text{Compat}_{25,28}^{(1)} :=$ (workflow declares TR1 or TR2 and provides required inverses/adjoints deterministically).

Predicate $\text{Compat}_{25,28}^{(2)}$ **(adjoint/calc coherence).**

$\text{Compat}_{25,28}^{(2)} :=$ (AdjointSpec and Calc in both regimes are coherent with the EY28 mapping semantics).

Predicate $\text{Compat}_{25,28}^{(3)}$ **(transcript coherence).**

$\text{Compat}_{25,28}^{(3)} :=$ (TranscriptSpec binds to the transported objects deterministically and preserves ordering/rounding).

Compatibility status.

$$\text{Compat}(\text{EY25}, \text{EY28}) := \text{Compat}_{25,28}^{(1)} \wedge \text{Compat}_{25,28}^{(2)} \wedge \text{Compat}_{25,28}^{(3)}.$$

If TRUE, mark OK*.

6.4 Mini Compatibility Matrix Row (EY25 Perspective)

This minimal row is informative and must be backed by predicates above:

	EY23	EY24	EY25	EY26	EY27	EY28	EY29	EY30
EY25	OK*	OK*	N/A	OK*	OK*	OK*	OK*	OK*

Notes (informational).

- EY25×EY26 is OK* because Floquet program tokens can generate deterministic step schedules or input sequences.
- EY25×EY27 is OK* because commutant generators can be used as symmetry constraints or generator decompositions.
- EY25×EY30 is OK* because Koopman export can map the audited evolution into observable-space representations; mapping must be declared.

6.5 Interop Failure Codes (Required)

Interop verification MUST use deterministic failure codes, at minimum:

- **FAIL_DOMAIN_MISMATCH**
- **FAIL_CALC_MISMATCH**
- **FAIL_NUMERICMODEL_MISMATCH**
- **FAIL_TRANSCRIPT_MISMATCH**
- **FAIL_INVARIANT_NOT_PRESERVED**
- **FAIL_COMPAT_PREDICATE**

6.6 Chapter Summary

This chapter provided minimal EY25 interoperability declarations, defining concrete compatibility predicates for composing $\mathcal{L}_{\text{audit}}$ with EY23 filtering, EY24 validity penalties, EY29 coarse-grain reduction, and EY28 transport across regimes. It also provided an EY25 mini compatibility-matrix row and fixed the required failure codes for interoperability failures.

Chapter 7 next defines the baseline LIOUV-0 reference profile (a concrete SpecID instance with fixed generator class, discretization protocol, truncation schedule, transcript commitments, and certificate requirements).

7 LIOUV-0 Reference Profile and Example SpecID Instance

7.1 Purpose

Chapters 1–6 defined EY25 in full generality. This chapter provides a concrete, audit-friendly baseline configuration called **LIOUV-0**. LIOUV-0 is designed to be:

- simple enough to verify by third parties without specialized solvers,
- deterministic by construction (replayable transcript),
- minimal in degrees of freedom (anti-smuggling),
- and stable as a canonical meaning of “audit-native evolution” for Vol. No. IV.

Rule. A LIOUV-0 instance **MUST** be fully specified by explicit constants (or hash commitments) and **MUST NOT** require instance-specific tuning.

7.2 LIOUV-0 Summary (Normative)

LIOUV-0 fixes:

- picture: $\text{Pic} = \text{state}$ (state-side evolution),
- generator class: G1 commutator generator $\mathcal{L}(\rho) = -i[H, \rho]$,
- discretization: D1 Forward Euler with explicit schedule binding,
- transcript: full-state commitment every step (chained commitments allowed but must be fixed),
- truncation: a fixed basis and cut rule with a fixed truncation schedule $\{N_\ell\}$,
- certificate: EY22-CCE with required checks L1–L8 (and L10 if interop claimed).

Normative note. LIOUV-0 is intentionally conservative. It defines a clean replay baseline. Optimized profiles may replace D1 with RK2 or Trotter, or enable controlled dissipation, but **MUST** still remain deterministic and transcript-replayable.

7.3 LIOUV-0 Generator Profile (G1) Details

Admissible Family $\mathcal{A}_{\mathbb{G}, \text{LIOUV-0}}$

$\mathcal{A}_{\mathbb{G}, \text{LIOUV-0}} := \left\{ \mathbb{G} = (H) \mid H = H^\dagger \text{ under AdjointSpec, HRef is canonical or committed, BoundaryPolicy} = \text{E} \right\}$

Fixed constants (LIOUV-0). The following **MUST** be fixed in SpecID:

$$\text{BP}_0, \quad \text{UnitsSpec}_0, \quad \text{HRefEncodingRule}_0.$$

Self-adjointness check. For required truncation levels N_ℓ , the verifier **MUST** check:

$$\|H_{N_\ell} - H_{N_\ell}^\dagger\|_{\text{proxy}} \leq \varepsilon_H,$$

with ε_H fixed in NumericModel. If exact fixed-point arithmetic is used, $\varepsilon_H = 0$.

7.4 LIOUV-0 Discretization Profile (D1) Details

Disc₀

$\text{Disc}_0 := (\text{StepType} = \text{D1}, T_0, \Delta t_0, N_{\text{steps},0}, \text{GridRule}_0, \text{CanonPoints}_0).$

Two-of-three binding (example normative pattern). LIOUV-0 fixes T_0 and Δt_0 , derives:

$$N_{\text{steps},0} := \frac{T_0}{\Delta t_0}.$$

If not an integer under GridRule_0 , reject with **FAIL_DISC_MISMATCH**. Final partial steps are forbidden in LIOUV-0.

Canonicalization points. CanonPoints_0 MUST include:

- after computing $\Phi(\rho_n) = -i[H, \rho_n]$ (optional in LIOUV-0; if enabled must be fixed),
- after producing ρ_{n+1} (required),
- at any transcript commitment point (required).

7.5 LIOUV-0 Truncation Profile

Trunc₀

$\text{Trunc}_0 := (\text{Basis}_0, \text{CutRule}_0, \text{BoundaryRule}_0, \{N_\ell\}_{\ell=1}^L, \text{CrossNPolicy}_0).$

Baseline requirements.

- Basis_0 MUST be declared and committed by hash if external.
- CutRule_0 MUST be deterministic.
- BoundaryRule_0 MUST be deterministic (no implicit padding).
- $\{N_\ell\}$ MUST be fixed and used consistently for H , ρ , and transcript objects when required.
- CrossNPolicy_0 MUST declare how (if at all) objects at different N are compared or aggregated.

7.6 LIOUV-0 Transcript Profile

TranscriptSpec₀

$\text{TranscriptSpec}_0 := (\text{CommitType} = \text{full-state}, \text{HashAlg}_0, \text{SerializeRule}_0, \text{CommitRate} = \text{every step}, \text{ChainRule}_0).$

ChainRule. LIOUV-0 permits either:

- $\text{ChainRule}_0 = \text{none}$ (independent per-step commitments), or
- $\text{ChainRule}_0 = \text{hash-chain}$ (chained commitments).

The choice MUST be fixed in SpecID .

SerializeRule. Serialization MUST use canonical encoding (EY22 Appendix A.1). If a compressed representation is used, the compression rule MUST be deterministic and committed.

7.7 LIOUV-0 NumericModel (Minimal Deterministic Profile)

LIOUV-0 requires a deterministic NumericModel sufficient to fix all replay degrees of freedom:

- arithmetic type (fixed-point or deterministic float) for matrix products,
- rounding policy and precision,
- matrix multiplication order and accumulation order,
- canonicalization points CanonPoints_0 ,
- tolerances $\varepsilon_H, \varepsilon_{\text{step}}$ (if approximate),
- divergence bound B_{div} (optional; off by default in LIOUV-0).

Recommended (informational). Use exact fixed-point for inputs/parameters (operator encodings) and deterministic float for step computations with canonical rounding.

7.8 LIOUV-0 Certificate Requirements

LIOUV-0 requires at minimum the checks:

- L1 (CCE structural),
- L2 (SpecID consistency),
- L3 (no hidden inputs),
- L4 (generator membership and bounds),
- L5 (discretization protocol),
- L6 (truncation protocol),
- L7 (step replay),
- L8 (transcript matching).

Interop checks (L10) are required only if interop is claimed.

7.9 Reference SpecID Instance (LIOUV-0 Template)

The following is a template for a LIOUV-0 SpecID instance. Numeric constants are placeholders until locked.

```
SpecID_EY25_LIOUV0 :=
(
  ArtifactType = LIOUV,
  Pic = "state",
  G = {
    class: "G1-commutator",
    HRef: Hash(H) or Enc(H),
    units: "UnitsSpec0",
```

```

    boundary: "BP0"
  },
  A_G = {
    class: "G1-commutator",
    require_self_adjoint: true,
    eps_H: 0 (or 1e-12),
    HRefEncodingRule: "canonical-or-committed"
  },
  Disc = {
    StepType: "D1-euler",
    bind: { T: 1.0, dt: 0.001, derive: "Nsteps=T/dt", final_partial: false },
    GridRule: { rounding: "reject-if-not-integer" },
    CanonPoints: ["after-step", "at-commit"]
  },
  Trunc = {
    Basis: Hash(basis0),
    CutRule: "cut0",
    BoundaryRule: "boundary0",
    N: [256, 512, 1024],
    CrossNPolicy: "none"
  },
  NumericModel = {
    arithmetic: "det-float-or-fixed",
    rounding: { mode: "nearest", precision: 64 },
    matmul: { order: "row-major", accumulate: "sequential" },
    tolerances: { eps_H: 1e-12, eps_step: 1e-10 },
    divergence: null
  },
  TranscriptSpec = {
    CommitType: "full-state",
    HashAlg: "sha256",
    SerializeRule: "EY22-CCE",
    CommitRate: "every-step",
    ChainRule: "hash-chain"
  },
  Cert = {
    schema: "EY22-CCE",
    required_checks: ["L1", "L2", "L3", "L4", "L5", "L6", "L7", "L8"]
  },
  Interop = {
    allowed_forms: ["C4", "C1"],
    compat: {
      EY23: "Compat_25_23",
      EY24: "Compat_25_24",
      EY28: "Compat_25_28",
      EY29: "Compat_25_29"
    }
  },
  Version = "VID-2026-02"
)

```

Rule. Once LIOUV-0 is published, all placeholders **MUST** be replaced by fixed values (or hash commitments) and the Version **MUST** be treated as immutable for that meaning.

7.10 Chapter Summary

This chapter defined LIOUV-0, a concrete baseline profile for EY25 designed for deterministic replay and third-party auditability. It fixed the default generator class (G1 commutator), baseline discretization (Euler), fixed truncation and transcript profiles, required certificate checks, and a reference SpecID instance template.

Next. Chapter 8 fixes the publication boundary and non-claims for EY25 and states licensing intent, consistent with Vol. No. IV.

8 Publication Boundary, Non-Claims, and Licensing Intent

8.1 Purpose

EY25 makes “dynamics” publishable as an auditable artifact. That power also creates a common failure mode: people read an evolution protocol as a claim about truth, correctness, or security. This chapter fixes:

- what **MUST** be public for an EY25 publication,
- what EY25 explicitly does *not* claim,
- and the QFC-consistent licensing intent for implementations and derivative catalogs.

8.2 Publication Boundary (Normative)

Rule (what must be public). An EY25 publication **MUST** include, at minimum:

- the full $\text{SpecID}_{\text{EY25}}$ tuple,
- the canonical encoding (or hash commitment) of the generator $\mathbb{G}$,
- canonical encodings (or commitments) for all referenced operators/channels (e.g. H , $\{L_k\}$, C , V , EY24Ref),
- the full Disc object (step type, schedule binding, ordering, canonicalization points),
- the full Trunc object (basis, cut rule, boundary rule, required truncation levels),
- the full NumericModel (rounding, ordering, tolerances, aggregation, canonical encoding rules),
- the full TranscriptSpec (commit type, serialization, hash algorithm, commit rate, chaining),
- the transcript commitments Tx at all required commitment points,
- and the certificate π_{LIOUV} with required check identifiers.

Rule (no hidden solver degrees of freedom). Any choice that can change the evolution output **MUST** be included in SpecID or committed by hash. In particular:

- adaptive step size is forbidden unless the schedule is fully specified as a deterministic function of public inputs,
- any tolerance must be explicit and fixed,
- any randomness must be seeded deterministically and the seed must be public/committed.

Rule (transcript is part of the publication). If an EY25 result is cited as an artifact-derived claim (trajectory, invariant, regime classification), then the transcript commitments **MUST** be published at the declared rate. Claims without transcript commitments **MUST** be treated as non-audited narratives.

Rule (commitment for external objects). If the instance references external objects (basis definitions, generator catalogs, datasets, code snapshots), those objects **MUST** be committed by hash according to EY22 Appendix A.1.

8.3 Non-Claims (Explicit)

EY25 explicitly does *not* claim the following:

NC1: Physical Correctness

EY25 does not claim that the chosen generator $\mathbb{G}$ describes physical reality, nor that the resulting evolution is the “true” time evolution of any system. EY25 only claims:

- the artifact is well-defined under SpecID,
- the verifier checks pass deterministically,
- and replay reproduces the transcript commitments under NumericModel.

NC2: Numerical Optimality

EY25 does not claim that the chosen discretization is numerically optimal, stable, or efficient. A method can be audit-native and still be a poor integrator. Optimization is outside the baseline contract (though future profiles may standardize it).

NC3: Security Guarantees Beyond Declared Checks

EY25 does not claim cryptographic security, adversarial robustness, or protection against malicious producers beyond what is explicitly covered by:

- membership checks for $\mathbb{G}$,
- transcript commitments and deterministic replay,
- and explicit interop predicates.

NC4: Equivalence of Pictures and Semantics

EY25 does not claim that state-side and observable-side evolution are equivalent in all truncation schemes or numeric models. Pic is part of SpecID, and workflows **MUST** not conflate them.

NC5: Completeness of the Transcript

A transcript commitment proves replay under the declared commitment policy. If TranscriptSpec commits only to summaries, EY25 does not claim full-state replayability. Any claim about full trajectories MUST publish full-state commitments at the required rate.

8.4 Responsible Use and Misuse Boundary

Rule. EY25 SHOULD be used to publish evolutions as reproducible artifacts:

- publish SpecID and all committed objects,
- publish transcript commitments,
- and allow third parties to replay and dispute both the generator choice and the discretization choice.

Misuse examples (non-normative).

- Publishing a trajectory plot without publishing transcript commitments.
- Claiming physical or security guarantees from auditability alone.
- Hiding step-size adaptation or tolerances behind “implementation details”.
- Using non-canonical libraries for matrix exponentials without pinning the algorithm and rounding rules.

8.5 Licensing Intent (QFC-Consistent)

EY25 is part of the QFC operator suite. The intent is that the *specification* remains public and auditable, while enabling controlled licensing of implementations or derivative operator catalogs if desired.

Normative statement of intent.

- The definitions, SpecID schemas, transcript rules, and verifier requirements are intended to be publicly readable and reusable.
- Implementations may be licensed under terms chosen by the project (open, dual-license, or commercial), provided they preserve auditability (SpecID + commitments + transcript + checks).
- Trademark and naming conventions (QFC, Vol. No. IV, EY-IDs, artifact names) may be governed separately from the text license.

Rule (no audit stripping). No license arrangement may remove the requirement to publish SpecID, commitments, and transcript commitments for any claim that relies on EY25.

8.6 Chapter Summary

This chapter fixed the publication boundary for EY25 and listed explicit non-claims: no claims of physical correctness, numerical optimality, or security beyond declared checks; no implicit equivalence of pictures; and no full-state replay claim without full-state commitments. It also

stated QFC-consistent licensing intent: keep the spec auditable and public while allowing implementation licensing provided auditability is preserved.

Next. With EY25 complete, Vol. No. IV proceeds to EY26 (Floquet Program Token U_F), which packages deterministic, replayable discrete-time programs that can generate data for EY23 probing and can be audited under the same transcript conventions.

EY26: Floquet Program Token U_F

Canonical Operator Artifacts for QFC

Enter Yourname

Abstract

This whitepaper defines the Floquet Program Token U_F , a canonical discrete-time execution artifact for QFC workflows. A token packages a *finite, replayable* program—an ordered list of operator updates, schedule semantics, and numeric/truncation rules—into a single auditable object. Unlike informal “circuits” or opaque simulation scripts, U_F is bound by SpecID: its step list, parameter grids, rounding behavior, tie-breaks, and failure modes are fixed and verifiable.

U_F is designed to generate deterministic datasets for canonical spectral probing (EY23), to interact with validity penalties (EY24), and to be evolved or audited under audit-native superoperators (EY25). The token also serves as a portable program commitment: a verifier can check that execution exactly matches the published transcript commitments, enabling public reproducibility without trust.

1 Motivation, Scope, and Design Goals

1.1 Motivation: Programs Must Become Publishable Objects

In Vol. No. IV we treat operators as publishable artifacts: they have SpecIDs, deterministic evaluation rules, and certificates. However, a large class of real workflows is not a single operator but a *finite program*: an ordered sequence of operator updates applied over discrete time.

Without a canonical representation, programs drift into:

- informal “circuits” whose semantics depend on libraries,
- scripts with hidden randomness, implicit tolerances, and uncommitted environment details,
- non-deterministic execution due to parallel reduction or GPU scheduling,
- and unverifiable claims about what was actually executed.

EY26 defines the Floquet Program Token U_F as a remedy: a program is packaged as a single auditable object whose meaning is fixed by SpecID and whose execution produces a replayable transcript.

1.2 What This Whitepaper Defines

EY26 defines:

- the Floquet Program Token U_F as a canonical discrete-time execution artifact,

- a program encoding: an ordered step list with explicit update semantics,
- schedule binding (period, step count, time slicing) and deterministic replay rules,
- numeric and truncation semantics required for deterministic execution,
- certificate payload and verifier checks for acceptance,
- minimal interoperability declarations with EY23 (CSF), EY24 (VPO), EY25 (LIOUV), and optionally EY29/EY28.

1.3 Scope

In-scope.

- **Discrete-time** programs with a finite step list (or a finite loop with bounded iteration count).
- Programs that can be executed deterministically under a declared NumericModel and Truncation policy.
- Transcript commitments that bind the exact executed sequence to public inputs.
- Composition with operator artifacts: filtering, validity, audit evolution, coarse-grain reduction, and transport.

Out-of-scope.

- Arbitrary Turing-complete programs with unbounded loops or hidden I/O.
- Programs whose semantics depend on non-committed external state (OS time, network calls, nondeterministic RNG).
- Claims of physical correctness, optimality, or security beyond verifier checks.

1.4 Interpretation: What U_F Is

A Floquet Program Token is a **portable commitment to a finite execution trace**.

Two equivalent views (declared by SpecID). EY26 supports two canonical interpretations:

(P1) Operator product token. U_F denotes a finite ordered product:

$$U_F := \prod_{j=1}^m U_j,$$

with a declared order convention (left-to-right or right-to-left) fixed by SpecID.

(P2) Step-list execution token. U_F denotes a step list executed on an object X :

$$X_{j+1} := \text{Step}_j(X_j), \quad j = 0, \dots, m-1.$$

The token commits to the full step list, schedule, and numeric rules.

Rule. A token MUST declare whether it is interpreted as (P1), (P2), or both with an explicit equivalence mapping. If undeclared, the token MUST be rejected as ambiguous.

1.5 Design Goals (Normative)

G1: Deterministic Replay

Executing U_F under the declared NumericModel MUST yield the same intermediate states and transcript commitments across verifiers. No hidden randomness is permitted.

G2: Canonical Program Encoding

The step list MUST be canonically encoded. Reordering steps, changing rounding points, or changing schedule semantics produces a different token.

G3: Boundedness and Anti-Smuggling

Tokens MUST be finite and bounded:

- bounded step count $m \leq m_{\max}$,
- bounded parameter ranges on quantized grids,
- bounded external references, committed by hash if external.

Tokens MUST NOT hide tuning freedom under “implementation details”.

G4: Explicit Failure Codes

When execution fails, it MUST fail deterministically with structured failure codes: domain mismatch, truncation mismatch, overflow, transcript mismatch, etc.

G5: Composability as a First-Class Contract

Tokens SHOULD compose with:

- EY23 (generate deterministic datasets for probing),
- EY24 (include validity constraints or produce validity-audited outputs),
- EY25 (be embedded inside an audit-native evolution workflow),

under explicit interoperability predicates (EY22 Chapter 6).

G6: Conservative Baseline Profile

EY26 MUST provide a baseline token profile (FLOQ-0) that is trivial to verify: small step lists, explicit rounding points, and full transcript commitments.

1.6 Chapter Summary

This chapter motivated EY26 as the missing primitive for making *programs* publishable artifacts. It fixed the scope (finite discrete-time step lists), defined U_F as a portable commitment to execu-

tion semantics, and stated design goals: deterministic replay, canonical encoding, boundedness, explicit failure codes, and composability.

Chapter 2 next defines the normative artifact structure and SpecID binding for U_F : step encoding, schedule semantics, truncation and NumericModel requirements, transcript commitments, and verifier acceptance.

2 Normative Definition and SpecID Binding

2.1 Conformance to EY22

EY26 inherits Vol. No. IV conventions from EY22:

- SpecID binding (EY22 Chapter 3),
- NumericModel determinism (EY22 Chapter 4),
- certificate schema and verifier checks (EY22 Chapter 5, Appendix A.1),
- interoperability vocabulary (EY22 Chapter 6).

Any deviation MUST be explicitly stated in $\text{SpecID}_{\text{EY26}}$.

2.2 Core Objects

A Floquet Program Token packages a finite program as a canonical object.

Program target. A token acts on a declared target type X :

$$X \in \mathcal{X},$$

where $\mathcal{X}$ is a declared space (states, observables, operator tuples, summary vectors, or a declared hybrid object).

Program semantics. EY26 supports:

- **Step-list semantics** (execution): $X_{j+1} := \text{Step}_j(X_j)$,
- **Operator-product semantics** (token operator): $U_F := \prod_{j=1}^m U_j$,

with the choice bound by SpecID.

2.3 Artifact Definition

Definition 2.1 (Floquet Program Token). An EY26 artifact instance is a tuple:

$$U_F := (\text{Sem}, \mathcal{X}, \text{Steps}, \text{Sched}, \text{Trunc}, \text{NumericModel}),$$

together with a certificate π_{FLOQ} that proves:

- the step list Steps is admissible under a declared admissible family $\mathcal{A}_{\text{Steps}}$,
- Sched deterministically binds time slicing / period semantics,
- Trunc and NumericModel deterministically bind evaluation and canonicalization,
- and the published transcript commitments match deterministic replay.

Rule (finite and bounded). A token MUST be finite. Any loop MUST have a bounded iteration count fixed in SpecID. Tokens with unbounded control flow are invalid.

2.4 Step Encoding Steps

Step List Structure

$$\text{Steps} := (\text{Step}_1, \dots, \text{Step}_m), \quad 1 \leq m \leq m_{\max}.$$

Each Step_j MUST be canonically encoded as:

$$\text{Step}_j := (\text{OpType}_j, \text{OpRef}_j, \theta_j, \text{ApplyRule}_j, \text{CanonPoint}_j).$$

Fields.

- OpType_j : step type identifier (e.g. unitary update, channel update, projection, filter call, summary extraction).
- OpRef_j : operator reference: either canonical encoding or hash commitment to the operator object.
- θ_j : parameter tuple on a declared quantized grid.
- ApplyRule_j : how the operator acts on X (left, right, conjugation, superoperator action, etc.).
- CanonPoint_j : whether to canonicalize after this step and how.

Rule. Any field that can change execution output MUST be inside the step encoding. Hidden defaults are forbidden.

Step Types (Baseline Set)

EY26 standardizes a minimal baseline set of step types; future profiles may extend it.

S1: Left action.

$$X \mapsto AX.$$

S2: Right action.

$$X \mapsto XA.$$

S3: Conjugation action.

$$X \mapsto AXA^\dagger,$$

with AdjointSpec and rounding points declared in NumericModel .

S4: Superoperator action.

$$X \mapsto \mathcal{S}(X),$$

where $\mathcal{S}$ is a declared superoperator (may reference EY25).

S5: Projection / penalty step.

$$X \mapsto \text{Proj}(X; V) \quad \text{or} \quad X \mapsto X - \eta \nabla \Phi_V(X),$$

where V may reference EY24 and all hyperparameters η are quantized and explicit.

S6: Filter call. Invoke EY23 filter artifact on a declared object:

$$X \mapsto \text{Filter}(X; F_\star).$$

If used, it MUST bind to EY23 SpecID.

Rule. If a step type references another artifact (EY23/EY24/EY25), the token MUST include interop predicates and references to those artifacts' SpecIDs (or commitments).

2.5 Schedule Semantics Sched

Sched binds Floquet/period semantics and time slicing.

Sched Component (required)

$$\text{Sched} := (\text{PeriodType}, T, \Delta t, m, \text{OrderConv}, \text{RepeatCount}, \text{StopRule}).$$

PeriodType.

- **finite:** execute exactly m steps once,
- **floquet:** interpret the step list as one period and repeat R times (bounded),
- **schedule-list:** execute steps according to an explicit index list (bounded).

Rule (bounded repeats). If repeats are allowed, $\text{RepeatCount} = R$ MUST be fixed and bounded: $R \leq R_{\max}$.

OrderConv. Order convention MUST be explicit:

$$\text{OrderConv} \in \{\text{left-to-right}, \text{right-to-left}\}.$$

This matters for operator-product semantics and for step-list semantics if intermediate canonicalization points are present.

2.6 Truncation Semantics Trunc

Tokens may depend on truncation calculus (finite representations of operators and objects).

Trunc Component (required if infinite objects exist).

$$\text{Trunc} := (\text{Basis}, \text{CutRule}, \text{BoundaryRule}, \{N_\ell\}, \text{CrossNPolicy}).$$

This reuses EY22 truncation vocabulary.

Rule. If truncation is used, the required truncation levels $\{N_\ell\}$ MUST be fixed in SpecID and applied consistently to all steps that require finite representation.

2.7 Transcript Contract

A token MUST publish transcript commitments that bind execution.

TranscriptSpec

$\text{TranscriptSpec} := (\text{CommitType}, \text{HashAlg}, \text{SerializeRule}, \text{CommitRate}, \text{ChainRule}).$

CommitType.

- **full-X:** hash full canonical encoding of X_j at commitment points,
- **summary:** hash a declared summary vector extracted from X_j ,
- **hybrid:** periodic full-X plus per-step summary.

Rule. If a token claims full replay of intermediate states, it MUST commit full-X at the required rate. Summary-only commitments cannot claim full-state replay determinism.

2.8 SpecID for EY26

ArtifactType. For EY26:

$\text{ArtifactType} = \text{FLOQ}.$

SpecID components (EY26 binding). The full SpecID tuple is:

$\text{SpecID}_{\text{EY26}} := (\text{FLOQ}, \text{Sem}, \mathcal{X}, \text{Steps}, \mathcal{A}_{\text{Steps}}, \text{Sched}, \text{Trunc}, \text{NumericModel}, \text{TranscriptSpec}, \text{Cert}, \text{Interop},$

Sem **(required)**

$\text{Sem} \in \{\text{step-list}, \text{operator-product}, \text{both}\}$

MUST be declared.

$\mathcal{X}$ **(required)**

$\mathcal{X}$ MUST declare the target object type and its canonical encoding rule.

Steps and $\mathcal{A}_{\text{Steps}}$ (required)

Even if the step list is provided, $\mathcal{A}_{\text{Steps}}$ MUST exist to enforce bounds, canonical ordering, and parameter grids.

Sched, Trunc, NumericModel, TranscriptSpec (required)

These bind deterministic execution and transcript commitments.

Cert, Interop, Version (**required**)

Cert binds verifier checks; Interop binds compatibility predicates; Version binds meaning and MUST change if any component changes.

2.9 Deterministic Outcomes

EY26 verification MUST return:

$$\{\text{ACCEPT}, \text{REJECT}\} \cup \{\text{FAIL_}\ast\},$$

and MUST support at minimum:

- **FAIL__SCHED__MISMATCH**
- **FAIL__STEP__MALFORMED**
- **FAIL__TRUNC__MISMATCH**
- **FAIL__DOMAIN__MISMATCH**
- **FAIL__OVERFLOW**
- **FAIL__TRANSCRIPT__MISMATCH**
- **FAIL__NONDETERMINISM**
- **FAIL__CONSTRAINT**

2.10 Chapter Summary

This chapter defined EY26 as a canonical token artifact that packages a finite program: a step list with explicit apply rules, a schedule binding for Floquet semantics, truncation and numeric rules for deterministic evaluation, and a transcript contract for verifiable replay. It fixed the SpecID binding and minimal failure codes.

Chapter 3 next defines admissible step families $\mathcal{A}_{\text{Steps}}$: step bounds, parameter grids, canonical ordering, allowed step-type catalogs, and anti-smuggling constraints for program tokens.

3 Admissible Step Families $\mathcal{A}_{\text{Steps}}$: Canonical Program Classes

3.1 Purpose of $\mathcal{A}_{\text{Steps}}$

A token is meaningful only if its step list is meaningfully constrained. Otherwise, “a token” can smuggle arbitrary behavior: hidden tuning via continuous parameters, non-canonical operator encodings, or external dependencies that cannot be replayed.

Therefore, $\mathcal{A}_{\text{Steps}}$ is normative. It specifies:

- which step types are allowed,
- how each step is parameterized and bounded,
- which operator references are permitted and how they are committed,
- and how the full program is canonically ordered and verified.

Rule. A step list Steps is admissible only if:

- each step belongs to an allowed step-type catalog,
- all parameters lie on declared quantized grids with explicit bounds,
- all operator references are canonical or hash-committed,
- the overall program satisfies global boundedness (step count, repeat count, memory bounds),
- and membership is checkable deterministically under NumericModel.

3.2 Canonical Encoding and Anti-Ambiguity

Rule (canonical step serialization). Every step Step_j MUST have a canonical encoding $\text{Enc}(\text{Step}_j)$ and MUST be serializable in a fixed field order:

$$(\text{OpType}, \text{OpRef}, \theta, \text{ApplyRule}, \text{CanonPoint}).$$

No optional fields may be omitted silently; if a field is not used it MUST be explicitly `null`.

Rule (program encoding). The token encoding is:

$$\text{Enc}(U_F) := \text{Enc}(\text{Sem}) \parallel \text{Enc}(\mathcal{X}) \parallel \text{Enc}(\text{Sched}) \parallel \text{Enc}(\text{Trunc}) \parallel \text{Enc}(\text{NumericModel}) \parallel \text{Enc}(\text{Steps}),$$

with Steps encoded as a length-prefixed list of step encodings.

Rule (no hidden defaults). Any behavior that can affect outputs (rounding points, boundary conventions, order conventions, stopping rules) MUST be explicitly encoded. If it is not encoded, it does not exist.

3.3 Step-Type Catalog (Baseline)

EY26 defines a baseline catalog $\mathcal{C}_{\text{step}}$. Tokens may extend it only by declaring a new catalog ID and publishing its formal semantics and verifier rules.

Baseline catalog.

$$\mathcal{C}_{\text{step}}^{(0)} := \{\text{S1}, \text{S2}, \text{S3}, \text{S4}, \text{S5}, \text{S6}\},$$

where S1–S6 are the baseline step types from Chapter 2.

Rule. A FLOQ-0 token (baseline profile) MUST use only $\mathcal{C}_{\text{step}}^{(0)}$.

3.4 Parameter Grids and Bounds

To prevent tuning smuggling, all parameters MUST be quantized.

Per-step parameter grid

Each step type OpType has an associated parameter grid:

$$\theta_j \in \mathcal{G}_{\text{OpType}_j}.$$

Grid structure (normative). A grid MUST be defined by:

$$\mathcal{G} := (\text{Type}, \text{Step}, \text{Bound}, \text{EncodingRule}),$$

where:

- $\text{Type} \in \{\text{fixed-point}, \text{integer}, \text{enum}\}$,
- Step is the quantization step size (if numeric),
- Bound is an explicit max magnitude (or min/max range),
- EncodingRule defines canonical bytes.

Rule. Continuous parameters are forbidden unless they are discretized by a declared grid.

Global bounds

In addition to per-step bounds, a token MUST satisfy:

$$m \leq m_{\max}, \quad R \leq R_{\max},$$

and may include resource bounds (informational unless declared):

$$\text{WorkProxy}(U_F) \leq B_{\text{work}}, \quad \text{MemProxy}(U_F) \leq B_{\text{mem}}.$$

3.5 Operator Reference Policy

Each step may reference an operator or superoperator object.

OpRef modes. OpRef MUST be one of:

- **inline:** canonical encoding of the operator at required truncation levels,
- **commit:** a hash commitment, plus a deterministic reconstruction/truncation rule,
- **artifact-ref:** a reference to another Vol. No. IV artifact SpecID (e.g. EY23/EY24/EY25), plus its commitment hash.

Rule. If OpRef is **commit** or **artifact-ref**, the commitment MUST be included in the certificate and MUST match replay.

Rule (no external state). OpRefs MUST NOT depend on non-committed external state. If an operator is loaded from a file, the file hash must be committed.

3.6 Canonical Ordering and Normal Form

Even though execution order matters, there are token-level canonicalization requirements.

Program normal form

A step list is in *normal form* if:

- step indices are consecutive $1, \dots, m$,
- all CanonPoint_j are explicitly declared,
- all ApplyRule_j are explicit and match Sem conventions,
- all parameters θ_j are in canonical encoding,
- and all OpRefs satisfy the reference policy.

Rule. Tokens not in normal form MUST be rejected (**FAIL_STEP_MALFORMED**).

Optional canonical step reordering (disabled by default)

EY26 allows the possibility of a *canonical reordering rule* only if the token declares an equivalence class (e.g. commuting steps). This is **disabled by default**.

Rule (default). Unless a token explicitly declares a step-commutation equivalence and a deterministic reordering rule, *no* reordering is permitted. The listed order is the executed order.

3.7 Membership Check Procedure

A verifier checks $\text{Steps} \in \mathcal{A}_{\text{Steps}}$ deterministically:

- parse token encoding and step list,
- check $m \leq m_{\text{max}}$ and bounded repeats,
- for each step: check OpType is allowed, OpRef is valid, parameters lie on grids, ApplyRule and CanonPoint are valid,
- check all commitments resolve deterministically,
- check program is in normal form,
- return deterministic failure codes if violated.

3.8 Baseline Profile FLOQ-0 (Admissibility Defaults)

EY26 defines a conservative baseline admissibility profile.

FLOQ-0 admissibility. A FLOQ-0 token MUST satisfy:

- allowed step catalog: $\mathcal{C}_{\text{step}}^{(0)}$ only,
- m_{max} fixed and small (e.g. $m_{\text{max}} = 256$),
- repeat count disabled by default ($R = 1$) unless explicitly enabled,
- parameters on fixed-point grids with explicit bounds,
- OpRefs either inline or hash-committed (artifact-ref allowed only if full SpecIDs are provided),

- full-X transcript commitments at every step (unless declared otherwise).

3.9 Chapter Summary

This chapter defined $\mathcal{A}_{\text{Steps}}$ for EY26: a normative admissibility framework for program tokens. It fixed canonical step encoding, baseline step-type catalog, quantized parameter grids and global bounds, operator reference policy, program normal form, and deterministic membership checks. It also defined FLOQ-0 admissibility defaults.

Chapter 4 next specifies schedule and execution semantics in detail: Floquet period interpretation, repeat rules, canonicalization points during execution, transcript generation, and deterministic failure modes for token execution.

4 Schedule, Execution Semantics, and Deterministic Replay

4.1 Purpose

A token is not only a list of steps. It also fixes *how* steps are executed: time slicing, repetition semantics, ordering conventions, canonicalization points, and transcript commitments. This chapter defines:

- schedule semantics (finite vs Floquet period vs schedule-list),
- execution semantics for step-list and operator-product interpretations,
- canonicalization points and rounding boundaries during execution,
- transcript generation rules,
- and deterministic failure codes for token execution.

Rule. If the same token encoding can produce different outputs on different runtimes, it is not an EY26 token. Execution **MUST** be replayable.

4.2 Schedule Semantics Sched

Recall:

$$\text{Sched} := (\text{PeriodType}, T, \Delta t, m, \text{OrderConv}, \text{RepeatCount}, \text{StopRule}).$$

PeriodType: finite

Meaning. Execute the step list exactly once:

$$j = 1, \dots, m.$$

RepeatCount **MUST** equal 1. If RepeatCount $\neq 1$, reject with **FAIL_SCHED_MISMATCH**.

PeriodType: floquet

Meaning. Interpret the step list as one period and repeat it R times:

$$(j, r) \in \{1, \dots, m\} \times \{1, \dots, R\},$$

with $R = \text{RepeatCount}$ fixed and bounded.

Rule (bounded). R MUST be fixed in SpecID and satisfy $R \leq R_{\max}$. Otherwise fail **FAIL_SCHED_MISMATCH**.

PeriodType: schedule-list

Meaning. Execute steps according to an explicit index list:

$$\mathcal{I} = (i_1, \dots, i_L), \quad i_\ell \in \{1, \dots, m\},$$

where $\mathcal{I}$ is fixed and canonically encoded in SpecID.

Rule. The index list MUST be bounded ($L \leq L_{\max}$) and canonical. Out-of-range indices cause **FAIL_SCHED_MISMATCH**.

4.3 Time Binding (Two-of-Three Rule)

Because Floquet language suggests time, EY26 standardizes time binding even for discrete programs.

Rule (two-of-three binding). The SpecID MUST fix any two of:

$$(T, \Delta t, N_{\text{ticks}}),$$

and deterministically derive the third. Here N_{ticks} is the number of executed step applications:

$$N_{\text{ticks}} := \begin{cases} m, & \text{PeriodType} = \text{finite} \\ mR, & \text{PeriodType} = \text{floquet} \\ L, & \text{PeriodType} = \text{schedule-list} \end{cases}$$

If derivation is not consistent, fail **FAIL_SCHED_MISMATCH**.

Note. EY26 does not require that each program step corresponds to physical time; time is a *semantic coordinate* for audit and interop.

4.4 Execution Semantics

Let X_0 be the public initial object in the declared space $\mathcal{X}$.

Step-List Semantics (Primary)

If Sem = step-list or both, execution is:

Tick order. Define a tick order $\tau = 1, \dots, N_{\text{ticks}}$ and map each tick to a step index $j(\tau)$ by:

$$j(\tau) := \begin{cases} \tau, & \text{finite} \\ 1 + ((\tau - 1) \bmod m), & \text{floquet} \\ i_\tau, & \text{schedule-list} \end{cases}$$

Then:

$$X_\tau := \text{Apply}(\text{Step}_{j(\tau)}, X_{\tau-1}).$$

OrderConv. OrderConv MUST still be declared, because it fixes how products are interpreted if the token also claims operator-product semantics, and it fixes the interpretation of composite ApplyRules that involve multiple operator multiplications.

Operator-Product Semantics (Secondary)

If Sem = operator-product or both, the token defines:

$$U_F := \prod_{j=1}^m U_j$$

under the declared OrderConv. If PeriodType = floquet, then the total token is:

$$U_{\text{tot}} := (U_F)^R.$$

Rule. If Sem = both, the token MUST declare whether:

- (E1) product semantics are only an *annotation*, or
- (E2) product semantics are asserted equivalent to step-list execution under declared canonicalization points.

If equivalence is asserted, the verifier MUST check it at the declared commitment points, otherwise **FAIL_CONSTRAINT**.

4.5 ApplyRule Semantics

Each step includes an ApplyRule. EY26 standardizes a minimal set.

Baseline ApplyRule forms

- **AR1 left:** $X \mapsto AX$
- **AR2 right:** $X \mapsto XA$
- **AR3 conj:** $X \mapsto AXA^\dagger$
- **AR4 super:** $X \mapsto \mathcal{S}(X)$
- **AR5 update:** $X \mapsto \text{Update}(X; \theta)$ with explicit formula

Rule. If ApplyRule refers to an algorithmic update (AR5), the algorithm MUST be fully specified and deterministic under NumericModel (no adaptive stopping unless explicitly bound).

4.6 Canonicalization Points During Execution

NumericModel MUST declare canonicalization points for token execution.

CanonPoint per step. Each Step_j includes CanonPoint_j ∈ {none, after, after + summary} (or an explicitly defined enum).

Rule (minimum canonicalization). A FLOQ-0 token MUST canonicalize after every tick:

$$X_\tau \xrightarrow{\text{Canon}} \hat{X}_\tau$$

and use $\hat{X}_\tau$ for transcript commitments.

Rule (deterministic rounding). Canonicalization MUST specify:

- rounding mode,
- precision or fixed-point grid,
- ordering for reductions and serialization.

4.7 Transcript Generation

Recall:

$\text{TranscriptSpec} := (\text{CommitType}, \text{HashAlg}, \text{SerializeRule}, \text{CommitRate}, \text{ChainRule}).$

Commitment points

CommitRate defines a set of commitment ticks $\mathcal{T}_{\text{commit}} \subseteq \{0, \dots, N_{\text{ticks}}\}$.

Rule. $\mathcal{T}_{\text{commit}}$ MUST be derivable deterministically from SpecID and public inputs. If not, fail **FAIL__SCHED__MISMATCH**.

Commitment computation

Let Y_τ be the committed object at tick τ (full-X or summary). Then:

$$\text{Tx}_\tau := \text{Hash}(\text{Enc}(Y_\tau))$$

or, if chained:

$$\text{Tx}_\tau := \text{Hash}(\text{Tx}_{\tau^-} \parallel \text{Enc}(Y_\tau)),$$

where τ^- is the previous commitment tick.

Rule. If summary commitments are used, the summary function MUST be deterministic and MUST be included in SpecID (or referenced by artifact-ref).

4.8 Deterministic Failure Modes

Token execution MUST fail deterministically when requirements are violated.

Required failure codes

At minimum:

- **FAIL__SCHED__MISMATCH:** schedule derivation fails, invalid indices, repeats out of bounds.

- **FAIL_STEP_MALFORMED**: invalid step encoding, missing fields, non-canonical params.
- **FAIL_CONSTRAINT**: admissibility constraints violated (grid/bounds, equivalence assertions fail).
- **FAIL_DOMAIN_MISMATCH**: ApplyRule not defined for the declared $\mathcal{X}$ under Calc/AdjointSpec.
- **FAIL_TRUNC_MISMATCH**: truncation calculus inconsistent across steps or required N .
- **FAIL_OVERFLOW**: NaN/Inf/overflow occurs where forbidden.
- **FAIL_NONDETERMINISM**: replay differs at declared canonicalization points.
- **FAIL_TRANSCRIPT_MISMATCH**: transcript commitments do not match replay.

4.9 Deterministic Replay Definition

Definition 4.1 (Token Replay Determinism). An EY26 token is replay-deterministic if, for any verifier executing the declared schedule, step list, truncation, and NumericModel on the same public inputs, the produced commitment sequence T_x and the committed objects Y_τ match exactly (or within declared tolerance at declared canonicalization points).

Rule. If a token claims full-state replay, it **MUST** commit full-X at the declared commitment rate. Summary-only commitments can claim only summary replay.

4.10 Chapter Summary

This chapter fixed schedule semantics (finite/Floquet/schedule-list), execution semantics for step-list and operator-product interpretations, canonicalization points and rounding boundaries, transcript generation rules, and deterministic failure modes. It defined replay determinism as the central correctness notion for program tokens.

Chapter 5 next specifies the certificate payload π_{FLOQ} and verifier checks: step admissibility, schedule derivation, execution replay, transcript matching, and interoperability checks when composing with EY23/EY24/EY25/EY29/EY28.

5 Certificate and Verifier: Execution Replay and Acceptance

5.1 Purpose

EY26 publishes programs as auditable objects. This chapter specifies:

- the certificate payload π_{FLOQ} required to publish a Floquet Program Token,
- deterministic verifier checks for admissibility, replay, and transcript commitment matching,
- and required failure codes.

Certificates follow EY22 canonical certificate encoding (CCE) conventions (EY22 Chapter 5, Appendix A.1) and add token-specific fields.

5.2 Certificate Object π_{FLOQ}

Rule. An EY26 token MUST be published with a certificate π_{FLOQ} that includes, at minimum, the following canonical content.

Header (required)

- artifact type: FLOQ
- artifact symbol: U_F
- SpecID short form and full SpecID tuple
- version identifier

SpecID (required)

The full SpecID tuple MUST be embedded:

(FLOQ, Sem, $\mathcal{X}$, Steps, $\mathcal{A}_{\text{Steps}}$, Sched, Trunc, NumericModel, TranscriptSpec, Cert, Interop, Version).

Inputs (required)

Inputs MUST include:

- canonical encoding (or hash commitment) of the full step list Steps,
- canonical encoding of $\mathcal{A}_{\text{Steps}}$ (or its ID + committed definition),
- full Sched object and all public inputs used to derive schedule values,
- full Trunc object (if used) and required truncation levels $\{N_\ell\}$,
- full NumericModel (rounding, ordering, canonicalization points, tolerances),
- full TranscriptSpec (commit type, hash algorithm, serialization, commit rate, chain rule),
- initial object X_0 (or its commitment + deterministic reconstruction rule),
- all external OpRefs (operators, catalogs, datasets) committed by hash according to EY22 Appendix A.1,
- any artifact-refs used by steps (EY23/EY24/EY25 etc.) as SpecIDs + commitment hashes.

Outputs (required)

Outputs MUST include:

- transcript commitments Tx at all required commitment ticks,
- final committed object $Y_{N_{\text{ticks}}}$ (full-X or summary as declared),
- any declared summary vectors used by commitments or checks,
- hash commitments to the final token encoding and to the transcript sequence itself.

Checks (required)

The certificate MUST list check IDs and what each check verifies, grouped as:

- structural and SpecID checks,
- step admissibility checks,
- schedule derivation checks,
- truncation checks (if used),
- replay execution checks,
- transcript commitment checks,
- interoperability checks (if claimed).

5.3 Verifier Contract

A verifier takes π_{FLOQ} and deterministically returns:

$$\{\text{ACCEPT}, \text{REJECT}\} \cup \{\text{FAIL_}\ast\}.$$

Rule. The verifier MUST NOT depend on producer identity, informal descriptions, or uncommitted runtime details.

5.4 Required Verifier Checks (Normative)

This section enumerates minimal checks required for EY26 acceptance.

F1: Structural Validity (CCE)

- parse canonical certificate encoding (EY22 CCE rules),
- reject malformed encodings, duplicate keys, non-canonical numbers,
- confirm required fields exist.

Failure: **REJECT_MALFORMED**.

F2: SpecID Completeness and Consistency

- SpecID tuple is present and complete,
- ArtifactType = FLOQ,
- Sem, $\mathcal{X}$, Steps, Sched are present,
- NumericModel and TranscriptSpec are present,
- Trunc is present iff required by target space or OpRefs.

Failure: **REJECT_SPECID_MISSING**, **REJECT_SPECID_MISMATCH**.

F3: No Hidden Inputs

- all OpRefs resolve to canonical encodings or committed objects,
- schedule derivations depend only on public committed inputs,
- no undeclared randomness, I/O, or external state exists.

Failure: **REJECT_HIDDEN_INPUT**, **FAIL_HASH_MISMATCH**.

F4: Step Admissibility (Membership in $\mathcal{A}_{\text{Steps}}$)

Verify $\text{Steps} \in \mathcal{A}_{\text{Steps}}$ deterministically:

- parse canonical step encodings; all required fields present,
- verify $m \leq m_{\max}$,
- verify each OpType belongs to the allowed catalog ID,
- verify θ_j lies on declared grids and satisfies bounds,
- verify ApplyRule is valid for declared $\mathcal{X}$ and Calc/AdjointSpec,
- verify CanonPoint fields are valid and explicit,
- verify any artifact-refs provide full SpecIDs + commitments,
- verify program normal form (Chapter 3).

Failure: **FAIL_STEP_MALFORMED** or **FAIL_CONSTRAINT**.

F5: Schedule Derivation and Consistency

- derive N_{ticks} from PeriodType and bounds (finite/floquet/schedule-list),
- verify RepeatCount $R \leq R_{\max}$ if applicable,
- verify time binding (two-of-three rule) where used,
- verify index list bounds and in-range indices for schedule-list.

Failure: **FAIL_SCHED_MISMATCH**.

F6: Truncation Protocol Check (If Used)

- verify Trunc object is complete (Basis, CutRule, BoundaryRule, required N_{ℓ}),
- verify truncation maps for all OpRefs are deterministic,
- verify CrossNPolicy if multiple truncation levels appear.

Failure: **FAIL_TRUNC_MISMATCH**.

F7: Replay Execution (Deterministic)

Run the declared token execution from X_0 for all ticks:

- apply steps using the declared tick mapping $j(\tau)$,

- enforce OrderConv and ApplyRules,
- apply canonicalization at declared CanonPoints,
- compute commitments at commitment ticks.

Failures:

- **FAIL_DOMAIN_MISMATCH** if a step action is not defined,
- **FAIL_OVERFLOW** if NaN/Inf/overflow occurs where forbidden,
- **FAIL_NONDETERMINISM** if replay differs at canonicalization points.

F8: Transcript Commitment Check

Compare replayed commitments to published commitments:

$$Tx_{\tau}^{(\text{replay})} \stackrel{?}{=} Tx_{\tau}^{(\text{published})} \quad \text{for all } \tau \in \mathcal{T}_{\text{commit}}.$$

Failure: **FAIL_TRANSCRIPT_MISMATCH**.

F9: Optional Equivalence Check (If Sem = both)

If the token asserts equivalence between step-list execution and operator-product semantics, the verifier MUST check equivalence at declared commitment points:

$$\hat{X}_{\tau}^{(\text{step})} \stackrel{?}{=} \hat{X}_{\tau}^{(\text{prod})} \quad \text{for all required } \tau.$$

Failure: **FAIL_CONSTRAINT**.

F10: Interop Checks (Only if Claimed)

If interop is claimed with EY23/EY24/EY25/EY29/EY28, the verifier MUST evaluate the declared compatibility predicates under shared NumericModel and declared calculus (EY22 Chapter 6). Failure: **FAIL_COMPAT_PREDICATE** (or the specific EY22 interop failure code).

5.5 Required Failure Codes

EY26 MUST support at minimum:

- **REJECT_MALFORMED**
- **REJECT_SPECID_MISSING**
- **REJECT_SPECID_MISMATCH**
- **REJECT_HIDDEN_INPUT**
- **FAIL_HASH_MISMATCH**
- **FAIL_SCHED_MISMATCH**
- **FAIL_STEP_MALFORMED**
- **FAIL_TRUNC_MISMATCH**

- **FAIL_DOMAIN_MISMATCH**
- **FAIL_OVERFLOW**
- **FAIL_NONDETERMINISM**
- **FAIL_TRANSCRIPT_MISMATCH**
- **FAIL_CONSTRAINT**
- **FAIL_COMPAT_PREDICATE**

5.6 Chapter Summary

This chapter specified the EY26 certificate payload π_{FLOQ} and deterministic verifier checks required to accept a Floquet Program Token. It fixed checks for step admissibility, schedule derivation, truncation protocol correctness, replay execution, transcript commitment matching, and interoperability predicates when claimed.

Chapter 6 next defines interoperability declarations (mini) for EY26: composition rules with EY23 probing, EY24 validity penalties, and embedding within EY25 audit-native evolution, including required compatibility predicates and failure codes.

6 Interop Declarations (Mini): Composition with EY23/EY24/EY25/EY29

6.1 Purpose

This chapter instantiates EY22 interoperability rules (EY22 Chapter 6) for EY26. It defines minimal compatibility predicates for composing the Floquet Program Token U_F with:

- EY23 (CSF) Canonical Spectral Filter $F_\star(H)$,
- EY24 (VPO) Validity Penalty Operator $V = C^\dagger C$,
- EY25 (LIOUV) Audit-Native Liouvillian $\mathcal{L}_{\text{audit}}$,
- EY29 (CGR) Coarse-Grain Reduction R ,
- EY28 (INTW) Regime Transport $T_{1 \rightarrow 2}$.

Rule. If an EY26 token claims interoperability beyond what is stated here, it **MUST** introduce additional compatibility predicates inside its Interop SpecID component and **MUST** remain checkable under the declared NumericModel.

6.2 Interop Component (EY26)

EY26 declares the following baseline composition forms (EY22 C1–C5):

$$\text{Interop}_{\text{EY26}} := \{C5, C4, C1\},$$

i.e. token execution (C5), superoperator embedding (C4), and algebraic operator composition (C1). Coarse-grain (C3) and transport (C2) are supported only via explicit predicates below.

6.3 Compatibility Predicates

All compatibility predicates below are deterministic booleans under the declared `NumericModel(s)`. Unless explicitly stated, compatibility is asymmetric (EY22).

FLOQ→CSF: Token-Generated Data for Spectral Probing

A common workflow is:

$$X_0 \xrightarrow{U_F} \{X_\tau\}_{\tau \in \mathcal{T}} \Rightarrow \text{EIF feed } \{X_\tau\} \text{ (or summaries) into } F_\star(H)\text{-based probing.}$$

Predicate $\text{Compat}_{26,23}^{(1)}$ (**interface type**).

$\text{Compat}_{26,23}^{(1)} := (\text{EIF the EY23 input type matches what the token publishes: full objects or declared summaries})$.

Predicate $\text{Compat}_{26,23}^{(2)}$ (**NumericModel alignment**).

$\text{Compat}_{26,23}^{(2)} := (\text{NumericModel}_{\text{EY26}} = \text{NumericModel}_{\text{EY23}}) \vee (\text{declared deterministic coercion})$.

Predicate $\text{Compat}_{26,23}^{(3)}$ (**commitment sufficiency**).

$\text{Compat}_{26,23}^{(3)} := (\text{EIF the data used by EY23 probing is committed by the EY26 transcript at required ticks})$.

Compatibility status.

$$\text{Compat}(\text{EY26}, \text{EY23}) := \text{Compat}_{26,23}^{(1)} \wedge \text{Compat}_{26,23}^{(2)} \wedge \text{Compat}_{26,23}^{(3)}.$$

If TRUE, mark OK*.

FLOQ↔VPO: Validity-Constrained Tokens

Tokens may incorporate validity in two canonical ways:

- include explicit penalty/projection steps that reference V or C (S5),
- or commit to outputs together with validity metrics required by EY24.

Predicate $\text{Compat}_{26,24}^{(1)}$ (**EY24 binding**).

$\text{Compat}_{26,24}^{(1)} := (\text{EY24Ref binds to full EY24 SpecID and certificate commitment})$.

Predicate $\text{Compat}_{26,24}^{(2)}$ (**mode declaration**).

$\text{Compat}_{26,24}^{(2)} := (\text{EIF token declares how validity is used: S5 step, metrics only, or both})$.

Predicate $\text{Compat}_{26,24}^{(3)}$ (**deterministic validity computation**).

$\text{Compat}_{26,24}^{(3)} := (\text{EIF validity scores, projections, or penalties are computed deterministically under NumericModel})$.

Compatibility status.

$$\text{Compat}(\text{EY26}, \text{EY24}) := \text{Compat}_{26,24}^{(1)} \wedge \text{Compat}_{26,24}^{(2)} \wedge \text{Compat}_{26,24}^{(3)}.$$

If TRUE, mark OK*.

FLOQ $\leftrightarrow$ LIOUV: Embedding Tokens Inside Audit-Native Evolution

There are two canonical compositions:

(TL1) token as a discrete-time evolution map. Treat the token as an evolution operator per period and compare to an EY25 evolution transcript over matching ticks.

(TL2) token steps include EY25 superoperator calls. Include steps of type S4 where $\mathcal{S} = \mathcal{L}_{\text{audit}}$ (or its induced step map) is invoked.

Predicate $\text{Compat}_{26,25}^{(1)}$ (semantic declaration).

$\text{Compat}_{26,25}^{(1)} := (\text{EIF workflow declares TL1 or TL2 and fixes alignment of tick schedule to EY25 step schedule})$

Predicate $\text{Compat}_{26,25}^{(2)}$ (NumericModel and truncation alignment).

$\text{Compat}_{26,25}^{(2)} := (\text{NumericModel}_{\text{EY26}} = \text{NumericModel}_{\text{EY25}}) \vee (\text{declared deterministic coercion}),$
and truncation policies must match or be bridged deterministically.

Predicate $\text{Compat}_{26,25}^{(3)}$ (transcript cross-binding).

$\text{Compat}_{26,25}^{(3)} := (\text{EIF a claim depends on both artifacts, the published transcripts must include shared commitment})$

Compatibility status.

$$\text{Compat}(\text{EY26}, \text{EY25}) := \text{Compat}_{26,25}^{(1)} \wedge \text{Compat}_{26,25}^{(2)} \wedge \text{Compat}_{26,25}^{(3)}.$$

If TRUE, mark OK*.

FLOQ $\leftrightarrow$ CGR: Reduced Tokens and Coarse-Grain Execution

Reduction may occur:

- after each tick (commit reduced objects),
- or at selected commitment ticks only,
- or by defining a reduced token whose steps act on reduced objects.

Predicate $\text{Compat}_{26,29}^{(1)}$ (reduction placement).

$\text{Compat}_{26,29}^{(1)} := (\text{EIF workflow declares when reduction occurs relative to canonicalization and transcript commitment})$

Predicate $\text{Compat}_{26,29}^{(2)}$ (**invariant preservation**).

$\text{Compat}_{26,29}^{(2)} :=$ (EIF invariants required by commitments or downstream artifacts are preserved under R with

Predicate $\text{Compat}_{26,29}^{(3)}$ (**deterministic reduced commitments**).

$\text{Compat}_{26,29}^{(3)} :=$ (EIF reduced objects/summaries are computed deterministically under NumericModel and com

Compatibility status.

$$\text{Compat}(\text{EY26}, \text{EY29}) := \text{Compat}_{26,29}^{(1)} \wedge \text{Compat}_{26,29}^{(2)} \wedge \text{Compat}_{26,29}^{(3)}.$$

If TRUE, mark OK*.

FLOQ $\leftrightarrow$ INTW: Transporting Tokens Across Regimes

A token may be transported by transporting its referenced operators, step semantics, and target space.

Canonical semantics.

- (TR-T1) transport each referenced operator A via EY28 and rebuild the token,
- (TR-T2) transport the executed trajectory objects X_τ and commit to transported transcripts.

Predicate $\text{Compat}_{26,28}^{(1)}$ (**transport declaration**).

$\text{Compat}_{26,28}^{(1)} :=$ (EIF workflow declares TR-T1 or TR-T2 and provides required inverses/adjoints deterministically)

Predicate $\text{Compat}_{26,28}^{(2)}$ (**semantic coherence**).

$\text{Compat}_{26,28}^{(2)} :=$ (EIF ApplyRules and AdjointSpec remain coherent under the mapping semantics).

Predicate $\text{Compat}_{26,28}^{(3)}$ (**transcript coherence**).

$\text{Compat}_{26,28}^{(3)} :=$ (EIF transported execution produces deterministic commitments under the declared serializat

Compatibility status.

$$\text{Compat}(\text{EY26}, \text{EY28}) := \text{Compat}_{26,28}^{(1)} \wedge \text{Compat}_{26,28}^{(2)} \wedge \text{Compat}_{26,28}^{(3)}.$$

If TRUE, mark OK*.

6.4 Mini Compatibility Matrix Row (EY26 Perspective)

This minimal row is informative and must be backed by predicates above:

	EY23	EY24	EY25	EY26	EY27	EY28	EY29	EY30
EY26	OK*	OK*	OK*	N/A	OK*	OK*	OK*	OK*

Notes (informational).

- EY26×EY27 is OK* because commutant/symmetry constraints can restrict allowed step families.
- EY26×EY30 is OK* because Koopman export can map token-generated trajectories into observable-space representations.

6.5 Interop Failure Codes (Required)

Interop verification MUST use deterministic failure codes, at minimum:

- **FAIL__DOMAIN__MISMATCH**
- **FAIL__CALC__MISMATCH**
- **FAIL__NUMERICMODEL__MISMATCH**
- **FAIL__TRUNC__MISMATCH**
- **FAIL__TRANSCRIPT__MISMATCH**
- **FAIL__INVARIANT__NOT__PRESERVED**
- **FAIL__COMPAT__PREDICATE**

6.6 Chapter Summary

This chapter provided minimal EY26 interoperability declarations, defining concrete compatibility predicates for composing U_F with EY23 probing, EY24 validity penalties, and embedding or alignment with EY25 audit-native evolution, as well as optional composition with EY29 coarse-grain reduction and EY28 transport across regimes. It also provided an EY26 mini compatibility-matrix row and fixed required failure codes for interoperability failures.

Chapter 7 next defines the FLOQ-0 reference profile: a concrete SpecID instance with fixed admissible step catalog, schedule binding, canonicalization points, transcript commitments, and certificate requirements suitable for universal verification.

7 FLOQ-0 Reference Profile and Example SpecID Instance

7.1 Purpose

Chapters 1–6 defined EY26 in full generality. This chapter provides a concrete, audit-friendly baseline configuration called **FLOQ-0**. FLOQ-0 is designed to be:

- simple enough to verify by third parties without specialized tooling,
- deterministic by construction (replayable transcript),
- conservative in degrees of freedom (anti-smuggling),
- and a stable reference meaning of “program token” for Vol. No. IV.

Rule. A FLOQ-0 token MUST be fully specified by explicit constants (or hash commitments) and MUST NOT require instance-specific tuning.

7.2 FLOQ-0 Summary (Normative)

FLOQ-0 fixes:

- semantics: Sem = step-list (operator-product semantics optional but not asserted equivalent),
- schedule: PeriodType = finite (no repeats),
- step catalog: baseline $\mathcal{C}_{\text{step}}^{(0)}$ only,
- bounded length: $m \leq m_{\text{max},0}$ with a small fixed bound,
- canonicalization: after every tick (and at any commitment point),
- transcript: full-X commitment every tick, optionally hash-chained,
- truncation: fixed basis and cut rule if required by OpRefs,
- certificate: EY22-CCE with required checks F1–F8 (and F10 only if interop is claimed).

Normative note. FLOQ-0 is intentionally conservative. It makes “a program” maximally verifiable: everything is explicit, bounded, and committed.

7.3 FLOQ-0 Step Admissibility Profile

Admissible Family $\mathcal{A}_{\text{Steps},0}$

$\mathcal{A}_{\text{Steps},0} := \left\{ \text{Steps} \mid \text{Steps} \in (\mathcal{C}_{\text{step}}^{(0)})^{\leq m_{\text{max},0}}, \theta_j \in \mathcal{G}_0(\text{OpType}_j), \text{OpRef}_j \in \{\text{inline}, \text{commit}, \text{artifact-ref}^*\} \right\}$,
where artifact-ref* is allowed only if full SpecID + commitment hash are included.

Fixed constants (FLOQ-0). FLOQ-0 fixes:

$$m_{\text{max},0}, \quad \mathcal{C}_{\text{step}}^{(0)}, \quad \mathcal{G}_0, \quad \text{ApplyRuleCatalog}_0, \quad \text{CanonPointEnum}_0.$$

Grid defaults. All numeric parameters use fixed-point grids:

$$\theta \in \{k \cdot \delta_0 \mid k \in \mathbb{Z}, |k| \leq K_0\},$$

with δ_0 and K_0 fixed in SpecID.

Rule. If any parameter is not on the declared grid, reject with **FAIL_CONSTRAINT**.

7.4 FLOQ-0 Schedule Profile

Sched₀

$\text{Sched}_0 := (\text{PeriodType} = \text{finite}, T_0, \Delta t_0, m, \text{OrderConv}_0, \text{RepeatCount} = 1, \text{StopRule} = \text{none}).$

Time binding. FLOQ-0 fixes T_0 and Δt_0 , and binds:

$$m = N_{\text{ticks}} = \frac{T_0}{\Delta t_0}.$$

If this is not an integer under the declared GridRule (reject-if-not-integer), fail **FAIL_SCHED_MISMATCH**.

Order convention.

$$\text{OrderConv}_0 \in \{\text{left-to-right}, \text{right-to-left}\}$$

MUST be fixed, even though Sem is step-list, because it fixes product annotations and multi-mul apply rules.

7.5 FLOQ-0 Truncation Profile (If Needed)

Trunc₀

If truncation is required:

$$\text{Trunc}_0 := (\text{Basis}_0, \text{CutRule}_0, \text{BoundaryRule}_0, N_0, \text{CrossNPolicy} = \text{none}).$$

Rule. FLOQ-0 uses a *single* truncation level N_0 unless the token explicitly declares a finite list. Multiple truncation levels introduce complexity and are reserved for non-baseline profiles.

7.6 FLOQ-0 NumericModel and Canonicalization

FLOQ-0 requires a minimal deterministic NumericModel:

- arithmetic type: deterministic fixed-point or deterministic float with explicit rounding,
- rounding mode and precision,
- matrix multiplication order and accumulation order,
- canonicalization after each tick,
- tolerances (default exact for fixed-point; otherwise explicit $\varepsilon_{\text{step}}$).

CanonPoints. FLOQ-0 canonicalizes:

$$X_\tau \xrightarrow{\text{Canon}} \hat{X}_\tau \quad \text{for all } \tau = 0, \dots, m,$$

and transcript commitments bind $\hat{X}_\tau$.

7.7 FLOQ-0 Transcript Profile

TranscriptSpec₀

$\text{TranscriptSpec}_0 := (\text{CommitType} = \text{full-X}, \text{HashAlg}_0, \text{SerializeRule} = \text{EY22-CCE}, \text{CommitRate} = \text{every tick},$

ChainRule. FLOQ-0 permits:

- $\text{ChainRule}_0 = \text{none}$, or
- $\text{ChainRule}_0 = \text{hash-chain}$.

The choice MUST be fixed in SpecID.

7.8 FLOQ-0 Certificate Requirements

FLOQ-0 requires at minimum the checks:

- F1 (CCE structural),
- F2 (SpecID consistency),
- F3 (no hidden inputs),
- F4 (step admissibility),
- F5 (schedule derivation),
- F6 (truncation protocol, if used),
- F7 (replay execution),
- F8 (transcript matching).

Interop checks (F10) are required only if interop is claimed.

7.9 Reference SpecID Instance (FLOQ-0 Template)

The following is a template for a FLOQ-0 SpecID instance. Numeric constants are placeholders until locked.

```
SpecID_EY26_FLOQ0 :=
(
  ArtifactType = FLOQ,
  Sem = "step-list",
  XSpace = {
    type: "state-or-hybrid",
    encoding: "canonical"
  },
  Steps = [
    { OpType:"S3-conj", OpRef:Hash(U1), theta:null, ApplyRule:"AR3", CanonPoint:"after" },
    { OpType:"S3-conj", OpRef:Hash(U2), theta:null, ApplyRule:"AR3", CanonPoint:"after" },
    ...
  ],
  A_Steps = {
    catalog: "C_step_0",
    m_max: 256,
    grids: { default_fixed_point: { delta: 2-20, bound: 8.0 } },
    opref_policy: ["inline","commit","artifact-ref*"],
    normal_form: true
  },
  Sched = {
    PeriodType:"finite",
    bind: { T: 1.0, dt: 0.001, derive: "m=T/dt", final_partial:false },
    OrderConv:"left-to-right",
    RepeatCount: 1,
    StopRule: "none"
  },
  Trunc = {
```

```

Basis: Hash(basis0),
CutRule: "cut0",
BoundaryRule: "boundary0",
N: 1024,
CrossNPolicy: "none"
},
NumericModel = {
  arithmetic: "det-float-or-fixed",
  rounding: { mode:"nearest", precision:64 },
  matmul: { order:"row-major", accumulate:"sequential" },
  canon: { after_each_tick:true },
  tolerances: { eps_step: 1e-10 }
},
TranscriptSpec = {
  CommitType:"full-X",
  HashAlg:"sha256",
  SerializeRule:"EY22-CCE",
  CommitRate:"every-tick",
  ChainRule:"hash-chain"
},
Cert = {
  schema:"EY22-CCE",
  required_checks:["F1","F2","F3","F4","F5","F6","F7","F8"]
},
Interop = {
  allowed_forms:["C5","C4","C1"],
  compat:{
    EY23:"Compat_26_23",
    EY24:"Compat_26_24",
    EY25:"Compat_26_25",
    EY28:"Compat_26_28",
    EY29:"Compat_26_29"
  }
},
Version = "VID-2026-02"
)

```

Rule. Once FLOQ-0 is published, all placeholders MUST be replaced by fixed values (or hash commitments) and the Version MUST be treated as immutable for that meaning.

7.10 Chapter Summary

This chapter defined FLOQ-0, a conservative baseline profile for EY26 designed for deterministic replay and universal verifiability. It fixed baseline semantics (step-list), schedule (finite), admissible step catalog and parameter grids, canonicalization after every tick, full transcript commitments, and required certificate checks. It also provided a reference SpecID instance template.

Next. Chapter 8 fixes the publication boundary and non-claims for EY26 and states licensing intent consistent with Vol. No. IV.

8 Publication Boundary, Non-Claims, and Licensing Intent

8.1 Purpose

EY26 makes *programs* publishable. That also creates a predictable risk: readers treat an executed token as a claim about truth, security, or physical correctness. This chapter fixes:

- what **MUST** be public for an EY26 publication,
- what EY26 explicitly does *not* claim,
- and QFC-consistent licensing intent for implementations and token catalogs.

8.2 Publication Boundary (Normative)

Rule (what must be public). An EY26 publication **MUST** include, at minimum:

- the full $\text{SpecID}_{\text{EY26}}$ tuple,
- the full step list Steps in canonical encoding *or* a hash commitment plus deterministic reconstruction rule,
- the admissible family $\mathcal{A}_{\text{Steps}}$ definition (or catalog ID + committed definition),
- the full schedule object Sched including all bound constants and derivation rules,
- the full Trunc object Trunc if truncation is used,
- the full NumericModel (rounding, ordering, canonicalization points, tolerances),
- the full TranscriptSpec (commit type, serialization, hash algorithm, commit rate, chaining),
- transcript commitments Tx at all required commitment ticks,
- the certificate π_{FLOQ} listing required verifier checks (F1–F8 at minimum),
- and hash commitments for all external OpRefs (operators, catalogs, datasets, code snapshots) used by steps.

Rule (no hidden environment dependencies). A token **MUST NOT** depend on:

- OS time, network calls, non-committed files, device identity,
- nondeterministic RNG,
- unspecified GPU parallelism or unspecified reduction order.

If any such dependency exists, the artifact is invalid.

Rule (boundedness is part of publication). Bounds are not optional. The publication **MUST** include:

m_{max} , R_{max} , parameter grids and bounds, and any declared work/memory proxies.

Rule (transcript is part of the claim). If any claim relies on the execution of U_F , the transcript commitments **MUST** be published at the declared rate. Claims without transcript commitments **MUST** be treated as non-audited narratives.

8.3 Non-Claims (Explicit)

EY26 explicitly does *not* claim the following:

NC1: Physical Correctness

EY26 does not claim that the token describes physical time evolution or that its steps correspond to real-world processes. It claims only that the execution is well-defined, bounded, and replayable under SpecID.

NC2: Computational Optimality

EY26 does not claim that a token is efficient, minimal, optimal, or stable under perturbations. A token can be perfectly auditable and still be a poor program.

NC3: Security Beyond Declared Checks

EY26 does not claim cryptographic security, robustness to adversarial producers, or resistance to side-channels beyond what is enforced by:

- admissibility constraints $\mathcal{A}_{\text{Steps}}$,
- transcript commitments and replay determinism,
- explicit interop predicates (if claimed).

NC4: Equivalence of Interpretations

If Sem includes operator-product semantics, EY26 does not automatically claim equivalence to step-list semantics. Equivalence must be explicitly asserted and verified, otherwise operator-product is at most an annotation.

NC5: Completeness of Commitments

Summary-only transcript commitments do not imply full-state replayability. If a publication commits only to summaries, it cannot claim full trajectory reproducibility.

8.4 Responsible Use and Misuse Boundary

Recommended use (informational). Use EY26 to publish discrete-time programs where:

- the step list is explicit and bounded,
- parameters are quantized and committed,
- canonicalization points are explicit,
- transcript commitments allow third parties to replay exactly.

Misuse examples (non-normative).

- Publishing a token execution plot without publishing the token encoding and transcript commitments.

- Using “library defaults” for floating point rounding or matrix multiplication order without pinning NumericModel.
- Smuggling adaptivity (variable step counts, hidden early stopping) behind implementation freedom.
- Claiming security, privacy, or physical truth from auditability alone.

8.5 Licensing Intent (QFC-Consistent)

EY26 is part of the QFC operator suite. The intent is:

- keep the *specification* (definitions, SpecID schema, verifier checks) publicly readable and reusable,
- allow implementations and token catalogs to be licensed under project-chosen terms (open, dual, or commercial),
- preserve auditability as non-negotiable: SpecID + commitments + transcript + verifier rules remain required for any claim.

Rule (no audit stripping). No license arrangement may remove the requirement to publish SpecID, commitments, and transcript commitments for any claim that depends on EY26.

8.6 Chapter Summary

This chapter fixed the publication boundary for EY26 and stated explicit non-claims: no physical correctness, no computational optimality, no security guarantees beyond declared checks, no implicit equivalence of semantics, and no full-state reproducibility without full-state commitments. It also stated QFC-consistent licensing intent: keep the spec auditable and public while allowing implementation/catalog licensing provided auditability is preserved.

Next. With EY26 complete, Vol. No. IV proceeds to EY27 (Commutant Generator G_{comm}), which standardizes symmetry and commutation constraints as canonical artifacts that restrict operator families, tokens, and audit-native evolutions.

EY27: Commutant Generator G_{comm}

Canonical Operator Artifacts for QFC

Enter Yourname

Abstract

This whitepaper defines the Commutant Generator G_{comm} , a canonical artifact that encodes symmetry and compatibility constraints as a *generator of commutation requirements*. Given a target operator or workflow component, G_{comm} produces a testable predicate—and when needed, a constructive constraint operator—that forces membership in a declared commutant class.

The intent is to make “symmetry”, “invariants”, and “allowed transformations” auditable: instead of informal statements (“*these operators should commute*”), EY27 provides SpecID-bound commutation rules, numeric and truncation semantics, and certificates enabling deterministic verification. G_{comm} is designed to compose with canonical spectral filtering (EY23), validity penalties (EY24), audit-native evolution (EY25), program tokens (EY26), regime transport (EY28), and coarse-grain reduction (EY29).

1 Normative Commutation Modes and Commutator Measures

1.1 Purpose

To make “commutes” machine-checkable, EY27 fixes a minimal vocabulary:

- commutation *modes* (what equality means and where it is required),
- commutator *measures* (how violation is quantified),
- tolerance semantics under NumericModel,
- and canonical predicate identifiers that other artifacts can cite in Interop.

Rule. Every EY27 instance MUST declare a commutation mode Mode, a measure Meas, and a tolerance model. Undeclared commutation semantics are invalid.

1.2 Objects and Domains

Let A, B be operators (or derived objects that admit operator evaluation) acting on a declared space $\mathcal{H}$, with truncation policy Trunc (if applicable).

Commutator and anticommutator.

$$[A, B] := AB - BA, \quad \{A, B\} := AB + BA.$$

Rule (domain compatibility). All products appearing in the declared commutation relation MUST be well-defined under declared truncation and calculus. If not, the check MUST fail deterministically with **FAIL_DOMAIN_MISMATCH** or **FAIL_TRUNC_MISMATCH**.

1.3 Commutation Modes Mode

EY27 standardizes the following commutation modes.

M1: Exact Commutation (Finite Representation)

Meaning. In the declared finite representation (e.g. truncation level N):

$$[A, B] = 0 \quad \text{exactly in the declared arithmetic model.}$$

When used. Preferred when NumericModel is exact (fixed-point / rational) and truncation is explicit.

Predicate form.

$$\text{Comm}_{\text{M1}}(A, B) := ([A, B] = 0).$$

M2: Approximate Commutation (Norm-Toleranced)

Meaning.

$$\|[A, B]\|_{\text{Meas}} \leq \varepsilon,$$

where ε is fixed in SpecID (or quantized on a declared grid).

Rule (tolerance binding). ε MUST be part of SpecID. “Within numerical error” is forbidden unless ε is explicit.

Predicate form.

$$\text{Comm}_{\text{M2}}(A, B; \varepsilon, \text{Meas}) := (\|[A, B]\|_{\text{Meas}} \leq \varepsilon).$$

M3: Projected Commutation (Subspace/Projector)

Meaning. Commutation is required only on a declared subspace, represented by a projector P (or idempotent):

$$P[A, B]P = 0 \quad (\text{exact or toleranced}).$$

Variants.

- **M3a exact:** $P[A, B]P = 0$.
- **M3b approx:** $\|P[A, B]P\|_{\text{Meas}} \leq \varepsilon$.

Rule (projector binding). P MUST be canonically encoded or committed by hash and MUST satisfy idempotence (exact or within declared tolerance).

Predicate form.

$$\text{Comm}_{\text{M3}}(A, B; P, \varepsilon) := (\|P[A, B]P\|_{\text{Meas}} \leq \varepsilon),$$

with $\varepsilon = 0$ for exact.

M4: Graded Commutation (Commutator vs Anticommutator)

Meaning. Declare whether the relation is a commutator or anticommutator constraint:

$$[A, B]_{\sigma} := \begin{cases} AB - BA, & \sigma = - \\ AB + BA, & \sigma = + \end{cases}$$

and require $[A, B]_{\sigma} = 0$ (exact or approx).

Predicate form.

$$\text{Comm}_{\text{M4}}(A, B; \sigma, \varepsilon) := (\|[A, B]_{\sigma}\|_{\text{Meas}} \leq \varepsilon).$$

M5: Central-Phase Commutation (Projective)

Meaning. Some workflows require commutation up to a central phase (typical for unitary tokens):

$$AB \approx e^{i\phi} BA,$$

with ϕ on a declared grid.

Predicate form.

$$\text{Comm}_{\text{M5}}(A, B; \phi, \varepsilon) := (\|AB - e^{i\phi} BA\|_{\text{Meas}} \leq \varepsilon).$$

Rule (phase binding). ϕ MUST be quantized and bounded, and the grid MUST be declared in SpecID.

1.4 Commutator Measures Meas

A commutation mode needs a measure for “how much” it fails.

Rule. Every EY27 instance MUST declare one commutator measure identifier Meas and its computation semantics under NumericModel.

N1: Frobenius Norm

$$\|X\|_F := \sqrt{\text{Tr}(X^\dagger X)}.$$

Deterministic under canonical multiplication and trace ordering.

N2: Operator Norm Proxy

Exact operator norm is expensive; EY27 standardizes a *proxy*:

$$\|X\|_{\text{op_proxy}} := \max_{k \in \mathcal{K}} \|X v_k\|_2,$$

where $\mathcal{K}$ is a fixed, finite, canonically declared test-vector set. $\mathcal{K}$ MUST be committed by hash or generated deterministically from a public seed.

N3: Entrywise Max Norm

$$\|X\|_\infty := \max_{i,j} |X_{ij}|.$$

Simple and deterministic.

N4: Spectral Commutator Proxy (CSF-Aware)

For workflows aligned with EY23, a proxy can be defined using filtered spectral summaries:

$$\|[A, B]\|_{\text{spec_proxy}} := \|\text{Summary}(F_\star([A, B]))\|,$$

where Summary and $F_\star$ are explicitly referenced artifacts (and must be deterministic). This measure is allowed only if the EY23 reference is included in SpecID and interop predicates are provided.

1.5 Tolerance Semantics

Fixed tolerance

$\varepsilon \in \mathbb{R}_{\geq 0}$ is fixed in SpecID .

Grid tolerance

$$\varepsilon \in \{k \cdot \delta_\varepsilon : 0 \leq k \leq K_\varepsilon\}$$

with $\delta_\varepsilon, K_\varepsilon$ fixed.

Rule. Floating “machine epsilon” tolerances are forbidden unless explicitly bound as a numeric constant in SpecID .

1.6 Canonical Predicate Identifiers

To enable interoperability, EY27 standardizes predicate identifiers:

$$\text{PredID} := \text{COMM}-\langle \text{Mode} \rangle - \langle \text{Meas} \rangle - \langle \text{ParamHash} \rangle.$$

ParamHash commits to $(\varepsilon, P, \sigma, \phi, \mathcal{K})$ as applicable.

Rule. If an artifact claims to use an EY27 commutation constraint in its Interop component, it MUST reference the full PredID .

1.7 Deterministic Failure Codes (Mode/Measure Layer)

At minimum:

- **FAIL_DOMAIN_MISMATCH:** products not defined on declared space.
- **FAIL_TRUNC_MISMATCH:** truncation level(s) incompatible.
- **FAIL_NUMERICMODEL_MISMATCH:** rounding/order differs from declared.

- **FAIL_PROJECTOR_INVALID**: P fails idempotence/self-adjointness checks beyond tolerance.
- **FAIL_MEASURE_UNDEFINED**: measure requires missing committed test vectors or undefined summary.
- **FAIL_COMMUTATION**: predicate evaluates to FALSE.

1.8 Chapter Summary

This chapter defined the normative commutation vocabulary for EY27: commutation modes (exact, approximate, projected, graded, central-phase) and commutator measures (Frobenius, operator-norm proxy, max norm, spectral proxy). It fixed tolerance semantics, canonical predicate identifiers, and mode/measure-layer failure codes.

Chapter 3 next defines the EY27 artifact structure and SpecID binding for G_{comm} : its inputs, outputs (predicate + optional constraint object), admissible families, numeric/truncation requirements, and certificate payload.

2 Artifact Structure and SpecID Binding

2.1 Purpose

Chapters ??–1 fixed the meaning of commutation modes and measures. This chapter defines EY27 as a canonical *generator artifact*: it produces a commutation predicate (PredID), a deterministic verification routine, and (optionally) a constructive constraint object that can be used by other artifacts (EY24/EY25/EY26) to enforce the commutation condition.

2.2 Core Objects

EY27 binds commutation to declared objects under deterministic evaluation semantics.

Target objects. An EY27 instance declares:

$$(A, B) \in \mathcal{O} \times \mathcal{O}$$

(or a single target A commuted against a declared reference set $\mathcal{B}$; see below), where $\mathcal{O}$ is a declared operator class with canonical encoding rules.

Commutation contract. The contract is a predicate:

$$\text{CompatComm}(A, B) \in \{\text{TRUE}, \text{FALSE}\}$$

implemented as a deterministic routine.

2.3 Artifact Definition

Definition 2.1 (Commutant Generator). An EY27 artifact instance is a tuple:

$$G_{\text{comm}} := (\text{Mode}, \text{Meas}, \Theta, \mathcal{O}, \text{Trunc}, \text{NumericModel}, \text{RoutineSpec}),$$

that induces:

$$\text{PredID} := \text{COMM-}\langle \text{Mode} \rangle\text{-}\langle \text{Meas} \rangle\text{-}\langle \text{ParamHash} \rangle,$$

and a deterministic predicate evaluation routine

$$\text{Eval}_{G_{\text{comm}}}(A, B) \in \{\text{TRUE}, \text{FALSE}\}.$$

Optionally, G_{comm} may also output a constraint object K such that downstream artifacts can enforce commutation by constraining admissible families or by adding penalties.

Rule (generator semantics). An EY27 publication **MUST** specify:

- the predicate form used (from Chapter 1),
- the exact measure computation semantics,
- the tolerance and any additional parameters Θ ,
- and the deterministic routine $\text{Eval}_{G_{\text{comm}}}$.

2.4 Parameter Tuple Θ

Θ collects mode/measure parameters:

$$\Theta := (\varepsilon, P, \sigma, \phi, \mathcal{K}, \text{EqPolicy}),$$

where only the fields relevant to the chosen Mode/Meas are non-null.

EqPolicy. EqPolicy fixes:

- exact equality vs toleranced comparison,
- canonical rounding points before measuring,
- tie-break behavior for max operations,
- and serialization of intermediate objects if required for commitments.

Rule (anti-smuggling). All numeric parameters (including ε and ϕ) **MUST** be bounded and on declared grids if not exact constants. If a parameter is not bound, the artifact **MUST** be rejected.

2.5 Operator Class $\mathcal{O}$ and Encoding

EY27 **MUST** declare the operator class:

$$\mathcal{O} := (\text{Type}, \text{EncodingRule}, \text{AdjointSpec}, \text{CalcSpec}).$$

Examples (informational).

- bounded matrices at truncation level N ,
- sparse operator encodings with deterministic multiplication semantics,
- operator families generated by EY23/EY24/EY25/EY26 references.

Rule. If operators are external (files, catalogs), they **MUST** be committed by hash (EY22 Appendix A.1).

2.6 RoutineSpec: Deterministic Verification Routine

The routine is not “implementation detail”; it is part of SpecID.

RoutineSpec fields

$\text{RoutineSpec} := (\text{RoutineID}, \text{MulOrder}, \text{ReduceOrder}, \text{CanonPoints}, \text{MeasAlg}, \text{ProjectorChecks})$.

RoutineID. A short identifier referencing a fully described routine (inline or committed by hash). For baseline routines, EY27 defines a standard set.

MulOrder and ReduceOrder. Matrix multiplication order and reduction order MUST be fixed:

- row-major vs column-major accumulation,
- deterministic associativity policy,
- deterministic max tie-break rules for entrywise measures.

CanonPoints. Canonical rounding points MUST be declared:

- before computing AB and BA ,
- after computing the commutator object,
- before measuring.

MeasAlg. Measure algorithm must be fixed (e.g. Frobenius via trace order, operator proxy via fixed test vectors $\mathcal{K}$).

ProjectorChecks. If P is present (Mode M3), checks MUST be fixed:

$$\|P^2 - P\|_{\text{Meas}_P} \leq \varepsilon_P, \quad \|P - P^\dagger\|_{\text{Meas}_P} \leq \varepsilon_P,$$

with ε_P fixed in Θ (or exact).

2.7 SpecID for EY27

ArtifactType. For EY27:

$$\text{ArtifactType} = \text{COMM}.$$

SpecID components (EY27 binding). The full SpecID tuple is:

$\text{SpecID}_{\text{EY27}} := (\text{COMM}, \text{Mode}, \text{Meas}, \Theta, \mathcal{O}, \text{Trunc}, \text{NumericModel}, \text{RoutineSpec}, \text{TranscriptSpec}, \text{Cert}, \text{Inte})$

TranscriptSpec (required)

Even if the artifact only outputs a boolean, EY27 requires commitments that bind the check inputs and outputs:

- commitments to canonical encodings (or hashes) of A, B (or their references),

- commitment to the predicate parameters (Θ),
- commitment to the evaluation output and measured value(s).

Rule. If an EY27 result is cited inside another artifact (e.g. to restrict admissible families), the referenced PredID MUST match the committed parameters.

2.8 Optional Output: Constraint Object K

EY27 MAY output a constraint object K (committed by hash) for downstream enforcement. Two canonical forms are permitted:

K1: Penalty operator (VPO-ready). Provide a penalty V_{comm} such that:

$$\| [A, B] \|_{\text{Meas}} \leq \varepsilon \quad \Leftarrow \quad \langle X, V_{\text{comm}} X \rangle \text{ small,}$$

with explicit construction rules.

K2: Admissibility constraint (token-ready). Provide a constraint descriptor that restricts admissible steps/operators to those satisfying the predicate (checked by verifier).

Rule. If K is provided, it MUST be fully specified by deterministic construction rules and committed by hash.

2.9 Deterministic Failure Codes (Structure Layer)

At minimum:

- REJECT_MALFORMED
- REJECT_SPECID_MISSING
- REJECT_SPECID_MISMATCH
- FAIL_HASH_MISMATCH
- FAIL_DOMAIN_MISMATCH
- FAIL_TRUNC_MISMATCH
- FAIL_NUMERICMODEL_MISMATCH
- FAIL_PROJECTOR_INVALID
- FAIL_MEASURE_UNDEFINED
- FAIL_COMMUTATION

2.10 Chapter Summary

This chapter defined EY27 as a canonical generator artifact: it binds commutation mode and measure, parameters, operator class, truncation and numeric semantics, and a deterministic verification routine into SpecID. It introduced TranscriptSpec commitments and the optional constraint object K for downstream enforcement.

Chapter 4 next specifies admissible families and baseline routines: standard RoutineIDs, parameter grids and bounds, and FLOQ/LIOUV restrictions induced by G_{comm} in a machine-checkable way.

3 Artifact Structure and SpecID Binding

3.1 Purpose

Chapters ??–1 fixed the meaning of commutation modes and measures. This chapter defines EY27 as a canonical *generator artifact*: it produces a commutation predicate (PredID), a deterministic verification routine, and (optionally) a constructive constraint object that can be used by other artifacts (EY24/EY25/EY26) to enforce the commutation condition.

3.2 Core Objects

EY27 binds commutation to declared objects under deterministic evaluation semantics.

Target objects. An EY27 instance declares:

$$(A, B) \in \mathcal{O} \times \mathcal{O}$$

(or a single target A commuted against a declared reference set $\mathcal{B}$; see below), where $\mathcal{O}$ is a declared operator class with canonical encoding rules.

Commutation contract. The contract is a predicate:

$$\text{CompatComm}(A, B) \in \{\text{TRUE}, \text{FALSE}\}$$

implemented as a deterministic routine.

3.3 Artifact Definition

Definition 3.1 (Commutant Generator). An EY27 artifact instance is a tuple:

$$G_{\text{comm}} := (\text{Mode}, \text{Meas}, \Theta, \mathcal{O}, \text{Trunc}, \text{NumericModel}, \text{RoutineSpec}),$$

that induces:

$$\text{PredID} := \text{COMM-}\langle \text{Mode} \rangle\text{-}\langle \text{Meas} \rangle\text{-}\langle \text{ParamHash} \rangle,$$

and a deterministic predicate evaluation routine

$$\text{Eval}_{G_{\text{comm}}}(A, B) \in \{\text{TRUE}, \text{FALSE}\}.$$

Optionally, G_{comm} may also output a constraint object $\mathbf{K}$ such that downstream artifacts can enforce commutation by constraining admissible families or by adding penalties.

Rule (generator semantics). An EY27 publication MUST specify:

- the predicate form used (from Chapter 1),
- the exact measure computation semantics,
- the tolerance and any additional parameters Θ ,
- and the deterministic routine $\text{Eval}_{G_{\text{comm}}}$.

3.4 Parameter Tuple Θ

Θ collects mode/measure parameters:

$$\Theta := (\varepsilon, P, \sigma, \phi, \mathcal{K}, \text{EqPolicy}),$$

where only the fields relevant to the chosen Mode/Meas are non-null.

EqPolicy. EqPolicy fixes:

- exact equality vs toleranced comparison,
- canonical rounding points before measuring,
- tie-break behavior for max operations,
- and serialization of intermediate objects if required for commitments.

Rule (anti-smuggling). All numeric parameters (including ε and ϕ) MUST be bounded and on declared grids if not exact constants. If a parameter is not bound, the artifact MUST be rejected.

3.5 Operator Class $\mathcal{O}$ and Encoding

EY27 MUST declare the operator class:

$$\mathcal{O} := (\text{Type}, \text{EncodingRule}, \text{AdjointSpec}, \text{CalcSpec}).$$

Examples (informational).

- bounded matrices at truncation level N ,
- sparse operator encodings with deterministic multiplication semantics,
- operator families generated by EY23/EY24/EY25/EY26 references.

Rule. If operators are external (files, catalogs), they MUST be committed by hash (EY22 Appendix A.1).

3.6 RoutineSpec: Deterministic Verification Routine

The routine is not “implementation detail”; it is part of SpecID.

RoutineSpec fields

$\text{RoutineSpec} := (\text{RoutineID}, \text{MulOrder}, \text{ReduceOrder}, \text{CanonPoints}, \text{MeasAlg}, \text{ProjectorChecks}).$

RoutineID. A short identifier referencing a fully described routine (inline or committed by hash). For baseline routines, EY27 defines a standard set.

MulOrder and ReduceOrder. Matrix multiplication order and reduction order MUST be fixed:

- row-major vs column-major accumulation,
- deterministic associativity policy,
- deterministic max tie-break rules for entrywise measures.

CanonPoints. Canonical rounding points MUST be declared:

- before computing AB and BA ,
- after computing the commutator object,
- before measuring.

MeasAlg. Measure algorithm must be fixed (e.g. Frobenius via trace order, operator proxy via fixed test vectors $\mathcal{K}$).

ProjectorChecks. If P is present (Mode M3), checks MUST be fixed:

$$\|P^2 - P\|_{\text{Meas}_P} \leq \varepsilon_P, \quad \|P - P^\dagger\|_{\text{Meas}_P} \leq \varepsilon_P,$$

with ε_P fixed in Θ (or exact).

3.7 SpecID for EY27

ArtifactType. For EY27:

$$\text{ArtifactType} = \text{COMM}.$$

SpecID components (EY27 binding). The full SpecID tuple is:

$$\text{SpecID}_{\text{EY27}} := (\text{COMM}, \text{Mode}, \text{Meas}, \Theta, \mathcal{O}, \text{Trunc}, \text{NumericModel}, \text{RoutineSpec}, \text{TranscriptSpec}, \text{Cert}, \text{Inte})$$

TranscriptSpec (required)

Even if the artifact only outputs a boolean, EY27 requires commitments that bind the check inputs and outputs:

- commitments to canonical encodings (or hashes) of A, B (or their references),
- commitment to the predicate parameters (Θ),
- commitment to the evaluation output and measured value(s).

Rule. If an EY27 result is cited inside another artifact (e.g. to restrict admissible families), the referenced PredID MUST match the committed parameters.

3.8 Optional Output: Constraint Object K

EY27 MAY output a constraint object K (committed by hash) for downstream enforcement. Two canonical forms are permitted:

K1: Penalty operator (VPO-ready). Provide a penalty V_{comm} such that:

$$\| [A, B] \|_{\text{Meas}} \leq \varepsilon \quad \Leftarrow \quad \langle X, V_{\text{comm}} X \rangle \text{ small,}$$

with explicit construction rules.

K2: Admissibility constraint (token-ready). Provide a constraint descriptor that restricts admissible steps/operators to those satisfying the predicate (checked by verifier).

Rule. If K is provided, it MUST be fully specified by deterministic construction rules and committed by hash.

3.9 Deterministic Failure Codes (Structure Layer)

At minimum:

- **REJECT__MALFORMED**
- **REJECT__SPECID__MISSING**
- **REJECT__SPECID__MISMATCH**
- **FAIL__HASH__MISMATCH**
- **FAIL__DOMAIN__MISMATCH**
- **FAIL__TRUNC__MISMATCH**
- **FAIL__NUMERICMODEL__MISMATCH**
- **FAIL__PROJECTOR__INVALID**
- **FAIL__MEASURE__UNDEFINED**
- **FAIL__COMMUTATION**

3.10 Chapter Summary

This chapter defined EY27 as a canonical generator artifact: it binds commutation mode and measure, parameters, operator class, truncation and numeric semantics, and a deterministic verification routine into SpecID. It introduced TranscriptSpec commitments and the optional constraint object K for downstream enforcement.

Chapter 4 next specifies admissible families and baseline routines: standard RoutineIDs, parameter grids and bounds, and FLOQ/LIOUV restrictions induced by G_{comm} in a machine-checkable way.

4 Admissible Families and Baseline Routines (COMM-0)

4.1 Purpose

EY27 must be usable as a *constraint primitive* across Vol. No. IV. That requires:

- admissible families (anti-smuggling bounds) for parameters and references,
- baseline deterministic routines that every verifier can implement,

- and a standard “reference profile” that other artifacts can cite.

This chapter defines those pieces and introduces the baseline profile **COMM-0**.

4.2 Admissible Family $\mathcal{A}_{\text{COMM}}$

An EY27 instance is admissible only if its SpecID parameters lie in a declared admissible family.

Definition 4.1 (Admissible EY27 Instance). An EY27 instance is admissible if:

1. all parameters in Θ are bounded and canonically encoded,
2. all referenced objects (operators, projectors, test vectors) are inline or hash-committed,
3. Mode is one of the standardized modes $\{M1, \dots, M5\}$ or a declared extension mode,
4. Meas is one of $\{N1, \dots, N4\}$ or a declared extension measure,
5. RoutineSpec references a deterministic routine with fixed multiply/reduce/canonicalization behavior,
6. truncation levels are fixed and consistent (or bridged deterministically via CrossNPolicy),
7. and all checks are deterministic under NumericModel.

Rule (anti-smuggling). Any EY27 instance with free continuous parameters, adaptive test-vector sets, or unspecified evaluation order **MUST** be rejected.

4.3 Parameter Bounds and Grids

Tolerance grid

For toleranced modes (M2/M3b/M4/M5), ε **MUST** be:

$$\varepsilon \in \{k \cdot \delta_\varepsilon : 0 \leq k \leq K_\varepsilon\},$$

with $\delta_\varepsilon, K_\varepsilon$ fixed in SpecID, unless ε is a single fixed constant.

Phase grid (M5)

If ϕ is used:

$$\phi \in \left\{ \frac{2\pi q}{Q} : q \in \{0, \dots, Q-1\} \right\},$$

with Q fixed and bounded.

Projector tolerance grid (M3)

If projector checks are toleranced:

$$\varepsilon_P \in \{k \cdot \delta_P : 0 \leq k \leq K_P\}.$$

Rule. All grids **MUST** be declared. Defaulting to “machine epsilon” is forbidden.

4.4 Baseline Routine Set

EY27 defines baseline routines $\text{RoutineID} \in \mathcal{R}_0$ that are universally implementable.

Routine COMM-R0: Direct Commutator + Frobenius (N1)

Inputs. (A, B) at truncation level N .

Steps (deterministic).

1. Canonicalize A, B (NumericModel).
2. Compute AB and BA using declared MulOrder and ReduceOrder.
3. Compute $C := AB - BA$.
4. Canonicalize C .
5. Compute $m := \|C\|_F$ via trace ordering.
6. Return TRUE iff $m \leq \varepsilon$ (or $m = 0$ in M1).

Outputs. Measured value m , boolean output, and transcript commitments.

Routine COMM-R1: Entrywise Max (N3)

Same as COMM-R0, but measure is:

$$m := \|C\|_\infty = \max_{i,j} |C_{ij}|,$$

with deterministic tie-break rules:

- scan order (row-major) fixed,
- if equal maxima occur, select the first occurrence.

Routine COMM-R2: Operator Norm Proxy (N2)

Inputs. (A, B) , a committed test-vector set $\mathcal{K} = \{v_1, \dots, v_s\}$.

Steps.

1. Compute $C := AB - BA$ deterministically.
2. For each v_k in a fixed order, compute $w_k := Cv_k$.
3. Compute $m_k := \|w_k\|_2$ deterministically.
4. Set $m := \max_k m_k$ with deterministic tie-break.
5. Return TRUE iff $m \leq \varepsilon$.

Rule (test vectors). $\mathcal{K}$ MUST be committed by hash or generated from a public seed with a deterministic generator.

Routine COMM-R3: Projected Commutator (M3)

Steps. Compute:

$$C_P := P(AB - BA)P$$

then apply N1/N3/N2 measure as declared. Projector validity checks must be executed first:

$$\|P^2 - P\| \leq \varepsilon_P, \quad \|P - P^\dagger\| \leq \varepsilon_P.$$

If checks fail: **FAIL_PROJECTOR_INVALID**.

Routine COMM-R4: Central-Phase Search (M5)

Steps. For ϕ in the declared grid order:

$$C(\phi) := AB - e^{i\phi}BA, \quad m(\phi) := \|C(\phi)\|_{\text{Meas}}.$$

Choose ϕ^* by deterministic minimization with tie-break (smallest grid index), and accept if $m(\phi^*) \leq \varepsilon$. If the token fixes ϕ instead of searching, this routine is not used.

Rule. Search order and tie-break MUST be explicit. Otherwise: **FAIL_NONDETERMINISM**.

4.5 COMM-0 Reference Profile (Baseline)

COMM-0 is the conservative baseline profile for EY27, analogous to FLOQ-0 and LIOUV-0.

COMM-0 Fixes

COMM-0 fixes:

- allowed modes: M1 and M2 only,
- allowed measures: N1 or N3 only,
- routines: COMM-R0 (N1) or COMM-R1 (N3),
- truncation: single level N_0 (or an explicit finite list with CrossNPolicy=**none**),
- tolerance: fixed ε (grid-tolerance allowed but must be fixed),
- canonicalization points: before products, after commutator, before measuring.

Rule. COMM-0 forbids projectors (M3), graded commutation (M4), and phase commutation (M5) unless an explicit non-baseline profile is declared.

4.6 Using EY27 to Restrict Other Artifacts (Normative Patterns)

EY27 is most valuable when used to restrict admissible families elsewhere.

Pattern P1: Restricting EY26 Tokens

A token may require:

$$\forall j, \text{Comm}(U_j, B) = \text{TRUE}$$

for a declared B (symmetry generator). This becomes part of $\mathcal{A}_{\text{Steps}}$ for EY26, and the token verifier must evaluate the referenced PredID for each step operator.

Rule. If an EY26 token claims an EY27 restriction, it MUST:

- reference PredID,
- define the set of step objects that are checked,
- define whether checks are per-step or per-period,
- and ensure all checks are deterministic and committed.

Pattern P2: Restricting EY25 Generators

An EY25 generator may require:

$$[H, B] = 0 \quad \text{or} \quad \|[H, B]\| \leq \varepsilon,$$

as a membership condition for $\mathcal{A}_{\mathbb{G}}$.

Pattern P3: Certifying Interop Stability

Interop predicates may require commutation stability under reduction or transport:

$$\text{Comm}(A, B) = \text{TRUE} \Rightarrow \text{Comm}(R(A), R(B)) = \text{TRUE}$$

or an explicitly weakened version. Such implications MUST be declared and checked (not assumed).

4.7 Deterministic Failure Codes (Admissibility Layer)

At minimum:

- **FAIL_CONSTRAINT:** parameter bounds/grids violated.
- **FAIL_ROUTINE_UNKNOWN:** RoutineID not recognized and not provided by committed definition.
- **FAIL_NONDETERMINISM:** routine requires unspecified order/tie-break.
- **FAIL_PROJECTOR_INVALID:** projector checks fail (if used).
- **FAIL_COMMUTATION:** predicate evaluates to FALSE.

4.8 Chapter Summary

This chapter defined admissible EY27 instances $\mathcal{A}_{\text{COMM}}$, fixed bounded parameter grids, introduced baseline deterministic routines (COMM-R0..R4), and defined the conservative reference profile COMM-0. It also specified normative patterns for using EY27 to restrict EY26 tokens and EY25 generators and to support interop stability claims.

Chapter 5 next specifies certificate payload and verifier checks for EY27: how commutation results are committed, how routines are replayed, and how PredIDs are validated for downstream composition.

5 Certificate and Verifier: PredID, Replay, and Acceptance

5.1 Purpose

EY27 must be citeable as a *constraint artifact*. That requires more than “TRUE/FALSE”: the publication must bind the exact predicate meaning (PredID), the routine used to evaluate it, the measured values, and the deterministic replay path.

This chapter specifies:

- the certificate payload π_{COMM} for EY27,
- deterministic verifier checks for acceptance,
- transcript commitments for commutation evaluations,
- and required failure codes.

EY27 certificates follow EY22 canonical certificate encoding (CCE) conventions and include commutation-specific fields.

5.2 Certificate Object π_{COMM}

Rule. An EY27 publication MUST include a certificate π_{COMM} with the following canonical content.

Header (required)

- artifact type: COMM
- artifact symbol: G_{comm}
- PredID
- Version
- Certificate schema identifier (EY22-CCE)

SpecID (required)

The full SpecID tuple MUST be embedded:

$\text{SpecID}_{\text{EY27}} := (\text{COMM}, \text{Mode}, \text{Meas}, \Theta, \mathcal{O}, \text{Trunc}, \text{NumericModel}, \text{RoutineSpec}, \text{TranscriptSpec}, \text{Cert}, \text{Inte})$

Committed Inputs (required)

π_{COMM} MUST commit to:

- canonical encoding or hash-commitment of A and B (or of their references and deterministic reconstruction rules),
- the parameter tuple Θ (including $\varepsilon, P, \sigma, \phi, \mathcal{K}$, EqPolicy with nulls explicit),
- truncation configuration (levels, basis, cut rule, boundary rule),
- NumericModel (rounding points, multiplication/reduction order, tolerances),
- RoutineSpec (RoutineID and all routine-critical settings),

- any external objects: projector P , test vector set $\mathcal{K}$, EY23 references (if measure N4), committed by hash.

Committed Outputs (required)

π_{COMM} MUST commit to:

- measured commutator value(s) m (and, if applicable, ϕ^* for M5 search),
- boolean predicate outcome $b \in \{\text{TRUE}, \text{FALSE}\}$,
- transcript commitments Tx as required by TranscriptSpec,
- deterministic failure code if evaluation fails.

Optional Constraint Object K (if published)

If a constraint object K is provided (Chapter 3), the certificate MUST include:

- canonical encoding or hash-commitment of K,
- deterministic construction rules (inline or committed by hash),
- statement of how K is intended to be used (penalty vs admissibility restriction).

5.3 PredID Computation (Normative)

PredID is the canonical identifier for commutation semantics.

Definition.

$$\text{PredID} := \text{COMM} - \langle \text{Mode} \rangle - \langle \text{Meas} \rangle - \langle \text{ParamHash} \rangle,$$

where ParamHash is the hash of the canonical encoding of all commutation parameters:

$$\text{ParamHash} := \text{Hash}(\text{Enc}(\Theta) \parallel \text{Enc}(\mathcal{O}) \parallel \text{Enc}(\text{Trunc}) \parallel \text{Enc}(\text{NumericModel}) \parallel \text{Enc}(\text{RoutineSpec})).$$

Rule. A verifier MUST recompute PredID from SpecID and reject if it does not match the published PredID (**REJECT_PREDID_MISMATCH**).

5.4 Verifier Contract

A verifier consumes π_{COMM} and returns:

$$\{\text{ACCEPT}, \text{REJECT}\} \cup \{\text{FAIL_}*\}.$$

Rule. Verification MUST be deterministic under NumericModel and MUST not depend on producer identity or runtime secrets.

5.5 Required Verifier Checks (Normative)

C1: Structural Validity (EY22-CCE)

Parse canonical certificate encoding and confirm all required fields exist. Failure: **REJECT_MALFORMED**.

C2: SpecID Completeness and Consistency

- `ArtifactType = COMM`,
- `Mode`, `Meas`, Θ , $\mathcal{O}$, `NumericModel`, `RoutineSpec` are present,
- `Trunc` present if required by operator encodings,
- `TranscriptSpec` present and valid.

Failure: **REJECT_SPECID_MISSING**, **REJECT_SPECID_MISMATCH**.

C3: PredID Recompute and Match

Recompute PredID and compare. Failure: **REJECT_PREDID_MISMATCH**.

C4: No Hidden Inputs (Commitment Integrity)

- all external objects are hash-committed and match,
- all operator references resolve deterministically,
- test vectors $\mathcal{K}$ or EY23 references exist if required.

Failure: **FAIL_HASH_MISMATCH**, **REJECT_HIDDEN_INPUT**.

C5: Admissibility Check $\mathcal{A}_{\text{COMM}}$

Verify bounds and grids:

- ε on declared grid or fixed constant,
- ϕ on declared grid if used,
- projector tolerances on declared grids if used,
- `RoutineID` recognized or committed definition supplied,
- canonical encoding is in normal form (nulls explicit).

Failure: **FAIL_CONSTRAINT**, **FAIL_ROUTINE_UNKNOWN**.

C6: Domain/Truncation Validity

- check products AB , BA are defined under `CalcSpec`,
- check truncation levels and basis are consistent with `SpecID`,
- for multi-level truncation, apply `CrossNPolicy` deterministically.

Failure: **FAIL_DOMAIN_MISMATCH**, **FAIL_TRUNC_MISMATCH**.

C7: Routine Replay and Measurement

Execute the declared routine (`COMM-R0..` or committed routine) deterministically:

- apply canonicalization points,

- compute commutator object (or graded/projected/phase variant),
- compute measure value(s) m ,
- apply EqPolicy to decide b .

Failure:

- **FAIL_NONDETERMINISM** if required tie-breaks/order not fixed,
- **FAIL_PROJECTOR_INVALID** if P validation fails,
- **FAIL_MEASURE_UNDEFINED** if measure prerequisites missing,
- **FAIL_COMMUTATION** if predicate returns FALSE.

C8: Transcript Commitment Check

Recompute transcript commitments per TranscriptSpec and compare to published:

$$T_X^{(\text{replay})} \stackrel{?}{=} T_X^{(\text{published})}.$$

Failure: **FAIL_TRANSCRIPT_MISMATCH**.

C9: Optional Constraint Object Verification

If K is published:

- verify commitment hash for K ,
- verify deterministic construction rules (if constructive),
- verify K usage declaration is consistent (penalty vs admissibility).

Failure: **FAIL_HASH_MISMATCH**, **FAIL_CONSTRAINT**.

5.6 Acceptance Rule

Rule. A verifier returns ACCEPT iff:

- C1–C6 pass,
- the declared routine executes deterministically,
- transcript commitments match,
- and the published boolean output matches replay output.

If the predicate output is FALSE, the artifact may still be *well-formed* but it does not certify commutation; in that case the verifier returns **FAIL_COMMUTATION** (not ACCEPT).

5.7 Required Failure Codes

At minimum:

- **REJECT_MALFORMED**
- **REJECT_SPECID_MISSING**

- **REJECT__SPECID__MISMATCH**
- **REJECT__PREDID__MISMATCH**
- **REJECT__HIDDEN__INPUT**
- **FAIL__HASH__MISMATCH**
- **FAIL__CONSTRAINT**
- **FAIL__ROUTINE__UNKNOWN**
- **FAIL__DOMAIN__MISMATCH**
- **FAIL__TRUNC__MISMATCH**
- **FAIL__NUMERICMODEL__MISMATCH**
- **FAIL__PROJECTOR__INVALID**
- **FAIL__MEASURE__UNDEFINED**
- **FAIL__NONDETERMINISM**
- **FAIL__COMMUTATION**
- **FAIL__TRANSCRIPT__MISMATCH**

5.8 Chapter Summary

This chapter specified the EY27 certificate π_{COMM} and deterministic verifier checks: structural validity, SpecID completeness, PredID recomputation, commitment integrity, admissibility constraints, domain/truncation validity, routine replay and measurement, and transcript matching. It also defined acceptance rules and required failure codes, making EY27 citeable as a strict commutation-constraint artifact.

Chapter 6 next provides interoperability declarations (mini) for EY27: how PredIDs are referenced in EY22 interop components, how commutation constraints restrict EY26 tokens and EY25 generators, and how they interact with EY23 filters, EY24 validity penalties, EY28 transport, and EY29 coarse-grain reduction.

6 Interop Declarations (Mini): How COMM Constraints Compose

6.1 Purpose

EY27 is intended to be a constraint layer that other artifacts can cite. This chapter instantiates the EY22 interoperability framework for commutation constraints:

- how EY27 PredIDs are referenced inside Interop components,
- how commutation constraints restrict EY26 admissible step families and EY25 admissible generator families,
- and what stability requirements apply under EY23 filtering, EY24 validity, EY28 transport, and EY29 reduction.

Rule. Interop is a contract. If an artifact claims it is “symmetry-compatible” or “commutant-preserving”, it MUST reference a concrete EY27 PredID and MUST provide a checkable compatibility predicate under the declared NumericModel.

6.2 Interop Component in SpecID (EY27 Requirement)

Every artifact in EY23–EY30 that claims a commutation constraint MUST include an interop entry:

$$\text{Interop} \ni (\text{COMM-Ref} = \text{PredID}, \text{CommUse}, \text{CommCheckScope}).$$

Required fields.

- **COMM-Ref:** full PredID and commitment hash of π_{COMM} (or the SpecID hash).
- **CommUse:** how the constraint is used: **admissibility**, **penalty**, **certificate**, or **annotation**.
- **CommCheckScope:** where checks are applied: **per-step**, **per-period**, **final-only**, or an explicit tick set.

Rule. If $\text{CommUse} = \text{admissibility}$ or **certificate**, then **CommCheckScope** MUST be explicit and MUST be enforced by verifiers.

6.3 Compatibility Predicates (EY27 Perspective)

EY27 defines minimal predicate IDs for interoperability. All are deterministic under NumericModel.

COMM $\rightarrow$ FLOQ: Restricting Program Tokens

A common claim is: “this token respects symmetry B ”. Formally, a token U_F is commutant-compatible if required step operators commute (or approximately commute) with a declared reference B under a declared PredID.

Predicate $\text{Compat}_{27,26}^{(1)}$ (**binding**).

$\text{Compat}_{27,26}^{(1)} := (\text{EIF the token's Interop includes } \text{COMM-Ref} = \text{PredID} \text{ and matches the committed } \pi_{\text{COMM}})$

Predicate $\text{Compat}_{27,26}^{(2)}$ (**scope**).

$\text{Compat}_{27,26}^{(2)} := (\text{EIF } \text{CommCheckScope} \text{ is explicit and refers to a deterministic set of checked step objects}).$

Predicate $\text{Compat}_{27,26}^{(3)}$ (**replay check**).

$\text{Compat}_{27,26}^{(3)} := (\text{EIF the EY26 verifier evaluates the COMM predicate at the declared scope under aligned N})$

Compatibility status.

$$\text{Compat}(\text{EY27}, \text{EY26}) := \text{Compat}_{27,26}^{(1)} \wedge \text{Compat}_{27,26}^{(2)} \wedge \text{Compat}_{27,26}^{(3)}.$$

If TRUE, mark OK*.

Failure codes. **FAIL_COMPAT_PREDICATE**, **FAIL_NUMERICMODEL_MISMATCH**, **FAIL_TRUNC_MISMATCH**, **FAIL_COMMUTATION**.

COMM → LIOUV: Restricting Audit-Native Generators

EY25 generators often claim invariant preservation or symmetry. EY27 provides a canonical way to bind that claim.

Predicate $\text{Compat}_{27,25}^{(1)}$ (**generator restriction**).

$\text{Compat}_{27,25}^{(1)} :=$ (EIF EY25's admissible generator family $\mathcal{A}_{\mathbb{G}}$ references PredID and declares what commutes).

Predicate $\text{Compat}_{27,25}^{(2)}$ (**object of commutation**).

$\text{Compat}_{27,25}^{(2)} :=$ (EIF EY25 declares whether COMM applies to H , to $\mathcal{L}$, to noise terms, or to induced step ma

Predicate $\text{Compat}_{27,25}^{(3)}$ (**stability under evolution**). For invariance claims, the workflow must declare what is preserved:

$$[A, B] = 0 \Rightarrow [\mathcal{L}(A), B] = 0 \quad (\text{or a toleranced variant}),$$

and provide a deterministic check routine at declared checkpoints.

Compatibility status.

$$\text{Compat}(\text{EY27}, \text{EY25}) := \text{Compat}_{27,25}^{(1)} \wedge \text{Compat}_{27,25}^{(2)} \wedge \text{Compat}_{27,25}^{(3)}.$$

If TRUE, mark OK*.

COMM ↔ CSF: Filtered Symmetry and Commutation Probing

EY23 may operate on operators or derived commutators.

Predicate $\text{Compat}_{27,23}^{(1)}$ (**measure dependence**). If EY27 uses the spectral proxy measure (N4), then:

$\text{Compat}_{27,23}^{(1)} :=$ (EIF EY27 SpecID includes an EY23 reference and NumericModel alignment is declared).

Otherwise, compatibility is optional.

Predicate $\text{Compat}_{27,23}^{(2)}$ (**filter commutation stability, optional**). If a workflow claims that filtering preserves commutation:

$$\text{Comm}(A, B) \Rightarrow \text{Comm}(F_{\star}(A), F_{\star}(B)),$$

it MUST include a deterministic check routine (usually at truncation N and checkpoint set). Failure: **FAIL_INVARIANT_NOT_PRESERVED**.

COMM $\leftrightarrow$ VPO: Commutation-Enforced Validity

EY24 validity penalties can encode symmetry constraints.

Predicate $\text{Compat}_{27,24}^{(1)}$ (**penalty binding**). If EY24 claims to penalize commutation violation, it MUST:

- reference PredID,
- declare the penalty construction (e.g. $V_{\text{comm}} = C^\dagger C$ with C encoding commutator residual),
- and commit to construction rules.

Predicate $\text{Compat}_{27,24}^{(2)}$ (**equivalence non-claim**). The existence of a penalty does not imply exact commutation. If a claim needs commutation, it must still cite a COMM predicate check, or explicitly define what “valid” means in terms of commutation residual.

COMM $\leftrightarrow$ INTW: Transport of Symmetry Constraints

Transport can preserve commutation only under explicit semantics.

Predicate $\text{Compat}_{27,28}^{(1)}$ (**transported constraint**). If $T_{1 \rightarrow 2}$ transports $A_1 \mapsto A_2$, $B_1 \mapsto B_2$, then a workflow may claim:

$$\text{Comm}(A_1, B_1) \Rightarrow \text{Comm}(A_2, B_2).$$

This is permitted only if the workflow declares:

- whether transport is unitary/isometric/generalized,
- how adjoints/inverses are computed deterministically,
- and a deterministic check routine at declared truncation.

Failure. `FAIL_CALC_MISMATCH`, `FAIL_DOMAIN_MISMATCH`, `FAIL_INVARIANT_NOT`

COMM $\leftrightarrow$ CGR: Reduction and Symmetry

Reduction often weakens commutation. Therefore, any claim of preservation MUST be explicit.

Predicate $\text{Compat}_{27,29}^{(1)}$ (**reduced commutation**). A workflow may declare either:

- **strong:** $\text{Comm}(A, B) \Rightarrow \text{Comm}(R(A), R(B))$,
- **weak:** $\|[R(A), R(B)]\| \leq g(\|[A, B]\|)$ for a declared deterministic bound function g .

In either case, it MUST provide a deterministic check routine and tolerances.

6.4 Mini Compatibility Matrix Row (EY27 Perspective)

Informative (must be backed by predicates above):

	EY23	EY24	EY25	EY26	EY27	EY28	EY29	EY30
EY27	OK*	OK*	OK*	OK*	N/A	OK*	OK*	OK*

6.5 Interop Failure Codes (Required)

Interop verification MUST use deterministic failure codes, at minimum:

- **FAIL_COMPAT_PREDICATE**
- **FAIL_COMMUTATION**
- **FAIL_NUMERICMODEL_MISMATCH**
- **FAIL_TRUNC_MISMATCH**
- **FAIL_DOMAIN_MISMATCH**
- **FAIL_CALC_MISMATCH**
- **FAIL_INVARIANT_NOT_PRESERVED**
- **FAIL_TRANSCRIPT_MISMATCH**

6.6 Chapter Summary

This chapter defined minimal interoperability declarations for EY27. It standardized how PredIDs are referenced in other artifacts' Interop components and provided compatibility predicates for restricting EY26 tokens and EY25 generators, and for coordinating with EY23 filtering, EY24 validity penalties, EY28 transport, and EY29 reduction. It also provided an EY27 mini compatibility row and fixed required failure codes for interop failures.

Chapter 7 next defines the COMM-0 reference instance (fully spelled-out SpecID template) and states the publication boundary and non-claims for EY27 in a QFC-consistent way.

7 COMM-0 Reference Instance, Publication Boundary, and Non-Claims

7.1 Purpose

This chapter closes EY27 by:

- providing a concrete COMM-0 reference SpecID template (a stable meaning of “commutation constraint artifact”),
- fixing what must be published for a valid EY27 artifact,
- stating explicit non-claims (what EY27 does *not* guarantee),
- and recording licensing intent consistent with Vol. No. IV.

7.2 COMM-0 Reference Instance (Template)

COMM-0 is the conservative baseline profile defined in Chapter 4. The following template fixes a “default” SpecID for universal verifiers.

COMM-0 profile choices.

- $\text{Mode} \in \{\text{M1}, \text{M2}\},$

- $\text{Meas} \in \{\text{N1}, \text{N3}\}$,
- $\text{RoutineID} \in \{\text{COMM-R0}, \text{COMM-R1}\}$,
- single truncation level N_0 (or an explicit finite list with `CrossNPolicy=none`),
- explicit constant ε (or an ε -grid with fixed $\delta_\varepsilon, K_\varepsilon$).

Reference SpecID (COMM-0)

```

SpecID_EY27_COMM0 :=
(
  ArtifactType = COMM,

  Mode = "M2-approx"          # or "M1-exact" if eps=0 exact under numeric model
  Meas = "N1-frobenius",      # or "N3-max"

  Theta = {
    eps: 1e-10,               # fixed (or grid: {delta:..., K:...})
    P: null,                  # COMM-0 forbids projectors
    sigma: "-",               # default commutator (COMM-0 forbids graded variants)
    phi: null,                # COMM-0 forbids phase commutation
    K: null,                  # COMM-0 forbids operator-norm proxy vectors
    EqPolicy: {
      compare: "<= eps",
      canon_before_products: true,
      canon_after_commutator: true,
      canon_before_measure: true,
      tie_break: "first-in-scan"
    }
  },

  OperatorClass 0 = {
    Type: "finite-matrix",
    EncodingRule: "canonical",
    AdjointSpec: "dagger",
    CalcSpec: "matmul"
  },

  Trunc = {
    Basis: Hash(basis0),      # or inline if small
    CutRule: "cut0",
    BoundaryRule: "boundary0",
    N: 1024,
    CrossNPolicy: "none"
  },

  NumericModel = {
    arithmetic: "det-float-or-fixed",
    rounding: { mode:"nearest", precision:64 },
    matmul: { order:"row-major", accumulate:"sequential" },
    canon: { rule:"normalize-encoding" },
    tolerances: { eps_step: 0.0 } # COMM-0 recommends exact canon after each stage

```

```
},
```

```
RoutineSpec = {  
  RoutineID: "COMM-R0",      # (N1) or "COMM-R1" (N3)  
  MulOrder: "row-major",  
  ReduceOrder: "sequential",  
  CanonPoints: ["before-products", "after-commutator", "before-measure"],  
  MeasAlg: "trace-ordered", # for N1; or "entrywise-scan" for N3  
  ProjectorChecks: null  
},
```

```
TranscriptSpec = {  
  CommitType: "summary",    # summary is allowed: commits to A-hash, B-hash, m, b  
  HashAlg: "sha256",  
  SerializeRule: "EY22-CCE",  
  CommitRate: "single",    # one commitment per evaluation  
  ChainRule: "none"  
},
```

```
Cert = {  
  schema: "EY22-CCE",  
  required_checks: ["C1", "C2", "C3", "C4", "C5", "C6", "C7", "C8"]  
},
```

```
Interop = {  
  allowed_forms: ["C1", "C4"], # algebraic composition and superoperator embedding (information)  
  references: {}  
},
```

```
Version = "VID-2026-02"  
)
```

Rule. If a COMM-0 instance forbids a feature (projectors, phase, graded), the corresponding Θ fields **MUST** be explicitly null. Omitting them is not allowed.

7.3 Publication Boundary (Normative)

Rule (what must be public). An EY27 publication **MUST** include:

- full $\text{SpecID}_{\text{EY27}}$,
- PredID and the rule used to compute it (ParamHash construction),
- canonical encodings or hash commitments for A and B (or references + deterministic reconstruction),
- full Θ including all nulls explicitly,
- truncation and NumericModel (including multiply/reduce order and canonicalization points),
- RoutineSpec (RoutineID and all routine-critical settings),
- measured value(s) m and boolean outcome b ,

- transcript commitment(s) Tx (at least one),
- and certificate π_{COMM} with required checks (Chapter 5).

Rule (no hidden commutation objects). If B is “a symmetry generator”, it must be published or committed like any other operator. “Known symmetry” is not an input.

7.4 Non-Claims (Explicit)

EY27 explicitly does *not* claim:

NC1: Analytic Commutation in the Infinite Limit

Passing a commutation check at truncation N does not imply exact commutation in the full infinite-dimensional system. Any such limit statement must be made separately and is outside EY27.

NC2: Uniqueness of the Symmetry

If $[A, B] = 0$, EY27 does not claim that B is the only symmetry, or that the commutant is fully characterized.

NC3: Physical Meaning

Commutation constraints may encode physical symmetries, but EY27 does not assert physical correctness. It asserts only that the declared predicate holds under declared numeric/truncation semantics.

NC4: Robustness to Adversarial Encoding

EY27 reduces smuggling by bounds and commitments, but does not guarantee adversarial robustness beyond the declared verifier checks.

NC5: Preservation Under Transformations Unless Declared

Reduction (EY29), transport (EY28), filtering (EY23), and audit evolution (EY25) do not automatically preserve commutation. Preservation must be explicitly declared and checked.

7.5 Licensing Intent (Vol. No. IV)

The specification of EY27 (SpecID, PredID rules, baseline routines, verifier checks) is intended to remain publicly readable. Implementations and operator catalogs may be licensed under project-chosen terms, but:

auditability may not be removed.

Rule (no audit stripping). No distribution or license arrangement may claim EY27 compliance while omitting SpecID binding, commitments, and verifier-replay rules.

7.6 Chapter Summary

This chapter provided a COMM-0 reference SpecID template, fixed the publication boundary (what must be public and committed), stated explicit non-claims (no infinite-limit guarantees, no uniqueness, no physical meaning, no implicit robustness, no automatic preservation), and recorded licensing intent consistent with Vol. No. IV: the spec remains auditable and public; implementations may vary, but auditability is mandatory.

Next. With EY27 complete, Vol. No. IV proceeds to EY28: the Regime Intertwiner $T_{1 \rightarrow 2}$, which standardizes how operators and constraints are transported across regimes while preserving declared invariants.

8 Worked Examples (Non-Normative): How COMM Constraints Are Used

8.1 Purpose

Chapters ??–7 defined EY27 normatively. This chapter is *non-normative*: it illustrates typical ways commutation constraints appear in Vol. No. IV workflows, and how to publish them so that third parties can replay checks deterministically.

Note. Examples are schematic. They are valid only if all referenced objects are canonically encoded or hash-committed and NumericModel/Truncation are fixed.

8.2 Example 1: Symmetry Constraint on a Hamiltonian (EY25-Ready)

Goal. Publish the claim that a Hamiltonian H respects a symmetry generator B :

$$\|[H, B]\|_F \leq \varepsilon.$$

EY27 instance. Choose:

$$\text{Mode} = \text{M2}, \quad \text{Meas} = \text{N1}, \quad \varepsilon = 10^{-10}, \quad \text{RoutineID} = \text{COMM-R0}.$$

Commit to H and B encodings (or their hashes), fix truncation N , and publish π_{COMM} .

How EY25 uses it. EY25 includes in $\mathcal{A}_{\text{G}}$ an admissibility clause:

$$\text{PredID}_{HB} \text{ must return TRUE.}$$

If an EY25 generator family changes H , it must re-commit and re-check at declared checkpoints.

8.3 Example 2: Token Restriction (EY26 Per-Step Symmetry)

Goal. Publish a token $U_F = \prod_{j=1}^m U_j$ where each step respects the same symmetry B :

$$\forall j, \|[U_j, B]\|_{\infty} \leq \varepsilon.$$

EY27 instance. Choose:

$$\text{Mode} = \text{M2}, \quad \text{Meas} = \text{N3}, \quad \text{RoutineID} = \text{COMM-R1}.$$

Bind B by hash in the PredID parameters.

How EY26 encodes it. In EY26 Interop:

- $\text{COMM-Ref} = \text{PredID}$,
- $\text{CommUse} = \text{admissibility}$,
- $\text{CommCheckScope} = \text{per-step}$.

The EY26 verifier evaluates the COMM predicate for each step operator U_j against B , using the same truncation and NumericModel.

8.4 Example 3: Projected Symmetry (Subspace Invariance)

Goal. Require commutation only on a declared subspace (e.g. a “valid” or “physical” subspace):

$$\|P[H, B]P\|_F \leq \varepsilon.$$

EY27 instance (non-baseline). Choose:

$$\text{Mode} = \text{M3b}, \quad \text{Meas} = \text{N1}, \quad \text{RoutineID} = \text{COMM-R3}.$$

Publish P (or commit by hash), and publish projector validity checks:

$$\|P^2 - P\|_F \leq \varepsilon_P, \quad \|P - P^\dagger\|_F \leq \varepsilon_P.$$

How EY24 can embed it. EY24 can define a penalty operator that punishes leaving the projected commutant:

$$V_{\text{comm}} := C^\dagger C, \quad C := P(HB - BH)P,$$

with deterministic construction and truncation binding. (Still: penalties do not imply exact commutation outside P .)

8.5 Example 4: Central-Phase Commutation for Unitary Steps

Goal. Allow commutation up to a central phase:

$$\|UB - e^{i\phi}BU\|_{\text{Meas}} \leq \varepsilon.$$

EY27 instance (non-baseline). Choose:

$$\text{Mode} = \text{M5}, \quad \phi \in \left\{ \frac{2\pi q}{Q} \right\}, \quad \text{RoutineID} = \text{COMM-R4}.$$

Two patterns are valid:

- **fixed** ϕ : ϕ is part of SpecID and checked directly,
- **searched** ϕ^* : routine searches grid in fixed order and commits ϕ^* .

8.6 Example 5: “Commutation Stability Under Reduction” (EY29)

Goal. Claim that reduction R preserves symmetry:

$$\|[A, B]\| \leq \varepsilon \Rightarrow \|[R(A), R(B)]\| \leq \tilde{\varepsilon}.$$

Declaration (required). The workflow must declare:

- the exact reduction placement (before/after canonicalization),
- whether $R(B)$ is computed by the same R or by a specialized map,
- the relation between tolerances ($\tilde{\varepsilon}$ fixed or via a declared bound function g).

Then it must provide a deterministic routine that checks both predicates at the declared truncation and NumericModel.

8.7 Example 6: “Commutation Stability Under Transport” (EY28)

Goal. Claim that an intertwiner transports a symmetry:

$$[A_1, B_1] = 0 \Rightarrow [A_2, B_2] = 0, \quad A_2 = T A_1 T^{-1}, \quad B_2 = T B_1 T^{-1}.$$

Declaration (required). The workflow must declare:

- whether T^{-1} or $T^\dagger$ is used (unitary vs generalized),
- how T and inverses/adjoints are computed deterministically,
- truncation alignment across regimes,
- and the exact EY27 predicate used before and after transport.

8.8 Chapter Summary

This non-normative chapter illustrated how EY27 commutation constraints are used in practice: symmetry constraints on Hamiltonians (EY25), per-step token restrictions (EY26), projected commutation (M3), central-phase commutation (M5), and stability claims under reduction (EY29) and transport (EY28). In each case, the key principle remains: the commutation meaning must be bound to SpecID and replayable by independent verifiers.

Next. Proceed to EY28: Regime Intertwiner $T_{1 \rightarrow 2}$ (transport across regimes), which will inherit and reuse EY27 constraints as transported predicates with explicit preservation checks.

EY28: Regime Intertwiner $T_{1 \rightarrow 2}$

Canonical Operator Artifacts for QFC

Enter Yourname

Abstract

This whitepaper defines the Regime Intertwiner $T_{1 \rightarrow 2}$, a canonical artifact for transporting operators, states, and constraints between two spectral regimes (or layers) while preserving explicitly declared invariants. Unlike informal “change of basis” or “mapping” narratives, EY28 binds transport semantics to SpecID: the mapping type (unitary, isometric, partial, or generalized), the required adjoint/inverse behavior, truncation alignment across regimes, and deterministic verification routines.

$T_{1 \rightarrow 2}$ is intended as the bridge artifact for Vol. No. IV workflows: it allows canonical spectral filters (EY23), validity penalties (EY24), audit-native Liouvillians (EY25), program tokens (EY26), and commutation constraints (EY27) to be moved across layers without ambiguity, producing interop certificates that verifiers can replay deterministically.

1 Motivation, Scope, and Intuition: Transport Without Smuggling

1.1 Motivation: “Map It to Another Regime” Is Usually Undefined

Operator workflows often rely on statements like:

“move this operator to another basis/regime” or “transport the constraint to the next layer”.

But without a formal transport artifact, such statements hide:

- what the two regimes (1) and (2) actually are (spaces, truncations, encodings),
- whether the map is unitary, isometric, partial, or merely heuristic,
- what replaces the inverse if the map is not invertible,
- how adjoints are treated and what “preservation” means numerically,
- how truncation levels align (or fail) across regimes,
- and how a third party can verify that transport did not change the claim.

Vol. No. IV exists to remove hidden semantics. Therefore, “transport” must become an auditable artifact.

EY28 defines $T_{1 \rightarrow 2}$: a canonical regime intertwiner that binds transport semantics to SpecID and enables deterministic verification.

1.2 What EY28 Defines

EY28 defines:

- a **transport artifact** $T_{1 \rightarrow 2}$ between two declared regimes,
- a normative vocabulary of transport semantics (unitary, isometric, partial, generalized),
- canonical pushforward/pullback forms and when each is permitted,
- truncation alignment policies (same- N , mapped- N , multi- N with CrossNPolicy),
- preservation predicates for invariants (spectral class, trace, positivity, commutation),
- certificate and verifier routines for deterministic replay,
- interoperability mini-declarations for composing transported artifacts in Vol. No. IV.

1.3 Scope

In-scope.

- Transport of operators A_1 on regime 1 into operators A_2 on regime 2.
- Transport of states/objects X_1 into X_2 .
- Transport of *constraints* (e.g. EY27 commutation predicates) into transported constraints with explicit check routines.
- Deterministic verification under declared NumericModel and truncation policies.

Out-of-scope.

- “Learned” mappings without committed parameters and deterministic replay path.
- Heuristic embeddings that are not SpecID-bound and cannot be verified by third parties.
- Statements about infinite-dimensional equivalence without explicit truncation semantics.

1.4 Intuition: Regimes as Typed Spaces

A **regime** is not just a label; it is a typed execution/representation environment. EY28 treats each regime as:

$$\mathcal{R}_k := (\mathcal{H}_k, \text{Enc}_k, \text{Trunc}_k, \text{NumericModel}_k, \text{CalcSpec}_k).$$

Regime examples (informational).

- $\mathcal{H}_1$ = full-resolution truncation, $\mathcal{H}_2$ = coarse-grained truncation.
- $\mathcal{H}_1$ = operator-on-states space, $\mathcal{H}_2$ = observable-space for Koopman export.
- $\mathcal{H}_1$ = “physical” subspace, $\mathcal{H}_2$ = “audit” space augmented with certificates.

Transport must specify how encodings and truncations relate. Otherwise, a map can silently change the meaning of the computation.

1.5 Canonical Transport Forms

EY28 standardizes two canonical transport directions, mirroring EY22 C2.

Pushforward (operator transport)

Given an operator A_1 in regime 1, define the transported operator A_2 by:

$$A_2 := T_{1 \rightarrow 2} A_1 S_{2 \rightarrow 1},$$

where $S_{2 \rightarrow 1}$ is the declared *back-map* (inverse, adjoint, pseudo-inverse, or left/right inverse as specified).

Special cases.

- **Unitary:** $S_{2 \rightarrow 1} = T_{1 \rightarrow 2}^{-1} = T_{1 \rightarrow 2}^\dagger$.
- **Isometric:** $S_{2 \rightarrow 1} = T_{1 \rightarrow 2}^\dagger$ (left inverse on the image).
- **Generalized:** $S_{2 \rightarrow 1} = T_{1 \rightarrow 2}^+$ (a declared pseudo-inverse routine).

Pullback (observable/constraint transport)

For an observable g_2 or constraint defined in regime 2, pull it back to regime 1 by:

$$g_1 := S_{2 \rightarrow 1} g_2 T_{1 \rightarrow 2},$$

with the same back-map semantics.

Rule. Every EY28 instance MUST declare both the forward map $T_{1 \rightarrow 2}$ and the back-map semantics $S_{2 \rightarrow 1}$. If $S_{2 \rightarrow 1}$ is not unique (e.g. pseudo-inverse), the selection routine MUST be deterministic.

1.6 Why a Dedicated Artifact (Not “Just a Basis Change”)

A basis change is only one transport subtype. EY28 is broader:

- regime changes can include truncation change, coarse-graining, projection, augmentation, or encoding change,
- transport may be partial: only a subspace or subset of operators is meaningful in the target regime,
- constraints must be transported as *checkable predicates*, not as informal claims.

Therefore, $T_{1 \rightarrow 2}$ is treated as a first-class auditable object with its own SpecID, certificate, and verifier checks.

1.7 Design Goals (Normative)

G1: Deterministic Transport

Transport MUST be replayable: given committed inputs and NumericModel, every verifier computes the same transported object.

G2: No Smuggling Across Regimes

Transport must not be used to hide changes. Any change of truncation, tolerance, or calculus semantics must be explicit in SpecID.

G3: Preservation Is Declared, Not Assumed

Invariants are preserved only if a preservation predicate is declared and checked. Otherwise, no preservation claim is implied.

G4: Composability Across Vol. No. IV

EY28 must compose with:

- EY23 filtering (transporting H and filter inputs),
- EY24 penalties (transporting V and constraint operators),
- EY25 Liouvillian evolution (transporting generators or trajectories),
- EY26 tokens (transporting step operators or executed trajectories),
- EY27 commutation predicates (transporting constraints with explicit checks),
- EY29 coarse-graining (explicitly declaring when transport is itself a reduction).

1.8 Chapter Summary

This chapter motivated EY28: transport is usually informal and hides semantics. EY28 defines the regime intertwiner $T_{1 \rightarrow 2}$, a SpecID-bound artifact that makes mapping between regimes auditable and deterministic. It introduced regimes as typed spaces, canonical pushforward/pullback forms, the requirement to declare back-map semantics, and the normative design goals: deterministic transport, anti-smuggling, declared preservation, and composability.

Chapter 2 next defines the normative transport semantics classes (unitary, isometric, partial, generalized), the allowed back-map types, and the exact preservation predicate forms used throughout EY28.

2 Transport Semantics Classes and Preservation Predicates

2.1 Purpose

EY28 must remove ambiguity from the word “transport”. This chapter defines:

- normative transport semantics classes (unitary, isometric, partial, generalized),
- the allowed back-map types $S_{2 \rightarrow 1}$,
- preservation predicate vocabulary (what can be claimed preserved and how it is checked),
- and deterministic failure codes for semantic mismatch.

Rule. Every EY28 instance **MUST** declare a semantics class `SemClass` and a back-map type `BackMapType`. If either is missing, the artifact is invalid.

2.2 Semantics Class SemClass

Let $T := T_{1 \rightarrow 2}$ be the forward map and $S := S_{2 \rightarrow 1}$ the declared back-map. Let $\mathcal{H}_1, \mathcal{H}_2$ be the regime spaces.

S1: Unitary Transport

Definition. SemClass = UNIT if:

$$T^\dagger T = I_{\mathcal{H}_1}, \quad TT^\dagger = I_{\mathcal{H}_2},$$

and the back-map is fixed as:

$$S = T^{-1} = T^\dagger.$$

Implications (informational). Unitary transport preserves:

- inner products and norms,
- spectrum (up to multiplicity in finite truncation),
- trace and positivity under conjugation.

But such preservation claims are still required to be *declared* if they are used in interop certificates.

S2: Isometric Transport

Definition. SemClass = ISO if:

$$T^\dagger T = I_{\mathcal{H}_1},$$

but $TT^\dagger \neq I_{\mathcal{H}_2}$ in general. The natural back-map is:

$$S = T^\dagger,$$

which is a left inverse on the image of T .

Interpretation. Regime 1 embeds into regime 2 as a subspace. Transport can preserve structure on the embedded image, but not globally.

S3: Partial/Projected Transport

Definition. SemClass = PART if transport is defined only on a declared subset:

$$\mathcal{D}_1 \subseteq \mathcal{H}_1 \quad \text{and/or} \quad \mathcal{D}_2 \subseteq \mathcal{H}_2,$$

typically via projectors P_1, P_2 . Transport obligations are stated only for objects restricted to these domains.

Canonical form. A projected transport may declare:

$$T := P_2 \tilde{T} P_1, \quad S := P_1 \tilde{S} P_2,$$

with explicit projector validity checks (idempotence/self-adjointness) and deterministic routines.

Rule. If $\text{SemClass} = \text{PART}$, the artifact **MUST** publish the domain declarations (projectors or explicit admissible sets), otherwise claims of preservation are meaningless.

S4: Generalized Transport (Non-Invertible / Pseudo-Inverse)

Definition. $\text{SemClass} = \text{GEN}$ if T is not invertible and the back-map is a declared generalized inverse:

$$S := T^+,$$

computed by a deterministic routine (e.g. truncated SVD with fixed thresholds).

Rule (pseudo-inverse determinism). If a pseudo-inverse is used, the artifact **MUST** declare:

- the algorithm (e.g. SVD) and ordering conventions,
- threshold/grid policy for singular values,
- truncation and rounding policies,
- and tie-breaks for degenerate singular values.

Otherwise, reject with **FAIL__NONDETERMINISM**.

2.3 Back-Map Types BackMapType

EY28 standardizes the allowed back-map types:

B1: Adjoint

$$S = T^\dagger.$$

Allowed for UNIT and ISO, and sometimes PART (with explicit domain restrictions).

B2: Inverse

$$S = T^{-1}.$$

Allowed only if the inverse exists in the declared truncation and is computed deterministically.

B3: Pseudo-Inverse

$$S = T^+.$$

Allowed only for GEN (and sometimes PART) with explicit deterministic routine.

B4: Left/Right Inverse on Image

Declare S such that:

$$ST = I_{\mathcal{H}_1} \quad (\text{left inverse}) \quad \text{or} \quad TS = I_{\mathcal{H}_2} \quad (\text{right inverse}),$$

only on a declared image/domain. Requires PART semantics with explicit domain declaration.

Rule. If BackMapType is not $\dagger$ or (-1) , the artifact MUST include a RoutineSpec for computing S .

2.4 Preservation Predicate Vocabulary

Preservation is declared as explicit predicates. EY28 standardizes a minimal vocabulary.

Definition 2.1 (Preservation Predicate). A preservation predicate is a deterministic boolean condition:

$$\text{Preserve}_\alpha(T; \text{inputs}) \in \{\text{TRUE}, \text{FALSE}\},$$

with explicit tolerance and check routines under declared NumericModel(s).

P1: Inner Product Preservation

$$\langle Tx, Ty \rangle_2 \approx \langle x, y \rangle_1$$

checked on a fixed test set $\mathcal{V}$ (committed by hash or deterministic seed). Exact for UNIT/ISO in exact arithmetic; toleranced otherwise.

P2: Norm Preservation (Isometry Test)

$$\|Tx\|_2 \approx \|x\|_1$$

on $\mathcal{V}$, with explicit tolerance.

P3: Operator Conjugation Consistency

For a declared operator family $\mathcal{F}$:

$$A_2 \stackrel{?}{=} T A_1 S$$

checked either exactly (finite exact arithmetic) or within tolerance using a declared matrix/operator norm.

P4: Spectrum-Class Preservation (Filter-Aware)

For workflows that care about spectral classes (EY23), define:

$$\text{Class}(A_1) \stackrel{?}{=} \text{Class}(A_2)$$

under a declared classifier, typically based on committed spectral summaries or canonical filters. This predicate is permitted only if the classifier is committed and deterministic.

P5: Trace / Positivity Preservation (Density-Aware)

For states $\rho_1 \mapsto \rho_2 = T \rho_1 S$:

$$\text{Tr}(\rho_2) \approx \text{Tr}(\rho_1), \quad \rho_2 \succeq 0,$$

checked with explicit tolerance and positivity test routine (deterministic eigenvalue threshold grid).

P6: Commutation Preservation (EY27-Aware)

If $\text{Comm}(A_1, B_1) = \text{TRUE}$ under a declared EY27 PredID, then a workflow may claim:

$$\text{Comm}(A_2, B_2) = \text{TRUE}, \quad A_2 = T A_1 S, \quad B_2 = T B_1 S,$$

but only if it declares:

- the transported PredID (or the transported objects and re-check under an EY27 instance in regime 2),
- truncation alignment,
- and explicit tolerance semantics.

Rule. No commutation preservation is implied by default, even for UNIT transport, unless explicitly declared and checked.

2.5 Tolerance and Test-Set Rules

Several preservation predicates rely on finite test sets $\mathcal{V}$ or $\mathcal{F}$.

Rule (test sets). Any finite test set used for preservation MUST be:

- explicitly listed (if small), or
- committed by hash (if large), or
- generated deterministically from a public seed with a declared generator routine.

Otherwise: **REJECT_HIDDEN_INPUT**.

Rule (tolerance binding). All tolerances used in preservation predicates MUST be explicit constants or quantized grids fixed in SpecID.

2.6 Deterministic Failure Codes (Semantics Layer)

At minimum:

- **REJECT_SPECID_MISSING**
- **REJECT_SEMCLASS_MISSING**
- **REJECT_BACKMAP_MISSING**
- **FAIL_NONDETERMINISM**
- **FAIL_DOMAIN_MISMATCH**
- **FAIL_TRUNC_MISMATCH**
- **FAIL_NUMERICMODEL_MISMATCH**
- **FAIL_PRESERVATION**

2.7 Chapter Summary

This chapter defined normative transport semantics classes (UNIT, ISO, PART, GEN), allowed back-map types (adjoint, inverse, pseudo-inverse, left/right inverse on image), and a standardized vocabulary of preservation predicates (inner product, norm, conjugation consistency, spectrum-class, trace/positivity, commutation). It fixed test-set and tolerance binding rules and introduced semantics-layer failure codes.

Chapter 3 next defines the EY28 artifact structure and SpecID schema: how regimes are declared, how T and S are encoded/committed, how truncation alignment is specified, and what certificates are required for transported claims.

3 Artifact Structure and SpecID Schema

3.1 Purpose

Chapters 1–2 defined why transport must be auditable and fixed the semantics classes and preservation predicates. This chapter defines EY28 as an artifact with a precise SpecID schema:

- how the two regimes are declared,
- how the forward map $T_{1 \rightarrow 2}$ and the back-map $S_{2 \rightarrow 1}$ are represented and committed,
- how truncation alignment is specified,
- how preservation claims are bound to check routines,
- and what transcript/certificate payload is required.

3.2 Regime Declarations

Each regime is a typed environment:

$$\mathcal{R}_k := (\mathcal{H}_k, \text{Enc}_k, \text{Trunc}_k, \text{NumericModel}_k, \text{CalcSpec}_k).$$

RegimeSpec tuple

EY28 standardizes a regime declaration object:

$$\text{RegimeSpec}_k := (\text{RegimeID}_k, \text{SpaceType}_k, \text{EncodingRule}_k, \text{Trunc}_k, \text{NumericModel}_k, \text{CalcSpec}_k).$$

Rule. Both RegimeSpec_1 and RegimeSpec_2 MUST appear in SpecID. A transport without regime typing is invalid.

3.3 Map Objects T and S

Let:

$$T := T_{1 \rightarrow 2}, \quad S := S_{2 \rightarrow 1}.$$

MapSpec tuple

EY28 represents each map by a MapSpec:

$$\text{MapSpec}(M) := (\text{MapType}, \text{EncodingRule}, \text{AdjointSpec}, \text{InverseSpec}, \text{RoutineSpec}).$$

MapType. At minimum one of:

- **matrix-explicit** (finite matrix in canonical encoding),
- **sparse-linear** (sparse operator with deterministic apply),
- **factored** (product of committed factors in a declared order),
- **constructed** (deterministic construction from committed data/rules).

EncodingRule. Must bind canonical serialization and hashing. If the map is external, it MUST be committed by hash (EY22 Appendix A).

AdjointSpec / InverseSpec. Must declare whether adjoints/inverses are:

- computed explicitly and committed,
- computed by a deterministic routine,
- or not available (only allowed when SemClass declares it, e.g. PART with domain limits).

RoutineSpec. If S is a pseudo-inverse or generalized inverse, RoutineSpec MUST be present and fully specified (Chapter 2).

Commitment Rules

Rule (map commitment). SpecID MUST bind map content by one of:

- inline canonical encoding (small maps),
- hash commitment to the encoded object,
- or hash commitment to a deterministic construction recipe + committed inputs.

Otherwise: **REJECT_HIDDEN_INPUT**.

3.4 Transport Declaration Object

EY28 standardizes the transport declaration:

$\text{TransportDecl} := (\text{SemClass}, \text{BackMapType}, \text{PushPullForms}, \text{AlignPolicy}, \text{PreserveSet}).$

PushPullForms

Declare which forms are used:

- **pushforward:** $A_2 := TA_1S$,
- **pullback:** $g_1 := Sg_2T$,
- or both.

Rule. If an artifact uses pushforward/pullback in later claims, the form MUST be declared here.

AlignPolicy (Truncation Alignment Policy)

Truncation alignment is one of:

- **same-N**: both regimes use identical truncation N ,
- **mapped-N**: $N_2 = f(N_1)$ with declared deterministic f ,
- **multi-N**: explicit finite lists $N_1 \in \mathcal{N}_1$, $N_2 \in \mathcal{N}_2$ with a declared CrossNPolicy,
- **projected**: transport defined only on declared projectors P_1, P_2 (PART semantics).

Rule. If AlignPolicy is not **same-N**, the policy MUST include all mapping rules and tie-breaks. Otherwise: **FAIL_TRUNC_MISMATCH**.

PreserveSet

PreserveSet is a finite set of declared preservation predicates from Chapter 2:

$$\text{PreserveSet} \subseteq \{P1, \dots, P6\}$$

plus any declared extensions (each with committed definition).

Rule. Only predicates listed in PreserveSet may be claimed preserved by this transport. Anything else is non-claim.

3.5 SpecID for EY28

ArtifactType. For EY28:

$$\text{ArtifactType} = \text{INTW}.$$

SpecID components (EY28 binding). The full SpecID tuple is:

$$\text{SpecID}_{\text{EY28}} := (\text{INTW}, \text{RegimeSpec}_1, \text{RegimeSpec}_2, \text{MapSpec}(T), \text{MapSpec}(S), \text{TransportDecl}, \text{NumericMod})$$

NumericModelLink

Because two regimes may declare different NumericModels, EY28 includes a linking component:

$$\text{NumericModelLink} := (\text{NMPolicy}, \text{Coercions}, \text{OrderFixes}).$$

NMPolicy. One of:

- **must-match**: $\text{NumericModel}_1 = \text{NumericModel}_2$,
- **coerce**: a declared coercion rule exists,
- **separate**: computations are done in each regime separately with explicit cross-check rules.

Rule. If NMPolicy is **coerce**, the coercion routine MUST be deterministic and committed by hash.

3.6 TranscriptSpec and Required Commitments

Transport is often used to justify downstream claims. Therefore EY28 requires transcript commitments that bind:

- the committed representation (or construction recipe) of T and S ,
- all regime declarations and truncation alignment policies,
- all preservation predicate results that are claimed,
- and any transported objects A_2 , B_2 , etc. when those objects are published as outputs.

Rule. If a downstream artifact claims to depend on transported content (e.g. “we applied EY23 in regime 2 after transporting H ”), the EY28 transcript commitments MUST be sufficient to replay that transport.

3.7 Interop Component (Mini)

EY28 interop binds which artifacts are transported and how.

Required fields.

- **TransportedArtifacts:** list of artifact references whose objects are transported (EY23/EY24/EY25/EY26)
- **FormsUsed:** which pushforward/pullback forms were used per artifact,
- **PreservationClaims:** which predicates in PreserveSet are asserted for which transported objects,
- **CompatPredicates:** predicate IDs (EY22 style) used to certify compatibility.

Rule. If an interop claim asserts preservation of commutation constraints, it MUST cite the relevant EY27 PredID(s) and re-check routine(s).

3.8 Deterministic Failure Codes (Schema Layer)

At minimum:

- **REJECT_MALFORMED**
- **REJECT_SPECID_MISSING**
- **REJECT_HIDDEN_INPUT**
- **FAIL_HASH_MISMATCH**
- **FAIL_DOMAIN_MISMATCH**
- **FAIL_TRUNC_MISMATCH**
- **FAIL_NUMERICMODEL_MISMATCH**
- **FAIL_NONDETERMINISM**
- **FAIL_PRESERVATION**

3.9 Chapter Summary

This chapter defined the EY28 artifact schema: typed regime declarations, committed map specifications for T and S , a transport declaration object binding semantics class, back-map type, push/pull forms, truncation alignment policy, and a finite preservation predicate set. It also specified NumericModel linking, required transcript commitments, interop mini fields, and schema-layer failure codes.

Chapter 4 next defines primitive composition forms and baseline routines for computing T , S , transported objects, and preservation checks deterministically (INTW-0 reference profile).

4 Primitive Transport Forms and Baseline Routines (INTW-0)

4.1 Purpose

EY28 must be practically verifiable. That requires baseline routines that:

- compute transported objects deterministically,
- compute back-maps (if not explicitly provided) deterministically,
- evaluate preservation predicates on fixed test sets,
- and produce replayable transcript commitments.

This chapter defines primitive transport forms and introduces the conservative reference profile **INTW-0**.

4.2 Primitive Transport Forms (Normative)

The following primitive forms are permitted. Each EY28 instance **MUST** declare which are used in TransportDecl.

T1: Operator Pushforward

Given A_1 in regime 1:

$$A_2 := T A_1 S.$$

This is the primary operator-transport form.

Domain rule. If A_1 is only defined on a subspace, the transport **MUST** include explicit projectors or domain declarations. Otherwise: **FAIL_DOMAIN_MISMATCH**.

T2: Operator Pullback

Given an operator/observable g_2 in regime 2:

$$g_1 := S g_2 T.$$

Used when constraints or observables are defined in the target regime but need evaluation in the source regime.

T3: State Transport (Density/State Object)

Given ρ_1 in regime 1:

$$\rho_2 := T\rho_1 S.$$

Special case where positivity/trace predicates may be asserted (P5).

T4: Constraint Transport (Predicate Carryover)

Constraints are transported *by re-instantiation*:

$$\text{Comm}(A_1, B_1) \rightsquigarrow \text{Comm}(A_2, B_2), \quad A_2 = T A_1 S, \quad B_2 = T B_1 S,$$

and then checked in regime 2 via an EY27 instance bound to regime 2 NumericModel/Truncation.

Rule. Constraints are never “transported by narrative”. They are transported by transported objects + re-check routines.

4.3 Baseline Routine Set

EY28 defines baseline deterministic routines $\text{RoutineID} \in \mathcal{R}_0^{\text{INTW}}$.

Routine INTW-R0: Direct Pushforward (Explicit T, S)

Inputs. Canonical encodings or hashes for T, S, A_1 .

Steps.

1. Canonicalize T, S, A_1 under declared NumericModelLink rules.
2. Compute $M := T A_1$ with fixed MulOrder/ReduceOrder.
3. Compute $A_2 := M S$.
4. Canonicalize A_2 at declared CanonPoints.
5. Commit transcript: hashes of T, S, A_1, A_2 and intermediate rules.

Outputs. Transported operator A_2 (or its commitment), and transcript commitments.

Routine INTW-R1: Pullback (Explicit T, S)

Analogous to INTW-R0 but computes $g_1 := S g_2 T$.

Routine INTW-R2: Adjoint Back-Map Construction

If BackMapType is $\dagger$ and S not explicitly provided:

$$S := T^\dagger$$

computed deterministically:

- conjugate + transpose in a fixed order,

- canonicalize result,
- commit S by hash.

Routine INTW-R3: Inverse Back-Map Construction (Finite)

If BackMapType is (-1) and S not explicitly provided:

$$S := T^{-1}$$

computed deterministically by a declared algorithm, baseline:

- LU decomposition with fixed pivot policy and tie-breaks,
- deterministic rounding/canonicalization at fixed points,
- reject if determinant test fails (within tolerance).

Failure. **FAIL_NONINVERTIBLE** if inversion fails deterministically.

Routine INTW-R4: Pseudo-Inverse Back-Map Construction (GEN)

If BackMapType is $+$:

$$S := T^+$$

baseline algorithm: truncated SVD.

1. Compute SVD with fixed ordering conventions for singular vectors.
2. Apply a fixed singular-value threshold policy: $\sigma_i < \tau \Rightarrow \sigma_i^{-1} = 0$, with τ on a declared grid.
3. Recompose S deterministically and canonicalize.

Rule. All thresholds and tie-breaks MUST be declared. Otherwise: **FAIL_NONDETERMINISM**.

Routine INTW-R5: Preservation Check Harness

A generic harness that evaluates PreserveSet predicates. It consumes:

- test sets $\mathcal{V}$ / $\mathcal{F}$ (committed by hash or seed),
- tolerances (explicit constants or grids),
- and the declared predicate list.

It outputs:

$$\text{PreserveReport} := \{(\text{Pred}, \text{Result}, \text{Value})\}_{\text{Pred} \in \text{PreserveSet}}$$

committed by hash.

4.4 INTW-0 Reference Profile (Baseline)

INTW-0 is the conservative baseline profile for EY28, analogous to FLOQ-0 and COMM-0.

INTW-0 Fixes

INTW-0 fixes:

- $\text{SemClass} \in \{\text{UNIT}, \text{ISO}\}$ only,
- $\text{BackMapType} = \dagger$ (so $S = T^\dagger$),
- $\text{AlignPolicy} = \text{same-N}$ (single truncation level N_0),
- pushforward and pullback allowed; constraints transported only by re-check (T4),
- preservation predicates allowed: P1, P2, P3 (optional), P6 (only if explicitly re-checked via EY27),
- pseudo-inverses and projected domains forbidden in baseline.

Rule. INTW-0 forbids GEN and PART semantics. If an artifact needs pseudo-inverses, projectors, or multi- N alignment, it must declare a non-baseline profile.

4.5 Deterministic Failure Codes (Routine Layer)

At minimum:

- **FAIL_HASH_MISMATCH**
- **FAIL_NONDETERMINISM**
- **FAIL_NONINVERTIBLE**
- **FAIL_DOMAIN_MISMATCH**
- **FAIL_TRUNC_MISMATCH**
- **FAIL_NUMERICMODEL_MISMATCH**
- **FAIL_PRESERVATION**

4.6 Chapter Summary

This chapter defined primitive transport forms (pushforward, pullback, state transport, constraint transport by re-check), introduced baseline deterministic routines (INTW-R0..R5) for computing transported objects and back-maps, and defined the conservative INTW-0 reference profile: unitary/isometric transport with adjoint back-map, same truncation alignment, and limited preservation claims.

Chapter 5 next specifies certificate payload and verifier checks for EY28: how transported objects and preservation reports are committed, how back-maps are validated, and how interop claims are certified deterministically.

5 Certificate and Verifier: Transport Replay and Preservation Acceptance

5.1 Purpose

EY28 is often used as the justification layer for “this workflow in regime 2 corresponds to that workflow in regime 1”. Therefore, EY28 must publish more than a map: it must publish replayable

commitments for:

- the regime declarations,
- the maps T and S (or deterministic construction rules),
- the transported objects (when published),
- and the preservation predicate results used to support interop claims.

This chapter specifies:

- the certificate payload π_{INTW} ,
- deterministic verifier checks for acceptance,
- transcript commitments for transport computations,
- and required failure codes.

5.2 Certificate Object π_{INTW}

Rule. An EY28 publication **MUST** include a certificate π_{INTW} in canonical certificate encoding (EY22-CCE).

Header (required)

- artifact type: INTW
- artifact symbol: $T_{1 \rightarrow 2}$
- RegimeID₁, RegimeID₂
- SemClass, BackMapType, AlignPolicy
- Version
- Certificate schema identifier (EY22-CCE)

SpecID (required)

The full SpecID tuple **MUST** be embedded:

$\text{SpecID}_{\text{EY28}} := (\text{INTW}, \text{RegimeSpec}_1, \text{RegimeSpec}_2, \text{MapSpec}(T), \text{MapSpec}(S), \text{TransportDecl}, \text{NumericMod})$

Committed Inputs (required)

π_{INTW} **MUST** commit to:

- RegimeSpec₁, RegimeSpec₂ (including Trunc and NumericModels),
- MapSpec(T) and its content: inline encoding or hash-commitment or construction recipe + inputs,
- MapSpec(S) and its content: explicit or derived by declared routine,
- TransportDecl fields (SemClass, BackMapType, PushPullForms, AlignPolicy, PreserveSet),
- NumericModelLink (NMPolicy and any coercion routines),

- all tolerances and test sets used by preservation predicates (explicit, hashed, or deterministic seed).

Committed Outputs (required)

π_{INTW} MUST commit to:

- a **Transport Transcript Commitment** Tx_{intw} ,
- a **Preservation Report Commitment** Tx_{pres} covering all predicates in `PreserveSet`,
- transported objects if published as outputs (e.g. A_2, ρ_2) or their hashes,
- deterministic failure code if evaluation fails.

Transport Transcript (minimum contents)

At minimum, the transcript commitment MUST bind:

- hashes of T and S (or construction recipes + input hashes),
- hashes of the input objects transported (if any are part of the claim),
- declared routine IDs used (INTW-R0..R5 or committed equivalents),
- canonicalization points and multiplication/reduction order settings,
- truncation alignment decisions (including CrossNPolicy results if multi- N).

Preservation Report (minimum contents)

A preservation report must include a list:

$$\text{PreserveReport} := \{(\text{PredID}, \text{Result}, \text{Value}, \text{Tol})\}$$

for each predicate in `PreserveSet`, with deterministic serialization and hashing.

Rule. Preservation predicates not listed in `PreserveSet` MUST NOT appear in the report as “implied”. If they appear, reject as malformed.

5.3 Verifier Contract

A verifier consumes π_{INTW} and returns:

$$\{\text{ACCEPT}, \text{REJECT}\} \cup \{\text{FAIL_}\ast\}.$$

Acceptance meaning. ACCEPT means: the transport artifact is well-formed, replayable, and the declared preservation predicates (if any) hold under the declared routines. It does not mean the regimes are “the same” or that any undeclared property is preserved.

5.4 Required Verifier Checks (Normative)

V1: Structural Validity (EY22-CCE)

Parse canonical certificate encoding and confirm required fields exist. Failure: **REJECT_MALFORMED**.

V2: SpecID Completeness and Consistency

Confirm:

- `ArtifactType=INTW`,
- both `RegimeSpecs` present,
- `MapSpecs` for T and S present,
- `TransportDecl` present (`SemClass`, `BackMapType`, `AlignPolicy`, `PreserveSet`),
- `NumericModelLink` present.

Failure: **REJECT_SPECID_MISSING**, **REJECT_SEMCLASS_MISSING**, **REJECT_BACKMAP_MISSING**.

V3: Commitment Integrity

Verify all hash-committed objects resolve and match:

- T content or construction recipe,
- S content or construction recipe,
- test sets $\mathcal{V}/\mathcal{F}$ and tolerances,
- any transported objects that are part of the claim.

Failure: **FAIL_HASH_MISMATCH**, **REJECT_HIDDEN_INPUT**.

V4: NumericModel Linking Rules

Enforce `NMPolicy`:

- **must-match**: check equality of `NumericModels`,
- **coerce**: replay coercion routine deterministically,
- **separate**: enforce declared cross-check procedure.

Failure: **FAIL_NUMERICMODEL_MISMATCH**, **FAIL_NONDETERMINISM**.

V5: Truncation Alignment

Enforce `AlignPolicy`:

- **same- N** : confirm $N_1 = N_2 = N_0$,
- **mapped- N** : recompute $N_2 = f(N_1)$,
- **multi- N** : apply `CrossNPolicy` and bind decisions,
- **projected**: validate projectors and domain declarations.

Failure: **FAIL_TRUNC_MISMATCH**, **FAIL_PROJECTOR_INVALID**.

V6: Back-Map Construction/Validation

If S is derived:

- adjoint: replay INTW-R2,
- inverse: replay INTW-R3,
- pseudo-inverse: replay INTW-R4.

Then validate required identities within tolerance based on SemClass:

- UNIT: $TS \approx I_2$ and $ST \approx I_1$,
- ISO: $ST \approx I_1$,
- PART: identities only on declared domains,
- GEN: pseudo-inverse conditions per routine spec.

Failure: **FAIL_NONINVERTIBLE**, **FAIL_PRESERVATION**, **FAIL_NONDETERMINISM**.

V7: Transport Replay (if objects are part of the claim)

If the certificate includes transported objects (or hashes), the verifier MUST replay transport:

- pushforward/pullback forms as declared,
- INTW-R0/INTW-R1 (or committed equivalents),
- canonicalization points and ordering rules.

Failure: **FAIL_DOMAIN_MISMATCH**, **FAIL_TRANSCRIPT_MISMATCH**.

V8: Preservation Predicate Checks

For each predicate in PreserveSet, run the declared predicate routine (INTW-R5 harness) on committed test sets and tolerances, and compare to the published Preservation Report. Failure: **FAIL_PRESERVATION**, **FAIL_TRANSCRIPT_MISMATCH**.

V9: Interop Claim Consistency

If Interop claims composition with other artifacts:

- verify that referenced artifacts (by hash/SpecID) exist and match,
- verify that preservation claims listed are a subset of PreserveSet,
- for commutation preservation, verify that the workflow includes an EY27 re-check in regime 2.

Failure: **FAIL_COMPAT_PREDICATE**, **FAIL_INVARIANT_NOT_PRESERVED**.

5.5 Acceptance Rule

Rule. A verifier returns ACCEPT iff:

- V1–V6 pass,

- any transported objects that are part of the claim replay deterministically,
- preservation report matches replay checks for all predicates in PreserveSet,
- transcript commitments match.

If any preservation predicate fails, the artifact may still be well-formed but must return **FAIL_PRESERVATION** (not **ACCEPT**).

5.6 Required Failure Codes

At minimum:

- **REJECT_MALFORMED**
- **REJECT_SPECID_MISSING**
- **REJECT_SEMCLASS_MISSING**
- **REJECT_BACKMAP_MISSING**
- **REJECT_HIDDEN_INPUT**
- **FAIL_HASH_MISMATCH**
- **FAIL_NUMERICMODEL_MISMATCH**
- **FAIL_TRUNC_MISMATCH**
- **FAIL_PROJECTOR_INVALID**
- **FAIL_NONDETERMINISM**
- **FAIL_NONINVERTIBLE**
- **FAIL_DOMAIN_MISMATCH**
- **FAIL_PRESERVATION**
- **FAIL_COMPAT_PREDICATE**
- **FAIL_INVARIANT_NOT_PRESERVED**
- **FAIL_TRANSCRIPT_MISMATCH**

5.7 Chapter Summary

This chapter defined the EY28 certificate π_{INTW} and deterministic verifier checks: structural validity, SpecID completeness, commitment integrity, NumericModel linking, truncation alignment, back-map construction/validation, transport replay, preservation predicate verification, and interop claim consistency. It also fixed acceptance rules and required failure codes, making EY28 a replayable transport artifact.

Chapter 7 next provides interoperability declarations (mini) for EY28: how transport composes with EY23–EY30, how compatibility matrices should mark transported workflows, and how interop certificates bind regime bridges deterministically.

6 Interop Declarations (Mini): Regime Bridges as Composable Contracts

6.1 Purpose

EY28 exists so that *workflows* can move between regimes without informal interpretation. This chapter instantiates EY22 interoperability rules for regime transport:

- how EY28 is referenced by other artifacts (and vice versa),
- what it means for a composition to be “transport-compatible”,
- how to write mini compatibility predicates for typical pairings (EY23–EY30),
- and what interop certificates must bind when a result depends on a regime bridge.

Rule. If a published claim depends on applying an artifact in regime 2 to an object transported from regime 1 (or vice versa), the publication **MUST** include an interop certificate π_{interop} referencing EY28 π_{INTW} .

6.2 Interop Component Fields (Standard)

Any artifact that composes with EY28 **MUST** include an interop entry:

Interop $\ni$ (INTW-Ref, InteropRole, FormsUsed, PreserveClaims, Scope, CompatPredicates).

Required fields.

- INTW-Ref: hash/SpecID reference to π_{INTW} (or SpecID hash) and RegimeIDs.
- InteropRole: one of **source-bridge**, **target-bridge**, **two-way**, **constraint-bridge**.
- FormsUsed: pushforward/pullback/state/constraint transport forms actually used.
- PreserveClaims: subset of PreserveSet asserted for this composition.
- Scope: which objects are transported (e.g. H , V , $\mathcal{L}$, step set, summaries).
- CompatPredicates: identifiers of the compatibility predicates used (defined below).

Rule. PreserveClaims **MUST** be a subset of EY28 PreserveSet. Otherwise: **FAIL_COMPAT_PREDICATES**.

6.3 Compatibility Predicates (EY28 Mini Set)

EY28 standardizes a minimal predicate set for common interop reasoning.

INTW-C0: SpecID/Certificate Binding

INTW-C0 := (EIF INTW-Ref matches a valid π_{INTW} and RegimeIDs align).

INTW-C1: Form Consistency

INTW-C1 := (EIF the composition uses only forms declared in TransportDecl.PushPullForms).

INTW-C2: NumericModel/Truncation Alignment

INTW-C2 := (EIF NMPolicy and AlignPolicy requirements are satisfied for the composed workflow).

INTW-C3: Preservation Claims Supported

INTW-C3 := (EIF each asserted preserve claim has a corresponding PASS entry in PreserveReport).

INTW-C4: Object Commitment Closure

INTW-C4 := (EIF all transported objects used downstream are committed by hash or replayable by routine t

INTW-C5: Constraint Transport Is Re-Checked

For constraint transport (e.g. commutation):

INTW-C5 := (EIF constraints are transported by transporting objects and re-checking via the target predicate

Rule. Any workflow asserting preservation of an EY27 commutation predicate MUST satisfy INTW-C5 and MUST cite the target-regime EY27 PredID.

6.4 EY28 Pairwise Interop: Standard Mini Declarations

This section lists the intended composition semantics with EY23–EY30. These are *templates*; real workflows must cite predicate IDs.

INTW $\leftrightarrow$ CSF (EY23): Transported Filtering

Typical workflow. Transport $H_1 \mapsto H_2$, then apply $F_\star(H_2)$ in regime 2.

Compatibility template. Mark OK* if:

$$\text{INTW-C0} \wedge \text{INTW-C1} \wedge \text{INTW-C2} \wedge \text{INTW-C4}$$

and if any spectral-class preservation is claimed, require:

$$\text{INTW-C3 with P4 present in PreserveSet.}$$

INTW $\leftrightarrow$ VPO (EY24): Transported Validity Penalties

Typical workflow. Transport a validity operator $V_1 \mapsto V_2$ or transport constraint object K used to build V.

Compatibility template. OK* if:

$$\text{INTW-C0} \wedge \text{INTW-C2} \wedge \text{INTW-C4}$$

and if positivity/trace preservation is claimed for state evolution under validity, require P5 with PASS.

INTW $\leftrightarrow$ LIOUV (EY25): Transported Generators and Evolutions

Two patterns are allowed:

Pattern A (generator transport). Transport the generator (Hamiltonian/Liouvillian) and evolve in regime 2.

Pattern B (trajectory transport). Evolve in regime 1 and transport snapshots/states to regime 2.

Compatibility template. OK* if:

$$\text{INTW-C0} \wedge \text{INTW-C2} \wedge \text{INTW-C4}$$

plus explicit declaration of which pattern is used. If invariants are claimed preserved, require INTW-C3 with those predicates present.

INTW $\leftrightarrow$ FLOQ (EY26): Transported Step Sets and Tokens

Typical workflow. Transport step operators $U_{j,1} \mapsto U_{j,2}$ and build/verify a token in regime 2.

Compatibility template. OK* if:

$$\text{INTW-C0} \wedge \text{INTW-C1} \wedge \text{INTW-C2} \wedge \text{INTW-C4}$$

and if the token claims to preserve a commutation symmetry, require INTW-C5 and a target-regime EY27 PredID.

INTW $\leftrightarrow$ COMM (EY27): Transported Commutation Constraints

Canonical rule. Commutation constraints are transported by transporting the objects and re-checking:

$$A_2 = T A_1 S, \quad B_2 = T B_1 S, \quad \text{Comm}(A_2, B_2) \text{ checked via EY27 in regime 2.}$$

Compatibility template. OK* iff INTW-C5 holds and NumericModel/Truncation alignment is satisfied.

INTW $\leftrightarrow$ CGR (EY29): Transport as Reduction

Transport can *be* a reduction (when $N_2 < N_1$ or when PART semantics apply). In that case, the workflow **MUST** declare whether EY28 is effectively performing coarse-grain and must use EY29 invariants/tolerances.

Compatibility template. OK* if:

$$\text{INTW-C0} \wedge \text{INTW-C2} \wedge \text{INTW-C4}$$

and any reduction-preservation claim requires explicit EY29 certificates.

INTW $\leftrightarrow$ KOOP (EY30): Export Across Representations

Koopman export often changes representation (state-space to observable-space). EY28 provides the bridge typing, but preservation claims must be explicit (P3/P4/P5 depending on semantics).

Compatibility template. OK* if INTW-C0..C4 hold and the workflow declares the export semantics mapping.

6.5 Interop Certificates for Regime-Bridged Workflows

Whenever a claim depends on an EY28 bridge, the workflow SHOULD publish an interop certificate:

$\pi_{\text{interop}} := (\text{SpecIDs of all artifacts, INTW-Ref, FormsUsed, Scope, PreserveClaims, CompatPredicates, Trans})$

Rule. If the outcome is published as an artifact-derived claim (e.g. spectral bucket assignment, regime bridge, Koopman invariant), an interop certificate MUST be provided if the claim depends on cross-regime composition.

6.6 Deterministic Interop Failure Codes

At minimum:

- **FAIL_COMPAT_PREDICATE**
- **FAIL_NUMERICMODEL_MISMATCH**
- **FAIL_TRUNC_MISMATCH**
- **FAIL_DOMAIN_MISMATCH**
- **FAIL_PRESERVATION**
- **FAIL_INVARIANT_NOT_PRESERVED**
- **FAIL_TRANSCRIPT_MISMATCH**

6.7 Chapter Summary

This chapter defined the EY28 interoperability mini-language: how INTW references appear in other artifacts' Interop components, a minimal set of compatibility predicates (INTW-C0..C5), and standard pairwise interop templates for composing EY28 with EY23–EY30. It also defined interop certificates for regime-bridged workflows and required deterministic failure codes.

Chapter ?? next provides an INTW-0 reference SpecID template, fixes the publication boundary and non-claims for EY28, and closes the artifact in a QFC-consistent way.

7 Interop Declarations (Mini): Regime Bridges as Composable Contracts

7.1 Purpose

EY28 exists so that *workflows* can move between regimes without informal interpretation. This chapter instantiates EY22 interoperability rules for regime transport:

- how EY28 is referenced by other artifacts (and vice versa),
- what it means for a composition to be “transport-compatible”,
- how to write mini compatibility predicates for typical pairings (EY23–EY30),
- and what interop certificates must bind when a result depends on a regime bridge.

Rule. If a published claim depends on applying an artifact in regime 2 to an object transported from regime 1 (or vice versa), the publication **MUST** include an interop certificate π_{interop} referencing EY28 π_{INTW} .

7.2 Interop Component Fields (Standard)

Any artifact that composes with EY28 **MUST** include an interop entry:

Interop $\ni$ (INTW-Ref, InteropRole, FormsUsed, PreserveClaims, Scope, CompatPredicates).

Required fields.

- INTW-Ref: hash/SpecID reference to π_{INTW} (or SpecID hash) and RegimeIDs.
- InteropRole: one of **source-bridge**, **target-bridge**, **two-way**, **constraint-bridge**.
- FormsUsed: pushforward/pullback/state/constraint transport forms actually used.
- PreserveClaims: subset of PreserveSet asserted for this composition.
- Scope: which objects are transported (e.g. H , V , $\mathcal{L}$, step set, summaries).
- CompatPredicates: identifiers of the compatibility predicates used (defined below).

Rule. PreserveClaims **MUST** be a subset of EY28 PreserveSet. Otherwise: **FAIL_COMPAT_PREDICATES**.

7.3 Compatibility Predicates (EY28 Mini Set)

EY28 standardizes a minimal predicate set for common interop reasoning.

INTW-C0: SpecID/Certificate Binding

INTW-C0 := (EIF INTW-Ref matches a valid π_{INTW} and RegimeIDs align).

INTW-C1: Form Consistency

INTW-C1 := (EIF the composition uses only forms declared in TransportDecl.PushPullForms).

INTW-C2: NumericModel/Truncation Alignment

INTW-C2 := (EIF NMPolicy and AlignPolicy requirements are satisfied for the composed workflow).

INTW-C3: Preservation Claims Supported

INTW-C3 := (EIF each asserted preserve claim has a corresponding PASS entry in PreserveReport).

INTW-C4: Object Commitment Closure

INTW-C4 := (EIF all transported objects used downstream are committed by hash or replayable by routine t

INTW-C5: Constraint Transport Is Re-Checked

For constraint transport (e.g. commutation):

INTW-C5 := (EIF constraints are transported by transporting objects and re-checking via the target predicat

Rule. Any workflow asserting preservation of an EY27 commutation predicate **MUST** satisfy INTW-C5 and **MUST** cite the target-regime EY27 PredID.

7.4 EY28 Pairwise Interop: Standard Mini Declarations

This section lists the intended composition semantics with EY23–EY30. These are *templates*; real workflows must cite predicate IDs.

INTW $\leftrightarrow$ CSF (EY23): Transported Filtering

Typical workflow. Transport $H_1 \mapsto H_2$, then apply $F_\star(H_2)$ in regime 2.

Compatibility template. Mark OK* if:

$$\text{INTW-C0} \wedge \text{INTW-C1} \wedge \text{INTW-C2} \wedge \text{INTW-C4}$$

and if any spectral-class preservation is claimed, require:

$$\text{INTW-C3 with P4 present in PreserveSet.}$$

INTW $\leftrightarrow$ VPO (EY24): Transported Validity Penalties

Typical workflow. Transport a validity operator $V_1 \mapsto V_2$ or transport constraint object K used to build V.

Compatibility template. OK* if:

$$\text{INTW-C0} \wedge \text{INTW-C2} \wedge \text{INTW-C4}$$

and if positivity/trace preservation is claimed for state evolution under validity, require P5 with PASS.

INTW $\leftrightarrow$ LIOUV (EY25): Transported Generators and Evolutions

Two patterns are allowed:

Pattern A (generator transport). Transport the generator (Hamiltonian/Liouvillian) and evolve in regime 2.

Pattern B (trajectory transport). Evolve in regime 1 and transport snapshots/states to regime 2.

Compatibility template. OK* if:

$$\text{INTW-C0} \wedge \text{INTW-C2} \wedge \text{INTW-C4}$$

plus explicit declaration of which pattern is used. If invariants are claimed preserved, require INTW-C3 with those predicates present.

INTW $\leftrightarrow$ FLOQ (EY26): Transported Step Sets and Tokens

Typical workflow. Transport step operators $U_{j,1} \mapsto U_{j,2}$ and build/verify a token in regime 2.

Compatibility template. OK* if:

$$\text{INTW-C0} \wedge \text{INTW-C1} \wedge \text{INTW-C2} \wedge \text{INTW-C4}$$

and if the token claims to preserve a commutation symmetry, require INTW-C5 and a target-regime EY27 PredID.

INTW $\leftrightarrow$ COMM (EY27): Transported Commutation Constraints

Canonical rule. Commutation constraints are transported by transporting the objects and re-checking:

$$A_2 = T A_1 S, \quad B_2 = T B_1 S, \quad \text{Comm}(A_2, B_2) \text{ checked via EY27 in regime 2.}$$

Compatibility template. OK* iff INTW-C5 holds and NumericModel/Truncation alignment is satisfied.

INTW $\leftrightarrow$ CGR (EY29): Transport as Reduction

Transport can *be* a reduction (when $N_2 < N_1$ or when PART semantics apply). In that case, the workflow MUST declare whether EY28 is effectively performing coarse-grain and must use EY29 invariants/tolerances.

Compatibility template. OK* if:

$$\text{INTW-C0} \wedge \text{INTW-C2} \wedge \text{INTW-C4}$$

and any reduction-preservation claim requires explicit EY29 certificates.

INTW $\leftrightarrow$ KOOP (EY30): Export Across Representations

Koopman export often changes representation (state-space to observable-space). EY28 provides the bridge typing, but preservation claims must be explicit (P3/P4/P5 depending on semantics).

Compatibility template. OK* if INTW-C0..C4 hold and the workflow declares the export semantics mapping.

7.5 Interop Certificates for Regime-Bridged Workflows

Whenever a claim depends on an EY28 bridge, the workflow SHOULD publish an interop certificate:

$\pi_{\text{interop}} := (\text{SpecIDs of all artifacts, INTW-Ref, FormsUsed, Scope, PreserveClaims, CompatPredicates, Trans}$

Rule. If the outcome is published as an artifact-derived claim (e.g. spectral bucket assignment, regime bridge, Koopman invariant), an interop certificate MUST be provided if the claim depends on cross-regime composition.

7.6 Deterministic Interop Failure Codes

At minimum:

- **FAIL_COMPAT_PREDICATE**
- **FAIL_NUMERICMODEL_MISMATCH**
- **FAIL_TRUNC_MISMATCH**
- **FAIL_DOMAIN_MISMATCH**
- **FAIL_PRESERVATION**
- **FAIL_INVARIANT_NOT_PRESERVED**
- **FAIL_TRANSCRIPT_MISMATCH**

7.7 Chapter Summary

This chapter defined the EY28 interoperability mini-language: how INTW references appear in other artifacts' Interop components, a minimal set of compatibility predicates (INTW-C0..C5), and standard pairwise interop templates for composing EY28 with EY23–EY30. It also defined interop certificates for regime-bridged workflows and required deterministic failure codes.

Chapter ?? next provides an INTW-0 reference SpecID template, fixes the publication boundary and non-claims for EY28, and closes the artifact in a QFC-consistent way.

8 Worked Examples (Non-Normative): Regime Bridges in Vol. No. IV

8.1 Purpose

This chapter is *non-normative*. It illustrates typical EY28 usage patterns:

- transporting a Hamiltonian and applying a canonical spectral filter (EY23),
- transporting a validity penalty (EY24) and preserving positivity claims,
- bridging a Liouvillian evolution (EY25) by generator-transport vs trajectory-transport,
- transporting a program token step set (EY26),
- transporting commutation constraints (EY27) by re-check,
- and bridging into observable space for Koopman export (EY30).

Reminder. Examples are schematic. They become valid only when all objects are canonically encoded or hash-committed, and NumericModel/Truncation alignment is explicit.

8.2 Example 1: Transport H_1 and Filter in Regime 2 (EY23)

Goal. Compute a filtered operator in regime 2:

$$H_2 := TH_1S, \quad \tilde{H}_2 := F_\star(H_2).$$

Artifact chain.

- EY28 publishes T, S and commits H_2 (or commits the transcript needed to replay it).
- EY23 in regime 2 takes H_2 as input and publishes $\text{SpecID}_{\text{EY23}}$ with its own NumericModel.
- Interop certificate binds: INTW-Ref, CSF-Ref, the exact forms used (**pushforward** then **filter**), and the preserved predicates needed for the claim (often none beyond transport replay).

Common pitfall. Claiming “spectral class preserved” without P4 in PreserveSet and a PASS entry in PreserveReport.

8.3 Example 2: Transport a Validity Penalty V (EY24)

Goal. Move a validity operator to regime 2:

$$V_2 := TV_1S,$$

so a verifier can enforce the same validity constraints in regime 2.

If positivity of states is claimed. If the workflow asserts that mapping preserves positivity of a transported density:

$$\rho_2 := T\rho_1S \succeq 0,$$

then EY28 must include P5 in PreserveSet with deterministic positivity test rules (eigenvalue threshold grid).

Common pitfall. Assuming positivity is preserved just because $S = T^\dagger$. That is only guaranteed if the map is truly unitary/isometric under the declared truncation and numeric policy—and even then, Vol. No. IV requires explicit declaration and check.

8.4 Example 3: Liouvillian Bridge (EY25) — Two Patterns

Pattern A: Generator Transport

Goal. Transport the generator $\mathcal{L}_1$ to $\mathcal{L}_2$ and evolve in regime 2:

$$\mathcal{L}_2 := T\mathcal{L}_1S, \quad \rho_2(t) = e^{t\mathcal{L}_2}\rho_2(0).$$

This requires explicit declaration of how superoperators are transported and represented (Map-Type may be factored/constructed).

Pattern B: Trajectory Transport

Goal. Evolve in regime 1 and transport snapshots:

$$\rho_2(t_k) := T\rho_1(t_k)S,$$

for a declared checkpoint set $\{t_k\}$.

Common pitfall. Mixing the two patterns without declaring it; verifiers cannot know whether generator transport or trajectory transport was used.

8.5 Example 4: Token Step-Set Transport (EY26)

Goal. Build a token in regime 2 from steps defined in regime 1:

$$U_{j,2} := TU_{j,1}S, \quad U_{F,2} := \prod_{j=1}^m U_{j,2}.$$

Interop binding. Interop must declare:

- scope: “step operators” $\{U_j\}$,
- forms: pushforward,
- and whether token verification occurs per-step or per-period (CommCheckScope if symmetry is involved).

8.6 Example 5: Transporting an EY27 Constraint (Re-check in Regime 2)

Goal. Given a commutation claim in regime 1:

$$\text{Comm}(A_1, B_1) = \text{TRUE},$$

transport the objects and re-check in regime 2:

$$A_2 := TA_1S, \quad B_2 := TB_1S, \quad \text{Comm}(A_2, B_2) = \text{TRUE}.$$

Required publication.

- EY28 transcript commitments for A_2, B_2 or replay construction.
- A target-regime EY27 instance (PredID₂) that defines commutation under regime-2 NumericModel/Trunc.
- Interop certificate that includes INTW-C5 and cites both PredID₁ and PredID₂ where relevant.

Common pitfall. Claiming that commutation is preserved under transport without re-check. Vol. No. IV forbids this as a default.

8.7 Example 6: Bridge Into Observable Space (Koopman Export, EY30)

Goal. Move from a state/operator regime into an observable regime where a Koopman operator acts:

$$g_1 \xleftarrow{\text{pullback}} g_2, \quad \mathcal{K}_2(g_2) = g_2 \circ \Phi,$$

and bind the relationship between state evolution and observable evolution.

What must be declared.

- whether the bridge is pullback-only,
- how observables are represented (`RegimeSpec2.SpaceType`),
- what preservation predicates are meaningful (often P3 and possibly P4),
- and which claims are non-claims (e.g. state positivity may not translate to observable space).

8.8 Chapter Summary

This non-normative chapter illustrated EY28 in common Vol. No. IV workflows: transporting Hamiltonians for filtering (EY23), transporting validity penalties and positivity claims (EY24), bridging Liouvillian evolution by generator vs trajectory transport (EY25), transporting token step sets (EY26), transporting commutation constraints via object transport + re-check (EY27), and bridging into observable space for Koopman export (EY30).

Next. Proceed to EY29: Coarse-Grain Reducer R , which makes explicit how reductions weaken or preserve invariants and how bucketed, verifier-friendly summaries are produced from high-resolution operator artifacts.

EY29: Coarse-Grain Reducer R

Canonical Operator Artifacts for QFC

Enter Yourname

Abstract

This whitepaper defines the Coarse-Grain Reducer R , a canonical artifact that reduces high-resolution operator objects into verifier-friendly summaries and lower-resolution representations while preserving (or explicitly weakening) declared invariants. EY29 standardizes reduction semantics: which degrees of freedom are eliminated, how truncation levels are mapped, what invariants are preserved, and how loss is measured and certified.

R is the bridge from spectral probing to machine-checkable buckets: it converts outputs of canonical spectral filters (EY23), validity penalties (EY24), audit-native evolutions (EY25), and regime transport (EY28) into reduced operators, reduced certificates, and coarse observables usable for classification, compatibility matrices, and Koopman export (EY30). Every reduction claim in Vol. No. IV must be replayable under a declared NumericModel, with deterministic failure codes and interop certificates.

1 Motivation and Scope: From High-Resolution Operators to Verifier-Friendly Summaries

1.1 Motivation: Why Coarse-Grain Must Be a First-Class Artifact

Vol. No. IV introduces operator artifacts intended to be composed and verified: filters (EY23), validity penalties (EY24), audit-native evolution (EY25), tokens (EY26), commutation constraints (EY27), and regime bridges (EY28). In practice, however, these objects are often too large or too detailed to:

- transmit widely,
- replay quickly by many verifiers,
- compare across runs and machines,
- or use as stable classification labels for registries and compatibility matrices.

Therefore, workflows require *reduction*: turning a detailed operator object into a smaller representation or summary. But reduction is also the primary place where meaning gets smuggled:

- hidden thresholds,
- unlogged truncations,
- heuristic feature extraction,

- and “it looks the same” reasoning.

EY29 exists to prevent that. It makes coarse-graining an auditable artifact with a SpecID, deterministic routines, and certificates.

1.2 What EY29 Defines

EY29 defines:

- a canonical reducer R that maps detailed objects to reduced objects,
- reduction semantics classes (projection, aggregation, bucketing, sketching),
- truncation mapping policies (how N changes),
- invariant preservation/weakening rules (what remains true, what becomes approximate, what is discarded),
- loss metrics that are explicitly computed and committed,
- a certificate/verifier contract for deterministic replay,
- interop declarations with other artifacts (especially EY23, EY24, EY28, EY30).

1.3 Scope

In-scope.

- Reduction of operators: $A \mapsto \tilde{A} = R(A)$.
- Reduction of state objects and summaries (when explicitly declared).
- Reduction of certificates: producing reduced certificates or summary commitments.
- Declaring and checking which invariants are preserved vs weakened.

Out-of-scope.

- “Black-box compression” without deterministic reconstruction rules.
- Learned embeddings without committed parameters and deterministic replay.
- Claims of infinite-dimensional equivalence inferred solely from reduced summaries.

1.4 Intuition: Reduction as a Typed Map with a Loss Budget

A reducer is not merely a function; it is a *typed map* with declared semantics:

$$R : \mathcal{X} \rightarrow \tilde{\mathcal{X}},$$

where $\mathcal{X}$ is an object class (operators, states, spectra, transcripts) and $\tilde{\mathcal{X}}$ is a smaller class intended for verifier-friendly handling.

Reduction must bind:

- **what is removed** (degrees of freedom),
- **what is kept** (invariants and summaries),
- **how much is lost** (loss metrics),
- **how to reproduce it** (deterministic routines).

1.5 Why EY29 Is Central to “Buckets” and Registries

Vol. No. IV intends to support a registry and compatibility matrix (EY22): to compare artifacts across runs and to classify them into stable spectral classes.

But classification requires stable signatures. High-resolution objects are too sensitive to:

- truncation edges,
- floating point drift,
- minor parameter changes,
- ordering differences.

A reducer supplies a canonical, auditable *signature layer*:

$$\text{Sig}(A) := \text{Enc}(R(A)),$$

where Enc is canonical encoding/hashing bound to NumericModel.

Rule. If a workflow uses a reduced signature as a registry key, it MUST cite EY29 SpecID and include loss metrics and tolerance policies so the key is meaningful and replayable.

1.6 Design Goals (Normative)

G1: Deterministic Reduction

Given the same inputs, SpecID, truncation policy, and NumericModel, every verifier MUST compute the same reduced object.

G2: Explicit Loss Accounting

Every reduction MUST produce explicit loss metrics and commit them in the certificate. If a loss metric is “not applicable”, it must be explicitly **null**.

G3: Declared Invariant Preservation

Invariants are preserved only if explicitly declared and checked. Otherwise, no preservation claim is implied.

G4: Compatibility with Vol. No. IV Workflows

EY29 must compose with:

- EY23 filters (reducing filtered outputs),
- EY24 penalties (reducing validity objects or their summaries),
- EY25 evolutions (reducing snapshots/summary observables),
- EY26 tokens (reducing step sets or token-derived summaries),
- EY27 constraints (reducing objects while tracking commutation loss),
- EY28 bridges (treating transport as reduction when N shrinks),
- EY30 Koopman export (reducing to observable-space summaries).

1.7 Chapter Summary

This chapter motivated EY29: coarse-graining is required for verifier-friendly operation, but it is also the primary place where meaning is smuggled. EY29 makes reduction an auditable artifact. It defined scope, intuition (typed map + loss budget), the role of reducers in registries and buckets, and the normative design goals: deterministic reduction, explicit loss accounting, declared preservation, and composability across Vol. No. IV.

Chapter ?? next defines reduction semantics classes, truncation mapping policies, and the invariant vocabulary used to express preservation and weakening under coarse-graining.

2 Motivation and Scope: From High-Resolution Operators to Verifier-Friendly Summaries

2.1 Motivation: Why Coarse-Grain Must Be a First-Class Artifact

Vol. No. IV introduces operator artifacts intended to be composed and verified: filters (EY23), validity penalties (EY24), audit-native evolution (EY25), tokens (EY26), commutation constraints (EY27), and regime bridges (EY28). In practice, however, these objects are often too large or too detailed to:

- transmit widely,
- replay quickly by many verifiers,
- compare across runs and machines,
- or use as stable classification labels for registries and compatibility matrices.

Therefore, workflows require *reduction*: turning a detailed operator object into a smaller representation or summary. But reduction is also the primary place where meaning gets smuggled:

- hidden thresholds,
- unlogged truncations,
- heuristic feature extraction,
- and “it looks the same” reasoning.

EY29 exists to prevent that. It makes coarse-graining an auditable artifact with a SpecID, deterministic routines, and certificates.

2.2 What EY29 Defines

EY29 defines:

- a canonical reducer R that maps detailed objects to reduced objects,
- reduction semantics classes (projection, aggregation, bucketing, sketching),
- truncation mapping policies (how N changes),
- invariant preservation/weakening rules (what remains true, what becomes approximate, what is discarded),
- loss metrics that are explicitly computed and committed,

- a certificate/verifier contract for deterministic replay,
- interop declarations with other artifacts (especially EY23, EY24, EY28, EY30).

2.3 Scope

In-scope.

- Reduction of operators: $A \mapsto \tilde{A} = R(A)$.
- Reduction of state objects and summaries (when explicitly declared).
- Reduction of certificates: producing reduced certificates or summary commitments.
- Declaring and checking which invariants are preserved vs weakened.

Out-of-scope.

- “Black-box compression” without deterministic reconstruction rules.
- Learned embeddings without committed parameters and deterministic replay.
- Claims of infinite-dimensional equivalence inferred solely from reduced summaries.

2.4 Intuition: Reduction as a Typed Map with a Loss Budget

A reducer is not merely a function; it is a *typed map* with declared semantics:

$$R : \mathcal{X} \rightarrow \tilde{\mathcal{X}},$$

where $\mathcal{X}$ is an object class (operators, states, spectra, transcripts) and $\tilde{\mathcal{X}}$ is a smaller class intended for verifier-friendly handling.

Reduction must bind:

- **what is removed** (degrees of freedom),
- **what is kept** (invariants and summaries),
- **how much is lost** (loss metrics),
- **how to reproduce it** (deterministic routines).

2.5 Why EY29 Is Central to “Buckets” and Registries

Vol. No. IV intends to support a registry and compatibility matrix (EY22): to compare artifacts across runs and to classify them into stable spectral classes.

But classification requires stable signatures. High-resolution objects are too sensitive to:

- truncation edges,
- floating point drift,
- minor parameter changes,
- ordering differences.

A reducer supplies a canonical, auditable *signature layer*:

$$\text{Sig}(A) := \text{Enc}(R(A)),$$

where `Enc` is canonical encoding/hashing bound to `NumericModel`.

Rule. If a workflow uses a reduced signature as a registry key, it **MUST** cite EY29 SpecID and include loss metrics and tolerance policies so the key is meaningful and replayable.

2.6 Design Goals (Normative)

G1: Deterministic Reduction

Given the same inputs, SpecID, truncation policy, and `NumericModel`, every verifier **MUST** compute the same reduced object.

G2: Explicit Loss Accounting

Every reduction **MUST** produce explicit loss metrics and commit them in the certificate. If a loss metric is “not applicable”, it must be explicitly `null`.

G3: Declared Invariant Preservation

Invariants are preserved only if explicitly declared and checked. Otherwise, no preservation claim is implied.

G4: Compatibility with Vol. No. IV Workflows

EY29 must compose with:

- EY23 filters (reducing filtered outputs),
- EY24 penalties (reducing validity objects or their summaries),
- EY25 evolutions (reducing snapshots/summary observables),
- EY26 tokens (reducing step sets or token-derived summaries),
- EY27 constraints (reducing objects while tracking commutation loss),
- EY28 bridges (treating transport as reduction when N shrinks),
- EY30 Koopman export (reducing to observable-space summaries).

2.7 Chapter Summary

This chapter motivated EY29: coarse-graining is required for verifier-friendly operation, but it is also the primary place where meaning is smuggled. EY29 makes reduction an auditable artifact. It defined scope, intuition (typed map + loss budget), the role of reducers in registries and buckets, and the normative design goals: deterministic reduction, explicit loss accounting, declared preservation, and composability across Vol. No. IV.

Chapter ?? next defines reduction semantics classes, truncation mapping policies, and the invariant vocabulary used to express preservation and weakening under coarse-graining.

3 Artifact Structure and SpecID Schema

3.1 Purpose

Chapters 2–?? motivated reduction and defined semantics classes, truncation mapping policies, and invariant vocabulary. This chapter defines EY29 as an auditable artifact with a precise SpecID schema:

- how the reducer R is represented (explicit, factored, constructed),
- how partitions/seeds/thresholds/classifiers are committed,
- how invariants and loss metrics are bound to check routines,
- and what transcript/certificate payload is required for deterministic replay.

3.2 Reducer as a Typed Object

EY29 treats a reducer as a typed map:

$$R : \mathcal{X} \rightarrow \tilde{\mathcal{X}},$$

where $\mathcal{X}$ is an input object class and $\tilde{\mathcal{X}}$ is the reduced object class.

InputType and OutputType

Each EY29 instance MUST declare:

$\text{InputType} \in \{\text{OP}, \text{STATE}, \text{CERT}, \text{SUMMARY}\}, \quad \text{OutputType} \in \{\text{OP}, \text{STATE}, \text{CERT}, \text{SUMMARY}, \text{BUCKET}\}$

Rule. If $\text{OutputType} = \text{BUCKET}$, the artifact MUST publish bucket label rules and classifier commitments ($\text{RedClass} = \text{BUCKET}$).

3.3 ReducerSpec

EY29 standardizes a reducer declaration object:

$$\text{ReducerSpec} := (\text{RedClass}, \text{NMap}, \text{ReducerCore}, \text{InvPolicy}, \text{LossSpec}).$$

ReducerCore

ReducerCore encodes the mechanism of reduction and is class-dependent.

Class PROJ (R1).

$$\text{ReducerCore}_{\text{PROJ}} := (\text{ProjType}, \text{ProjEncoding}, \text{ProjRoutine}, \text{ProjChecks})$$

where P is explicit or constructed deterministically, and projector checks include idempotence/self-adjointness tolerances.

Class AGG (R2).

$$\text{ReducerCore}_{\text{AGG}} := (\text{PartitionSpec}, \Phi, \text{OrderSpec})$$

where `PartitionSpec` binds the block partition and Φ binds the aggregation functional and its deterministic evaluation order.

Class BUCK (R3).

$$\text{ReducerCore}_{\text{BUCK}} := (\text{ClassifierSpec}, \text{SummarySpec}, \text{CrossNPolicy})$$

where `ClassifierSpec` commits bucket rules and `SummarySpec` commits a bounded-size summary encoding.

Class SKETCH (R4).

$$\text{ReducerCore}_{\text{SKETCH}} := (\text{PRNGSpec}, \text{Seed}, \text{SketchGen}, \text{SketchForm})$$

where `SketchGen` deterministically generates the sketch operator or sketch vector transform.

Class THR (R5).

$$\text{ReducerCore}_{\text{THR}} := (\tau, \text{ThreshGrid}, \text{TieBreak}, \text{ScanOrder})$$

where thresholds are explicit or quantized grids, and `ScanOrder` fixes entry iteration deterministically.

Rule (null discipline). Any `ReducerCore` field that is not used **MUST** appear as `null` (explicit). Omission is not allowed.

3.4 Invariant Policy Binding

`InvPolicy` is the contract for what the reduction preserves/weakens/discards.

Schema.

$$\text{InvPolicy} := \{(\text{InvName}, \text{Status}, \text{TolSpec}, \text{CheckRoutineID})\}.$$

Rule. If `Status` = `preserved` or `weakened`, then `TolSpec` and `CheckRoutineID` **MUST** be present. If `Status` = `discarded`, both **MUST** be `null`.

3.5 LossSpec

`LossSpec` binds loss metrics to explicit deterministic routines.

Schema.

$$\text{LossSpec} := \{(\text{LossName}, \text{ValueType}, \text{TolSpec}, \text{RoutineID})\}.$$

Rule. At least one loss metric **MUST** be declared (Chapter ??). If a loss metric is not applicable, it must be explicitly `null` and justified by `RedClass`.

3.6 NumericModel and Truncation Binding

EY29 is only meaningful under declared numeric/truncation semantics.

TruncSpec

$$\text{TruncSpec} := (\text{Basis}, \text{CutRule}, \text{BoundaryRule}, N_{\text{in}}, N_{\text{out}}, \text{CrossNPolicy}).$$

Rule. If NMap is same- N , then $N_{\text{in}} = N_{\text{out}}$. If NMap is shrink- N , the mapping rule and rounding MUST be declared.

NumericModel

NumericModel MUST bind:

- arithmetic type (float/fixed/exact),
- rounding/precision,
- deterministic matmul rules (order and accumulation),
- canonicalization rules and points.

3.7 TranscriptSpec and Commitments

Reduction must be replayable. Therefore, transcript commitments MUST bind:

- reducer core parameters (projector/partition/classifier/seed/threshold),
- truncation mapping decisions (especially for multi- N),
- reduced output object $\tilde{A}$ (or its hash),
- all invariant check results and loss metric values,
- deterministic scan orders and tie-breaks used.

Rule. If the reduced output is used as a registry key or bucket label, the transcript MUST bind enough data to recompute it exactly.

3.8 SpecID for EY29

ArtifactType. For EY29:

$$\text{ArtifactType} = \text{CGR}.$$

SpecID components (EY29 binding). The full SpecID tuple is:

$$\text{SpecID}_{\text{EY29}} := (\text{CGR}, \text{InputType}, \text{OutputType}, \text{ReducerSpec}, \text{TruncSpec}, \text{NumericModel}, \text{TranscriptSpec}, \text{CGR})$$

Interop (Mini)

Interop binds intended compositions:

- references to upstream artifacts whose outputs are reduced (EY23/EY24/EY25/EY26/EY28/EY27),
- invariant obligations inherited from those artifacts (e.g. validity, commutation),
- and compatibility predicates for workflows.

Rule. If EY29 reduces an object whose meaning depends on EY27 or EY24, InvPolicy MUST include **I6 Commutation** and/or **I7 Validity** as preserved/weakened (not discarded), unless the workflow explicitly declares it does not depend on those invariants.

3.9 Deterministic Failure Codes (Schema Layer)

At minimum:

- **REJECT_MALFORMED**
- **REJECT_SPECID_MISSING**
- **REJECT_REDCCLASS_MISSING**
- **REJECT_NMAP_MISSING**
- **REJECT_INVPOLICY_MISSING**
- **REJECT_HIDDEN_INPUT**
- **FAIL_HASH_MISMATCH**
- **FAIL_NONDETERMINISM**
- **FAIL_TRUNC_MISMATCH**
- **FAIL_DOMAIN_MISMATCH**
- **FAIL_INVARIANT_NOT_PRESERVED**
- **FAIL_LOSS_UNDEFINED**

3.10 Chapter Summary

This chapter defined the EY29 artifact schema: InputType/OutputType, a ReducerSpec binding reduction class, truncation mapping, reducer core parameters, invariant policy, and loss specification; plus truncation and NumericModel binding, transcript commitments, and an interop mini layer. It fixed the full SpecID tuple for EY29 and required schema-layer failure codes.

Chapter 4 next defines baseline deterministic routines (CGR-0) for each reduction class and specifies how reducers compute reduced outputs, loss metrics, and invariant checks in a verifier-replayable way.

4 Baseline Deterministic Routines (CGR-0)

4.1 Purpose

A reducer is only useful if verifiers can replay it. This chapter defines:

- a baseline routine set for computing reduced objects $\tilde{A} = R(A)$,
- deterministic harnesses for invariant checks and loss metrics,
- and a conservative reference profile **CGR-0**.

Rule. Every EY29 instance MUST declare the RoutineID used for reduction, invariant checks, and loss metrics (either from this baseline set or via a committed equivalent). Hidden routines are forbidden.

4.2 Routine Set Overview

EY29 defines baseline routines $\text{RoutineID} \in \mathcal{R}_0^{\text{CGR}}$:

- CGR-R0: canonical input preparation and validation
- CGR-R1: PROJ reduction
- CGR-R2: AGG reduction
- CGR-R3: BUCK reduction
- CGR-R4: SKETCH reduction
- CGR-R5: THR reduction
- CGR-R6: invariant check harness
- CGR-R7: loss metric harness
- CGR-R8: multi- N aggregation (CrossNPolicy harness)

4.3 CGR-R0: Canonical Input Preparation

Inputs. A (operator/state/cert/summary) with declared encoding, plus SpecID, TruncSpec, NumericModel.

Steps.

1. Parse and validate InputType/encoding.
2. Canonicalize A under NumericModel (encoding normalization + declared canon points).
3. Validate domain/type constraints (e.g. square matrix for OP).
4. Commit input hash h_A .

Failure. **FAIL__DOMAIN__MISMATCH** if type constraints fail; **REJECT__HIDDEN__INPUT** if encoding is not bound.

4.4 CGR-R1: PROJ Reduction (Projection/Restriction)

Inputs. Prepared A , projector P (explicit or constructed), projector checks/tolerances.

Steps.

1. Obtain P by inline encoding, hash resolution, or deterministic construction recipe.
2. Validate projector properties (deterministic):

$$\|P^2 - P\|_F \leq \varepsilon_P, \quad \|P - P^\dagger\|_F \leq \varepsilon_P.$$

3. Compute $\tilde{A} := PAP$ with fixed MulOrder/ReduceOrder.
4. Canonicalize $\tilde{A}$.
5. Commit $h_{\tilde{A}}$ and intermediate hashes as declared.

Failure. **FAIL_PROJECTOR_INVALID** if projector checks fail; **FAIL_NONDETERMINISM** if routines are underspecified.

4.5 CGR-R2: AGG Reduction (Block Aggregation)

Inputs. Prepared A , PartitionSpec $\{B_i\}$, aggregation functional Φ , deterministic OrderSpec.

Steps.

1. Resolve PartitionSpec deterministically (explicit list, hash, or seed+generator).
2. For each output block pair (i, j) in deterministic scan order: compute the multiset $\{A_{pq}\}$ for $p \in B_i, q \in B_j$, then compute $\tilde{A}_{ij} := \Phi(\cdot)$ using a fixed reduction order.
3. Canonicalize $\tilde{A}$ and commit hash.

Failure. **FAIL_PARTITION_INVALID** if blocks overlap/omit indices beyond declared policy.

4.6 CGR-R3: BUCK Reduction (Bucket + Summary)

Inputs. Prepared A , ClassifierSpec, SummarySpec, truncation policy (single- N or multi- N).

Steps (single- N).

1. Compute a deterministic feature vector $f(A)$ (must be defined in ClassifierSpec).
2. Evaluate deterministic classifier C to produce a discrete label:

$$\text{BucketID} := C(f(A)).$$

3. Compute bounded summary $s(A)$ as in SummarySpec (e.g. top- k spectral stats, bounded moments).
4. Output $R(A) := (\text{BucketID}, s(A))$ and commit by canonical encoding.

Steps (multi- N). For $\mathcal{N}_{\text{in}} = \{N^{(1)}, \dots, N^{(K)}\}$, repeat the above per $N^{(k)}$ and invoke CGR-R8 CrossNPolicy.

Failure. **FAIL_CLASSIFIER_UNDEFINED** if classifier rules are not fully committed.

4.7 CGR-R4: SKETCH Reduction (Deterministic Pseudorandom Sketch)

Inputs. Prepared A , PRNGSpec, Seed, SketchGen routine, SketchForm.

Steps.

1. Generate sketch operator/transform S deterministically from (PRNGSpec, Seed).
2. Compute sketch output (one of):

$$\tilde{A} := SAS^\dagger \quad \text{or} \quad \tilde{v} := S \cdot \text{vec}(A).$$

3. Canonicalize output and commit hash.

Failure. **FAIL_NONDETERMINISM** if PRNGSpec or generator is underspecified.

4.8 CGR-R5: THR Reduction (Thresholding/Sparsification)

Inputs. Prepared A , threshold τ (or grid), ScanOrder, TieBreak.

Steps.

1. Fix τ deterministically (explicit constant or quantized grid index).
2. For each entry in ScanOrder: apply threshold rule; on equality, apply TieBreak policy deterministically.
3. Canonicalize $\tilde{A}$, commit hash, and record sparsity statistics.

Failure. **FAIL_LOSS_UNDEFINED** if the reduction does not compute $\|A - \tilde{A}\|$ (unless explicitly `null` and justified).

4.9 CGR-R6: Invariant Check Harness

Inputs. InvPolicy list, input A , reduced output $\tilde{A}$, and any referenced artifacts (EY27 PredID, EY24 validity, etc.).

Steps. For each (InvName, Status, TolSpec, CheckRoutineID):

1. If Status=`discarded`: record `null` and continue.
2. Else run CheckRoutineID deterministically and record: (InvName, Result, Value, Tol).

Outputs. Invariant report committed by canonical encoding/hashing.

4.10 CGR-R7: Loss Metric Harness

Inputs. LossSpec list, A , $\tilde{A}$, truncation, NumericModel.

Steps. For each declared loss metric:

1. compute its value deterministically under declared norms/orders,
2. record the value and tolerance (if applicable),
3. commit the loss report.

Outputs. Loss report committed by canonical encoding/hashing.

4.11 CGR-R8: Multi- N CrossNPolicy Harness

Inputs. Per- N outputs $\{R_{N(k)}(A)\}$ and declared CrossNPolicy.

Steps.

1. Apply CrossNPolicy deterministically to aggregate: BucketID and/or summaries across N .
2. Record stabilization diagnostics (e.g. first k where label stabilizes) if declared.

Outputs. Aggregated reduced object and CrossN transcript commitments.

4.12 CGR-0 Reference Profile (Baseline)

CGR-0 is the conservative baseline profile for EY29.

CGR-0 Fixes

CGR-0 fixes:

- RedClass $\in \{\text{PROJ}, \text{AGG}, \text{BUCK}\}$ only,
- NMap $\in \{\text{same-N}, \text{shrink-N}\}$ (multi- N is non-baseline),
- no SKETCH unless explicitly seeded and committed (recommended non-baseline),
- no THR unless thresholds are fixed constants (recommended non-baseline),
- mandatory loss metrics: $\|A - \tilde{A}\|_F$ for PROJ/AGG/THR; “margin/stability” for BUCK,
- invariant vocabulary must include I1 (Domain) and I8 (Replay), and any workflow-dependent invariants (I6/I7) cannot be discarded.

Rule. If a reducer uses SKETCH or THR with a grid of thresholds, it MUST declare a non-baseline profile and publish full determinism rules.

4.13 Routine-Layer Failure Codes

At minimum:

- **FAIL_PROJECTOR_INVALID**
- **FAIL_PARTITION_INVALID**
- **FAIL_CLASSIFIER_UNDEFINED**

- **FAIL_NONDETERMINISM**
- **FAIL_DOMAIN_MISMATCH**
- **FAIL_TRUNC_MISMATCH**
- **FAIL_INVARIANT_NOT_PRESERVED**
- **FAIL_LOSS_UNDEFINED**

4.14 Chapter Summary

This chapter defined baseline deterministic routines (CGR-R0..R8) for computing reduced objects under each reduction class, evaluating invariant policies, computing loss metrics, and aggregating across truncations with CrossNPolicy. It also defined the conservative CGR-0 reference profile: limited reduction classes, restricted N -mapping, and mandatory loss/invariant accounting.

Chapter 6 next specifies certificate payload and verifier checks for EY29: how reduced outputs, invariant reports, and loss reports are committed and replayed, and how reducers are accepted or rejected deterministically.

5 Certificate and Verifier: Reduction Replay, Loss Accounting, and Invariant Acceptance

5.1 Purpose

EY29 reductions are used as registry keys, bucket labels, and verifier-friendly summaries. Therefore, a reducer must publish:

- the full reducer SpecID (including ReducerCore parameters),
- deterministic transcripts that allow replay,
- invariant check results (per InvPolicy),
- loss metrics (per LossSpec),
- and failure codes when reduction cannot be accepted.

This chapter specifies:

- the certificate payload π_{CGR} ,
- deterministic verifier checks for acceptance,
- transcript commitments for reduction computations,
- and required failure codes.

5.2 Certificate Object π_{CGR}

Rule. An EY29 publication MUST include a certificate π_{CGR} in canonical certificate encoding (EY22-CCE).

Header (required)

- artifact type: `CGR`
- artifact symbol: R
- `RedClass`, `NMap`, `InputType`, `OutputType`
- `Version`
- Certificate schema identifier (EY22-CCE)

SpecID (required)

The full SpecID tuple MUST be embedded:

$\text{SpecID}_{\text{EY29}} := (\text{CGR}, \text{InputType}, \text{OutputType}, \text{ReducerSpec}, \text{TruncSpec}, \text{NumericModel}, \text{TranscriptSpec}, \text{C})$

Committed Inputs (required)

π_{CGR} MUST commit to:

- `ReducerSpec` including `ReducerCore` (projector/partition/classifier/seed/threshold),
- `TruncSpec` including N_{in} , N_{out} and any `CrossNPolicy` rules,
- `NumericModel` (including ordering/canonicalization),
- `InvPolicy` (names, statuses, tolerances, check routines),
- `LossSpec` (definitions, routines, tolerances or explicit `null`),
- any external referenced objects used by invariant checks (e.g. EY27 PredID, EY24 validity, EY28 bridge transcripts),
- input object commitment h_A (or a reference to the upstream artifact output hash).

Committed Outputs (required)

π_{CGR} MUST commit to:

- reduced output object $\tilde{A}$ (or its hash) and canonical encoding rule,
- reduction transcript commitment Tx_{cgr} ,
- invariant report commitment Tx_{inv} ,
- loss report commitment Tx_{loss} ,
- deterministic failure code if reduction or checks fail.

5.3 Reduction Transcript (Minimum Contents)

The reduction transcript MUST bind enough information to replay:

- input hash h_A and any upstream references (artifact IDs, output hashes),
- reducer core resolution (projector/partition/classifier/seed/threshold) and their hashes,
- truncation mapping decisions (including rounding/tie-breaks and `CrossNPolicy` outputs),

- deterministic scan orders (block order, entry order, reduction order),
- canonicalization points and rules,
- reduced output hash $h_{\tilde{A}}$ (or hash of bucket+summary encoding).

Rule (registry-grade output). If $\tilde{A}$ or BucketID is used as a registry key, π_{CGR} MUST include:

- a stable canonical encoding of the key material,
- and the exact CrossNPolicy (if any) that produced it.

5.4 Invariant Report (Required Structure)

An invariant report is the canonical serialization of:

$$\text{InvReport} := \{(\text{InvName}, \text{Status}, \text{Result}, \text{Value}, \text{Tol})\}.$$

Rule. For $\text{Status} = \text{discarded}$, Result , Value , Tol MUST be `null`. For preserved/weakened invariants, they MUST be non-null.

5.5 Loss Report (Required Structure)

A loss report is the canonical serialization of:

$$\text{LossReport} := \{(\text{LossName}, \text{Value}, \text{Tol})\}.$$

Rule. If a loss metric is declared but not computed, Value MUST be `null`, and the verifier must check that the SpecID justification exists. Otherwise: **FAIL_LOSS_UNDEFINED**.

5.6 Verifier Contract

A verifier consumes π_{CGR} and returns:

$$\{\text{ACCEPT}, \text{REJECT}\} \cup \{\text{FAIL_}\ast\}.$$

Acceptance meaning. ACCEPT means: the reducer artifact is well-formed, replayable, the reduced output matches replay, loss metrics are computed as declared, and invariants marked preserved/weakened satisfy their check rules within tolerance. It does not imply any invariant marked discarded, and it does not imply preservation beyond InvPolicy.

5.7 Required Verifier Checks (Normative)

V1: Structural Validity (EY22-CCE)

Parse canonical certificate encoding and confirm required fields exist. Failure: **REJECT_MALFORMED**.

V2: SpecID Completeness

Confirm:

- ArtifactType=CGR,
- InputType, OutputType,
- ReducerSpec (RedClass, NMap, ReducerCore, InvPolicy, LossSpec),
- TruncSpec, NumericModel,
- TranscriptSpec, Cert, Interop, Version.

Failure: **REJECT_SPECID_MISSING**, **REJECT_REDCLASS_MISSING**, **REJECT_NMAP_MISSING**, **REJECT_INVPOLICY_MISSING**.

V3: Commitment Integrity

Verify all hash-committed objects resolve and match:

- reducer core components (projector/partition/classifier/seed/threshold),
- input commitment h_A (or upstream output hash),
- any referenced external artifacts used by invariant checks (PredIDs, validity objects),
- transcript commitments.

Failure: **FAIL_HASH_MISMATCH**, **REJECT_HIDDEN_INPUT**.

V4: Truncation and NMap Enforcement

Enforce NMap and TruncSpec:

- same- N : check $N_{\text{in}} = N_{\text{out}}$,
- shrink- N : recompute declared mapping and rounding,
- multi- N : verify lists and CrossNPolicy selection (if used).

Failure: **FAIL_TRUNC_MISMATCH**.

V5: Reduction Replay

Replay reduction deterministically:

- run CGR-R0 preparation,
- run the declared reduction routine for RedClass (CGR-R1..R5) and CGR-R8 if multi- N ,
- apply declared scan orders and canonicalization points.

Compare the replayed reduced output hash to published $h_{\tilde{A}}$ (or bucket+summary encoding hash). Failure: **FAIL_TRANSCRIPT_MISMATCH**, **FAIL_NONDETERMINISM**, **FAIL_DOMAIN_MISMATCH**.

V6: Loss Metric Replay

Replay all loss metrics in LossSpec via CGR-R7. Compare to published LossReport. Failure: **FAIL_LOSS_UNDEFINED, FAIL_TRANSCRIPT_MISMATCH**.

V7: Invariant Check Replay

Replay all preserved/weakened invariants via CGR-R6, including any referenced EY27/EY24/EY28 routines. Compare to published InvReport. Failure: **FAIL_INVARIANT_NOT_PRESERVED, FAIL_COMPAT_PREDICATE, FAIL_TRANSCRIPT_MISMATCH**.

V8: Interop Claim Consistency

If Interop claims composition with other artifacts:

- verify referenced artifacts (by hash/SpecID) exist and match,
- verify that invariants required by those workflows are not marked discarded in InvPolicy,
- verify that BUCK outputs used as keys include CrossNPolicy and stability diagnostics if declared.

Failure: **FAIL_COMPAT_PREDICATE**.

5.8 Acceptance Rule

Rule. A verifier returns ACCEPT iff:

- V1–V5 pass,
- LossReport matches replayed loss metrics for all declared metrics,
- InvReport matches replayed invariant checks for all preserved/weakened invariants,
- transcript commitments match.

If an invariant marked preserved fails, return **FAIL_INVARIANT_NOT_PRESERVED** (not ACCEPT).

5.9 Required Failure Codes

At minimum:

- **REJECT_MALFORMED**
- **REJECT_SPECID_MISSING**
- **REJECT_REDCONTEXT_MISSING**
- **REJECT_NMAP_MISSING**
- **REJECT_INVPOLICY_MISSING**
- **REJECT_HIDDEN_INPUT**
- **FAIL_HASH_MISMATCH**
- **FAIL_TRUNC_MISMATCH**

- **FAIL_DOMAIN_MISMATCH**
- **FAIL_NONDETERMINISM**
- **FAIL_TRANSCRIPT_MISMATCH**
- **FAIL_LOSS_UNDEFINED**
- **FAIL_INVARIANT_NOT_PRESERVED**
- **FAIL_COMPAT_PREDICATE**

5.10 Chapter Summary

This chapter defined the EY29 certificate π_{CGR} and deterministic verifier checks: SpecID completeness, commitment integrity, truncation enforcement, reduction replay, loss metric replay, invariant check replay, and interop claim consistency. It fixed acceptance rules and required failure codes, making reducers replayable and registry-grade.

Chapter ?? next provides interoperability declarations (mini) for EY29: how reductions compose with EY23–EY30, how compatibility matrices should represent reduction steps, and how to publish interop certificates for bucketed workflows.

6 Certificate and Verifier: Reduction Replay, Loss Accounting, and Invariant Acceptance

6.1 Purpose

EY29 reductions are used as registry keys, bucket labels, and verifier-friendly summaries. Therefore, a reducer must publish:

- the full reducer SpecID (including ReducerCore parameters),
- deterministic transcripts that allow replay,
- invariant check results (per InvPolicy),
- loss metrics (per LossSpec),
- and failure codes when reduction cannot be accepted.

This chapter specifies:

- the certificate payload π_{CGR} ,
- deterministic verifier checks for acceptance,
- transcript commitments for reduction computations,
- and required failure codes.

6.2 Certificate Object π_{CGR}

Rule. An EY29 publication MUST include a certificate π_{CGR} in canonical certificate encoding (EY22-CCE).

Header (required)

- artifact type: `CGR`
- artifact symbol: R
- `RedClass`, `NMap`, `InputType`, `OutputType`
- `Version`
- Certificate schema identifier (EY22-CCE)

SpecID (required)

The full SpecID tuple MUST be embedded:

$\text{SpecID}_{\text{EY29}} := (\text{CGR}, \text{InputType}, \text{OutputType}, \text{ReducerSpec}, \text{TruncSpec}, \text{NumericModel}, \text{TranscriptSpec}, \text{C})$

Committed Inputs (required)

π_{CGR} MUST commit to:

- `ReducerSpec` including `ReducerCore` (projector/partition/classifier/seed/threshold),
- `TruncSpec` including N_{in} , N_{out} and any `CrossNPolicy` rules,
- `NumericModel` (including ordering/canonicalization),
- `InvPolicy` (names, statuses, tolerances, check routines),
- `LossSpec` (definitions, routines, tolerances or explicit `null`),
- any external referenced objects used by invariant checks (e.g. EY27 PredID, EY24 validity, EY28 bridge transcripts),
- input object commitment h_A (or a reference to the upstream artifact output hash).

Committed Outputs (required)

π_{CGR} MUST commit to:

- reduced output object $\tilde{A}$ (or its hash) and canonical encoding rule,
- reduction transcript commitment Tx_{cgr} ,
- invariant report commitment Tx_{inv} ,
- loss report commitment Tx_{loss} ,
- deterministic failure code if reduction or checks fail.

6.3 Reduction Transcript (Minimum Contents)

The reduction transcript MUST bind enough information to replay:

- input hash h_A and any upstream references (artifact IDs, output hashes),
- reducer core resolution (projector/partition/classifier/seed/threshold) and their hashes,
- truncation mapping decisions (including rounding/tie-breaks and `CrossNPolicy` outputs),

- deterministic scan orders (block order, entry order, reduction order),
- canonicalization points and rules,
- reduced output hash $h_{\tilde{A}}$ (or hash of bucket+summary encoding).

Rule (registry-grade output). If $\tilde{A}$ or BucketID is used as a registry key, π_{CGR} MUST include:

- a stable canonical encoding of the key material,
- and the exact CrossNPolicy (if any) that produced it.

6.4 Invariant Report (Required Structure)

An invariant report is the canonical serialization of:

$$\text{InvReport} := \{(\text{InvName}, \text{Status}, \text{Result}, \text{Value}, \text{Tol})\}.$$

Rule. For $\text{Status} = \text{discarded}$, Result , Value , Tol MUST be `null`. For preserved/weakened invariants, they MUST be non-null.

6.5 Loss Report (Required Structure)

A loss report is the canonical serialization of:

$$\text{LossReport} := \{(\text{LossName}, \text{Value}, \text{Tol})\}.$$

Rule. If a loss metric is declared but not computed, Value MUST be `null`, and the verifier must check that the SpecID justification exists. Otherwise: **FAIL_LOSS_UNDEFINED**.

6.6 Verifier Contract

A verifier consumes π_{CGR} and returns:

$$\{\text{ACCEPT}, \text{REJECT}\} \cup \{\text{FAIL_}\ast\}.$$

Acceptance meaning. ACCEPT means: the reducer artifact is well-formed, replayable, the reduced output matches replay, loss metrics are computed as declared, and invariants marked preserved/weakened satisfy their check rules within tolerance. It does not imply any invariant marked discarded, and it does not imply preservation beyond InvPolicy.

6.7 Required Verifier Checks (Normative)

V1: Structural Validity (EY22-CCE)

Parse canonical certificate encoding and confirm required fields exist. Failure: **REJECT_MALFORMED**.

V2: SpecID Completeness

Confirm:

- ArtifactType=CGR,
- InputType, OutputType,
- ReducerSpec (RedClass, NMap, ReducerCore, InvPolicy, LossSpec),
- TruncSpec, NumericModel,
- TranscriptSpec, Cert, Interop, Version.

Failure: **REJECT_SPECID_MISSING**, **REJECT_REDCCLASS_MISSING**, **REJECT_NMAP_MISSING**, **REJECT_INVPOLICY_MISSING**.

V3: Commitment Integrity

Verify all hash-committed objects resolve and match:

- reducer core components (projector/partition/classifier/seed/threshold),
- input commitment h_A (or upstream output hash),
- any referenced external artifacts used by invariant checks (PredIDs, validity objects),
- transcript commitments.

Failure: **FAIL_HASH_MISMATCH**, **REJECT_HIDDEN_INPUT**.

V4: Truncation and NMap Enforcement

Enforce NMap and TruncSpec:

- same- N : check $N_{\text{in}} = N_{\text{out}}$,
- shrink- N : recompute declared mapping and rounding,
- multi- N : verify lists and CrossNPolicy selection (if used).

Failure: **FAIL_TRUNC_MISMATCH**.

V5: Reduction Replay

Replay reduction deterministically:

- run CGR-R0 preparation,
- run the declared reduction routine for RedClass (CGR-R1..R5) and CGR-R8 if multi- N ,
- apply declared scan orders and canonicalization points.

Compare the replayed reduced output hash to published $h_{\tilde{A}}$ (or bucket+summary encoding hash). Failure: **FAIL_TRANSCRIPT_MISMATCH**, **FAIL_NONDETERMINISM**, **FAIL_DOMAIN_MISMATCH**.

V6: Loss Metric Replay

Replay all loss metrics in LossSpec via CGR-R7. Compare to published LossReport. Failure: **FAIL_LOSS_UNDEFINED, FAIL_TRANSCRIPT_MISMATCH**.

V7: Invariant Check Replay

Replay all preserved/weakened invariants via CGR-R6, including any referenced EY27/EY24/EY28 routines. Compare to published InvReport. Failure: **FAIL_INVARIANT_NOT_PRESERVED, FAIL_COMPAT_PREDICATE, FAIL_TRANSCRIPT_MISMATCH**.

V8: Interop Claim Consistency

If Interop claims composition with other artifacts:

- verify referenced artifacts (by hash/SpecID) exist and match,
- verify that invariants required by those workflows are not marked discarded in InvPolicy,
- verify that BUCK outputs used as keys include CrossNPolicy and stability diagnostics if declared.

Failure: **FAIL_COMPAT_PREDICATE**.

6.8 Acceptance Rule

Rule. A verifier returns ACCEPT iff:

- V1–V5 pass,
- LossReport matches replayed loss metrics for all declared metrics,
- InvReport matches replayed invariant checks for all preserved/weakened invariants,
- transcript commitments match.

If an invariant marked preserved fails, return **FAIL_INVARIANT_NOT_PRESERVED** (not ACCEPT).

6.9 Required Failure Codes

At minimum:

- **REJECT_MALFORMED**
- **REJECT_SPECID_MISSING**
- **REJECT_REDCONTEXT_MISSING**
- **REJECT_NMAP_MISSING**
- **REJECT_INVPOLICY_MISSING**
- **REJECT_HIDDEN_INPUT**
- **FAIL_HASH_MISMATCH**
- **FAIL_TRUNC_MISMATCH**

- **FAIL_DOMAIN_MISMATCH**
- **FAIL_NONDETERMINISM**
- **FAIL_TRANSCRIPT_MISMATCH**
- **FAIL_LOSS_UNDEFINED**
- **FAIL_INVARIANT_NOT_PRESERVED**
- **FAIL_COMPAT_PREDICATE**

6.10 Chapter Summary

This chapter defined the EY29 certificate π_{CGR} and deterministic verifier checks: SpecID completeness, commitment integrity, truncation enforcement, reduction replay, loss metric replay, invariant check replay, and interop claim consistency. It fixed acceptance rules and required failure codes, making reducers replayable and registry-grade.

Chapter ?? next provides interoperability declarations (mini) for EY29: how reductions compose with EY23–EY30, how compatibility matrices should represent reduction steps, and how to publish interop certificates for bucketed workflows.

7 CGR-0 Reference Instance, Publication Boundary, and Non-Claims

7.1 Purpose

This chapter closes EY29 by:

- providing a concrete CGR-0 reference SpecID template (a stable meaning of “coarse-grain reducer artifact”),
- fixing what must be published for a valid EY29 reducer,
- stating explicit non-claims (what reduction does *not* guarantee),
- and recording licensing intent consistent with Vol. No. IV.

7.2 CGR-0 Reference Instance (Template)

CGR-0 is the conservative baseline profile defined in Chapter 4. The following template fixes a “default” reducer SpecID for universal verifiers.

CGR-0 profile choices.

- $\text{RedClass} \in \{\text{PROJ}, \text{AGG}, \text{BUCK}\}$,
- $\text{NMap} \in \{\text{same-N}, \text{shrink-N}\}$ (multi- N is non-baseline),
- deterministic reducer core commitments (projector/partition/classifier),
- mandatory loss metrics: $\|A - \tilde{A}\|_F$ for PROJ/AGG, and stability/margin diagnostics for BUCK,
- InvPolicy must include I1 (Domain) and I8 (Replay), and any workflow-dependent invariants (I6/I7) MUST NOT be discarded.

Reference SpecID (CGR-0)

```
SpecID_EY29_CGR0 :=
(
  ArtifactType = CGR,

  InputType = OP,
  OutputType = BUCKET,      # or OP / SUMMARY (depends on use)

  ReducerSpec = {
    RedClass = BUCK,
    NMap      = "shrink-N",

    ReducerCore = {
      ClassifierSpec = {
        FeatureSpec: "spectral-summary-v1",      # committed definition or hash
        ClassRule:   Hash("bucket-rules-v1"),    # committed classifier rule set
        TieBreak:    "lexicographic-first",
        MarginSpec:  "distance-to-boundary-v1"
      },

      SummarySpec = {
        SummaryType: "bounded-vector",
        Fields: ["trA","froA","topk-spec","gap-est"],
        k: 16,
        EncodeRule: "canonical"
      },

      CrossNPolicy = null
    },

    InvPolicy = [
      { InvName:"I1_DOMAIN", Status:"preserved", Tol:0,      Check:"CHK-DOMAIN-v1" },
      { InvName:"I8_REPLAY", Status:"preserved", Tol:0,      Check:"CHK-REPLAY-v1" },
      { InvName:"I2_SPECTRUM",Status:"weakened", Tol:"eps", Check:"CHK-SPECTRAL-STAB-v1" },
      { InvName:"I6_COMM",    Status:"discarded", Tol:null, Check:null }, # only if workflow c
      { InvName:"I7_VALID",   Status:"discarded", Tol:null, Check:null }  # only if workflow c
    ],

    LossSpec = [
      { LossName:"L_bucket_margin", ValueType:"real", Tol:null, Routine:"LOSS-MARGIN-v1" },
      { LossName:"L_stability_N",   ValueType:"real", Tol:null, Routine:"LOSS-STAB-v1" }
    ],

    TruncSpec = {
      Basis: Hash("basis-in"),
      CutRule: "cut-in",
      BoundaryRule: "bnd-in",
      N_in: 4096,
      N_out: 512,
      CrossNPolicy: null
    }
  )
```

```

},

NumericModel = {
  arithmetic:"det-float-or-fixed",
  precision:64,
  rounding:"nearest",
  matmul:{order:"row-major", accumulate:"sequential"},
  canon:{rule:"normalize-encoding", points:["input","post-reduce","final"]}
},

TranscriptSpec = {
  CommitType:"closure",
  HashAlg:"sha256",
  SerializeRule:"EY22-CCE",
  CommitRate:"per-reduction",
  ChainRule:"none"
},

Cert = {
  schema:"EY22-CCE",
  required_checks:["V1","V2","V3","V4","V5","V6","V7","V8"]
},

Interop = {
  UpstreamArtifacts: [],
  Scope: {},
  InvObligations: [],
  LossObligations: [],
  CompatPredicates: ["CGR-C0","CGR-C1","CGR-C2","CGR-C3","CGR-C4","CGR-C6"]
},

Version = "VID-2026-02"
)

```

Rule. All null fields **MUST** be explicit. Omitting them is not allowed, because it can hide semantics.

7.3 Publication Boundary (Normative)

Rule (what must be public). An EY29 publication **MUST** include:

- full SpecID_{EY29},
- ReducerCore commitments (projector/partition/classifier/seed/threshold) sufficient to replay,
- TruncSpec and NMap decisions (including rounding/tie-breaks),
- NumericModel (including ordering/canonicalization),
- InvPolicy and CheckRoutineIDs for all preserved/weakened invariants,
- LossSpec and RoutineIDs for all declared loss metrics (or explicit null-justifications),

- certificate π_{CGR} with transcript commitments, invariant report, and loss report,
- deterministic failure codes (or PASS) for all declared checks.

Rule (no hidden classifier). If RedClass=BUCK, the classifier rules (bucket boundaries, feature definitions, and tie-breaks) MUST be committed by hash or explicit encoding. A bucket label without classifier commitments is invalid.

Rule (no hidden partition). If RedClass=AGG, the partition must be public (explicit list, committed hash, or seed+generator). Averaging across hidden blocks is invalid.

7.4 Non-Claims (Explicit)

EY29 explicitly does *not* claim:

NC1: Truth Preservation Beyond Declared InvPolicy

Reduction is not assumed to preserve any property not listed in InvPolicy. In particular, commutation, validity, positivity, and spectral class are not preserved unless explicitly declared and checked.

NC2: Infinite-Dimensional Guarantees

Passing reduction checks at finite truncations does not imply anything about the infinite-dimensional limit. No analytic equivalence is implied by a stable bucket label.

NC3: Uniqueness of the Reduction

Multiple reducers may map the same A to different reduced objects. EY29 does not claim uniqueness; it binds *this* reducer instance by SpecID.

NC4: Adversarial Robustness Beyond Verifier Checks

EY29 reduces smuggling by commitment and replay, but does not guarantee robustness against adversarial feature engineering beyond declared checks.

NC5: Cross-Workflow Comparability Without Shared SpecID

Reduced outputs are comparable only when reducer SpecIDs match (or a declared coercion exists). A bucket label from a different classifier is a different object.

7.5 Licensing Intent (Vol. No. IV)

The EY29 specification (schemas, baseline routines, invariant vocabulary, verifier checks) is intended to remain publicly readable. Implementations and classifier catalogs may be licensed under project-chosen terms, but:

loss accounting and replayability may not be removed.

Rule (no audit stripping). No distribution or license arrangement may claim EY29 compliance while omitting:

- reducer core commitments,
- loss reports,
- invariant reports,
- and verifier replay rules.

7.6 Chapter Summary

This chapter provided a CGR-0 reference SpecID template, fixed the EY29 publication boundary (what must be committed and public), stated explicit non-claims (no implied preservation beyond InvPolicy, no infinite-limit guarantees, no uniqueness, no extra robustness, no comparability without shared SpecID), and recorded licensing intent consistent with Vol. No. IV: the spec remains auditable and public; implementations may vary, but replayability and loss accounting are mandatory.

Next. Proceed to EY30: Koopman Export Operator $\mathcal{K}$, which defines the canonical mapping from state/operator workflows into observable-space dynamics, and binds how Vol. No. IV artifacts can be exported into Koopman form with interop and verification.

8 Worked Examples (Non-Normative): Reduction Patterns for Vol. No. IV

8.1 Purpose

This chapter is *non-normative*. It illustrates typical EY29 usage patterns:

- projecting onto a low-frequency subspace (PROJ),
- aggregating an operator into block form (AGG),
- producing bucket labels and summaries (BUCK) for registries,
- treating regime transport as reduction when N shrinks ($\text{INTW} \rightarrow \text{CGR}$),
- and exporting reduced summaries into Koopman workflows ($\text{CGR} \rightarrow \text{KOOP}$).

Reminder. Examples become valid only when reducer core parameters, truncation mapping, invariants, and loss metrics are fully committed and replayable.

8.2 Example 1: PROJ — Low-Mode Projection of a Filtered Hamiltonian

Goal. Reduce a filtered operator to its low-mode subspace:

$$\tilde{H} := P F_{\star}(H) P,$$

with P selecting the lowest N_{out} basis modes.

ReducerSpec sketch.

- RedClass=PROJ, NMap=shrink- N
- ProjType=basis-prefix, selection rule: “first N_{out} modes”
- Loss metric: $\|F_{\star}(H) - \tilde{H}\|_F$
- Invariants:
 - I1 (Domain) preserved,
 - I5 (Norm Bound) weakened with tolerance,
 - I2 (Spectrum) weakened (spectral summary computed only on low block).

Interpretation. The reduced operator is not claimed to reproduce high-frequency spectral details; the loss metric explicitly quantifies discarded content.

8.3 Example 2: AGG — Block-Coarse Operator for Compatibility Matrices

Goal. Partition the basis into m blocks and aggregate:

$$\tilde{A}_{ij} := \frac{1}{|B_i||B_j|} \sum_{p \in B_i} \sum_{q \in B_j} A_{pq}.$$

Use case. A compatibility matrix entry may require a small operator (e.g. $m \leq 32$) that can be replayed quickly by many verifiers.

ReducerSpec sketch.

- RedClass=AGG, NMap=same- N (output is $m \times m$)
- PartitionSpec committed by hash (explicit block list)
- Aggregation Φ =mean, with deterministic reduction order
- Loss metrics:
 - $\|A - \hat{A}\|_F$ where $\hat{A}$ is the lifted block operator,
 - within-block variance (stability diagnostic).

Interpretation. AGG produces a coarse operator that is meaningful only under the declared partition. Without that partition commitment, the summary is meaningless.

8.4 Example 3: BUCK — Registry Key for “Spectral Class”

Goal. Produce a stable bucket label and a bounded summary:

$$R(A) := (\text{BucketID}(A), s(A)).$$

Typical inputs.

- $A = F_{\star}(H)$ (output of EY23),
- $A = V$ (validity object from EY24),
- or a token-derived object from EY26.

Multi- N stability (optional). Compute labels across $\mathcal{N}_{\text{in}}$ and aggregate with CrossNPolicy=`stable-first`: choose the first N where label stabilizes and record the stabilization index as diagnostic.

Loss metrics.

- bucket margin (distance to decision boundary),
- stability score across truncations.

Interpretation. BUCK makes registry-grade keys possible, but only because the classifier and features are committed and replayable.

8.5 Example 4: INTW $\rightarrow$ CGR — Transport as Reduction

Goal. Bridge from regime 1 to regime 2 and shrink N :

$$H_2 := TH_1S, \quad \tilde{H}_2 := R(H_2), \quad N_{\text{out}} < N_{\text{in}}.$$

Two equivalent-looking orders (not equivalent).

- **Order A:** transport then reduce,
- **Order B:** reduce then transport.

Rule reminder. The workflow MUST declare the order (EY29 Chapter ??). The order affects loss accounting and invariants.

Interop certificate must bind.

- EY28 π_{INTW} reference,
- EY29 π_{CGR} reference,
- the order and scope (which object was transported/reduced),
- combined truncation policy and deterministic tie-breaks.

8.6 Example 5: CGR $\rightarrow$ KOOP — Reduced Summaries for Koopman Export

Goal. Export a reduced observable basis into Koopman form:

$$(\text{BucketID}, s) := R(\text{trajectory summary}), \quad \mathcal{K}(g) \text{ acts on the reduced observable set.}$$

Interpretation. Koopman export becomes feasible for many verifiers when the observable basis and summaries are bounded and replayable. CGR binds which details were discarded and how much was lost.

8.7 Example 6: Reduction Under Constraints (EY27 + EY29)

Goal. Reduce A and B while tracking commutation weakening:

$$\delta_{\text{comm}} := \|[A, B]\|, \quad \tilde{\delta}_{\text{comm}} := \|\tilde{[A, B]}\|.$$

Policy choices.

- preserve: require $\tilde{\delta}_{\text{comm}} \leq \varepsilon$ (strict),
- weaken: require $\tilde{\delta}_{\text{comm}} \leq g(\delta_{\text{comm}})$ for a declared function g ,
- discard: allowed only if the workflow explicitly does not rely on the constraint.

Interpretation. Reduction is not allowed to silently break constraints if the workflow depends on them.

8.8 Chapter Summary

This non-normative chapter illustrated EY29 in common Vol. No. IV workflows: PROJ low-mode projection, AGG block-aggregation for small verifier-friendly operators, BUCK bucket labels + summaries for registries, INTW-as-reduction when truncation shrinks across regimes, reduction feeding Koopman export, and reduction under commutation constraints.

Next. Proceed to EY30: Koopman Export Operator $\mathcal{K}$.

EY30: Koopman Export Operator $\mathcal{K}$

Canonical Operator Artifacts for QFC

Enter Yourname

Abstract

This whitepaper defines the Koopman Export Operator $\mathcal{K}$, a canonical artifact that maps state/operator-based workflows into observable-space dynamics, enabling regime-agnostic comparison, reduced summaries, and verifier-friendly export of Vol. No. IV pipelines. EY30 standardizes: the observable function space declaration, the induced Koopman action, finite-basis truncations, and deterministic certificates that bind how an underlying evolution (EY25), token execution (EY26), or filtered/operator workflow (EY23–EY24) is represented as a Koopman operator.

$\mathcal{K}$ is treated as an export boundary: it does not claim physical equivalence of regimes or completeness of observable spaces. Instead, it commits to a specific observable basis, sampling/evaluation rules, and replayable construction routines. EY30 composes explicitly with regime intertwiners (EY28) and coarse-grain reducers (EY29), and publishes interop declarations so that Koopman-space statements remain auditable under QFC-style deterministic replay.

1 Motivation and Scope: Exporting Operator Workflows into Observable Space

1.1 Motivation: Why an Export Operator Exists in Vol. No. IV

Vol. No. IV defines canonical operator artifacts that live naturally in “state/operator space”: filters (EY23), validity penalties (EY24), audit-native evolution (EY25), tokens (EY26), commutation constraints (EY27), regime intertwiners (EY28), and coarse-grain reducers (EY29). These artifacts compose into workflows, but their outputs are often:

- high-dimensional,
- basis-dependent,
- regime-specific,
- and difficult to compare across implementations.

Koopman theory provides a complementary viewpoint: instead of evolving states, evolve *observables* (functions on state space). The induced operator (the Koopman operator) often admits:

- linear structure even for nonlinear dynamics,
- representation-agnostic comparison via shared observable dictionaries,

- stable low-dimensional summaries (eigenfunctions, modes),
- and a natural interface for coarse-grain and classification.

EY30 exists to make this translation auditable. It defines a canonical export operator $\mathcal{K}$ that binds:

- what the “observable space” is,
- how observables are evaluated/sampled,
- which finite-basis truncation is used,
- and how the induced Koopman action is constructed deterministically from upstream artifacts.

1.2 What EY30 Defines

EY30 defines:

- an **Observable Space Declaration** (function class + basis/dictionary + evaluation rules),
- a **Koopman Action** specification (pullback by a declared evolution map),
- finite-basis truncation rules that produce a matrix/operator representation,
- deterministic construction routines and certificates that bind the export,
- interop rules for composing Koopman export with EY23–EY29 workflows.

1.3 Core Intuition: Koopman as Pullback of Observables

Let x be a state (in some regime-dependent representation) and let Φ be the evolution map:

$$x_{t+\Delta} = \Phi(x_t).$$

An observable is a function g of the state. The Koopman operator $\mathcal{K}$ acts by pullback:

$$(\mathcal{K}g)(x) := g(\Phi(x)).$$

This is the export boundary:

- upstream artifacts define or constrain Φ (explicitly or implicitly),
- EY30 commits to how Φ is realized (via evolution/tokens/snapshots),
- and the resulting linear operator on observables is constructed deterministically.

1.4 Scope

In-scope.

- Export from EY25 evolution (generator- or trajectory-defined) to a Koopman operator on a declared observable basis.
- Export from EY26 program tokens to a Koopman operator induced by token step execution.
- Export from general operator workflows (EY23/EY24) when an evolution map is provided (by token or by explicit map artifact).

- Finite-basis Koopman representations (dictionary truncations) and their verification.
- Interop with EY28 (regime bridges) and EY29 (coarse-grain).

Out-of-scope.

- Claims of completeness of an observable dictionary (no “we span everything”).
- Unverifiable learned Koopman embeddings without committed parameters and deterministic replay.
- Infinite-dimensional Koopman spectral claims inferred solely from finite truncations.

1.5 Why EY30 Matters for Registries and “Regime-Agnostic” Claims

Many “same phenomenon” claims are hard to justify directly in state/operator representations because bases differ. Koopman export provides a unifying comparison layer:

$$\text{workflow} \Rightarrow \mathcal{K}\text{-representation} \Rightarrow \text{registry} / \text{bucket} / \text{invariant signatures}.$$

But this only works if the export is auditable:

- the observable basis must be committed,
- sampling/evaluation rules must be deterministic,
- and the constructed Koopman operator must be replayable.

EY30 supplies exactly that.

1.6 Design Goals (Normative)

G1: Deterministic Export

Given the same upstream artifact commitments and the same EY30 SpecID, every verifier **MUST** compute the same Koopman representation.

G2: Explicit Observable Space

The observable dictionary (basis) and evaluation rules **MUST** be explicit and hash-committed.

G3: Explicit Evolution Realization

EY30 **MUST** declare how Φ is realized:

- by generator evolution (EY25),
- by token execution (EY26),
- or by trajectory snapshots (committed).

G4: Interop by Construction

If Koopman export depends on regime transport (EY28) or coarse-grain (EY29), the workflow **MUST** publish interop certificates binding:

- order of operations,
- truncation policies,
- and preserved/weakened invariants.

1.7 Chapter Summary

This chapter motivated EY30: Koopman export provides a regime-agnostic observable-space layer for comparing and summarizing Vol. No. IV workflows, but only if export semantics are explicit and replayable. It defined scope, core intuition (Koopman pullback), and the normative design goals: deterministic export, explicit observable dictionary, explicit evolution realization, and interop-by-construction.

Chapter 2 next defines the observable space declaration: function classes, dictionaries/bases, sampling grids, evaluation rules, truncation policies, and the minimal invariant vocabulary used to express correctness and stability in Koopman space.

2 Observable Space Declaration: Dictionaries, Evaluation Rules, and Truncation

2.1 Purpose

Koopman export is meaningless without a precise statement of *what an observable is* and *how it is evaluated*. This chapter defines the EY30 **Observable Space Declaration**:

- the observable function class,
- the dictionary/basis used for finite export,
- the sampling/evaluation rules (grids, measures, trajectories),
- truncation policies for finite-dimensional Koopman representations,
- and the invariant vocabulary for stability and verification in observable space.

Rule. Every EY30 instance **MUST** include an ObservableDecl component in its SpecID. If ObservableDecl is missing, the export artifact is invalid.

2.2 ObservableDecl: Schema

An observable declaration is a tuple:

$$\text{ObservableDecl} := (\text{StateRep}, \mathcal{G}, \mathcal{D}, \text{EvalRule}, \text{TruncObs}, \text{InnerProd}, \text{Canon}).$$

StateRep (state representation reference)

StateRep is a reference that fixes how a state x is represented for evaluation:

- regime identifier (RegimeID or reference to EY28 RegimeSpec),
- state encoding type (vector, density, operator snapshot, reduced summary),
- coordinate/basis declaration (or a hash reference),
- normalization rule (if any).

Rule. If EY30 uses a regime bridge (EY28), StateRep MUST cite the target regime specification and include INTW-Ref in Interop.

Function class $\mathcal{G}$

$\mathcal{G}$ specifies the allowable observable functions. At minimum, EY30 supports the following classes (choose one):

G1: Polynomial dictionary class.

$$\mathcal{G} = \text{POLY}(d, p)$$

meaning: multivariate polynomials up to degree p on a d -dimensional state vector.

G2: Fourier/trigonometric dictionary class.

$$\mathcal{G} = \text{FOURIER}(d, \Omega)$$

meaning: trigonometric basis with frequency set Ω (explicitly enumerated or deterministically generated).

G3: Kernel feature class (explicit parameters).

$$\mathcal{G} = \text{KERNEL}(\kappa, \theta)$$

meaning: kernel feature map with kernel type κ and committed parameters θ . (*Learned* parameters are allowed only if committed and deterministic.)

G4: Operator-derived observables.

$$\mathcal{G} = \text{OP-FEAT}(\Psi)$$

meaning: observables are features computed from operator/state objects (trace, norms, spectral stats) via a committed feature map Ψ .

Rule. If $\mathcal{G}$ involves learnable parameters, those parameters MUST be committed and the evaluation must be deterministic under NumericModel.

Dictionary/Basis $\mathcal{D}$

Koopman export typically uses a finite dictionary $\mathcal{D} = \{g_1, \dots, g_m\} \subset \mathcal{G}$.

Schema.

$$\mathcal{D} := (\text{DictType}, \text{DictGen}, \text{DictParams}, \text{OrderSpec}).$$

DictType options.

- **explicit:** the list of basis functions is explicitly enumerated.
- **seeded:** generated deterministically from a public seed + generator.
- **reduced:** derived from EY29 reduced summaries (must cite CGR-Ref).

Rule. A dictionary MUST be committed by explicit listing, hash, or deterministic seed+generator. Omission is forbidden.

OrderSpec. Basis ordering must be fixed (lexicographic or committed order), because it changes the Koopman matrix representation.

EvalRule (how observables are evaluated)

EvalRule binds the evaluation points and evaluation semantics.

Schema.

$\text{EvalRule} := (\text{EvalType}, \text{PointSet}, \text{MeasureSpec}, \text{TimeSpec}, \text{TieBreak}).$

EvalType options.

- **trajectory:** points are states along a committed trajectory (EY25 snapshots or EY26 token execution).
- **grid:** points are from a deterministic grid in state coordinates.
- **sampled:** points are sampled from a deterministic PRNG and a committed distribution (seeded).

Rule. If **trajectory** is used, EvalRule MUST cite the upstream trajectory source (EY25 certificate or EY26 token transcript). If **sampled** is used, PRNGSpec and seed MUST be public and deterministic.

TimeSpec. TimeSpec declares:

- discrete step Δ or token period,
- time index set $\{t_k\}$,
- and whether Koopman is constructed per-step or per-period.

TruncObs (observable truncation policy)

Finite export requires choosing m , the dictionary size.

Schema.

$\text{TruncObs} := (m, \text{SelectRule}, \text{CrossMPolicy}).$

SelectRule examples.

- **prefix:** first m in canonical order,
- **degree:** all polynomials up to degree p (implies m),
- **energy:** select basis by a committed score and threshold.

Rule. Selection must be deterministic. If a score is used, the scoring routine must be committed.

InnerProd (how Koopman is projected)

Most practical Koopman constructions compute a finite operator by regression/projection in an inner product:

$$\langle g_i, g_j \rangle := \mathbb{E}_{x \sim \mu}[g_i(x) \overline{g_j(x)}].$$

Schema.

InnerProd := (Type, μ , Estimator, Regularization).

Type options.

- **empirical**: empirical average over EvalRule point set,
- **grid**: weighted sum over grid,
- **trajectory**: time-average along trajectory (must bind window and weights).

Regularization. If regression is ill-conditioned, a deterministic regularization (e.g. ridge λI) may be declared.

Rule. Any regularization MUST be explicit and committed; hidden conditioning fixes are forbidden.

Canon (canonicalization rules)

Canon binds:

- numeric model for evaluation and regression,
- rounding/precision rules,
- canonicalization points (after evaluation, after regression, final).

2.3 Invariant Vocabulary in Observable Space

EY30 requires a minimal invariant vocabulary for export correctness and stability.

K1: Eval Determinism. Given the same EvalRule and NumericModel, all evaluators produce identical observable values on all points.

K2: Dictionary Commitment. The dictionary $\mathcal{D}$ used is exactly the committed one (hash or explicit list).

K3: Projection Stability. Koopman operator construction is stable under declared tolerances (e.g. conditioning diagnostics are within bounds).

K4: Time-Step Semantics. The constructed operator corresponds to the declared Δ or period (no hidden time rescaling).

K5: Cross-M Stability (optional). If CrossMPolicy is used, the aggregated Koopman representation is stable across dictionary sizes.

Rule. If a workflow uses Koopman eigenvalues/modes as registry keys, it MUST include K3 and K5 (if applicable) with explicit tolerances.

2.4 Deterministic Failure Codes (ObservableDecl Layer)

At minimum:

- **REJECT_OBSDECL_MISSING**
- **REJECT_DICT_MISSING**
- **REJECT_EVALRULE_MISSING**
- **REJECT_TRUNC OBS_MISSING**
- **REJECT_INNERPROD_MISSING**
- **FAIL_HASH_MISMATCH**
- **FAIL_NONDETERMINISM**
- **FAIL_EVAL_SOURCE_MISSING**
- **FAIL_ILL_CONDITIONED**

2.5 Chapter Summary

This chapter defined the Observable Space Declaration (ObservableDecl): state representation reference, observable function class, finite dictionary/basis and ordering, evaluation rules (trajectory/grid/sampled), observable truncation, inner product/projection estimator, and canonicalization. It also defined a minimal invariant vocabulary (K1–K5) for correctness and stability of Koopman exports and fixed failure codes.

Chapter ?? next defines the Koopman construction semantics: how the evolution map Φ is realized from EY25/EY26 pipelines, how the finite Koopman operator is computed (regression/projection), and how the export is represented as a canonical operator artifact with SpecID.

3 Motivation and Scope: Exporting Operator Workflows into Observable Space

3.1 Motivation: Why an Export Operator Exists in Vol. No. IV

Vol. No. IV defines canonical operator artifacts that live naturally in “state/operator space”: filters (EY23), validity penalties (EY24), audit-native evolution (EY25), tokens (EY26), commutation constraints (EY27), regime intertwiners (EY28), and coarse-grain reducers (EY29). These artifacts compose into workflows, but their outputs are often:

- high-dimensional,
- basis-dependent,
- regime-specific,

- and difficult to compare across implementations.

Koopman theory provides a complementary viewpoint: instead of evolving states, evolve *observables* (functions on state space). The induced operator (the Koopman operator) often admits:

- linear structure even for nonlinear dynamics,
- representation-agnostic comparison via shared observable dictionaries,
- stable low-dimensional summaries (eigenfunctions, modes),
- and a natural interface for coarse-grain and classification.

EY30 exists to make this translation auditable. It defines a canonical export operator $\mathcal{K}$ that binds:

- what the “observable space” is,
- how observables are evaluated/sampled,
- which finite-basis truncation is used,
- and how the induced Koopman action is constructed deterministically from upstream artifacts.

3.2 What EY30 Defines

EY30 defines:

- an **Observable Space Declaration** (function class + basis/dictionary + evaluation rules),
- a **Koopman Action** specification (pullback by a declared evolution map),
- finite-basis truncation rules that produce a matrix/operator representation,
- deterministic construction routines and certificates that bind the export,
- interop rules for composing Koopman export with EY23–EY29 workflows.

3.3 Core Intuition: Koopman as Pullback of Observables

Let x be a state (in some regime-dependent representation) and let Φ be the evolution map:

$$x_{t+\Delta} = \Phi(x_t).$$

An observable is a function g of the state. The Koopman operator $\mathcal{K}$ acts by pullback:

$$(\mathcal{K}g)(x) := g(\Phi(x)).$$

This is the export boundary:

- upstream artifacts define or constrain Φ (explicitly or implicitly),
- EY30 commits to how Φ is realized (via evolution/tokens/snapshots),
- and the resulting linear operator on observables is constructed deterministically.

3.4 Scope

In-scope.

- Export from EY25 evolution (generator- or trajectory-defined) to a Koopman operator on a declared observable basis.
- Export from EY26 program tokens to a Koopman operator induced by token step execution.
- Export from general operator workflows (EY23/EY24) when an evolution map is provided (by token or by explicit map artifact).
- Finite-basis Koopman representations (dictionary truncations) and their verification.
- Interop with EY28 (regime bridges) and EY29 (coarse-grain).

Out-of-scope.

- Claims of completeness of an observable dictionary (no “we span everything”).
- Unverifiable learned Koopman embeddings without committed parameters and deterministic replay.
- Infinite-dimensional Koopman spectral claims inferred solely from finite truncations.

3.5 Why EY30 Matters for Registries and “Regime-Agnostic” Claims

Many “same phenomenon” claims are hard to justify directly in state/operator representations because bases differ. Koopman export provides a unifying comparison layer:

$$\text{workflow} \Rightarrow \mathcal{K}\text{-representation} \Rightarrow \text{registry} / \text{bucket} / \text{invariant signatures}.$$

But this only works if the export is auditable:

- the observable basis must be committed,
- sampling/evaluation rules must be deterministic,
- and the constructed Koopman operator must be replayable.

EY30 supplies exactly that.

3.6 Design Goals (Normative)

G1: Deterministic Export

Given the same upstream artifact commitments and the same EY30 SpecID, every verifier **MUST** compute the same Koopman representation.

G2: Explicit Observable Space

The observable dictionary (basis) and evaluation rules **MUST** be explicit and hash-committed.

G3: Explicit Evolution Realization

EY30 MUST declare how Φ is realized:

- by generator evolution (EY25),
- by token execution (EY26),
- or by trajectory snapshots (committed).

G4: Interop by Construction

If Koopman export depends on regime transport (EY28) or coarse-grain (EY29), the workflow MUST publish interop certificates binding:

- order of operations,
- truncation policies,
- and preserved/weakened invariants.

3.7 Chapter Summary

This chapter motivated EY30: Koopman export provides a regime-agnostic observable-space layer for comparing and summarizing Vol. No. IV workflows, but only if export semantics are explicit and replayable. It defined scope, core intuition (Koopman pullback), and the normative design goals: deterministic export, explicit observable dictionary, explicit evolution realization, and interop-by-construction.

Chapter 2 next defines the observable space declaration: function classes, dictionaries/bases, sampling grids, evaluation rules, truncation policies, and the minimal invariant vocabulary used to express correctness and stability in Koopman space.

4 Artifact Structure and SpecID Schema

4.1 Purpose

Chapters 3-?? defined why EY30 exists, what observable space is, and how Koopman export is constructed. This chapter defines EY30 as an auditable artifact with a precise SpecID schema:

- how ObservableDecl and EvolutionDecl are represented,
- how Koopman construction class and numeric routines are bound,
- how outputs (matrix K , spectral summaries) are committed,
- and what transcript/certificate payload is required for deterministic replay.

4.2 Koopman Export as a Typed Artifact

EY30 treats Koopman export as a typed map:

$$\mathcal{K}_{\text{export}} : (\text{workflow commitments}) \rightarrow (\text{observable-space operator artifact}).$$

InputType and OutputType

Each EY30 instance MUST declare:

$\text{InputType} \in \{\text{EVO}, \text{TOKEN}, \text{SNAP}, \text{WORKFLOW}\}, \quad \text{OutputType} \in \{\text{KOOP_OP}, \text{KOOP_SPEC}, \text{KOOP_OP_SPEC}\}$

Interpretation.

- $\text{InputType} = \text{EVO}$: export from EY25 generator-based evolution.
- $\text{InputType} = \text{TOKEN}$: export from EY26 token execution.
- $\text{InputType} = \text{SNAP}$: export from committed paired snapshots (x_k, x'_k) .
- $\text{InputType} = \text{WORKFLOW}$: export from a composed pipeline (must list upstream SpecIDs in Interop).

4.3 KoopmanSpec

EY30 standardizes a Koopman export declaration:

$\text{KoopmanSpec} := (\text{ObservableDecl}, \text{EvolutionDecl}, \text{KoopClass}, \text{BuildSpec}, \text{DiagSpec}).$

KoopClass

$\text{KoopClass} \in \{\text{KOP1}, \text{KOP2}, \text{KOP3}\}$

as defined in Chapter ??.

BuildSpec

BuildSpec binds the deterministic construction routine:

$\text{BuildSpec} := (\text{RoutineID}, \text{LinAlgSpec}, \text{CanonPoints}, \text{TieBreaks}).$

LinAlgSpec (required). LinAlgSpec MUST bind:

- matrix multiplication order and accumulation,
- pseudo-inverse method (if used) and its truncation policy (threshold/grid),
- solver/inversion method and regularization (if used),
- eigensolver method and ordering (if eigenvalues/modes are published).

Rule (null discipline). Any BuildSpec component not used MUST be explicitly `null`. Omission is forbidden.

DiagSpec

DiagSpec binds required diagnostics and tolerances:

$\text{DiagSpec} := \{(\text{DiagName}, \text{ValueType}, \text{TolSpec}, \text{RoutineID})\}.$

Rule. At minimum, DiagSpec MUST include:

- reconstruction error L_{rec} ,
- a conditioning diagnostic,

unless explicitly justified as `null` (rare; generally non-acceptable for registry-grade exports).

4.4 NumericModel and Truncation Binding

Koopman export is only meaningful under declared numeric semantics and observable truncation.

TruncObs Binding

ObservableDecl.TruncObs binds:

- dictionary size m ,
- selection rule and ordering,
- optional CrossMPolicy (if multi- m used).

NumericModel

EY30 NumericModel MUST bind:

- arithmetic type (float/fixed/exact),
- precision and rounding,
- deterministic evaluation rules for observables,
- deterministic linear algebra rules (as referenced in BuildSpec),
- canonicalization rules and points.

4.5 TranscriptSpec and Commitments

Export must be replayable. Therefore, transcript commitments MUST bind:

- all upstream references (EY25/EY26/SNAP dataset hashes, and/or workflow SpecIDs),
- dictionary commitments (hash/seed/generator, ordering),
- evaluation point set commitments (trajectory/window/grid/seeded samples),
- time semantics Δ (or period) and per-step vs per-period choice,
- construction class and routine IDs,
- diagnostics and their computed values,
- output Koopman operator K (or its hash) and optional spectral summaries,
- deterministic tie-break decisions (SVD truncation, eigenvalue ordering, CrossM decisions).

Rule. If K is published as a matrix, its canonical serialization MUST be committed. If K is published as a factorization, the factorization must be sufficient to reproduce K exactly under NumericModel.

4.6 SpecID for EY30

ArtifactType. For EY30:

ArtifactType = KOOP.

SpecID components (EY30 binding). The full SpecID tuple is:

$\text{SpecID}_{\text{EY30}} := (\text{KOOP}, \text{InputType}, \text{OutputType}, \text{KoopmanSpec}, \text{NumericModel}, \text{TranscriptSpec}, \text{Cert}, \text{Interop})$

Interop (Mini)

Interop binds intended compositions and references to upstream artifacts:

- UpstreamSpecIDs: list of EY23–EY29 artifacts used to generate the evolution or the observables,
- INTW-Ref (if regime bridging is used),
- CGR-Ref (if coarse-grain reduction is used),
- Scope: what is exported (trajectory, token, filtered operator pipeline),
- CompatPredicates: identifiers of interop predicates (Koopman-layer + EY22 baseline).

Rule. If the export depends on reduction (EY29) or transport (EY28), Interop MUST declare the order. If order is missing: **FAIL_COMPAT_PREDICATE**.

4.7 Schema-Layer Failure Codes

At minimum:

- **REJECT_MALFORMED**
- **REJECT_SPECID_MISSING**
- **REJECT_OBSDECL_MISSING**
- **REJECT_EVODECL_MISSING**
- **REJECT_KOOPCLASS_MISSING**
- **REJECT_BUILDSPEC_MISSING**
- **REJECT_DIAGSPEC_MISSING**
- **REJECT_HIDDEN_INPUT**
- **FAIL_HASH_MISMATCH**
- **FAIL_NONDETERMINISM**

4.8 Chapter Summary

This chapter defined the EY30 artifact schema: InputType/OutputType, a KoopmanSpec binding ObservableDecl, EvolutionDecl, KoopClass, construction routines (BuildSpec), and diagnostics (DiagSpec); plus NumericModel and transcript commitments. It fixed the full SpecID tuple for

EY30 and required schema-layer failure codes and interop mini declarations (including order binding for EY28/EY29).

Chapter 5 next specifies certificate payload and verifier checks: how K , diagnostics, and optional spectral summaries are committed, how export replay is performed, and how Koopman exports are accepted or rejected deterministically.

5 Certificate and Verifier: Replayable Export, Diagnostics, and Acceptance Rules

5.1 Purpose

EY30 exports are meant to be used as:

- verifier-friendly operator artifacts K in observable space,
- registry keys (via bucket labels, spectral summaries, or Koopman signatures),
- and interop bridges between regimes and pipelines.

Therefore, every export must be replayable and must publish diagnostics that quantify how well the finite K represents the declared Koopman action.

This chapter specifies:

- the certificate payload π_{KOOP} ,
- deterministic verifier checks for acceptance,
- transcript commitments for evaluation and construction,
- mandatory diagnostics (reconstruction error, conditioning),
- and required failure codes.

5.2 Certificate Object π_{KOOP}

Rule. An EY30 publication **MUST** include a certificate π_{KOOP} in canonical certificate encoding (EY22-CCE).

Header (required)

- artifact type: KOOP
- artifact symbol: $\mathcal{K}$
- InputType, OutputType
- KoopClass
- Version
- Certificate schema identifier (EY22-CCE)

SpecID (required)

The full SpecID tuple MUST be embedded:

$\text{SpecID}_{\text{EY30}} := (\text{Koop}, \text{InputType}, \text{OutputType}, \text{KoopmanSpec}, \text{NumericModel}, \text{TranscriptSpec}, \text{Cert}, \text{Inter})$

Committed Inputs (required)

π_{Koop} MUST commit to:

- ObservableDecl (StateRep, $\mathcal{G}$, $\mathcal{D}$, EvalRule, TruncObs, InnerProd, Canon),
- EvolutionDecl (Φ realization mode, source references, Δ , semantics, replay rules),
- KoopClass and BuildSpec (RoutineID, LinAlgSpec, thresholds, regularization, tie-breaks),
- DiagSpec (diagnostic names, routines, tolerances),
- NumericModel (evaluation + linear algebra determinism),
- all upstream references:
 - EY25 π_{LIOUV} or
 - EY26 token transcript or
 - SNAP dataset commitment,
- evaluation point set commitment (trajectory window/grid/seeded samples),
- any interop references used (INTW-Ref, CGR-Ref, upstream SpecIDs for WORKFLOW input).

Committed Outputs (required)

π_{Koop} MUST commit to:

- Koopman operator representation K (or committed factorization) and its hash h_K ,
- (optional) spectral summaries: eigenvalues/modes and their hashes,
- evaluation transcript commitment Tx_{eval} ,
- construction transcript commitment Tx_{build} ,
- diagnostics report commitment Tx_{diag} ,
- deterministic failure code if export fails.

5.3 Evaluation Transcript (Minimum Contents)

The evaluation transcript MUST bind enough information to replay the formation of X and Y :

- dictionary commitment resolution (explicit list/hash/seed+generator) and ordering,
- evaluation point set commitment:
 - trajectory source and window indices (if trajectory),
 - grid specification (if grid),
 - PRNGSpec+seed+distribution (if sampled),

- time semantics: Δ or period and per-step/per-period choice,
- pairing rule for $(x_k, \Phi(x_k))$ (or (x_k, x_{k+1})),
- canonicalization points and numeric evaluation rules,
- hashes of the resulting X and Y matrices (or their canonical encodings).

Rule. If X and Y are too large to publish, their canonical hashes MUST be published, and the upstream commitments must allow reconstructing them exactly.

5.4 Construction Transcript (Minimum Contents)

The construction transcript MUST bind:

- KoopClass and BuildSpec resolution (including thresholds and regularization),
- linear algebra tie-breaks (SVD ordering, truncation choices),
- canonicalization points,
- intermediate matrix hashes as needed to replay deterministically,
- output K hash h_K (or factorization hashes).

5.5 Diagnostics Report (Required Structure)

A diagnostics report is the canonical serialization of:

$$\text{DiagReport} := \{(\text{DiagName}, \text{Value}, \text{Tol}, \text{Result})\}.$$

Mandatory diagnostics. At minimum:

- **D1 Reconstruction error:**

$$L_{\text{rec}} := \frac{\|Y - KX\|_F}{\|Y\|_F}.$$

- **D2 Conditioning diagnostic:** a committed estimate of $\kappa(X)$ (or $\kappa(G)$ for Galerkin).

Rule. If OutputType includes spectral summaries, a spectral solver diagnostic SHOULD be included (e.g. residual norms of eigenpairs).

5.6 Verifier Contract

A verifier consumes π_{Koop} and returns:

$$\{\text{ACCEPT}, \text{REJECT}\} \cup \{\text{FAIL_}\ast\}.$$

Acceptance meaning. ACCEPT means: the export artifact is well-formed, replayable, K matches replay under NumericModel, and mandatory diagnostics are computed and within tolerances (if tolerances are declared). It does *not* claim that the dictionary is complete, that the Koopman truncation is physically exact, or that any cross-regime equivalence holds beyond declared interop.

5.7 Required Verifier Checks (Normative)

V1: Structural Validity (EY22-CCE)

Parse canonical certificate encoding and confirm required fields exist. Failure: **REJECT__MALFORMED**.

V2: SpecID Completeness

Confirm presence of:

- `ArtifactType=KOOP`,
- `InputType`, `OutputType`,
- `KoopmanSpec` (`ObservableDecl`, `EvolutionDecl`, `KoopClass`, `BuildSpec`, `DiagSpec`),
- `NumericModel`, `TranscriptSpec`, `Cert`, `Interop`, `Version`.

Failure: **REJECT__SPECID__MISSING**, **REJECT__OBSDECL__MISSING**, **REJECT__EVODECL__MISSING**, **REJECT__KOOPCLASS__MISSING**, **REJECT__BUILDSPEC__MISSING**, **REJECT__DIAGSPEC__MISSING**.

V3: Commitment Integrity

Verify that all committed inputs resolve and match: dictionary, evaluation point set source, upstream evolution source (EY25/EY26/SNAP), and any interop references (INTW-Ref, CGR-Ref). Failure: **FAIL__HASH__MISMATCH**, **REJECT__HIDDEN__INPUT**.

V4: Evaluation Replay (X, Y)

Replay `EvalRule` deterministically:

- reconstruct points $\{x_k\}$,
- realize Φ per `EvolutionDecl` (generator/token/snap),
- evaluate dictionary $\psi(x_k)$ and $\psi(\Phi(x_k))$,
- construct X, Y in committed order and canonicalize.

Compare hashes to committed X, Y (or rebuild them directly if published). Failure: **FAIL__NONDETERMINISTIC__HASH__MISMATCH**, **FAIL__EVAL__SOURCE__MISSING**, **FAIL__TRANSCRIPT__MISMATCH**.

V5: Koopman Construction Replay (K)

Replay `KoopClass` construction under `BuildSpec`:

- compute K via `KOP1/KOP2/KOP3`,
- apply deterministic SVD/pseudo-inverse/inversion rules and thresholds,
- canonicalize K ,
- compare h_K to committed value.

Failure: **FAIL__SVD__UNSPECIFIED**, **FAIL__PSEUDOINV__UNDEFINED**, **FAIL__ILL__CONDITIONED**, **FAIL__TRANSCRIPT__MISMATCH**.

V6: Diagnostics Replay

Replay all diagnostics in DiagSpec:

- compute L_{rec} ,
- compute conditioning diagnostic,
- compute any optional diagnostics (CrossM stability, eigenpair residuals).

Compare to DiagReport commitments and tolerances. Failure: **FAIL_RECONSTRUCTION_TOO_HIGH**, **FAIL_ILL_CONDITIONED**, **FAIL_TRANSCRIPT_MISMATCH**.

V7: Spectral Summary Replay (if published)

If eigenvalues/modes are published:

- replay eigensolver with committed algorithm and ordering,
- verify residual norms (if declared),
- compare to committed spectral summary hash.

Failure: **FAIL_EIGEN_UNSPECIFIED**, **FAIL_TRANSCRIPT_MISMATCH**.

V8: Interop Claim Consistency

If Interop declares EY28/EY29 dependencies:

- verify the order of operations is declared and consistent with transcripts,
- verify truncation policies do not conflict,
- verify that any reduction/transport obligations are satisfied by referenced certificates.

Failure: **FAIL_COMPAT_PREDICATE**.

5.8 Acceptance Rule

Rule. A verifier returns ACCEPT iff:

- V1–V5 pass,
- mandatory diagnostics are present and replay-match,
- and any declared tolerances are satisfied.

5.9 Required Failure Codes

At minimum:

- **REJECT_MALFORMED**
- **REJECT_SPECID_MISSING**
- **REJECT_OBSDECL_MISSING**
- **REJECT_EVODECL_MISSING**

- **REJECT_KOOPCLASS_MISSING**
- **REJECT_BUILDSPEC_MISSING**
- **REJECT_DIAGSPEC_MISSING**
- **REJECT_HIDDEN_INPUT**
- **FAIL_HASH_MISMATCH**
- **FAIL_EVAL_SOURCE_MISSING**
- **FAIL_NONDETERMINISM**
- **FAIL_TRANSCRIPT_MISMATCH**
- **FAIL_SVD_UNSPECIFIED**
- **FAIL_PSEUDOINV_UNDEFINED**
- **FAIL_ILL_CONDITIONED**
- **FAIL_RECONSTRUCTION_TOO_HIGH**
- **FAIL_EIGEN_UNSPECIFIED**
- **FAIL_COMPAT_PREDICATE**

5.10 Chapter Summary

This chapter defined the EY30 certificate π_{KOOP} and deterministic verifier checks: SpecID completeness, commitment integrity, evaluation replay of X, Y , Koopman construction replay of K , diagnostics replay (reconstruction + conditioning), optional spectral summary replay, and interop claim consistency for EY28/EY29 dependencies. It fixed acceptance rules and required failure codes.

Chapter 6 next provides interoperability declarations (mini) for EY30: compatibility predicates for composing Koopman export with EY23–EY29 workflows, and how Koopman artifacts enter the Vol. No. IV compatibility matrix and registry.

6 Interop Declarations (Mini): Koopman-Space Compatibility and Registry Use

6.1 Purpose

EY30 sits at an export boundary: it converts Vol. No. IV workflows into observable-space operator artifacts. This chapter instantiates EY22 interoperability rules for Koopman export:

- how EY30 is referenced by upstream and downstream artifacts,
- what it means for a workflow to be “Koopman-compatible”,
- a minimal set of compatibility predicates for Koopman export,
- and how interop certificates bind observable dictionaries, evaluation sources, and construction routines.

Rule. If a published claim depends on Koopman-space objects (Koopman matrix K , Koopman eigenvalues/modes, Koopman signatures), the publication MUST include an interop certificate π_{interop} referencing π_{KOOP} and the upstream artifact(s) that define Φ , the evaluation points, and any reduction/transport steps.

6.2 Interop Component Fields (Standard)

Any artifact that composes with EY30 MUST include an interop entry:

Interop $\ni$ (KOOP-Ref, InteropRole, Scope, DictObligations, EvalObligations, DiagObligations, CompatPredi

Required fields.

- KOOP-Ref: hash/SpecID reference to π_{KOOP} (and h_K).
- InteropRole: one of `producer-export`, `consumer-koops`, `registry-key`, `signature-layer`.
- Scope: what is exported (trajectory window, token period, workflow stage).
- DictObligations: required dictionary identity (DictType, ordering, truncation m , CrossM-Policy if any).
- EvalObligations: required evaluation source identity (trajectory/token/dataset), time semantics Δ , pairing rules.
- DiagObligations: required diagnostics and bounds (e.g. $\max L_{\text{rec}}$, $\max$ conditioning).
- CompatPredicates: identifiers of compatibility predicates used (defined below).

Rule. If a workflow relies on Koopman comparability, it MUST bind dictionary identity and ordering. Two Koopman operators are not comparable unless their ObservableDecl matches (or a declared coercion exists).

6.3 Compatibility Predicates (EY30 Mini Set)

EY30 standardizes a minimal predicate set for Koopman export interop.

KOOP-C0: SpecID/Certificate Binding

KOOP-C0 := (EIF KOOP-Ref matches a valid π_{KOOP} and SpecID hashes align).

KOOP-C1: Export Replay Closure

KOOP-C1 := (EIF evaluation and construction are replayable and h_K matches replay).

KOOP-C2: Dictionary Identity

KOOP-C2 := (EIF ObservableDecl.Dictionary (type+hash/seed+order) matches exactly).

KOOP-C3: Evaluation Source Identity

KOOP-C3 := (EIF EvalRule point set source and pairing rules match; Δ semantics match).

KOOP-C4: Diagnostics Present and Within Bounds

KOOP-C4 := (EIF mandatory diagnostics are present, replay-match, and satisfy DiagObligations).

KOOP-C5: CrossMPolicy Determinism (if used)

KOOP-C5 := (EIF CrossMPolicy is declared, deterministic, and transcript binds decisions).

KOOP-C6: Registry-Key Stability (if used)

KOOP-C6 := (EIF key material (eigenvalues/modes/signature) is canonically encoded with ordering+tie-breaker).

KOOP-C7: Upstream Interop Closure

KOOP-C7 := (EIF all upstream dependencies (EY25/EY26/EY28/EY29) referenced by KOOP are themselves KOOP).

6.4 Pairwise Interop Templates (EY23–EY29 → EY30)

This section lists intended composition semantics. These are templates; real workflows MUST cite predicate IDs.

LIUV (EY25) → KOOP (EY30): Generator-Based Export

Typical workflow.

$$\Phi_{\Delta} \text{ from EY25} \Rightarrow K \text{ on dictionary } \mathcal{D}.$$

Compatibility template. OK if:

$$\text{KOOP-C0} \wedge \text{KOOP-C1} \wedge \text{KOOP-C2} \wedge \text{KOOP-C3} \wedge \text{KOOP-C4} \wedge \text{KOOP-C7}.$$

FLOQ (EY26) → KOOP (EY30): Token-Based Export

Typical workflow. Per-step or per-period Koopman:

$$x_{k+1} = U_F(x_k) \Rightarrow K.$$

Compatibility template. OK* if KOOP-C3 includes time-slicing semantics and KOOP-C7 confirms token transcript integrity.

INTW (EY28) ↔ KOOP (EY30): Regime-Bridged Export

Two options:

- **Export-in-target:** transport states first (EY28), then evaluate observables and build K .
- **Export-in-source:** build K in source regime, then map observable dictionary via a declared observable-bridge.

Rule. The workflow MUST declare the order. Otherwise: **FAIL_COMPAT_PREDICATE**.

Compatibility template. Require:

$$\text{INTW-C0..C4} \wedge \text{KOOP-C0..C4} \wedge \text{KOOP-C7},$$

and bind StateRep and dictionary identity in the target regime.

CGR (EY29) $\rightarrow$ KOOP (EY30): Reduced Dictionaries / Reduced Evaluation

If dictionary is **reduced** (derived from EY29 outputs), or if evaluation points are reduced summaries, then KOOP-C7 must include a verified CGR-Ref and KOOP-C2 must bind the reduced dictionary construction recipe.

Compatibility template. OK* iff:

$$\text{CGR-C0..C3} \wedge \text{KOOP-C0..C4} \wedge \text{KOOP-C7}.$$

CSF/VPO (EY23/EY24) $\rightarrow$ KOOP (EY30): Workflow Export

Filtering/validity alone do not define Φ . Therefore EY30 requires:

- Φ realized via EY25 or EY26, or
- committed snapshots derived from a verified workflow.

Compatibility template. NO unless EvolutionDecl cites EY25/EY26/SNAP with commitments (KOOP-C3, KOOP-C7).

6.5 Interop Certificates for Koopman Workflows

Whenever a claim depends on Koopman export, the workflow SHOULD publish an interop certificate:

$\pi_{\text{interop}} := (\text{UpstreamSpecIDs}, \text{KOOP-Ref}, \text{Scope}, \text{DictObligations}, \text{EvalObligations}, \text{DiagObligations}, \text{Compa}$

Rule (registry-grade Koopman keys). If Koopman eigenvalues/modes/signatures are used as registry keys, π_{interop} MUST include:

- KOOP-C6 evidence (canonical key material + ordering/tie-breaks),
- KOOP-C4 evidence (diagnostics within bounds),
- KOOP-C2 and KOOP-C3 evidence (dictionary and evaluation source identity).

6.6 Deterministic Interop Failure Codes

At minimum:

- **FAIL_COMPAT_PREDICATE**
- **FAIL_HASH_MISMATCH**
- **FAIL_NONDETERMINISM**
- **FAIL_EVAL_SOURCE_MISSING**

- **FAIL_TRANSCRIPT_MISMATCH**
- **FAIL_ILL_CONDITIONED**
- **FAIL_RECONSTRUCTION_TOO_HIGH**

6.7 Chapter Summary

This chapter defined EY30 interoperability mini-declarations: how Koopman exports are referenced (KOOP-Ref), how workflows state scope, dictionary/evaluation/diagnostic obligations, and a minimal set of Koopman compatibility predicates (KOOP-C0..C7). It provided pairwise interop templates for composing Koopman export with EY25/EY26/EY28/EY29, and fixed interop certificate requirements for registry-grade Koopman keys.

Chapter 7 next provides a KOOP-0 reference SpecID template, fixes the publication boundary and non-claims for EY30, and closes the artifact in a QFC-consistent way.

7 KOOP-0 Reference Instance, Publication Boundary, and Non-Claims

7.1 Purpose

This chapter closes EY30 by:

- providing a concrete KOOP-0 reference SpecID template (a stable meaning of “Koopman export artifact”),
- fixing what must be published for a valid EY30 export,
- stating explicit non-claims (what Koopman export does *not* guarantee),
- and recording licensing intent consistent with Vol. No. IV.

7.2 KOOP-0 Reference Instance (Template)

KOOP-0 is the conservative baseline profile for Koopman export. It aims to minimize ambiguity and maximize verifier replayability.

KOOP-0 profile choices.

- KoopClass = KOP2 (ridge-regularized normal equation) as a conditioning-stable default,
- EvalType = **trajectory** using committed EY25 or EY26 trajectories,
- DictType = **seeded** or **explicit** (no hidden learned dictionary),
- mandatory diagnostics: reconstruction error L_{rec} and a conditioning estimate,
- no CrossMPolicy unless explicitly declared (non-baseline),
- optional spectral summaries only if eigensolver rules are fully committed.

Reference SpecID (KOOP-0)

Note (ASCII-safe). This template is intentionally ASCII-only inside the `verbatim` block: use `Delta` as a field name and `"delta=1"` as a value string. Greek symbols (e.g. Δ) are used only in math mode outside code blocks.

```
SpecID_EY30_KOOP0 :=
(
  ArtifactType = KOOP,

  InputType = EVO,                # or TOKEN / SNAP / WORKFLOW
  OutputType = KOOP_OP,          # KOOP_SPEC / KOOP_SUMMARY optional

  KoopmanSpec = {
    ObservableDecl = {
      StateRep = Hash("regime-state-rep-v1"),

      G = OP-FEAT(Psi_v1),

      D = {
        DictType: "seeded",
        DictGen: "poly-monomials-v1",    # or "fourier-v1" / "op-feat-v1"
        DictParams:{d:16, p:2},
        OrderSpec: "lexicographic"
      },

      EvalRule = {
        EvalType: "trajectory",
        PointSet: Ref("EY25:trajectory-window"),    # or EY26 token-derived
        MeasureSpec: "time-average",
        TimeSpec: {Delta:"delta=1", mode:"per-step", indices:"k=1..n"},
        TieBreak: "lexicographic-first"
      },

      TruncObs = {m:256, SelectRule:"prefix", CrossMPolicy:null},

      InnerProd = {
        Type: "empirical",
        mu: "empirical",
        Estimator: "avg",
        Regularization:{ridge:1e-8}
      },

      Canon = {rule:"canonical-eval+linAlg", points:["post-eval","post-build","final"]}
    },

    EvolutionDecl = {
      PhiType: "generator",
      PhiSource: Ref("EY25:pi_LIOUV"),
      Delta: "delta=1",
      Semantics: "declared-flow-v1",
      ReplaySpec: "replay-by-EY25-routineID"
```



```

},

KoopClass = "KOP2",

BuildSpec = {
  RoutineID: "K00P-BUILD-KOP2-v1",
  LinAlgSpec:{
    matmul:{order:"row-major", accumulate:"sequential"},
    inv:    {method:"cholesky-or-svd", tieBreak:"lexicographic"},
    regularization:{lambda:1e-8},
    pinv:null,
    eig:null
  },
  CanonPoints:["post-X-Y","post-K","final"],
  TieBreaks:{svd:"lexicographic", eigen:"none"}
},

DiagSpec = [
  {DiagName:"D_rec",  ValueType:"real", TolSpec:{max:0.05},  RoutineID:"DIAG-REC-v1"},
  {DiagName:"D_cond", ValueType:"real", TolSpec:{max:1e12},  RoutineID:"DIAG-COND-v1"}
]
},

NumericModel = {
  arithmetic:"det-float-or-fixed",
  precision:64,
  rounding:"nearest",
  eval:    {complex:"pair-of-reals", order:"fixed"},
  linAlg: {accumulate:"sequential", thresholds:"explicit-grid"},
  canon:  {rule:"normalize-encoding", points:["post-eval","post-build","final"]}
},

TranscriptSpec = {
  CommitType:    "closure",
  HashAlg:       "sha256",
  SerializeRule: "EY22-CCE",
  CommitRate:    "per-export",
  ChainRule:     "none"
},

Cert = {
  schema:"EY22-CCE",
  required_checks:["V1","V2","V3","V4","V5","V6","V8"]
},

Interop = {
  UpstreamSpecIDs: ["EY25:...", "EY28:...", "EY29:..."], # as applicable
  Scope: {what:"trajectory->Koopman", window:"k=1..n", Delta:"delta=1"},
  DictObligations:{DictHash:Hash("dict-v1"), order:"lexicographic", m:256},
  EvalObligations:{source:"EY25", pairing:"(x_k,x_{k+1})", Delta:"delta=1"},
  DiagObligations:{D_rec_max:0.05, D_cond_max:1e12},

```

```

    CompatPredicates: ["K00P-C0", "K00P-C1", "K00P-C2", "K00P-C3", "K00P-C4", "K00P-C7"]
  },

  Version = "VID-2026-02"
)

```

Rule. All `null` fields **MUST** be explicit. Omitting them is not allowed.

7.3 Publication Boundary (Normative)

Rule (what must be public). An EY30 publication **MUST** include:

- full `SpecIDEY30`,
- `ObservableDecl` (dictionary commitments, evaluation rule, truncation m , ordering),
- `EvolutionDecl` (`PhiType`, `PhiSource` reference, declared time step Δ semantics),
- `KoopClass` and `BuildSpec` (routine IDs, thresholds, regularization, tie-breaks),
- `DiagSpec` and computed `DiagReport` (including reconstruction error L_{rec} and conditioning),
- `NumericModel` (evaluation + linear algebra determinism),
- certificate π_{K00P} with transcript commitments: Tx_{eval} , Tx_{build} , Tx_{diag} ,
- output commitment h_K (and optional spectral summary commitments),
- deterministic failure codes (or PASS) for all declared checks.

Rule (no hidden dictionary). A Koopman export is invalid if dictionary identity cannot be reconstructed. Learned dictionaries are allowed only if parameters are committed and evaluation is deterministic under `NumericModel`.

Rule (no hidden evaluation points). If `EvalType` is `sampled`, `PRNGSpec` and seed **MUST** be public. If `EvalType` is `trajectory`, the trajectory source (EY25/EY26) **MUST** be cited and committed.

Rule (no hidden time semantics). The declared time step Δ **MUST** be bound as a value in `EvolutionDecl` and `EvalRule.TimeSpec`. In code/spec blocks use ASCII (e.g. `"delta=1"`); in math text use Δ freely.

7.4 Non-Claims (Explicit)

EY30 explicitly does *not* claim:

NC1: Completeness of Observable Dictionaries

No claim is made that $\mathcal{D}$ spans a Koopman-invariant subspace or captures all dynamics.

NC2: Infinite-Dimensional Koopman Truth from Finite Truncations

Spectral claims about the true Koopman operator are not implied by finite K eigenvalues. Finite- m eigenvalues are only export artifacts under a declared dictionary and evaluation rule.

NC3: Regime Equivalence

Using the same dictionary in two regimes does not imply those regimes are physically equivalent. Any equivalence claim must be mediated by EY28 (transport) and explicit interop certificates.

NC4: Robustness Beyond Declared Diagnostics

Passing L_{rec} and conditioning thresholds does not guarantee robustness to adversarial perturbations, sampling bias, or unmodeled dynamics. EY30 only certifies what it measures and commits.

NC5: Comparability Without Shared ObservableDecl

Two Koopman exports are comparable only if ObservableDecl (dictionary, ordering, evaluation source semantics) matches or a declared coercion/bridge exists. Otherwise they are different objects.

7.5 Licensing Intent (Vol. No. IV)

The EY30 specification (schemas, baseline routines, diagnostics, verifier checks) is intended to remain publicly readable. Implementations may be licensed under project-chosen terms, but:

dictionary commitments, evaluation determinism, and diagnostics may not be removed.

Rule (no audit stripping). No distribution may claim EY30 compliance while omitting:

- explicit dictionary identity,
- explicit evaluation point set source semantics,
- reconstruction error and conditioning diagnostics,
- and replayable construction rules.

7.6 Chapter Summary

This chapter provided a KOOP-0 reference SpecID template (ASCII-safe), fixed the EY30 publication boundary (what must be committed and public), stated explicit non-claims (no completeness, no infinite-dimensional truth, no regime equivalence, no robustness beyond diagnostics, no comparability without shared ObservableDecl), and recorded licensing intent consistent with Vol. No. IV.

Next. Proceed to Chapter 8 for worked examples (non-normative), or close Vol. No. IV by compiling the full EY22 compatibility matrix and publishing one canonical end-to-end reference workflow with interop certificates.

8 Worked Examples (Non-Normative): Export Patterns and Koopman Signatures

8.1 Purpose

This chapter is *non-normative*. It illustrates common EY30 usage patterns:

- generator-based Koopman export from EY25 (EDMD-style),
- token-based export from EY26 (per-step vs per-period),
- regime-bridged export using EY28 (export-in-target vs export-in-source),
- reduced dictionaries and reduced evaluation via EY29,
- and registry-grade Koopman signatures (bounded, canonical, replayable).

Reminder. Examples become valid only when ObservableDecl, EvolutionDecl, KoopClass, BuildSpec, diagnostics, and transcripts are fully committed.

8.2 Example 1: Generator-Based Export (EY25 $\rightarrow$ EY30)

Setup. Assume EY25 provides a committed evolution routine that produces a trajectory:

$$x_{k+1} = \Phi_{\Delta}(x_k),$$

with Δ fixed.

ObservableDecl (sketch). Choose a polynomial dictionary on a d -dimensional state vector:

$$\mathcal{D} = \{1, x_1, \dots, x_d, x_1^2, x_1x_2, \dots\},$$

ordered lexicographically, truncated to size m .

Construction. Form:

$$X = [\psi(x_1) \cdots \psi(x_n)], \quad Y = [\psi(x_2) \cdots \psi(x_{n+1})],$$

then compute K via KOP2 (ridge-regularized normal equation).

Diagnostics. Compute:

$$L_{\text{rec}} = \frac{\|Y - KX\|_F}{\|Y\|_F},$$

and a conditioning estimate. Publish both.

Interpretation. The export produces an auditable K that encodes how the chosen observables behave under the EY25-evolution. It does not claim completeness of the dictionary.

8.3 Example 2: Token-Based Export (EY26 $\rightarrow$ EY30)

Setup. Assume EY26 defines a Floquet token U_F and a step factorization $U_F = U_J \cdots U_1$.

Two exports.

- **Per-step Koopman:** $\Phi(x) = U_j(x)$ for a fixed j .
- **Per-period Koopman:** $\Phi(x) = U_F(x)$ (one full period).

Rule reminder. EvolutionDecl MUST specify which one is used, including time-slicing semantics.

Interpretation. Per-step K_j can reveal the internal structure of the token; per-period K_F is a compressed summary suitable for registries.

8.4 Example 3: Regime-Bridged Export (EY28 + EY30)

Setup. Two regimes with a bridge $T_{1 \rightarrow 2}$ (EY28).

Option A: Export-in-target. Transport states first:

$$x_k^{(2)} := T_{1 \rightarrow 2}(x_k^{(1)}),$$

then evaluate observables in regime 2 and build $K^{(2)}$.

Option B: Export-in-source with observable bridge. Build $K^{(1)}$ in regime 1, then map dictionaries:

$$g_i^{(2)} := g_i^{(1)} \circ T_{2 \rightarrow 1},$$

if such an observable bridge is committed and deterministic.

Interop requirement. The workflow MUST publish interop certificates that bind:

- the chosen order,
- the StateRep in the target regime,
- and dictionary identity in the target evaluation semantics.

Interpretation. This enables “same observable dictionary across regimes” comparisons without claiming physical equivalence.

8.5 Example 4: Reduced Dictionary via EY29 (CGR $\rightarrow$ KOOP)

Setup. Suppose a high-dimensional operator/state yields a coarse-grain summary via EY29:

$$(\text{BucketID}, s) := R(\text{trajectory summary}),$$

where s is a bounded feature vector.

Dictionary construction. Define an OP-FEAT dictionary using coordinates of s as observables:

$$g_i(x) := s_i(x), \quad i = 1, \dots, m,$$

with dictionary identity bound by CGR-Ref + SummarySpec.

Interpretation. This yields a Koopman export on a reduced, registry-friendly observable space. But KOOP-C7 requires that the reduction certificate is verified.

8.6 Example 5: Koopman Signatures as Registry Keys (Bounded)

Registry-grade usage requires *bounded* canonical signatures. Two examples:

Signature A: Spectral Window Summary

Compute eigenvalues of K (if published) and store a bounded windowed summary:

$$\sigma_{\text{spec}} := (\text{hist}(\Re \lambda; \text{bins}), \text{hist}(\Im \lambda; \text{bins})).$$

This avoids dependence on exact eigenvalue ordering beyond histogram bins.

Signature B: Mode Energy Profile

Pick a committed set of probe vectors v_j and store:

$$\sigma_{\text{mode}} := (\|K^t v_j\|)_{t \in T, j \in J},$$

with T, J committed and small.

Rule reminder. If signatures are used as keys:

- dictionary identity (KOOP-C2) and evaluation semantics (KOOP-C3) must match,
- diagnostics (KOOP-C4) must be within bounds,
- and the signature encoding must be canonical (KOOP-C6).

Interpretation. Signatures are designed for registry stability and comparison, not for claiming analytic truth about the infinite-dimensional Koopman operator.

8.7 Example 6: Cross- m Stability (CrossMPolicy)

Setup. Run export at $m \in \{m_1, m_2, m_3\}$ and aggregate via `CrossMPolicy=stable-first`.

Diagnostic. Compute a stability score:

$$D_{\text{stab}} := \max_{i < j} \frac{\|K_{m_i} - \Pi_{ij}^\top K_{m_j} \Pi_{ij}\|_F}{\|K_{m_i}\|_F},$$

where Π_{ij} is the canonical truncation projector from m_j to m_i .

Interpretation. Cross- m stability can justify using a smaller m for registries while demonstrating that the representation has stabilized.

8.8 Chapter Summary

This non-normative chapter illustrated EY30 export patterns: generator-based export from EY25, token-based export from EY26, regime-bridged export using EY28, reduced dictionary/evaluation via EY29, registry-grade Koopman signatures, and CrossMPolicy stability.

Next. Vol. No. IV (EY23–EY30) is now complete. Next practical step is to populate the EY22 compatibility matrix entries for the full suite and generate one or two end-to-end reference workflows (with interop certificates) to serve as canonical examples.